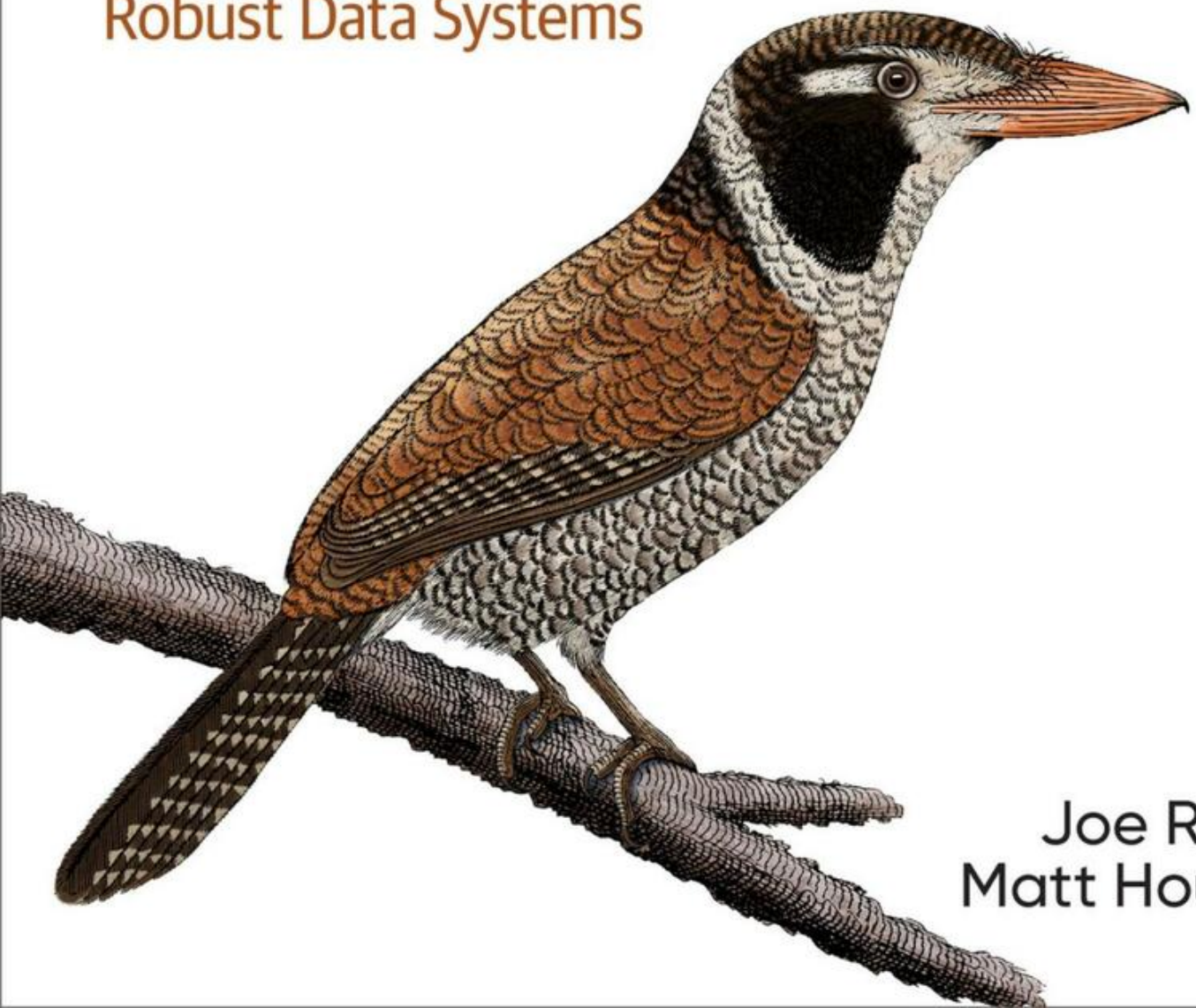
Machine Translated by Google

O'REILLY®

Fundamentals of Data Engineering

Plan and Build Robust Data Systems



Joe Reis &
Matt Housley

Machine Translated by Google

Fundamentos de dados Engenharia

Planeje e crie sistemas de dados robustos

Joe Reis e Matt Housley

Machine Translated by Google

Fundamentos da Engenharia de Dados

por Joe Reis e Matt Housley

Copyright © 2022 Joseph Reis e Matthew Housley. Todos os direitos reservados.

Impresso nos Estados Unidos da América.

Publicado por O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

Os livros da O'Reilly podem ser adquiridos para uso educacional, comercial ou promocional de vendas. Edições online também estão disponíveis para a maioria dos títulos ***(http:// oreilly.com).*** Para mais informações, entre em contato com nosso departamento de vendas corporativo/institucional: 800-998-9938 ou ***corporate@oreilly.com.***

Editor de Aquisições: Jessica Haberman

Editor de Desenvolvimento: Michele Cronin

Editor de Produção: Gregory Hyman

Editor de texto: Sharon Wilkey

Revisor: Amnet Systems, LLC

Indexador: Judith McConville

Designer de Interiores: David Futato

Designer de capa: Karen Montgomery

Ilustrador: Kate Dullea

Junho de 2022: primeira edição

Histórico de revisões para a primeira edição

Machine Translated by Google

22/06/2022: Primeiro lançamento

Consulte ***http://oreilly.com/catalog/errata.csp?isbn=9781098108304*** para detalhes do lançamento.

O logotipo O'Reilly é uma marca registrada da O'Reilly Media, Inc.
Fundamentos de Engenharia de Dados, a imagem da capa e a imagem comercial relacionada são marcas registradas da O'Reilly Media, Inc.

As opiniões expressas neste trabalho são dos autores e não representam a opinião da editora. Embora o editor e os autores tenham feito esforços de boa fé para garantir que as informações e instruções contidas neste trabalho sejam precisas, o editor e os autores se isentam de qualquer responsabilidade por erros ou omissões, incluindo, sem limitação, responsabilidade por danos resultantes do uso ou confiança neste trabalho. O uso das informações e instruções contidas neste trabalho é por sua conta e risco. Se qualquer amostra de código ou outra tecnologia que este trabalho contém ou descreve está sujeita a licenças de código aberto ou direitos de propriedade intelectual de terceiros, é sua responsabilidade garantir que seu uso esteja em conformidade com tais licenças e/ou direitos.

978-1-098-10830-4

[LSI]

Machine Translated by Google

Prefácio

Como surgiu este livro? A origem está profundamente enraizada em nossa jornada da ciência de dados à engenharia de dados. Muitas vezes, brincando, nos referimos a nós mesmos como ***cientistas de dados em recuperação.*** Nós dois tivemos a experiência de ser designados para projetos de ciência de dados, depois lutando para executar esses projetos devido à falta de fundamentos adequados. Nossa jornada na engenharia de dados começou quando realizamos tarefas de engenharia de dados para construir fundações e infraestrutura.

Com a ascensão da ciência de dados, as empresas investiram abundantemente em talentos em ciência de dados, na esperança de colher grandes recompensas. Muitas vezes, os cientistas de dados lutavam com problemas básicos que sua formação e treinamento não abordavam – coleta de dados, limpeza de dados, acesso a dados, transformação de dados e infraestrutura de dados. Esses são problemas que a engenharia de dados visa resolver.

Machine Translated by Google

O que este livro não é

Antes de falarmos sobre o que é este livro e o que você vai tirar dele, vamos falar rapidamente sobre o que este livro ***não é.*** Este livro não é sobre engenharia de dados usando uma ferramenta, tecnologia ou plataforma específica. Embora muitos livros excelentes abordem as tecnologias de engenharia de dados dessa perspectiva, esses livros têm uma vida útil curta. Em vez disso, tentamos nos concentrar nos conceitos fundamentais por trás da engenharia de dados.

Machine Translated by Google

Sobre o que é este livro

Este livro visa preencher uma lacuna no conteúdo e materiais atuais de engenharia de dados. Embora não haja escassez de recursos técnicos que abordam ferramentas e tecnologias específicas de engenharia de dados, as pessoas lutam para entender como montar esses componentes em um todo coerente que se aplique ao mundo real. Este livro conecta os pontos do ciclo de vida de dados de ponta a ponta. Ele mostra como unir várias tecnologias para atender às necessidades de consumidores de dados downstream, como analistas, cientistas de dados e engenheiros de aprendizado de máquina. Este livro funciona como um complemento aos livros da O'Reilly que cobrem os detalhes de tecnologias, plataformas e linguagens de programação específicas.

A grande ideia deste livro é o ***ciclo de vida da engenharia de*** dados: geração, armazenamento, ingestão, transformação e serviço de dados fases do ciclo permaneceram essencialmente inalteradas. Com essa estrutura, o leitor terá um bom entendimento para aplicar tecnologias a problemas de negócios do mundo real.

Nosso objetivo aqui é mapear princípios que abrangem dois eixos. Primeiro, queremos destilar a engenharia de dados em princípios que possam abranger ***qualquer tecnologia relevante.*** Em segundo lugar, desejamos apresentar princípios que resistirão ao teste do ***tempo.*** Esperamos que essas ideias reflitam as lições aprendidas durante a revolução da tecnologia de dados dos últimos vinte anos e que nossa estrutura mental permaneça útil por uma década ou mais no futuro.

Uma coisa a ser observada: adotamos, sem remorso, uma abordagem que prioriza a nuvem. Vemos a nuvem como um desenvolvimento fundamentalmente transformador que durará décadas; a maioria dos sistemas de dados e cargas de trabalho locais acabará migrando para a hospedagem na nuvem. Assumimos que a infraestrutura e os sistemas são ***efêmeros*** e ***escaláveis,*** e que os engenheiros de dados se inclinarão para a implantação de serviços gerenciados na nuvem. Dito isso, a maioria dos conceitos neste livro serão traduzidos para ambientes não-nuvem.

Machine Translated by Google

Quem deveria ler esse livro

Nosso principal público-alvo para este livro consiste em profissionais técnicos, engenheiros de software de nível médio a sênior, cientistas de dados ou analistas interessados em ingressar na engenharia de dados; ou engenheiros de dados trabalhando nas entranhas de tecnologias específicas, mas querendo desenvolver uma perspectiva mais abrangente. Nosso público-alvo secundário consiste em partes interessadas de dados que trabalham ao lado de profissionais técnicos - por exemplo, um líder de equipe de dados com formação técnica supervisionando uma equipe de engenheiros de dados ou um diretor de armazenamento de dados que deseja migrar de tecnologia local para uma nuvem solução baseada.

Idealmente, você é curioso e quer aprender – por que mais você estaria lendo este livro? Você se mantém atualizado com as tecnologias e tendências de dados lendo livros e artigos sobre data warehousing/data lakes, sistemas de lote e streaming, orquestração, modelagem, gerenciamento, análise, desenvolvimentos em tecnologias de nuvem etc. leia em uma imagem completa da engenharia de dados em todas as tecnologias e paradigmas.

Pré-requisitos

Assumimos uma boa familiaridade com os tipos de sistemas de dados encontrados em um ambiente corporativo. Além disso, presumimos que os leitores tenham alguma familiaridade com SQL e Python (ou alguma outra linguagem de programação) e experiência com serviços em nuvem.

Vários recursos estão disponíveis para aspirantes a engenheiros de dados praticarem Python e SQL. Recursos online gratuitos são abundantes (postagens em blogs, sites de tutoriais, vídeos do YouTube) e muitos novos livros sobre Python são publicados todos os anos.

A nuvem oferece oportunidades sem precedentes para obter experiência prática com ferramentas de dados. Sugerimos que aspirantes a engenheiros de dados configurem contas com serviços em nuvem, como AWS, Azure, Google Cloud Platform, Snowflake, Databricks, etc. Observe que muitas dessas plataformas têm ***nível gratuito***

Machine Translated by Google

opções, mas os leitores devem ficar de olho nos custos e trabalhar com pequenas quantidades de dados e clusters de nó único enquanto estudam.

Desenvolver familiaridade com sistemas de dados corporativos fora de um ambiente corporativo continua difícil e isso cria certas barreiras para aspirantes a engenheiros de dados que ainda não conseguiram seu primeiro emprego de dados. Este livro pode ajudar.
Sugerimos que os novatos em dados leiam as ideias de alto nível e, em seguida, examinem os materiais na seção de ***recursos adicionais*** no final de cada capítulo. Em uma segunda leitura, observe quaisquer termos e tecnologias desconhecidos. Você pode utilizar o Google, Wikipedia, postagens de blog, vídeos do YouTube e sites de fornecedores para se familiarizar com novos termos e preencher lacunas em sua compreensão.

O que você aprenderá e como isso melhorará Suas habilidades

Este livro tem como objetivo ajudá-lo a construir uma base sólida para resolver problemas de engenharia de dados do mundo real.

Ao final deste livro você entenderá:

- Como a engenharia de dados afeta sua função atual (cientista de dados, engenheiro de software ou líder de equipe de dados).
- Como superar o hype de marketing e escolher as tecnologias, arquitetura de dados e processos certos.
- Como usar o ciclo de vida de engenharia de dados para projetar e construir uma arquitetura robusta.
- Práticas recomendadas para cada estágio do ciclo de vida dos dados.

E você será capaz de:

- Incorpore princípios de engenharia de dados em sua função atual (cientista de dados, analista, engenheiro de software, líder de equipe de dados etc.)
- Junte uma variedade de tecnologias de nuvem para atender às necessidades dos consumidores de dados downstream.

Machine Translated by Google

Avalie problemas de engenharia de dados com uma estrutura completa de melhores práticas

Incorpore governança e segurança de dados em todo o ciclo de vida da engenharia de dados.

Machine Translated by Google

O esboço do livro

Este livro é composto de quatro partes:

- Parte I, “Fundação e blocos de construção”
- Parte II, “O ciclo de vida da engenharia de dados em profundidade”
- Parte III, “Segurança, Privacidade e o Futuro da Engenharia de Dados”
- Apêndices A e B: rede em nuvem, serialização e compressão

Na Parte I, começamos definindo a engenharia de dados no Capítulo 1, depois mapeamos o ciclo de vida da engenharia de dados no Capítulo 2. No Capítulo 3, discutimos a ***boa arquitetura.*** No Capítulo 4, apresentamos uma estrutura para escolher a tecnologia certa — embora frequentemente vejamos tecnologia e arquitetura confundidas, esses são, na verdade, tópicos muito diferentes.

A Parte II baseia-se no Capítulo 2 para cobrir o ciclo de vida da engenharia de dados em profundidade; cada estágio do ciclo de vida — geração, armazenamento, ingestão, transformação e serviço de dados — é abordado em seu próprio capítulo. A Parte II é indiscutivelmente o coração do livro, e os outros capítulos existem para apoiar as ideias centrais abordadas aqui.

A Parte III cobre tópicos adicionais. No Capítulo 10, discutimos ***segurança e privacidade.*** Embora a segurança sempre tenha sido uma parte importante da profissão de engenharia de dados, ela só se tornou mais crítica com o surgimento de hackers com fins lucrativos e ataques cibernéticos patrocinados pelo estado. E o que podemos dizer da privacidade? A era do niilismo de privacidade corporativa acabou - nenhuma empresa quer ver seu nome aparecer na manchete de um artigo sobre práticas de privacidade descuidadas. O manuseio imprudente de dados pessoais também pode ter ramificações legais significativas com o advento do GDPR, CCPA e outros regulamentos. Em suma, segurança e privacidade devem ser as principais prioridades em qualquer trabalho de engenharia de dados.

Enquanto trabalhamos em engenharia de dados, fazendo pesquisas para este livro e entrevistando vários especialistas, pensamos bastante sobre o rumo que o campo está tomando no curto e longo prazo. O Capítulo 11 descreve nosso altamente

Machine Translated by Google

idéias especulativas sobre o futuro da engenharia de dados. Por sua natureza, o futuro é uma coisa escorregadia. O tempo dirá se algumas de nossas ideias estão corretas. Adoraríamos ouvir de nossos leitores como suas visões do futuro concordam ou diferem das nossas.

No apêndice, abordamos alguns tópicos técnicos extremamente relevantes para a prática cotidiana da engenharia de dados, mas que não se encaixavam no corpo principal do texto. Especificamente, a rede na nuvem é um tópico crítico à medida que a engenharia de dados muda para a nuvem, e os engenheiros precisam entender a serialização e a compactação para trabalhar diretamente com arquivos de dados e para avaliar as considerações de desempenho em sistemas de dados.

Convenções utilizadas neste livro

As seguintes convenções tipográficas são usadas neste livro:

itálico

Indica novos termos, URLs, endereços de e-mail, nomes de arquivo e extensões de arquivo.

Largura constante

Usado para listagens de programas, bem como dentro de parágrafos para se referir a elementos de programa, como nomes de variáveis ou funções, bancos de dados, tipos de dados, variáveis de ambiente, instruções e palavras-chave.

Largura constante em negrito

Mostra comandos ou outro texto que deve ser digitado literalmente pelo usuário.

Largura constante em itálico

Mostra o texto que deve ser substituído por valores fornecidos pelo usuário ou por valores determinados pelo contexto.

Machine Translated by Google

GORJETA

Este elemento significa uma dica ou sugestão.

NOTA

Este elemento significa uma nota geral.

AVISO

Este elemento indica um aviso ou cuidado.

Como entrar em contato conosco

Envie comentários e perguntas sobre este livro à editora:

O'Reilly Media, Inc.

1005 Gravestein Highway North

Sebastopol, CA 95472

800-998-9938 (nos Estados Unidos ou Canadá)

707-829-0515 (internacional ou local)

707-829-0104 (fax)

Temos uma página web para este livro, onde listamos erratas, exemplos e qualquer informação adicional. Você pode acessar esta página em https://oreil.ly/fundamentals-of-data.

Machine Translated by Google

E- mail ***bookquestions@oreilly.com*** para comentar ou fazer perguntas técnicas sobre este livro.

Para notícias e informações sobre nossos livros e cursos, visite https://oreilly.com.

Encontre-nos no LinkedIn: https://linkedin.com/company/oreilly-media

Siga-nos no Twitter: https://twitter.com/oreillymedia

Assista-nos no YouTube: https://www.youtube.com/oreillymedia

Agradecimentos

Quando começamos a escrever este livro, fomos avisados por muitas pessoas de que enfrentávamos uma tarefa difícil. Um livro como este tem muitas partes móveis e, devido à sua visão abrangente do campo da engenharia de dados, exigiu uma tonelada de pesquisas, entrevistas, discussões e reflexão profunda. Não afirmaremos ter capturado todas as nuances da engenharia de dados, mas esperamos que os resultados ressoem com você. Inúmeras pessoas contribuíram para nossos esforços e somos gratos pelo apoio que recebemos de muitos especialistas.

Primeiro, obrigado à nossa incrível equipe de revisores técnicos. Eles se arrastaram por muitas leituras e deram um feedback inestimável (e muitas vezes implacavelmente contundente). Este livro seria uma fração de si mesmo sem seus esforços. Em nenhuma ordem específica, agradecemos infinitamente a Bill Inmon, Andy Petrella, Matt Sharp, Tod Hanseman, Chris Tabb, Danny Lebzyon, Martin Kleppman, Scott Lorimor, Nick Schrock, Lisa Steckman e Alex Woolford.

Em segundo lugar, tivemos uma oportunidade única de conversar com os principais especialistas na área de dados em nossos shows ao vivo, podcasts, encontros e intermináveis chamadas privadas. Suas ideias ajudaram a moldar nosso livro. Há muitas pessoas para citar individualmente, mas gostaríamos de agradecer a Bill Inmon, Jordan Tigani, Zhamak Dehghani, Shruti Bhat, Eric Tschetter, Benn Stancil, Kevin Hu, Michael Rogove, Ryan Wright, Egor Gryaznov, Chad Sanderson , Julie Price, Matt Turck, Monica Rogati, Mars Lan, Pardhu Gunnam, Brian Suk, Barr Moses, Lior Gavish, Bruno Aziza, Gian Merlino, DeVaris Brown,

Machine Translated by Google

Todd Beauchene, Tudor Girba, Scott Taylor, Ori Rafael, Lee Edwards, Bryan Offutt, Ollie Hughes, Gilbert Eijkelenboom, Chris Bergh, Fabiana Clemente, Andreas Kretz, Ori Reshef, Nick Singh, Mark Balkenende, Kenten Danas, Brian Olsen, Lior Gavish , Rhaghu Murthy, Greg Coquillo, David Aponte, Demetrios Brinkmann, Sarah Catanzaro, Michel Tricot, Levi Davis, Ted Walker, Carlos Kemeny, Josh Benamram, Chanin Nantasenamat, George Firican, Jordan Goldmeir, Minhaaj Rehmam, Luigi Patruno, Vin Vashista, Danny Ma, Jesse Anderson, Alessya Visnjic, Vishal Singh, Dave Langer, Roy Hasson, Todd Odess, Che Sharma, Scott Breitenother, Ben Taylor, Thom Ives, John Thompson, Brent Dykes, Josh Tobin, Mark Kosiba, Tyler Pugliese, Douwe Maan, Martin Traverso, Curtis Kowalski, Bob Davis, Koo Ping Shung, Ed Chenard, Matt Sciorma, Tyler Folkman, Jeff Baird, Tejas Manohar, Paul Singman, Kevin Stumpf, Willem Pineaar e Michael Del Balso de Tecton, Emma Dahl, Harpreet Sahota, Ken Jee, Scott Taylor, Kate Strachnyi, Kristen Kehrer, Taylor Miller, Abe Gong, Ben Castleton, Ben Rogojan, David Mertz, Emmanuel Raj, Andrew Jones, Avery Smith, Brock Cooper, Jeff Larson, Jon King, Holden Ackerman, Miriah Peterson, Felipe Hoffa, David Gonzalez, Richard Wellman , Susan Walsh, Ravit Jain, Lauren Balik, Mikiko Bazeley, Mark Freeman, Mike Wimmer, Alexey Shchedrin, Mary Clair Thompson, Julie Burroughs, Jason Pedley, Freddy Drennan, Jake Carter, Jason Pedley, Kelly e Matt Phillipps, Brian Campbell, Faris Chebib, Dylan Gregerson, Ken Myers e muitos outros.

Se você não for mencionado especificamente, não leve para o lado pessoal. Você sabe quem você é. Deixe-nos saber e vamos levá-lo na próxima edição.

Também gostaríamos de agradecer à equipe Ternary Data, nossos alunos e as inúmeras pessoas ao redor do mundo que nos apoiaram. É um ótimo lembrete de que o mundo é um lugar muito pequeno.

Trabalhar com a equipe da O'Reilly foi incrível! Agradecimentos especiais a Jess Haberman por confiar em nós durante o processo de proposta do livro, nossas incríveis e extremamente pacientes editoras de desenvolvimento Nicole Taché e Michele Cronin pela edição, feedback e suporte inestimáveis. Obrigado também à excelente equipe de produção da O'Reilly (Greg e equipe).

Machine Translated by Google

Joe gostaria de agradecer a sua família — Cassie, Milo e Ethan — por deixá-lo escrever um livro. Eles tiveram que aguentar uma tonelada, e Joe promete nunca mais escrever outro livro ;)

Matt gostaria de agradecer a seus amigos e familiares por sua paciência e apoio duradouros. Ele ainda está esperançoso de que Sêneca se dignará a dar uma avaliação de cinco estrelas depois de muito trabalho e tempo perdido com a família nos feriados.

Machine Translated by Google

Parte I. Fundação e Blocos de Construção

Machine Translated by Google

Capítulo 1. Engenharia de Dados Descrita

Se você trabalha com dados ou software, deve ter notado a engenharia de dados emergindo das sombras e agora dividindo o palco com a ciência de dados.
A engenharia de dados é um dos campos mais quentes em dados e tecnologia, e por um bom motivo. Ele cria a base para a ciência de dados e a análise em produção. Este capítulo explora o que é engenharia de dados, como o campo nasceu e sua evolução, as habilidades dos engenheiros de dados e com quem eles trabalham.

O que é Engenharia de Dados?

Apesar da popularidade atual da engenharia de dados, há muita confusão sobre o que significa engenharia de dados e o que engenheiros de dados fazem. A engenharia de dados existe de alguma forma desde que as empresas começaram a fazer coisas com dados – como análise preditiva, análise descritiva e relatórios – e entrou em foco junto com a ascensão da ciência de dados na década de 2010.
Para os propósitos deste livro, é fundamental definir o que significa engenharia de ***dados*** e ***engenheiro*** de dados .

Primeiro, vamos analisar o cenário de como a engenharia de dados é descrita e desenvolver alguma terminologia que podemos usar ao longo deste livro. Existem infinitas definições de ***engenharia de dados .*** No início de 2022, uma pesquisa de correspondência exata no Google por “o que é engenharia de dados?” retorna mais de 91.000 resultados exclusivos. Antes de darmos nossa definição, aqui estão alguns exemplos de como alguns especialistas na área definem a engenharia de dados:

Machine Translated by Google

A engenharia de dados é um conjunto de operações que visa criar interfaces e mecanismos para o fluxo e acesso de informações. São necessários especialistas dedicados — engenheiros de dados — para manter os dados para que permaneçam disponíveis e possam ser usados por outras pessoas. Em suma, os engenheiros de dados configuram e operam a infraestrutura de dados da organização, preparando-a para análise posterior por analistas de dados e cientistas.

—De “Engenharia de Dados e Seus Principais Conceitos” por AlexSoft [1]

O primeiro tipo de engenharia de dados é focado em SQL. O trabalho e o armazenamento primário dos dados estão em bancos de dados relacionais. Todo o processamento de dados é feito com SQL ou uma linguagem baseada em SQL. Às vezes, esse processamento de dados é feito com uma ferramenta ETL. O segundo tipo de engenharia de dados é focado em Big Data. O trabalho e o armazenamento primário dos dados estão em tecnologias de Big Data como Hadoop, Cassandra e HBase. Todo o processamento de dados é feito em estruturas de Big Data como MapReduce, Spark e Flink. Enquanto o SQL é usado, o processamento primário é feito com linguagens de programação como Java, Scala e Python.

—Jesse Anderson [3]

Em relação às funções existentes anteriormente, o campo de engenharia de dados pode ser pensado como um superconjunto de inteligência de negócios e armazenamento de dados que traz mais elementos da engenharia de software. Essa disciplina também integra a especialização em torno da operação dos chamados sistemas distribuídos de “big data”, juntamente com conceitos em torno do ecossistema Hadoop estendido, processamento de fluxo e computação em escala. [4]

- Maxime Beauchemin

A engenharia de dados tem tudo a ver com movimentação, manipulação e gerenciamento de dados.

—Lewis Gavin [5]

Uau! É totalmente compreensível se você está confuso sobre engenharia de dados. Isso é apenas um punhado de definições, e elas contêm uma enorme variedade de opiniões sobre o significado da ***engenharia de dados.***

Machine Translated by Google

Engenharia de dados definida

Quando descompactamos os tópicos comuns de como várias pessoas definem a engenharia de dados, surge um padrão óbvio: um engenheiro de dados obtém dados, armazena-os e prepara-os para serem consumidos por cientistas de dados, analistas e outros. Definimos ***engenharia*** de dados e engenheiro de ***dados*** da seguinte forma:

> ***A engenharia de dados é o desenvolvimento, implementação e manutenção de sistemas e processos que recebem dados brutos e produzem informações consistentes e de alta qualidade que suportam casos de uso downstream, como análise e aprendizado de máquina. A engenharia de dados é a interseção de segurança, gerenciamento de dados, DataOps, arquitetura de dados, orquestração e engenharia de software. Um engenheiro de dados gerencia o ciclo de vida da engenharia de dados, começando com a obtenção de dados dos sistemas de origem e terminando com o fornecimento de dados para casos de uso, como análise ou aprendizado***

O ciclo de vida da engenharia de dados

É muito fácil fixar-se na tecnologia e perder o panorama geral míope. Este livro gira em torno de uma grande ideia chamada ***ciclo de vida da engenharia de dados*** (Figura 1-1), que acreditamos dar aos engenheiros de dados o contexto holístico para visualizar sua função.

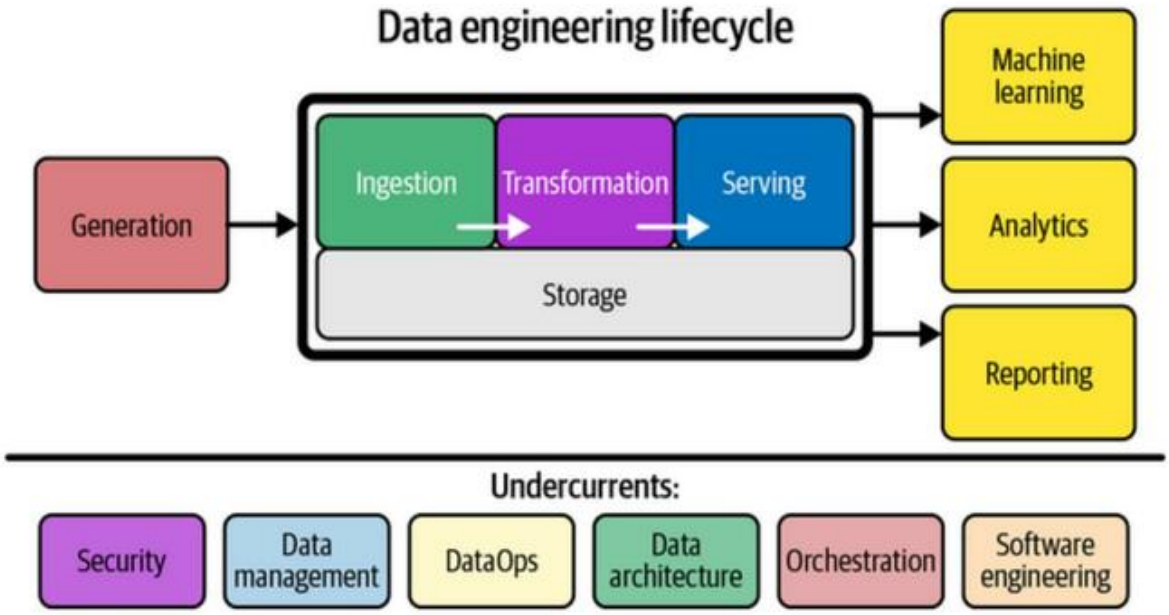


Figura 1-1. O ciclo de vida da engenharia de dados

Machine Translated by Google

O ciclo de vida da engenharia de dados muda a conversa da tecnologia para os próprios dados e os objetivos finais que eles devem atender.
As etapas do ciclo de vida da engenharia de dados são as seguintes:

Geração

Armazenar

Ingestão

Transformação

Servindo

O ciclo de vida da engenharia de dados também tem uma noção de ***subcorrentes –*** ideias críticas em todo o ciclo de vida. Isso inclui segurança, gerenciamento de dados, DataOps, arquitetura de dados, orquestração e engenharia de software. Cobrimos o ciclo de vida da engenharia de dados e suas subcorrentes mais amplamente no Capítulo 2. Ainda assim, o apresentamos aqui porque é essencial para nossa definição de engenharia de dados e para a discussão que se segue neste capítulo.

Agora que você tem uma definição funcional de engenharia de dados e uma introdução ao seu ciclo de vida, vamos dar um passo atrás e examinar um pouco da história.

Evolução do Engenheiro de Dados

A história não se repete, mas rima.

—Um famoso ditado frequentemente atribuído a Mark Twain

Compreender a engenharia de dados hoje e amanhã requer um contexto de como o campo evoluiu. Esta seção não é uma lição de história, mas olhar para o passado é inestimável para entender onde estamos hoje e para onde as coisas estão indo. Um tema comum reaparece constantemente: o que é velho é novo novamente.

Os primórdios: 1980 a 2000, do armazenamento de dados à web

Machine Translated by Google

O nascimento do engenheiro de dados provavelmente tem suas raízes no armazenamento de dados, datando da década de 1970, com o ***armazenamento de dados de negócios*** tomando forma na década de 1980 e Bill Inmon cunhando oficialmente o termo ***data warehouse*** em 1990. Depois que os engenheiros da IBM desenvolveram o banco de dados relacional e Structured Query Language (SQL), a Oracle popularizou a tecnologia. À medida que os sistemas de dados nascentes cresciam, as empresas precisavam de ferramentas e pipelines de dados dedicados para relatórios e inteligência de negócios (BI). Para ajudar as pessoas a modelar corretamente sua lógica de negócios no data warehouse, Ralph Kimball e Inmon desenvolveram suas respectivas técnicas e abordagens de modelagem de dados de mesmo nome, que ainda são amplamente utilizadas hoje.

O armazenamento de dados inaugurou a primeira era da análise escalável, com novos bancos de dados de processamento paralelo massivo (MPP) que usam vários processadores para processar grandes quantidades de dados que chegam ao mercado e suportam volumes de dados sem precedentes. Funções como engenheiro de BI, desenvolvedor de ETL e engenheiro de data warehouse atenderam às várias necessidades do data warehouse. Data warehouse e engenharia de BI foram precursores da engenharia de dados de hoje e ainda desempenham um papel central na disciplina.

A internet se popularizou em meados da década de 1990, criando toda uma nova geração de empresas pioneiras na web, como AOL, Yahoo e Amazon. O boom das pontocom gerou uma tonelada de atividades em aplicativos da Web e os sistemas de back-end para suportá-los - servidores, bancos de dados e armazenamento. Grande parte da infraestrutura era cara, monolítica e fortemente licenciada. Os fornecedores que vendem esses sistemas de back-end provavelmente não previram a escala dos dados que os aplicativos da Web produziriam.

O início dos anos 2000: o nascimento da engenharia de dados contemporânea

Avancemos para o início dos anos 2000, quando o boom das pontocom do final dos anos 90 quebrou, deixando para trás um pequeno grupo de sobreviventes. Algumas dessas empresas, como Yahoo, Google e Amazon, se tornariam poderosas empresas de tecnologia. Inicialmente, essas empresas continuaram a contar com os bancos de dados e data warehouses monolíticos e relacionais tradicionais da década de 1990, levando esses sistemas ao limite. À medida que esses sistemas falhavam, eram necessárias abordagens atualizadas para lidar com o crescimento de dados. O novo

Machine Translated by Google

A geração dos sistemas deve ser econômica, escalável, disponível e confiável.

Coincidindo com a explosão de dados, hardwares comuns – como servidores, RAM, discos e drives flash – também se tornaram baratos e onipresentes. Várias inovações permitiram computação distribuída e armazenamento em clusters de computação massivos em grande escala. Essas inovações começaram a descentralizar e romper serviços tradicionalmente monolíticos. A era do "big data" havia começado.

O ***Oxford English Dictionary*** define big data como "conjuntos de dados extremamente grandes que podem ser analisados computacionalmente para revelar padrões, tendências e associações, especialmente relacionadas ao comportamento e interações humanas". Outra descrição famosa e sucinta de big data são os três V's de dados: velocidade, variedade e volume.

Em 2003, o Google publicou um artigo sobre o Google File System e, pouco depois, em 2004, um artigo sobre o MapReduce, um paradigma de processamento de dados ultraescalável. Na verdade, big data tem antecedentes anteriores em data warehouses MPP e gerenciamento de dados para projetos de física experimental, mas as publicações do Google constituíram um "big bang" para tecnologias de dados e as raízes culturais da engenharia de dados como a conhecemos hoje. Você aprenderá mais sobre sistemas MPP e MapReduce nos Capítulos 3 e 8, respectivamente.

Os documentos do Google inspiraram engenheiros do Yahoo a desenvolver e mais tarde o Apache Hadoop de código aberto em 2006.[6] É difícil exagerar o impacto do Hadoop. Engenheiros de software interessados em problemas de dados em larga escala foram atraídos para as possibilidades desse novo ecossistema de tecnologia de código aberto. À medida que empresas de todos os tamanhos e tipos viram seus dados crescerem em muitos terabytes e até petabytes, nasceu a era do engenheiro de big data.

Na mesma época, a Amazon teve que acompanhar suas próprias necessidades de dados explosivas e criou ambientes de computação elástica (Amazon Elastic Compute Cloud ou EC2), sistemas de armazenamento infinitamente escaláveis (Amazon Simple Storage Service ou S3), bancos de dados NoSQL altamente escaláveis (Amazon DynamoDB) e muitos outros blocos de construção de dados principais. A Amazon optou por oferecer esses serviços para consumo interno e externo por meio de[7]

Machine Translated by Google

Amazon Web Services (AWS), tornando-se a primeira nuvem pública popular. A AWS criou um mercado de recursos de pagamento conforme o uso ultraflexível, virtualizando e revendendo vastos pools de hardware de commodity. Em vez de comprar hardware para um data center, os desenvolvedores poderiam simplesmente alugar computação e armazenamento da AWS.

À medida que a AWS se tornou um mecanismo de crescimento altamente lucrativo para a Amazon, outras nuvens públicas seguiriam em breve, como Google Cloud, Microsoft Azure e DigitalOcean. A nuvem pública é indiscutivelmente uma das inovações mais significativas do século 21 e gerou uma revolução na forma como os aplicativos de software e dados são desenvolvidos e implantados.

As primeiras ferramentas de big data e a nuvem pública lançaram as bases para o ecossistema de dados atual. O cenário de dados moderno – e a engenharia de dados como a conhecemos agora – não existiria sem essas inovações.

Os anos 2000 e 2010: engenharia de big data As ferramentas

de big data de código aberto no ecossistema Hadoop amadureceram rapidamente e se espalharam do Vale do Silício para empresas com experiência em tecnologia em todo o mundo. Pela primeira vez, qualquer empresa teve acesso às mesmas ferramentas de dados de ponta usadas pelas principais empresas de tecnologia. Outra revolução ocorreu com a transição da computação em lote para o streaming de eventos, inaugurando uma nova era de big data “em tempo real”. Você aprenderá sobre streaming de lotes e eventos ao longo deste livro.

Os engenheiros podiam escolher o que há de mais recente e melhor—Hadoop, Apache Pig, Apache Hive, Dremel, Apache HBase, Apache Storm, Apache Cassandra, Apache Spark, Presto e várias outras novas tecnologias que entraram em cena. As ferramentas de dados tradicionais orientadas para empresas e baseadas em GUI de repente se sentiram ultrapassadas, e a engenharia de código estava em voga com a ascensão do MapReduce. Nós (os autores) estávamos por perto durante esse período, e parecia que velhos dogmas morreram de repente no altar do big data.

A explosão de ferramentas de dados no final dos anos 2000 e 2010 deu início ao ***engenheiro de big data.*** Para usar efetivamente essas ferramentas e técnicas, ou seja, o ecossistema Hadoop, incluindo Hadoop, YARN, Hadoop Distributed File

Machine Translated by Google

System (HDFS) e MapReduce — os engenheiros de big data precisavam ser proficientes em desenvolvimento de software e hacking de infraestrutura de baixo nível, mas com ênfase diferente. Os engenheiros de big data normalmente mantinham grandes clusters de hardware comum para fornecer dados em escala. Embora eles possam ocasionalmente enviar solicitações de pull ao código principal do Hadoop, eles mudaram seu foco do desenvolvimento da tecnologia principal para a entrega de dados.

O big data rapidamente se tornou vítima de seu próprio sucesso. Como palavra da moda, ***big data*** ganhou popularidade no início dos anos 2000 até meados dos anos 2010. O big data capturou a imaginação das empresas que tentam entender os volumes cada vez maiores de dados e a interminável enxurrada de marketing descarado de empresas que vendem ferramentas e serviços de big data. Por causa do imenso hype, era comum ver empresas usando ferramentas de big data para pequenos problemas de dados, às vezes montando um cluster Hadoop para processar apenas alguns gigabytes. Parecia que todos queriam participar da ação de big data. Dan Ariely twittou, “Big data é como sexo na adolescência: todo mundo fala sobre isso, ninguém sabe realmente como fazer, todo mundo acha que todo mundo está fazendo, então todo mundo afirma que está fazendo.”

A Figura 1-2 mostra um instantâneo do Google Trends para o termo de pesquisa “big data” para ter uma ideia da ascensão e queda do big data.

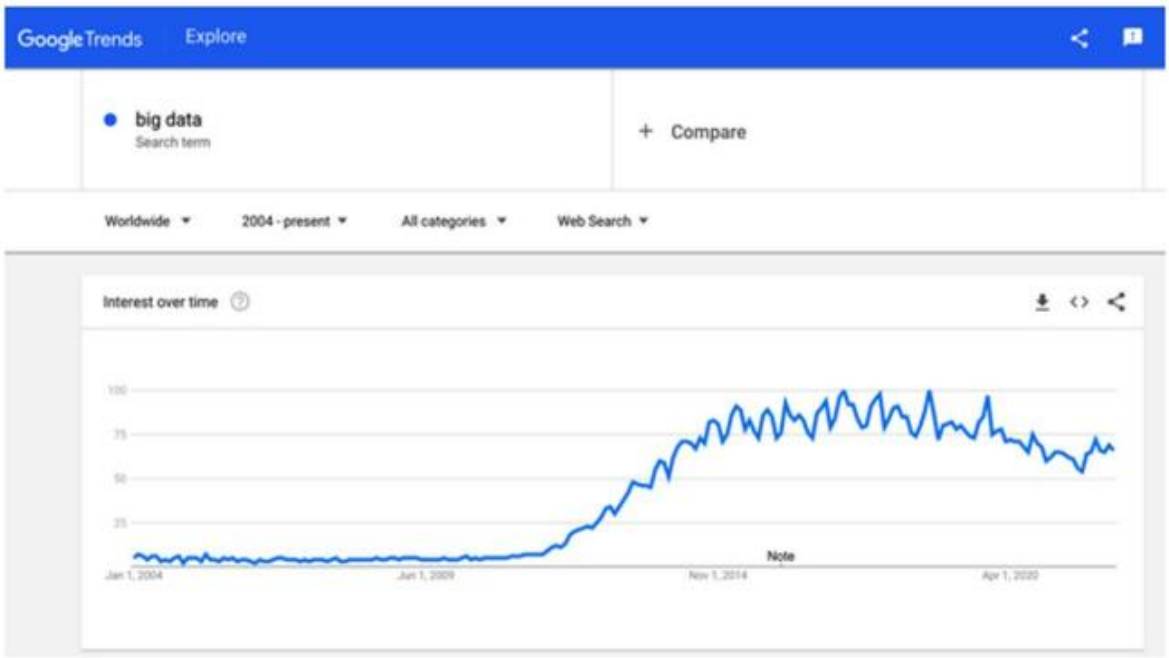


Figura 1-2. Google Trends para “big data” (março de 2022)

Machine Translated by Google

Apesar da popularidade do termo, o big data perdeu força. O que aconteceu? Uma palavra: simplificação. Apesar do poder e sofisticação das ferramentas de big data de código aberto, gerenciá-las era muito trabalhoso e exigia atenção constante. Muitas vezes, as empresas empregavam equipes inteiras de engenheiros de big data, que custavam milhões de dólares por ano, para cuidar dessas plataformas. Os engenheiros de big data geralmente gastavam muito tempo mantendo ferramentas complicadas e provavelmente não tanto tempo entregando os insights e o valor do negócio.

Desenvolvedores de código aberto, nuvens e terceiros começaram a procurar maneiras de abstrair, simplificar e disponibilizar big data sem a alta sobrecarga administrativa e o custo de gerenciar seus clusters e instalar, configurar e atualizar seu código de código aberto. O termo ***big data*** é essencialmente uma relíquia para descrever um determinado momento e abordagem para lidar com grandes quantidades de dados.

Hoje, os dados estão se movendo mais rápido do que nunca e crescendo cada vez mais, mas o processamento de big data tornou-se tão acessível que não merece mais um termo separado; toda empresa visa resolver seus problemas de dados, independentemente do tamanho real dos dados. Engenheiros de big data agora são simplesmente ***engenheiros de dados.***

A década de 2020: Engenharia para o ciclo de vida dos dados No

momento da redação deste artigo, a função da engenharia de dados está evoluindo rapidamente. Esperamos que essa evolução continue em ritmo acelerado no futuro próximo.
Enquanto os engenheiros de dados historicamente tendiam aos detalhes de baixo nível de estruturas monolíticas como Hadoop, Spark ou Informatica, a tendência está se movendo para ferramentas descentralizadas, modularizadas, gerenciadas e altamente abstratas.

De fato, as ferramentas de dados proliferaram a uma taxa surpreendente (veja a Figura 1-3).
As tendências populares no início da década de 2020 incluem a ***pilha de dados moderna,*** representando uma coleção de produtos de código aberto e de terceiros prontos para uso, reunidos para facilitar a vida dos analistas. Ao mesmo tempo, as fontes de dados e os formatos de dados estão crescendo em variedade e tamanho. A engenharia de dados é cada vez mais uma disciplina de interoperação e conecta várias tecnologias, como peças LEGO, para atender aos objetivos de negócios finais.

Machine Translated by Google

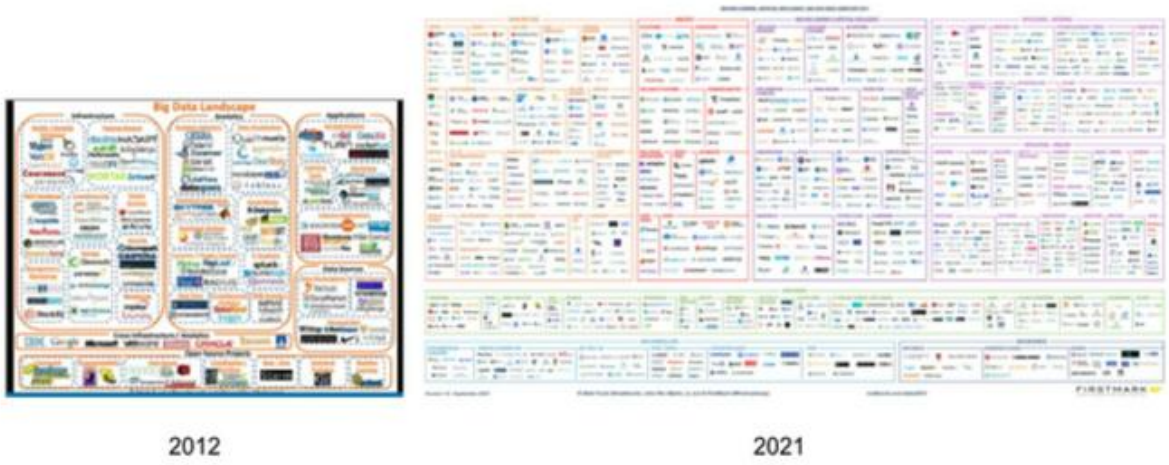


Figura 1-3. Cenário de dados de Matt Turck em 2012 versus 2021

O engenheiro de dados que discutimos neste livro pode ser descrito mais precisamente como um ***engenheiro de ciclo de vida de dados.*** Com maior abstração e simplificação, um engenheiro de ciclo de vida de dados não está mais sobrecarregado pelos detalhes sangrentos das estruturas de big data de ontem. Embora os engenheiros de dados mantenham habilidades em programação de dados de baixo nível e as usem conforme necessário, eles encontram cada vez mais seu papel focado em coisas mais altas na cadeia de valor: segurança, gerenciamento de dados, DataOps, arquitetura de dados, orquestração e gerenciamento geral do ciclo de vida dos dados.[8]

À medida que as ferramentas e os fluxos de trabalho são simplificados, observamos uma mudança notável nas atitudes dos engenheiros de dados. Em vez de se concentrar em quem tem os “maiores dados”, os projetos e serviços de código aberto estão cada vez mais preocupados em gerenciar e governar dados, facilitando o uso e a descoberta e melhorando sua qualidade. Os engenheiros de dados agora estão familiarizados com siglas como ***CCPA*** e ***GDPR;*** à medida que projetam pipelines, eles se preocupam com privacidade, anonimização, coleta de lixo de dados e conformidade com os regulamentos.[9]

O que é velho é novo de novo. Embora coisas “empresariais” como gerenciamento de dados (incluindo qualidade e governança de dados) fossem comuns para grandes empresas na era pré-big-data, não foram amplamente adotadas em empresas menores. Agora que muitos dos problemas desafiadores dos sistemas de dados de ontem estão resolvidos, perfeitamente produzidos e empacotados, tecnólogos e empreendedores mudaram o foco de volta para as coisas “empresariais”, mas com ênfase em

Machine Translated by Google

descentralização e agilidade, que contrasta com a abordagem tradicional de comando e controle empresarial.

Vemos o presente como uma era de ouro do gerenciamento do ciclo de vida dos dados. Os engenheiros de dados que gerenciam o ciclo de vida da engenharia de dados têm ferramentas e técnicas melhores do que nunca. Discutimos o ciclo de vida da engenharia de dados e suas subcorrentes com mais detalhes no próximo capítulo.

Engenharia de dados e ciência de dados

Onde a engenharia de dados se encaixa com a ciência de dados? Há algum debate, com alguns argumentando que a engenharia de dados é uma subdisciplina da ciência de dados. Acreditamos que a engenharia de dados é ***separada*** da ciência e análise de dados. Eles se complementam, mas são bem diferentes. A engenharia de dados fica a montante da ciência de dados (Figura 1-4), o que significa que os engenheiros de dados fornecem as entradas usadas pelos cientistas de dados (a jusante da engenharia de dados), que convertem essas entradas em algo útil.

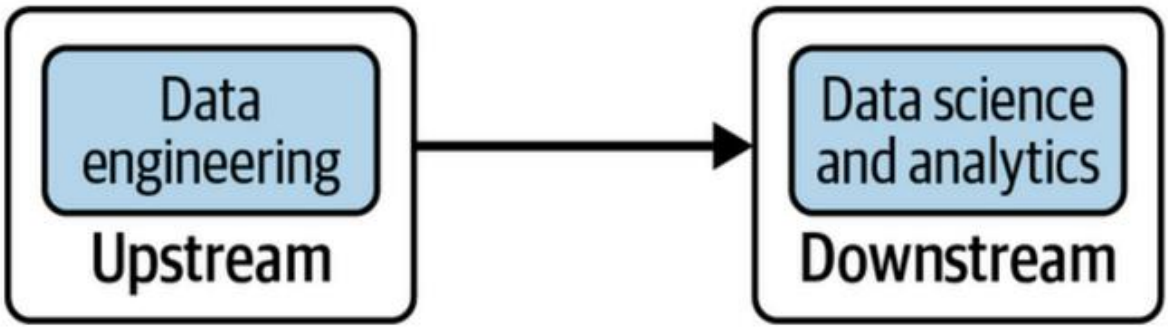


Figura 1-4. A engenharia de dados fica a montante da ciência de dados

Considere a Hierarquia de Necessidades da Ciência de Dados (Figura 1-5). Em 2017, Monica Rogati publicou essa hierarquia em um artigo que mostrou onde a IA e o aprendizado de máquina (ML) estavam próximos de áreas mais “mundanas”, como movimentação/armazenamento de dados, coleta e infraestrutura.

Machine Translated by Google

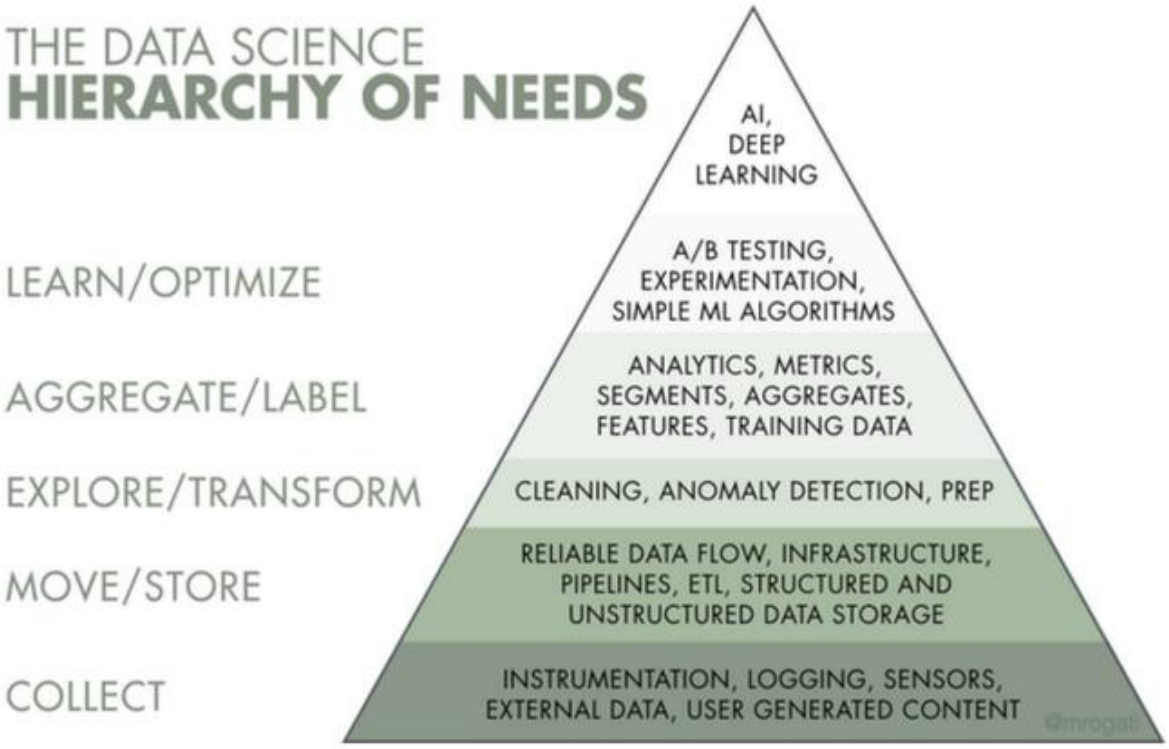


Figura 1-5. A hierarquia de necessidades da ciência de dados

Embora muitos cientistas de dados estejam ansiosos para construir e ajustar modelos de ML, a realidade é que cerca de 70% a 80% de seu tempo é gasto trabalhando nas três partes inferiores da hierarquia – coleta de dados, limpeza de dados, processamento de dados – e apenas um pequena fatia de seu tempo em análise e ML.
Rogati argumenta que as empresas precisam construir uma base de dados sólida (os três níveis inferiores da hierarquia) antes de abordar áreas como IA e ML.

Os cientistas de dados normalmente não são treinados para projetar sistemas de dados de nível de produção e acabam fazendo esse trabalho ao acaso porque não têm o suporte e os recursos de um engenheiro de dados. Em um mundo ideal, os cientistas de dados deveriam passar mais de 90% de seu tempo focados nas camadas superiores da pirâmide: análise, experimentação e ML. Quando os engenheiros de dados se concentram nessas partes inferiores da hierarquia, eles constroem uma base sólida para o sucesso dos cientistas de dados.

Com a ciência de dados impulsionando análises avançadas e ML, a engenharia de dados abrange a divisão entre obter dados e obter valor a partir dos dados (consulte a Figura 1-6). Acreditamos que a engenharia de dados é de igual importância e

Machine Translated by Google

visibilidade à ciência de dados, com engenheiros de dados desempenhando um papel vital para tornar a ciência de dados bem-sucedida na produção.

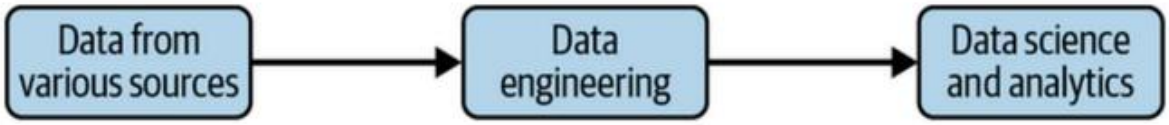


Figura 1-6. Um engenheiro de dados obtém dados e fornece valor a partir dos dados

Habilidades e Atividades de Engenharia de Dados

O conjunto de habilidades de um engenheiro de dados abrange as “correntes ocultas” da engenharia de dados: segurança, gerenciamento de dados, DataOps, arquitetura de dados e engenharia de software. Esse conjunto de habilidades requer uma compreensão de como avaliar as ferramentas de dados e como elas se encaixam no ciclo de vida da engenharia de dados. Também é fundamental saber como os dados são produzidos nos sistemas de origem e como os analistas e cientistas de dados consumirão e criarão valor após o processamento e a curadoria dos dados. Finalmente, um engenheiro de dados manipula muitas partes móveis complexas e deve otimizar constantemente ao longo dos eixos de custo, agilidade, escalabilidade, simplicidade, reutilização e interoperabilidade (Figura 1-7). Abordaremos esses tópicos com mais detalhes nos próximos capítulos.



Figura 1-7. O ato de equilíbrio da engenharia de dados

Como discutimos, no passado recente, esperava-se que um engenheiro de dados conhecesse e entendesse como usar um pequeno punhado de tecnologias poderosas e monolíticas (Hadoop, Spark, Teradata, Hive e muitas outras) para criar uma solução de dados. A utilização dessas tecnologias geralmente requer uma compreensão sofisticada de engenharia de software, rede, computação distribuída, armazenamento ou outros detalhes de baixo nível. Seu trabalho seria dedicado à administração e manutenção do cluster, gerenciamento de sobrecarga e criação de trabalhos de pipeline e transformação, entre outras tarefas.

Atualmente, o cenário de ferramentas de dados é muito menos complicado de gerenciar e implantar. As ferramentas de dados modernas abstraem e simplificam consideravelmente

Machine Translated by Google

fluxos de trabalho. Como resultado, os engenheiros de dados agora estão focados em equilibrar os serviços mais simples e econômicos, os melhores da categoria, que agregam valor aos negócios. O engenheiro de dados também deve criar arquiteturas de dados ágeis que evoluem à medida que surgem novas tendências.

Quais são algumas coisas que um engenheiro de dados ***não*** faz? Um engenheiro de dados normalmente não cria modelos de ML diretamente, cria relatórios ou painéis, executa análise de dados, cria indicadores-chave de desempenho (KPIs) ou desenvolve aplicativos de software. Um engenheiro de dados deve ter uma boa compreensão do funcionamento dessas áreas para atender melhor as partes interessadas.

Maturidade de dados e o engenheiro de dados

O nível de complexidade da engenharia de dados dentro de uma empresa depende muito da maturidade dos dados da empresa. Isso afeta significativamente as responsabilidades de trabalho diárias de um engenheiro de dados e a progressão na carreira. O que é maturidade de dados, exatamente?

A maturidade dos dados é a progressão para uma maior utilização, recursos e integração de dados em toda a organização, mas a maturidade dos dados não depende simplesmente da idade ou da receita de uma empresa. Uma startup em estágio inicial pode ter maior maturidade de dados do que uma empresa de 100 anos com receita anual na casa dos bilhões. O que importa é a forma como os dados são aproveitados como vantagem competitiva.

Os modelos de maturidade de dados têm muitas versões, como Data Management Maturity (DMM) e outros, e é difícil escolher um que seja simples e útil para engenharia de dados. Então, vamos criar nosso próprio modelo simplificado de maturidade de dados. Nosso modelo de maturidade de dados (Figura 1-8) tem três estágios: começar com dados, dimensionar com dados e liderar com dados. Vejamos cada um desses estágios e o que um engenheiro de dados normalmente faz em cada estágio.

Machine Translated by Google

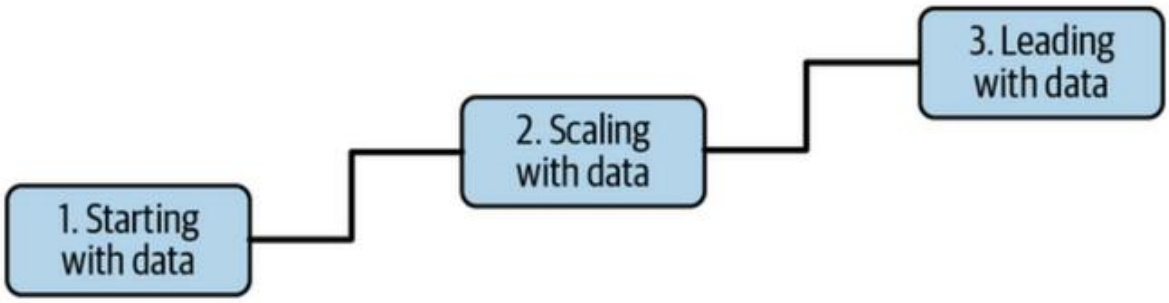


Figura 1-8. Nosso modelo simplificado de maturidade de dados para uma empresa

Etapa 1: começando com dados

Uma empresa que está começando com dados está, por definição, nos estágios iniciais de sua maturidade de dados. A empresa pode ter metas difusas e definidas de maneira vaga ou não ter metas. A arquitetura e a infraestrutura de dados estão nos estágios iniciais de planejamento e desenvolvimento. A adoção e a utilização são provavelmente baixas ou inexistentes. A equipe de dados é pequena, geralmente com um número de funcionários de um dígito. Nesse estágio, um engenheiro de dados geralmente é um generalista e normalmente desempenha várias outras funções, como cientista de dados ou engenheiro de software. O objetivo de um engenheiro de dados é mover-se rapidamente, obter tração e agregar valor.

Os aspectos práticos de obter valor a partir de dados geralmente são mal compreendidos, mas o desejo existe. Relatórios ou análises carecem de estrutura formal e a maioria das solicitações de dados é ad hoc. Embora seja tentador pular de cabeça no ML neste estágio, não recomendamos. Vimos inúmeras equipes de dados ficarem paralisadas e falharem ao tentar pular para o ML sem construir uma base de dados sólida.

Isso não quer dizer que você não possa obter vitórias do ML neste estágio - é raro, mas possível. Sem uma base de dados sólida, você provavelmente não terá os dados para treinar modelos de ML confiáveis nem os meios para implantar esses modelos na produção de maneira escalável e repetível. Nós nos chamamos de brincadeira de “cientistas de dados em recuperação”, principalmente da experiência pessoal de estar envolvido em projetos prematuros de ciência de dados sem maturidade de dados adequada ou suporte de engenharia de dados.

Um engenheiro de dados deve se concentrar no seguinte nas organizações que estão começando com dados:

Machine Translated by Google

Obtenha a adesão das principais partes interessadas, incluindo a gestão executiva. Idealmente, o engenheiro de dados deve ter um patrocinador para iniciativas críticas para projetar e construir uma arquitetura de dados para apoiar os objetivos da empresa.

Defina a arquitetura de dados correta (geralmente sozinho, já que um arquiteto de dados provavelmente não está disponível). Isso significa determinar as metas de negócios e a vantagem competitiva que você pretende alcançar com sua iniciativa de dados. Trabalhe em direção a uma arquitetura de dados que suporte esses objetivos.

Consulte o Capítulo 3 para obter nossos conselhos sobre uma “boa” arquitetura de dados.

Identifique e audite os dados que darão suporte às principais iniciativas e operem dentro da arquitetura de dados que você projetou.

Crie uma base de dados sólida para futuros analistas de dados e cientistas de dados para gerar relatórios e modelos que forneçam valor competitivo. Enquanto isso, você também pode ter que gerar esses relatórios e modelos até que essa equipe seja contratada.

Esta é uma fase delicada com muitas armadilhas. Seguem algumas dicas para esta etapa:

A força de vontade organizacional pode diminuir se muitos sucessos visíveis não ocorrerem com os dados. Obter ganhos rápidos estabelecerá a importância dos dados dentro da organização. Apenas tenha em mente que vitórias rápidas provavelmente criarão dívida técnica. Tenha um plano para reduzir essa dívida, pois, caso contrário, aumentará o atrito para entregas futuras.

Saia e converse com as pessoas e evite trabalhar em silos. Muitas vezes vemos a equipe de dados trabalhando em uma bolha, não se comunicando com pessoas de fora de seus departamentos e obtendo perspectivas e feedback das partes interessadas do negócio. O perigo é que você passará muito tempo trabalhando em coisas de pouca utilidade para as pessoas.

Evite levantamento pesado indiferenciado. Não se encaixote com complexidade técnica desnecessária. Use soluções prontas para uso sempre que possível.

Machine Translated by Google

Crie soluções personalizadas e codifique apenas onde isso cria uma vantagem competitiva.

Etapa 2: dimensionamento com dados

Neste ponto, uma empresa se afastou das solicitações de dados ad hoc e tem práticas formais de dados. Agora, o desafio é criar arquiteturas de dados escaláveis e planejar um futuro em que a empresa seja genuinamente orientada por dados. As funções de engenharia de dados passam de generalistas para especialistas, com pessoas focando em aspectos específicos do ciclo de vida da engenharia de dados.

Em organizações que estão no estágio 2 de maturidade de dados, os objetivos de um engenheiro de dados são:

Estabeleça práticas formais de dados

Crie arquiteturas de dados escaláveis e robustas

Adote práticas de DevOps e DataOps

Crie sistemas que suportem ML

Continuar a evitar o trabalho pesado indiferenciado e personalizar apenas quando resultar em vantagem competitiva

Voltaremos a cada um desses objetivos mais adiante neste livro.

Os problemas a serem observados incluem o seguinte:

À medida que nos tornamos mais sofisticados com os dados, há a tentação de adotar tecnologias de ponta baseadas em provas sociais de empresas do Vale do Silício. Isso raramente é um bom uso do seu tempo e energia. Quaisquer decisões de tecnologia devem ser orientadas pelo valor que entregarão aos seus clientes.

O principal gargalo para dimensionamento não são nós de cluster, armazenamento ou tecnologia, mas a equipe de engenharia de dados. Concentre-se em soluções simples de implantar e gerenciar para expandir o rendimento de sua equipe.

Machine Translated by Google

Você ficará tentado a se enquadrar como um tecnólogo, um gênio dos dados que pode entregar produtos mágicos. Mude seu foco para a liderança pragmática e comece a transição para o próximo estágio de maturidade; comunicar com outras equipes sobre a utilidade prática dos dados. Ensine a organização a consumir e alavancar dados.

Estágio 3: Liderando com dados Neste

estágio, a empresa é orientada por dados. Os pipelines e sistemas automatizados criados por engenheiros de dados permitem que as pessoas dentro da empresa façam análises de autoatendimento e ML. A introdução de novas fontes de dados é perfeita e o valor tangível é derivado. Os engenheiros de dados implementam controles e práticas adequados para garantir que os dados estejam sempre disponíveis para as pessoas e os sistemas.
As funções de engenharia de dados continuam a se especializar mais profundamente do que no estágio 2.

Nas organizações no estágio 3 de maturidade de dados, um engenheiro de dados continuará desenvolvendo os estágios anteriores, além de fazer o seguinte:

Crie automação para a introdução e uso contínuos de novos dados

Concentre-se na criação de ferramentas e sistemas personalizados que aproveitem os dados como uma vantagem competitiva

Concentre-se nos aspectos “empresariais” dos dados, como gerenciamento de dados (incluindo governança e qualidade de dados) e DataOps

Implante ferramentas que expõem e divulgam dados em toda a organização, incluindo catálogos de dados, ferramentas de linhagem de dados e sistemas de gerenciamento de metadados

Colabore de forma eficiente com engenheiros de software, engenheiros de ML, analistas e outros

Crie uma comunidade e um ambiente onde as pessoas possam colaborar e falar abertamente, independentemente de sua função ou posição

Os problemas a serem observados incluem o seguinte:

Machine Translated by Google

Nesta fase, a complacência é um perigo significativo. Quando as organizações atingem o estágio 3, elas devem se concentrar constantemente na manutenção e melhoria ou correm o risco de voltar a um estágio inferior.

As distrações tecnológicas são um perigo mais significativo aqui do que nos outros estágios. Existe a tentação de perseguir projetos caros de hobby que não agregam valor ao negócio. Utilize a tecnologia personalizada apenas onde ela oferece uma vantagem competitiva.

O histórico e as habilidades de um engenheiro de dados

A engenharia de dados é um campo em rápido crescimento, e muitas perguntas permanecem sobre como se tornar um engenheiro de dados. Como a engenharia de dados é uma disciplina relativamente nova, pouco treinamento formal está disponível para entrar no campo.
As universidades não têm um caminho padrão de engenharia de dados. Embora um punhado de treinamentos de engenharia de dados e tutoriais on-line abordem tópicos aleatórios, ainda não existe um currículo comum para o assunto.

As pessoas que entram na engenharia de dados chegam com diferentes formações em educação, carreira e conjunto de habilidades. Todos que entram no campo devem esperar investir uma quantidade significativa de tempo em autoestudo. Ler este livro é um bom ponto de partida; um dos principais objetivos deste livro é fornecer uma base para o conhecimento e as habilidades que acreditamos serem necessárias para ter sucesso como engenheiro de dados.

Se você está transformando sua carreira em engenharia de dados, descobrimos que a transição é mais fácil quando se muda de um campo adjacente, como engenharia de software, desenvolvimento de ETL, administração de banco de dados, ciência de dados ou análise de dados. Essas disciplinas tendem a ser “conscientes de dados” e fornecem um bom contexto para funções de dados em uma organização. Eles também equipam as pessoas com as habilidades técnicas e contexto relevantes para resolver problemas de engenharia de dados.

Apesar da falta de um caminho formalizado, existe um corpo de conhecimento necessário que acreditamos que um engenheiro de dados deve saber para ter sucesso. Por definição, um engenheiro de dados deve entender tanto os dados quanto a tecnologia. Com relação aos dados, isso implica conhecer várias práticas recomendadas em torno do gerenciamento de dados. No lado da tecnologia, um engenheiro de dados deve estar ciente de

Machine Translated by Google

várias opções de ferramentas, sua interação e seus trade-offs. Isso requer um bom entendimento de engenharia de software, DataOps e arquitetura de dados.

Ao diminuir o zoom, um engenheiro de dados também deve entender os requisitos dos consumidores de dados (analistas de dados e cientistas de dados) e as implicações mais amplas dos dados em toda a organização. A engenharia de dados é uma prática holística; os melhores engenheiros de dados veem suas responsabilidades por meio de lentes comerciais e técnicas.

Responsabilidades Empresariais

As responsabilidades macro que listamos nesta seção não são exclusivas dos engenheiros de dados, mas são cruciais para quem trabalha em um campo de dados ou tecnologia. Como uma simples pesquisa no Google renderá muitos recursos para aprender sobre essas áreas, vamos simplesmente listá-los para brevidade:

Saber se comunicar com pessoas não técnicas e técnicas.

A comunicação é fundamental, e você precisa ser capaz de estabelecer rapport e confiança com as pessoas em toda a organização. Sugerimos prestar muita atenção às hierarquias organizacionais, quem se reporta a quem, como as pessoas interagem e quais silos existem. Essas observações serão inestimável para o seu sucesso.

Entenda como definir o escopo e reunir os requisitos de negócios e de produtos.

Você precisa saber o que construir e garantir que seus stakeholders concordem com sua avaliação. Além disso, desenvolva um senso de como os dados e decisões de tecnologia impactam o negócio.

Entenda os fundamentos culturais do Agile, DevOps e DataOps.

Muitos tecnólogos acreditam erroneamente que essas práticas são resolvidas através da tecnologia. Achamos que isso é perigosamente errado. Ágil, DevOps,

Machine Translated by Google

e DataOps são fundamentalmente culturais, exigindo adesão de toda a organização.

Controlar custos.

Você será bem-sucedido quando puder manter os custos baixos e, ao mesmo tempo, fornecer um valor descomunal. Saiba como otimizar o time to value, o custo total de propriedade e custo de oportunidade. Aprenda a monitorar os custos para evitar surpresas.

Aprenda continuamente.

O campo de dados parece estar mudando na velocidade da luz. Pessoas que têm sucesso nele são ótimos em pegar coisas novas enquanto aprimoram seus conhecimentos fundamentais. Eles também são bons em filtrar, determinando quais novos desenvolvimentos são mais relevantes para seu trabalho, que ainda são imaturos, e que são apenas modismos. Fique a par do campo e aprenda como aprender.

Um engenheiro de dados bem-sucedido sempre diminui o zoom para entender o quadro geral e como obter um valor descomunal para o negócio. A comunicação é vital, tanto para pessoas técnicas quanto para não-técnicas. Muitas vezes vemos equipes de dados bem-sucedidas com base em sua comunicação com outras partes interessadas; sucesso ou fracasso raramente é um problema de tecnologia. Saber como navegar em uma organização, definir o escopo e coletar requisitos, controlar custos e aprender continuamente o diferenciará dos engenheiros de dados que dependem apenas de suas habilidades técnicas para seguir sua carreira.

Responsabilidades Técnicas

Você deve entender como construir arquiteturas que otimizem desempenho e custo em alto nível, usando componentes pré-empacotados ou desenvolvidos internamente.

Machine Translated by Google

Em última análise, as arquiteturas e as tecnologias constituintes são blocos de construção para atender ao ciclo de vida da engenharia de dados. Lembre-se dos estágios do ciclo de vida da engenharia de dados:

Geração

Armazenar

Ingestão

Transformação

Servindo

As correntes subjacentes do ciclo de vida da engenharia de dados são as seguintes:

Segurança

Gestão de dados

DataOps

Arquitetura de dados

Engenharia de software

Ampliando um pouco, discutimos alguns dos dados táticos e habilidades tecnológicas que você precisará como engenheiro de dados nesta seção; discutimos isso com mais detalhes nos capítulos subsequentes.

As pessoas costumam perguntar: um engenheiro de dados deve saber codificar? Resposta curta: sim. Um engenheiro de dados deve ter habilidades de engenharia de software de nível de produção. Observamos que a natureza dos projetos de desenvolvimento de software realizados por engenheiros de dados mudou fundamentalmente nos últimos anos. Os serviços totalmente gerenciados agora substituem uma grande parte do esforço de programação de baixo nível anteriormente esperado dos engenheiros, que agora usam software como serviço (SaaS) plug-and-play simples e software como serviço (SaaS). Por exemplo, os engenheiros de dados agora se concentram em abstrações de alto nível ou escrevem pipelines como código dentro de uma estrutura de orquestração.

Machine Translated by Google

Mesmo em um mundo mais abstrato, as melhores práticas de engenharia de software fornecem uma vantagem competitiva, e os engenheiros de dados que podem mergulhar nos detalhes profundos da arquitetura de uma base de código dão a suas empresas uma vantagem quando surgem necessidades técnicas específicas. Em suma, um engenheiro de dados que não pode escrever código de nível de produção será severamente prejudicado, e não vemos isso mudando tão cedo. Os engenheiros de dados continuam sendo engenheiros de software, além de suas muitas outras funções.

Quais linguagens um engenheiro de dados deve conhecer? Dividimos as linguagens de programação de engenharia de dados em categorias primárias e secundárias. No momento da redação deste artigo, as principais linguagens de engenharia de dados são SQL, Python, uma linguagem Java Virtual Machine (JVM) (geralmente Java ou Scala) e bash:

SQL

A interface mais comum para bancos de dados e data lakes. Após breve sendo marginalizado pela necessidade de escrever código MapReduce personalizado para processamento de big data, o SQL (em várias formas) ressurgiu como o idioma franca de dados.

Pitão

A linguagem de ponte entre engenharia de dados e ciência de dados. Um número crescente de ferramentas de engenharia de dados são escritas em Python ou têm APIs Python. É conhecido como “a segunda melhor linguagem em tudo”.
O Python está subjacente a ferramentas de dados populares, como pandas, NumPy, Airflow, sci-kit learn, TensorFlow, PyTorch e PySpark. Python é a cola entre os componentes subjacentes e é frequentemente uma API de primeira classe linguagem para interface com um framework.

Linguagens JVM como Java e Scala

Prevalente para projetos de código aberto Apache, como Spark, Hive e Druida. A JVM geralmente tem mais desempenho que o Python e pode

Machine Translated by Google

fornecer acesso a recursos de nível inferior do que uma API do Python (por exemplo, esse é o caso do Apache Spark e do Beam). Entendendo Java ou Scala será benéfico se você estiver usando um software de código aberto popular estrutura.

festança

A interface de linha de comando para sistemas operacionais Linux. Conhecendo comandos bash e se sentir confortável usando CLIs melhorarão significativamente sua produtividade e fluxo de trabalho quando você precisar criar scripts ou executar operações do SO. Ainda hoje, engenheiros de dados usam frequentemente ferramentas de linha de comando como awk ou sed para processar arquivos em um pipeline de dados ou chamar comandos bash de estruturas de orquestração. Se você estiver usando Windows, sinta-se à vontade para substituir o PowerShell por bash.

Machine Translated by Google

A EFICÁCIA IRRACIONAL DO SQL

O advento do MapReduce e a era do big data relegaram o SQL ao status de ultrapassado. Desde então, vários desenvolvimentos melhoraram drasticamente a utilidade do SQL no ciclo de vida da engenharia de dados. Spark SQL, Google BigQuery, Snowflake, Hive e muitas outras ferramentas de dados podem processar grandes quantidades de dados usando semântica SQL declarativa e teórica de conjunto. SQL também é suportado por muitas estruturas de streaming, como Apache Flink, Beam e Kafka. Acreditamos que engenheiros de dados competentes devem ser altamente proficientes em SQL.

Estamos dizendo que o SQL é uma linguagem essencial e final? De jeito nenhum. O SQL é uma ferramenta poderosa que pode resolver rapidamente problemas complexos de análise e transformação de dados. Dado que o tempo é a principal restrição para o rendimento da equipe de engenharia de dados, os engenheiros devem adotar ferramentas que combinem simplicidade e alta produtividade. Os engenheiros de dados também fazem bem em desenvolver experiência na composição de SQL com outras operações, seja em estruturas como Spark e Flink ou usando orquestração para combinar várias ferramentas. Os engenheiros de dados também devem aprender a semântica SQL moderna para lidar com a análise de JavaScript Object Notation (JSON) e dados aninhados e considerar o aproveitamento de uma estrutura de gerenciamento SQL, como dbt (Data Build Tool).

Um engenheiro de dados proficiente também reconhece quando o SQL não é a ferramenta certa para o trabalho e pode escolher e codificar uma alternativa adequada. Um especialista em SQL provavelmente poderia escrever uma consulta para derivar e tokenizar texto bruto em um pipeline de processamento de linguagem natural (NLP), mas também reconheceria que codificar em Spark nativo é uma alternativa muito superior a esse exercício masoquista.

Os engenheiros de dados também podem precisar desenvolver proficiência em linguagens de programação secundárias, incluindo R, JavaScript, Go, Rust, C/C++, C# e Julia. O desenvolvimento nesses idiomas geralmente é necessário quando é popular em toda a empresa ou usado com ferramentas de dados específicas de domínio. Por exemplo, JavaScript provou ser popular como uma linguagem para funções definidas pelo usuário em

Machine Translated by Google

armazenamentos de dados em nuvem. Ao mesmo tempo, C# e Power Shell são essenciais em empresas que utilizam o Azure e o ecossistema da Microsoft.

MANTENDO O RITMO EM UM CAMPO RÁPIDO

> ***Uma vez que uma nova tecnologia passa por cima de você, se você não faz parte do rolo compressor, você faz parte da estrada.***
>
> —Marca Stewart

Como você mantém suas habilidades afiadas em um campo em rápida mudança, como a engenharia de dados? Você deve se concentrar nas ferramentas mais recentes ou mergulhar profundamente nos fundamentos? Aqui está nosso conselho: concentre-se nos fundamentos para entender o que não vai mudar; preste atenção aos desenvolvimentos em curso para saber para onde o campo está indo. Novos paradigmas e práticas são introduzidos o tempo todo, e cabe a você se manter atualizado. Esforce-se para entender como as novas tecnologias serão úteis no ciclo de vida.

A continuidade das funções de engenharia de dados, de A a B

Embora as descrições de trabalho pintem um engenheiro de dados como um “unicórnio” que deve possuir todas as habilidades de dados imagináveis, os engenheiros de dados não fazem o mesmo tipo de trabalho ou têm o mesmo conjunto de habilidades. A maturidade de dados é um guia útil para entender os tipos de desafios de dados que uma empresa enfrentará à medida que aumenta sua capacidade de dados. É benéfico observar algumas distinções críticas nos tipos de trabalho que os engenheiros de dados realizam. Embora essas distinções sejam simplistas, elas esclarecem o que cientistas de dados e engenheiros de dados fazem e evitam colocar qualquer um dos papéis no balde do unicórnio.

Na ciência de dados, há a noção de cientistas de dados tipo A e tipo B.
Os cientistas de dados do tipo A – onde A significa ***análise – concentram-se*** em entender e obter insights dos dados. ***Cientistas de dados do tipo B*** – onde ***B*** significa ***construção – compartilham*** experiências semelhantes aos cientistas de dados do tipo A e possuem fortes habilidades de programação. O cientista de dados tipo B constrói sistemas que fazem a ciência de dados funcionar na produção. Empréstimo de

Machine Translated by Google

neste continuum de cientistas de dados, criaremos uma distinção semelhante para dois tipos de engenheiros de dados:

Engenheiros de dados tipo A

A significa ***abstração.*** Nesse caso, o engenheiro de dados evita o trabalho pesado indiferenciado, mantendo a arquitetura de dados o mais abstrata e direta possível e não reinventando a roda. Dados tipo A
engenheiros gerenciam o ciclo de vida da engenharia de dados principalmente usando produtos totalmente prontos para uso, serviços gerenciados e ferramentas. Os engenheiros de dados do Tipo A trabalham em empresas de todos os setores e em todos os níveis de maturidade de dados.

Engenheiros de dados tipo B

B significa ***construção.*** Os engenheiros de dados do tipo B constroem ferramentas e sistemas de dados que escalam e alavancam a competência central e competitiva de uma empresa vantagem. Na faixa de maturidade de dados, um engenheiro de dados tipo B é mais comumente encontrado em empresas nos estágios 2 e 3 (escalonamento e liderança com dados) ou quando um caso de uso de dados inicial é tão único e essencial que ferramentas de dados personalizadas são necessárias para começar.

Os engenheiros de dados do tipo A e do tipo B podem trabalhar na mesma empresa e podem até ser a mesma pessoa! Mais comumente, um engenheiro de dados tipo A é contratado pela primeira vez para estabelecer a base, com conjuntos de habilidades de engenheiro de dados tipo B aprendidos ou contratados conforme a necessidade surgir dentro de uma empresa.

Engenheiros de dados dentro de uma organização

Os engenheiros de dados não trabalham no vácuo. Dependendo do que eles estão trabalhando, eles irão interagir com pessoas técnicas e não técnicas e

Machine Translated by Google

enfrentar diferentes direções (interna e externa). Vamos explorar o que os engenheiros de dados fazem dentro de uma organização e com quem eles interagem.

Engenheiros de dados internos versus externos

Um engenheiro de dados atende a vários usuários finais e enfrenta muitas direções internas e externas (Figura 1-9). Como nem todas as cargas de trabalho e responsabilidades de engenharia de dados são as mesmas, é essencial entender a quem o engenheiro de dados atende. Dependendo dos casos de uso final, as principais responsabilidades de um engenheiro de dados são externas, internas ou uma combinação das duas.

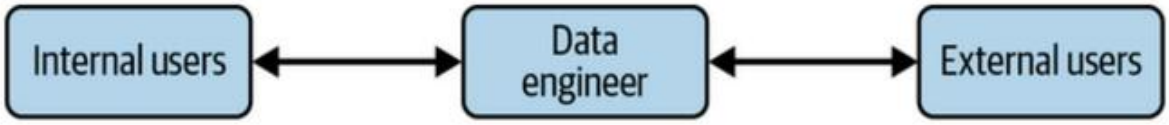


Figura 1-9. As direções que um engenheiro de dados enfrenta

Um engenheiro ***de dados externo*** normalmente se alinha com os usuários de aplicativos externos, como aplicativos de mídia social, dispositivos da Internet das Coisas (IoT) e plataformas de comércio eletrônico. Esse engenheiro de dados arquiteta, constrói e gerencia os sistemas que coletam, armazenam e processam dados transacionais e de eventos desses aplicativos. Os sistemas construídos por esses engenheiros de dados têm um ciclo de feedback do aplicativo para o pipeline de dados e, em seguida, de volta ao aplicativo (Figura 1-10).

Machine Translated by Google

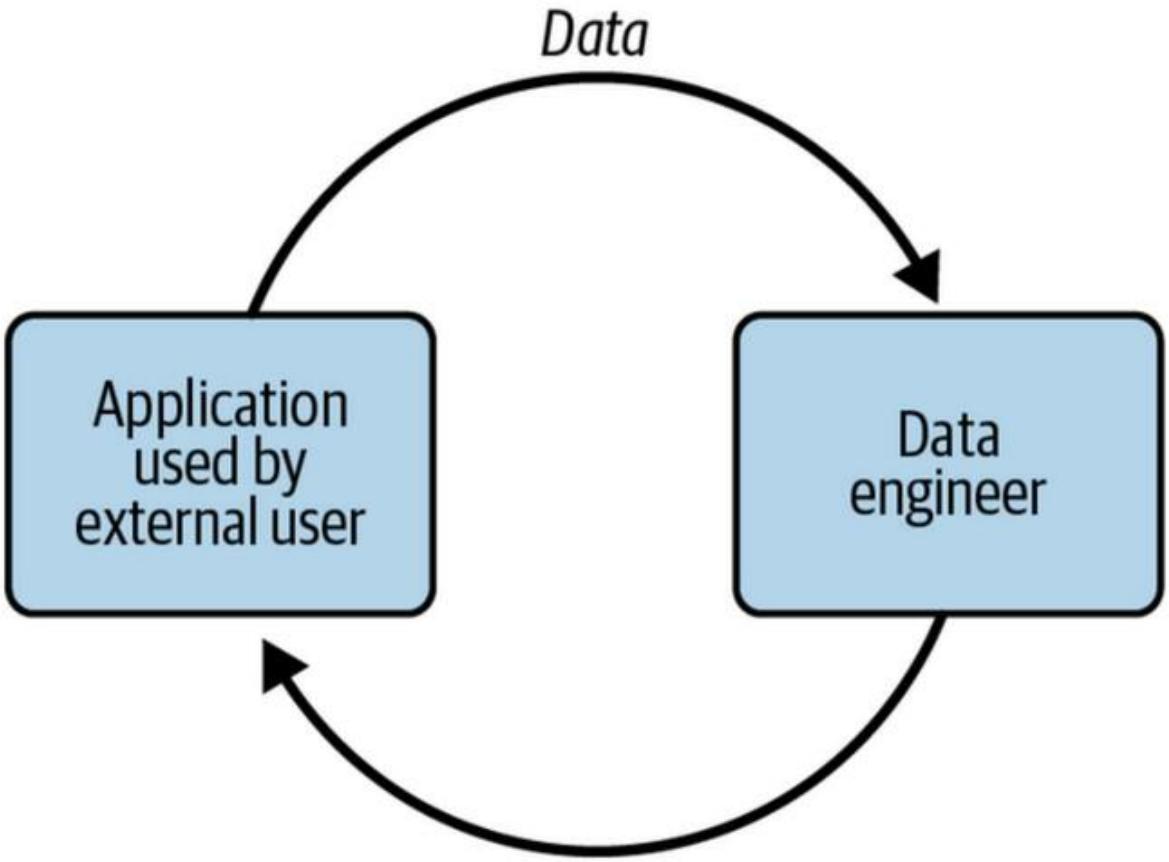


Figura 1-10. Sistemas de engenharia de dados voltados para o exterior

A engenharia de dados externa vem com um conjunto único de problemas.
Os mecanismos de consulta externos geralmente lidam com cargas de simultaneidade muito maiores do que os sistemas internos. Os engenheiros também precisam considerar colocar limites rígidos nas consultas que os usuários podem executar para limitar o impacto na infraestrutura de um único usuário. Além disso, a segurança é um problema muito mais complexo e sensível para consultas externas, especialmente se os dados consultados forem multilocatários (dados de muitos clientes e armazenados em uma única tabela).

Um ***engenheiro de dados voltado para o interno*** normalmente se concentra em atividades cruciais para as necessidades dos negócios e das partes interessadas internas (Figura 1-11). Os exemplos incluem a criação e manutenção de pipelines de dados e data warehouses para painéis de BI, relatórios, processos de negócios, ciência de dados e modelos de ML.



Figura 1-11. Engenheiro de dados voltado para o interior

Machine Translated by Google

As responsabilidades externas e internas são muitas vezes combinadas. Na prática, os dados internos geralmente são um pré-requisito para os dados externos. O engenheiro de dados tem dois conjuntos de usuários com requisitos muito diferentes para simultaneidade de consulta, segurança e muito mais.

Engenheiros de dados e outras funções técnicas

Na prática, o ciclo de vida da engenharia de dados abrange muitos domínios de responsabilidade. Os engenheiros de dados estão no nexo de várias funções, diretamente ou por meio de gerentes, interagindo com muitas unidades organizacionais.

Vejamos quem um engenheiro de dados pode impactar. Nesta seção, discutiremos as funções técnicas relacionadas à engenharia de dados (Figura 1-12).

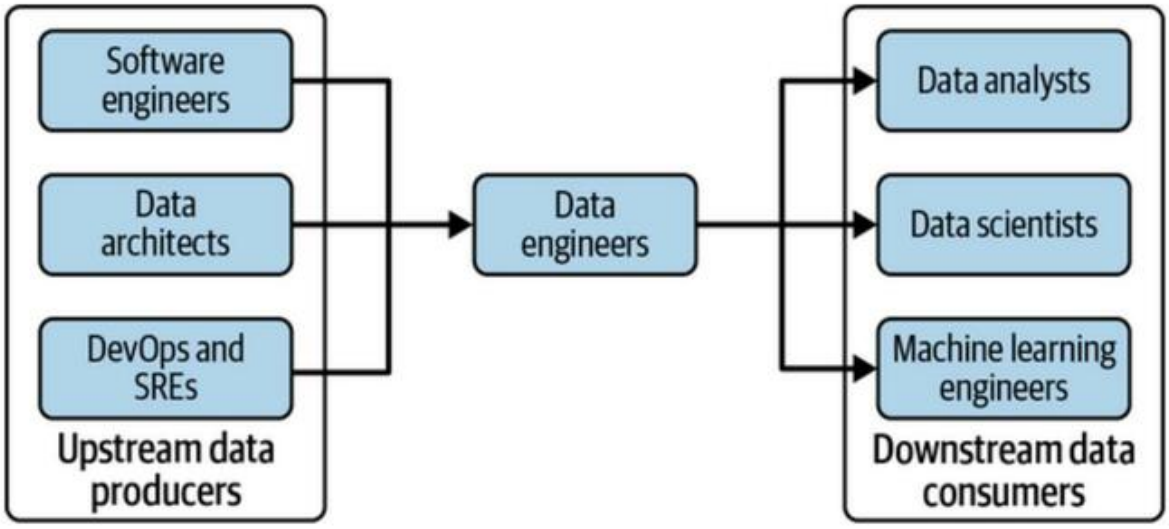


Figura 1-12. Principais interessados técnicos da engenharia de dados

O engenheiro de dados é um hub entre ***produtores de dados,*** como engenheiros de software, arquitetos de dados e DevOps ou engenheiros de confiabilidade do site (SREs) e ***consumidores de dados,*** como analistas de dados, cientistas de dados e engenheiros de ML. Além disso, os engenheiros de dados irão interagir com aqueles em funções operacionais, como engenheiros de DevOps.

Dado o ritmo em que novas funções de dados entram em voga (engenheiros de análise e ML vêm à mente), essa não é uma lista exaustiva.

Partes interessadas a montante

Machine Translated by Google

Para ter sucesso como engenheiro de dados, você precisa entender a arquitetura de dados que está usando ou projetando e os sistemas de origem que produzem os dados necessários. Em seguida, discutimos algumas partes interessadas de upstream familiares: arquitetos de dados, engenheiros de software e engenheiros de DevOps.

Arquitetos de dados

Os arquitetos de dados funcionam em um nível de abstração a um passo dos engenheiros de dados. Os arquitetos de dados projetam o plano para o gerenciamento de dados organizacionais, mapeando processos e arquitetura e sistemas gerais de dados. Eles também servem como uma ponte entre os lados técnicos e não técnicos de uma organização. Arquitetos de dados bem-sucedidos geralmente têm “cicatrizes de batalha” de extensa experiência em engenharia, permitindo que eles guiem e ajudem os engenheiros enquanto comunicam com sucesso os desafios de engenharia para as partes interessadas não técnicas.

Os arquitetos de dados implementam políticas para gerenciar dados em silos e unidades de negócios, orientam estratégias globais, como gerenciamento de dados e governança de dados, e orientam iniciativas significativas. Os arquitetos de dados geralmente desempenham um papel central nas migrações de nuvem e no design de nuvem greenfield.

O advento da nuvem mudou a fronteira entre arquitetura de dados e engenharia de dados. As arquiteturas de dados em nuvem são muito mais fluidas do que os sistemas locais, portanto, as decisões de arquitetura que tradicionalmente envolviam estudo extensivo, longos prazos de entrega, contratos de compra e instalação de hardware agora são tomadas durante o processo de implementação, apenas uma etapa de uma estratégia maior. No entanto, os arquitetos de dados continuarão sendo visionários influentes nas empresas, trabalhando lado a lado com engenheiros de dados para determinar o panorama geral das práticas de arquitetura e estratégias de dados.

Dependendo da maturidade e tamanho dos dados da empresa, um engenheiro de dados pode se sobrepor ou assumir as responsabilidades de um arquiteto de dados. Portanto, um engenheiro de dados deve ter um bom entendimento das melhores práticas e abordagens de arquitetura.

Observe que colocamos arquitetos de dados na seção de ***partes interessadas upstream .*** Os arquitetos de dados geralmente ajudam a projetar camadas de dados de aplicativos que são sistemas de origem para engenheiros de dados. Arquitetos também podem interagir com dados

Machine Translated by Google

engenheiros em vários outros estágios do ciclo de vida da engenharia de dados. Cobrimos a "boa" arquitetura de dados no Capítulo 3.

Engenheiros de software

Os engenheiros de software constroem o software e os sistemas que administram um negócio; eles são amplamente responsáveis por gerar os ***dados internos*** que os engenheiros de dados consumirão e processarão. Os sistemas criados por engenheiros de software geralmente geram dados e logs de eventos de aplicativos, que são ativos significativos por si só. Esses dados internos contrastam com ***dados externos*** extraídos de plataformas SaaS ou empresas parceiras. Em organizações técnicas bem administradas, engenheiros de software e engenheiros de dados coordenam desde o início de um novo projeto para projetar dados de aplicativos para consumo por aplicativos de análise e ML.

Um engenheiro de dados deve trabalhar em conjunto com engenheiros de software para entender os aplicativos que geram dados, o volume, a frequência e o formato dos dados gerados e qualquer outra coisa que afete o ciclo de vida da engenharia de dados, como segurança de dados e conformidade regulatória. Por exemplo, isso pode significar definir expectativas de upstream sobre o que os engenheiros de software de dados precisam para fazer seus trabalhos. Os engenheiros de dados devem trabalhar em estreita colaboração com os engenheiros de software.

Engenheiros de DevOps e engenheiros de confiabilidade do site

DevOps e SREs geralmente produzem dados por meio de monitoramento operacional. Nós os classificamos como upstream de engenheiros de dados, mas eles também podem ser downstream, consumindo dados por meio de painéis ou interagindo com engenheiros de dados diretamente na coordenação de operações de sistemas de dados.

Partes interessadas a jusante

A profissão moderna de engenharia de dados existe para atender consumidores de dados downstream e casos de uso. Esta seção discute como os engenheiros de dados interagem com várias funções downstream. Também apresentaremos alguns modelos de serviço, incluindo equipes de engenharia de dados centralizadas e equipes multifuncionais.

Cientistas de dados

Machine Translated by Google

Os cientistas de dados criam modelos prospectivos para fazer previsões e recomendações. Esses modelos são então avaliados em dados ao vivo para fornecer valor de várias maneiras. Por exemplo, a pontuação do modelo pode determinar ações automatizadas em resposta a condições em tempo real, recomendar produtos aos clientes com base no histórico de navegação em sua sessão atual ou fazer previsões econômicas ao vivo usadas pelos traders.

De acordo com o folclore comum da indústria, os cientistas de dados gastam de 70% a 80% de seu tempo coletando, limpando e preparando dados. Em nossa experiência, esses números geralmente refletem práticas imaturas de ciência de dados e engenharia de dados. Em particular, muitas estruturas populares de ciência de dados podem se tornar gargalos se não forem dimensionadas adequadamente. Os cientistas de dados que trabalham exclusivamente em uma única estação de trabalho se forçam a reduzir a amostragem de dados, tornando a preparação de dados significativamente mais complicada e potencialmente comprometendo a qualidade dos modelos que produzem. Além disso, código e ambientes desenvolvidos localmente são muitas vezes difíceis de implantar na produção, e a falta de automação dificulta significativamente os fluxos de trabalho de ciência de dados. Se os engenheiros de dados fizerem seu trabalho e colaborarem com sucesso, os cientistas de dados não deverão gastar seu tempo coletando, limpando e preparando dados após o trabalho exploratório inicial. Os engenheiros de dados devem automatizar esse trabalho o máximo possível.

A necessidade de ciência de dados pronta para produção é um fator significativo por trás do surgimento da profissão de engenharia de dados. Os engenheiros de dados devem ajudar os cientistas de dados a permitir um caminho para a produção. Na verdade, nós (os autores) passamos da ciência de dados para a engenharia de dados depois de reconhecer essa necessidade fundamental. Os engenheiros de dados trabalham para fornecer automação e escala de dados que tornam a ciência de dados mais eficiente.

Analistas de dados

Os analistas de dados (ou analistas de negócios) procuram entender o desempenho e as tendências dos negócios. Enquanto os cientistas de dados estão voltados para o futuro, um analista de dados normalmente se concentra no passado ou no presente. Os analistas de dados geralmente executam consultas SQL em um data warehouse ou data lake. Eles também podem utilizar planilhas para cálculo e análise e várias ferramentas de BI, como

Machine Translated by Google

Microsoft Power BI, Looker ou Tableau. Os analistas de dados são especialistas no domínio dos dados com os quais trabalham com frequência e se familiarizam intimamente com as definições, características e problemas de qualidade dos dados. Os clientes downstream típicos de um analista de dados são usuários de negócios, gerentes e executivos.

Os engenheiros de dados trabalham com analistas de dados para criar pipelines para novas fontes de dados exigidas pelos negócios. A experiência no assunto dos analistas de dados é inestimável para melhorar a qualidade dos dados, e eles frequentemente colaboram com engenheiros de dados nessa capacidade.

Engenheiros de aprendizado de máquina e pesquisadores de IA

Engenheiros de aprendizado de máquina (engenheiros de ML) se sobrepõem a engenheiros de dados e cientistas de dados. Os engenheiros de ML desenvolvem técnicas avançadas de ML, treinam modelos e projetam e mantêm a infraestrutura que executa processos de ML em um ambiente de produção em escala. Os engenheiros de ML geralmente têm conhecimentos avançados de ML e técnicas de aprendizado profundo e estruturas como PyTorch ou TensorFlow.

Os engenheiros de ML também entendem o hardware, os serviços e os sistemas necessários para executar essas estruturas, tanto para treinamento de modelos quanto para implantação de modelos em escala de produção. É comum que os fluxos de ML sejam executados em um ambiente de nuvem em que os engenheiros de ML possam aumentar e dimensionar recursos de infraestrutura sob demanda ou contar com serviços gerenciados.

Como mencionamos, os limites entre engenharia de ML, engenharia de dados e ciência de dados são confusos. Os engenheiros de dados podem ter algumas responsabilidades de DevOps sobre os sistemas de ML, e os cientistas de dados podem trabalhar em estreita colaboração com a engenharia de ML na criação de processos avançados de ML.

O mundo da engenharia de ML está se tornando uma bola de neve e é paralelo a muitos dos mesmos desenvolvimentos que ocorrem na engenharia de dados. Enquanto há vários anos a atenção do ML estava focada em como construir modelos, a engenharia de ML agora enfatiza cada vez mais a incorporação das melhores práticas de operações de aprendizado de máquina (MLOps) e outras práticas maduras anteriormente adotadas em engenharia de software e DevOps.

Machine Translated by Google

Os pesquisadores de IA trabalham em novas e avançadas técnicas de ML. Os pesquisadores de IA podem trabalhar dentro de grandes empresas de tecnologia, startups especializadas em propriedade intelectual (OpenAI, DeepMind) ou instituições acadêmicas. Alguns profissionais se dedicam à pesquisa em meio período em conjunto com as responsabilidades de engenharia de ML dentro de uma empresa. Aqueles que trabalham em laboratórios especializados de ML geralmente são 100% dedicados à pesquisa. Os problemas de pesquisa podem visar aplicações práticas imediatas ou demonstrações mais abstratas de IA.
AlphaGo e GPT-3/GPT-4 são ótimos exemplos de projetos de pesquisa de ML.
Fornecemos algumas referências em “Recursos Adicionais”.

Os pesquisadores de IA em organizações bem financiadas são altamente especializados e operam com equipes de engenheiros de apoio para facilitar seu trabalho. Os engenheiros de ML na academia geralmente têm menos recursos, mas contam com equipes de estudantes de pós-graduação, pós-doutorandos e funcionários da universidade para fornecer suporte de engenharia. Os engenheiros de ML que se dedicam parcialmente à pesquisa geralmente contam com as mesmas equipes de suporte para pesquisa e produção.

Engenheiros de dados e liderança de negócios

Discutimos as funções técnicas com as quais um engenheiro de dados interage. Mas os engenheiros de dados também operam de forma mais ampla como conectores organizacionais, geralmente em uma capacidade não técnica. As empresas passaram a confiar cada vez mais nos dados como parte central de muitos produtos ou como um produto em si. Os engenheiros de dados agora participam do planejamento estratégico e lideram as principais iniciativas que vão além dos limites da TI. Os engenheiros de dados geralmente dão suporte aos arquitetos de dados atuando como a cola entre os negócios e a ciência/análise de dados.

Dados no C-suite

Os executivos de nível C estão cada vez mais envolvidos em dados e análises, pois são reconhecidos como ativos significativos para as empresas modernas. Os CEOs agora se preocupam com iniciativas que antes eram exclusivas da TI, como migrações para a nuvem ou implantação de uma nova plataforma de dados de clientes.

Machine Translated by Google

Diretor executivo

Os diretores executivos (CEOs) de empresas não tecnológicas geralmente não se preocupam com o âmago da questão das estruturas de dados e do software.
Em vez disso, eles definem uma visão em colaboração com funções técnicas do C-suite e liderança de dados da empresa. Os engenheiros de dados fornecem uma janela para o que é possível com os dados. Os engenheiros de dados e seus gerentes mantêm um mapa de quais dados estão disponíveis para a organização - tanto internamente quanto de terceiros - em que período de tempo. Eles também são encarregados de estudar as principais alterações arquitetônicas de dados em colaboração com outras funções de engenharia. Por exemplo, os engenheiros de dados geralmente estão fortemente envolvidos em migrações de nuvem, migrações para novos sistemas de dados ou implantação de tecnologias de streaming.

Diretor de informações

Um diretor de informação (CIO) é o executivo sênior C-suite responsável pela tecnologia da informação dentro de uma organização; é um papel voltado para o interior. Um CIO deve possuir um conhecimento profundo de tecnologia da informação e processos de negócios – ou sozinho é insuficiente. Os CIOs dirigem a organização de tecnologia da informação, definindo políticas contínuas e, ao mesmo tempo, definindo e executando iniciativas significativas sob a direção do CEO.

Os CIOs geralmente colaboram com a liderança de engenharia de dados em organizações com uma cultura de dados bem desenvolvida. Se uma organização não tiver maturidade de dados muito alta, um CIO normalmente ajudará a moldar sua cultura de dados. Os CIOs trabalharão com engenheiros e arquitetos para mapear as principais iniciativas e tomar decisões estratégicas sobre a adoção dos principais elementos de arquitetura, como sistemas de planejamento de recursos empresariais (ERP) e gerenciamento de relacionamento com o cliente (CRM), migrações de nuvem, sistemas de dados e TI interna

Diretor de tecnologia

Um diretor de tecnologia (CTO) é semelhante a um CIO, mas voltado para fora. Um CTO possui a principal estratégia tecnológica e arquiteturas para aplicativos externos, como dispositivos móveis, aplicativos da Web e IoT - todas as fontes de dados críticas para engenheiros de dados. O CTO é provavelmente um tecnólogo habilidoso e tem

Machine Translated by Google

um bom senso de fundamentos de engenharia de software e arquitetura de sistema. Em algumas organizações sem um CIO, o CTO ou, às vezes, o diretor de operações (COO) desempenha o papel de CIO. Os engenheiros de dados geralmente se reportam direta ou indiretamente por meio de um CTO.

Diretor de dados

O Chief Data Officer (CDO) foi criado em 2002 na Capital One para reconhecer a crescente importância dos dados como um ativo de negócios. O CDO é responsável pelos ativos de dados e estratégia de uma empresa. Os CDOs estão focados na utilidade comercial dos dados, mas devem ter uma base técnica sólida. Os CDOs supervisionam produtos de dados, estratégias, iniciativas e funções essenciais, como gerenciamento de dados mestre e privacidade. Ocasionalmente, os CDOs gerenciam análises de negócios e engenharia de dados.

Diretor de análise

O diretor de análise (CAO) é uma variante da função de CDO. Onde existem ambas as funções, o CDO se concentra na tecnologia e na organização necessárias para fornecer dados. O CAO é responsável pela análise, estratégia e tomada de decisões para o negócio. Um CAO pode supervisionar a ciência de dados e o ML, embora isso dependa muito se a empresa tem uma função de CDO ou CTO.

Diretor de algoritmos Um

diretor de algoritmos (CAO-2) é uma inovação recente no C-suite, uma função altamente técnica focada especificamente em ciência de dados e ML. Os CAO-2s normalmente têm experiência como colaboradores individuais e líderes de equipe em projetos de ciência de dados ou ML. Frequentemente, eles têm experiência em pesquisa de ML e um grau avançado relacionado.

Espera-se que os CAO-2s estejam familiarizados com a pesquisa atual de ML e tenham profundo conhecimento técnico das iniciativas de ML de sua empresa. Além de criar iniciativas de negócios, eles fornecem liderança técnica, definem agendas de pesquisa e desenvolvimento e formam equipes de pesquisa.

Engenheiros de dados e gerentes de projeto

Machine Translated by Google

Os engenheiros de dados geralmente trabalham em iniciativas significativas, que podem durar muitos anos. Enquanto escrevemos este livro, muitos engenheiros de dados estão trabalhando em migrações de nuvem, migrando pipelines e armazéns para a próxima geração de ferramentas de dados. Outros engenheiros de dados estão iniciando projetos greenfield, montando novas arquiteturas de dados do zero, selecionando entre um número surpreendente de opções de arquitetura e ferramentas de ponta.

Essas grandes iniciativas geralmente se beneficiam do ***gerenciamento de projetos*** (em contraste com o gerenciamento de produtos, discutido a seguir). Enquanto os engenheiros de dados funcionam em uma infraestrutura e capacidade de entrega de serviços, os gerentes de projeto direcionam o tráfego e atuam como gatekeepers. A maioria dos gerentes de projeto opera de acordo com alguma variação de Agile e Scrum, com Waterfall ainda aparecendo ocasionalmente. Os negócios nunca dormem, e as partes interessadas nos negócios geralmente têm um acúmulo significativo de coisas que desejam resolver e novas iniciativas que desejam lançar. Os gerentes de projeto devem filtrar uma longa lista de solicitações e priorizar entregas críticas para manter os projetos no caminho certo e atender melhor à empresa.

Os engenheiros de dados interagem com os gerentes de projeto, muitas vezes planejando sprints para projetos e realizando standups relacionados ao sprint. O feedback ocorre nos dois sentidos, com engenheiros de dados informando os gerentes de projeto e outras partes interessadas sobre o progresso e os bloqueadores, e os gerentes de projeto equilibrando a cadência das equipes de tecnologia em relação às necessidades em constante mudança dos negócios.

Engenheiros de dados e gerentes de produto

Os gerentes de produto supervisionam o desenvolvimento de produtos, geralmente possuindo linhas de produtos. No contexto dos engenheiros de dados, esses produtos são chamados ***de produtos de dados.*** Os produtos de dados são construídos desde o início ou são melhorias incrementais em produtos existentes. Os engenheiros de dados interagem com mais frequência com ***os gerentes de produto ,*** pois o mundo corporativo adotou um foco centrado em dados. Assim como os gerentes de projeto, os gerentes de produto equilibram a atividade das equipes de tecnologia com as necessidades do cliente e do negócio.

Engenheiros de dados e outras funções de gerenciamento

Machine Translated by Google

Os engenheiros de dados interagem com vários gerentes além dos gerentes de projeto e produto. No entanto, essas interações geralmente seguem os serviços ou modelos multifuncionais. Os engenheiros de dados atendem a uma variedade de solicitações recebidas como uma equipe centralizada ou trabalham como um recurso atribuído a um gerente, projeto ou produto específico.

Para obter mais informações sobre equipes de dados e como estruturá-las, recomendamos o ***Building Analytics Teams*** (Packt) de John Thompson e o ***Data Teams*** de Jesse Anderson (Apress). Ambos os livros fornecem estruturas e perspectivas sólidas sobre os papéis dos executivos com dados, quem contratar e como construir a equipe de dados mais eficaz para sua empresa.

NOTA

As empresas não contratam engenheiros simplesmente para hackear o código isoladamente. Para serem dignos de seu título, os engenheiros devem desenvolver uma compreensão profunda dos problemas que têm a tarefa de resolver, as ferramentas tecnológicas à sua disposição e as pessoas com quem trabalham e servir.

Conclusão

Este capítulo forneceu uma breve visão geral do cenário de engenharia de dados, incluindo o seguinte:

- Definindo a engenharia de dados e descrevendo o que os engenheiros de dados fazem
- Descrevendo os tipos de maturidade de dados em uma empresa
- Engenheiros de dados tipo A e tipo B
- Com quem os engenheiros de dados trabalham

Esperamos que este primeiro capítulo tenha aguçado seu apetite, seja você um praticante de desenvolvimento de software, cientista de dados, engenheiro de ML, stakeholder de negócios, empresário ou capitalista de risco. É claro que muito ainda precisa ser elucidado nos capítulos subsequentes. O Capítulo 2 cobre o ciclo de vida da engenharia de dados, seguido pela arquitetura no Capítulo 3.

Machine Translated by Google

capítulos abordam o âmago da questão das decisões de tecnologia para cada parte do ciclo de vida. Todo o campo de dados está em fluxo e, tanto quanto possível, cada capítulo se concentra nos imutáveis — perspectivas que serão válidas por muitos anos em meio a mudanças implacáveis.

Recursos adicionais

"Sobre a Complexidade em Big Data" por Jesse Anderson (O'Reilly)

"Qual profissão é mais complexa de se tornar, um engenheiro de dados ou um cientista de dados?" tópico no Quora

"O futuro da engenharia de dados é a convergência de disciplinas" por Liam Hausmann

O site do Information Management Body of Knowledge

"Fazendo Ciência de Dados no Twitter" por Robert Chang

"Uma Breve História do Big Data" por Mark van Rijmenam

"Engenharia de Dados: Uma Definição Rápida e Simples" por James Furbush (O'Reilly)

"Big Data estará morto em cinco anos" por Lewis Gavin

"A Hierarquia das Necessidades da IA" por Monica Rogati

Capítulo 1 ***de O que é engenharia de dados?*** por Lewis Gavin (O'Reilly)

"Os três níveis de análise de dados: uma estrutura para avaliação de dados Maturidade da Organização" de Emilie Schario

"Dados como Produto versus Dados como Serviço" por Justin Gage

"A queda do engenheiro de dados" por Maxime Beauchemin

"A Ascensão do Engenheiro de Dados" por Maxime Beauchemin

"Habilidades do Arquiteto de Dados" por Bob Lambert

Machine Translated by Google

"O que é um arquiteto de dados? Visionário da estrutura de dados de TI" por Thor Olavsrud

"O novo gerador de linguagem GPT-3 da OpenAI é chocantemente bom – e Completamente Idiota" por Will Douglas Heaven

A página de pesquisa AlphaGo

"Como os CEOs podem liderar uma cultura orientada a dados" por Thomas H. Davenport e Nitin Mittal

"Por que os CEOs devem liderar iniciativas de Big Data" por John Weathington

"Como a criação de uma cultura orientada a dados pode impulsionar o sucesso" por Frederik Bussler

Construindo equipes de análise por John K. Thompson (Pacote)

Equipes de dados por Jesse Anderson (Apress)

"Conjunto de Conhecimentos em Gestão da Informação" Página da Wikipédia

"Gestão da Informação" Página da Wikipédia

1 "Data Engineering and Its Main Concepts", AlexSoft, atualizado pela última vez em 26 de agosto de 2021, https://oreil.ly/e94py.

2 ***ETL*** significa ***extrair, transformar, carregar,*** um padrão comum que abordamos no livro.

3 Jesse Anderson, "The Two Types of Data Engineering", 27 de junho de 2018, ***https://oreil.ly/dxDt6.***

4 Maxime Beauchemin, "The Rise of the Data Engineer", 20 de janeiro de 2017, ***https://oreil.ly/kNDmd.***

5 Lewis Gavin, ***O que é engenharia de dados?*** (Sebastapol, CA: O'Reilly, 2020), ***https://oreil.ly/ELxLi.***

6 Cade Metz, "How Yahoo Spawned Hadoop, the Future of Big Data", ***Wired,*** 18 de outubro de 2011, ***https://oreil.ly/iaD9G.***

7 Ron Miller, "How AWS Came to Be", ***TechCrunch,*** 2 de julho de 2016, ***https://oreil.ly/VJehv.***

8 ***DataOps*** é uma abreviação de ***operações de dados.*** Cobrimos esse tópico no Capítulo 2. Para obter mais informações, leia o Manifesto DataOps.

Machine Translated by Google

9 Essas siglas significam ***Lei de Privacidade do Consumidor da Califórnia*** e ***Proteção Geral de Dados regulamento,*** respectivamente.

10 Robert Chang, “Doing Data Science at Twitter”, ***Medium,*** 20 de junho de 2015, ***https://oreil.ly/xqjAx.***

11 Paramita (Guha) Ghosh, “Data Architect vs. Data Engineer”, Dataversity, 12 de novembro de 2021, ***https://oreil.ly/TlyZY.***

12 Existe uma variedade de referências para essa noção. Embora esse clichê seja amplamente conhecido, um debate saudável surgiu em torno de sua validade em diferentes contextos práticos. Para obter mais detalhes, consulte Leigh Dodds, “Os cientistas de dados gastam 80% de seu tempo limpando dados? Acontece, não?” Blog Lost Boy, 31 de janeiro de 2020, ***https://oreil.ly/szFww;*** e Alex Woodie, “Data Prep Still Dominates Data Scientists' Time, Survey Finds”, ***Datanami,*** 6 de julho de 2020, ***https://oreil.ly/jDVWF.***

Machine Translated by Google

Capítulo 2. O Ciclo de Vida da Engenharia de Dados

O principal objetivo deste livro é incentivá-lo a ir além da visão da engenharia de dados como uma coleção específica de tecnologias de dados. O cenário de dados está passando por uma explosão de novas tecnologias e práticas de dados, com níveis cada vez maiores de abstração e facilidade de uso.
Devido ao aumento da abstração técnica, os engenheiros de dados se tornarão cada vez mais ***engenheiros do ciclo de vida dos dados,*** pensando e operando em termos dos ***princípios*** do gerenciamento do ciclo de vida dos dados.

Neste capítulo, você aprenderá sobre o ***ciclo de vida da engenharia de dados,*** que é o tema central deste livro. O ciclo de vida da engenharia de dados é nossa estrutura que descreve a engenharia de dados "do berço ao túmulo". Você também aprenderá sobre as correntes ocultas do ciclo de vida da engenharia de dados, que são as principais bases que suportam todos os esforços de engenharia de dados.

Qual é o ciclo de vida da engenharia de dados?

O ciclo de vida da engenharia de dados compreende estágios que transformam ingredientes de dados brutos em um produto final útil, pronto para consumo por analistas, cientistas de dados, engenheiros de ML e outros. Este capítulo apresenta os principais estágios do ciclo de vida da engenharia de dados, concentrando-se nos conceitos principais de cada estágio e salvando detalhes para capítulos posteriores.

Dividimos o ciclo de vida da engenharia de dados nos cinco estágios a seguir (Figura 2-1, superior):

Geração

Armazenar

Ingestão

Machine Translated by Google

- Transformação
- Dados de veiculação

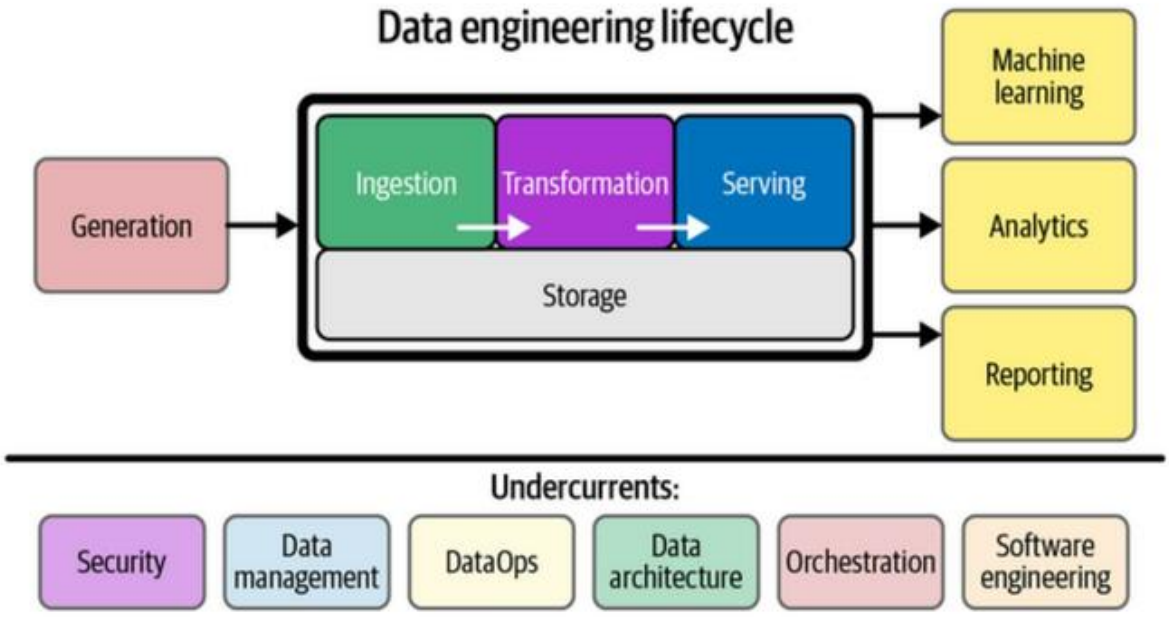


Figura 2-1. Componentes e subcorrentes do ciclo de vida da engenharia de dados

Começamos o ciclo de vida da engenharia de dados obtendo dados dos sistemas de origem e armazenando-os. Em seguida, transformamos os dados e prosseguimos para nosso objetivo central, fornecendo dados para analistas, cientistas de dados, engenheiros de ML e outros. Na realidade, o armazenamento ocorre durante todo o ciclo de vida, à medida que os dados fluem do início ao fim – portanto, o diagrama mostra o "estágio" de armazenamento como uma base que sustenta outros estágios.

Em geral, os estágios intermediários – armazenamento, ingestão, transformação – podem ficar um pouco confusos. E tudo bem. Embora dividamos as partes distintas do ciclo de vida da engenharia de dados, nem sempre é um fluxo puro e contínuo. Vários estágios do ciclo de vida podem se repetir, ocorrer fora de ordem, sobrepor-se ou entrelaçar-se de maneiras interessantes e inesperadas.

Atuando como um alicerce estão as ***correntes ocultas*** (Figura 2-1, inferior) que atravessam vários estágios do ciclo de vida da engenharia de dados: segurança, gerenciamento de dados, DataOps, arquitetura de dados, orquestração e engenharia de software. Nenhuma parte do ciclo de vida da engenharia de dados pode funcionar adequadamente sem essas tendências.

Machine Translated by Google

O ciclo de vida dos dados versus a engenharia de dados

Ciclo da vida

Você pode estar se perguntando sobre a diferença entre o ciclo de vida geral dos dados e o ciclo de vida da engenharia de dados. Há uma distinção sutil entre os dois. O ciclo de vida da engenharia de dados é um subconjunto de todo o ciclo de vida dos dados (Figura 2-2). Enquanto o ciclo de vida de dados completo abrange dados em toda a sua vida útil, o ciclo de vida de engenharia de dados se concentra nos estágios que um engenheiro de dados controla.

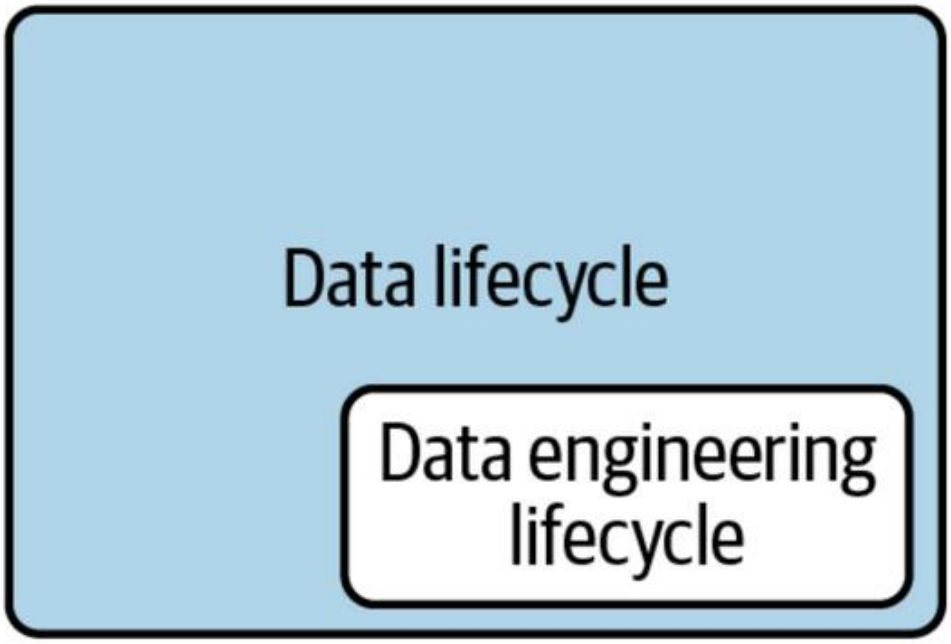


Figura 2-2. O ciclo de vida de engenharia de dados é um subconjunto do ciclo de vida de dados completo

Geração: Sistemas de Origem

Um ***sistema de origem*** é a origem dos dados usados no ciclo de vida da engenharia de dados. Por exemplo, um sistema de origem pode ser um dispositivo IoT, uma fila de mensagens do aplicativo ou um banco de dados transacional. Um engenheiro de dados consome dados de um sistema de origem, mas normalmente não possui ou controla o próprio sistema de origem. O engenheiro de dados precisa ter uma compreensão prática da maneira como os sistemas de origem funcionam, como eles geram dados, a frequência e a velocidade dos dados e a variedade de dados que eles geram.

Machine Translated by Google

Os engenheiros também precisam manter uma linha aberta de comunicação com os proprietários do sistema de origem sobre alterações que possam interromper pipelines e análises. O código do aplicativo pode alterar a estrutura dos dados em um campo ou a equipe do aplicativo pode até optar por migrar o back-end para uma tecnologia de banco de dados totalmente nova.

Um grande desafio na engenharia de dados é a variedade estonteante de engenheiros de sistemas de fonte de dados com os quais devem trabalhar e entender. Como ilustração, vejamos dois sistemas de origem comuns, um muito tradicional (um banco de dados de aplicativos) e outro um exemplo mais recente (enxames de IoT).

A Figura 2-3 ilustra um sistema de origem tradicional com vários servidores de aplicativos suportados por um banco de dados. Esse padrão de sistema de origem tornou-se popular na década de 1980 com o sucesso explosivo dos sistemas de gerenciamento de banco de dados relacional (RDBMSs). O padrão aplicativo + banco de dados permanece popular hoje com várias evoluções modernas de práticas de desenvolvimento de software.
Por exemplo, os aplicativos geralmente consistem em muitos pequenos pares de serviço/ banco de dados com microsserviços, em vez de um único monólito.

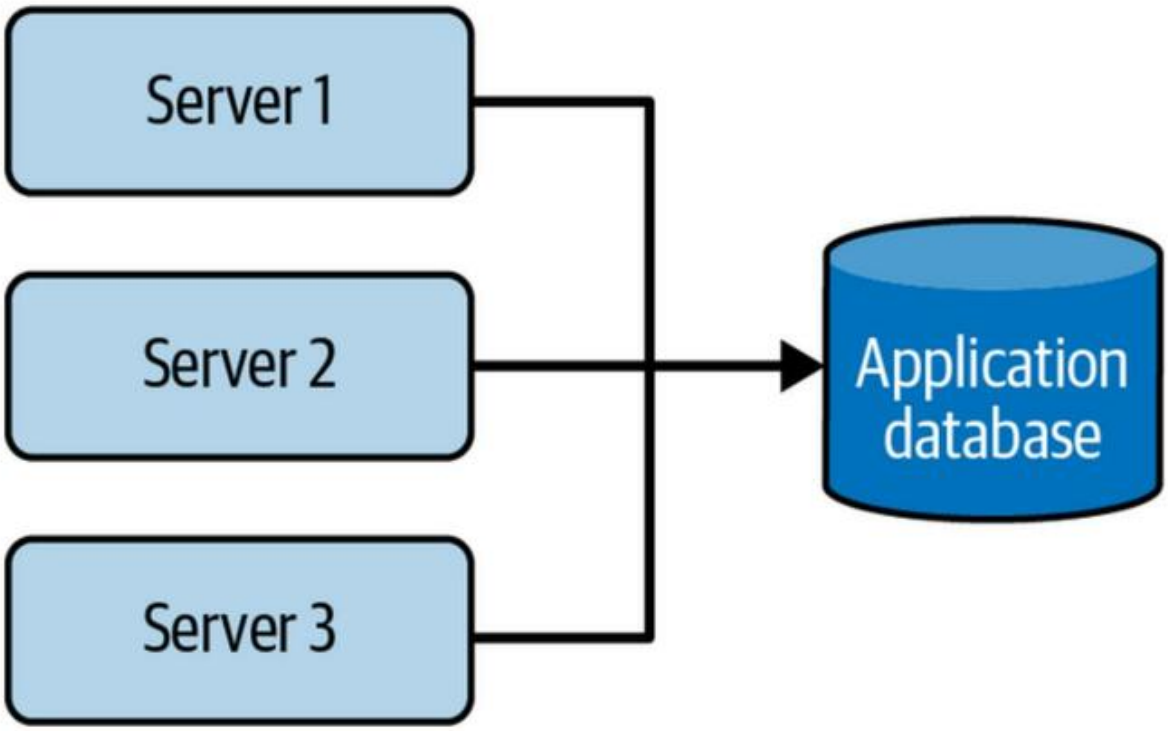


Figura 2-3. Exemplo de sistema de origem: um banco de dados de aplicativos

Machine Translated by Google

Vejamos outro exemplo de um sistema de origem. A Figura 2-4 ilustra um enxame de IoT: uma frota de dispositivos (círculos) envia mensagens de dados (retângulos) para um sistema de coleta central. Esse sistema de origem IoT é cada vez mais comum à medida que dispositivos IoT, como sensores, dispositivos inteligentes e muito mais, aumentam na natureza.

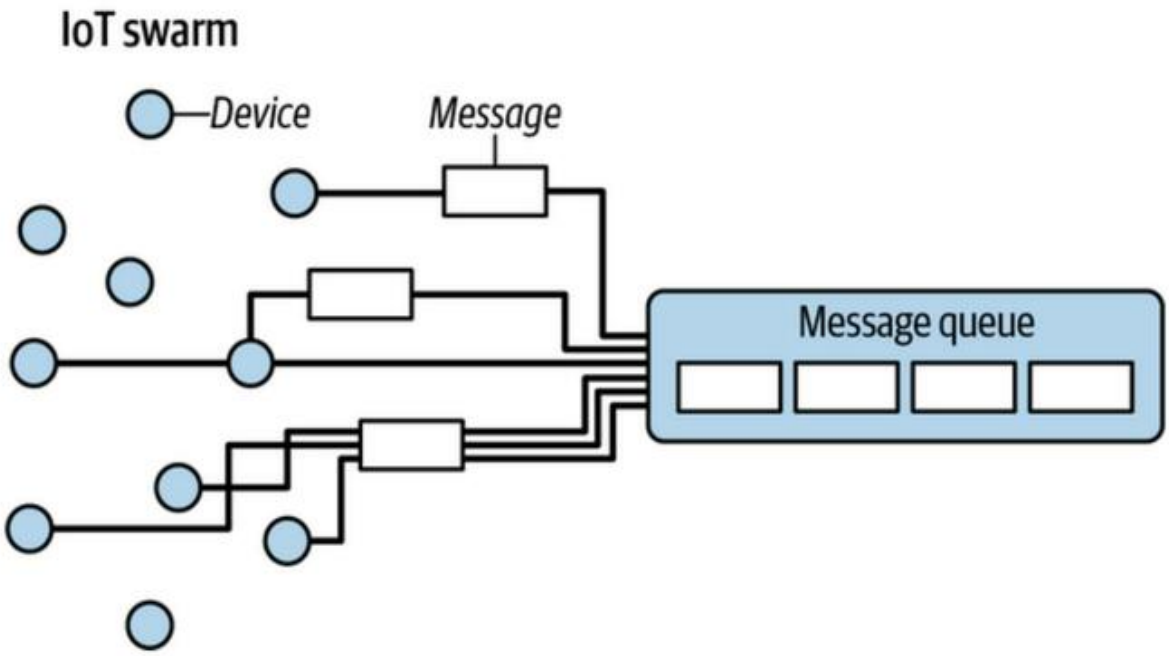


Figura 2-4. Exemplo de sistema de origem: um enxame de IoT e uma fila de mensagens

Avaliando sistemas de origem: principais considerações de engenharia

Há muitas coisas a serem consideradas ao avaliar os sistemas de origem, incluindo como o sistema lida com a ingestão, o estado e a geração de dados. A seguir, um conjunto inicial de perguntas de avaliação de sistemas de origem que os engenheiros de dados devem considerar:

- Quais são as características essenciais da fonte de dados? É um aplicativo? Um enxame de dispositivos IoT?
- Como os dados são persistidos no sistema de origem? Os dados persistem a longo prazo ou são temporários e rapidamente excluídos?
- A que taxa os dados são gerados? Quantos eventos por segundo? Quantos gigabytes por hora?

Machine Translated by Google

Que nível de consistência os engenheiros de dados podem esperar dos dados de saída? Se você estiver executando verificações de qualidade de dados em relação aos dados de saída, com que frequência ocorrem inconsistências de dados — nulos onde não são esperados, formatação ruim etc.?

Com que frequência ocorrem os erros?

Os dados conterão duplicatas?

Alguns valores de dados chegarão atrasados, possivelmente muito mais tarde do que outras mensagens produzidas simultaneamente?

Qual é o esquema dos dados ingeridos? Os engenheiros de dados precisarão unir várias tabelas ou mesmo vários sistemas para obter uma visão completa dos dados?

Se o esquema muda (digamos, uma nova coluna é adicionada), como isso é tratado e comunicado aos interessados a jusante?

Com que frequência os dados devem ser extraídos do sistema de origem?

Para sistemas com estado (por exemplo, um banco de dados que rastreia as informações da conta do cliente), os dados são fornecidos como instantâneos periódicos ou eventos de atualização do change data capture (CDC)? Qual é a lógica de como as alterações são executadas e como elas são rastreadas no banco de dados de origem?

Quem/o que é o provedor de dados que transmitirá os dados para consumo downstream?

A leitura de uma fonte de dados afetará seu desempenho?

O sistema de origem tem dependências de dados upstream? Quais são as características desses sistemas upstream?

Existem verificações de qualidade de dados para verificar se há dados atrasados ou ausentes?

As fontes produzem dados consumidos por sistemas downstream, incluindo planilhas geradas por humanos, sensores de IoT e aplicativos da Web e móveis. Cada fonte tem seu volume e cadência únicos de geração de dados. Um engenheiro de dados deve saber como a fonte gera dados, incluindo dados relevantes

Machine Translated by Google

peculiaridades ou nuances. Os engenheiros de dados também precisam entender os limites dos sistemas de origem com os quais interagem. Por exemplo, as consultas analíticas em um banco de dados do aplicativo de origem causarão contenção de recursos e problemas de desempenho?

Uma das nuances mais desafiadoras dos dados de origem é o esquema. O ***esquema*** define a organização hierárquica dos dados. Logicamente, podemos pensar em dados no nível de todo um sistema de origem, detalhando tabelas individuais, até a estrutura dos respectivos campos. O esquema de dados enviados de sistemas de origem é tratado de várias maneiras.

Duas opções populares são esquema fixo e sem esquema.

Schemaless não significa ausência de esquema. Em vez disso, significa que o aplicativo define o esquema à medida que os dados são gravados, seja em uma fila de mensagens, um arquivo simples, um blob ou um banco de dados de documentos como o MongoDB. Um modelo mais tradicional construído no armazenamento de banco de dados relacional usa um ***esquema fixo*** aplicado no banco de dados, ao qual as gravações do aplicativo devem estar em conformidade.

Qualquer um desses modelos apresenta desafios para engenheiros de dados. Os esquemas mudam ao longo do tempo; na verdade, a evolução do esquema é incentivada na abordagem Ágil para o desenvolvimento de software. Uma parte fundamental do trabalho do engenheiro de dados é receber a entrada de dados brutos no esquema do sistema de origem e transformá-la em uma saída valiosa para análise. Esse trabalho se torna mais desafiador à medida que o esquema de origem evolui.

Mergulhamos nos sistemas de origem com mais detalhes no Capítulo 5; também abordamos esquemas e modelagem de dados nos Capítulos 6 e 8, respectivamente.

Armazenar

Após a ingestão de dados, você precisa de um local para armazená-los. A escolha de uma solução de armazenamento é fundamental para o sucesso no restante do ciclo de vida dos dados e também é um dos estágios mais complicados do ciclo de vida dos dados por vários motivos.
Primeiro, as arquiteturas de dados na nuvem geralmente aproveitam ***várias*** soluções de armazenamento. Em segundo lugar, poucas soluções de armazenamento de dados funcionam apenas como armazenamento, com muitas suportando consultas de transformação complexas; até mesmo as soluções de armazenamento de objetos podem oferecer suporte a recursos avançados de consulta, por exemplo, Amazon S3

Machine Translated by Google

Selecione. Terceiro, embora o armazenamento seja um estágio do ciclo de vida da engenharia de dados, ele frequentemente toca em outros estágios, como ingestão, transformação e veiculação.

O armazenamento é executado em todo o ciclo de vida da engenharia de dados, geralmente ocorrendo em vários locais em um pipeline de dados, com sistemas de armazenamento cruzando com sistemas de origem, ingestão, transformação e serviço. De muitas maneiras, a forma como os dados são armazenados impacta como eles são usados em todos os estágios do ciclo de vida da engenharia de dados. Por exemplo, os data warehouses na nuvem podem armazenar dados, processar dados em pipelines e vendê-los para analistas. Estruturas de streaming como Apache Kafka e Pulsar podem funcionar simultaneamente como sistemas de ingestão, armazenamento e consulta de mensagens, sendo o armazenamento de objetos uma camada padrão para transmissão de dados.

Avaliando sistemas de armazenamento: principais considerações de engenharia

Aqui estão algumas perguntas importantes de engenharia a serem feitas ao escolher um sistema de armazenamento para um data warehouse, data lakehouse, banco de dados ou armazenamento de objetos:

- Esta solução de armazenamento é compatível com as velocidades de gravação e leitura exigidas pela arquitetura?
- O armazenamento criará um gargalo para os processos downstream?
- Você entende como essa tecnologia de armazenamento funciona? Você está utilizando o sistema de armazenamento de forma otimizada ou cometendo atos não naturais?
  Por exemplo, você está aplicando uma alta taxa de atualizações de acesso aleatório em um sistema de armazenamento de objetos? (Este é um antipadrão com sobrecarga de desempenho significativa.)
- Este sistema de armazenamento lidará com a escala futura prevista? Você deve considerar todos os limites de capacidade do sistema de armazenamento: armazenamento total disponível, taxa de operação de leitura, volume de gravação etc.
- Os usuários e processos downstream poderão recuperar dados no acordo de nível de serviço (SLA) exigido?

Machine Translated by Google

Você está capturando metadados sobre evolução de esquema, fluxos de dados, linhagem de dados e assim por diante? Os metadados têm um impacto significativo na utilidade dos dados. Os metadados representam um investimento no futuro, aumentando drasticamente a capacidade de descoberta e o conhecimento institucional para otimizar projetos futuros e mudanças de arquitetura.

Esta é uma solução de armazenamento puro (armazenamento de objetos) ou suporta padrões de consulta complexos (ou seja, um data warehouse em nuvem)?

O sistema de armazenamento é agnóstico de esquema (armazenamento de objetos)? Esquema flexível (Cassandra)? Esquema imposto (um data warehouse na nuvem)?

Como você está rastreando dados mestre, qualidade de dados de registros dourados e linhagem de dados para governança de dados? (Temos mais a dizer sobre isso em "Gerenciamento de Dados".)

Como você está lidando com a conformidade regulatória e a soberania de dados? Por exemplo, você pode armazenar seus dados em determinadas localizações geográficas, mas não em outras?

Entendendo a frequência de acesso a dados

Nem todos os dados são acessados da mesma maneira. Os padrões de recuperação variam muito com base nos dados armazenados e consultados. Isso traz à tona a noção das "temperaturas" dos dados. A frequência de acesso aos dados determinará a temperatura dos seus dados.

Os dados acessados com mais frequência são chamados ***de dados quentes.*** Os dados quentes geralmente são recuperados muitas vezes por dia, talvez até várias vezes por segundo, em sistemas que atendem a solicitações de usuários. Esses dados devem ser armazenados para recuperação rápida, onde "rápido" é relativo ao caso de uso. ***Dados mornos*** podem ser acessados de vez em quando – digamos, toda semana ou mês.

Os dados frios raramente são consultados e são apropriados para armazenamento em um sistema de arquivamento. Os dados frios geralmente são retidos para fins de conformidade ou em caso de falha catastrófica em outro sistema. Nos "antigos tempos", os dados frios eram armazenados em fitas e enviados para instalações remotas de arquivamento. Na nuvem

Machine Translated by Google

ambientes, os fornecedores oferecem camadas de armazenamento especializadas com custos mensais de armazenamento muito baratos, mas preços altos para recuperação de dados.

Selecionando um sistema de armazenamento

Que tipo de solução de armazenamento você deve usar? Isso depende de seus casos de uso, volumes de dados, frequência de ingestão, formato e tamanho dos dados que estão sendo ingeridos — basicamente, as principais considerações listadas nas perguntas com marcadores anteriores. Não há uma recomendação de armazenamento universal de tamanho único. Toda tecnologia de armazenamento tem suas desvantagens. Existem inúmeras variedades de tecnologias de armazenamento e é fácil ficar sobrecarregado ao decidir a melhor opção para sua arquitetura de dados.

O Capítulo 6 aborda as melhores práticas e abordagens de armazenamento com mais detalhes, bem como o cruzamento entre armazenamento e outros estágios do ciclo de vida.

Ingestão

Depois de entender a fonte de dados e as características do sistema de origem que está usando, você precisa coletar os dados. A segunda etapa do ciclo de vida da engenharia de dados é a ingestão de dados dos sistemas de origem.

Em nossa experiência, os sistemas de origem e a ingestão representam os gargalos mais significativos do ciclo de vida da engenharia de dados. Os sistemas de origem normalmente estão fora de seu controle direto e podem deixar de responder aleatoriamente ou fornecer dados de baixa qualidade. Ou seu serviço de ingestão de dados pode parar de funcionar misteriosamente por vários motivos. Como resultado, o fluxo de dados para ou fornece dados insuficientes para armazenamento, processamento e veiculação.

Sistemas de origem e ingestão não confiáveis têm um efeito cascata em todo o ciclo de vida da engenharia de dados. Mas você está em boa forma, supondo que tenha respondido às grandes questões sobre sistemas de origem.

Principais considerações de engenharia para a fase de ingestão

Ao se preparar para arquitetar ou construir um sistema, aqui estão algumas perguntas principais sobre o estágio de ingestão:

Machine Translated by Google

Quais são os casos de uso para os dados que estou ingerindo? Posso reutilizar esses dados em vez de criar várias versões do mesmo conjunto de dados?

Os sistemas estão gerando e ingerindo esses dados de forma confiável e os dados estão disponíveis quando eu preciso deles?

Qual é o destino dos dados após a ingestão?

Com que frequência precisarei acessar os dados?

Em que volume os dados normalmente chegarão?

Em que formato estão os dados? Meus sistemas de armazenamento e transformação downstream podem lidar com esse formato?

Os dados de origem estão em boas condições para uso imediato a jusante? Em caso afirmativo, por quanto tempo e o que pode fazer com que seja inutilizável?

Se os dados forem de uma fonte de streaming, eles precisam ser transformados antes de chegar ao destino? Uma transformação em andamento seria apropriada, onde os dados são transformados dentro do próprio fluxo?

Esses são apenas uma amostra dos fatores que você precisará considerar com a ingestão, e abordaremos essas questões e muito mais no Capítulo 7. Antes de sairmos, vamos voltar brevemente nossa atenção para dois conceitos principais de ingestão de dados: lote versus streaming e push versus puxar.

Lote versus streaming

Praticamente todos os dados com os quais lidamos são inerentemente ***streaming.*** Os dados são quase sempre produzidos e atualizados continuamente em sua fonte. ***A ingestão em lote*** é simplesmente uma maneira especializada e conveniente de processar esse fluxo em grandes partes — por exemplo, manipular dados de um dia inteiro em um único lote.

A ingestão de streaming nos permite fornecer dados para sistemas downstream — sejam outros aplicativos, bancos de dados ou sistemas de análise — de maneira contínua e em tempo real. Aqui, ***em tempo real*** (ou ***quase em tempo real)*** significa que os dados estão disponíveis para um sistema downstream pouco tempo depois de serem

Machine Translated by Google

produzido (por exemplo, menos de um segundo depois). A latência necessária para se qualificar como em tempo real varia de acordo com o domínio e os requisitos.

Os dados de lote são ingeridos em um intervalo de tempo predeterminado ou quando os dados atingem um limite de tamanho predefinido. A ingestão de lotes é uma porta de mão única: uma vez que os dados são divididos em lotes, a latência para os consumidores downstream é inerentemente restrita. Por causa das limitações dos sistemas legados, o batch foi por muito tempo a forma padrão de ingerir dados. O processamento em lote continua sendo uma maneira extremamente popular de ingerir dados para consumo downstream, principalmente em análises e ML.

No entanto, a separação de armazenamento e computação em muitos sistemas e a onipresença das plataformas de streaming e processamento de eventos tornam o processamento contínuo de fluxos de dados muito mais acessível e cada vez mais popular.
A escolha depende em grande parte do caso de uso e das expectativas de pontualidade dos dados.

Principais considerações para ingestão de lote versus fluxo

Você deve ir primeiro ao streaming? Apesar da atratividade de uma primeira abordagem de streaming, há muitas compensações para entender e pensar. Veja a seguir algumas perguntas a serem feitas ao determinar se a ingestão de streaming é uma escolha apropriada em relação à ingestão em lote:

> Se eu ingerir os dados em tempo real, os sistemas de armazenamento downstream podem lidar com a taxa de fluxo de dados?
>
> Preciso de ingestão de dados em tempo real de milissegundos? Ou uma abordagem de micro lote funcionaria, acumulando e ingerindo dados, digamos, a cada minuto?
>
> Quais são meus casos de uso para ingestão de streaming? Quais benefícios específicos eu percebo ao implementar o streaming? Se eu obtiver dados em tempo real, que ações posso tomar nesses dados que seriam uma melhoria no lote?
>
> Minha abordagem de streaming em primeiro lugar custará mais em termos de tempo, dinheiro, manutenção, tempo de inatividade e custo de oportunidade do que simplesmente fazer?

Machine Translated by Google

lote?

Meu pipeline e sistema de streaming são confiáveis e redundantes se a infraestrutura falhar?

Quais ferramentas são mais apropriadas para o caso de uso? Devo usar um serviço gerenciado (Amazon Kinesis, Google Cloud Pub/Sub, Google Cloud Dataflow) ou criar minhas próprias instâncias de Kafka, Flink, Spark, Pulsar etc.? Se eu fizer o último, quem irá gerenciá-lo? Quais são os custos e trade-offs?

Se estou implantando um modelo de ML, quais benefícios tenho com previsões online e possivelmente treinamento contínuo?

Estou recebendo dados de uma instância de produção ativa? Em caso afirmativo, qual é o impacto do meu processo de ingestão neste sistema de origem?

Como você pode ver, transmitir primeiro pode parecer uma boa ideia, mas nem sempre é simples; custos e complexidades extras ocorrem inerentemente.
Muitas estruturas de ingestão excelentes lidam com estilos de ingestão em lote e microlote. Achamos que o lote é uma abordagem excelente para muitos casos de uso comuns, como treinamento de modelo e relatórios semanais. Adote o streaming em tempo real somente após identificar um caso de uso de negócios que justifique as compensações em relação ao uso de lote.

Empurrar versus puxar

No modelo ***push*** de ingestão de dados, um sistema de origem grava os dados em um destino, seja um banco de dados, armazenamento de objetos ou sistema de arquivos. No modelo ***pull ,*** os dados são recuperados do sistema de origem. A linha entre os paradigmas de empurrar e puxar pode ser bastante tênue; os dados geralmente são empurrados e puxados à medida que percorrem os vários estágios de um pipeline de dados.

Considere, por exemplo, o processo de extração, transformação, carregamento (ETL), comumente usado em fluxos de trabalho de ingestão orientados a lote. A parte de ***extração (E)*** do ETL esclarece que estamos lidando com um modelo de ingestão de pull. No ETL tradicional, o sistema de ingestão consulta um instantâneo da tabela de origem atual em um

Machine Translated by Google

cronograma. Você aprenderá mais sobre ETL e extrair, carregar, transformar (ELT) ao longo deste livro.

Em outro exemplo, considere o CDC contínuo, que é alcançado de algumas maneiras. Um método comum aciona uma mensagem sempre que uma linha é alterada no banco de dados de origem. Essa mensagem é ***enviada*** para uma fila, onde o sistema de ingestão a coleta. Outro método CDC comum usa logs binários, que registram cada confirmação no banco de dados. O banco de dados ***envia*** para seus logs. O sistema de ingestão lê os logs, mas não interage diretamente com o banco de dados. Isso adiciona pouca ou nenhuma carga adicional ao banco de dados de origem. Algumas versões do CDC em lote usam o padrão ***pull .*** Por exemplo, no CDC baseado em carimbo de data/hora, um sistema de ingestão consulta o banco de dados de origem e extrai as linhas que foram alteradas desde a atualização anterior.

Com a ingestão de streaming, os dados ignoram um banco de dados de back-end e são enviados diretamente para um endpoint, normalmente com dados armazenados em buffer por uma plataforma de streaming de eventos. Esse padrão é útil com frotas de sensores de IoT que emitem dados de sensores. Em vez de depender de um banco de dados para manter o estado atual, simplesmente pensamos em cada leitura registrada como um evento. Esse padrão também está crescendo em popularidade em aplicativos de software, pois simplifica o processamento em tempo real, permite que os desenvolvedores de aplicativos personalizem suas mensagens para análises downstream e simplifica muito a vida dos engenheiros de dados.

Discutimos as melhores práticas e técnicas de ingestão em profundidade no Capítulo 7.
Em seguida, vamos nos voltar para o estágio de transformação do ciclo de vida da engenharia de dados.

Transformação

Depois de ingerir e armazenar dados, você precisa fazer algo com eles.
O próximo estágio do ciclo de vida da engenharia de dados é a ***transformação,*** o que significa que os dados precisam ser alterados de sua forma original para algo útil para casos de uso downstream. Sem as transformações adequadas, os dados ficarão inertes e não estarão em uma forma útil para relatórios, análises ou ML. Normalmente, o estágio de transformação é onde os dados começam a criar valor para o consumo do usuário downstream.

Machine Translated by Google

Imediatamente após a ingestão, as transformações básicas mapeiam os dados em tipos corretos (alterando os dados de string ingeridos em tipos numéricos e de data, por exemplo), colocando registros em formatos padrão e removendo os inválidos.
Os estágios posteriores de transformação podem transformar o esquema de dados e aplicar a normalização. A jusante, podemos aplicar agregação em larga escala para relatórios ou apresentar dados para processos de ML.

Principais considerações para a fase de transformação

Ao considerar as transformações de dados dentro do ciclo de vida da engenharia de dados, é útil considerar o seguinte:

Qual é o custo e o retorno do investimento (ROI) da transformação?
Qual é o valor de negócio associado?

A transformação é o mais simples e auto-isolada possível?

Quais regras de negócios as transformações suportam?

Estou minimizando a movimentação de dados entre a transformação e o sistema de armazenamento durante a transformação?

Você pode transformar dados em lote ou durante o streaming em voo. Conforme mencionado em “Ingestão”, praticamente todos os dados iniciam a vida como um fluxo contínuo; batch é apenas uma forma especializada de processar um fluxo de dados. As transformações em lote são extremamente populares, mas dada a crescente popularidade das soluções de processamento de fluxo e o aumento geral na quantidade de dados de streaming, esperamos que a popularidade das transformações de streaming continue crescendo, talvez substituindo inteiramente o processamento em lote em determinados domínios
em breve.

Logicamente, tratamos a transformação como uma área autônoma do ciclo de vida da engenharia de dados, mas as realidades do ciclo de vida podem ser muito mais complicadas na prática. A transformação geralmente está emaranhada em outras fases do ciclo de vida. Normalmente, os dados são transformados em sistemas de origem ou em voo durante a ingestão. Por exemplo, um sistema de origem pode adicionar um carimbo de data/hora de evento a um registro antes de encaminhá-lo para um processo de ingestão. Ou um registro em um pipeline de streaming pode ser “enriquecido” com campos adicionais

Machine Translated by Google

e cálculos antes de serem enviados para um data warehouse. As transformações são onipresentes em várias partes do ciclo de vida. Preparação de dados, organização de dados e limpeza — essas tarefas transformadoras agregam valor para os consumidores finais de dados.

A lógica de negócios é um dos principais impulsionadores da transformação de dados, geralmente na modelagem de dados. Os dados traduzem a lógica de negócios em elementos reutilizáveis (por exemplo, uma venda significa “alguém comprou 12 porta-retratos de mim por US$ 30 cada, ou US$ 360 no total”). Nesse caso, alguém comprou 12 porta-retratos por US$ 30 cada. A modelagem de dados é fundamental para obter uma imagem clara e atual dos processos de negócios. Uma visão simples das transações brutas de varejo pode não ser útil sem adicionar a lógica das regras contábeis para que o CFO tenha uma visão clara da saúde financeira. Garanta uma abordagem padrão para implementar a lógica de negócios em suas transformações.

A caracterização de dados para ML é outro processo de transformação de dados. A featurização pretende extrair e aprimorar recursos de dados úteis para o treinamento de modelos de ML. A caracterização pode ser uma arte obscura, combinando conhecimento de domínio (para identificar quais recursos podem ser importantes para previsão) com ampla experiência em ciência de dados. Para este livro, o ponto principal é que, uma vez que os cientistas de dados determinem como caracterizar os dados, os processos de caracterização podem ser automatizados por engenheiros de dados no estágio de transformação de um pipeline de dados.

A transformação é um assunto profundo, e não podemos fazer justiça a isso nesta breve introdução. O Capítulo 8 aprofunda consultas, modelagem de dados e várias práticas e nuances de transformação.

Dados de veiculação

Você atingiu o último estágio do ciclo de vida da engenharia de dados. Agora que os dados foram ingeridos, armazenados e transformados em estruturas coerentes e úteis, é hora de obter valor de seus dados. “Obter valor” dos dados significa coisas diferentes para usuários diferentes.

Os dados têm ***valor*** quando são usados para fins práticos. Os dados que não são consumidos ou consultados são simplesmente inertes. Os projetos de vaidade de dados são um grande risco para as empresas. Muitas empresas buscaram projetos de vaidade na era do big data,

Machine Translated by Google

reunindo grandes conjuntos de dados em data lakes que nunca foram consumidos de forma útil. A era da nuvem está desencadeando uma nova onda de projetos de vaidade construídos nos mais recentes data warehouses, sistemas de armazenamento de objetos e tecnologias de streaming. Os projetos de dados devem ser intencionais em todo o ciclo de vida. Qual é o objetivo comercial final dos dados coletados, limpos e armazenados com tanto cuidado?

O serviço de dados talvez seja a parte mais empolgante do ciclo de vida da engenharia de dados. É aqui que a mágica acontece. É aqui que os engenheiros de ML podem aplicar as técnicas mais avançadas. Vejamos alguns dos usos populares de dados: análise, ML e ETL reverso.

Análise

A análise é o núcleo da maioria dos esforços de dados. Depois que seus dados forem armazenados e transformados, você estará pronto para gerar relatórios ou painéis e fazer análises ad hoc dos dados. Enquanto a maior parte das análises costumava abranger o BI, agora inclui outras facetas, como análises operacionais e análises voltadas para o cliente (Figura 2-5). Vamos abordar brevemente essas variações de análise.

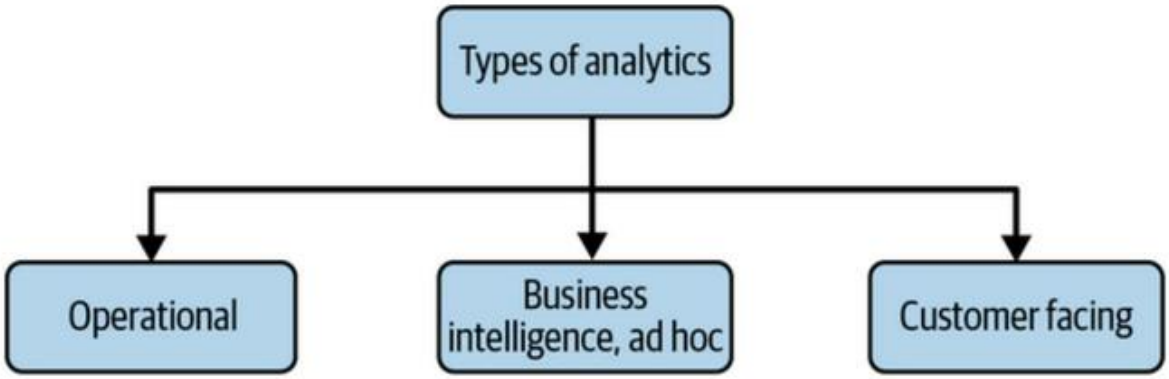


Figura 2-5. Tipos de análise

Inteligência de negócios

Os marechais de BI coletaram dados para descrever o estado passado e atual de uma empresa. O BI requer o uso de lógica de negócios para processar dados brutos. Observe que o fornecimento de dados para análise é outra área em que os estágios do ciclo de vida da engenharia de dados podem se confundir. Como mencionamos anteriormente, a lógica de negócios geralmente é aplicada aos dados no estágio de transformação do ciclo de vida da engenharia de dados, mas uma abordagem lógica na leitura tornou-se cada vez mais popular. Os dados são

Machine Translated by Google

armazenado em uma forma limpa, mas bastante bruta, com lógica de negócios de pós-processamento mínima. Um sistema de BI mantém um repositório de definições e lógica de negócios. Essa lógica de negócios é usada para consultar o data warehouse para que os relatórios e painéis se alinhem às definições de negócios.

À medida que uma empresa aumenta sua maturidade de dados, ela passará da análise de dados ad hoc para a análise de autoatendimento, permitindo acesso democratizado aos dados para usuários de negócios sem a necessidade de intervenção da TI. A capacidade de fazer análises de autoatendimento pressupõe que os dados são bons o suficiente para que as pessoas em toda a organização possam simplesmente acessá-los, cortá-los como quiserem e obter insights imediatos. Embora a análise de autoatendimento seja simples na teoria, é difícil de realizar na prática. A principal razão é que a baixa qualidade dos dados, os silos organizacionais e a falta de habilidades de dados adequadas impedem o uso generalizado de análises.

Análise operacional

A análise operacional se concentra nos detalhes refinados das operações, promovendo ações nas quais um usuário dos relatórios pode agir imediatamente.
A análise operacional pode ser uma visualização ao vivo do inventário ou um painel em tempo real da integridade do site. Nesse caso, os dados são consumidos em tempo real, diretamente de um sistema de origem ou de um pipeline de dados de streaming. Os tipos de insights na análise operacional diferem do BI tradicional, pois a análise operacional é focada no presente e não necessariamente diz respeito às tendências históricas.

Análise incorporada

Você pode se perguntar por que dividimos a análise incorporada (análise voltada para o cliente) separadamente do BI. Na prática, as análises fornecidas aos clientes em uma plataforma SaaS vêm com um conjunto separado de requisitos e complicações. O BI interno enfrenta um público limitado e geralmente apresenta um número limitado de visualizações unificadas. Os controles de acesso são críticos, mas não particularmente complicados. O acesso é gerenciado usando várias funções e camadas de acesso.

Machine Translated by Google

Com a análise voltada para o cliente, a taxa de solicitação de relatórios e a carga correspondente nos sistemas de análise aumentam drasticamente; o controle de acesso é significativamente mais complicado e crítico. As empresas podem estar servindo análises e dados separados para milhares ou mais clientes. Cada cliente deve ver seus dados e apenas seus dados. Um erro interno de acesso a dados em uma empresa provavelmente levaria a uma revisão processual. Um vazamento de dados entre clientes seria considerado uma grande quebra de confiança, levando à atenção da mídia e a uma perda significativa de clientes. Minimize seu raio de explosão relacionado a vazamentos de dados e vulnerabilidades de segurança. Aplique segurança em nível de locatário ou de dados em seu armazenamento e em qualquer lugar onde haja a possibilidade de vazamento de dados.

MÚLTIPLOS INQUILINOS

Muitos sistemas atuais de armazenamento e análise oferecem suporte a multilocação de várias maneiras. Os engenheiros de dados podem optar por hospedar dados de muitos clientes em tabelas comuns para permitir uma visualização unificada para análises internas e ML. Esses dados são apresentados externamente a clientes individuais por meio de visualizações lógicas com controles e filtros adequadamente definidos. Cabe aos engenheiros de dados entender as minúcias da multilocação nos sistemas que eles implantam para garantir a segurança e o isolamento absolutos dos dados.

Aprendizado de máquina

O surgimento e o sucesso do ML é uma das revoluções tecnológicas mais empolgantes. Quando as organizações atingem um alto nível de maturidade de dados, elas podem começar a identificar problemas passíveis de ML e começar a organizar uma prática em torno disso.

As responsabilidades dos engenheiros de dados se sobrepõem significativamente em análise e ML, e os limites entre engenharia de dados, engenharia de ML e engenharia de análise podem ser confusos. Por exemplo, um engenheiro de dados pode precisar dar suporte a clusters Spark que facilitam pipelines de análise e treinamento de modelo de ML. Eles também podem precisar fornecer um sistema que orquestre tarefas

Machine Translated by Google

entre equipes e suporte a metadados e sistemas de catalogação que rastreiam o histórico e a linhagem dos dados. Definir esses domínios de responsabilidade e as estruturas de relatórios relevantes é uma decisão organizacional crítica.

A feature store é uma ferramenta desenvolvida recentemente que combina engenharia de dados e engenharia de ML. Os repositórios de recursos são projetados para reduzir a carga operacional dos engenheiros de ML, mantendo o histórico e as versões de recursos, oferecendo suporte ao compartilhamento de recursos entre equipes e fornecendo recursos operacionais e de orquestração básicos, como preenchimento. Na prática, os engenheiros de dados fazem parte da equipe principal de suporte para repositórios de recursos para dar suporte à engenharia de ML.

Um engenheiro de dados deve estar familiarizado com ML? Certamente ajuda. Independentemente do limite operacional entre engenharia de dados, engenharia de ML, análise de negócios e assim por diante, os engenheiros de dados devem manter o conhecimento operacional sobre suas equipes. Um bom engenheiro de dados conhece as técnicas fundamentais de ML e os requisitos de processamento de dados relacionados, os casos de uso de modelos em sua empresa e as responsabilidades das várias equipes de análise da organização. Isso ajuda a manter uma comunicação eficiente e facilita a colaboração. Idealmente, os engenheiros de dados criarão ferramentas em parceria com outras equipes que nenhuma das equipes pode criar de forma independente.

Este livro não pode cobrir ML em profundidade. Um ecossistema crescente de livros, vídeos, artigos e comunidades está disponível se você estiver interessado em aprender mais; incluímos algumas sugestões em “Recursos Adicionais”.

A seguir estão algumas considerações para a fase de veiculação de dados específica para ML:

- Os dados são de qualidade suficiente para realizar engenharia de recursos confiável? Os requisitos e avaliações de qualidade são desenvolvidos em estreita colaboração com as equipes que consomem os dados.

- Os dados são detectáveis? Os cientistas de dados e engenheiros de ML podem encontrar facilmente dados valiosos?

Machine Translated by Google

Onde estão os limites técnicos e organizacionais entre engenharia de dados e engenharia de ML? Essa questão organizacional tem implicações arquitetônicas significativas.

O conjunto de dados representa adequadamente a verdade do terreno? É injustamente tendencioso?

Embora o ML seja empolgante, nossa experiência é que as empresas muitas vezes mergulham nele prematuramente. Antes de investir muitos recursos em ML, reserve um tempo para construir uma base de dados sólida. Isso significa configurar os melhores sistemas e arquitetura em todo o ciclo de vida de engenharia de dados e ML. Geralmente, é melhor desenvolver competência em análise antes de mudar para ML. Muitas empresas frustraram seus sonhos de ML porque empreenderam iniciativas sem fundamentos adequados.

ETL reverso

O ETL reverso tem sido uma realidade prática em dados, visto como um antipadrão que não gostamos de falar ou dignificar com um nome. ***O ETL reverso*** pega os dados processados do lado de saída do ciclo de vida da engenharia de dados e os realimenta nos sistemas de origem, conforme mostrado na Figura 2-6. Na realidade, esse fluxo é benéfico e muitas vezes necessário; O ETL reverso nos permite obter análises, modelos pontuados, etc., e alimentá-los de volta em sistemas de produção ou plataformas SaaS.

Machine Translated by Google

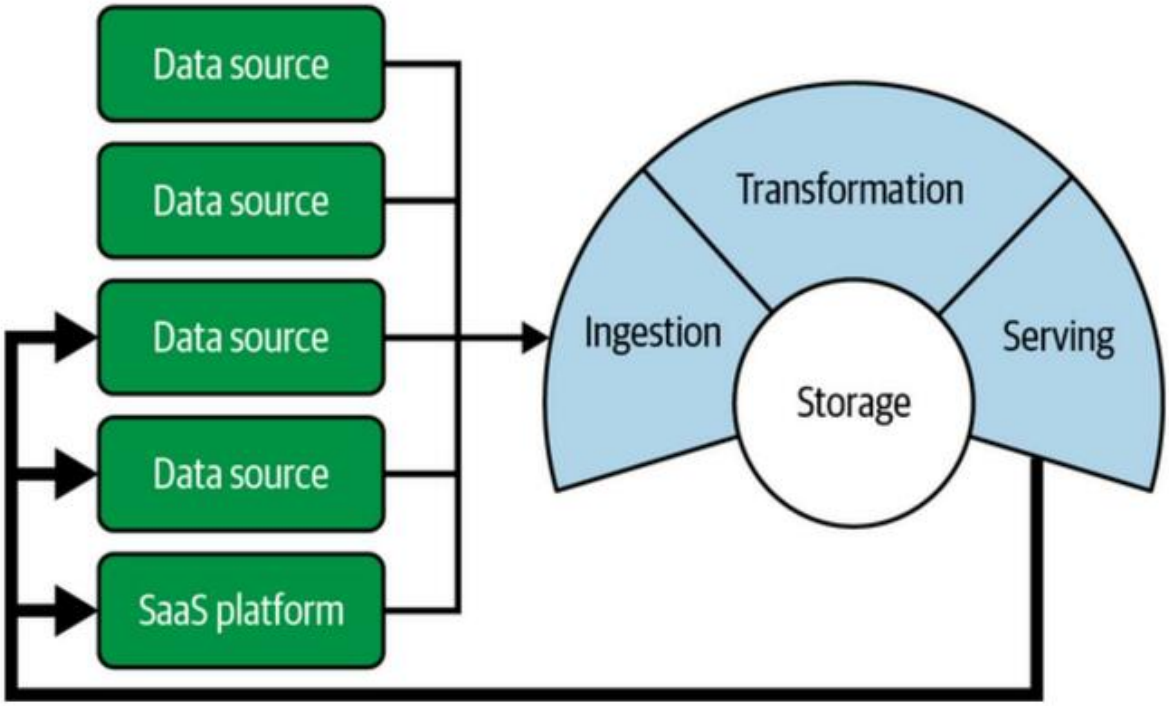


Figura 2-6. ETL reverso

Os analistas de marketing podem calcular lances no Microsoft Excel usando os dados em seu data warehouse e, em seguida, fazer upload desses lances para o Google Ads. Esse processo era muitas vezes inteiramente manual e primitivo.

Enquanto escrevíamos este livro, vários fornecedores adotaram o conceito de ETL reverso e criaram produtos em torno dele, como Hightouch e Census. O ETL reverso permanece incipiente como um campo, mas suspeitamos que ele veio para ficar.

O ETL reverso tornou-se especialmente importante à medida que as empresas dependem cada vez mais de SaaS e plataformas externas. Por exemplo, as empresas podem querer enviar métricas específicas de seu data warehouse para uma plataforma de dados do cliente ou sistema de CRM. As plataformas de publicidade são outro caso de uso diário, como no exemplo do Google Ads. Espere ver mais atividade no ETL reverso, com uma sobreposição na engenharia de dados e na engenharia de ML.

O júri está decidindo se o termo ***ETL reverso*** se manterá. E a prática pode evoluir. Alguns engenheiros afirmam que podemos eliminar o ETL reverso manipulando transformações de dados em um fluxo de eventos e enviando esses eventos de volta aos sistemas de origem conforme necessário. Percebendo a adoção generalizada deste

Machine Translated by Google

padrão entre as empresas é outra questão. A essência é que os dados transformados precisarão ser devolvidos aos sistemas de origem de alguma maneira, idealmente com a linhagem e o processo de negócios corretos associados ao sistema de origem.

Principais subcorrentes nos dados Ciclo de vida de engenharia

A engenharia de dados está amadurecendo rapidamente. Enquanto os ciclos anteriores de engenharia de dados simplesmente se concentravam na camada de tecnologia, a abstração e simplificação contínuas de ferramentas e práticas mudaram esse foco. A engenharia de dados agora abrange muito mais do que ferramentas e tecnologia. O campo agora está subindo na cadeia de valor, incorporando práticas empresariais tradicionais, como gerenciamento de dados e otimização de custos, e práticas mais recentes, como DataOps.

Chamamos essas práticas ***de subcorrentes — segurança,*** gerenciamento de dados, DataOps, arquitetura de dados, orquestração e engenharia de software — que dão suporte a todos os aspectos do ciclo de vida da engenharia de dados (Figura 2-7). Nesta seção, apresentamos uma breve visão geral dessas subcorrentes e seus principais componentes, que você verá com mais detalhes ao longo do livro.

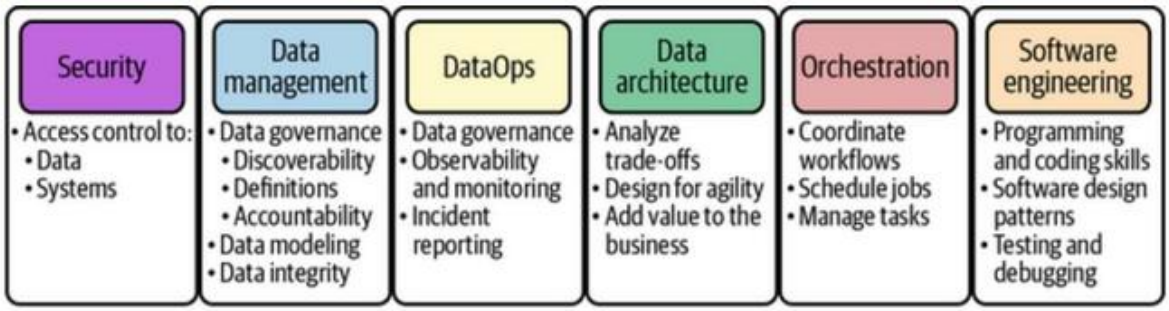


Figura 2-7. As principais tendências da engenharia de dados

Segurança

A segurança deve ser prioridade para os engenheiros de dados, e aqueles que a ignoram o fazem por sua conta e risco. É por isso que a segurança é a primeira tendência. Os engenheiros de dados devem entender tanto a segurança de dados quanto de acesso, exercendo o princípio de

Machine Translated by Google

Ultimo privilégio. O princípio do menor privilégio significa dar a um usuário ou sistema acesso apenas aos dados e recursos essenciais para executar uma função pretendida. Um antipadrão comum que vemos com engenheiros de dados com pouca experiência em segurança é dar acesso de administrador a todos os usuários. Esta é uma catástrofe esperando para acontecer!

Dê aos usuários apenas o acesso de que precisam para realizar seus trabalhos hoje, nada mais. Não opere a partir de um shell root quando estiver apenas procurando por arquivos visíveis com acesso de usuário padrão. Ao consultar tabelas com uma função menor, não use a função de superusuário em um banco de dados. Impor o princípio do menor privilégio a nós mesmos pode evitar danos acidentais e mantê-lo em uma mentalidade de segurança em primeiro lugar.

Pessoas e estrutura organizacional são sempre as maiores vulnerabilidades de segurança em qualquer empresa. Quando ouvimos falar de grandes violações de segurança na mídia, muitas vezes acontece que alguém na empresa ignorou as precauções básicas, foi vítima de um ataque de phishing ou agiu de forma irresponsável. A primeira linha de defesa para a segurança de dados é criar uma cultura de segurança que permeie a organização. Todos os indivíduos que têm acesso aos dados devem compreender sua responsabilidade na proteção dos dados confidenciais da empresa e de seus clientes.

A segurança dos dados também tem a ver com o tempo – fornecer acesso aos dados exatamente para as pessoas e sistemas que precisam acessá-los e ***apenas pela duração necessária para realizar seu trabalho.*** Os dados devem ser protegidos contra visibilidade indesejada, tanto em voo quanto em repouso, usando criptografia, tokenização, mascaramento de dados, ofuscação e controles de acesso simples e robustos.

Os engenheiros de dados devem ser administradores de segurança competentes, pois a segurança é de seu domínio. Um engenheiro de dados deve entender as práticas recomendadas de segurança para a nuvem e no local. O conhecimento de funções, políticas, grupos, segurança de rede, políticas de senha e criptografia de gerenciamento de acesso de identidade e usuário (IAM) são bons pontos de partida.

Ao longo do livro, destacamos as áreas em que a segurança deve ser prioridade no ciclo de vida da engenharia de dados. Você também pode obter informações mais detalhadas sobre segurança no Capítulo 10.

Machine Translated by Google

Gestão de dados

Você provavelmente acha que o gerenciamento de dados soa muito... corporativo. As práticas de gerenciamento de dados “old school” chegam aos dados e à engenharia de ML. O que é velho é novo de novo. O gerenciamento de dados existe há décadas, mas não obteve muita tração na engenharia de dados até recentemente.

As ferramentas de dados estão se tornando mais simples e há menos complexidade para os engenheiros de dados gerenciarem. Como resultado, o engenheiro de dados sobe na cadeia de valor em direção ao próximo degrau das melhores práticas. As melhores práticas de dados antes reservadas para grandes empresas – governança de dados, gerenciamento de dados mestre, gerenciamento de qualidade de dados, gerenciamento de metadados – agora estão sendo filtradas para empresas de todos os tamanhos e níveis de maturidade. Como gostamos de dizer, a engenharia de dados está se tornando “empresarial”. Isso é, em última análise, uma grande coisa!

Dados da Data Management Association International (DAMA)
Management Body of Knowledge (DMBOK), que consideramos ser o livro definitivo para gerenciamento de dados corporativos, oferece esta definição:

> ***O gerenciamento de dados é o desenvolvimento, execução e supervisão de planos, políticas, programas e práticas que fornecem, controlam, protegem e aumentam o valor dos ativos de dados e informações ao longo de seu ciclo de vida.***

Isso é um pouco longo, então vamos ver como isso se relaciona com a engenharia de dados. Os engenheiros de dados gerenciam o ciclo de vida dos dados e o gerenciamento de dados abrange o conjunto de práticas recomendadas que os engenheiros de dados usarão para realizar essa tarefa, tanto técnica quanto estrategicamente. Sem uma estrutura para gerenciar dados, os engenheiros de dados são simplesmente técnicos operando no vácuo. Os engenheiros de dados precisam de uma perspectiva mais ampla da utilidade dos dados em toda a organização, desde os sistemas de origem até o C-suite e em todos os lugares entre eles.

Por que o gerenciamento de dados é importante? O gerenciamento de dados demonstra que os dados são vitais para as operações diárias, assim como as empresas veem recursos financeiros, produtos acabados ou imóveis como ativos. As práticas de gerenciamento de dados formam uma estrutura coesa que todos podem adotar para garantir que a organização obtenha valor dos dados e os trate adequadamente.

Machine Translated by Google

O gerenciamento de dados tem algumas facetas, incluindo o seguinte:

- Governança de dados, incluindo descoberta e responsabilidade
- Modelagem e design de dados
- Linhagem de dados
- Armazenamento e operações
- Integração e interoperabilidade de dados
- Gerenciamento do ciclo de vida dos dados
- Sistemas de dados para análises avançadas e ML
- Ética e privacidade

Embora este livro não seja um recurso exaustivo sobre gerenciamento de dados, vamos abordar brevemente alguns pontos importantes de cada área no que se refere à engenharia de dados.

Governança de dados

De acordo com ***Data Governance: The Definitive Guide,*** "A governança de dados é, antes de tudo, uma função de gerenciamento de dados para garantir a qualidade, integridade, segurança e usabilidade dos dados coletados por uma organização".[1]

Podemos expandir essa definição e dizer que a governança de dados envolve pessoas, processos e tecnologias para maximizar o valor dos dados em uma organização enquanto protege os dados com controles de segurança apropriados.
A governança de dados eficaz é desenvolvida com intenção e apoiada pela organização. Quando a governança de dados é acidental e aleatória, os efeitos colaterais podem variar de dados não confiáveis a violações de segurança e tudo mais. Ser intencional sobre a governança de dados maximizará os recursos de dados da organização e o valor gerado a partir dos dados. Também (espero) manterá uma empresa fora das manchetes por práticas de dados questionáveis ou totalmente imprudentes.

Machine Translated by Google

Pense no exemplo típico de governança de dados mal feita. Um analista de negócios recebe uma solicitação de relatório, mas não sabe quais dados usar para responder à pergunta. Eles podem passar horas vasculhando dezenas de tabelas em um banco de dados transacional, tentando adivinhar quais campos podem ser úteis. O analista compila um relatório "direcionalmente correto", mas não tem certeza absoluta de que os dados subjacentes do relatório são precisos ou sólidos. O destinatário do relatório também questiona a validade dos dados. A integridade do analista — e de todos os dados nos sistemas da empresa — é questionada. A empresa está confusa quanto ao seu desempenho, impossibilitando o planejamento do negócio.

A governança de dados é uma base para práticas de negócios orientadas por dados e uma parte essencial do ciclo de vida da engenharia de dados. Quando a governança de dados é bem praticada, pessoas, processos e tecnologias se alinham para tratar os dados como um fator-chave de negócios; se ocorrerem problemas de dados, eles serão prontamente tratados.

As principais categorias de governança de dados são descoberta, segurança e responsabilidade. Dentro dessas categorias principais estão subcategorias, como qualidade de dados, metadados e privacidade. Vejamos cada categoria principal por sua vez.

Capacidade de

descoberta Em uma empresa orientada a dados, os dados devem estar disponíveis e detectáveis. Os usuários finais devem ter acesso rápido e confiável aos dados de que precisam para realizar seus trabalhos. Eles devem saber de onde vêm os dados, como eles se relacionam com outros dados e o que os dados significam.

Algumas áreas-chave de descoberta de dados incluem gerenciamento de metadados e gerenciamento de dados mestre. Vamos descrever brevemente essas áreas.

Metadados

Metadados são "dados sobre dados" e sustentam cada seção do ciclo de vida da engenharia de dados. Metadados são exatamente os dados necessários para tornar os dados detectáveis e governáveis.

Dividimos os metadados em duas categorias principais: gerados automaticamente e gerados por humanos. A engenharia de dados moderna gira em torno da automação, mas

Machine Translated by Google

a coleta de metadados geralmente é manual e propensa a erros.

A tecnologia pode ajudar nesse processo, removendo grande parte do trabalho propenso a erros da coleta manual de metadados. Estamos vendo uma proliferação de catálogos de dados, sistemas de rastreamento de linhagem de dados e ferramentas de gerenciamento de metadados. As ferramentas podem rastrear bancos de dados para procurar relacionamentos e monitorar pipelines de dados para rastrear de onde os dados vêm e para onde vão. Uma abordagem manual de baixa fidelidade usa um esforço conduzido internamente em que várias partes interessadas fazem crowdsourcing da coleta de metadados dentro da organização. Essas ferramentas de gerenciamento de dados são abordadas em profundidade ao longo do livro, pois prejudicam grande parte do ciclo de vida da engenharia de dados.

Os metadados tornam-se um subproduto dos dados e dos processos de dados. No entanto, os principais desafios permanecem. Em particular, ainda faltam interoperabilidade e padrões. As ferramentas de metadados são tão boas quanto seus conectores para sistemas de dados e sua capacidade de compartilhar metadados. Além disso, as ferramentas de metadados automatizadas não devem tirar totalmente os humanos do circuito.

Os dados têm um elemento social; cada organização acumula capital social e conhecimento em torno de processos, conjuntos de dados e pipelines. Os sistemas de metadados orientados para humanos concentram-se no aspecto social dos metadados. Isso é algo que o Airbnb enfatizou em suas várias postagens de blog sobre ferramentas de dados, particularmente seu conceito original de Dataportal. Essas ferramentas devem fornecer um local para divulgar proprietários de dados, consumidores de dados e especialistas de domínio. As ferramentas de documentação e wiki internas fornecem uma base fundamental para o gerenciamento de metadados, mas essas ferramentas também devem se integrar à catalogação automatizada de dados. Por exemplo, ferramentas de varredura de dados podem gerar páginas wiki com links para objetos de dados relevantes.

Uma vez que existam sistemas e processos de metadados, os engenheiros de dados podem consumir metadados de maneiras úteis. Os metadados se tornam a base para projetar pipelines e gerenciar dados ao longo do ciclo de vida.

O DMBOK identifica quatro categorias principais de metadados que são úteis para engenheiros de dados:

Metadados comerciais

Machine Translated by Google

Metadados técnicos

Metadados operacionais

Metadados de referência

Vamos descrever brevemente cada categoria de metadados.

Os metadados de negócios estão relacionados à maneira como os dados são usados nos negócios, incluindo definições de negócios e dados, regras e lógica de dados, como e onde os dados são usados e o(s) proprietário(s) dos dados.

Um engenheiro de dados usa metadados de negócios para responder a perguntas não técnicas sobre quem, o quê, onde e como. Por exemplo, um engenheiro de dados pode ser encarregado de criar um pipeline de dados para análise de vendas do cliente. Mas o que é um cliente? É alguém que comprou nos últimos 90 dias? Ou alguém que comprou a qualquer momento o negócio foi aberto? Um engenheiro de dados usaria os dados corretos para se referir aos metadados de negócios (dicionário de dados ou catálogo de dados) para pesquisar como um “cliente” é definido. Os metadados de negócios fornecem a um engenheiro de dados o contexto e as definições corretos para usar os dados corretamente.

Os ***metadados técnicos*** descrevem os dados criados e usados pelos sistemas em todo o ciclo de vida da engenharia de dados. Inclui o modelo e esquema de dados, linhagem de dados, mapeamentos de campo e fluxos de trabalho de pipeline. Um engenheiro de dados usa metadados técnicos para criar, conectar e monitorar vários sistemas ao longo do ciclo de vida da engenharia de dados.

Aqui estão alguns tipos comuns de metadados técnicos que um engenheiro de dados usará:

Metadados de pipeline (geralmente produzidos em sistemas de orquestração)

Linhagem de dados

Esquema

A orquestração é um hub central que coordena o fluxo de trabalho em vários sistemas.

Metadados de pipeline capturados em sistemas de orquestração fornecem

Machine Translated by Google

detalhes da programação do fluxo de trabalho, dependências de sistema e dados, configurações, detalhes de conexão e muito mais.

Os metadados de linhagem de dados rastreiam a origem e as alterações nos dados e suas dependências ao longo do tempo. À medida que os dados fluem pelo ciclo de vida da engenharia de dados, eles evoluem por meio de transformações e combinações com outros dados. A linhagem de dados fornece uma trilha de auditoria da evolução dos dados à medida que se movem por vários sistemas e fluxos de trabalho.

Os ***metadados do esquema*** descrevem a estrutura dos dados armazenados em um sistema, como um banco de dados, um data warehouse, um data lake ou um sistema de arquivos; é um dos principais diferenciais em diferentes sistemas de armazenamento. Armazenamentos de objetos, por exemplo, não gerenciam metadados de esquema; em vez disso, isso deve ser gerenciado em um ***metastore.*** Por outro lado, os data warehouses na nuvem gerenciam os metadados do esquema internamente.

Estes são apenas alguns exemplos de metadados técnicos que um engenheiro de dados deve conhecer. Esta não é uma lista completa e abordamos aspectos adicionais de metadados técnicos em todo o livro.

Os metadados operacionais descrevem os resultados operacionais de vários sistemas e incluem estatísticas sobre processos, IDs de trabalhos, logs de tempo de execução do aplicativo, dados usados em um processo e logs de erros. Um engenheiro de dados usa metadados operacionais para determinar se um processo foi bem-sucedido ou falhou e os dados envolvidos no processo.

Os sistemas de orquestração podem fornecer uma imagem limitada de metadados operacionais, mas estes ainda tendem a estar espalhados por muitos sistemas. A necessidade de metadados operacionais de melhor qualidade e melhor gerenciamento de metadados é uma das principais motivações para os sistemas de orquestração e gerenciamento de metadados de última geração.

Metadados de referência são dados usados para classificar outros dados. Isso também é conhecido como ***dados de pesquisa.*** Exemplos padrão de dados de referência são códigos internos, códigos geográficos, unidades de medida e padrões de calendário internos. Observe que muitos dos dados de referência são totalmente gerenciados internamente, mas itens como códigos geográficos podem vir de referências externas padrão.

Machine Translated by Google

Os dados de referência são essencialmente um padrão para interpretar outros dados, portanto, se houver alterações, essa alteração ocorrerá lentamente ao longo do tempo.

Responsabilidade de dados

Responsabilidade de dados significa designar um indivíduo para controlar uma parte dos dados. A pessoa responsável coordena então as atividades de governança de outras partes interessadas. Gerenciar a qualidade dos dados é difícil se ninguém for responsável pelos dados em questão.

Observe que as pessoas responsáveis pelos dados não precisam ser engenheiros de dados. A pessoa responsável pode ser um engenheiro de software ou gerente de produto, ou servir em outra função. Além disso, o responsável geralmente não possui todos os recursos necessários para manter a qualidade dos dados. Em vez disso, eles coordenam com todas as pessoas que tocam os dados, incluindo engenheiros de dados.

A responsabilidade de dados pode acontecer em vários níveis; a responsabilidade pode ocorrer no nível de uma tabela ou de um fluxo de logs, mas pode ser tão refinada quanto uma entidade de campo único que ocorre em muitas tabelas. Um indivíduo pode ser responsável pelo gerenciamento de um ID de cliente em vários sistemas. Para gerenciamento de dados corporativos, um domínio de dados é o conjunto de todos os valores possíveis que podem ocorrer para um determinado tipo de campo, como neste exemplo de ID. Isso pode parecer excessivamente burocrático e meticuloso, mas pode afetar significativamente a qualidade dos dados.

Qualidade dos dados

Posso confiar nesses dados?

—Todos no negócio

A qualidade dos dados é a otimização dos dados em direção ao estado desejado e orbita a pergunta: “O que você obtém em comparação com o que espera?” Os dados devem estar em conformidade com as expectativas nos metadados de negócios. Os dados correspondem à definição acordada pela empresa?

Um engenheiro de dados garante a qualidade dos dados em todo o ciclo de vida da engenharia de dados. Isso envolve realizar testes de qualidade de dados e garantir a conformidade dos dados com as expectativas do esquema, integridade e precisão dos dados.

Machine Translated by Google

De acordo com o ***Data Governance: The Definitive Guide,*** a qualidade dos dados é definida por três características principais: [4]

Precisão

Os dados coletados estão realmente corretos? Existem valores duplicados? São os valores numéricos são precisos?

Completude

Os registros estão completos? Todos os campos obrigatórios contêm valores válidos?

Pontualidade

Os registros estão disponíveis em tempo hábil?

Cada uma dessas características é bastante matizada. Por exemplo, como pensamos em bots e web scrapers ao lidar com dados de eventos da web? Se pretendemos analisar a jornada do cliente, devemos ter um processo que nos permita separar humanos do tráfego gerado por máquina. Quaisquer eventos gerados por bots classificados erroneamente como ***humanos*** apresentam problemas de precisão de dados e vice-versa.

Uma variedade de problemas interessantes surgem em relação à completude e pontualidade. No artigo do Google que apresenta o modelo Dataflow, os autores dão o exemplo de uma plataforma de vídeo offline que exibe anúncios. A plataforma baixa vídeos e anúncios [5] enquanto uma conexão está presente, permite que o usuário assista enquanto estiver offline e, em seguida, carrega dados de visualização de anúncios quando uma conexão está presente novamente. Esses dados podem chegar tarde, bem depois que os anúncios forem assistidos. Como a plataforma lida com o faturamento dos anúncios?

Fundamentalmente, este problema não pode ser resolvido por meios puramente técnicos. Em vez disso, os engenheiros precisarão determinar seus padrões para dados de chegada tardia e aplicá-los de maneira uniforme, possivelmente com a ajuda de várias ferramentas de tecnologia.

A qualidade dos dados está no limite dos problemas humanos e tecnológicos.

Os engenheiros de dados precisam de processos robustos para coletar feedback humano acionável sobre a qualidade dos dados e usar ferramentas de tecnologia para detectar problemas de qualidade

Machine Translated by Google

preventivamente antes que os usuários a jusante os vejam. Cobrimos esses processos de coleta nos capítulos apropriados ao longo deste livro.

GESTÃO DE DADOS PRINCIPAIS

Dados mestres são dados sobre entidades de negócios, como funcionários, clientes, produtos e locais. À medida que as organizações crescem e se tornam mais complexas por meio de crescimento orgânico e aquisições, e colaboram com outros negócios, manter uma imagem consistente de entidades e identidades se torna cada vez mais desafiador.

O gerenciamento de dados mestre (MDM) é a prática de criar definições de entidade consistentes conhecidas como ***registros dourados.*** Os registros dourados harmonizam os dados da entidade em uma organização e com seus parceiros. O MDM é um processo de operações de negócios facilitado pela criação e implantação de ferramentas de tecnologia. Por exemplo, uma equipe de MDM pode determinar um formato padrão para endereços e, em seguida, trabalhar com engenheiros de dados para criar uma API para retornar endereços consistentes e um sistema que use dados de endereço para corresponder registros de clientes nas divisões da empresa.

O MDM alcança todo o ciclo de dados em bancos de dados operacionais. Pode estar diretamente sob a alçada da engenharia de dados, mas geralmente é responsabilidade atribuída a uma equipe dedicada que trabalha em toda a organização. Mesmo que não possuam MDM, os engenheiros de dados devem estar sempre cientes disso, pois colaborarão em iniciativas de MDM.

Modelagem e design de dados Para

obter insights de negócios a partir de dados, por meio de análise de negócios e ciência de dados, os dados devem estar em um formato utilizável. O processo de conversão de dados em um formato utilizável é conhecido como ***modelagem e design de dados.*** Enquanto tradicionalmente pensamos na modelagem de dados como um problema para administradores de banco de dados (DBAs) e desenvolvedores de ETL, a modelagem de dados pode acontecer em praticamente qualquer lugar de uma organização. Os engenheiros de firmware desenvolvem o formato de dados de um registro para um dispositivo IoT ou o design de desenvolvedores de aplicativos da Web

Machine Translated by Google

a resposta JSON a uma chamada de API ou a um esquema de tabela MySQL — todas são instâncias de modelagem e design de dados.

A modelagem de dados tornou-se mais desafiadora devido à variedade de novas fontes de dados e casos de uso. Por exemplo, a normalização estrita não funciona bem com dados de eventos. Felizmente, uma nova geração de ferramentas de dados aumenta a flexibilidade dos modelos de dados, mantendo as separações lógicas de medidas, dimensões, atributos e hierarquias. Os data warehouses em nuvem suportam a ingestão de enormes quantidades de dados desnormalizados e semiestruturados, enquanto ainda suportam padrões comuns de modelagem de dados, como Kimball, Inmon e cofre de dados. Estruturas de processamento de dados, como o Spark, podem ingerir todo um espectro de dados, desde registros relacionais estruturados simples até texto bruto não estruturado. Discutimos esses padrões de modelagem e transformação de dados com mais detalhes no Capítulo 8.

Com a grande variedade de dados com os quais os engenheiros precisam lidar, existe a tentação de jogar as mãos para o alto e desistir da modelagem de dados. Esta é uma ideia terrível com consequências angustiantes, evidenciadas quando as pessoas murmuram sobre o padrão de acesso write uma vez, read never (WORN) ou referem-se a um ***pântano de dados.*** Os engenheiros de dados precisam entender as práticas recomendadas de modelagem, bem como desenvolver a flexibilidade para aplicar o nível e o tipo apropriados de modelagem à fonte de dados e ao caso de uso.

Linhagem de dados

À medida que os dados se movem ao longo de seu ciclo de vida, como você sabe qual sistema afetou os dados ou do que os dados são compostos à medida que são transmitidos e transformados? ***A linhagem*** de dados descreve o registro de uma trilha de auditoria de dados ao longo de seu ciclo de vida, rastreando os sistemas que processam os dados e os dados upstream dos quais depende.

A linhagem de dados ajuda no rastreamento de erros, responsabilidade e depuração de dados e dos sistemas que os processam. Ele tem o benefício óbvio de fornecer uma trilha de auditoria para o ciclo de vida dos dados e ajuda na conformidade. Por exemplo, se um usuário deseja que seus dados sejam excluídos de seus sistemas, ter a linhagem desses dados permite que você saiba onde esses dados estão armazenados e suas dependências.

Machine Translated by Google

A linhagem de dados existe há muito tempo em empresas maiores com padrões rígidos de conformidade. No entanto, agora está sendo mais amplamente adotado em empresas menores à medida que o gerenciamento de dados se torna popular. Também observamos que o conceito de Desenvolvimento Orientado à Observabilidade de Dados (DODD) de Andy Petrella está intimamente relacionado à linhagem de dados. O DODD observa dados ao longo de sua linhagem. Este processo é aplicado durante o desenvolvimento, teste e, finalmente, produção para entregar qualidade e conformidade com as expectativas.

Integração e interoperabilidade de dados

A integração e interoperabilidade de dados é o processo de integração de dados entre ferramentas e processos. À medida que nos afastamos de uma abordagem de pilha única para análise e em direção a um ambiente de nuvem heterogêneo no qual várias ferramentas processam dados sob demanda, a integração e a interoperabilidade ocupam uma faixa cada vez maior do trabalho do engenheiro de dados.

Cada vez mais, a integração acontece por meio de APIs de uso geral, em vez de conexões de banco de dados personalizadas. Por exemplo, um pipeline de dados pode extrair dados da API do Salesforce, armazená-los no Amazon S3, chamar a API do Snowflake para carregá-los em uma tabela, chamar a API novamente para executar uma consulta e exportar os resultados para o S3, onde o Spark pode consumi-los.

Toda essa atividade pode ser gerenciada com código Python relativamente simples que se comunica com sistemas de dados em vez de manipular os dados diretamente. Embora a complexidade da interação com os sistemas de dados tenha diminuído, o número de sistemas e a complexidade dos pipelines aumentaram drasticamente.
Engenheiros começando do zero rapidamente superam os recursos de scripts sob medida e se deparam com a necessidade de ***orquestração.*** A orquestração é uma de nossas correntes, e discutimos isso em detalhes em “Orquestração”.

Gerenciamento do ciclo de vida dos dados

O advento dos data lakes encorajou as organizações a ignorar o arquivamento e a destruição de dados. Por que descartar dados quando você pode simplesmente adicionar mais armazenamento ad infinitum? Duas mudanças incentivaram os engenheiros a prestar mais atenção ao que acontece no final do ciclo de vida da engenharia de dados.

Machine Translated by Google

Primeiro, os dados são cada vez mais armazenados na nuvem. Isso significa que temos custos de armazenamento com pagamento conforme o uso, em vez de grandes despesas de capital iniciais para um data lake local. Quando cada byte aparece em um extrato mensal da AWS, os CFOs veem oportunidades de economia. Os ambientes em nuvem tornam o arquivamento de dados um processo relativamente simples. Os principais fornecedores de nuvem oferecem classes de armazenamento de objetos específicas de arquivamento que permitem a retenção de dados a longo prazo a um custo extremamente baixo, assumindo acesso muito pouco frequente (deve-se notar que a recuperação de dados não é tão barata, mas isso é para outra conversa).
Essas classes de armazenamento também oferecem suporte a controles de política extras para evitar a exclusão acidental ou deliberada de arquivos críticos.

Em segundo lugar, as leis de privacidade e retenção de dados, como o GDPR e o CCPA, exigem que os engenheiros de dados gerenciem ativamente a destruição de dados para respeitar o "direito de ser esquecido" dos usuários. Os engenheiros de dados devem saber quais dados do consumidor eles retêm e devem ter procedimentos para destruir dados em resposta a solicitações e requisitos de conformidade.

A destruição de dados é direta em um data warehouse na nuvem. A semântica SQL permite a exclusão de linhas em conformidade com uma cláusula where. A destruição de dados era mais desafiadora em data lakes, onde escrever uma vez, ler muitos era o padrão de armazenamento padrão. Ferramentas como Hive ACID e Delta Lake permitem o gerenciamento fácil de transações de exclusão em escala. Novas gerações de ferramentas de gerenciamento de metadados, linhagem de dados e catalogação também simplificarão o final do ciclo de vida da engenharia de dados.

Ética e privacidade

> ***Comportamento ético é fazer a coisa certa quando ninguém mais está olhando.***
>
> —Aldo Leopoldo

Os últimos anos de violações de dados, desinformação e manuseio incorreto de dados deixam uma coisa clara: os dados afetam as pessoas. Os dados costumavam viver no Velho Oeste, coletados e negociados livremente como cartões de beisebol. Esses dias já se foram. Enquanto as implicações éticas e de privacidade dos dados já foram consideradas boas de se ter, como segurança, agora elas são centrais para o ciclo de vida geral dos dados. Os engenheiros de dados precisam fazer a coisa certa quando ninguém mais está

Machine Translated by Google

assistindo, porque todo mundo vai assistir um dia. Esperamos que mais organizações encorajem uma cultura de boa ética e privacidade de dados.

Como a ética e a privacidade afetam o ciclo de vida da engenharia de dados? Os engenheiros de dados precisam garantir que os conjuntos de dados mascarem informações de identificação pessoal (PII) e outras informações confidenciais; o viés pode ser identificado e rastreado em conjuntos de dados à medida que são transformados. Os requisitos regulamentares e as penalidades de conformidade estão crescendo. Garanta que seus ativos de dados estejam em conformidade com um número crescente de regulamentações de dados, como GDPR e CCPA. Por favor, leve isso a sério. Oferecemos dicas ao longo do livro para garantir que você esteja introduzindo ética e privacidade no ciclo de vida da engenharia de dados.

Orquestração

> ***Achamos que a orquestração é importante porque a vemos realmente como o centro de gravidade tanto da plataforma de dados quanto do ciclo de vida dos dados, o ciclo de vida do desenvolvimento de software no que diz respeito aos dados.***
>
> —Nick Schrock, fundador da Elementl

A orquestração não é apenas um processo central de DataOps, mas também uma parte crítica do fluxo de engenharia e implantação para tarefas de dados. Então, o que é orquestração?

A ***orquestração*** é o processo de coordenar muitos trabalhos para serem executados da forma mais rápida e eficiente possível em uma cadência programada. Por exemplo, as pessoas costumam se referir a ferramentas de orquestração como o Apache Airflow como ***agendadores.*** Isso não é muito preciso. Um agendador puro, como o cron, está ciente apenas do tempo; um mecanismo de orquestração cria metadados em dependências de trabalho, geralmente na forma de um gráfico acíclico direcionado (DAG). O DAG pode ser executado uma vez ou programado para ser executado em um intervalo fixo diário, semanal, a cada hora, a cada cinco minutos, etc.

À medida que discutimos a orquestração ao longo deste livro, presumimos que um sistema de orquestração permaneça online com alta disponibilidade. Isso permite que o sistema de orquestração detecte e monitore constantemente sem intervenção humana e execute novos trabalhos sempre que forem implantados. Uma orquestração

Machine Translated by Google

o sistema monitora os trabalhos que gerencia e inicia novas tarefas à medida que as dependências internas do DAG são concluídas. Ele também pode monitorar sistemas e ferramentas externas para observar a chegada de dados e o cumprimento de critérios. Quando certas condições saem dos limites, o sistema também define condições de erro e envia alertas por e-mail ou outros canais. Você pode definir um tempo de conclusão esperado de 10h para pipelines de dados diários noturnos. Se os trabalhos não forem concluídos até esse momento, os alertas serão enviados para engenheiros de dados e consumidores.

Os sistemas de orquestração também criam recursos de histórico de trabalho, visualização e alertas. Mecanismos de orquestração avançados podem preencher novos DAGs ou tarefas individuais à medida que são adicionados a um DAG. Eles também suportam dependências em um intervalo de tempo. Por exemplo, um trabalho de relatório mensal pode verificar se um trabalho de ETL foi concluído no mês inteiro antes de iniciar.

A orquestração tem sido um recurso essencial para o processamento de dados, mas muitas vezes não era a prioridade nem acessível a ninguém, exceto as maiores empresas. As empresas usavam várias ferramentas para gerenciar fluxos de trabalho, mas eram caras, fora do alcance de pequenas startups e geralmente não extensíveis. O Apache Oozie era extremamente popular na década de 2010, mas foi projetado para funcionar em um cluster Hadoop e era difícil de usar em um ambiente mais heterogêneo. O Facebook desenvolveu o Dataswarm para uso interno no final dos anos 2000; isso inspirou ferramentas populares como o Airflow, lançado pelo Airbnb em 2014.

O Airflow era de código aberto desde o início e foi amplamente adotado. Foi escrito em Python, tornando-o altamente extensível a quase todos os casos de uso imagináveis. Embora existam muitos outros projetos interessantes de orquestração de código aberto, como Luigi e Conductor, o Airflow é sem dúvida o líder de mindshare por enquanto. O fluxo de ar chegou no momento em que o processamento de dados estava se tornando mais abstrato e acessível, e os engenheiros estavam cada vez mais interessados em coordenar fluxos complexos em vários processadores e sistemas de armazenamento, especialmente em ambientes de nuvem.

No momento da redação deste artigo, vários projetos de código aberto nascentes visam imitar os melhores elementos do design principal do Airflow, aprimorando-o em áreas-chave. Algum

Machine Translated by Google

dos exemplos mais interessantes são Prefect e Dagster, que visam melhorar a portabilidade e testabilidade dos DAGs para permitir que os engenheiros passem do desenvolvimento local para a produção com mais facilidade. Argo é um mecanismo de orquestração construído em torno de primitivos do Kubernetes; Metaflow é um projeto de código aberto da Netflix que visa melhorar a orquestração da ciência de dados.

Devemos salientar que a orquestração é estritamente um conceito de lote. A alternativa de streaming para DAGs de tarefas orquestradas é o DAG de streaming. Os DAGs de streaming continuam sendo um desafio para construir e manter, mas as plataformas de streaming de próxima geração, como o Pulsar, visam reduzir drasticamente a carga de engenharia e operacional. Falamos mais sobre esses desenvolvimentos no Capítulo 8.

DataOps

O DataOps mapeia as melhores práticas da metodologia Agile, DevOps e controle estatístico de processos (SPC) para os dados. Enquanto o DevOps visa melhorar o lançamento e a qualidade dos produtos de software, o DataOps faz o mesmo com os produtos de dados.

Os produtos de dados diferem dos produtos de software devido à maneira como os dados são usados. Um produto de software fornece funcionalidades e recursos técnicos específicos para usuários finais. Por outro lado, um produto de dados é construído em torno de métricas e lógica de negócios sólidas, cujos usuários tomam decisões ou constroem modelos que executam ações automatizadas. Um engenheiro de dados deve entender os aspectos técnicos da criação de produtos de software e a lógica de negócios, qualidade e métricas que criarão excelentes produtos de dados.

Assim como o DevOps, o DataOps empresta muito da manufatura enxuta e do gerenciamento da cadeia de suprimentos, misturando pessoas, processos e tecnologia para reduzir o tempo de retorno. Como o Data Kitchen (especialista em DataOps) descreve:6

Machine Translated by Google

DataOps é uma coleção de práticas técnicas, fluxos de trabalho, normas culturais e padrões de arquitetura que permitem:

Inovação e experimentação rápidas, fornecendo novos insights aos clientes com velocidade crescente

Qualidade de dados extremamente alta e taxas de erro muito baixas

Colaboração em conjuntos complexos de pessoas, tecnologia e ambientes

Medição clara, monitoramento e transparência dos resultados

Práticas Lean (como redução de lead time e minimização de defeitos) e as melhorias resultantes de qualidade e produtividade são coisas que estamos felizes em ver ganhando força tanto em operações de software quanto de dados.

Em primeiro lugar, DataOps é um conjunto de hábitos culturais; a equipe de engenharia de dados precisa adotar um ciclo de comunicação e colaboração com os negócios, quebra de silos, aprendizado contínuo com acertos e erros e iteração rápida. Somente quando esses hábitos culturais são estabelecidos, a equipe pode obter os melhores resultados de tecnologia e ferramentas.

Dependendo da maturidade de dados de uma empresa, um engenheiro de dados tem algumas opções para incorporar o DataOps na estrutura do ciclo de vida geral da engenharia de dados. Se a empresa não tiver infraestrutura de dados ou práticas preexistentes, o DataOps é uma oportunidade inexplorada que pode ser incorporada desde o primeiro dia. Com um projeto ou infraestrutura existente sem DataOps, um engenheiro de dados pode começar a adicionar DataOps aos fluxos de trabalho. Sugerimos primeiro começar com observabilidade e monitoramento para obter uma visão geral do desempenho de um sistema e, em seguida, adicionar automação e resposta a incidentes. Um engenheiro de dados pode trabalhar ao lado de uma equipe de DataOps existente para melhorar o ciclo de vida de engenharia de dados em uma empresa com maturidade de dados. Em todos os casos, um engenheiro de dados deve estar ciente da filosofia e dos aspectos técnicos do DataOps.

O DataOps tem três elementos técnicos principais: automação, monitoramento e observabilidade e resposta a incidentes (Figura 2-8). Vejamos cada uma dessas partes e como elas se relacionam com o ciclo de vida da engenharia de dados.

Machine Translated by Google

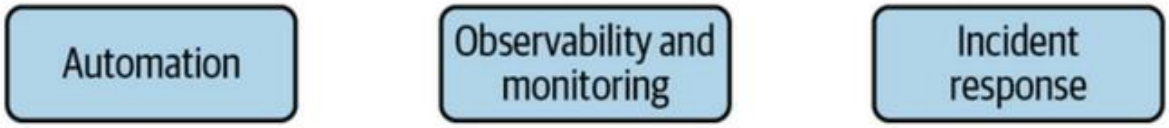


Figura 2-8. Os três pilares do DataOps

Automação

A automação permite confiabilidade e consistência no processo DataOps e permite que engenheiros de dados implantem rapidamente novos recursos de produtos e melhorias nos fluxos de trabalho existentes. A automação de DataOps tem uma estrutura e fluxo de trabalho semelhantes ao DevOps, consistindo em gerenciamento de mudanças (ambiente, código e controle de versão de dados), integração contínua/implantação contínua (CI/CD) e configuração como código.
Assim como o DevOps, as práticas do DataOps monitoram e mantêm a confiabilidade da tecnologia e dos sistemas (pipelines de dados, orquestração etc.), com a dimensão adicional de verificação de qualidade de dados, desvio de dados/modelo, integridade de metadados e muito mais.

Vamos discutir brevemente a evolução da automação de DataOps dentro de uma organização hipotética. Uma organização com um baixo nível de maturidade de DataOps muitas vezes tenta agendar vários estágios de processos de transformação de dados usando cron jobs. Isso funciona bem por um tempo. À medida que os pipelines de dados se tornam mais complicados, é provável que várias coisas aconteçam. Se os cron jobs estiverem hospedados em uma instância de nuvem, a instância poderá ter um problema operacional, fazendo com que os jobs parem de ser executados inesperadamente. À medida que o espaçamento entre os trabalhos se torna mais estreito, um trabalho acabará por ser executado por muito tempo, fazendo com que um trabalho subsequente falhe ou produza dados obsoletos. Os engenheiros podem não estar cientes das falhas de trabalho até ouvirem dos analistas que seus relatórios estão desatualizados.

À medida que a maturidade de dados da organização cresce, os engenheiros de dados normalmente adotam uma estrutura de orquestração, talvez Airflow ou Dagster. Os engenheiros de dados estão cientes de que o Airflow representa um fardo operacional, mas os benefícios da orquestração eventualmente superam a complexidade. Os engenheiros migrarão gradualmente seus trabalhos cron para trabalhos do Airflow. Agora, as dependências são verificadas antes da execução dos jobs. Mais tarefas de transformação podem ser empacotadas em um determinado horário porque cada tarefa pode ser iniciada assim que os dados upstream estiverem prontos, em vez de em um horário fixo e predeterminado.

Machine Translated by Google

A equipe de engenharia de dados ainda tem espaço para melhorias operacionais. Um cientista de dados eventualmente implanta um DAG quebrado, derrubando o servidor web do Airflow e deixando a equipe de dados operacionalmente cega. Depois de tantas dores de cabeça, os membros da equipe de engenharia de dados percebem que precisam parar de permitir implantações manuais de DAG. Em sua próxima fase de maturidade operacional, eles adotam a implantação automatizada de DAG. Os DAGs são testados antes da implantação e os processos de monitoramento garantem que os novos DAGs comecem a funcionar corretamente. Além disso, os engenheiros de dados bloqueiam a implantação de novas dependências do Python até que a instalação seja validada.
Depois que a automação é adotada, a equipe de dados fica muito mais feliz e experimenta muito menos dores de cabeça.

Um dos princípios do Manifesto DataOps é “Abrace a mudança”. Isso não significa mudança pela mudança, mas mudança orientada a objetivos. Em cada estágio de nossa jornada de automação, existem oportunidades de melhoria operacional. Mesmo no alto nível de maturidade que descrevemos aqui, ainda há espaço para melhorias. Os engenheiros podem adotar uma estrutura de orquestração de próxima geração que incorpora melhores recursos de metadados. Ou eles podem tentar desenvolver uma estrutura que cria DAGs automaticamente com base em especificações de linhagem de dados. O ponto principal é que os engenheiros buscam constantemente implementar melhorias na automação que reduzam sua carga de trabalho e aumentem o valor que entregam ao negócio.

Observabilidade e monitoramento

Como dizemos aos nossos clientes: “Os dados são um assassino silencioso”. Vimos inúmeros exemplos de dados ruins que permanecem em relatórios por meses ou anos. Os executivos podem tomar decisões importantes a partir desses dados ruins, descobrindo o erro apenas muito mais tarde. Os resultados geralmente são ruins e às vezes catastróficos para o negócio. As iniciativas são minadas e destruídas, anos de trabalho desperdiçados. Em alguns dos piores casos, dados ruins podem levar as empresas à ruína financeira.

Outra história de horror ocorre quando os sistemas que criam os dados para relatórios param de funcionar aleatoriamente, resultando em relatórios atrasados por

Machine Translated by Google

muitos dias. A equipe de dados não sabe até que seja perguntado pelas partes interessadas por que os relatórios estão atrasados ou produzem informações obsoletas. Eventualmente, várias partes interessadas perdem a confiança nos recursos da equipe de dados principal e iniciam suas próprias equipes fragmentadas. O resultado são muitos sistemas instáveis diferentes, relatórios inconsistentes e silos.

Se você não estiver observando e monitorando seus dados e os sistemas que os produzem, inevitavelmente experimentará sua própria história de horror de dados. Observabilidade, monitoramento, registro, alerta e rastreamento são essenciais para se antecipar a quaisquer problemas ao longo do ciclo de vida da engenharia de dados. Recomendamos que você incorpore o SPC para entender se os eventos monitorados estão fora de linha e quais incidentes valem a pena responder.

O método DODD de Petrella mencionado anteriormente neste capítulo fornece uma excelente estrutura para pensar sobre a observabilidade de dados. O DODD é muito parecido com o desenvolvimento orientado a testes (TDD) na engenharia de software:

> ***O objetivo do DODD é dar a todos os envolvidos na cadeia de dados visibilidade dos dados e dos aplicativos de dados, para que todos os envolvidos na cadeia de valor de dados tenham a capacidade de identificar alterações nos dados ou nos aplicativos de dados em cada etapa - da ingestão à transformação para análise — para ajudar a solucionar ou evitar problemas de dados. O DODD se concentra em tornar a observabilidade de dados uma consideração de primeira classe no ciclo de vida da engenharia de dados.*** [7]

Cobrimos muitos aspectos de monitoramento e observabilidade em todo o ciclo de vida da engenharia de dados em capítulos posteriores.

Resposta a incidentes

Uma equipe de dados de alto funcionamento usando DataOps poderá enviar novos produtos de dados rapidamente. Mas erros inevitavelmente acontecerão. Um sistema pode ter tempo de inatividade, um novo modelo de dados pode interromper relatórios de downstream, um modelo de ML pode ficar obsoleto e fornecer previsões ruins - inúmeros problemas podem interromper o ciclo de vida da engenharia de dados. ***A resposta a incidentes*** é usar os recursos de automação e observabilidade mencionados anteriormente para

Machine Translated by Google

identificar as causas-raiz de um incidente e resolvê-lo da forma mais confiável e rápida possível.

A resposta a incidentes não envolve apenas tecnologia e ferramentas, embora sejam benéficas; trata-se também de comunicação aberta e irrepreensível, tanto na equipe de engenharia de dados quanto em toda a organização. Como Werner Vogels é famoso por dizer: “Tudo quebra o tempo todo”. Os engenheiros de dados devem estar preparados para um desastre e prontos para responder da forma mais rápida e eficiente possível.

Os engenheiros de dados devem encontrar problemas de forma proativa antes que a empresa os relate. A falha acontece e, quando as partes interessadas ou usuários finais veem problemas, eles os apresentam. Eles ficarão infelizes ao fazê-lo. A sensação é diferente quando eles vão levantar esses problemas para uma equipe e veem que eles já estão sendo trabalhados ativamente para resolver. Em qual estado de equipe você confiaria mais como usuário final? A confiança leva muito tempo para ser construída e pode ser perdida em minutos. A resposta a incidentes consiste tanto em responder retroativamente a incidentes quanto em abordá-los proativamente antes que eles aconteçam.

Resumo do DataOps

Neste ponto, o DataOps ainda é um trabalho em andamento. Os profissionais fizeram um bom trabalho ao adaptar os princípios de DevOps ao domínio de dados e mapear uma visão inicial por meio do Manifesto de DataOps e outros recursos.
Os engenheiros de dados fariam bem em tornar as práticas de DataOps uma alta prioridade em todo o seu trabalho. O esforço inicial trará um retorno significativo a longo prazo por meio da entrega mais rápida de produtos, melhor confiabilidade e precisão dos dados e maior valor geral para os negócios.

O estado das operações na engenharia de dados ainda é bastante imaturo em comparação com a engenharia de software. Muitas ferramentas de engenharia de dados, especialmente monólitos legados, não são automatizadas em primeiro lugar. Um movimento recente surgiu para adotar as melhores práticas de automação em todo o ciclo de vida da engenharia de dados. Ferramentas como o Airflow abriram caminho para uma nova geração de ferramentas de automação e gerenciamento de dados. As práticas gerais que descrevemos para DataOps são aspiracionais, e sugerimos que as empresas tentem adotá-las ao máximo, dadas as ferramentas e o conhecimento disponíveis hoje.

Machine Translated by Google

Arquitetura de dados

Uma arquitetura de dados reflete o estado atual e futuro dos sistemas de dados que suportam as necessidades e a estratégia de dados de longo prazo de uma organização. Como os requisitos de dados de uma organização provavelmente mudarão rapidamente e novas ferramentas e práticas parecem chegar quase diariamente, os engenheiros de dados devem entender uma boa arquitetura de dados. O Capítulo 3 cobre a arquitetura de dados em profundidade, mas queremos destacar aqui que a arquitetura de dados é uma tendência do ciclo de vida da engenharia de dados.

Um engenheiro de dados deve primeiro entender as necessidades do negócio e reunir requisitos para novos casos de uso. Em seguida, um engenheiro de dados precisa traduzir esses requisitos para projetar novas maneiras de capturar e fornecer dados, equilibrados por custo e simplicidade operacional. Isso significa conhecer os trade-offs com padrões de design, tecnologias e ferramentas em sistemas de origem, ingestão, armazenamento, transformação e serviço de dados.

Isso não significa que um engenheiro de dados seja um arquiteto de dados, pois normalmente são duas funções separadas. Se um engenheiro de dados trabalha ao lado de um arquiteto de dados, o engenheiro de dados deve ser capaz de entregar os projetos do arquiteto de dados e fornecer feedback de arquitetura.

Engenharia de software

A engenharia de software sempre foi uma habilidade central para engenheiros de dados. Nos primeiros dias da engenharia de dados contemporânea (2000–2010), engenheiros de dados trabalhavam em estruturas de baixo nível e escreviam trabalhos MapReduce em C, C++ e Java. No auge da era do big data (meados da década de 2010), os engenheiros começaram a usar estruturas que abstraíam esses detalhes de baixo nível.

Essa abstração continua até hoje. Os data warehouses na nuvem oferecem suporte a transformações poderosas usando a semântica do SQL; ferramentas como o Spark tornaram-se mais fáceis de usar, deixando de lado os detalhes de codificação de baixo nível para dataframes fáceis de usar. Apesar dessa abstração, a engenharia de software ainda é fundamental para a engenharia de dados. Queremos discutir brevemente algumas áreas comuns da engenharia de software que se aplicam ao ciclo de vida da engenharia de dados.

Machine Translated by Google

Código de processamento de dados principal

Embora tenha se tornado mais abstrato e fácil de gerenciar, o código principal de processamento de dados ainda precisa ser escrito e aparece em todo o ciclo de vida da engenharia de dados. Seja em ingestão, transformação ou serviço de dados, os engenheiros de dados precisam ser altamente proficientes e produtivos em estruturas e linguagens como Spark, SQL ou Beam; rejeitamos a noção de que SQL não é código.

Também é imperativo que um engenheiro de dados entenda as metodologias adequadas de teste de código, como unidade, regressão, integração, ponta a ponta e fumaça.

Desenvolvimento de frameworks de código aberto

Muitos engenheiros de dados estão fortemente envolvidos no desenvolvimento de estruturas de código aberto. Eles adotam essas estruturas para resolver problemas específicos no ciclo de vida da engenharia de dados e, em seguida, continuam desenvolvendo o código da estrutura para melhorar as ferramentas para seus casos de uso e contribuir com a comunidade.

Na era do big data, vimos uma explosão cambriana de estruturas de processamento de dados dentro do ecossistema Hadoop. Essas ferramentas se concentraram principalmente em transformar e servir partes do ciclo de vida da engenharia de dados. A especiação de ferramentas de engenharia de dados não cessou ou desacelerou, mas a ênfase mudou para cima na escada da abstração, longe do processamento direto de dados. Essa nova geração de ferramentas de código aberto auxilia os engenheiros no gerenciamento, aprimoramento, conexão, otimização e monitoramento de dados.

Por exemplo, o Airflow dominou o espaço de orquestração de 2015 até o início dos anos 2020. Agora, um novo lote de concorrentes de código aberto (incluindo Prefect, Dagster e Metaflow) surgiu para corrigir as limitações percebidas do Airflow, fornecendo melhor manuseio de metadados, portabilidade e gerenciamento de dependências. Para onde vai o futuro da orquestração é uma incógnita.

Antes que os engenheiros de dados comecem a projetar novas ferramentas internas, eles fariam bem em pesquisar o cenário de ferramentas disponíveis publicamente. Fique de olho no custo total de propriedade (TCO) e no custo de oportunidade associado à implementação de uma ferramenta. Há uma boa chance de que um projeto de código aberto

Machine Translated by Google

já existe para resolver o problema que eles procuram resolver, e eles fariam bem em colaborar em vez de reinventar a roda.

Transmissão

O processamento de dados em streaming é inerentemente mais complicado do que em lote, e as ferramentas e paradigmas são indiscutivelmente menos maduros. À medida que o streaming de dados se torna mais difundido em todas as etapas do ciclo de vida da engenharia de dados, os engenheiros de dados enfrentam problemas interessantes de engenharia de software.

Por exemplo, tarefas de processamento de dados, como junções, que consideramos garantidas no mundo do processamento em lote, geralmente se tornam mais complicadas em tempo real, exigindo engenharia de software mais complexa. Os engenheiros também devem escrever código para aplicar uma variedade de métodos de ***janelas .*** A janela permite que os sistemas em tempo real calculem métricas valiosas, como estatísticas finais. Os engenheiros têm muitas estruturas para escolher, incluindo várias plataformas de funções (OpenFaaS, AWS Lambda, Google Cloud Functions) para lidar com eventos individuais ou processadores de fluxo dedicados (Spark, Beam, Flink ou Pulsar) para analisar fluxos para oferecer suporte a relatórios e ações em tempo real .

Infraestrutura como código

Infraestrutura como código (IaC) aplica práticas de engenharia de software à configuração e gerenciamento de infraestrutura. A carga de gerenciamento de infraestrutura da era do big data diminuiu à medida que as empresas migraram para sistemas de big data gerenciados (como Databricks e Amazon EMR) e data warehouses na nuvem. Quando os engenheiros de dados precisam gerenciar sua infraestrutura em um ambiente de nuvem, eles fazem isso cada vez mais por meio de estruturas IaC, em vez de criar instâncias manualmente e instalar software. Várias estruturas de uso geral e específicas de plataforma de nuvem permitem a implantação de infraestrutura automatizada com base em um conjunto de especificações. Muitas dessas estruturas podem gerenciar serviços em nuvem, bem como infraestrutura. Há também uma noção de IaC com containers e Kubernetes, usando ferramentas como o Helm.

Machine Translated by Google

Essas práticas são uma parte vital do DevOps, permitindo o controle de versão e a repetibilidade das implantações. Naturalmente, esses recursos são preciosos em todo o ciclo de vida da engenharia de dados, especialmente quando adotamos práticas de DataOps.

Pipelines como código

Pipelines como código é o conceito central dos sistemas de orquestração atuais, que tocam todas as etapas do ciclo de vida da engenharia de dados. Engenheiros de dados usam código (normalmente Python) para declarar tarefas de dados e dependências entre eles. O mecanismo de orquestração interpreta essas instruções para executar etapas usando os recursos disponíveis.

Resolução de problemas de uso geral

Na prática, independentemente de quais ferramentas de alto nível eles adotem, os engenheiros de dados se deparam com casos de canto ao longo do ciclo de vida da engenharia de dados que exigem que eles resolvam problemas fora dos limites de suas ferramentas escolhidas e escrevam código personalizado. Ao usar estruturas como Fivetran, Airbyte ou Singer, os engenheiros de dados encontrarão fontes de dados sem conectores existentes e precisarão escrever algo personalizado. Eles devem ser proficientes em engenharia de software para entender APIs, extrair e transformar dados, lidar com exceções e assim por diante.

Conclusão

A maioria das discussões que vimos no passado sobre engenharia de dados envolve tecnologias, mas perde a visão geral do gerenciamento do ciclo de vida dos dados. À medida que as tecnologias se tornam mais abstratas e fazem mais trabalho pesado, um engenheiro de dados tem a oportunidade de pensar e agir em um nível mais alto. O ciclo de vida da engenharia de dados, apoiado por suas correntes subjacentes, é um modelo mental extremamente útil para organizar o trabalho da engenharia de dados.

Dividimos o ciclo de vida da engenharia de dados nas seguintes etapas:

- Geração

Machine Translated by Google

Armazenar

Ingestão

Transformação

Dados de veiculação

Vários temas também atravessam o ciclo de vida da engenharia de dados. Essas são as correntes subjacentes do ciclo de vida da engenharia de dados. Em um nível alto, as subcorrentes são as seguintes:

Segurança

Gestão de dados

DataOps

Arquitetura de dados

Orquestração

Engenharia de software

Um engenheiro de dados tem vários objetivos de alto nível em todo o ciclo de vida dos dados: produzir ROI ideal e reduzir custos (financeiros e de oportunidade), reduzir riscos (segurança, qualidade dos dados) e maximizar o valor e a utilidade dos dados.

Os próximos dois capítulos discutem como esses elementos afetam um bom projeto de arquitetura, juntamente com a escolha das tecnologias certas. Se você se sentir confortável com esses dois tópicos, sinta-se à vontade para pular para a Parte II, onde abordamos cada um dos estágios do ciclo de vida da engenharia de dados.

Recursos adicionais

Gerenciando o ciclo de vida da engenharia de dados:

"Estar à frente das dívidas" por Etai Mizrahi

Transformação e processamento de dados:

Machine Translated by Google

"Transformação de dados" Página da Wikipédia

"Processamento de dados" Página da Wikipédia

"Uma comparação de estruturas de processamento de dados" por Ludovic Santos

Subcorrentes:

"O Modelo de Fluxo de Dados: Uma Abordagem Prática para Equilibrar Correção, Latência e Custo no Processamento de Dados em Grande Escala, Sem Limites e Fora de Ordem" por Tyler Akidau et al.

Operações de dados:

"Introdução à automação de DevOps" por Jared Murrel

"Gerenciamento de Incidentes na Era do DevOps" página da Atlassian

"Os sete estágios de uma resposta eficaz a incidentes" Web Atlassian página

"O DevOps está relacionado ao DataOps?" por Carol Jang e Jove Kuang

Gestão de dados:

DAMA Internacional local na rede Internet

Gerenciamento de dados e metadados:

"O que são metadados" por Michelle Knight

Portal de dados do Airbnb:

"Democratizando Dados no Airbnb" por Chris Williams et ai.

"Cinco passos para começar a coletar o valor de seus dados" Página da web Lean-Data

Orquestração:

"Uma introdução ao Dagster: o orquestrador para os dados completos Lifecycle" vídeo de Nick Schrock

Machine Translated by Google

1 Evren Eryurek et al., ***Data Governance: The Definitive Guide*** (Sebastopol, CA: O'Reilly, 2021), 1, ***https:// oreil.ly/ LFT4d.***

2 Eryurek, ***Governança de Dados,*** 5.

3 Chris Williams et al., “Democratizing Data at Airbnb”, ***The Airbnb Tech Blog,*** 12 de maio de 2017, ***https:// oreil.ly/ dM332.***

4 Eryurek, ***Governança de Dados,*** 113.

5 Tyler Akidau et al., “O Modelo de Fluxo de Dados: Uma Abordagem Prática para Equilibrar a Correção, Latência e Custo em Processamento de Dados em Grande Escala, Ilimitado e Fora de Ordem”, ***Anais do VLDB Endowment*** 8 (2015): 1792–1803, https://oreil.ly/Z6XYy.

6 “O que é DataOps”, página de perguntas frequentes do DataKitchen, acessada em 5 de maio de 2022, ***https:// oreil.ly/ Ns06w.***

7 Andy Petrella, “Desenvolvimento Orientado à Observabilidade de Dados: A Analogia Perfeita para Iniciantes”, Kensu, acessado em 5 de maio de 2022, ***https:// oreil.ly/ MxvSX.***

Machine Translated by Google

Capítulo 3. Projetando uma boa arquitetura de dados

Uma boa arquitetura de dados fornece recursos contínuos em todas as etapas do ciclo de vida e subcorrente dos dados. Começaremos definindo ***a arquitetura de dados*** e, em seguida, discutiremos os componentes e as considerações. Em seguida, abordaremos padrões de lote específicos (data warehouses, data lakes), padrões de streaming e padrões que unificam lote e streaming. Durante todo o processo, enfatizaremos o aproveitamento dos recursos da nuvem para fornecer escalabilidade, disponibilidade e confiabilidade.

O que é arquitetura de dados?

A engenharia de dados bem-sucedida é construída sobre uma arquitetura de dados sólida. Este capítulo tem como objetivo revisar algumas abordagens e estruturas de arquitetura populares e, em seguida, elaborar nossa definição opinativa do que torna uma arquitetura de dados “boa”. Na verdade, não vamos fazer todo mundo feliz. Ainda assim, apresentaremos uma definição pragmática, específica de domínio, funcional para ***arquitetura de dados*** que achamos que funcionará para empresas de escalas, processos de negócios e necessidades muito diferentes.

O que é arquitetura de dados? Quando você para para descompactá-lo, o assunto fica um pouco obscuro; pesquisar arquitetura de dados gera muitas definições inconsistentes e muitas vezes desatualizadas. É muito parecido com quando definimos ***engenharia de dados*** no Capítulo 1 — não há consenso. Em um campo que está em constante mudança, isso é de se esperar. Então, o que queremos dizer com ***arquitetura de dados*** para os propósitos deste livro? Antes de definir o termo, é essencial entender o contexto em que ele se situa. Vamos abordar brevemente a arquitetura corporativa, que enquadrará nossa definição de arquitetura de dados.

Arquitetura Corporativa, Definida

Machine Translated by Google

A arquitetura corporativa tem muitos subconjuntos, incluindo negócios, técnicos, aplicativos e dados (Figura 3-1). Como tal, muitos frameworks e recursos são dedicados à arquitetura corporativa. Na verdade, a arquitetura é um tema surpreendentemente controverso.

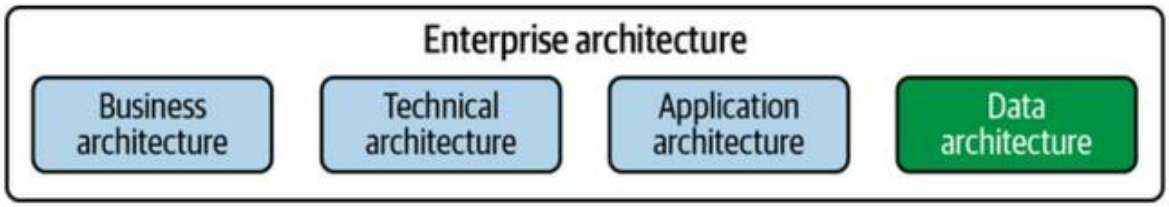


Figura 3-1. A arquitetura de dados é um subconjunto da arquitetura corporativa

O termo ***empresa*** recebe reações mistas. Isso traz à mente escritórios corporativos estéreis, planejamento de comando e controle/cascata, culturas de negócios estagnadas e frases de efeito vazias. Mesmo assim, podemos aprender algumas coisas aqui.

Antes de definirmos e descrevermos ***a arquitetura corporativa,*** vamos descompactar este termo. Vejamos como a arquitetura corporativa é definida por alguns líderes de pensamento importantes: TOGAF, Gartner e EABOK.

Definição do TOGAF

TOGAF é ***o Open Group Architecture Framework,*** um padrão do The Open Group. É apresentado como o framework de arquitetura mais usado atualmente. Aqui está a definição do TOGAF:

> ***O termo "empresa" no contexto de "arquitetura corporativa" pode denotar uma empresa inteira – abrangendo todos os seus serviços de informação e tecnologia, processos e infraestrutura – ou um domínio específico dentro da empresa. Em ambos os casos, a arquitetura cruza vários sistemas e vários grupos funcionais dentro da empresa.***

Definição do Gartner

Gartner é uma empresa global de consultoria e pesquisa que produz artigos de pesquisa e relatórios sobre tendências relacionadas a empresas. Entre outras coisas, é responsável pelo (in)famoso Gartner Hype Cycle. Definição do Gartner é o seguinte:

Machine Translated by Google

> ***A arquitetura corporativa (EA) é uma disciplina para liderar de forma proativa e holística as respostas corporativas às forças disruptivas, identificando e analisando a execução da mudança em direção à visão e aos resultados de negócios desejados. A EA agrega valor ao apresentar aos líderes de negócios e de TI recomendações prontas para o ajuste de políticas e projetos para alcançar resultados de negócios direcionados que capitalizam interrupções relevantes nos negócios.***

Definição do EABOK

EABOK é o ***Enterprise Architecture Book of Knowledge,*** uma referência de arquitetura corporativa produzida pela MITRE Corporation. EABOK foi lançado como um rascunho incompleto em 2004 e não foi atualizado desde então. Embora aparentemente obsoleto, o EABOK ainda é frequentemente referenciado em descrições de arquitetura corporativa; achamos muitas de suas ideias úteis enquanto escrevíamos este livro. Aqui está a definição EABOK:

> ***A Arquitetura Empresarial (EA) é um modelo organizacional; uma representação abstrata de uma empresa que alinha estratégia, operações e tecnologia para criar um roteiro para o sucesso.***

Nossa definição

Extraímos alguns tópicos comuns nessas definições de arquitetura corporativa: mudança, alinhamento, organização, oportunidades, solução de problemas e migração. Aqui está nossa definição de ***arquitetura corporativa,*** que consideramos mais relevante para o cenário de dados em rápida evolução de hoje:

> ***A arquitetura corporativa é o design de sistemas para suportar mudanças na empresa, alcançada por decisões flexíveis e reversíveis alcançadas por meio de uma avaliação cuidadosa de compensações.***

Aqui, abordamos algumas áreas-chave às quais retornaremos ao longo do livro: decisões flexíveis e reversíveis, gerenciamento de mudanças e avaliação de compensações. Discutimos detalhadamente cada tema nesta seção e, em seguida, tornamos a definição mais concreta na última parte do capítulo, dando vários exemplos de arquitetura de dados.

Machine Translated by Google

Decisões flexíveis e reversíveis são essenciais por duas razões. Primeiro, o mundo está em constante mudança e é impossível prever o futuro.
Decisões reversíveis permitem que você ajuste o curso à medida que o mundo muda e você reúne novas informações. Em segundo lugar, há uma tendência natural à ossificação empresarial à medida que as organizações crescem. À medida que as organizações crescem, tornam-se cada vez mais avessas ao risco e complicadas. Adotar uma cultura de decisões reversíveis ajuda a superar essa tendência, reduzindo o risco associado a uma decisão.

Jeff Bezos é creditado com a ideia de portas unidirecionais e bidirecionais. Uma[1] ***porta*** de mão única é uma decisão quase impossível de reverter. Por exemplo, a Amazon poderia ter decidido vender a AWS ou fechá-la. Seria quase impossível para a Amazon reconstruir uma nuvem pública com a mesma posição de mercado após tal ação.

Por outro lado, uma ***porta de mão dupla*** é uma decisão facilmente reversível: você entra e prossegue se gostar do que vê na sala ou recua pela porta se não gostar. A Amazon pode decidir exigir o uso do DynamoDB para um novo banco de dados de microsserviços. Se essa política não funcionar, a Amazon tem a opção de revertê-la e refatorar alguns serviços para usar outros bancos de dados. Como os riscos associados a cada decisão reversível (porta de mão dupla) são baixos, as organizações podem tomar mais decisões, iterando, melhorando e coletando dados rapidamente.

O gerenciamento de mudanças está intimamente relacionado a decisões reversíveis e é um tema central das estruturas de arquitetura corporativa. Mesmo com ênfase em decisões reversíveis, as empresas geralmente precisam empreender grandes iniciativas. Estes são idealmente divididos em mudanças menores, cada uma delas uma decisão reversível em si. Voltando à Amazon, notamos um intervalo de cinco anos (2007 a 2012) desde a publicação de um artigo sobre o conceito do DynamoDB até o anúncio de Werner Vogels do serviço DynamoDB na AWS. Nos bastidores, as equipes realizaram várias pequenas ações para tornar o DynamoDB uma realidade concreta para os clientes da AWS. Gerenciar essas pequenas ações está no centro do gerenciamento de mudanças.

Machine Translated by Google

Os arquitetos não estão simplesmente mapeando os processos de TI e olhando vagamente para um futuro distante e utópico; eles resolvem ativamente problemas de negócios e criam novas oportunidades. As soluções técnicas existem não por si mesmas, mas para apoiar os objetivos de negócios. Os arquitetos identificam problemas no estado atual (baixa qualidade de dados, limites de escalabilidade, linhas de negócios que perdem dinheiro), definem estados futuros desejados (melhoria ágil da qualidade de dados, soluções de dados em nuvem escaláveis, processos de negócios aprimorados) e realizam iniciativas por meio da execução de passos pequenos e concretos.

> ***As soluções técnicas existem não por si mesmas, mas para apoiar os objetivos de negócios.***

Encontramos inspiração significativa em ***Fundamentos de Arquitetura de Software*** por Mark Richards e Neal Ford (O'Reilly). Eles enfatizam que os trade-offs são inevitáveis e onipresentes no espaço da engenharia. Às vezes, a natureza relativamente fluida do software e dos dados nos leva a acreditar que estamos livres das restrições que os engenheiros enfrentam no mundo físico duro e frio. De fato, isso é parcialmente verdade; corrigir um bug de software é muito mais fácil do que redesenhar e substituir uma asa de avião. No entanto, os sistemas digitais são limitados por limites físicos, como latência, confiabilidade, densidade e consumo de energia. Os engenheiros também enfrentam vários limites não físicos, como características de linguagens e estruturas de programação e restrições práticas no gerenciamento de complexidade, orçamentos, etc.

O pensamento mágico culmina em uma engenharia pobre. Os engenheiros de dados devem levar em conta as compensações em cada etapa para projetar um sistema ideal, minimizando a dívida técnica com juros altos.

Vamos reiterar um ponto central em nossa definição de arquitetura corporativa: a arquitetura corporativa equilibra flexibilidade e compensações. Isso nem sempre é um equilíbrio fácil, e os arquitetos devem avaliar e reavaliar constantemente com o reconhecimento de que o mundo é dinâmico. Dado o ritmo de mudança que as empresas enfrentam, as organizações – e sua arquitetura – não podem ficar paradas.

Arquitetura de dados definida

Machine Translated by Google

Agora que você entende a arquitetura corporativa, vamos mergulhar na arquitetura de dados estabelecendo uma definição funcional que preparará o cenário para o restante do livro. ***A arquitetura*** de dados é um subconjunto da arquitetura corporativa, herdando suas propriedades: processos, estratégia, gerenciamento de mudanças e tecnologia. Aqui estão algumas definições de arquitetura de dados que influenciam nossa definição.

Definição do TOGAF

TOGAF define a arquitetura de dados da seguinte forma:

> ***Uma descrição da estrutura e interação dos principais tipos e fontes de dados da empresa, ativos de dados lógicos, ativos de dados físicos e recursos de gerenciamento de dados.***

Definição da DAMA

O DAMA ***DMBOK*** define a arquitetura de dados da seguinte forma:

> ***Identificar as necessidades de dados da empresa (independentemente da estrutura) e projetar e manter os planos mestres para atender a essas necessidades.***
> ***Usando blueprints mestres para orientar a integração de dados, controlar ativos de dados e alinhar investimentos em dados com a estratégia de negócios.***

Nossa definição

Considerando as duas definições anteriores e nossa experiência, elaboramos nossa definição de ***arquitetura de dados:***

> ***A arquitetura de dados é o design de sistemas para dar suporte às necessidades de dados em evolução de uma empresa, alcançada por decisões flexíveis e reversíveis alcançadas por meio de uma avaliação cuidadosa das compensações.***

Como a arquitetura de dados se encaixa na engenharia de dados? Assim como o ciclo de vida da engenharia de dados é um subconjunto do ciclo de vida dos dados, a arquitetura de engenharia de dados é um subconjunto da arquitetura geral de dados. ***A arquitetura de engenharia de dados*** são os sistemas e estruturas que compõem as principais seções do ciclo de vida da engenharia de dados. Usaremos ***a arquitetura de dados*** de forma intercambiável com ***a arquitetura de engenharia de dados*** ao longo deste livro.

Machine Translated by Google

Outros aspectos da arquitetura de dados que você deve conhecer são operacionais e técnicos (Figura 3-2). ***A arquitetura operacional*** engloba os requisitos funcionais do que precisa acontecer em relação a pessoas, processos e tecnologia. Por exemplo, a quais processos de negócios os dados atendem? Como a organização gerencia a qualidade dos dados? Qual é o requisito de latência desde quando os dados são produzidos até quando ficam disponíveis para consulta? ***A arquitetura técnica*** descreve como os dados são ingeridos, armazenados, transformados e servidos ao longo do ciclo de vida da engenharia de dados. Por exemplo, como você moverá 10 TB de dados a cada hora de um banco de dados de origem para seu data lake? Em resumo, a arquitetura operacional descreve ***o que*** precisa ser feito e a arquitetura técnica detalha ***como*** isso acontecerá.

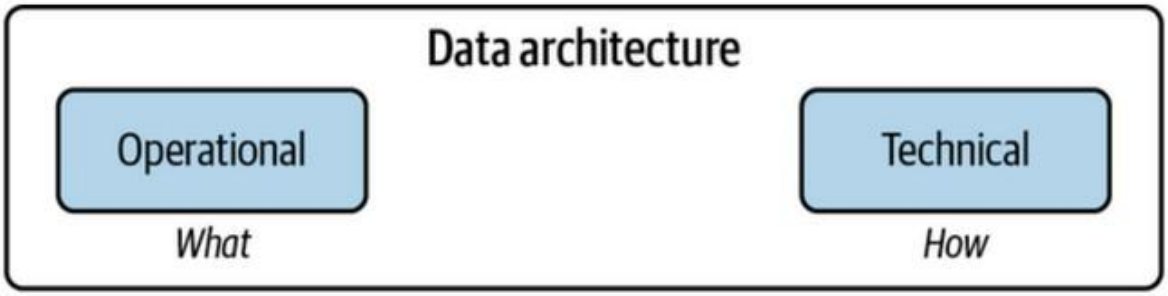


Figura 3-2. Arquitetura de dados operacionais e técnicos

Agora que temos uma definição funcional de arquitetura de dados, vamos abordar os elementos de uma “boa” arquitetura de dados.

Arquitetura de dados “boa”

> ***Nunca atire na melhor arquitetura, mas sim na menos pior arquitetura.***
>
> —Neal Ford, Mark Richards

Segundo Grady Booch, “A arquitetura representa as decisões significativas de design que moldam um sistema, onde o ***significativo*** é medido pelo custo da mudança.” Os arquitetos de dados visam tomar decisões significativas que levarão a uma boa arquitetura em um nível básico.

Machine Translated by Google

O que queremos dizer com “boa” arquitetura de dados? Parafraseando um velho clichê, você sabe bem quando o vê. Uma ***boa arquitetura de dados*** atende aos requisitos de negócios com um conjunto comum e amplamente reutilizável de blocos de construção, mantendo a flexibilidade e fazendo as compensações apropriadas. A arquitetura ruim é autoritária e tenta enfiar um monte de decisões de tamanho único em uma grande bola de lama.

A agilidade é a base para uma boa arquitetura de dados; reconhece que o mundo é fluido. ***Uma boa arquitetura de dados é flexível e de fácil manutenção.*** Ele evolui em resposta a mudanças dentro do negócio e novas tecnologias e práticas que podem liberar ainda mais valor no futuro.
As empresas e seus casos de uso de dados estão sempre evoluindo. O mundo é dinâmico e o ritmo das mudanças no espaço de dados está se acelerando. A arquitetura de dados do ano passado que lhe serviu bem pode não ser suficiente para hoje, muito menos para o próximo ano.

A arquitetura de dados ruim é fortemente acoplada, rígida, excessivamente centralizada ou usa as ferramentas erradas para o trabalho, dificultando o desenvolvimento e o gerenciamento de mudanças. Idealmente, ao projetar a arquitetura com a reversibilidade em mente, as mudanças serão menos dispendiosas.

As correntes ocultas do ciclo de vida da engenharia de dados formam a base de uma boa arquitetura de dados para empresas em qualquer estágio de maturidade de dados. Novamente, essas subcorrentes são segurança, gerenciamento de dados, DataOps, arquitetura de dados, orquestração e engenharia de software.

Uma boa arquitetura de dados é uma coisa viva que respira. Nunca está terminado. De fato, de acordo com nossa definição, mudança e evolução são centrais para o significado e propósito da arquitetura de dados. Vejamos agora os princípios de uma boa arquitetura de dados.

Princípios da Boa Arquitetura de Dados

Esta seção tem uma visão de 10.000 pés de boa arquitetura, concentrando-se em princípios – ideias-chave úteis para avaliar as principais decisões e práticas arquitetônicas. Tomamos emprestada inspiração para nossos princípios de arquitetura de vários

Machine Translated by Google

fontes, especialmente o AWS Well-Architected Framework e os cinco princípios do Google Cloud para arquitetura nativa em nuvem.

A estrutura bem arquitetada da AWS é composto por seis pilares:

- Excelência operacional
- Segurança
- Confiabilidade
- Eficiência de desempenho
- Otimização de custos
- Sustentabilidade

Cinco princípios do Google Cloud para arquitetura nativa da nuvem são como segue:

- Projeto para automação.
- Seja esperto com o estado.
- Favorecer os serviços gerenciados.
- Pratique a defesa em profundidade.
- Estar sempre arquitetando.

Aconselhamos que você estude cuidadosamente ambas as estruturas, identifique ideias valiosas e determine pontos de desacordo. Gostaríamos de expandir ou elaborar esses pilares com estes princípios da arquitetura de engenharia de dados:

1. Escolha componentes comuns com sabedoria.
2. Planeje o fracasso.
3. Arquiteto para escalabilidade.
4. Arquitetura é liderança.
5. Esteja sempre arquitetando.

Machine Translated by Google

6. Construir sistemas fracamente acoplados.
7. Tome decisões reversíveis.
8. Priorize a segurança.
9. Abrace FinOps.

Princípio 1: Escolha os componentes comuns com sabedoria

Um dos principais trabalhos de um engenheiro de dados é escolher componentes e práticas comuns que possam ser amplamente usados em uma organização. Quando os arquitetos escolhem bem e lideram com eficiência, os componentes comuns tornam-se um tecido que facilita a colaboração da equipe e elimina os silos. Componentes comuns permitem agilidade dentro e entre equipes em conjunto com conhecimentos e habilidades compartilhados.

Componentes comuns podem ser qualquer coisa que tenha ampla aplicabilidade dentro de uma organização. Os componentes comuns incluem armazenamento de objetos, sistemas de controle de versão, observabilidade, sistemas de monitoramento e orquestração e mecanismos de processamento. Componentes comuns devem ser acessíveis a todos com um caso de uso apropriado, e as equipes são incentivadas a confiar em componentes comuns já em uso, em vez de reinventar a roda. Os componentes comuns devem oferecer suporte a permissões e segurança robustas para permitir o compartilhamento de ativos entre as equipes, evitando o acesso não autorizado.

As plataformas em nuvem são o local ideal para adotar componentes comuns. Por exemplo, a separação de computação e armazenamento em sistemas de dados em nuvem permite que os usuários acessem uma camada de armazenamento compartilhado (mais comumente armazenamento de objetos) usando ferramentas especializadas para acessar e consultar os dados necessários para casos de uso específicos.

Escolher componentes comuns é um ato de equilíbrio. Por um lado, você precisa se concentrar nas necessidades em todo o ciclo de vida e nas equipes de engenharia de dados, utilizar componentes comuns que serão úteis para projetos individuais e, simultaneamente, facilitar a interoperação e a colaboração. Por outro lado, os arquitetos devem evitar decisões que prejudiquem a produtividade dos engenheiros que trabalham em problemas específicos de domínio, forçando-os a soluções tecnológicas de tamanho único. O Capítulo 4 fornece detalhes adicionais.

Machine Translated by Google

Princípio 2: Plano para Falha

Tudo falha, o tempo todo.

—Werner Vogels

O hardware moderno é altamente robusto e durável. Mesmo assim, qualquer componente de hardware falhará, com tempo suficiente. Para construir sistemas de dados altamente robustos, você deve considerar falhas em seus projetos. Aqui estão alguns termos-chave para avaliar cenários de falha; descrevemos isso com mais detalhes neste capítulo e em todo o livro:

Disponibilidade

A porcentagem de tempo que um serviço ou componente de TI está em operação Estado.

Confiabilidade

A probabilidade do sistema de atender a padrões definidos na execução de sua função pretendida durante um intervalo especificado.

Objetivo de tempo de recuperação

O tempo máximo aceitável para uma interrupção de serviço ou sistema. O objetivo de tempo de recuperação (RTO) geralmente é definido determinando o impacto comercial de uma interrupção. Um RTO de um dia pode ser bom para um sistema de relatórios internos. Uma interrupção do site de apenas cinco minutos pode ter um impacto negativo significativo nos negócios de um varejista on-line.

Objetivo do ponto de recuperação

O estado aceitável após a recuperação. Em sistemas de dados, os dados geralmente são perdidos durante uma interrupção. Nessa configuração, o objetivo do ponto de recuperação (RPO) refere-se à perda de dados máxima aceitável.

Machine Translated by Google

Os engenheiros precisam considerar confiabilidade, disponibilidade, RTO e RPO aceitáveis ao projetar para falhas. Eles guiarão suas decisões de arquitetura à medida que avaliam possíveis cenários de falha.

Princípio 3: Arquiteto para Escalabilidade

A escalabilidade em sistemas de dados engloba dois recursos principais. Primeiro, os sistemas escaláveis podem ser ***dimensionados*** para lidar com quantidades significativas de dados. Talvez seja necessário ativar um cluster grande para treinar um modelo em um petabyte de dados do cliente ou dimensionar um sistema de ingestão de streaming para lidar com um pico de carga transitório. Nossa capacidade de expansão nos permite lidar com cargas extremas temporariamente. Em segundo lugar, os sistemas escaláveis podem ser ***reduzidos.*** Uma vez que o pico de carga diminua, devemos remover automaticamente a capacidade para cortar custos. (Isso está relacionado ao princípio 9.) Um ***sistema elástico*** pode ser dimensionado dinamicamente em resposta à carga, idealmente de forma automatizada.

Alguns sistemas escaláveis também podem ser ***dimensionados para zero:*** eles desligam completamente quando não estão em uso. Depois que o trabalho de treinamento de modelo grande for concluído, podemos excluir o cluster. Muitos sistemas sem servidor (por exemplo, funções sem servidor e processamento analítico online sem servidor, ou OLAP, bancos de dados) podem ser dimensionados automaticamente para zero.

Observe que a implantação de estratégias de dimensionamento inadequadas pode resultar em sistemas supercomplicados e altos custos. Um banco de dados relacional direto com um nó de failover pode ser apropriado para um aplicativo em vez de um arranjo de cluster complexo. Meça sua carga atual, picos de carga aproximados e estime a carga nos próximos anos para determinar se sua arquitetura de banco de dados é apropriada. Se sua startup cresce muito mais rápido do que o previsto, esse crescimento também deve levar a mais recursos disponíveis para rearquitetar a escalabilidade.

Princípio 4: Arquitetura é Liderança

Os arquitetos de dados são responsáveis por decisões de tecnologia e descrições de arquitetura e pela divulgação dessas escolhas por meio de liderança e treinamento eficazes. Arquitetos de dados devem ser altamente competentes tecnicamente, mas

Machine Translated by Google

delegar a maior parte do trabalho individual dos colaboradores a outros. Fortes habilidades de liderança combinadas com alta competência técnica são raras e extremamente valiosas. Os melhores arquitetos de dados levam essa dualidade a sério.

Observe que a liderança não implica uma abordagem de comando e controle para a tecnologia. Não era incomum no passado os arquitetos escolherem uma tecnologia proprietária de banco de dados e forçar cada equipe a armazenar seus dados lá. Opomo-nos a esta abordagem porque pode prejudicar significativamente os atuais projetos de dados. Os ambientes de nuvem permitem que os arquitetos equilibrem escolhas de componentes comuns com flexibilidade que permite inovação nos projetos.

Voltando à noção de liderança técnica, Martin Fowler descreve um arquétipo específico de um arquiteto de software ideal, bem incorporado em seu colega Dave Rice:

> ***De muitas maneiras, a atividade mais importante do Architectus Oryzus é orientar a equipe de desenvolvimento, elevar seu nível para que possam lidar com questões mais complexas. Melhorar a capacidade da equipe de desenvolvimento dá a um arquiteto uma alavancagem muito maior do que ser o único tomador de decisões e, portanto, correr o risco de ser um gargalo de arquitetura.***[2]

Um arquiteto de dados ideal manifesta características semelhantes. Eles possuem as habilidades técnicas de um engenheiro de dados, mas não praticam mais a engenharia de dados no dia a dia; eles orientam os engenheiros de dados atuais, fazem escolhas tecnológicas cuidadosas em consulta com sua organização e disseminam conhecimentos por meio de treinamento e liderança. Eles treinam engenheiros nas melhores práticas e reúnem os recursos de engenharia da empresa para buscar objetivos comuns em tecnologia e negócios.

Como engenheiro de dados, você deve praticar a liderança em arquitetura e buscar orientação de arquitetos. Eventualmente, você pode muito bem ocupar o papel de arquiteto.

Princípio 5: Sempre esteja arquitetando

Pegamos emprestado esse princípio diretamente dos Cinco princípios para arquitetura nativa da nuvem do Google Cloud. Arquitetos de dados não servem em seu papel simplesmente

Machine Translated by Google

manter o estado existente; em vez disso, eles constantemente projetam coisas novas e empolgantes em resposta às mudanças nos negócios e na tecnologia. De acordo com o EABOK, o trabalho de um arquiteto é desenvolver um conhecimento profundo da ***arquitetura de linha*** de base (estado atual), desenvolver uma ***arquitetura de destino*** e mapear um ***plano de sequenciamento*** para determinar as prioridades e a ordem das mudanças na arquitetura.

Acrescentamos que a arquitetura moderna não deve ser de comando e controle ou cascata, mas colaborativa e ágil. O arquiteto de dados mantém uma arquitetura de destino e planos de sequenciamento que mudam com o tempo. A arquitetura alvo torna-se um alvo móvel, ajustado em resposta às mudanças de negócios e tecnologia interna e mundial. O plano de sequenciamento determina prioridades imediatas para entrega.

Princípio 6: Construir sistemas fracamente acoplados

> ***Quando a arquitetura do sistema é projetada para permitir que as equipes testem, implantem e alterem sistemas sem depender de outras equipes, as equipes exigem pouca comunicação para realizar o trabalho. Em outras palavras, tanto a arquitetura quanto as equipes são fracamente acopladas.***
>
> —Guia de arquitetura de tecnologia do Google DevOps

Em 2002, Bezos escreveu um e-mail para os funcionários da Amazon que ficou conhecido como 3 the Bezos API Mandate:

1. Todas as equipes, a partir de agora, exporão seus dados e funcionalidades por meio de interfaces de serviço.
2. As equipes devem se comunicar por meio dessas interfaces.
3. Não será permitida nenhuma outra forma de comunicação entre processos: nenhuma ligação direta, nenhuma leitura direta do armazenamento de dados de outra equipe, nenhum modelo de memória compartilhada, nenhum backdoor. A única comunicação permitida é por meio de chamadas de interface de serviço pela rede.
4. Não importa qual tecnologia eles usam. HTTP, Corba, Pubsub, protocolos personalizados — não importa.

Machine Translated by Google

5. Todas as interfaces de serviço, sem exceção, devem ser projetadas desde o início para serem externalizáveis. Ou seja, a equipe deve planejar e projetar para poder expor a interface aos desenvolvedores do mundo exterior. Sem exceções.

O advento do Mandato API de Bezos é amplamente visto como um divisor de águas para a Amazon. Colocar dados e serviços atrás de APIs permitiu o baixo acoplamento e acabou resultando na AWS como a conhecemos agora.
A busca do Google por baixo acoplamento permitiu que ele aumentasse seus sistemas em uma escala extraordinária.

Para arquitetura de software, um sistema fracamente acoplado tem as seguintes propriedades:

1. Os sistemas são divididos em muitos componentes pequenos.
2. Esses sistemas fazem interface com outros serviços por meio de camadas de abstração, como um barramento de mensagens ou uma API. Essas camadas de abstração ocultam e protegem detalhes internos do serviço, como um back-end de banco de dados ou classes internas e chamadas de método.
3. Como consequência da propriedade 2, mudanças internas em um sistema componente não requerem alterações em outras partes. Os detalhes das atualizações de código estão ocultos por trás de APIs estáveis. Cada peça pode evoluir e melhorar separadamente.
4. Como consequência da propriedade 3, não há ciclo de lançamento global em cascata para todo o sistema. Em vez disso, cada componente é atualizado separadamente à medida que são feitas alterações e melhorias.

Observe que estamos falando ***de sistemas técnicos.*** Precisamos pensar maior.
Vamos traduzir essas características técnicas em características organizacionais:

1. Muitas equipes pequenas projetam um sistema grande e complexo. Cada equipe é encarregada de projetar, manter e melhorar alguns componentes do sistema.

Machine Translated by Google

2. Essas equipes publicam os detalhes abstratos de seus componentes para outras equipes por meio de definições de API, esquemas de mensagens etc. As equipes não precisam se preocupar com os componentes de outras equipes; eles simplesmente usam a API publicada ou as especificações de mensagem para chamar esses componentes. Eles iteram sua parte para melhorar seu desempenho e recursos ao longo do tempo. Eles também podem publicar novos recursos à medida que são adicionados ou solicitar novos itens de outras equipes. Novamente, o último acontece sem que as equipes precisem se preocupar com os detalhes técnicos internos dos recursos solicitados. As equipes trabalham juntas por meio ***de comunicação fracamente acoplada.***

3. Como consequência da característica 2, cada equipe pode evoluir rapidamente e melhorar seu componente independentemente do trabalho de outras equipes.

4. Especificamente, a característica 3 implica que as equipes podem lançar atualizações para seus componentes com o mínimo de tempo de inatividade. As equipes lançam continuamente durante o horário normal de trabalho para fazer alterações no código e testá-las.

O baixo acoplamento entre tecnologia e sistemas humanos permitirá que suas equipes de engenharia de dados colaborem com mais eficiência entre si e com outras partes da empresa. Este princípio também facilita diretamente o princípio 7.

Princípio 7: Tome Decisões Reversíveis

O cenário de dados está mudando rapidamente. A tecnologia ou pilha quente de hoje é a reflexão tardia de amanhã. A opinião popular muda rapidamente. Você deve buscar decisões reversíveis, pois elas tendem a simplificar sua arquitetura e mantê-la ágil.

Como Fowler escreveu: “Uma das tarefas mais importantes de um arquiteto é remover a arquitetura encontrando maneiras de eliminar a irreversibilidade nos projetos de software”. O que era verdade quando Fowler escreveu isso em 2003 é tão preciso hoje.[4]

Como dissemos anteriormente, Bezos se refere a decisões reversíveis como “portas de mão dupla”. Como ele diz: “Se você anda e não gosta do que vê no

Machine Translated by Google

outro lado, você não pode voltar para antes. Podemos chamar essas decisões de Tipo 1. Mas a maioria das decisões não é assim – elas são mutáveis, reversíveis – são portas de mão dupla.” Aponte para portas de duas vias sempre que possível.

Dado o ritmo da mudança – e a dissociação/modularização de tecnologias em sua arquitetura de dados – sempre se esforce para escolher as melhores soluções de classe que funcionem para hoje. Além disso, esteja preparado para atualizar ou adotar melhores práticas à medida que a paisagem evolui.

Princípio 8: Priorize a Segurança

Todo engenheiro de dados deve assumir a responsabilidade pela segurança dos sistemas que constrói e mantém. Concentramo-nos agora em duas ideias principais: segurança de confiança zero e o modelo de segurança de responsabilidade compartilhada. Eles se alinham de perto a uma arquitetura nativa da nuvem.

Modelos de segurança de perímetro reforçado e confiança zero

Para definir ***a segurança de confiança zero,*** é útil começar entendendo o modelo tradicional de segurança de perímetro rígido e suas limitações, conforme detalhado em Os cinco princípios do Google Cloud:

> ***As arquiteturas tradicionais depositam muita fé na segurança do perímetro, grosseiramente um perímetro de rede reforçado com “coisas confiáveis” dentro e “coisas não confiáveis” fora. Infelizmente, essa abordagem sempre foi vulnerável a ataques internos, bem como a ameaças externas, como spear phishing.***

O filme ***Missão Impossível*** de 1996 apresenta um exemplo perfeito do modelo de segurança de perímetro rígido e suas limitações. No filme, a CIA hospeda dados altamente confidenciais em um sistema de armazenamento dentro de uma sala com segurança física extremamente rígida. Ethan Hunt se infiltra na sede da CIA e explora um alvo humano para obter acesso físico ao sistema de armazenamento. Uma vez dentro da sala segura, ele pode exfiltrar dados com relativa facilidade.

Por pelo menos uma década, reportagens alarmantes da mídia nos alertaram sobre a crescente ameaça de violações de segurança que exploram alvos humanos dentro de perímetros de segurança organizacional reforçados. Mesmo que os funcionários trabalhem

Machine Translated by Google

redes corporativas altamente seguras, eles permanecem conectados ao mundo exterior por meio de e-mail e dispositivos móveis. As ameaças externas tornam-se efetivamente ameaças internas.

Em um ambiente nativo da nuvem, a noção de um perímetro reforçado se desgasta ainda mais. Todos os ativos estão conectados ao mundo exterior em algum grau. Embora as redes de nuvem privada virtual (VPC) possam ser definidas sem conectividade externa, o plano de controle da API que os engenheiros usam para definir essas redes ainda está voltado para a Internet.

O modelo de responsabilidade compartilhada

A Amazon enfatiza o modelo de responsabilidade compartilhada, que divide a segurança em segurança ***da*** nuvem e segurança ***na*** nuvem. A AWS é responsável pela segurança da nuvem:

> ***A AWS é responsável por proteger a infraestrutura que executa os serviços da AWS na Nuvem AWS. A AWS também fornece serviços que você pode usar com segurança.***

Os usuários da AWS são responsáveis pela segurança na nuvem:

> ***Sua responsabilidade é determinada pelo serviço da AWS que você usa. Você também é responsável por outros fatores, incluindo a confidencialidade de seus dados, os requisitos de sua organização e as leis e regulamentos aplicáveis.***

Em geral, todos os provedores de nuvem operam nesse modelo de responsabilidade compartilhada. Eles protegem seus serviços de acordo com as especificações publicadas. Ainda assim, em última análise, é responsabilidade do usuário projetar um modelo de segurança para seus aplicativos e dados e aproveitar os recursos da nuvem para realizar esse modelo.

Engenheiros de dados como engenheiros de segurança

No mundo corporativo de hoje, uma abordagem de comando e controle para a segurança é bastante comum, na qual as equipes de segurança e rede gerenciam perímetros e práticas gerais de segurança. A nuvem transfere essa responsabilidade para engenheiros que não estão explicitamente em funções de segurança.
Devido a esta responsabilidade, em conjunto com a erosão mais geral da

Machine Translated by Google

no perímetro de segurança rígido, todos os engenheiros de dados devem se considerar engenheiros de segurança.

A falha em assumir essas novas responsabilidades implícitas pode levar a consequências terríveis. Várias violações de dados resultaram do simples erro de configurar buckets do Amazon S3 com acesso público. Aqueles que lidam com dados devem assumir que são os responsáveis por protegê-los.

Princípio 9: Abrace FinOps

Vamos começar considerando algumas definições de FinOps. Primeiro, a Fundação FinOps oferece isso:

> ***FinOps é uma prática cultural e disciplina de gerenciamento financeiro em nuvem em evolução que permite que as organizações obtenham o máximo valor comercial, ajudando as equipes de engenharia, finanças, tecnologia e negócios a colaborar em decisões de gastos orientadas por dados.***

Além disso, JR Sorment e Mike Fuller fornecem a seguinte definição no ***Cloud FinOps*** (O'Reilly):

> ***O termo "FinOps" normalmente se refere ao movimento profissional emergente que defende uma relação de trabalho colaborativa entre DevOps e Finanças, resultando em um gerenciamento iterativo e orientado por dados dos gastos com infraestrutura (ou seja, reduzindo a economia unitária da nuvem) ao mesmo tempo em que aumenta o custo eficiência e, em última análise, a lucratividade do ambiente de nuvem.***

A estrutura de custos dos dados evoluiu dramaticamente durante a era da nuvem. Em um ambiente local, os sistemas de dados geralmente são adquiridos com um gasto de capital (descrito mais no Capítulo 4) para um novo sistema a cada poucos anos em um ambiente local. As partes responsáveis precisam equilibrar seu orçamento em relação à capacidade desejada de computação e armazenamento. Comprar em excesso significa desperdício de dinheiro, enquanto que comprar menos significa prejudicar futuros projetos de dados e direcionar um tempo significativo do pessoal para controlar a carga do sistema e o tamanho dos dados; a subcompra pode exigir ciclos de atualização de tecnologia mais rápidos, com custos extras associados.

Machine Translated by Google

Na era da nuvem, a maioria dos sistemas de dados é paga conforme o uso e prontamente escalável. Os sistemas podem ser executados em um modelo de custo por consulta, modelo de custo por capacidade de processamento ou outra variante de um modelo de pagamento conforme o uso. Essa abordagem pode ser muito mais eficiente do que a abordagem de despesas de capital. Agora é possível expandir para alto desempenho e, em seguida, reduzir para economizar dinheiro. No entanto, a abordagem de pagamento conforme o uso torna os gastos muito mais dinâmicos. O novo desafio para os líderes de dados é gerenciar orçamentos, prioridades e eficiência.

As ferramentas de nuvem precisam de um conjunto de processos para gerenciar gastos e recursos. No passado, os engenheiros de dados pensavam em termos de engenharia de desempenho – maximizando o desempenho dos processos de dados em um conjunto fixo de recursos e comprando recursos adequados para necessidades futuras. Com o FinOps, os engenheiros precisam aprender a pensar nas estruturas de custo dos sistemas em nuvem.
Por exemplo, qual é a combinação apropriada de instâncias spot da AWS ao executar um cluster distribuído? Qual é a abordagem mais adequada para executar um trabalho diário considerável em termos de custo-benefício e desempenho?
Quando a empresa deve mudar de um modelo de pagamento por consulta para capacidade reservada?

FinOps evolui o modelo de monitoramento operacional para monitorar os gastos de forma contínua. Em vez de simplesmente monitorar as solicitações e a utilização da CPU para um servidor web, o FinOps pode monitorar o custo contínuo das funções sem servidor que lidam com o tráfego, bem como os picos nos alertas de gatilho de gastos. Assim como os sistemas são projetados para falhar graciosamente em tráfego excessivo, as empresas podem considerar a adoção de limites rígidos para gastos, com modos de falha graciosos em resposta a picos de gastos.

As equipes de operações também devem pensar em termos de ataques de custo. Assim como um ataque distribuído de negação de serviço (DDoS) pode bloquear o acesso a um servidor web, muitas empresas descobriram, para seu desgosto, que downloads excessivos de buckets do S3 podem aumentar os gastos e ameaçar uma pequena startup com falência. Ao compartilhar dados publicamente, as equipes de dados podem resolver esses problemas definindo políticas de pagamento do solicitante ou simplesmente monitorando gastos excessivos com acesso a dados e removendo rapidamente o acesso se os gastos começarem a atingir níveis inaceitáveis.

Machine Translated by Google

No momento da redação deste artigo, FinOps é uma prática recentemente formalizada. A Fundação FinOps foi iniciada apenas em 2019.[6] No entanto, é altamente recomendável que você comece a pensar no FinOps com antecedência, antes de encontrar altas contas de nuvem. Comece sua jornada com a FinOps Foundation e o ***Cloud FinOps da O'Reilly.*** Também sugerimos que os engenheiros de dados se envolvam no processo comunitário de criação de práticas de FinOps para engenharia de dados — em uma área de prática tão nova, uma boa parte do território ainda precisa ser mapeada.

Agora que você tem uma compreensão de alto nível dos bons princípios de arquitetura de dados, vamos nos aprofundar um pouco mais nos principais conceitos necessários para projetar e construir uma boa arquitetura de dados.

Principais Conceitos de Arquitetura

Se você seguir as tendências atuais em dados, parece que novos tipos de ferramentas e arquiteturas de dados estão chegando em cena a cada semana. Em meio a esse turbilhão de atividades, não devemos perder de vista o objetivo principal de todas essas arquiteturas: pegar dados e transformá-los em algo útil para consumo downstream.

Domínios e Serviços

> ***Uma esfera de conhecimento, influência ou atividade. A área de assunto à qual o usuário aplica um programa é o domínio do software.***
>
> —Eric Evans

Antes de mergulhar nos componentes da arquitetura, vamos abordar brevemente dois termos que você verá com muita frequência: domínio e serviços. Um ***domínio*** é a área de assunto do mundo real para a qual você está arquitetando. Um ***serviço*** é um conjunto de funcionalidades cujo objetivo é realizar uma tarefa. Por exemplo, você pode ter um serviço de processamento de pedidos de vendas cuja tarefa é processar pedidos à medida que são criados. O único trabalho do serviço de processamento de pedidos de vendas é processar pedidos; ele não fornece outras funcionalidades, como gerenciamento de inventário ou atualização de perfis de usuário.

Machine Translated by Google

Um domínio pode conter vários serviços. Por exemplo, você pode ter um domínio de vendas com três serviços: pedidos, faturamento e produtos. Cada serviço possui tarefas específicas que suportam o domínio de vendas. Outros domínios também podem compartilhar serviços (Figura 3-3). Nesse caso, o domínio contábil é responsável pelas funções contábeis básicas: faturamento, folha de pagamento e contas a receber (AR). Observe que o domínio de contabilidade compartilha o serviço de fatura com o domínio de vendas, pois uma venda gera uma fatura e a contabilidade deve acompanhar as faturas para garantir que o pagamento seja recebido. Vendas e contabilidade possuem seus respectivos domínios.

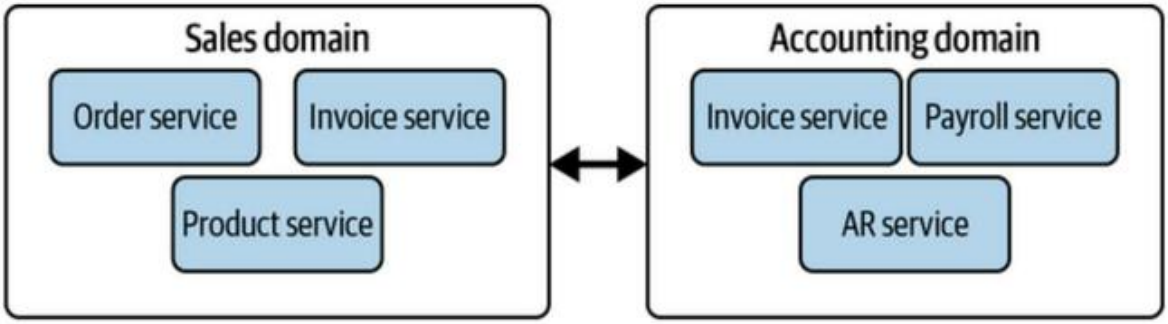


Figura 3-3. Dois domínios (vendas e contabilidade) compartilham um serviço comum (faturas), e vendas e contabilidade possuem seus respectivos domínios

Ao pensar sobre o que constitui um domínio, concentre-se no que o domínio representa no mundo real e trabalhe para trás. No exemplo anterior, o domínio de vendas deve representar o que acontece com a função de vendas em sua empresa. Ao arquitetar o domínio de vendas, evite copiar e colar do que outras empresas fazem. A função de vendas da sua empresa provavelmente tem aspectos únicos que exigem serviços específicos para que funcione da maneira que sua equipe de vendas espera.

Identifique o que deve ir no domínio. Ao determinar o que o domínio deve abranger e quais serviços incluir, o melhor conselho é simplesmente conversar com usuários e partes interessadas, ouvir o que eles estão dizendo e criar os serviços que os ajudarão a fazer seu trabalho. Evite a armadilha clássica de arquitetar no vácuo.

Sistemas distribuídos, escalabilidade e design para Falha

Machine Translated by Google

A discussão nesta seção está relacionada ao nosso segundo e terceiro princípios de arquitetura de engenharia de dados discutidos anteriormente: planejar para falhas e arquiteto para escalabilidade. Como engenheiros de dados, estamos interessados em quatro características intimamente relacionadas de sistemas de dados (disponibilidade e confiabilidade foram mencionadas anteriormente, mas as reiteramos aqui para completar):

Escalabilidade

Permite-nos aumentar a capacidade de um sistema para melhorar o desempenho e atender a demanda. Por exemplo, podemos querer dimensionar um sistema para lidar com uma alta taxa de consultas ou processar um grande conjunto de dados.

Elasticidade

A capacidade de um sistema escalável de escalar dinamicamente; um sistema altamente elástico pode aumentar e diminuir automaticamente com base na carga de trabalho atual. A ampliação é fundamental à medida que a demanda aumenta, enquanto a escalabilidade down economiza dinheiro em um ambiente de nuvem. Os sistemas modernos às vezes escala para zero, o que significa que eles podem desligar automaticamente quando ociosos.

Disponibilidade

A porcentagem de tempo que um serviço ou componente de TI está em operação Estado.

Confiabilidade

A probabilidade do sistema de atender aos padrões definidos na execução de seu função pretendida durante um intervalo especificado.

GORJETA

Consulte “Por que a disponibilidade e a confiabilidade são cruciais?” do PagerDuty? página da Internet para definições e antecedentes sobre disponibilidade e confiabilidade.

Machine Translated by Google

Como essas características estão relacionadas? Se um sistema não atender aos requisitos de desempenho durante um intervalo especificado, ele poderá parar de responder. Assim, a baixa confiabilidade pode levar à baixa disponibilidade. Por outro lado, o dimensionamento dinâmico ajuda a garantir o desempenho adequado sem intervenção manual dos engenheiros — a elasticidade melhora a confiabilidade.

A escalabilidade pode ser realizada de várias maneiras. Para seus serviços e domínios, uma única máquina lida com tudo? Uma única máquina pode ser dimensionada verticalmente; você pode aumentar os recursos (CPU, disco, memória, E/S).
Mas há limites rígidos para possíveis recursos em uma única máquina. Além disso, o que acontece se esta máquina morrer? Com tempo suficiente, alguns componentes eventualmente falharão. Qual é o seu plano para backup e failover? Máquinas únicas geralmente não podem oferecer alta disponibilidade e confiabilidade.

Utilizamos um sistema distribuído para obter maior capacidade de dimensionamento geral e maior disponibilidade e confiabilidade. ***A escala horizontal*** permite adicionar mais máquinas para satisfazer os requisitos de carga e recursos (Figura 3-4).
Sistemas comuns escalados horizontalmente têm um nó líder que atua como o principal ponto de contato para a instanciação, progresso e conclusão das cargas de trabalho. Quando uma carga de trabalho é iniciada, o nó líder distribui tarefas para os nós do trabalhador em seu sistema, concluindo as tarefas e retornando os resultados ao nó líder. Arquiteturas distribuídas modernas típicas também são construídas em redundância. Os dados são replicados para que, se uma máquina morrer, as outras máquinas possam continuar de onde o servidor ausente parou; o cluster pode adicionar mais máquinas para restaurar a capacidade.

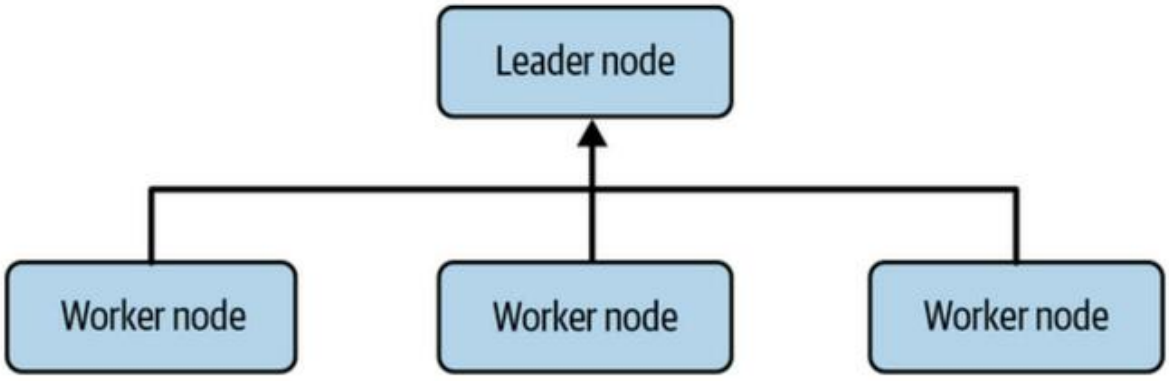


Figura 3-4. Um sistema distribuído horizontal simples utilizando uma arquitetura líder-seguidor, com um nó líder e três nós de trabalho

Machine Translated by Google

Os sistemas distribuídos são difundidos nas várias tecnologias de dados que você usará em sua arquitetura. Quase todo sistema de armazenamento de objetos de data warehouse em nuvem que você usa tem alguma noção de distribuição sob o capô.
Os detalhes de gerenciamento do sistema distribuído normalmente são abstraídos, permitindo que você se concentre na arquitetura de alto nível em vez de no encanamento de baixo nível. No entanto, é altamente recomendável que você aprenda mais sobre sistemas distribuídos, pois esses detalhes podem ser extremamente úteis para entender e melhorar o desempenho de seus pipelines; Projeto de aplicativos de ***dados intensivos de*** Martin Kleppmann (O'Reilly) é um excelente recurso.

Acoplamento apertado versus solto: camadas, monólitos e Microsserviços

Ao projetar uma arquitetura de dados, você escolhe quanta interdependência deseja incluir em seus vários domínios, serviços e recursos.
Em uma extremidade do espectro, você pode optar por ter dependências e fluxos de trabalho extremamente centralizados. Cada parte de um domínio e serviço é vitalmente dependente de todos os outros domínios e serviços. Esse padrão é conhecido como ***fortemente acoplado.***

Na outra ponta do espectro, você tem domínios e serviços descentralizados que não têm dependência estrita uns dos outros, em um padrão conhecido como ***acoplamento fraco.*** Em um cenário fracamente acoplado, é fácil para equipes descentralizadas criarem sistemas cujos dados podem não ser utilizáveis por seus pares. Certifique-se de atribuir padrões comuns, propriedade, responsabilidade e prestação de contas às equipes que possuem seus respectivos domínios e serviços.
Projetar uma arquitetura de dados “boa” depende de compensações entre o acoplamento forte e fraco de domínios e serviços.

Vale a pena notar que muitas das ideias nesta seção se originam no desenvolvimento de software. Tentaremos manter o contexto da intenção e do espírito originais dessas grandes ideias – mantendo-as agnósticas em relação aos dados – enquanto explicamos posteriormente algumas diferenças das quais você deve estar ciente ao aplicar esses conceitos aos dados especificamente.

Machine Translated by Google

Camadas de arquitetura

À medida que você desenvolve sua arquitetura, é útil estar ciente das camadas da arquitetura. Sua arquitetura tem camadas – dados, aplicativos, lógica de negócios, apresentação e assim por diante – e você precisa saber como desacoplar essas camadas. Como o acoplamento rígido de modalidades apresenta vulnerabilidades óbvias, lembre-se de como você estrutura as camadas de sua arquitetura para obter o máximo de confiabilidade e flexibilidade. Vejamos a arquitetura de camada única e multicamada.

Camada única

Em uma ***arquitetura de camada única,*** seu banco de dados e aplicativo são fortemente acoplados, residindo em um único servidor (Figura 3-5). Esse servidor pode ser seu laptop ou uma única máquina virtual (VM) na nuvem. A natureza fortemente acoplada significa que se o servidor, o banco de dados ou o aplicativo falhar, toda a arquitetura falhará. Embora as arquiteturas de camada única sejam boas para prototipagem e desenvolvimento, elas não são recomendadas para ambientes de produção devido aos riscos óbvios de falha.

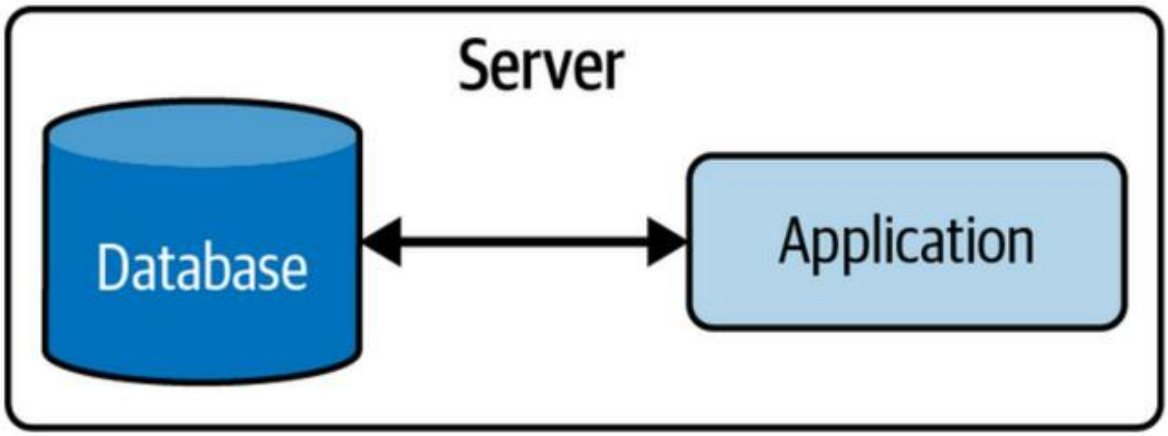


Figura 3-5. Arquitetura de camada única

Mesmo quando arquiteturas de camada única são construídas em redundância (por exemplo, uma réplica de failover), elas apresentam limitações significativas de outras maneiras. Por exemplo, muitas vezes é impraticável (e não aconselhável) executar consultas analíticas em bancos de dados de aplicativos de produção. Fazer isso corre o risco de sobrecarregar o banco de dados e fazer com que o aplicativo fique indisponível. Uma arquitetura de camada única é adequada para testar sistemas em uma máquina local, mas não é recomendada para usos de produção.

Machine Translated by Google

Multicamada

Os desafios de uma arquitetura de camada única fortemente acoplada são resolvidos com a dissociação dos dados e do aplicativo. Uma ***arquitetura multicamadas*** (também conhecida como ***n camadas)*** é composta de camadas separadas: dados, aplicativo, lógica de negócios, apresentação etc. Essas camadas são ascendentes e hierárquicas, o que significa que a camada inferior não depende necessariamente das camadas superiores ; as camadas superiores dependem das camadas inferiores. A noção é separar os dados do aplicativo e o aplicativo da apresentação.

Uma arquitetura multicamada comum é uma arquitetura de três camadas, um design cliente-servidor amplamente utilizado. Uma ***arquitetura de três camadas*** consiste em dados, lógica de aplicativo e camadas de apresentação (Figura 3-6). Cada camada é isolada da outra, permitindo a separação de interesses. Com uma arquitetura de três camadas, você pode usar as tecnologias que preferir em cada camada sem a necessidade de se concentrar monoliticamente.

Machine Translated by Google

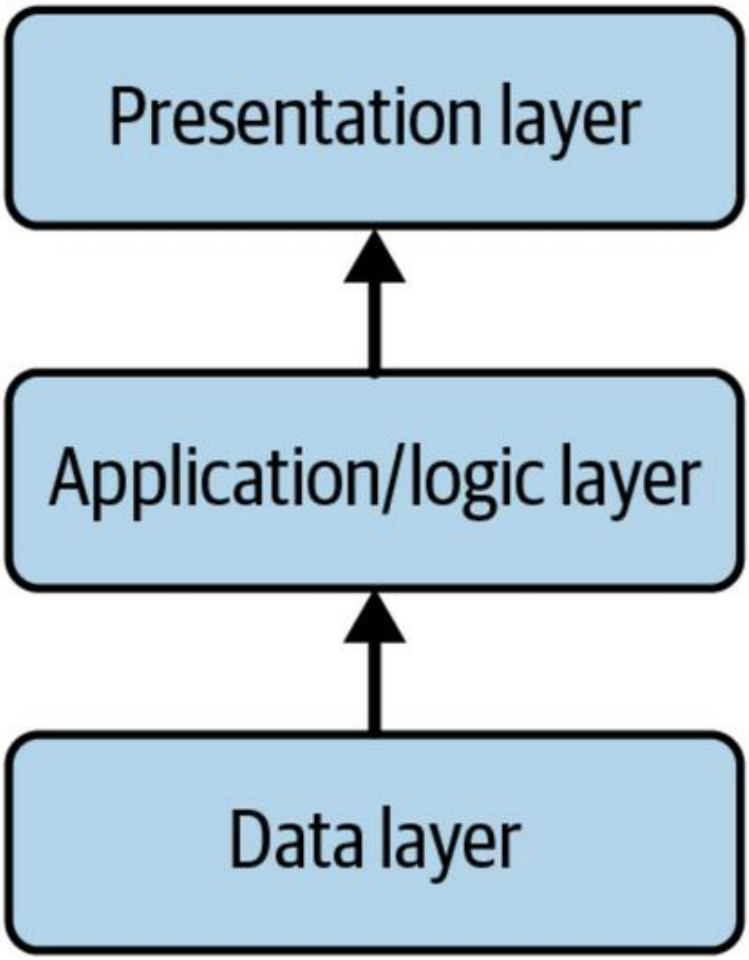


Figura 3-6. Uma arquitetura de três camadas

Vimos muitas arquiteturas de camada única em produção. As arquiteturas de camada única oferecem simplicidade, mas também limitações severas. Eventualmente, uma organização ou aplicativo supera esse arranjo; funciona bem até não funcionar. Por exemplo, em uma arquitetura de camada única, as camadas de dados e lógica compartilham e competem por recursos (disco, CPU e memória) de maneiras que são simplesmente evitadas em uma arquitetura de várias camadas. Os recursos estão distribuídos em vários níveis. Os engenheiros de dados devem usar camadas para avaliar sua arquitetura em camadas e a maneira como as dependências são tratadas. Novamente, comece de forma simples e evolua para camadas adicionais à medida que sua arquitetura se torna mais complexa.

Em uma arquitetura multicamada, você precisa considerar a separação de suas camadas e a maneira como os recursos são compartilhados nas camadas ao trabalhar com um sistema distribuído. Sistemas distribuídos sob o capô alimentam muitas tecnologias

Machine Translated by Google

você encontrará em todo o ciclo de vida da engenharia de dados. Primeiro, pense se você deseja contenção de recursos com seus nós. Caso contrário, exercite uma ***arquitetura de nada compartilhado:*** um único nó trata cada solicitação, o que significa que outros nós não compartilham recursos como memória, disco ou CPU com este nó ou entre si. Dados e recursos são isolados no nó.

Como alternativa, vários nós podem lidar com várias solicitações e compartilhar recursos, mas com o risco de contenção de recursos. Outra consideração é se os nós devem compartilhar o mesmo disco e memória acessível por todos os nós. Isso é chamado ***de arquitetura de disco compartilhado*** e é comum quando você deseja recursos compartilhados se ocorrer uma falha de nó aleatória.

Monólitos

A noção geral de um monólito inclui tanto quanto possível sob o mesmo teto; em sua versão mais extrema, um monólito consiste em uma única base de código executada em uma única máquina que fornece a lógica do aplicativo e a interface do usuário.

O acoplamento dentro de monólitos pode ser visto de duas maneiras: acoplamento técnico e acoplamento de domínio. O ***acoplamento técnico*** refere-se a camadas de arquitetura, enquanto ***o acoplamento de domínio*** refere-se à maneira como os domínios são acoplados. Um monólito tem vários graus de acoplamento entre tecnologias e domínios. Você pode ter um aplicativo com várias camadas desacopladas em uma arquitetura multicamada, mas ainda compartilhar vários domínios. Ou você pode ter uma arquitetura de camada única atendendo a um único domínio.

O acoplamento apertado de um monólito implica na falta de modularidade de seus componentes. Trocar ou atualizar componentes em um monólito geralmente é um exercício de trocar uma dor por outra. Devido à natureza fortemente acoplada, a reutilização de componentes na arquitetura é difícil ou impossível.
Ao avaliar como melhorar uma arquitetura monolítica, muitas vezes é um jogo de bater a toupeira: um componente é aprimorado, geralmente às custas de consequências desconhecidas com outras áreas do monólito.

As equipes de dados geralmente ignoram a crescente complexidade de seu monólito, deixando-o se transformar em uma grande bola de lama.

Machine Translated by Google

O Capítulo 4 fornece uma discussão mais extensa comparando monólitos a tecnologias distribuídas. Também discutimos o ***monólito distribuído,*** um estranho híbrido que surge quando os engenheiros constroem sistemas distribuídos com acoplamento rígido excessivo.

Microsserviços

Em comparação com os atributos de um monólito – serviços entrelaçados, centralização e acoplamento estreito entre serviços – os microsserviços são o oposto. ***A arquitetura de microsserviços*** compreende serviços separados, descentralizados e fracamente acoplados. Cada serviço tem uma função específica e é desacoplado de outros serviços que operam em seu domínio.
Se um serviço ficar temporariamente inativo, isso não afetará a capacidade de outros serviços continuarem funcionando.

Uma pergunta que surge com frequência é como converter seu monólito em muitos microsserviços (Figura 3-7). Isso depende completamente de quão complexo é o seu monólito e de quanto esforço será para começar a extrair serviços dele. É perfeitamente possível que seu monólito não possa ser desmembrado; nesse caso, você desejará começar a criar uma nova arquitetura paralela que tenha os serviços desacoplados de maneira amigável aos microsserviços. Não sugerimos uma refatoração inteira, mas, em vez disso, separamos os serviços. O monólito não chegou da noite para o dia e é uma questão de tecnologia como organizacional. Certifique-se de obter a adesão das partes interessadas do monólito se você planeja separá-lo.

Se você quiser saber mais sobre como quebrar um monólito, sugerimos a leitura do guia fantástico e pragmático ***Software Architecture: The Hard Parts ,*** de Neal Ford et al. (O'Reilly).

Considerações para arquitetura de dados

Como mencionamos no início desta seção, os conceitos de acoplamento rígido versus acoplamento flexível derivam do desenvolvimento de software, com alguns desses conceitos datando de mais de 20 anos. Embora as práticas de arquitetura em dados agora estejam adotando as do desenvolvimento de software, ainda é comum ver arquiteturas de dados muito monolíticas e fortemente acopladas. Parte disso se deve à natureza das tecnologias de dados existentes e à maneira como elas se integram.

Machine Translated by Google

Por exemplo, pipelines de dados podem consumir dados de várias fontes ingeridas em um data warehouse central. O data warehouse central é inerentemente monolítico. Uma mudança em direção a um microsserviço equivalente a um data warehouse é desacoplar o fluxo de trabalho com pipelines de dados específicos do domínio conectando-se aos data warehouses específicos do domínio correspondentes. Por exemplo, o pipeline de dados de vendas se conecta ao data warehouse específico de vendas e os domínios de estoque e produto seguem um padrão semelhante.

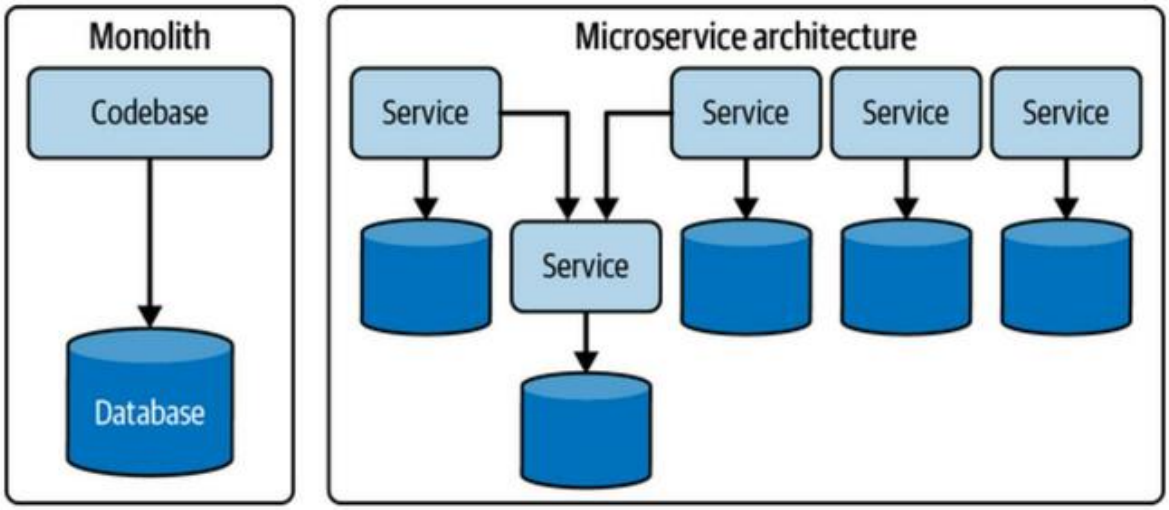


Figura 3-7. Uma arquitetura extremamente monolítica executa todas as funcionalidades dentro de uma única base de código, potencialmente colocando um banco de dados no mesmo servidor host

Em vez de pregar dogmaticamente os microsserviços sobre os monólitos (entre outros argumentos), sugerimos que você use o acoplamento flexível como ideal, reconhecendo o estado e as limitações das tecnologias de dados que você está usando em sua arquitetura de dados. Incorpore opções de tecnologia reversível que permitem modularidade e baixo acoplamento sempre que possível.

Como você pode ver na Figura 3-7, você separa os componentes de sua arquitetura em diferentes camadas de interesse de forma vertical. Embora uma arquitetura multicamada resolva os desafios técnicos de desacoplar recursos compartilhados, ela não aborda a complexidade de compartilhar domínios. Ao longo das linhas de arquitetura única versus multicamada, você também deve considerar como separa os domínios de sua arquitetura de dados. Por exemplo, sua equipe de analistas pode contar com dados de vendas e estoque. Os domínios de vendas e inventário são diferentes e devem ser vistos separadamente.

Machine Translated by Google

Uma abordagem para esse problema é a centralização: uma única equipe é responsável por coletar dados de todos os domínios e reconciliá-los para consumo em toda a organização. (Esta é uma abordagem comum no armazenamento de dados tradicional.) Outra abordagem é a ***malha de dados.*** Com a malha de dados, cada equipe de software é responsável por preparar seus dados para consumo em toda a organização. Falaremos mais sobre a malha de dados mais adiante neste capítulo.

Nosso conselho: monólitos não são necessariamente ruins, e pode fazer sentido começar com um sob certas condições. Às vezes você precisa se mover rápido, e é muito mais simples começar com um monólito. Apenas esteja preparado para quebrá-lo em pedaços menores eventualmente; não fique muito confortável.

Acesso de usuário: único versus multilocatário

Como engenheiro de dados, você precisa tomar decisões sobre o compartilhamento de sistemas entre várias equipes, organizações e clientes. De certa forma, todos os serviços de nuvem são multilocatários, embora essa multilocação ocorra em vários níveis. Por exemplo, uma instância de computação em nuvem geralmente está em um servidor compartilhado, mas a própria VM fornece algum grau de isolamento. O armazenamento de objetos é um sistema multilocatário, mas os fornecedores de nuvem garantem segurança e isolamento desde que os clientes configurem suas permissões corretamente.

Os engenheiros frequentemente precisam tomar decisões sobre multilocação em uma escala muito menor. Por exemplo, vários departamentos em uma grande empresa compartilham o mesmo data warehouse? A organização compartilha dados para vários clientes grandes na mesma tabela?

Temos dois fatores a serem considerados na multilocação: desempenho e segurança. Com vários locatários grandes em um sistema em nuvem, o sistema oferecerá suporte a desempenho consistente para todos os locatários ou haverá um problema de vizinho barulhento? (Ou seja, o alto uso de um locatário prejudicará o desempenho de outros locatários?) Em relação à segurança, os dados de diferentes locatários devem ser isolados adequadamente. Quando uma empresa tem vários locatários de clientes externos, esses locatários não devem estar cientes uns dos outros e os engenheiros devem evitar o vazamento de dados. As estratégias para isolamento de dados variam de acordo com o sistema. Por

Machine Translated by Google

Por exemplo, geralmente é perfeitamente aceitável usar tabelas multilocatário e isolar dados por meio de exibições. No entanto, você deve certificar-se de que essas exibições não podem vazar dados. Leia a documentação do fornecedor ou do projeto para entender as estratégias e riscos apropriados.

Arquitetura orientada a eventos

Seu negócio raramente é estático. Muitas vezes acontecem coisas em sua empresa, como obter um novo cliente, um novo pedido de um cliente ou um pedido de um produto ou serviço. Todos esses são exemplos de ***eventos*** amplamente definidos como algo que aconteceu, normalmente uma mudança no ***estado*** de algo.
Por exemplo, um novo pedido pode ser criado por um cliente ou um cliente pode posteriormente fazer uma atualização nesse pedido.

Um fluxo de trabalho orientado a eventos (Figura 3-8) abrange a capacidade de criar, atualizar e mover eventos de forma assíncrona em várias partes do ciclo de vida da engenharia de dados. Esse fluxo de trabalho se resume a três áreas principais: produção de eventos, roteamento e consumo. Um evento deve ser produzido e roteado para algo que o consuma sem dependências fortemente acopladas entre o produtor, roteador de eventos e consumidor.



Figura 3-8. Em um fluxo de trabalho orientado a eventos, um evento é produzido, roteado e então consumido

Uma arquitetura orientada a eventos (Figura 3-9) abrange o fluxo de trabalho orientado a eventos e o usa para se comunicar entre vários serviços. A vantagem de uma arquitetura orientada a eventos é que ela distribui o estado de um evento em vários serviços. Isso é útil se um serviço ficar offline, um nó falhar em um sistema distribuído ou se você quiser que vários consumidores ou serviços acessem os mesmos eventos. Sempre que você tiver serviços fracamente acoplados, este é um candidato à arquitetura orientada a eventos. Muitos dos exemplos que descrevemos posteriormente neste capítulo incorporam alguma forma de arquitetura orientada a eventos.

Machine Translated by Google

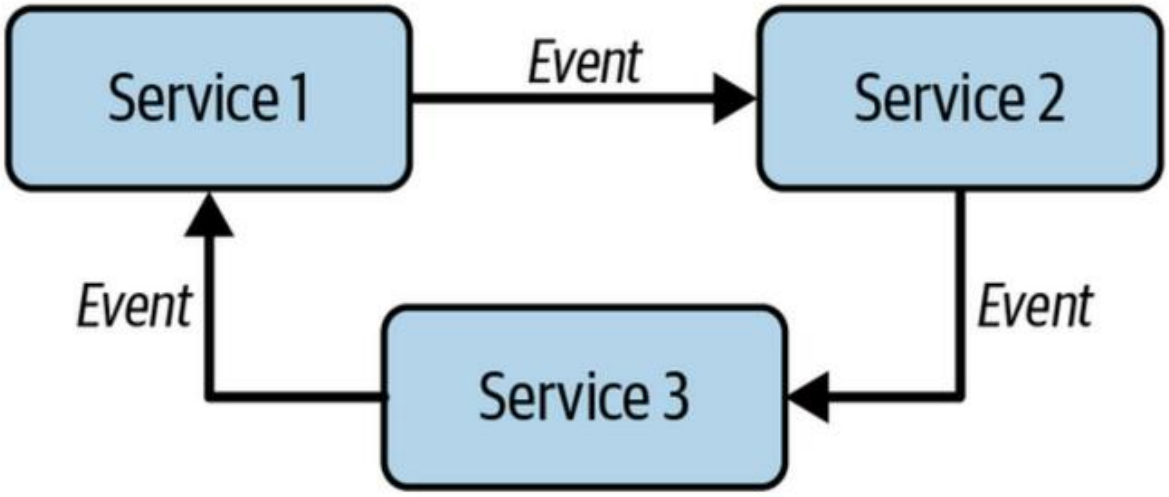


Figura 3-9. Em uma arquitetura orientada a eventos, os eventos são passados entre serviços fracamente acoplados

Você aprenderá mais sobre sistemas de transmissão e mensagens orientados a eventos no Capítulo 5.

Projetos Brownfield versus Greenfield

Antes de projetar seu projeto de arquitetura de dados, você precisa saber se está começando do zero ou redesenhando uma arquitetura existente.
Cada tipo de projeto requer a avaliação de trade-offs, embora com diferentes considerações e abordagens. Os projetos se dividem aproximadamente em dois grupos: brownfield e greenfield.

Projetos brownfield

Projetos brownfield geralmente envolvem refatoração e reorganização de uma arquitetura existente e são limitados pelas escolhas do presente e do passado.
Como uma parte fundamental da arquitetura é o gerenciamento de mudanças, você deve descobrir uma maneira de contornar essas limitações e projetar um caminho a seguir para alcançar seus novos objetivos comerciais e técnicos. Projetos brownfield exigem uma compreensão completa da arquitetura legada e a interação de várias tecnologias antigas e novas. Com muita frequência, é fácil criticar o trabalho e as decisões de uma equipe anterior, mas é muito melhor cavar fundo, fazer perguntas e entender por que as decisões foram tomadas. Empatia e contexto ajudam muito a diagnosticar problemas com a arquitetura existente, identificar oportunidades e reconhecer armadilhas.

Machine Translated by Google

Você precisará apresentar sua nova arquitetura e tecnologias e descontinuar as coisas antigas em algum momento. Vejamos algumas abordagens populares. Muitas equipes saltam de cabeça para uma revisão completa ou big-bang da arquitetura antiga, muitas vezes descobrindo a depreciação à medida que avançam.
Embora popular, não aconselhamos essa abordagem devido aos riscos associados e à falta de um plano. Esse caminho muitas vezes leva ao desastre, com muitas decisões irreversíveis e dispendiosas. Seu trabalho é tomar decisões reversíveis e de alto ROI.

Uma alternativa popular para uma reescrita direta é o padrão estrangulador: novos sistemas substituem lenta e incrementalmente os componentes de uma arquitetura legada. Eventualmente, a arquitetura legada é completamente substituída.

A atração para o padrão estrangulador é sua abordagem direcionada e cirúrgica de depreciar uma peça de um sistema de cada vez. Isso permite decisões flexíveis e reversíveis ao avaliar o impacto da descontinuação em sistemas dependentes.

É importante observar que a descontinuação pode ser um conselho de “torre de marfim” e não prático ou alcançável. Erradicar tecnologia ou arquitetura herdada pode ser impossível se você estiver em uma grande organização. Alguém, em algum lugar, está usando esses componentes legados. Como alguém disse uma vez: “Legado é uma maneira condescendente de descrever algo que gera dinheiro”.

Se você puder descontinuar, entenda que existem várias maneiras de descontinuar sua arquitetura antiga. É fundamental demonstrar valor na nova plataforma aumentando gradualmente sua maturidade para mostrar evidências de sucesso e seguir um plano de saída para encerrar sistemas antigos.

Projetos greenfield

No extremo oposto do espectro, um ***projeto greenfield*** permite que você seja pioneiro em um novo começo, sem restrições pela história ou legado de uma arquitetura anterior.
Projetos greenfield tendem a ser mais fáceis do que projetos brownfield, e muitos arquitetos e engenheiros de dados os consideram mais divertidos! Você tem a oportunidade de experimentar as ferramentas e padrões arquitetônicos mais novos e interessantes.
O que poderia ser mais emocionante?

Machine Translated by Google

Você deve tomar cuidado com algumas coisas antes de se empolgar demais. Vemos as equipes ficarem excessivamente exuberantes com a síndrome do objeto brilhante. Eles se sentem compelidos a buscar a maior e mais recente moda tecnológica sem entender como isso afetará o valor do projeto. Há também a tentação de fazer ***desenvolvimento orientado por currículo,*** empilhando novas tecnologias impressionantes sem priorizar os objetivos finais do projeto. Sempre priorize os requisitos sobre a construção de algo legal.

Esteja você trabalhando em um projeto brownfield ou greenfield, sempre concentre-se nos princípios da “boa” arquitetura de dados. Avalie as compensações, tome decisões flexíveis e reversíveis e lute por um ROI positivo.

Agora, veremos exemplos e tipos de arquiteturas - algumas estabelecidas há décadas (o data warehouse), algumas novas (o data lakehouse) e algumas que surgiram e desapareceram rapidamente, mas ainda influenciam os padrões de arquitetura atuais (arquitetura Lambda). .

Exemplos e tipos de arquitetura de dados

Como a arquitetura de dados é uma disciplina abstrata, ajuda a raciocinar pelo exemplo. Nesta seção, descrevemos exemplos e tipos proeminentes de arquitetura de dados que são populares hoje em dia. Embora este conjunto de exemplos não seja de forma alguma exaustivo, a intenção é expô-lo a alguns dos padrões de arquitetura de dados mais comuns e levá-lo a pensar sobre a flexibilidade necessária e a análise de compensação necessária ao projetar uma boa arquitetura para seu caso de uso .

Armazém de dados

Um ***data warehouse*** é um hub de dados central usado para relatórios e análises. Os dados em um data warehouse geralmente são altamente formatados e estruturados para casos de uso de análise. Está entre as arquiteturas de dados mais antigas e bem estabelecidas.

Em 1990, Bill Inmon originou a noção de data warehouse, que ele descreveu como “um sistema orientado a assunto, integrado, não volátil e variante no tempo”.

Machine Translated by Google

coleta de dados para apoiar as decisões da administração". Embora os aspectos técnicos do data warehouse tenham evoluído significativamente, sentimos que essa definição original ainda mantém seu peso hoje.

No passado, os data warehouses eram amplamente usados em empresas com orçamentos significativos (geralmente na casa dos milhões de dólares) para adquirir sistemas de dados e pagar equipes internas para fornecer suporte contínuo para manter o data warehouse. Isso era caro e trabalhoso. Desde então, o modelo escalável de pagamento conforme o uso tornou os data warehouses em nuvem acessíveis até mesmo para pequenas empresas. Como um provedor terceirizado gerencia a infraestrutura de data warehouse, as empresas podem fazer muito mais com menos pessoas, mesmo com o aumento da complexidade de seus dados.

Vale a pena notar dois tipos de arquitetura de data warehouse: organizacional e técnica. A ***arquitetura de data warehouse*** organizacional organiza os dados associados a determinadas estruturas e processos da equipe de negócios. A ***arquitetura do data warehouse*** técnico reflete a natureza técnica do data warehouse, como MPP. Uma empresa pode ter um data warehouse sem um sistema MPP ou executar um sistema MPP que não esteja organizado como um data warehouse. No entanto, as arquiteturas técnica e organizacional têm existido em um ciclo virtuoso e são frequentemente identificadas entre si.

A arquitetura de data warehouse organizacional tem duas características principais:

Separa os processos analíticos (OLAP) dos bancos de dados de produção (processamento de transações online)

> Essa separação é crítica à medida que os negócios crescem. Mover dados para um sistema físico separado direciona a carga dos sistemas de produção e melhora o desempenho da análise.

Centraliza e organiza os dados

> Tradicionalmente, um data warehouse extrai dados de sistemas de aplicativos usando ETL. A fase de extração extrai dados dos sistemas de origem. A fase de transformação limpa e padroniza os dados, organizando e

Machine Translated by Google

impondo lógica de negócios em uma forma altamente modelada. (O Capítulo 8 abrange transformações e modelos de dados.) A fase de carregamento envia os dados para o sistema de banco de dados de destino do data warehouse. Os dados são carregados em vários data marts que atendem às necessidades analíticas de linhas ou negócios e departamentos específicos. A Figura 3-10 mostra o fluxo de trabalho geral. O data warehouse e o ETL andam de mãos dadas com estruturas de negócios específicas, incluindo equipes de desenvolvedores de DBA e ETL que implementam a direção dos líderes de negócios para garantir que os dados para relatórios e análises correspondam aos processos de negócios. Um sistema ETL não é um pré-requisito para um data warehouse, como você aprenderá quando discutirmos outro padrão de carga e transformação, o ELT.

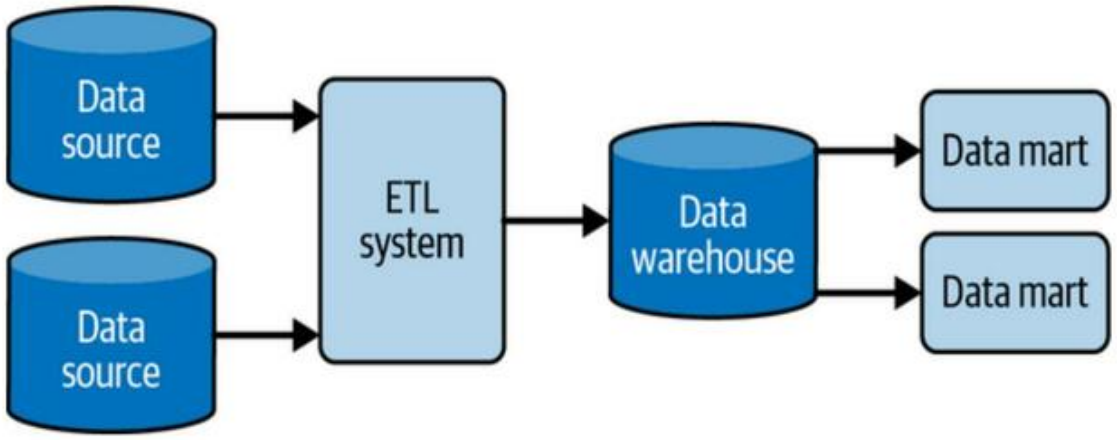


Figura 3-10. Data warehouse básico com ETL

Em relação à arquitetura de data warehouse técnico, os primeiros sistemas MPP no final da década de 1970 tornaram-se populares na década de 1980. Os MPPs suportam essencialmente a mesma semântica SQL usada em bancos de dados de aplicativos relacionais. Ainda assim, eles são otimizados para varrer grandes quantidades de dados em paralelo e, assim, permitir agregação de alto desempenho e cálculos estatísticos. Nos últimos anos, os sistemas MPP mudaram cada vez mais de uma arquitetura baseada em linhas para uma arquitetura colunar para facilitar dados e consultas ainda maiores, especialmente em data warehouses em nuvem. Os MPPs são indispensáveis para executar consultas de alto desempenho para grandes empresas à medida que as necessidades de dados e relatórios crescem.

Uma variação do ETL é o ELT. Com a arquitetura de data warehouse ELT, os dados são movidos mais ou menos diretamente dos sistemas de produção para um

Machine Translated by Google

área de preparação no data warehouse. A preparação nessa configuração indica que os dados estão em um formato bruto. Em vez de usar um sistema externo, as transformações são tratadas diretamente no data warehouse. A intenção é aproveitar o enorme poder computacional dos data warehouses na nuvem e das ferramentas de processamento de dados. Os dados são processados em lotes e a saída transformada é gravada em tabelas e visualizações para análise. A Figura 3-11 mostra o processo geral. O ELT também é popular em um arranjo de streaming, pois os eventos são transmitidos de um processo CDC, armazenados em uma área de teste e, posteriormente, transformados no data warehouse.

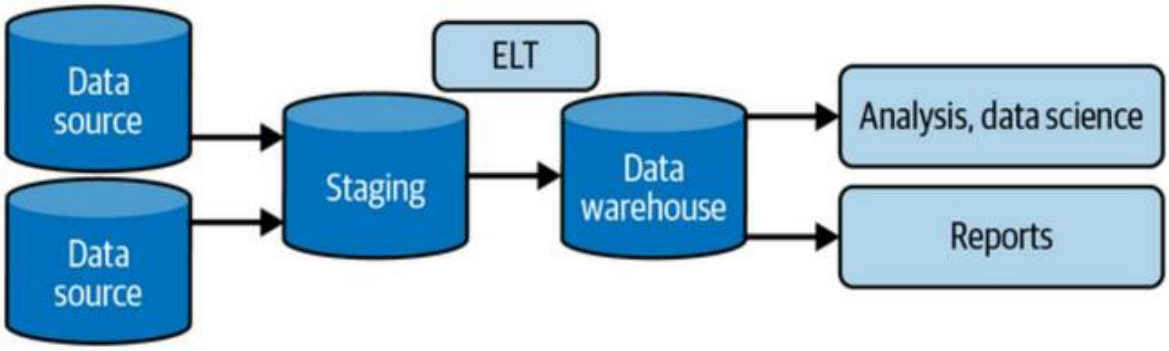


Figura 3-11. ELT—extrair, carregar e transformar

Uma segunda versão do ELT foi popularizada durante o crescimento de big data no ecossistema Hadoop. Este é ***o ELT de transformação na leitura,*** que discutimos em "Data Lake".

O armazenamento de dados em nuvem

Os data warehouses em nuvem representam uma evolução significativa da arquitetura de data warehouse local e, portanto, levaram a mudanças significativas na arquitetura organizacional. O Amazon Redshift deu início à revolução do data warehouse na nuvem. Em vez de precisar dimensionar adequadamente um sistema MPP para os próximos anos e assinar um contrato multimilionário para adquirir o sistema, as empresas tiveram a opção de criar um cluster Redshift sob demanda, ampliando-o ao longo do tempo à medida que a demanda de dados e análises crescia . Eles podem até criar novos clusters do Redshift sob demanda para atender a cargas de trabalho específicas e excluir clusters rapidamente quando não forem mais necessários.

Machine Translated by Google

Google BigQuery, Snowflake e outros concorrentes popularizaram a ideia de separar computação de armazenamento. Nesta arquitetura, os dados são alojados em armazenamento de objetos, permitindo armazenamento virtualmente ilimitado. Isso também oferece aos usuários a opção de aumentar o poder de computação sob demanda, fornecendo recursos de big data ad hoc sem o custo de longo prazo de milhares de nós.

Os data warehouses em nuvem expandem os recursos dos sistemas MPP para cobrir muitos casos de uso de big data que exigiam um cluster Hadoop em um passado muito recente. Eles podem processar prontamente petabytes de dados em uma única consulta. Eles normalmente suportam estruturas de dados que permitem o armazenamento de dezenas de megabytes de dados de texto bruto por linha ou documentos JSON extremamente ricos e complexos. À medida que os data warehouses na nuvem (e data lakes) amadurecem, a linha entre o data warehouse e o data lake continuará a se confundir.

O impacto dos novos recursos oferecidos pelos data warehouses em nuvem é tão significativo que podemos considerar descartar completamente o termo ***data warehouse .*** Em vez disso, esses serviços estão evoluindo para uma nova plataforma de dados com recursos muito mais amplos do que os oferecidos por um sistema MPP tradicional.

Data marts

Um ***data mart*** é um subconjunto mais refinado de um warehouse projetado para servir análises e relatórios, focado em uma única suborganização, departamento ou linha de negócios; cada departamento tem seu próprio data mart, específico para suas necessidades. Isso contrasta com o data warehouse completo que atende à organização ou negócio mais amplo.

Os data marts existem por dois motivos. Primeiro, um data mart torna os dados mais facilmente acessíveis a analistas e desenvolvedores de relatórios. Em segundo lugar, os data marts fornecem um estágio adicional de transformação além do fornecido pelos pipelines ETL ou ELT iniciais. Isso pode melhorar significativamente o desempenho se relatórios ou consultas analíticas exigirem junções e agregações complexas de dados, especialmente quando os dados brutos forem grandes. Os processos de transformação podem preencher o data mart com dados combinados e agregados para melhorar o desempenho das consultas ao vivo. A Figura 3-12 mostra o fluxo de trabalho geral. Discutimos data marts e modelagem de dados para data marts no Capítulo 8.

Machine Translated by Google

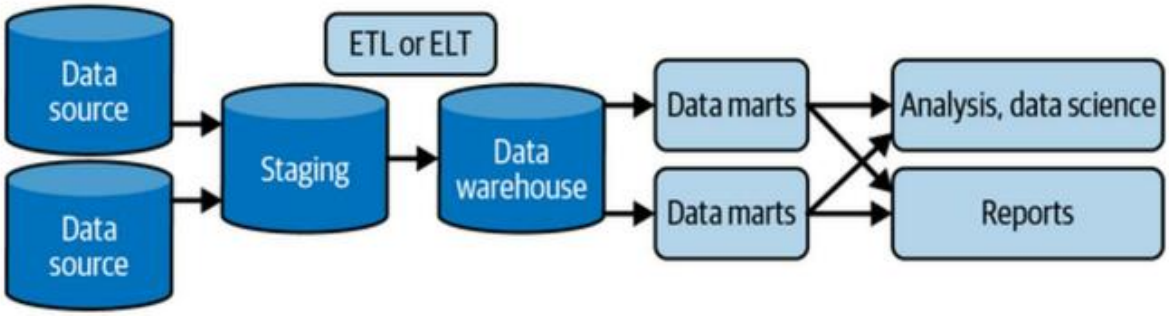


Figura 3-12. ETL ou ELT mais data marts

Data Lake

Entre as arquiteturas mais populares que surgiram durante a era do big data está o ***data lake.*** Em vez de impor limitações estruturais rígidas aos dados, por que não simplesmente despejar todos os seus dados – estruturados e não estruturados – em um local central? O data lake prometia ser uma força democratizadora, liberando os negócios para beber de uma fonte de dados ilimitados. O data lake de primeira geração, “data lake 1.0”, fez contribuições sólidas, mas geralmente não cumpriu sua promessa.

O Data Lake 1.0 começou com HDFS. À medida que a nuvem cresceu em popularidade, esses data lakes mudaram para armazenamento de objetos baseado em nuvem, com custos de armazenamento extremamente baratos e capacidade de armazenamento praticamente ilimitada. Em vez de depender de um data warehouse monolítico onde armazenamento e computação estão fortemente acoplados, o data lake permite que uma imensa quantidade de dados de qualquer tamanho e tipo seja armazenada. Quando esses dados precisam ser consultados ou transformados, você tem acesso a um poder de computação quase ilimitado ao criar um cluster sob demanda e pode escolher sua tecnologia de processamento de dados favorita para a tarefa em mãos — MapReduce, Spark, Ray, Presto, Colmeia, etc

Apesar da promessa e do hype, o data lake 1.0 tinha sérias deficiências. O data lake tornou-se um depósito de lixo; termos como ***data swamp, dark data*** e ***WORN*** foram cunhados quando projetos de dados outrora promissores falharam. Os dados cresceram para tamanhos não gerenciáveis, com pouco em termos de gerenciamento de esquema, catalogação de dados e ferramentas de descoberta. Além disso, o conceito original de data lake era essencialmente somente gravação, criando enormes dores de cabeça com a chegada de regulamentações como o GDPR, que exigia a exclusão direcionada de registros de usuários.

Machine Translated by Google

O processamento de dados também foi um desafio. Transformações de dados relativamente banais, como junções, eram uma grande dor de cabeça para codificar como trabalhos MapReduce. Estruturas posteriores, como Pig e Hive, melhoraram um pouco a situação do processamento de dados, mas pouco fizeram para resolver os problemas básicos do gerenciamento de dados.
As operações simples de linguagem de manipulação de dados (DML) comuns em SQL — excluir ou atualizar linhas — eram difíceis de implementar, geralmente obtidas com a criação de tabelas inteiramente novas. Enquanto os engenheiros de big data irradiavam um desdém particular por suas contrapartes em data warehousing, este último poderia apontar que os data warehouses forneciam recursos básicos de gerenciamento de dados prontos para uso e que o SQL era uma ferramenta eficiente para escrever consultas e transformações complexas e de alto desempenho.

O Data Lake 1.0 também não cumpriu outra promessa central do movimento de big data. O software de código aberto no ecossistema Apache foi apresentado como um meio de evitar contratos multimilionários para sistemas MPP proprietários.
Hardware barato e pronto para uso substituiria as soluções personalizadas de fornecedores. Na realidade, os custos de big data aumentaram à medida que as complexidades do gerenciamento de clusters Hadoop forçavam as empresas a contratar grandes equipes de engenheiros com altos salários. As empresas geralmente optam por comprar versões licenciadas e personalizadas do Hadoop de fornecedores para evitar os fios expostos e as bordas afiadas da base de código Apache bruta e adquirir um conjunto de ferramentas de andaimes para tornar o Hadoop mais fácil de usar. Mesmo as empresas que evitaram o gerenciamento de clusters do Hadoop usando armazenamento em nuvem tiveram que gastar muito com talentos para escrever trabalhos do MapReduce.

Devemos ter cuidado para não subestimar a utilidade e o poder dos data lakes de primeira geração. Muitas organizações encontraram valor significativo em data lakes – especialmente grandes empresas de tecnologia do Vale do Silício, fortemente focadas em dados, como Netflix e Facebook. Essas empresas tinham os recursos para criar práticas de dados bem-sucedidas e criar suas ferramentas e aprimoramentos personalizados baseados em Hadoop. Mas para muitas organizações, os data lakes se transformaram em um superfundo interno de desperdício, decepção e custos crescentes.

Convergência, Data Lakes de Próxima Geração e os Dados Plataforma

Machine Translated by Google

Em resposta às limitações dos data lakes de primeira geração, vários players procuraram aprimorar o conceito para cumprir plenamente sua promessa. Por exemplo, o Databricks introduziu a noção de ***data lakehouse.*** O lakehouse incorpora os controles, o gerenciamento de dados e as estruturas de dados encontrados em um data warehouse, ao mesmo tempo em que abriga dados no armazenamento de objetos e oferece suporte a uma variedade de mecanismos de consulta e transformação. Em particular, o data lakehouse oferece suporte a transações de atomicidade, consistência, isolamento e durabilidade (ACID), um grande afastamento do data lake original, onde você simplesmente insere dados e nunca os atualiza ou exclui. O termo ***data lakehouse*** sugere uma convergência entre data lakes e data warehouses.

A arquitetura técnica de data warehouses em nuvem evoluiu para ser muito semelhante a uma arquitetura de data lake. Os data warehouses na nuvem separam a computação do armazenamento, oferecem suporte a consultas em escala de petabytes, armazenam texto não estruturado e objetos semiestruturados e integram-se a tecnologias de processamento avançadas, como Spark ou Beam.

Acreditamos que a tendência de convergência só vai continuar. O data lake e o data warehouse ainda existirão como arquiteturas diferentes. Na prática, suas capacidades irão convergir para que poucos usuários percebam uma fronteira entre eles em seu trabalho diário. Agora vemos vários fornecedores oferecendo ***plataformas de dados*** que combinam recursos de data lake e data warehouse. Do nosso ponto de vista, AWS, Azure, Google Cloud, Floco de neve, e Databricks são líderes de classe, cada um oferecendo uma constelação de ferramentas totalmente integradas para trabalhar com dados, variando de relacional a completamente não estruturado. Em vez de escolher entre uma arquitetura de data lake ou data warehouse, os futuros engenheiros de dados terão a opção de escolher uma plataforma de dados convergente com base em vários fatores, incluindo fornecedor, ecossistema e abertura relativa.

Pilha de dados moderna

A ***pilha de dados moderna*** (Figura 3-13) é atualmente uma arquitetura analítica moderna que destaca o tipo de abstração que esperamos ver mais amplamente utilizado nos próximos anos. Enquanto as pilhas de dados anteriores dependiam de conjuntos de ferramentas monolíticos caros, o principal objetivo da pilha de dados moderna

Machine Translated by Google

é usar componentes baseados em nuvem, plug-and-play, fáceis de usar e prontos para uso para criar uma arquitetura de dados modular e econômica. Esses componentes incluem pipelines de dados, armazenamento, transformação, gerenciamento/governança de dados, monitoramento, visualização e exploração. O domínio ainda está em fluxo e as ferramentas específicas estão mudando e evoluindo rapidamente, mas o objetivo principal permanecerá o mesmo: reduzir a complexidade e aumentar a modularização. Observe que a noção de uma pilha de dados moderna se integra perfeitamente à ideia de plataforma de dados convergentes da seção anterior.

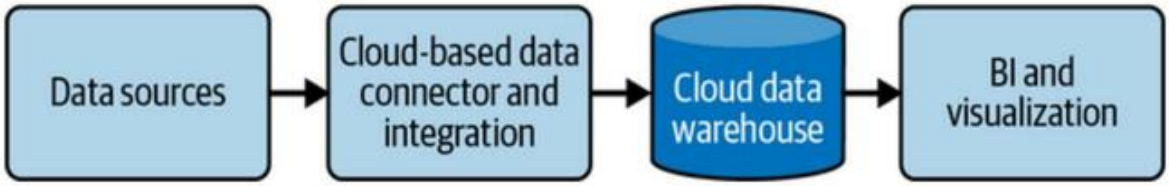


Figura 3-13. Componentes básicos da pilha de dados moderna

Os principais resultados da pilha de dados moderna são autoatendimento (análises e pipelines), gerenciamento ágil de dados e uso de ferramentas de código aberto ou ferramentas proprietárias simples com estruturas de preços claras. A comunidade também é um aspecto central da pilha de dados moderna. Ao contrário dos produtos do passado que tinham lançamentos e roteiros amplamente escondidos dos usuários, projetos e empresas que operam no espaço moderno de pilha de dados normalmente têm bases de usuários fortes e comunidades ativas que participam do desenvolvimento usando o produto antecipadamente, sugerindo recursos e enviando pull pedidos para melhorar o código.

Independentemente de onde o “moderno” vá (compartilhamos nossas ideias no Capítulo 11), achamos que o conceito-chave de modularidade plug-and-play com preços e implementação fáceis de entender é o caminho do futuro. Especialmente na engenharia analítica, a pilha de dados moderna é e continuará sendo a escolha padrão de arquitetura de dados. Ao longo do livro, a arquitetura a que nos referimos contém partes da pilha de dados moderna, como componentes modulares plug-and-play e baseados em nuvem.

Arquitetura lambda

Machine Translated by Google

Nos “velhos tempos” (início a meados da década de 2010), a popularidade de trabalhar com dados de streaming explodiu com o surgimento do Kafka como uma fila de mensagens altamente escalável e estruturas como Apache Storm e Samza para análise de streaming/em tempo real. Essas tecnologias permitiram que as empresas realizassem novos tipos de análise e modelagem em grandes quantidades de dados, agregação e classificação de usuários e recomendações de produtos. Os engenheiros de dados precisavam descobrir como reconciliar dados em lote e streaming em uma única arquitetura. A arquitetura Lambda foi uma das primeiras respostas populares a esse problema.

Em uma ***arquitetura Lambda*** (Figura 3-14), você tem sistemas operando independentemente uns dos outros — lote, streaming e serviço. O sistema de origem é idealmente imutável e somente anexado, enviando dados para dois destinos para processamento: fluxo e lote. O processamento in-stream pretende atender os dados com a menor latência possível em uma camada de “velocidade”, geralmente um banco de dados NoSQL. Na camada batch, os dados são processados e transformados em um sistema como um data warehouse, criando visualizações pré-computadas e agregadas dos dados. A camada de serviço fornece uma visualização combinada agregando os resultados da consulta das duas camadas.

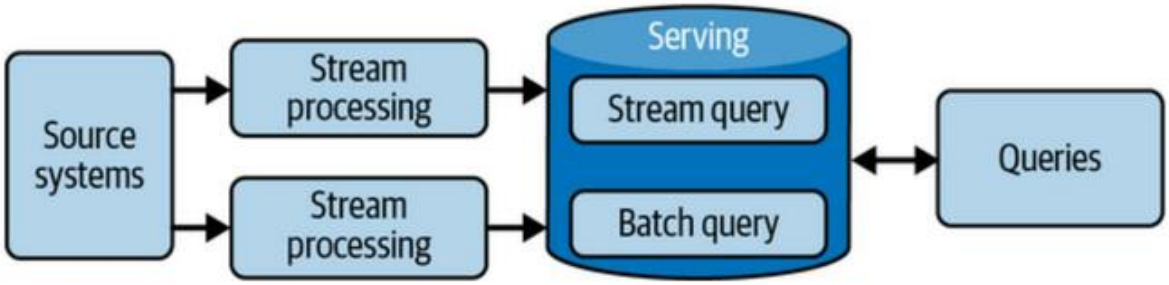


Figura 3-14. Arquitetura lambda

A arquitetura lambda tem sua parcela de desafios e críticas. Gerenciar vários sistemas com bases de código diferentes é tão difícil quanto parece, criando sistemas propensos a erros com código e dados extremamente difíceis de conciliar.

Mencionamos a arquitetura Lambda porque ela ainda chama atenção e é popular nos resultados de mecanismos de pesquisa para arquitetura de dados. Lambda não é o nosso primeiro

Machine Translated by Google

recomendação se você estiver tentando combinar dados de streaming e lote para análise. A tecnologia e as práticas mudaram.

A seguir, vamos ver uma reação à arquitetura Lambda, a arquitetura Kappa.

Arquitetura Kappa

Como resposta às deficiências da arquitetura Lambda, Jay Kreps propôs uma alternativa chamada ***arquitetura Kappa*** (Figura 3-15). A tese central é esta: por que[10] não usar apenas uma plataforma de processamento de fluxo como a espinha dorsal para todo o manuseio de dados – ingestão, armazenamento e serviço? Isso facilita uma verdadeira arquitetura baseada em eventos. O processamento em tempo real e em lote pode ser aplicado perfeitamente aos mesmos dados lendo o fluxo de eventos ao vivo diretamente e reproduzindo grandes blocos de dados para processamento em lote.

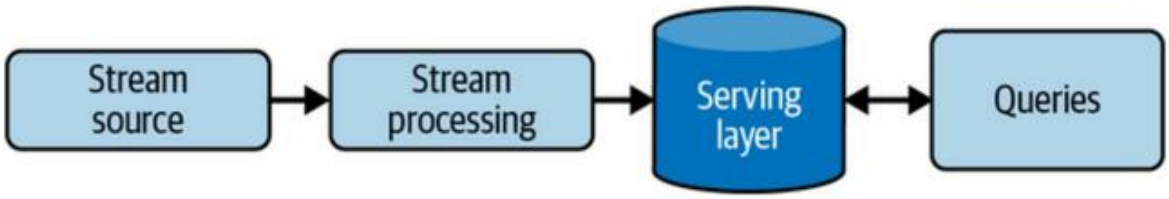


Figura 3-15. Arquitetura Kappa

Embora o artigo original da arquitetura Kappa tenha sido publicado em 2014, não o vimos amplamente adotado. Pode haver algumas razões para isso.
Primeiro, o streaming em si ainda é um mistério para muitas empresas.
Em segundo lugar, a arquitetura Kappa acaba sendo complicada e cara na prática. Embora alguns sistemas de streaming possam ser dimensionados para grandes volumes de dados, eles são complexos e caros; o armazenamento e o processamento em lote permanecem muito mais eficientes e econômicos para enormes conjuntos de dados históricos.

O modelo de fluxo de dados e o lote e streaming unificados

Tanto Lambda quanto Kappa procuraram abordar as limitações do ecossistema Hadoop da década de 2010, tentando unir ferramentas complicadas que provavelmente não eram encaixes naturais em primeiro lugar. O desafio central da

Machine Translated by Google

A unificação de dados de lote e streaming permaneceu, e Lambda e Kappa forneceram inspiração e base para o progresso contínuo nessa busca.

Um dos problemas centrais do gerenciamento de processamento em lote e fluxo é unificar vários caminhos de código. Embora a arquitetura Kappa dependa de uma camada unificada de enfileiramento e armazenamento, ainda é preciso enfrentar o uso de diferentes ferramentas para coletar estatísticas em tempo real ou executar tarefas de agregação em lote. Hoje, os engenheiros procuram resolver isso de várias maneiras. O Google deixou sua marca desenvolvendo o modelo Dataflow e o feixe Apache framework que implementa este modelo.

A ideia central do modelo Dataflow é visualizar todos os dados como eventos, pois a agregação é realizada em vários tipos de janelas. Os fluxos de eventos em tempo real contínuos são ***dados ilimitados.*** Os lotes de dados são simplesmente fluxos de eventos limitados e os limites fornecem uma janela natural. Os engenheiros podem escolher entre várias janelas para agregação em tempo real, como deslizamento ou queda. O processamento em tempo real e em lote ocorre no mesmo sistema usando código quase idêntico.

A filosofia de “lote como um caso especial de streaming” agora é mais difundida. Vários frameworks, como Flink e Spark, adotaram uma abordagem semelhante.

Arquitetura para IoT

A ***Internet das Coisas*** (IoT) é a coleção distribuída de dispositivos, também conhecidos como coisas – computadores, sensores, dispositivos móveis, dispositivos domésticos inteligentes e qualquer outra coisa com conexão à Internet. Em vez de gerar dados de entrada humana direta (pense na entrada de dados de um teclado), os dados de IoT são gerados a partir de dispositivos que coletam dados periodicamente ou continuamente do ambiente ao redor e os transmitem para um destino. Os dispositivos IoT geralmente são de baixa potência e operam em ambientes com poucos recursos/baixa largura de banda.

Embora o conceito de dispositivos IoT remonte pelo menos algumas décadas, a revolução dos smartphones criou um enorme enxame de IoT praticamente da noite para o dia. Desde então, várias novas categorias de IoT surgiram, como smart

Machine Translated by Google

termostatos, sistemas de entretenimento automotivo, TVs inteligentes e alto-falantes inteligentes. A IoT evoluiu de uma fantasia futurista para um domínio massivo de engenharia de dados. Esperamos que a IoT se torne uma das formas dominantes de geração e consumo de dados, e esta seção é um pouco mais profunda do que as outras que você leu.

Ter uma compreensão superficial da arquitetura IoT ajudará você a entender as tendências mais amplas da arquitetura de dados. Vejamos brevemente alguns conceitos de arquitetura de IoT.

Dispositivos

Dispositivos (também conhecidos como ***coisas)*** são o hardware físico conectado à Internet, detectando o ambiente ao seu redor e coletando e transmitindo dados para um destino downstream. Esses dispositivos podem ser usados em aplicativos de consumo, como uma câmera de campainha, smartwatch ou termostato.
O dispositivo pode ser uma câmera com inteligência artificial que monitora uma linha de montagem em busca de componentes defeituosos, um rastreador GPS para registrar a localização de veículos ou um Raspberry Pi programado para baixar os tweets mais recentes e preparar seu café. Qualquer dispositivo capaz de coletar dados de seu ambiente é um dispositivo IoT.

Os dispositivos devem ser minimamente capazes de coletar e transmitir dados.
No entanto, o dispositivo também pode processar dados ou executar ML nos dados que coleta antes de enviá-los para downstream – computação de borda e aprendizado de máquina de borda, respectivamente.

Um engenheiro de dados não precisa necessariamente conhecer os detalhes internos dos dispositivos IoT, mas deve saber o que o dispositivo faz, os dados que coleta, quaisquer cálculos de borda ou ML executados antes de transmitir os dados e com que frequência envia dados. Também ajuda a conhecer as consequências de um dispositivo ou interrupção da Internet, fatores ambientais ou outros fatores externos que afetam a coleta de dados e como isso pode afetar a coleta de dados a jusante do dispositivo.

Interface com dispositivos

Machine Translated by Google

Um dispositivo não é benéfico a menos que você possa obter seus dados. Esta seção abrange alguns dos principais componentes necessários para fazer a interface com dispositivos IoT em estado selvagem.

Gateway de IoT

Um ***gateway IoT*** é um hub para conectar dispositivos e rotear dispositivos com segurança para os destinos apropriados na Internet. Embora você possa conectar um dispositivo diretamente à Internet sem um gateway IoT, o gateway permite que os dispositivos se conectem usando muito pouca energia. Ele atua como uma estação intermediária para retenção de dados e gerencia uma conexão com a Internet até o destino final dos dados.

Novos padrões WiFi de baixo consumo são projetados para tornar os gateways IoT menos críticos no futuro, mas eles estão sendo lançados agora. Normalmente, um enxame de dispositivos utilizará muitos gateways IoT, um em cada local físico onde os dispositivos estão presentes (Figura 3-16).

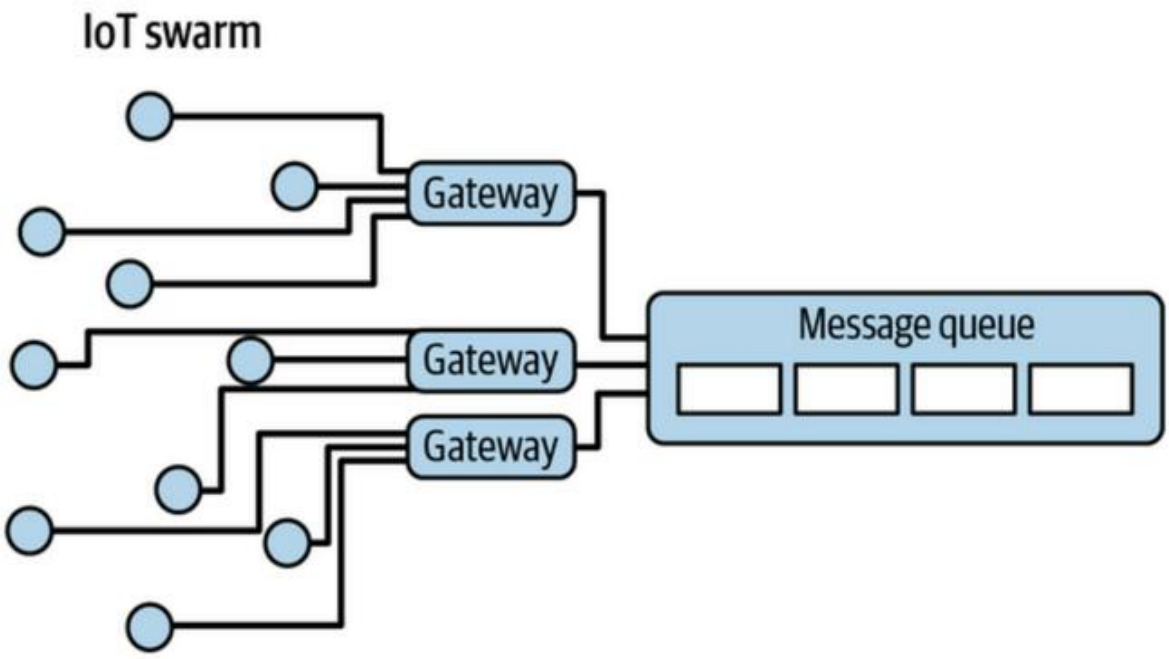


Figura 3-16. Um enxame de dispositivos (círculos), gateways IoT e fila de mensagens com mensagens (retângulos dentro da fila)

Ingestão

Machine Translated by Google

A ingestão começa com um gateway IoT, conforme discutido anteriormente. A partir daí, eventos e medições podem fluir para uma arquitetura de ingestão de eventos.

Claro, outros padrões são possíveis. Por exemplo, o gateway pode acumular dados e carregá-los em lotes para processamento analítico posterior. Em ambientes físicos remotos, os gateways podem não ter conectividade com uma rede na maior parte do tempo. Eles podem fazer o upload de todos os dados somente quando são trazidos ao alcance de uma rede celular ou Wi-Fi. O ponto é que a diversidade de sistemas e ambientes de IoT apresenta complicações – por exemplo, dados atrasados, estrutura de dados e disparidades de esquema, corrupção de dados e interrupção de conexão – que os engenheiros devem considerar em suas arquiteturas e análises downstream.

Armazenar

Os requisitos de armazenamento dependerão muito do requisito de latência para os dispositivos IoT no sistema. Por exemplo, para sensores remotos que coletam dados científicos para análise posterior, o armazenamento de objetos em lote pode ser perfeitamente aceitável. No entanto, respostas quase em tempo real podem ser esperadas de um back-end do sistema que analisa constantemente os dados em uma solução de monitoramento e automação residencial. Nesse caso, uma fila de mensagens ou banco de dados de série temporal é mais apropriado. Discutimos os sistemas de armazenamento com mais detalhes no Capítulo 6.

Servindo

Os padrões de serviço são incrivelmente diversos. Em um aplicativo científico em lote, os dados podem ser analisados usando um data warehouse na nuvem e, em seguida, exibidos em um relatório. Os dados serão apresentados e servidos de várias maneiras em um aplicativo de monitoramento doméstico. Os dados serão analisados em pouco tempo usando um mecanismo de processamento de fluxo ou consultas em um banco de dados de séries temporais para procurar eventos críticos, como incêndio, queda de energia ou arrombamento. A detecção de uma anomalia acionará alertas para o proprietário, o corpo de bombeiros ou outra entidade. Também existe um componente de análise de lote, por exemplo, um relatório mensal sobre o estado da casa.

Machine Translated by Google

Um padrão de serviço significativo para IoT parece ETL reverso (Figura 3-17), embora tendamos a não usar esse termo no contexto de IoT. Pense neste cenário: dados de sensores em dispositivos de fabricação são coletados e analisados. Os resultados dessas medições são processados para buscar otimizações que permitirão que os equipamentos operem com mais eficiência. Os dados são enviados de volta para reconfigurar os dispositivos e otimizá-los.

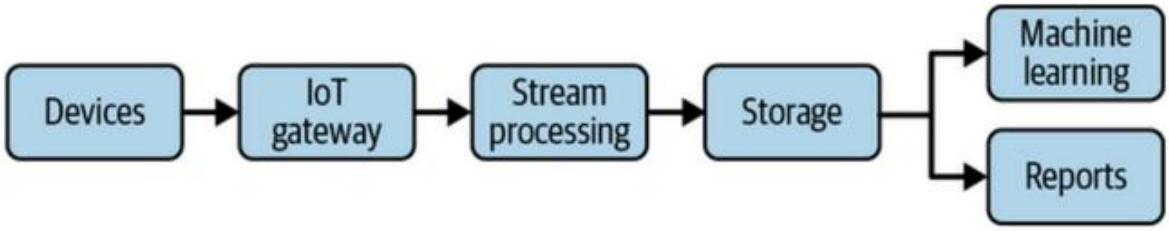


Figura 3-17. Padrão de veiculação de IoT para casos de uso downstream

Arranhando a superfície da IoT

Os cenários de IoT são incrivelmente complexos, e a arquitetura e os sistemas de IoT também são menos familiares aos engenheiros de dados que podem ter passado suas carreiras trabalhando com dados de negócios. Esperamos que esta introdução incentive os engenheiros de dados interessados a aprender mais sobre essa especialização fascinante e em rápida evolução.

Malha de dados

A ***malha de dados*** é uma resposta recente às plataformas de dados monolíticas em expansão, como data lakes e data warehouses centralizados, e "a grande divisão de dados", em que o cenário é dividido entre dados operacionais e dados analíticos.[11] A malha de dados tenta inverter os desafios da arquitetura de dados centralizada, pegando os conceitos de design orientado a domínio (comumente usados em arquiteturas de software) e aplicando-os à arquitetura de dados. Como a malha de dados capturou muita atenção recente, você deve estar ciente disso.

Uma grande parte da malha de dados é a descentralização, como observou Zhamak Dehghani em seu artigo inovador sobre o assunto:

Machine Translated by Google

> ***Para descentralizar a plataforma de dados monolítica, precisamos reverter a forma como pensamos sobre os dados, sua localidade e propriedade. Em vez de fluir os dados dos domínios para um data lake ou plataforma de propriedade central, os domínios precisam hospedar e servir seus conjuntos de dados de domínio de maneira facilmente consumível.***
>
> ***12***

Mais tarde, Dehghani identificou quatro componentes principais da malha de dados: 13

- Propriedade e arquitetura de dados descentralizados orientados ao domínio
- Dados como um produto
- Infraestrutura de dados de autoatendimento como plataforma
- Governança computacional federada

A Figura 3-18 mostra uma versão simplificada de uma arquitetura de malha de dados, com os três domínios interoperando.

Machine Translated by Google

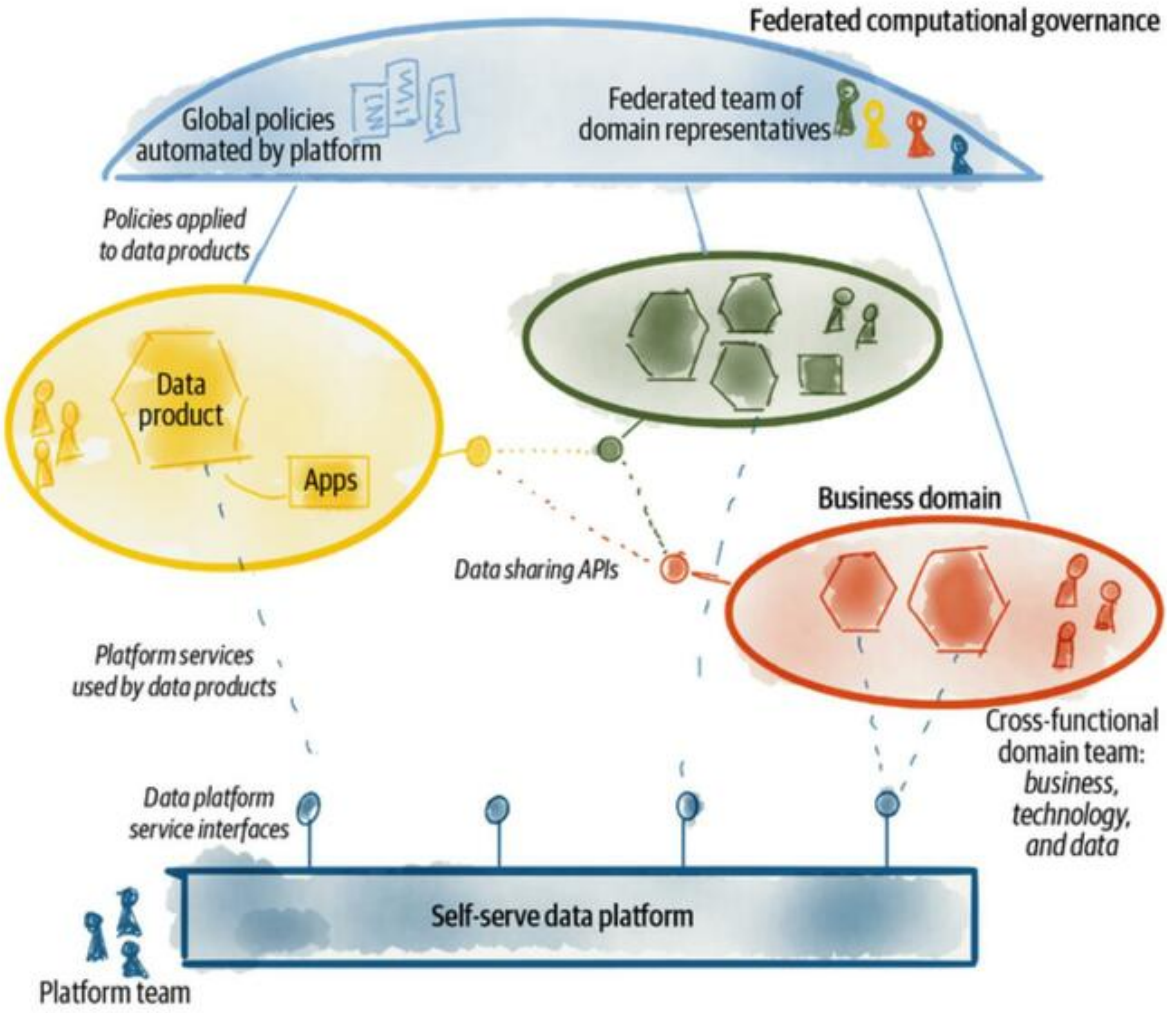


Figura 3-18. Exemplo simplificado de uma arquitetura de malha de dados. Fonte: From Data Mesh, de Zhamak Dehghani. Direitos autorais © 2022 Zhamak Dehghani. Publicado por O'Reilly Media, Inc. Usado com permissão.

Você pode aprender mais sobre malha de dados no livro ***Data Mesh*** de Dehghani (O'Reilly).

Outros exemplos de arquitetura de dados

As arquiteturas de dados têm inúmeras outras variações, como malha de dados, hub de dados, arquitetura em escala, arquitetura de metadados em primeiro lugar, arquitetura orientada a eventos, pilha de dados ao vivo (Capítulo 11) e muito mais. E novas arquiteturas continuarão a surgir à medida que as práticas se consolidam e amadurecem, e as ferramentas simplificam e melhoram. Nós nos concentramos em um punhado de

Machine Translated by Google

padrões de arquitetura de dados mais críticos que estão extremamente bem estabelecidos, evoluindo rapidamente, ou ambos.

Como engenheiro de dados, preste atenção em como as novas arquiteturas podem ajudar sua organização. Fique a par dos novos desenvolvimentos cultivando uma conscientização de alto nível sobre os desenvolvimentos do ecossistema de engenharia de dados. Tenha a mente aberta e não se apegue emocionalmente a uma abordagem. Depois de identificar o valor potencial, aprofunde seu aprendizado e tome decisões concretas. Quando feito corretamente, pequenos ajustes – ou grandes revisões – em sua arquitetura de dados podem impactar positivamente os negócios.

Quem está envolvido com o design de uma arquitetura de dados?

A arquitetura de dados não é projetada no vácuo. Empresas maiores ainda podem empregar arquitetos de dados, mas esses arquitetos precisarão estar fortemente sintonizados e atualizados com o estado da tecnologia e dos dados. Longe vão os dias da arquitetura de dados de torre de marfim. No passado, a arquitetura era em grande parte ortogonal à engenharia. Esperamos que essa distinção desapareça à medida que a engenharia de dados, e a engenharia em geral, evolua rapidamente, tornando-se mais ágil, com menos separação entre engenharia e arquitetura.

Idealmente, um engenheiro de dados trabalhará ao lado de um arquiteto de dados dedicado. No entanto, se uma empresa for pequena ou com baixo nível de maturidade de dados, um engenheiro de dados pode fazer dupla função como arquiteto. Como a arquitetura de dados é uma tendência do ciclo de vida da engenharia de dados, um engenheiro de dados deve entender a arquitetura “boa” e os vários tipos de arquitetura de dados.

Ao projetar a arquitetura, você trabalhará em conjunto com as partes interessadas do negócio para avaliar as compensações. Quais são as vantagens inerentes à adoção de um data warehouse em nuvem versus um data lake? Quais são os trade-offs de várias plataformas de nuvem? Quando uma estrutura unificada de lote/streaming (Beam, Flink) pode ser uma escolha apropriada? Estudar essas escolhas de forma abstrata irá prepará-lo para tomar decisões concretas e valiosas.

Machine Translated by Google

Conclusão

Você aprendeu como a arquitetura de dados se encaixa no ciclo de vida da engenharia de dados e o que torna a arquitetura de dados “boa” e viu vários exemplos de arquiteturas de dados. Como a arquitetura é uma base fundamental para o sucesso, incentivamos você a investir tempo para estudá-la profundamente e entender as vantagens e desvantagens inerentes a qualquer arquitetura. Você estará preparado para mapear a arquitetura que corresponde aos requisitos exclusivos de sua organização.

Em seguida, vamos analisar algumas abordagens para escolher as tecnologias certas a serem usadas na arquitetura de dados e em todo o ciclo de vida da engenharia de dados.

Recursos adicionais

- “Separando Utilidade de Valor Agregado” por Ross Petit
- “Táticas vs. Estratégia: SOA e o Tarpit da Irrelevância” por Neal Ford
- O Corpo de Conhecimento em Gestão da Informação local na rede Internet
- “A pilha de dados moderna: passado, presente e futuro” por Tristan Handy
- “Potemkin Data Science” por Michael Correll
- “Uma comparação de estruturas de processamento de dados” por Ludovik Santos
- “O CI moderno é muito complexo e mal direcionado” por Gregório Szorc
- “Questionando a Arquitetura Lambda” por Jay Kreps
- “Orquestração ETL sem servidor de ponta a ponta na AWS: um guia” por Rittika Jindal
- “Uma breve introdução a duas arquiteturas de processamento de dados – Lambda e Kappa para Big Data” por Iman Samizadeh
- “Como vencer o Teorema do Cap” por Nathan Marz

Machine Translated by Google

“O log: o que todo engenheiro de software deve saber sobre o real
A abstração unificadora dos dados de tempo” por Jay Kreps

“Gerencie sua equipe de dados como uma equipe de produto” por Emilie Schario e Taylor A. Murphy

“Dados como Produto versus Dados como Serviço” por Justin Gage

Artigos de Martin Fowler:

“EagerReadDerivação”

“AnemicDomainModel”

“DomainDrivenDesign”

“Fornecimento de eventos”

“Banco de dados de relatórios”

“BoundedContext”

“Foco em Eventos”

“Persistência Poliglota”

“UtilityVsStrategicDicotomia”

Aplicações de dados:

“Apresentando o Dagster: uma biblioteca Python de código aberto para construção Aplicações de dados” por Nick Schrock

“Abaixo a Dívida Pipeline: Apresentando Grandes Expectativas”, pelo Projeto Grandes Expectativas

“Engenharia de Dados Funcionais: Um Paradigma Moderno para Dados em Lote Processamento” por Maxime Beauchemin

“O que é o ecossistema de dados abertos e por que veio para ficar” por Casber Wang

Machine Translated by Google

“Estar à frente da dívida de dados” por Etai Mizrahi

“Escolhendo a abertura com sabedoria” por Benoit Dageville et al.

“A última lista de verificação de observabilidade de dados” por Molly Vorwerck

“Indo além do lote versus streaming” por David Yaffe

“Desastres que vi em um mundo de microsserviços” por João Alves

Convenções de nomenclatura:

“240 mesas e nenhuma documentação?” por Alexey Makhotkin

“Arquitetura de Data Warehouse: Visão Geral” por Roelant Vos

“O Projeto e Implementação de Sistemas Modernos de Banco de Dados Orientados a Colunas” por Daniel Abadi et al.

“SGBD orientado a colunas” Página da Wikipédia

“A dicotomia de dados: repensando a maneira como tratamos dados e Serviços” por Ben Stopford

“A Maldição do Monstro do Data Lake” por Kiran Prakash e Lucy Câmaras

“Infraestrutura de software 2.0: uma lista de desejos” por Erik Bernhardsson

“Plataforma de Equipe de Dados” por dados do GitLab

“Voltando a se apaixonar por pipelines de dados” por Sean Knapp

Falácia de construção versus compra:

“O que há de errado com MLOps?” por Laszlo Sragner

“Estrutura de Zachman” Página da Wikipédia

“Como o TOGAF define a arquitetura corporativa (EA)” por Avancier Limitado

Projeto EABOK, editado por Paula Hagan

Machine Translated by Google

“O que é arquitetura de dados? Uma estrutura para gerenciar dados” por Thor Olavsrud

"Estrutura de arquitetura do Google Cloud" Página da Web do Google Cloud Architecture

“Centro de Arquitetura Azure” da Microsoft

“Cinco princípios para arquitetura nativa em nuvem: o que é e como Domine-o” de Tom Gray

“Escolhendo a arquitetura certa para distribuição global de dados” Página da Web do Google Cloud Architecture

“Princípios de Engenharia de Dados, Parte I: Visão Geral da Arquitetura” por Hussein dinamarquês

“Uma Implementação Pessoal da Arquitetura de Dados Moderna: Obtendo Dados do Strava no Google Cloud Platform” por Matthew Reeve

“O custo da nuvem, um paradoxo de trilhões de dólares” por Sarah Wang e Martin Casado

Arquitetura de dados: uma cartilha para o cientista de dados por WH Inmon et al. (Imprensa Acadêmica)

“Arquitetura Empresarial” Definição do Glossário Gartner

Site EABOK

Site da estrutura TOGAF

“Definindo Arquitetura” Página da web ISO/IEC/IEEE 42010

“Os Seis Princípios da Arquitetura de Dados Moderna” por Joshua Klahr

“A Ascensão do Lago de Metadados” por Prukalpa

“As 5 principais tendências de dados para os CDOs a serem observados em 2021” por Prukalpa

“Testar a qualidade dos dados em escala com o Deequ” por Dustin Lange et al.

Machine Translated by Google

“O que diabos é malha de dados” por Chris Riccomini

“Troca de dados de microsserviços confiável com o padrão de caixa de saída” por Gunnar Morling

“Quem Precisa de um Arquiteto” por Martin Fowler

“O Papel da Arquitetura Empresarial na Construção de um Data-Driven Organização” por Ashutosh Gupta

“Como construir uma arquitetura de dados para impulsionar a inovação – hoje e amanhã” por Antonio Castro et al.

“Arquitetura de Data Warehouse” tutorial em Javatpoint

Snowflake “O que é arquitetura de data warehouse” página da Internet

“Arquitetura de três camadas” por IBM Educação

“Os blocos de construção de uma plataforma de dados moderna” por Prukalpa

“Arquitetura de Dados: Complexo vs. Complicado” por Dave Wells

“Definição de Data Fabric” por James Serra

“Arquitetura de Data Fabric é a chave para modernizar o gerenciamento e a integração de dados” por Ashutosh Gupta

“Análise unificada: onde o lote e o streaming se unem; SQL e além” Roteiro Apache Flink

“O que é um Data Lakehouse?” por Ben Lorica et al.

“Arquiteturas de Big Data” Documentação do Azure

“Arquitetura de referência do Microsoft Azure IoT” documentação

1 Jeff Haden, “Fundador da Amazon Jeff Bezos: é assim que pessoas bem-sucedidas tomam decisões inteligentes”, ***Inc.,*** 3 de dezembro de 2018, ***https:// oreil.ly/ Qwlm0.***

2 Martin Fowler, “Quem precisa de um arquiteto?” ***Software IEEE,*** julho/agosto de 2003, ***https:// oreil.ly/ wAMmZ.***

Machine Translated by Google

3 “The Bezos API Mandate: Amazon’s Manifesto for Externalization,” Nordic APIs, janeiro 19, 2021, ***https:// oreil.ly/ vIs8m.***

4 Fowler, “Quem precisa de um arquiteto?”

5 Ericka Chickowski, “Leaky Buckets: 10 Worst Amazon S3 Breaches”, Bitdefender ***Business*** Blog de ***insights ,*** 24 de janeiro de 2018, ***https:// oreil.ly/ pFEFO.***

6 “FinOps Foundation sobe para 300 membros e apresenta novos níveis de parceiros para provedores e fornecedores de serviços em nuvem”, Business Wire, 17 de junho de 2019, ***https:// oreil.ly/ XcwYO.***

7 Martin Fowler, “StranglerFigApplication”, 29 de junho de 2004, ***https:// oreil.ly/ PmqxB.***

8 Mike Loukides, “Resume Driven Development,” ***O'Reilly Radar,*** 13 de outubro de 2004, ***https:// oreil.ly/ BUHa8.***

9 HW Inmon, ***Construindo o Data Warehouse*** (Hoboken: Wiley, 2005).

10 Jay Kreps, “Questioning the Lambda Architecture,” O'Reilly, 2 de julho de 2014, ***https:// oreil.ly/ wWR3n.***

11 Martin Fowler, “Princípios de malha de dados e arquitetura lógica”, MartinFowler.com, 3 de dezembro de 2020, ***https:// oreil.ly/ ezWE7.***

12 Zhamak Dehghani, “How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh”, MartinFowler.com, 20 de maio de 2019, ***https:// oreil.ly/ SqMe8.***

13 Zhamak Dehghani, “Princípios de malha de dados”, MartinFowler.com, 3 de dezembro de 2020, ***https:// oreil.ly/ RymDi.***

Machine Translated by Google

Capítulo 4. Escolhendo tecnologias em todo o ciclo de vida da engenharia de dados

A engenharia de dados hoje sofre de um embaraço de riquezas. Não temos escassez de tecnologias para resolver vários tipos de problemas de dados.
As tecnologias de dados estão disponíveis como ofertas prontas e consumíveis em quase todas as formas: código aberto, código aberto gerenciado, software proprietário, serviço proprietário e muito mais. No entanto, é fácil ficar preso na busca por tecnologia de ponta e perder de vista o objetivo central da engenharia de dados: projetar sistemas robustos e confiáveis para transportar dados durante todo o ciclo de vida e atendê-los de acordo com as necessidades dos usuários finais.

Assim como os engenheiros estruturais escolhem cuidadosamente tecnologias e materiais para realizar a visão de um arquiteto para um edifício, os engenheiros de dados têm a tarefa de fazer escolhas tecnológicas apropriadas para orientar os dados ao longo do ciclo de vida para atender aos aplicativos e usuários de dados.

O Capítulo 3 discutiu a “boa” arquitetura de dados e por que ela é importante. Agora explicamos como escolher as tecnologias certas para atender a essa arquitetura. Os engenheiros de dados devem escolher boas tecnologias para criar o melhor produto de dados possível. Achamos que os critérios para escolher uma boa tecnologia de dados são simples: ela agrega valor a um produto de dados e aos negócios mais amplos?

Muita gente confunde arquitetura e ferramentas. A arquitetura é ***estratégica;*** ferramentas são ***táticas.*** Às vezes ouvimos: “Nossa arquitetura de dados são as ferramentas X, Y e Z”. Esta é a maneira errada de pensar sobre arquitetura. Arquitetura é o design de nível superior, roteiro e projeto de sistemas de dados que satisfazem os objetivos estratégicos do negócio. Arquitetura é o ***quê, por que*** e ***quando.***
Ferramentas são usadas para tornar a arquitetura uma realidade; ferramentas são o ***como.***

Muitas vezes vemos equipes saindo dos trilhos e escolhendo tecnologias antes de mapear uma arquitetura. As razões variam: síndrome do objeto brilhante,

Machine Translated by Google

desenvolvimento orientado a currículos e falta de experiência em arquitetura. Na prática, essa priorização da tecnologia geralmente significa que eles montam uma espécie de máquina de fantasia do Dr. Suess em vez de uma verdadeira arquitetura de dados. Desaconselhamos fortemente a escolha de tecnologia antes de acertar sua arquitetura. Arquitetura em primeiro lugar, tecnologia em segundo.

Este capítulo discute nosso plano tático para fazer escolhas de tecnologia assim que tivermos um plano de arquitetura estratégica. A seguir estão algumas considerações para escolher tecnologias de dados em todo o ciclo de vida de engenharia de dados:

Tamanho e capacidades da equipe

Velocidade para o mercado

Interoperabilidade

Otimização de custos e valor comercial

Hoje versus o futuro: tecnologias imutáveis versus transitórias

Localização (nuvem, local, nuvem híbrida, multicloud)

Construir versus comprar

Monólito versus modular

Sem servidor versus servidores

Otimização, desempenho e as guerras de benchmark.

As subcorrentes do ciclo de vida da engenharia de dados

Tamanho e capacidades da equipe

A primeira coisa que você precisa avaliar é o tamanho da sua equipe e suas capacidades com a tecnologia. Você está em uma equipe pequena (talvez uma equipe de uma pessoa) de pessoas que devem usar muitos chapéus, ou a equipe é grande o suficiente para que as pessoas trabalhem em funções especializadas? Um punhado de pessoas será responsável por vários estágios do ciclo de vida da engenharia de dados ou as pessoas cobrem

Machine Translated by Google

nichos específicos? O tamanho da sua equipe influenciará os tipos de tecnologias que você adotar.

Há um continuum de tecnologias simples a complexas, e o tamanho de uma equipe determina aproximadamente a quantidade de largura de banda que sua equipe pode dedicar a soluções complexas. Às vezes, vemos pequenas equipes de dados lendo postagens de blog sobre uma nova tecnologia de ponta em uma empresa de tecnologia gigante e, em seguida, tentam emular essas mesmas tecnologias e práticas extremamente complexas. Chamamos isso ***de engenharia de culto à carga,*** e geralmente é um grande erro que consome muito tempo e dinheiro valiosos, muitas vezes com pouco ou nada para mostrar em troca. Especialmente para equipes pequenas ou com habilidades técnicas mais fracas, use o maior número possível de ferramentas gerenciadas e SaaS e dedique sua largura de banda limitada para resolver os problemas complexos que agregam valor diretamente ao negócio.

Faça um inventário das habilidades de sua equipe. As pessoas se inclinam para ferramentas de baixo código ou preferem abordagens que priorizam o código? As pessoas são fortes em certas linguagens como Java, Python ou Go? As tecnologias estão disponíveis para atender a todas as preferências no espectro de código baixo a código pesado. Novamente, sugerimos manter as tecnologias e fluxos de trabalho com os quais a equipe está familiarizada. Vimos equipes de dados investirem muito tempo no aprendizado da nova e brilhante estrutura de dados, apenas para nunca usá-la em produção. Aprender novas tecnologias, idiomas e ferramentas é um investimento de tempo considerável, portanto, faça esses investimentos com sabedoria.

Velocidade para o mercado

Em tecnologia, a velocidade para o mercado ganha. Isso significa escolher as tecnologias certas que ajudam você a fornecer recursos e dados mais rapidamente, mantendo padrões e segurança de alta qualidade. Isso também significa trabalhar em um ciclo de feedback apertado de lançamento, aprendizado, iteração e melhorias.

O perfeito é inimigo do bom. Algumas equipes de dados deliberarão sobre as opções de tecnologia por meses ou anos sem chegar a nenhuma decisão. Decisões e resultados lentos são o beijo da morte para as equipes de dados. Nós vimos

Machine Translated by Google

mais do que algumas equipes de dados se dissolvem por se moverem muito devagar e por não entregarem o valor que foram contratados para produzir.

Entregue valor cedo e com frequência. Como mencionamos, use o que funciona. Os membros de sua equipe provavelmente obterão uma melhor alavancagem com ferramentas que já conhecem. Evite o trabalho pesado indiferenciado que envolve sua equipe em trabalhos desnecessariamente complexos que agregam pouco ou nenhum valor. Escolha ferramentas que o ajudem a se mover de forma rápida, confiável, segura e protegida.

Interoperabilidade

Raramente você usará apenas uma tecnologia ou sistema. Ao escolher uma tecnologia ou sistema, você precisará garantir que ele interaja e opere com outras tecnologias. ***A interoperabilidade*** descreve como várias tecnologias ou sistemas se conectam, trocam informações e interagem.

Digamos que você esteja avaliando duas tecnologias, A e B. Com que facilidade a tecnologia A se integra à tecnologia B quando se pensa em interoperabilidade? Isso geralmente é um espectro de dificuldade, variando de contínuo a demorado. A integração perfeita já está incorporada em cada produto, facilitando a configuração? Ou você precisa fazer muita configuração manual para integrar essas tecnologias?

Muitas vezes, fornecedores e projetos de código aberto terão como alvo plataformas e sistemas específicos para interoperar. A maioria das ferramentas de ingestão e visualização de dados possui integrações integradas com data warehouses e data lakes populares.
Além disso, ferramentas populares de ingestão de dados se integrarão a APIs e serviços comuns, como CRMs, software de contabilidade e muito mais.

Às vezes, existem padrões para interoperabilidade. Quase todos os bancos de dados permitem conexões via Java Database Connectivity (JDBC) ou Open Database Connectivity (ODBC), o que significa que você pode se conectar facilmente a um banco de dados usando esses padrões. Em outros casos, a interoperabilidade ocorre na ausência de padrões. A transferência de estado representacional (REST) não é realmente um padrão para APIs; cada API REST tem suas peculiaridades. Nesses casos, cabe

Machine Translated by Google

para o fornecedor ou projeto de software de código aberto (OSS) para garantir uma integração suave com outras tecnologias e sistemas.

Esteja sempre ciente de como será simples conectar suas várias tecnologias ao longo do ciclo de vida da engenharia de dados. Conforme mencionado em outros capítulos, sugerimos projetar para modularidade e dar a si mesmo a capacidade de trocar tecnologias facilmente à medida que novas práticas e alternativas se tornam disponíveis.

Otimização de custos e valor comercial

Em um mundo perfeito, você poderia experimentar todas as tecnologias mais recentes e interessantes sem considerar custo, investimento de tempo ou valor agregado ao negócio. Na realidade, os orçamentos e o tempo são finitos, e o custo é uma grande restrição para a escolha das arquiteturas e tecnologias de dados corretas. Sua organização espera um ROI positivo de seus projetos de dados, portanto, você deve entender os custos básicos que pode controlar. A tecnologia é um grande impulsionador de custos, portanto, suas escolhas de tecnologia e estratégias de gerenciamento afetarão significativamente seu orçamento. Analisamos os custos através de três lentes principais: custo total de propriedade, custo de oportunidade e FinOps.

Custo total de propriedade

O custo total de propriedade (TCO) é o custo total estimado de uma iniciativa, incluindo os custos diretos e indiretos dos produtos e serviços utilizados.
Os custos diretos podem ser atribuídos diretamente a uma iniciativa. Exemplos são os salários de uma equipe que trabalha na iniciativa ou a fatura da AWS por todos os serviços consumidos. ***Os custos indiretos,*** também conhecidos como ***despesas gerais,*** são independentes da iniciativa e devem ser pagos independentemente de onde são atribuídos.

Além dos custos diretos e indiretos, ***a forma como algo é comprado*** afeta a forma como os custos são contabilizados. As despesas se dividem em dois grandes grupos: despesas de capital e despesas operacionais.

As despesas de capital, também conhecidas como ***capex,*** exigem um investimento inicial. O pagamento é necessário ***hoje.*** Antes da nuvem existir, as empresas

Machine Translated by Google

normalmente compram hardware e software antecipadamente por meio de grandes contratos de aquisição. Além disso, foram necessários investimentos significativos para hospedar hardware em salas de servidores, data centers e instalações de colocation. Esses investimentos iniciais – geralmente centenas de milhares a milhões de dólares ou mais – seriam tratados como ativos e se depreciariam lentamente ao longo do tempo. Do ponto de vista orçamentário, era necessário capital para financiar toda a compra. Isso é capex, um desembolso de capital significativo com um plano de longo prazo para alcançar um ROI positivo sobre o esforço e as despesas realizadas.

As ***despesas operacionais,*** também conhecidas como ***opex,*** são o oposto do capex em alguns aspectos. Opex é gradual e distribuído ao longo do tempo. Enquanto o capex é focado no longo prazo, o opex é no curto prazo. Opex pode ser pago conforme o uso ou similar e permite muita flexibilidade. Opex está mais próximo de um custo direto, facilitando a atribuição a um projeto de dados.

Até recentemente, opex não era uma opção para grandes projetos de dados. Os sistemas de dados geralmente exigiam contratos multimilionários. Isso mudou com o advento da nuvem, pois os serviços de plataforma de dados permitem que os engenheiros paguem em um modelo baseado em consumo. Em geral, o opex permite uma capacidade muito maior para as equipes de engenharia escolherem seu software e hardware. Os serviços baseados em nuvem permitem que os engenheiros de dados interajam rapidamente com várias configurações de software e tecnologia, geralmente de forma barata.

Os engenheiros de dados precisam ser pragmáticos em relação à flexibilidade. O cenário de dados está mudando muito rapidamente para investir em hardware de longo prazo que inevitavelmente fica obsoleto, não pode ser dimensionado facilmente e potencialmente dificulta a flexibilidade de um engenheiro de dados para experimentar coisas novas. Dada a vantagem de flexibilidade e baixos custos iniciais, pedimos aos engenheiros de dados que adotem uma abordagem opex-first centrada na nuvem e em tecnologias flexíveis de pagamento conforme o uso.

Custo total de propriedade da oportunidade

Qualquer escolha exclui inerentemente outras possibilidades. ***O custo total de oportunidade de propriedade*** (TOCO) é o custo das oportunidades perdidas que incorremos ao escolher uma tecnologia, uma arquitetura ou um processo. Observe que a propriedade nessa configuração não exige compras de hardware ou licenças de longo prazo.

Machine Translated by Google

Mesmo em um ambiente de nuvem, possuímos efetivamente uma tecnologia, uma pilha ou um pipeline, uma vez que se torna uma parte essencial de nossos processos de dados de produção e é difícil sair dele. Os engenheiros de dados geralmente não avaliam o TOCO ao realizar um novo projeto; em nossa opinião, este é um ponto cego enorme.

Se você escolher a pilha de dados A, você escolheu os benefícios da pilha de dados A em vez de todas as outras opções, excluindo efetivamente as pilhas de dados B, C e D. Você está comprometido com a pilha de dados A e tudo o que ela envolve — a equipe para dar suporte isso, treinamento, configuração e manutenção. O que acontece se a pilha de dados A for uma má escolha? O que acontece quando a pilha de dados A se torna obsoleta? Você ainda pode mover para a pilha de dados B?

Quão rápido e barato você pode mudar para algo mais novo e melhor?
Esta é uma questão crítica no espaço de dados, onde novas tecnologias e produtos parecem aparecer em um ritmo cada vez mais rápido. A experiência que você acumulou na pilha de dados A se traduz na próxima onda? Ou você pode trocar os componentes da pilha de dados A e ganhar algum tempo e opções?

O primeiro passo para minimizar o custo de oportunidade é avaliá-lo com os olhos bem abertos. Vimos inúmeras equipes de dados ficarem presas a tecnologias que pareciam boas na época e não são flexíveis para o crescimento futuro ou simplesmente obsoletas. Tecnologias de dados inflexíveis são muito parecidas com armadilhas para ursos.
Eles são fáceis de entrar e extremamente dolorosos de escapar.

FinOps

Já abordamos o FinOps no “Princípio 9: Abrace o FinOps”. Conforme discutimos, os gastos típicos em nuvem são inerentemente opex: as empresas pagam por serviços para executar processos de dados críticos em vez de fazer compras antecipadas e recuperar o valor ao longo do tempo. O objetivo do FinOps é operacionalizar totalmente a responsabilidade financeira e o valor comercial, aplicando as práticas semelhantes ao DevOps de monitoramento e ajuste dinâmico de sistemas.

Neste capítulo, queremos enfatizar uma coisa sobre FinOps que está bem incorporada nesta citação:

Machine Translated by Google

> ***Se parece que FinOps é sobre economizar dinheiro, pense novamente. FinOps é sobre ganhar dinheiro. Os gastos com a nuvem podem gerar mais receita, sinalizar o crescimento da base de clientes, permitir mais velocidade de lançamento de produtos e recursos ou até mesmo ajudar a encerrar um data center.***[2]

Em nosso cenário de engenharia de dados, a capacidade de iterar rapidamente e dimensionar dinamicamente é inestimável para criar valor comercial. Essa é uma das principais motivações para transferir cargas de trabalho de dados para a nuvem.

Hoje versus o futuro: imutável versus Tecnologias Transitórias

Em um domínio empolgante como a engenharia de dados, é muito fácil se concentrar em um futuro em rápida evolução, ignorando as necessidades concretas do presente. A intenção de construir um futuro melhor é nobre, mas muitas vezes leva à superarquitetura e à superengenharia. As ferramentas escolhidas para o futuro podem estar obsoletas e desatualizadas quando esse futuro chegar; o futuro frequentemente se parece pouco com o que imaginamos anos antes.

Como muitos treinadores de vida lhe diriam, concentre-se no presente. Você deve escolher a melhor tecnologia para o momento e o futuro próximo, mas de uma forma que suporte as incógnitas futuras e a evolução. Pergunte a si mesmo: onde você está hoje e quais são seus objetivos para o futuro? Suas respostas a essas perguntas devem informar suas decisões sobre sua arquitetura e, portanto, as tecnologias usadas nessa arquitetura. Isso é feito entendendo o que provavelmente mudará e o que tende a permanecer o mesmo.

Temos duas classes de ferramentas a serem consideradas: imutáveis e transitórias. ***Tecnologias imutáveis*** podem ser componentes que sustentam a nuvem ou linguagens e paradigmas que resistiram ao teste do tempo. Na nuvem, exemplos de tecnologias imutáveis são armazenamento de objetos, rede, servidores e segurança. O armazenamento de objetos, como Amazon S3 e Azure Blob Storage, existirá de hoje até o final da década, e provavelmente por muito mais tempo. Armazenar seus dados no armazenamento de objetos é uma escolha sábia. O armazenamento de objetos continua a melhorar de várias maneiras e oferece constantemente novos

Machine Translated by Google

opções, mas seus dados estarão seguros e utilizáveis no armazenamento de objetos, independentemente da rápida evolução da tecnologia.

Para linguagens, SQL e bash existem há muitas décadas, e não os vemos desaparecendo tão cedo. As tecnologias imutáveis se beneficiam do efeito Lindy: quanto mais tempo uma tecnologia for estabelecida, mais tempo ela será usada. Pense na rede elétrica, nos bancos de dados relacionais, na linguagem de programação C ou na arquitetura do processador x86. Sugerimos aplicar o efeito Lindy como um teste decisivo para determinar se uma tecnologia é potencialmente imutável.

As ***tecnologias transitórias*** são aquelas que vêm e vão. A trajetória típica começa com muito hype, seguido por um crescimento meteórico em popularidade, depois uma lenta descida à obscuridade. O cenário de front-end JavaScript é um exemplo clássico. Quantas estruturas de front-end JavaScript surgiram e desapareceram entre 2010 e 2020? Backbone.js, Ember.js e Knockout eram populares no início dos anos 2010, AngularJS em meados dos anos 2010, e React e Vue.js têm um enorme compartilhamento hoje. Qual é a estrutura de front-end popular daqui a três anos? Quem sabe.

Novos participantes bem financiados e projetos de código aberto chegam à frente de dados todos os dias. Todo fornecedor dirá que seu produto mudará a indústria e "tornará o mundo um lugar melhor". A maioria dessas empresas e projetos não consegue tração de longo prazo e desaparece lentamente na obscuridade. Os principais VCs estão fazendo grandes apostas, sabendo que a maioria de seus investimentos em ferramentas de dados falhará. Como você pode saber em quais tecnologias investir para sua arquitetura de dados? É difícil. Basta considerar o número de tecnologias nas representações (in)famosas de Matt Turck do cenário de ML, IA e dados (MAD) que apresentamos no Capítulo 1 (Figura 4-1).

Machine Translated by Google



Figura 4-1. O cenário de dados MAD de 2021 de Matt Turck

Mesmo tecnologias relativamente bem-sucedidas geralmente desaparecem rapidamente na obscuridade, após alguns anos de rápida adoção, vítimas de seu sucesso. Por exemplo, no início de 2010, o Hive teve uma rápida aceitação porque permitiu que analistas e engenheiros consultassem grandes conjuntos de dados sem codificar manualmente trabalhos complexos do MapReduce. Inspirados pelo sucesso do Hive, mas desejando melhorar suas deficiências, os engenheiros desenvolveram o Presto e outras tecnologias. O Hive agora aparece principalmente em implantações legadas.

Nosso Conselho

Dado o ritmo acelerado das mudanças nas ferramentas e nas melhores práticas, sugerimos avaliar as ferramentas a cada dois anos (Figura 4-2). Sempre que possível, encontre as tecnologias imutáveis ao longo do ciclo de vida da engenharia de dados e use-as como base. Construa ferramentas transitórias em torno dos imutáveis.

Machine Translated by Google

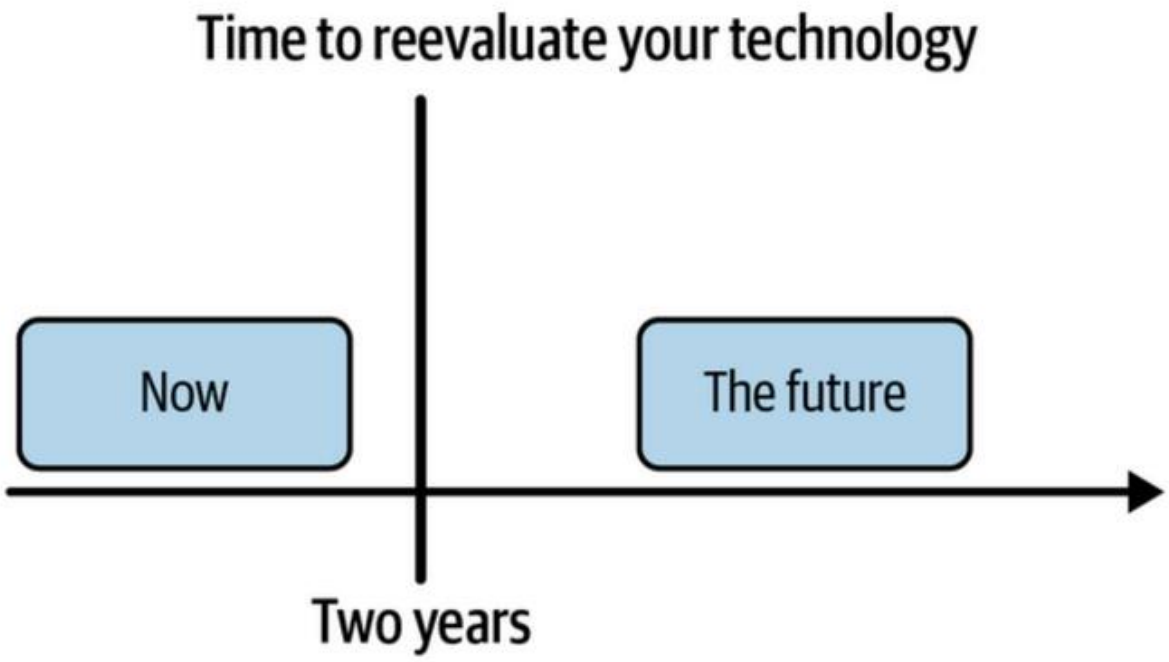


Figura 4-2. Use um horizonte de tempo de dois anos para reavaliar suas escolhas de tecnologia

Dada a probabilidade razoável de falha para muitas tecnologias de dados, você precisa considerar como é fácil fazer a transição de uma tecnologia escolhida.
Quais são as barreiras para sair? Conforme mencionado anteriormente em nossa discussão sobre custo de oportunidade, evite “armadilhas para ursos”. Entre em uma nova tecnologia com os olhos bem abertos, sabendo que o projeto pode ser abandonado, a empresa pode não ser viável ou a tecnologia simplesmente não é mais adequada.

Localização

As empresas agora têm várias opções ao decidir onde executar suas pilhas de tecnologia. Uma lenta mudança em direção à nuvem culmina em uma verdadeira debandada de empresas aumentando as cargas de trabalho na AWS, Azure e Google Cloud Platform (GCP). Na última década, muitos CTOs passaram a ver suas decisões sobre hospedagem de tecnologia como tendo significado existencial para suas organizações. Se eles se movem muito devagar, correm o risco de serem deixados para trás por sua concorrência mais ágil; por outro lado, uma migração para a nuvem mal planejada pode levar a falhas tecnológicas e custos catastróficos.

Vejamos os principais locais para executar sua pilha de tecnologia: local, nuvem, nuvem híbrida e multinuvem.

Machine Translated by Google

Nas instalações

Enquanto novas startups nascem cada vez mais na nuvem, os sistemas locais ainda são o padrão para empresas estabelecidas. Essencialmente, essas empresas são proprietárias de seu hardware, que pode residir em data centers próprios ou em espaço de colocation alugado. Em ambos os casos, as empresas são operacionalmente responsáveis por seu hardware e pelo software que é executado nele. Se o hardware falhar, eles precisam repará-lo ou substituí-lo. Eles também precisam gerenciar os ciclos de atualização a cada poucos anos à medida que o hardware novo e atualizado é lançado e o hardware mais antigo envelhece e se torna menos confiável. Eles devem garantir que tenham hardware suficiente para lidar com picos; para um varejista online, isso significa hospedar capacidade suficiente para lidar com os picos de carga da Black Friday. Para engenheiros de dados encarregados de sistemas locais, isso significa comprar sistemas grandes o suficiente para permitir um bom desempenho para cargas de pico e grandes trabalhos sem compras e gastos excessivos.

Por um lado, as empresas estabelecidas estabeleceram práticas operacionais que as serviram bem. Suponha que uma empresa que depende de tecnologia da informação esteja no mercado há algum tempo. Isso significa que ela conseguiu conciliar os requisitos de custo e pessoal para executar seu hardware, gerenciar ambientes de software, implantar código de equipes de desenvolvimento e executar bancos de dados e sistemas de big data.

Por outro lado, as empresas estabelecidas veem sua concorrência mais jovem e mais ágil escalando rapidamente e aproveitando os serviços gerenciados em nuvem. Eles também veem concorrentes estabelecidos fazendo incursões na nuvem, permitindo que aumentem temporariamente o enorme poder de computação para trabalhos de dados maciços ou o pico de compras da Black Friday.

Empresas em setores competitivos geralmente não têm a opção de ficar paradas. A concorrência é acirrada e sempre existe a ameaça de ser “interrompido” por uma concorrência mais ágil, apoiada por uma grande pilha de dólares de capital de risco. Toda empresa deve manter seus sistemas existentes funcionando de forma eficiente enquanto decide quais ações devem ser feitas em seguida. Isso pode envolver a adoção de práticas de DevOps mais recentes, como contêineres, Kubernetes, microsserviços e implantação contínua, mantendo o hardware

Machine Translated by Google

funcionando no local. Pode envolver uma migração completa para a nuvem, conforme discutido a seguir.

Nuvem

A nuvem inverte o modelo local. Em vez de comprar hardware, você simplesmente aluga hardware e serviços gerenciados de um provedor de nuvem (como AWS, Azure ou Google Cloud). Esses recursos muitas vezes podem ser reservados em prazos extremamente curtos; As VMs são ativadas em menos de um minuto e o uso subsequente é cobrado em incrementos por segundo.
Isso permite que os usuários da nuvem dimensionem dinamicamente recursos que eram inconcebíveis com servidores locais.

Em um ambiente de nuvem, os engenheiros podem lançar projetos e experimentar rapidamente sem se preocupar com o planejamento de hardware de longo prazo. Eles podem começar a executar servidores assim que seu código estiver pronto para implantação. Isso torna o modelo de nuvem extremamente atraente para startups com orçamento e tempo apertados.

A era inicial da nuvem foi dominada por ofertas de infraestrutura como serviço (IaaS) – produtos como VMs e discos virtuais que são essencialmente fatias de hardware alugadas. Lentamente, vimos uma mudança em direção à plataforma como serviço (PaaS), enquanto os produtos SaaS continuam a crescer rapidamente.

PaaS inclui produtos IaaS, mas adiciona serviços gerenciados mais sofisticados para dar suporte a aplicativos. Exemplos são bancos de dados gerenciados, como Amazon Relational Database Service (RDS) e Google Cloud SQL, plataformas de streaming gerenciadas, como Amazon Kinesis e Simple Queue Service (SQS), e Kubernetes gerenciados, como Google Kubernetes Engine (GKE) e Azure Kubernetes Service (AKS).). Os serviços de PaaS permitem que os engenheiros ignorem os detalhes operacionais do gerenciamento de máquinas individuais e da implantação de estruturas em sistemas distribuídos. Eles fornecem acesso pronto para uso a sistemas complexos de escalonamento automático com sobrecarga operacional mínima.

As ofertas de SaaS avançam um degrau adicional na escada da abstração. O SaaS normalmente fornece uma plataforma de software empresarial totalmente funcional com pouco gerenciamento operacional. Exemplos de SaaS incluem Salesforce,

Machine Translated by Google

Google Workspace, Microsoft 365, Zoom e Fivetran. Tanto as principais nuvens públicas quanto terceiros oferecem plataformas SaaS. O SaaS abrange todo um espectro de domínios corporativos, incluindo videoconferência, gerenciamento de dados, tecnologia de anúncios, aplicativos de escritório e sistemas de CRM.

Este capítulo também discute sem servidor, cada vez mais importante nas ofertas de PaaS e SaaS. Os produtos sem servidor geralmente oferecem dimensionamento automatizado de zero a taxas de uso extremamente altas. Eles são cobrados conforme o uso e permitem que os engenheiros operem sem o conhecimento operacional dos servidores subjacentes. Muitas pessoas discutem com o termo ***serverless;*** afinal, o código deve ser executado em algum lugar. Na prática, serverless geralmente significa ***muitos servidores.***

Os serviços em nuvem tornaram-se cada vez mais atraentes para empresas estabelecidas com data centers e infraestrutura de TI existentes. O dimensionamento dinâmico e contínuo é extremamente valioso para empresas que lidam com sazonalidade (por exemplo, empresas de varejo lidando com a carga da Black Friday) e picos de carga de tráfego na web. O advento do COVID-19 em 2020 foi um dos principais impulsionadores da adoção da nuvem, pois as empresas reconheceram o valor de aumentar rapidamente os processos de dados para obter insights em um clima de negócios altamente incerto; as empresas também tiveram que lidar com uma carga substancialmente maior devido a um aumento nas compras on-line, uso de aplicativos da web e trabalho remoto.

Antes de discutirmos as nuances da escolha de tecnologias na nuvem, vamos primeiro discutir por que a migração para a nuvem exige uma mudança drástica de pensamento, especificamente na frente de preços; isso está intimamente relacionado ao FinOps, introduzido em “FinOps”. As empresas que migram para a nuvem geralmente cometem grandes erros de implantação por não adaptarem adequadamente suas práticas ao modelo de precificação da nuvem.

Machine Translated by Google

UM BREVE DESVIO NA ECONOMIA DA NUVEM

Para entender como usar os serviços em nuvem com eficiência por meio da arquitetura nativa da nuvem , você precisa saber como as nuvens geram dinheiro. Este é um conceito extremamente complexo e no qual os provedores de nuvem oferecem pouca transparência. Considere esta barra lateral um ponto de partida para sua pesquisa, descoberta e desenvolvimento de processos.

Serviços de nuvem e swaps de inadimplência de crédito

Vamos tangenciar um pouco os swaps de crédito. Não se preocupe, isso vai fazer sentido daqui a pouco. Lembre-se de que os swaps de inadimplência de crédito chegaram à infâmia após a crise financeira global de 2007. Um credit default swap era um mecanismo para vender diferentes níveis de risco atrelados a um ativo (ou seja, uma hipoteca). Não é nossa intenção apresentar essa ideia em detalhes, mas oferecer uma analogia em que muitos serviços em nuvem são semelhantes a derivados financeiros; os provedores de nuvem não apenas dividem os ativos de hardware em pequenos pedaços por meio da virtualização, mas também vendem esses pedaços com características técnicas variadas e riscos associados. Embora os provedores sejam extremamente discretos sobre os detalhes de seus sistemas internos, existem enormes oportunidades de otimização e dimensionamento ao entender os preços da nuvem e trocar notas com outros usuários.

Veja o exemplo de armazenamento em nuvem de arquivamento. No momento da redação deste artigo, o GCP admite abertamente que seu armazenamento de classe de arquivamento é executado nos mesmos clusters do armazenamento em nuvem padrão, mas o preço por gigabyte por mês de armazenamento de arquivamento é aproximadamente 1/17 do armazenamento padrão. Como isso é possível?

Aqui está o nosso palpite. Ao comprar armazenamento em nuvem, cada disco em um cluster de armazenamento tem três ativos que os provedores e consumidores de nuvem usam. Primeiro, ele tem uma certa capacidade de armazenamento – digamos, 10 TB. Segundo, ele suporta um certo número de operações de entrada/saída (IOPs) por segundo — digamos, 100. Terceiro, os discos suportam uma certa largura de banda máxima, a velocidade máxima de leitura para arquivos organizados de maneira ideal. Uma unidade magnética pode ser capaz de ler a 200 MB/s.

Machine Translated by Google

Qualquer um desses limites (IOPs, capacidade de armazenamento, largura de banda) é um gargalo potencial para um provedor de nuvem. Por exemplo, o provedor de nuvem pode ter um disco armazenando 3 TB de dados, mas atingindo o máximo de IOPs. Uma alternativa para deixar os 7 TB restantes vazios é vender o espaço vazio sem vender IOPs. Ou, mais especificamente, venda espaço de armazenamento barato e IOPs caros para desencorajar as leituras.

Assim como os comerciantes de derivativos financeiros, os fornecedores de nuvem também lidam com riscos. No caso de armazenamento de arquivo, os fornecedores estão vendendo um tipo de seguro, mas que paga para a seguradora e não para o comprador da apólice no caso de uma catástrofe. Embora os custos de armazenamento de dados por mês sejam extremamente baratos, corro o risco de pagar um preço alto se precisar recuperar dados. Mas este é um preço que pagarei com prazer em uma verdadeira emergência.

Considerações semelhantes se aplicam a praticamente qualquer serviço de nuvem. Enquanto os servidores locais são vendidos essencialmente como hardware commodity, o modelo de custo na nuvem é mais sutil. Em vez de apenas cobrar por núcleos de CPU, memória e recursos, os fornecedores de nuvem monetizam características como durabilidade, confiabilidade, longevidade e previsibilidade; uma variedade de plataformas de computação descontam suas ofertas para cargas de trabalho que são efêmeras ou pode ser arbitrariamente interrompido quando a capacidade é necessária em outro lugar.

Nuvem ÿ No Local

Este título pode parecer uma tautologia boba, mas a crença de que os serviços em nuvem são como servidores locais familiares é um erro cognitivo generalizado que assola as migrações de nuvem e leva a contas horríveis. Isso demonstra uma questão mais ampla na tecnologia que chamamos de ***maldição da familiaridade.*** Muitos novos produtos de tecnologia são projetados intencionalmente para se parecerem com algo familiar para facilitar o uso e acelerar a adoção. Mas, qualquer novo produto de tecnologia tem sutilezas e rugas que os usuários devem aprender a identificar, acomodar e otimizar.

Mover servidores locais um por um para VMs na nuvem - conhecido como simples ***elevação e mudança - é*** uma estratégia perfeitamente razoável para a fase inicial da migração para a nuvem, especialmente quando uma empresa está enfrentando alguns problemas

Machine Translated by Google

tipo de precipício financeiro, como a necessidade de assinar uma nova locação significativa ou contrato de hardware se o hardware existente não for desligado. No entanto, as empresas que deixam seus ativos de nuvem nesse estado inicial sofrerão um grande choque. Em uma base de comparação direta, os servidores de longa duração na nuvem são significativamente mais caros do que seus equivalentes locais.

A chave para encontrar valor na nuvem é entender e otimizar o modelo de precificação da nuvem. Em vez de implantar um conjunto de servidores de longa execução capazes de lidar com a carga máxima de pico, use o dimensionamento automático para permitir que as cargas de trabalho sejam reduzidas para infraestrutura mínima quando as cargas são leves e até clusters massivos durante os horários de pico. Para obter descontos por meio de cargas de trabalho mais efêmeras e menos duráveis, use instâncias reservadas ou pontuais ou use funções sem servidor no lugar de servidores.

Muitas vezes pensamos que essa otimização leva a custos mais baixos, mas também devemos nos esforçar para ***aumentar o valor*** do negócio explorando a natureza dinâmica da nuvem. Os engenheiros de dados podem criar novo valor na nuvem realizando coisas que eram impossíveis em seu ambiente local. Por exemplo, é possível ativar rapidamente clusters de computação massivos para executar transformações complexas em escalas inacessíveis para hardware local.

Gravidade de dados

Além de erros básicos, como seguir práticas operacionais locais na nuvem, os engenheiros de dados precisam estar atentos a outros aspectos de preços e incentivos na nuvem que frequentemente pegam usuários
desprevenido.

Os fornecedores querem bloqueá-lo em suas ofertas. Obter dados na plataforma é barato ou gratuito na maioria das plataformas de nuvem, mas obter dados pode ser extremamente caro. Esteja ciente das taxas de saída de dados e seus impactos de longo prazo em seus negócios antes de ser pego de surpresa por uma conta grande. ***A gravidade*** dos dados é real: uma vez que os dados chegam à nuvem, o custo para extraí-los e migrar processos pode ser muito alto.

Machine Translated by Google

Nuvem Híbrida

À medida que empresas mais estabelecidas migram para a nuvem, o modelo de nuvem híbrida ganha importância. Praticamente nenhuma empresa pode migrar todas as suas cargas de trabalho da noite para o dia. O modelo de nuvem híbrida pressupõe que uma organização manterá indefinidamente algumas cargas de trabalho fora da nuvem.

Há muitas razões para considerar um modelo de nuvem híbrida. As organizações podem acreditar que alcançaram a excelência operacional em determinadas áreas, como a pilha de aplicativos e o hardware associado. Assim, eles podem migrar apenas para cargas de trabalho específicas onde veem benefícios imediatos no ambiente de nuvem. Por exemplo, uma pilha Spark local é migrada para clusters de nuvem efêmeros, reduzindo a carga operacional de gerenciamento de software e hardware para a equipe de engenharia de dados e permitindo dimensionamento rápido para grandes trabalhos de dados.

Esse padrão de colocar análises na nuvem é bonito porque os dados fluem principalmente em uma direção, minimizando os custos de saída de dados (Figura 4-3). Ou seja, os aplicativos locais geram dados de eventos que podem ser enviados para a nuvem essencialmente de graça. A maior parte dos dados permanece na nuvem onde é analisada, enquanto quantidades menores de dados são enviadas de volta ao local para implantação de modelos em aplicativos, ETL reverso etc.

Machine Translated by Google

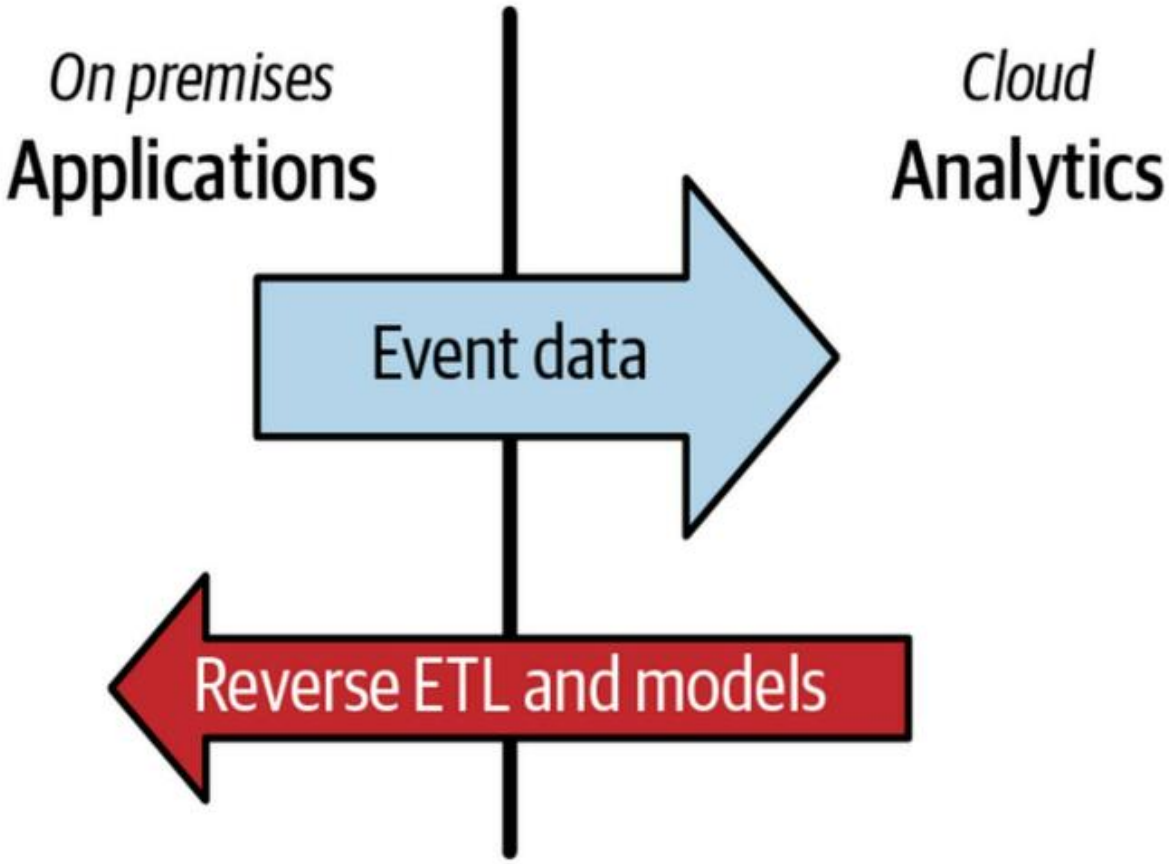


Figura 4-3. Um modelo de fluxo de dados de nuvem híbrida que minimiza os custos de saída

Uma nova geração de ofertas de serviços gerenciados de nuvem híbrida também permite que os clientes localizem servidores gerenciados em nuvem em seus data centers. Isso oferece aos usuários a capacidade de incorporar os melhores recursos em cada nuvem junto com a infraestrutura local.[4]

Multicloud

Multicloud simplesmente se refere à implantação de cargas de trabalho em várias nuvens públicas. As empresas podem ter várias motivações para implantações multicloud. As plataformas SaaS geralmente desejam oferecer serviços próximos às cargas de trabalho existentes na nuvem do cliente. Snowflake e Databricks fornecem suas ofertas de SaaS em várias nuvens por esse motivo. Isso é especialmente crítico para aplicativos com uso intenso de dados, em que a latência da rede e as limitações de largura de banda prejudicam o desempenho e os custos de saída de dados podem ser proibitivos.

Machine Translated by Google

Outra motivação comum para empregar uma abordagem multicloud é aproveitar os melhores serviços em várias nuvens. Por exemplo, uma empresa pode querer lidar com os dados do Google Ads e do Analytics no Google Cloud e implantar o Kubernetes por meio do GKE. E a empresa também pode adotar o Azure especificamente para cargas de trabalho da Microsoft. Além disso, a empresa pode gostar da AWS porque possui vários serviços de primeira classe (por exemplo, AWS Lambda) e desfruta de um enorme compartilhamento de ideias, tornando relativamente fácil contratar engenheiros proficientes em AWS. Qualquer combinação de vários serviços de provedores de nuvem é possível. Dada a intensa concorrência entre os principais provedores de nuvem, espere que eles ofereçam os melhores serviços, tornando a multicloud mais atraente.

Uma metodologia multicloud tem várias desvantagens. Como acabamos de mencionar, os custos de saída de dados e os gargalos de rede são críticos. A adoção da multicloud pode introduzir uma complexidade significativa. As empresas agora precisam gerenciar uma variedade estonteante de serviços em várias nuvens; a integração e a segurança entre nuvens representam um desafio considerável; a rede multicloud pode ser diabolicamente complicada.

Uma nova geração de serviços de “nuvem de nuvens” visa facilitar a multicloud com complexidade reduzida, oferecendo serviços entre nuvens e replicando dados entre nuvens ou gerenciando cargas de trabalho em várias nuvens por meio de um único painel. Para citar um exemplo, uma conta do Snowflake é executada em uma única região de nuvem, mas os clientes podem facilmente ativar outras contas no GCP, AWS ou Azure. O Snowflake fornece replicação de dados agendada simples entre essas várias contas de nuvem. A interface Snowflake é essencialmente a mesma em todas essas contas, removendo a carga de treinamento de alternar entre serviços de dados nativos da nuvem.

O espaço da “nuvem de nuvens” está evoluindo rapidamente; dentro de alguns anos da publicação deste livro, muitos mais desses serviços estarão disponíveis. Engenheiros e arquitetos de dados fariam bem em manter a consciência desse cenário em rápida mudança.

Descentralizado: Blockchain e Edge

Machine Translated by Google

Embora não seja amplamente utilizado agora, vale a pena mencionar brevemente uma nova tendência que pode se tornar popular na próxima década: computação descentralizada. Enquanto os aplicativos de hoje são executados principalmente no local e na nuvem, a ascensão do blockchain, Web 3.0 e computação de ponta pode inverter esse paradigma. No momento, as plataformas descentralizadas provaram ser extremamente populares, mas não tiveram um impacto significativo no espaço de dados; mesmo assim, vale a pena ficar de olho nessas plataformas ao avaliar as decisões de tecnologia.

Nosso Conselho

Do nosso ponto de vista, ainda estamos no início da transição para a nuvem. Assim, as evidências e os argumentos em torno do posicionamento e migração da carga de trabalho estão em constante mudança. A própria nuvem está mudando, com uma mudança do modelo IaaS construído em torno do Amazon EC2 que impulsionou o crescimento inicial da AWS, para ofertas de serviços mais gerenciadas, como AWS Glue, Google BigQuery e Snowflake.

Também vimos o surgimento de novas abstrações de posicionamento de carga de trabalho. Os serviços locais estão se tornando mais parecidos com a nuvem e abstratos. Os serviços de nuvem híbrida permitem que os clientes executem serviços totalmente gerenciados dentro de suas paredes, facilitando a integração entre ambientes locais e remotos. Além disso, a “nuvem de nuvens” está começando a tomar forma, alimentada por serviços de terceiros e fornecedores de nuvem pública.

Escolha tecnologias para o presente, mas olhe para o futuro

Como mencionamos em “Hoje versus o futuro: tecnologias imutáveis versus tecnologias transitórias”, você precisa ficar de olho no presente enquanto planeja o desconhecido. Agora é um momento difícil para planejar posicionamentos e migrações de carga de trabalho. Devido ao ritmo acelerado da concorrência e das mudanças no setor de nuvem, o espaço de decisão será muito diferente em cinco a dez anos. É tentador levar em conta todas as possíveis permutações de arquiteturas futuras.

Acreditamos que é fundamental evitar essa armadilha interminável de análise. Em vez disso, planeje para o presente. Escolha as melhores tecnologias para suas necessidades atuais

Machine Translated by Google

e planos concretos para o futuro próximo. Escolha sua plataforma de implantação com base nas necessidades reais dos negócios, concentrando-se na simplicidade e na flexibilidade.

Em particular, não escolha uma estratégia complexa de multicloud ou de nuvem híbrida, a menos que haja um motivo convincente. Você precisa fornecer dados próximos aos clientes em várias nuvens? As regulamentações do setor exigem que você aloje determinados dados em seus data centers? Você tem uma necessidade de tecnologia atraente para serviços específicos em duas nuvens diferentes? Escolha uma estratégia de implantação de nuvem única se esses cenários não se aplicarem a você.

Por outro lado, tenha um plano de fuga. Como enfatizamos antes, toda tecnologia – mesmo software de código aberto – vem com algum grau de bloqueio. Uma estratégia de nuvem única tem vantagens significativas de simplicidade e integração, mas vem com um bloqueio significativo anexado. Neste caso, estamos falando de flexibilidade mental, a flexibilidade de avaliar o estado atual do mundo e imaginar alternativas. Idealmente, seu plano de fuga permanecerá trancado atrás do vidro, mas preparar esse plano o ajudará a tomar melhores decisões no presente e lhe dará uma saída se as coisas derem errado no futuro.

Argumentos de repatriação na nuvem

Enquanto escrevíamos este livro, Sarah Wang e Martin Casado publicaram “The Cost of Cloud, A Trillion Dollar Paradox”, um artigo que gerou som e fúria significativos no espaço tecnológico. Os leitores interpretaram amplamente o artigo como um pedido de repatriação de cargas de trabalho na nuvem para servidores locais. Eles apresentam um argumento um pouco mais sutil de que as empresas devem gastar recursos significativos para controlar os gastos na nuvem e devem considerar a repatriação como uma opção possível.

Queremos ter um momento para dissecar uma parte de sua discussão. Wang e Casado citam a repatriação do Dropbox de cargas de trabalho significativas da AWS para servidores de propriedade do Dropbox como um estudo de caso para empresas que consideram movimentos de repatriação semelhantes.

Você não é Dropbox, nem é Cloudflare

Machine Translated by Google

Acreditamos que este estudo de caso é frequentemente usado sem contexto apropriado e é um exemplo convincente da falácia lógica da ***falsa equivalência .***
O Dropbox fornece serviços específicos em que a propriedade de hardware e data centers pode oferecer uma vantagem competitiva. As empresas não devem confiar excessivamente no exemplo do Dropbox ao avaliar as opções de implantação na nuvem e no local.

Primeiro, é importante entender que o Dropbox armazena enormes quantidades de dados. A empresa está de boca fechada sobre exatamente quantos dados hospeda, mas diz que são muitos exabytes e continua a crescer.

Em segundo lugar, o Dropbox lida com uma grande quantidade de tráfego de rede. Sabemos que seu consumo de largura de banda em 2017 foi significativo o suficiente para a empresa adicionar “centenas de gigabits de conectividade de internet com provedores de trânsito (ISPs regionais e globais) e centenas de novos parceiros de peering (onde trocamos tráfego diretamente em vez de um ISP).).” Os custos de saída de dados seriam extremamente altos em um ambiente de nuvem pública.[5]

Terceiro, o Dropbox é essencialmente um fornecedor de armazenamento em nuvem, mas com um produto de armazenamento altamente especializado que combina características de armazenamento em bloco e objeto. A competência central do Dropbox é um sistema diferencial de atualização de arquivos que pode sincronizar com eficiência arquivos editados ativamente entre os usuários, minimizando o uso da rede e da CPU. O produto não é adequado para armazenamento de objetos, armazenamento em bloco ou outras ofertas de nuvem padrão. Em vez disso, o Dropbox se beneficiou da construção de uma pilha de software e hardware personalizada e altamente integrada [6] .

Quarto, enquanto o Dropbox transferiu seu produto principal para o hardware, continuou desenvolvendo outras cargas de trabalho da AWS. Isso permite que o Dropbox se concentre na criação de um serviço de nuvem altamente ajustado em uma escala extraordinária, em vez de tentar substituir vários serviços. O Dropbox pode se concentrar em sua competência central em armazenamento em nuvem e sincronização de dados, enquanto descarrega software e gerenciamento de hardware em outras áreas, como análise de dados.[7]

Outras histórias de sucesso frequentemente citadas que as empresas construíram fora da nuvem incluem Backblaze e Cloudflare, mas oferecem lições semelhantes.
Backblaze começou a vida como um produto de backup de dados em nuvem pessoal, mas desde então

Machine Translated by Google

passou a oferecer B2, um serviço de armazenamento de objetos semelhante ao Amazon S3. Backblaze atualmente armazena mais de um exabyte de dados. Cloudflare afirma fornecer serviços para mais de 25 milhões de propriedades de internet, com pontos de presença em mais de 200 cidades e 51 terabits por segundo (Tbps) de capacidade total da rede.

A Netflix oferece mais um exemplo útil. A empresa é famosa por executar sua pilha de tecnologia na AWS, mas isso é apenas parcialmente verdade. A Netflix executa a transcodificação de vídeo na AWS, respondendo por aproximadamente 70% de suas necessidades de computação em 2017. A Netflix também executa seu back-end de aplicativos e análise de dados na AWS. No entanto, em vez de usar a rede de distribuição de conteúdo da AWS, a Netflix criou uma CDN personalizada em colaboração com provedores de serviços de Internet, utilizando uma combinação altamente especializada de software e hardware. Para uma empresa que consome uma fatia substancial de todo o tráfego da Internet, a construção dessa infraestrutura crítica permitiu que ela entregasse vídeo de alta qualidade a uma enorme base de clientes de maneira econômica.

Esses estudos de caso sugerem que faz sentido que as empresas gerenciem seu próprio hardware e conexões de rede em circunstâncias específicas.
As maiores histórias de sucesso modernas de empresas que criam e mantêm hardware envolvem escala extraordinária (exabytes de dados, terabits por segundo de largura de banda etc.) e casos de uso limitados em que as empresas podem obter uma vantagem competitiva ao projetar pilhas de hardware e software altamente integradas. Além disso, todas essas empresas consomem largura de banda de rede massiva, sugerindo que as cobranças de saída de dados seriam um grande custo se optassem por operar totalmente a partir de uma nuvem pública.

Considere continuar a executar cargas de trabalho no local ou repatriar cargas de trabalho na nuvem se você executar um serviço realmente em escala de nuvem. O que é escala de nuvem? Você pode estar em escala de nuvem se estiver armazenando um exabyte de dados ou manipulando terabits por segundo de tráfego ***de e para a Internet.*** (Atingir um terabit por segundo de tráfego de rede ***interno*** é bastante fácil.) Além disso, considere a propriedade de seus servidores se os custos de saída de dados forem um fator importante para seus negócios. Para dar um exemplo concreto de cargas de trabalho em escala de nuvem que podem se beneficiar da repatriação, a Apple pode obter uma vantagem financeira e de desempenho significativa ao migrar o armazenamento do iCloud para seus próprios servidores.

Machine Translated by Google

Construir versus comprar

Construir versus comprar é um debate antigo em tecnologia. O argumento para construir é que você tem controle de ponta a ponta sobre a solução e não está à mercê de um fornecedor ou comunidade de código aberto. O argumento que apoia a compra se resume a restrições de recursos e experiência; você tem a experiência para construir uma solução melhor do que algo já disponível? Qualquer uma das decisões se resume ao TCO, TOCO e se a solução oferece uma vantagem competitiva para sua organização.

Se você captou um tema do livro até agora, sugerimos investir na construção e customização , ***pois isso proporcionará uma vantagem competitiva*** para o seu negócio. Caso contrário, suba nos ombros de gigantes e ***use o que já está disponível*** no mercado. Dado o número de serviços de código aberto e pagos – os quais podem ter comunidades de voluntários ou equipes altamente pagas de engenheiros incríveis – você é tolo em construir tudo sozinho.

Como costumamos perguntar: “Quando você precisa de pneus novos para o seu carro, você obtém as matérias-primas, constrói os pneus do zero e os instala você mesmo?” Como a maioria das pessoas, você provavelmente está comprando pneus e mandando alguém instalá-los. O mesmo argumento se aplica a construir versus comprar. Vimos equipes que construíram seus bancos de dados do zero. Um simples RDBMS de código aberto teria servido muito melhor às suas necessidades após uma inspeção mais detalhada. Imagine a quantidade de tempo e dinheiro investidos neste banco de dados local. Fale sobre baixo ROI para TCO e custo de oportunidade.

É aqui que a distinção entre o engenheiro de dados tipo A e tipo B é útil. Como apontamos anteriormente, as funções do tipo A e do tipo B geralmente são incorporadas ao mesmo engenheiro, especialmente em uma pequena organização.
Sempre que possível, incline-se para o comportamento do tipo A; evite o trabalho pesado indiferenciado e abrace a abstração. Use estruturas de código aberto ou, se isso for muito problemático, procure comprar uma solução gerenciada ou proprietária adequada. Muitos ótimos serviços modulares estão disponíveis para escolha em ambos os casos.

Machine Translated by Google

Vale a pena mencionar a realidade mutável de como as empresas adotam o software. Enquanto no passado a TI costumava tomar a maioria das decisões de compra e adoção de software de forma top-down, hoje em dia, a tendência é para a adoção de software bottom-up em uma empresa, impulsionada por desenvolvedores, engenheiros de dados, cientistas de dados, e outras funções técnicas. A adoção de tecnologia dentro das empresas está se tornando um processo orgânico e contínuo.

Vejamos algumas opções para soluções de código aberto e proprietárias.

Software livre

O software de código aberto (OSS) é um modelo de distribuição de software no qual o software e a base de código subjacente são disponibilizados para uso geral, geralmente sob termos de licenciamento específicos. Muitas vezes o OSS é criado e mantido por uma equipe distribuída de colaboradores. O OSS é livre para usar, alterar e distribuir na maioria das vezes, mas com ressalvas específicas. Por exemplo, muitas licenças exigem que o código-fonte do software derivado de código aberto seja incluído quando o software for distribuído.

As motivações para criar e manter OSS variam. Às vezes, o OSS é orgânico, surgindo da mente de um indivíduo ou de uma pequena equipe que cria uma nova solução e opta por liberá-la para uso público. Outras vezes, uma empresa pode disponibilizar uma ferramenta ou tecnologia específica ao público sob uma licença de OSS.

O OSS tem dois sabores principais: OSS gerenciado pela comunidade e OSS comercial.

OSS gerenciados pela comunidade Os

projetos OSS são bem-sucedidos com uma comunidade forte e uma base de usuários vibrante. ***O OSS gerenciado pela comunidade*** é um caminho predominante para projetos de OSS. A comunidade abre altas taxas de inovações e contribuições de desenvolvedores em todo o mundo com projetos populares de OSS.

A seguir estão os fatores a serem considerados em um projeto de OSS gerenciado pela comunidade:

Mindshare

Machine Translated by Google

Evite adotar projetos OSS que não tenham tração e popularidade.
Veja o número de estrelas, bifurcações e volume de commits do GitHub e
recente. Outra coisa a se prestar atenção é a atividade da comunidade em
grupos de bate-papo e fóruns relacionados. O projeto tem um forte senso de comunidade? Uma
comunidade forte cria um ciclo virtuoso de adoção forte. Isso também significa que você terá mais
facilidade em obter informações técnicas
assistência e encontrar talentos qualificados para trabalhar com o framework.

Maturidade

Há quanto tempo o projeto existe, quão ativo está hoje e quão
utilizável estão as pessoas encontrando-o em produção? A maturidade de um projeto
indica que as pessoas o consideram útil e estão dispostas a incorporá-lo em seus fluxos de
trabalho de produção.

Solução de problemas

Como você terá que lidar com os problemas se eles surgirem? Você está sozinho para
solucionar problemas ou a comunidade pode ajudá-lo a resolver seu problema?

Gerenciamento de Projetos

Veja os problemas do Git e a maneira como eles são abordados. Eles são abordados
rapidamente? Em caso afirmativo, qual é o processo para enviar um problema e resolvê-lo?

Equipe

Uma empresa está patrocinando o projeto OSS? Quem é o núcleo
contribuintes?

Relacionamento com desenvolvedores e gerenciamento de comunidade

Machine Translated by Google

O que o projeto está fazendo para incentivar a aceitação e a adoção? Existe uma comunidade Slack vibrante que oferece incentivo e apoio?

Contribuindo

O projeto incentiva e aceita pull requests?

Roteiro

Existe um roteiro de projeto? Se sim, é claro e transparente?

Auto-hospedagem e manutenção

Você tem os recursos para hospedar e manter a solução OSS? Se for assim, qual é o TCO e o TOCO versus comprar um serviço gerenciado do Fornecedor de OSS?

Retribuindo à comunidade

Se você gosta do projeto e o está usando ativamente, considere investir nele. Você pode contribuir com a base de código, ajudar a corrigir problemas e dar conselhos nos fóruns e bate-papos da comunidade. Se o projeto permitir doações, considere fazer um. Muitos projetos OSS são essencialmente comunitários projetos de serviço, e os mantenedores geralmente têm empregos em tempo integral, além de ajudar no projeto OSS. Infelizmente, muitas vezes é um trabalho de amor que não dá ao mantenedor um salário digno. Se você pode dar ao luxo de doe, por favor, faça isso.

OSS comercial

Às vezes, o OSS tem algumas desvantagens. Ou seja, você precisa hospedar e manter a solução em seu ambiente. Isso pode ser trivial ou extremamente complicado e incômodo, dependendo da aplicação do OSS.

Os fornecedores comerciais tentam resolver essa dor de cabeça de gerenciamento hospedando e

Machine Translated by Google

gerenciando a solução OSS para você, normalmente como uma oferta de SaaS em nuvem. Exemplos de tais fornecedores incluem Databricks (Spark), Confluent (Kafka), DBT Labs (dbt) e há muitos, muitos outros.

Este modelo é chamado ***de OSS comercial*** (COSS). Normalmente, um fornecedor oferecerá o "núcleo" do OSS gratuitamente enquanto cobra por melhorias, distribuições de código com curadoria ou serviços totalmente gerenciados.

Um fornecedor geralmente é afiliado ao projeto OSS da comunidade. À medida que um projeto OSS se torna mais popular, os mantenedores podem criar um negócio separado para uma versão gerenciada do OSS. Isso normalmente se torna uma plataforma SaaS em nuvem construída em torno de uma versão gerenciada do código-fonte aberto.
Esta é uma tendência generalizada: um projeto OSS se torna popular, uma empresa afiliada levanta caminhões de dinheiro de capital de risco (VC) para comercializar o projeto OSS, e a empresa cresce como um foguete em movimento rápido.

Neste ponto, o engenheiro de dados tem duas opções. Você pode continuar usando a versão OSS gerenciada pela comunidade, que precisa continuar mantendo por conta própria (atualizações, manutenção de servidor/contêiner, solicitações de pull para correções de bugs, etc.). Ou você pode pagar ao fornecedor e deixar que ele cuide da gestão administrativa do produto COSS.

A seguir estão os fatores a serem considerados com um projeto OSS comercial:

Valor

O fornecedor está oferecendo um valor melhor do que se você mesmo gerenciasse a tecnologia OSS? Alguns fornecedores adicionarão muitos sinos e assobios para suas ofertas gerenciadas que não estão disponíveis no OSS da comunidade versão. Essas adições são atraentes para você?

Modelo de entrega

Como você acessa o serviço? O produto está disponível via download, API ou IU da Web/móvel? Certifique-se de que você pode acessar facilmente a versão inicial e as versões subsequentes.

Machine Translated by Google

Apoiar

O suporte não pode ser subestimado e muitas vezes é opaco para o comprador. o que é o modelo de suporte para o produto, e há um custo extra para Apoio, suporte? Frequentemente, os fornecedores vendem suporte por uma taxa adicional. Certifique-se de entender claramente os custos de obter suporte. Além disso, entenda o que é coberto no suporte e o que não é coberto.

Qualquer coisa que não seja coberta pelo suporte será de sua responsabilidade possuir e administrar.

Lançamentos e correções de bugs

O fornecedor é transparente sobre o cronograma de lançamento, melhorias e correções de bugs? Essas atualizações estão facilmente disponíveis para você?

Ciclo de vendas e preços

Muitas vezes, um fornecedor oferecerá preços sob demanda, especialmente para um SaaS produto e oferecer um desconto se você se comprometer com um contrato estendido. Certifique-se de entender as compensações de pagar à medida que avança versus pagar adiantado. Vale a pena pagar uma quantia fixa, ou é o seu dinheiro melhor gasto em outro lugar?

Finanças da empresa

A empresa é viável? Se a empresa levantou fundos de VC, você pode verifique seu financiamento em sites como Crunchbase. Quanta pista faz a empresa tem, e ainda estará no mercado em alguns anos?

Logos versus receita

Machine Translated by Google

A empresa está focada em aumentar o número de clientes (logotipos) ou está tentando aumentar a receita? Você pode se surpreender com o número de empresas preocupadas principalmente em aumentar o número de clientes, estrelas do GitHub ou os Clubes dos canais do Slack sem a receita para estabelecer finanças sólidas.

Suporte da comunidade

A empresa está realmente apoiando a versão comunitária do projeto OSS? Quanto a empresa está contribuindo para a base de código OSS da comunidade? Surgiram controvérsias com certos fornecedores cooptando projetos OSS e, posteriormente, fornecendo pouco valor de volta ao comunidade. Qual a probabilidade de o produto permanecer viável como um código aberto suportado pela comunidade se a empresa fechar?

Observe também que as nuvens oferecem seus próprios produtos de código aberto gerenciados. Se um fornecedor de nuvem vê tração com um determinado produto ou projeto, espere que esse fornecedor ofereça sua versão. Isso pode variar de exemplos simples (Linux de código aberto oferecido em VMs) a serviços gerenciados extremamente complexos (Kafka totalmente gerenciado). A motivação para essas ofertas é simples: as nuvens ganham dinheiro por meio do consumo. Mais ofertas em um ecossistema de nuvem significam uma chance maior de “aderência” e aumento dos gastos do cliente.

Jardins Murados Proprietários

Embora o OSS seja onipresente, também existe um grande mercado para tecnologias não OSS. Algumas das maiores empresas do setor de dados vendem produtos de código fechado. Vejamos dois tipos principais de ***jardins murados proprietários,*** empresas independentes e ofertas de plataforma em nuvem.

Ofertas independentes

Machine Translated by Google

O cenário de ferramentas de dados tem visto um crescimento exponencial nos últimos anos. Todos os dias, surgem novas ofertas independentes para ferramentas de dados. Com a capacidade de arrecadar fundos de VCs com capital, essas empresas de dados podem escalar e contratar ótimas equipes de engenharia, vendas e marketing. Isso apresenta uma situação em que os usuários têm ótimas opções de produtos no mercado, ao mesmo tempo em que precisam percorrer intermináveis vendas e confusão de marketing.

Muitas vezes, uma empresa que vende uma ferramenta de dados não a lança como OSS, oferecendo uma solução proprietária. Embora você não tenha a transparência de uma solução OSS pura, uma solução proprietária independente pode funcionar muito bem, especialmente como um serviço totalmente gerenciado na nuvem.

A seguir estão coisas a serem consideradas com uma oferta independente:

Interoperabilidade

Certifique-se de que a ferramenta interopere com outras ferramentas que você escolheu (OSS, outros independentes, ofertas de nuvem, etc.) A interoperabilidade é fundamental, então certifique-se de que você pode experimentá-lo antes de comprar.

Mindshare e market share

A solução é popular? Ele comanda uma presença no Mercado? Ele gosta de avaliações positivas dos clientes?

Documentação e suporte

Problemas e dúvidas surgirão inevitavelmente. Está claro como resolver seu problema, seja através de documentação ou suporte?

Preços

O preço é compreensível? Mapeie baixo, médio e alto cenários de utilização de probabilidade, com os respectivos custos. Você é capaz de negociar um contrato, juntamente com um desconto? Vale a pena? Quanta flexibilidade você perde se assinar um contrato, tanto na negociação quanto na

Machine Translated by Google

capacidade de experimentar novas opções? Você é capaz de obter compromissos contratuais sobre preços futuros?

Longevidade

A empresa sobreviverá tempo suficiente para que você obtenha valor de seu produto? Se a empresa arrecadou dinheiro, procure seu financiamento situação. Veja as avaliações dos usuários. Pergunte aos amigos e poste perguntas nas redes sociais redes sobre as experiências de outros usuários com o produto. Certifique-se de saber no que está se metendo.

Ofertas de serviços proprietários da plataforma em nuvem

Os fornecedores de nuvem desenvolvem e vendem seus serviços proprietários para armazenamento, bancos de dados e muito mais. Muitas dessas soluções são ferramentas internas usadas pelas respectivas empresas irmãs. Por exemplo, a Amazon criou o banco de dados DynamoDB para superar as limitações dos bancos de dados relacionais tradicionais e lidar com grandes quantidades de dados de usuários e pedidos à medida que a Amazon.com se tornava um gigante. Mais tarde, a Amazon ofereceu o serviço DynamoDB exclusivamente na AWS; agora é um produto de primeira linha usado por empresas de todos os tamanhos e níveis de maturidade. Os fornecedores de nuvem geralmente agrupam seus produtos para que funcionem bem juntos. Cada nuvem pode criar aderência com sua base de usuários criando um forte ecossistema integrado.

A seguir estão os fatores a serem considerados com uma oferta de nuvem proprietária:

Comparações de desempenho versus preço

A oferta de nuvem é substancialmente melhor do que uma versão independente ou OSS? Qual é o TCO de escolher uma oferta de nuvem?

Considerações de compra

O preço sob demanda pode ser caro. Você pode reduzir seu custo comprando capacidade reservada ou firmando um compromisso de longo prazo

Machine Translated by Google

acordo?

Nosso Conselho

Construir versus comprar volta a conhecer sua vantagem competitiva e onde faz sentido investir recursos em customização. Em geral, favorecemos OSS e COSS por padrão, o que libera você para se concentrar em melhorar as áreas onde essas opções são insuficientes. Concentre-se em algumas áreas onde construir algo irá agregar valor significativo ou reduzir substancialmente o atrito.

Não trate a sobrecarga operacional interna como um custo irrecuperável. Há um excelente valor em aprimorar sua equipe de dados existente para criar sistemas sofisticados em plataformas gerenciadas, em vez de cuidar de servidores locais. Além disso, pense em como uma empresa ganha dinheiro, especialmente suas equipes de vendas e experiência do cliente, que geralmente indicam como você é tratado durante o ciclo de vendas e quando é um cliente pagante.

Por fim, quem é responsável pelo orçamento da sua empresa? Como essa pessoa decide os projetos e tecnologias que serão financiados? Antes de fazer o business case para COSS ou serviços gerenciados, faz sentido tentar usar o OSS primeiro? A última coisa que você deseja é que sua escolha de tecnologia fique presa no limbo enquanto aguarda a aprovação do orçamento. Como diz o velho ditado, o ***tempo mata os negócios.*** No seu caso, mais tempo gasto no limbo significa uma maior probabilidade de que a aprovação do seu orçamento morra. Saiba de antemão quem controla o orçamento e o que será aprovado com sucesso.

Monólito Versus Modular

Monólitos versus sistemas modulares é outro debate de longa data no espaço de arquitetura de software. Os sistemas monolíticos são autocontidos, muitas vezes executando várias funções em um único sistema. O acampamento do monólito favorece a simplicidade de ter tudo em um só lugar. É mais fácil raciocinar sobre uma única entidade e você pode se mover mais rápido porque há menos partes móveis. O campo ***modular*** se inclina para desacoplado, o melhor da raça

Machine Translated by Google

tecnologias que executam tarefas nas quais são excepcionalmente excelentes. Especialmente considerando a taxa de mudança nos produtos no mundo dos dados, o argumento é que você deve buscar a interoperabilidade entre um conjunto de soluções em constante mudança.

Que abordagem você deve adotar em sua pilha de engenharia de dados? Vamos explorar os trade-offs.

Monólito

O ***monólito*** (Figura 4-4) tem sido um pilar da tecnologia por décadas. Os velhos tempos da cascata significavam que os lançamentos de software eram enormes, fortemente acoplados e se moviam em uma cadência lenta. Grandes equipes trabalharam juntas para entregar uma única base de código funcional. Os sistemas de dados monolíticos continuam até hoje, com fornecedores de software mais antigos, como a Informatica, e estruturas de código aberto, como o Spark.

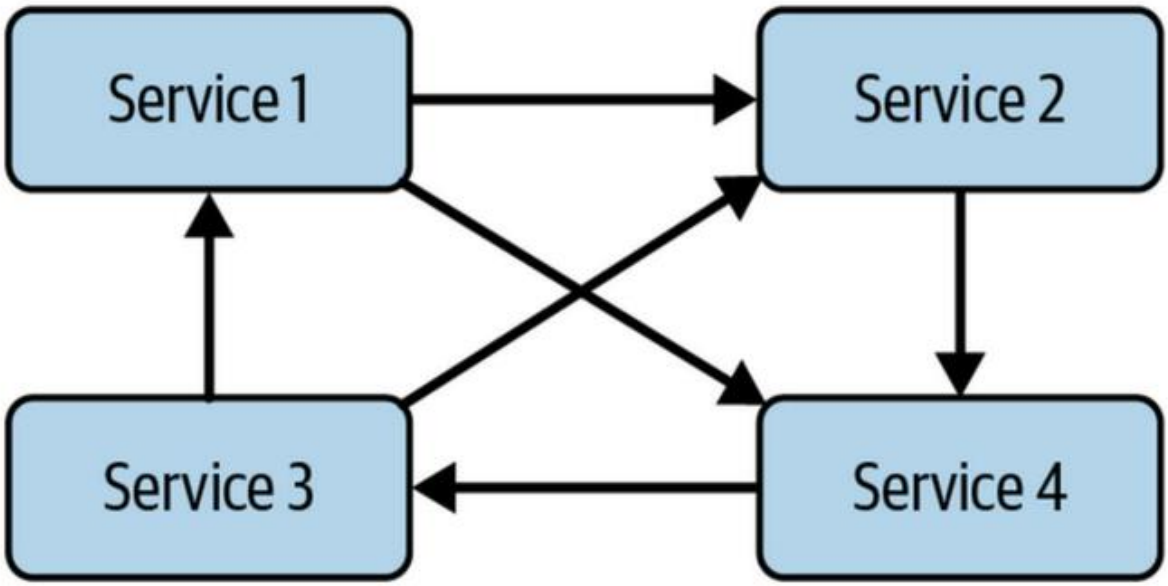


Figura 4-4. O monólito acopla firmemente seus serviços

As vantagens do monolito são a facilidade de raciocínio, e requer uma carga cognitiva menor e mudança de contexto, já que tudo é autocontido. Em vez de lidar com dezenas de tecnologias, você lida com “uma” tecnologia e normalmente uma linguagem de programação principal.
Os monólitos são uma excelente opção se você deseja simplicidade no raciocínio sobre sua arquitetura e processos.

Machine Translated by Google

Claro, o monólito tem contras. Por um lado, os monólitos são frágeis. Por causa do grande número de peças móveis, atualizações e lançamentos demoram mais e tendem a assar “na pia da cozinha”. Se o sistema tiver um bug — esperamos que o software tenha sido exaustivamente testado antes do lançamento! — isso pode prejudicar todo o sistema.

Problemas induzidos pelo usuário também acontecem com monólitos. Por exemplo, vimos um pipeline ETL monolítico que levou 48 horas para ser executado. Se algo quebrasse em qualquer lugar do pipeline, todo o processo teria que ser reiniciado. Enquanto isso, usuários de negócios ansiosos aguardavam seus relatórios, que já estavam com dois dias de atraso por padrão. As rupturas eram comuns o suficiente para que o sistema monolítico fosse eventualmente descartado.

A multilocação em um sistema monolítico também pode ser um problema significativo. Pode ser um desafio isolar as cargas de trabalho de vários usuários. Em um data warehouse local, uma função definida pelo usuário pode consumir CPU suficiente para reduzir a velocidade do sistema para outros usuários. Conflitos entre dependências e contenção de recursos são fontes frequentes de dores de cabeça.

Outro contra dos monólitos é que mudar para um novo sistema será doloroso se o fornecedor ou o projeto de código aberto morrer. Como todos os seus processos estão contidos no monólito, sair desse sistema e entrar em uma nova plataforma será caro em tempo e dinheiro.

Modularidade

A modularidade (Figura 4-5) é um conceito antigo na engenharia de software, mas os sistemas distribuídos modulares realmente entraram em voga com o surgimento dos microsserviços. Em vez de depender de um monólito maciço para lidar com suas necessidades, por que não separar sistemas e processos em suas áreas de interesse independentes? Os microsserviços podem se comunicar por meio de APIs, permitindo que os desenvolvedores se concentrem em seus domínios enquanto tornam seus aplicativos acessíveis a outros microsserviços. Essa é a tendência na engenharia de software e é cada vez mais vista em sistemas de dados modernos.

Machine Translated by Google



Figura 4-5. Com modularidade, cada serviço é desacoplado de outro

As principais empresas de tecnologia têm sido os principais impulsionadores do movimento de microsserviços. O famoso mandato da API Bezos diminui o acoplamento entre aplicativos, permitindo refatoração e decomposição. Bezos também impôs a regra de duas pizzas (nenhuma equipe deve ser tão grande que duas pizzas não possam alimentar todo o grupo). Efetivamente, isso significa que uma equipe terá no máximo cinco membros. Esse limite também limita a complexidade do domínio de responsabilidade de uma equipe — em particular, a base de código que ela pode gerenciar. Enquanto um aplicativo monolítico extenso pode envolver um grupo de cem pessoas, dividir os desenvolvedores em pequenos grupos de cinco exige que esse aplicativo seja dividido em partes pequenas, gerenciáveis e fracamente acopladas.

Em um ambiente de microsserviço modular, os componentes são intercambiáveis e é possível criar um aplicativo ***poliglota*** (linguagem de multiprogramação); um serviço Java pode substituir um serviço escrito em Python. Os clientes do serviço precisam se preocupar apenas com as especificações técnicas da API do serviço, não com os detalhes dos bastidores da implementação.

As tecnologias de processamento de dados mudaram para um modelo modular, fornecendo forte suporte à interoperabilidade. Os dados são armazenados no armazenamento de objetos em um formato padrão, como Parquet em data lakes e lakehouses. Qualquer ferramenta de processamento que suporte o formato pode ler os dados e gravar os resultados processados de volta no lago para processamento por outra ferramenta. Os data warehouses em nuvem oferecem suporte à interoperação com armazenamento de objetos por meio de importação/exportação usando formatos padrão e tabelas externas, ou seja, consultas executadas diretamente em dados em um data lake.

Novas tecnologias entram em cena em um ritmo vertiginoso no ecossistema de dados de hoje, e a maioria fica obsoleta e ultrapassada rapidamente. Enxague e repita. A capacidade de trocar ferramentas à medida que a tecnologia muda é inestimável. Vemos a modularidade de dados como um paradigma mais poderoso do que a engenharia de dados monolítica.
A modularidade permite que os engenheiros escolham a melhor tecnologia para cada trabalho ou passo ao longo do pipeline.

Machine Translated by Google

Os contras da modularidade são que há mais motivos para raciocinar. Em vez de lidar com um único sistema de preocupação, agora você tem potencialmente inúmeros sistemas para entender e operar. A interoperabilidade é uma dor de cabeça potencial; espero que todos esses sistemas funcionem bem juntos.

Esse mesmo problema nos levou a quebrar a orquestração como uma subcorrente separada, em vez de colocá-la sob gerenciamento de dados. A orquestração também é importante para arquiteturas de dados monolíticas; testemunhe o sucesso de ferramentas como o Control-M da BMC Software no espaço tradicional de armazenamento de dados. Mas orquestrar cinco ou dez ferramentas é dramaticamente mais complexo do que orquestrar uma. A orquestração se torna a cola que une os módulos da pilha de dados.

O Padrão de Monólito Distribuído

O ***padrão monolítico distribuído*** é uma arquitetura distribuída que ainda sofre de muitas das limitações da arquitetura monolítica. A ideia básica é que se execute um sistema distribuído com diferentes serviços para realizar diferentes tarefas. Ainda assim, serviços e nós compartilham um conjunto comum de dependências ou uma base de código comum.

Um exemplo padrão é um cluster Hadoop tradicional. Um cluster Hadoop pode hospedar simultaneamente várias estruturas, como Hive, Pig ou Spark. O cluster também tem muitas dependências internas. Além disso, o cluster executa os componentes principais do Hadoop: bibliotecas comuns do Hadoop, HDFS, YARN e Java. Na prática, um cluster geralmente tem uma versão de cada componente instalada.

Um sistema Hadoop padrão no local envolve o gerenciamento de um ambiente comum que funciona para todos os usuários e todos os trabalhos. Gerenciar atualizações e instalações é um desafio significativo. Forçar tarefas a atualizar dependências corre o risco de quebrá-las; manter duas versões de um framework envolve complexidade extra.

Algumas tecnologias modernas de orquestração baseadas em Python também sofrem com esse problema. Embora utilizem uma arquitetura altamente desacoplada e assíncrona, todos os serviços executam a mesma base de código com o mesmo

Machine Translated by Google

dependências. Qualquer executor pode executar qualquer tarefa, portanto, uma biblioteca cliente para uma única tarefa executada em um DAG deve ser instalada em todo o cluster.
A orquestração de muitas ferramentas envolve a instalação de bibliotecas de cliente para uma série de APIs. Os conflitos de dependência são um problema constante.

Uma solução para os problemas do monólito distribuído é a infraestrutura efêmera em um ambiente de nuvem. Cada trabalho obtém seu próprio servidor temporário ou cluster instalado com dependências. Cada cluster permanece altamente monolítico, mas separar tarefas reduz drasticamente os conflitos. Por exemplo, esse padrão agora é bastante comum para Spark com serviços como Amazon EMR e Google Cloud Dataproc.

Uma segunda solução é decompor adequadamente o monólito distribuído em vários ambientes de software usando contêineres. Temos mais a dizer sobre contêineres em “Serverless Versus Servers”.

Nosso Conselho

Embora os monólitos sejam atraentes devido à facilidade de compreensão e complexidade reduzida, isso tem um alto custo. O custo é a perda potencial de flexibilidade, custo de oportunidade e ciclos de desenvolvimento de alto atrito.

Aqui estão algumas coisas a serem consideradas ao avaliar monólitos versus opções modulares:

Interoperabilidade

Arquiteto para compartilhamento e interoperabilidade.

Evitando a “armadilha do urso”

Algo que é fácil de entrar pode ser doloroso ou impossível de escapar.

Flexibilidade

As coisas estão se movendo tão rápido no espaço de dados agora. Comprometer-se com um monólito reduz a flexibilidade e as decisões reversíveis.

Machine Translated by Google

Sem servidor versus servidores

Uma grande tendência para provedores de nuvem é a ***ausência de servidor,*** permitindo que desenvolvedores e engenheiros de dados executem aplicativos sem gerenciar servidores nos bastidores. O Serverless fornece um tempo rápido de valor para os casos de uso corretos. Para outros casos, pode não ser um bom ajuste. Vejamos como avaliar se o serverless é adequado para você.

Sem servidor

Embora o serverless já exista há algum tempo, a tendência serverless começou com força total com o AWS Lambda em 2014. Com a promessa de executar pequenos pedaços de código conforme necessário sem ter que gerenciar um servidor, o serverless explodiu em popularidade . As principais razões para sua popularidade são o custo e a conveniência. Em vez de pagar o custo de um servidor, por que não pagar apenas quando seu código for evocado?

Serverless tem muitos sabores. Embora a função como serviço (FaaS) seja muito popular, os sistemas sem servidor são anteriores ao advento do AWS Lambda. O BigQuery do Google Cloud não tem servidor, pois os engenheiros de dados não precisam gerenciar a infraestrutura de back-end, e o sistema é dimensionado para zero e dimensionado automaticamente para lidar com grandes consultas. Basta carregar os dados no sistema e começar a consultar. Você paga pela quantidade de dados que sua consulta consome e um pequeno custo para armazenar seus dados. Esse modelo de pagamento – pagando pelo consumo e armazenamento – está se tornando mais prevalente.

Quando o serverless faz sentido? Tal como acontece com muitos outros serviços em nuvem, depende; e os engenheiros de dados fariam bem em entender os detalhes dos preços da nuvem para prever quando as implantações sem servidor se tornarão caras. Olhando especificamente para o caso do AWS Lambda, vários engenheiros descobriram hacks para executar cargas de trabalho em lote a custos baixos. Por outro lado, as funções sem servidor sofrem de uma ineficiência de sobrecarga inerente. Manipular um evento por chamada de função em uma alta taxa de eventos pode ser catastroficamente caro.

Machine Translated by Google

Assim como em outras áreas de operações, é fundamental monitorar e modelar. ***Monitore*** para determinar o custo por evento em um ambiente real e ***modele*** usando esse custo por evento para determinar os custos gerais à medida que as taxas de eventos aumentam. A modelagem também deve incluir os piores cenários – o que acontece se meu site for atingido por um enxame de bots ou ataque DDoS?

Recipientes

Em conjunto com serverless e microsserviços, os ***contêineres*** são uma das tecnologias operacionais de tendência mais poderosas até o momento.
Os contêineres desempenham um papel em microsserviços e sem servidor.

Os contêineres geralmente são chamados de ***máquinas virtuais leves.*** Enquanto uma VM tradicional envolve um sistema operacional inteiro, um contêiner empacota um espaço de usuário isolado (como um sistema de arquivos e alguns processos); muitos desses contêineres podem coexistir em um único sistema operacional de host. Isso fornece alguns dos principais benefícios da virtualização (ou seja, dependência e isolamento de código) sem a sobrecarga de carregar um kernel de sistema operacional inteiro.

Um único nó de hardware pode hospedar vários contêineres com alocações de recursos refinadas. No momento da redação deste artigo, os contêineres continuam crescendo em popularidade, juntamente com o Kubernetes, um sistema de gerenciamento de contêineres. Ambientes sem servidor normalmente são executados em contêineres nos bastidores.
De fato, o Kubernetes é um tipo de ambiente sem servidor porque permite que desenvolvedores e equipes de operações implantem microsserviços sem se preocupar com os detalhes das máquinas em que estão implantados.

Os contêineres fornecem uma solução parcial para os problemas do monólito distribuído mencionado anteriormente neste capítulo. Por exemplo, o Hadoop agora oferece suporte a contêineres, permitindo que cada trabalho tenha suas próprias dependências isoladas.

Machine Translated by Google

> **AVISO**
>
> Os clusters de contêiner não fornecem a mesma segurança e isolamento que as VMs completas oferecem. A fuga de ***contêiner*** – amplamente, uma classe de explorações em que o código em um contêiner ganha privilégios fora do contêiner no nível do sistema operacional – é comum o suficiente para ser considerado um risco para multilocação. Embora o Amazon EC2 seja um ambiente verdadeiramente multilocatário com VMs de muitos clientes hospedadas no mesmo hardware, um cluster Kubernetes deve hospedar código apenas em um ambiente de confiança mútua (por exemplo, dentro dos muros de uma única empresa). Além disso, os processos de revisão de código e a verificação de vulnerabilidades são essenciais para garantir que um desenvolvedor não introduza uma falha de segurança.

Vários tipos de plataformas de contêiner adicionam recursos sem servidor adicionais. As plataformas de função em contêineres executam contêineres como unidades efêmeras acionadas por eventos em vez de serviços persistentes.[12] Isso oferece aos usuários a simplicidade do AWS Lambda com total flexibilidade de um ambiente de contêiner, em vez do tempo de execução altamente restritivo do Lambda. E serviços como AWS Fargate e Google App Engine executam contêineres sem gerenciar um cluster de computação necessário para Kubernetes. Esses serviços também isolam totalmente os contêineres, evitando os problemas de segurança associados à multilocação.

A abstração continuará trabalhando na pilha de dados. Considere o impacto do Kubernetes no gerenciamento de cluster. Embora você possa gerenciar seu cluster Kubernetes - e muitas equipes de engenharia o fazem - até mesmo o Kubernetes está amplamente disponível como um serviço gerenciado.

Quando a infraestrutura faz sentido

Por que você deseja executar seus próprios servidores em vez de usar sem servidor? Há algumas razões. O custo é um grande fator. Sem servidor faz menos sentido quando o uso e o custo excedem o custo contínuo de execução e manutenção de um servidor (Figura 4-6). No entanto, em certa escala, os benefícios econômicos do serverless podem diminuir e a execução de servidores se torna mais atraente.

Machine Translated by Google

Figura 4-6. Custo sem servidor versus utilizando um servidor

Personalização, poder e controle são outras razões principais para favorecer servidores em vez de servidores sem servidor. Algumas estruturas sem servidor podem ser fracas ou limitadas para determinados casos de uso. Aqui estão algumas coisas a serem consideradas ao usar servidores, principalmente na nuvem, onde os recursos do servidor são efêmeros:

Espere que os servidores falhem.

A falha do servidor acontecerá. Evite usar um servidor “floco de neve especial” que seja excessivamente personalizado e quebradiço, pois isso introduz um vulnerabilidade em sua arquitetura. Em vez disso, trate os servidores como efêmeros recursos que você pode criar conforme necessário e, em seguida, excluir. Se seu aplicativo exigir a instalação de um código específico no servidor, use um script de inicialização ou crie uma imagem. Implante o código no servidor por meio de um Pipeline CI/CD.

Use clusters e escalonamento automático.

Machine Translated by Google

Aproveite a capacidade da nuvem de aumentar e reduzir os recursos de computação sob demanda. À medida que seu aplicativo aumenta seu uso, o cluster seus servidores de aplicativos e use recursos de escalonamento automático para dimensione horizontalmente automaticamente seu aplicativo conforme a demanda cresce.

Trate sua infraestrutura como código.

A automação não se aplica apenas a servidores e deve se estender ao seu infraestrutura sempre que possível. Implante sua infraestrutura (servidores ou não) usando gerenciadores de implantação como Terraform, AWS CloudFormation e Google Cloud Deployment Manager.

Use recipientes.

Para cargas de trabalho mais sofisticadas ou pesadas com dependências instaladas complexas, considere o uso de contêineres em um único servidor ou Kubernetes.

Nosso Conselho

Aqui estão algumas considerações importantes para ajudá-lo a determinar se o serverless é ideal para você:

Tamanho e complexidade da carga de trabalho

O Serverless funciona melhor para tarefas e cargas de trabalho simples e discretas. Não é adequado se você tiver muitas partes móveis ou precisar de muita computação ou potência de memória. Nesse caso, considere usar contêineres e um estrutura de orquestração de fluxo de trabalho de contêiner como Kubernetes.

Frequência e duração da execução

Machine Translated by Google

Quantas solicitações por segundo seu aplicativo sem servidor processará?

Quanto tempo cada solicitação levará para ser processada? Plataformas sem servidor em nuvem têm limites na frequência de execução, simultaneidade e duração. Se seu aplicativo não pode funcionar perfeitamente dentro desses limites, é hora de considerar uma abordagem orientada a contêiner.

Pedidos e networking

As plataformas sem servidor geralmente utilizam alguma forma de rede simplificada e não oferecem suporte a todos os recursos de rede virtual na nuvem, como VPCs e firewalls.

Linguagem

Qual idioma você costuma usar? Se não for um dos idiomas oficialmente suportados pela plataforma sem servidor, você deve considere recipientes em vez disso.

Limitações de tempo de execução

Plataformas sem servidor não oferecem sistema operacional completo abstrações. Em vez disso, você está limitado a uma imagem de tempo de execução específica.

Custo

As funções sem servidor são incrivelmente convenientes, mas potencialmente caro. Quando sua função serverless processa apenas alguns eventos, seus custos são baixos; os custos aumentam rapidamente à medida que a contagem de eventos aumenta. Esse cenário é uma fonte frequente de contas de nuvem surpresa.

No final, a abstração tende a vencer. Sugerimos usar primeiro sem servidor e depois servidores - com contêineres e orquestração, se possível - uma vez

Machine Translated by Google

você superou as opções sem servidor.

Otimização, Desempenho e o Guerras de referência

Imagine que você é um bilionário comprando novos meios de transporte. Você reduziu sua escolha a duas opções:

Jato Executivo 787

- Alcance: 9.945 milhas náuticas (com 25 passageiros)
- Velocidade máxima: 0,90 Mach
- Velocidade de cruzeiro: 0,85 Mach
- Capacidade de combustível: 101.323 kg
- Peso máximo de decolagem: 227.930 kg
- Empuxo máximo: 128.000 libras

Xadrez Tesla Model S

- Alcance: 560 quilômetros
- Velocidade máxima: 322 quilômetros/hora
- 0–-100 quilômetros/hora: 2,1 segundos
- Capacidade da bateria: 100 quilowatts-hora
- Tempo de volta em Nurburgring: 7 minutos e 30,9 segundos
- Potência: 1020
- Torque: 1050 lb-ft

Qual dessas opções oferece melhor desempenho? Você não precisa saber muito sobre carros ou aeronaves para reconhecer que esta é uma comparação idiota.

Machine Translated by Google

Uma opção é um jato particular de fuselagem larga projetado para operação intercontinental, enquanto a outra é um supercarro elétrico.

Vemos essas comparações de maçãs com laranjas feitas o tempo todo no espaço do banco de dados. Os benchmarks comparam bancos de dados otimizados para casos de uso completamente diferentes ou usam cenários de teste que não têm nenhuma semelhança com as necessidades do mundo real.

Recentemente, vimos uma nova rodada de guerras de benchmark entre os principais fornecedores no espaço de dados. Aplaudimos os benchmarks e estamos felizes em ver muitos fornecedores de banco de dados finalmente eliminando as cláusulas DeWitt de seus contratos com clientes. Mesmo assim, cuidado com o comprador: o espaço de dados está cheio de benchmarks sem sentido. Aqui estão alguns truques comuns usados para colocar o polegar na escala de referência.[13][14]

Big Data... para os anos 90

Os produtos que alegam oferecer suporte a “big data” em escala de petabytes geralmente usam conjuntos de dados de referência pequenos o suficiente para caber facilmente no armazenamento do seu smartphone. Para sistemas que dependem de camadas de cache para fornecer desempenho, os conjuntos de dados de teste residem totalmente na unidade de estado sólido (SSD) ou na memória, e os benchmarks podem mostrar desempenho ultra-alto ao consultar repetidamente os mesmos dados. Um pequeno conjunto de dados de teste também minimiza os custos de RAM e SSD ao comparar preços.

Para comparar casos de uso do mundo real, você deve simular dados do mundo real e tamanho de consulta previstos. Avalie o desempenho da consulta e os custos dos recursos com base em uma avaliação detalhada de suas necessidades.

Comparações de custos sem sentido

As comparações de custos sem sentido são um truque padrão ao analisar um preço/desempenho ou TCO. Por exemplo, muitos sistemas MPP não podem ser criados e excluídos prontamente, mesmo quando residem em um ambiente de nuvem; esses sistemas são executados por anos a fio depois de configurados. Outros bancos de dados oferecem suporte a um modelo de computação dinâmico e cobram por consulta ou por segundo de uso. Comparando sistemas efêmeros e não efêmeros em um

Machine Translated by Google

base de custo por segundo não faz sentido, mas vemos isso o tempo todo em benchmarks.

Otimização assimétrica

O engano da otimização assimétrica aparece em muitas formas, mas aqui está um exemplo. Muitas vezes, um fornecedor compara um sistema MPP baseado em linha com um banco de dados colunar usando um benchmark que executa consultas de junção complexas em dados altamente normalizados. O modelo de dados normalizado é ideal para o sistema baseado em linhas, mas o sistema colunar realizaria todo o seu potencial apenas com algumas alterações de esquema. Para piorar as coisas, os fornecedores complementam seus sistemas com uma dose extra de otimização de junção (por exemplo, junções pré-indexação) sem aplicar ajustes comparáveis no banco de dados concorrente (por exemplo, colocando junções em uma visualização materializada).

Advertência Emptor

Tal como acontece com todas as coisas na tecnologia de dados, o comprador deve ficar atento. Faça sua lição de casa antes de confiar cegamente em benchmarks de fornecedores para avaliar e escolher a tecnologia.

Subcorrentes e seus impactos sobre Escolhendo tecnologias

Conforme visto neste capítulo, um engenheiro de dados tem muito a considerar ao avaliar tecnologias. Seja qual for a tecnologia que você escolher, certifique-se de entender como ela suporta as correntes ocultas do ciclo de vida da engenharia de dados. Vamos revisá-los brevemente novamente.

Gestão de dados

O gerenciamento de dados é uma área ampla e, em relação às tecnologias, nem sempre é aparente se uma tecnologia adota o gerenciamento de dados como uma preocupação principal. Por exemplo, nos bastidores, um fornecedor terceirizado pode

Machine Translated by Google

use as melhores práticas de gerenciamento de dados — conformidade regulatória, segurança, privacidade, qualidade de dados e governança — mas oculte esses detalhes por trás de uma camada de interface do usuário limitada. Nesse caso, ao avaliar o produto, ajuda perguntar à empresa sobre suas práticas de gerenciamento de dados. Aqui estão alguns exemplos de perguntas que você deve fazer:

> Como você está protegendo os dados contra violações, tanto de fora quanto de dentro?
>
> Qual é a conformidade do seu produto com GDPR, CCPA e outros regulamentos de privacidade de dados?
>
> Você me permite hospedar meus dados para cumprir esses regulamentos?
>
> Como você garante a qualidade dos dados e que estou visualizando os dados corretos em sua solução?

Há muitas outras perguntas a serem feitas, e essas são apenas algumas das maneiras de pensar sobre o gerenciamento de dados no que se refere à escolha das tecnologias certas. Essas mesmas perguntas também devem se aplicar às soluções de OSS que você está considerando.

DataOps

Problemas vão acontecer. Eles simplesmente vão. Um servidor ou banco de dados pode morrer, a região de uma nuvem pode ter uma interrupção, você pode implantar código de buggy, dados ruins podem ser introduzidos em seu data warehouse e outros problemas imprevistos podem ocorrer.

Ao avaliar uma nova tecnologia, quanto controle você tem sobre a implantação de um novo código, como você será alertado se houver um problema e como você responderá quando houver um problema? A resposta depende muito do tipo de tecnologia que você está considerando. Se a tecnologia for OSS, você provavelmente será responsável por configurar o monitoramento, hospedagem e implantação de código.
Como você vai lidar com os problemas? Qual é a sua resposta ao incidente?

Muitas das operações estão fora de seu controle se você estiver usando uma oferta gerenciada. Considere o SLA do fornecedor, a maneira como eles alertam você sobre problemas e

Machine Translated by Google

se eles são transparentes sobre como estão abordando o caso, incluindo fornecer um ETA para uma correção.

Arquitetura de dados

Conforme discutido no Capítulo 3, uma boa arquitetura de dados significa avaliar as compensações e escolher as melhores ferramentas para o trabalho, mantendo suas decisões reversíveis. Com o cenário de dados se transformando em velocidade de distorção, a ***melhor ferramenta*** para o trabalho é um alvo em movimento. Os principais objetivos são evitar bloqueios desnecessários, garantir a interoperabilidade em toda a pilha de dados e produzir alto ROI. Escolha suas tecnologias de acordo.

Exemplo de orquestração: fluxo de ar

Ao longo da maior parte deste capítulo, evitamos ativamente discutir qualquer tecnologia em particular muito extensivamente. Abrimos uma exceção para orquestração porque o espaço é atualmente dominado por uma tecnologia de código aberto, Apache Airflow.

Maxime Beauchemin iniciou o projeto Airflow no Airbnb em 2014.
O Airflow foi desenvolvido desde o início como um projeto de código aberto não comercial. A estrutura rapidamente cresceu significativamente fora do Airbnb, tornando-se um projeto Apache Incubator em 2016 e um projeto completo patrocinado pelo Apache em 2019.

O Airflow desfruta de muitas vantagens, em grande parte devido à sua posição dominante no mercado de código aberto. Primeiro, o projeto de código aberto Airflow é extremamente ativo, com uma alta taxa de commits e um tempo de resposta rápido para bugs e problemas de segurança, e o projeto lançou recentemente o Airflow 2, uma grande refatoração da base de código. Em segundo lugar, o Airflow desfruta de um enorme compartilhamento de ideias.
O Airflow tem uma comunidade vibrante e ativa em muitas plataformas de comunicação, incluindo Slack, Stack Overflow e GitHub. Os usuários podem encontrar facilmente respostas para perguntas e problemas. Terceiro, o Airflow está disponível comercialmente como um serviço gerenciado ou distribuição de software por meio de muitos fornecedores, incluindo GCP, AWS e Astronomer.io.

Machine Translated by Google

O fluxo de ar também tem algumas desvantagens. O Airflow depende de alguns componentes não escalonáveis principais (o agendador e o banco de dados de back-end) que podem se tornar gargalos para desempenho, escalabilidade e confiabilidade; as partes escaláveis do Airflow ainda seguem um padrão de monólito distribuído. (Consulte “Monolith Versus Modular”.) Por fim, o Airflow não tem suporte para muitas construções nativas de dados, como gerenciamento de esquema, linhagem e catalogação; e é um desafio desenvolver e testar fluxos de trabalho do Airflow.

Não tentamos uma discussão exaustiva das alternativas do Airflow aqui, mas apenas mencionamos alguns dos principais candidatos à orquestração no momento da redação. Prefect e Dagster visam resolver alguns dos problemas discutidos anteriormente repensando os componentes da arquitetura Airflow. Haverá outras estruturas e tecnologias de orquestração não discutidas aqui?
Planeje-se.

É altamente recomendável que qualquer pessoa que escolha uma tecnologia de orquestração estude as opções discutidas aqui. Eles também devem se familiarizar com a atividade no espaço, pois novos desenvolvimentos certamente ocorrerão no momento em que você ler isso.

Engenharia de software

Como engenheiro de dados, você deve se esforçar para simplificar e abstrair toda a pilha de dados. Compre ou use soluções de código aberto pré-criadas sempre que possível. Eliminar o trabalho pesado indiferenciado deve ser seu grande objetivo. Concentre seus recursos - codificação e ferramentas personalizadas - em áreas que oferecem uma vantagem competitiva sólida. Por exemplo, a codificação manual de uma conexão de banco de dados entre seu banco de dados de produção e seu data warehouse na nuvem é uma vantagem competitiva para você? Provavelmente não. Este é um problema muito resolvido. Escolha uma solução pronta para uso (código aberto ou SaaS gerenciado). O mundo não precisa do milionésimo conector +1 de banco de dados para armazenamento de dados em nuvem.

Por outro lado, por que os clientes compram de você? Sua empresa provavelmente tem algo especial na maneira como faz as coisas. Talvez seja um algoritmo específico que alimenta sua plataforma fintech. Ao abstrair muitos

Machine Translated by Google

os fluxos de trabalho e processos redundantes, você pode continuar desbastando, refinando e personalizando as coisas que movem a agulha para os negócios.

Conclusão

Escolher as tecnologias certas não é tarefa fácil, especialmente quando novas tecnologias e padrões surgem diariamente. Hoje é possivelmente o momento mais confuso da história para avaliar e selecionar tecnologias.

A escolha de tecnologias é um equilíbrio entre caso de uso, custo, construção versus compra e modularização. Sempre aborde a tecnologia da mesma forma que a arquitetura: avalie as compensações e aponte para decisões reversíveis.

1 Para mais detalhes, veja “Total Opportunity Cost of Ownership” de Joseph Reis em ***97 Things Todo Engenheiro de Dados Deve Saber*** (O'Reilly).

2 JR Storment e Mike Fuller, ***Cloud FinOps*** (Sebastopol, CA: O'Reilly, 2019), 6, ***https://oreil.ly/RvRvX.***

3 Este é um ponto importante de ênfase em Storment e Fuller, ***Cloud FinOps.***

4 exemplos incluem o Google Cloud Anthos e postos avançados da AWS.

5 Raghav Bhargava, “Evolution of Dropbox's Edge Network”, Dropbox.Tech, 19 de junho de 2017, ***https://oreil.ly/RAwPf.***

6 Akhil Gupta, “Scaling to Exabytes and Beyond”, Dropbox.Tech, 14 de março de 2016, ***https://oreil.ly/5XPKv.***

7 “O Dropbox migra 34 PB de dados para um Amazon S3 Data Lake for Analytics”, site da AWS, 2020, ***https://oreil.ly/wpVoM.***

8 Todd Hoff, “The Eternal Cost Savings of Netflix's Internal Spot Market,” High Scalability, 4 de dezembro de 2017, ***https://oreil.ly/LLoFt.***

9 Todd Spangler, “Consumo de largura de banda Netflix eclipsado pelo streaming de mídia da Web Applications”, ***Variety,*** 10 de setembro de 2019, ***https://oreil.ly/tTm3k.***

10 Amir Efrati e Kevin McLaughlin, “Os gastos da Apple com o armazenamento em nuvem do Google estão a caminho de Soar 50% This Year”, ***The Information,*** 29 de junho de 2021, ***https://oreil.ly/OlFyR.***

11 Evan Sangaline, “Executando o FFmpeg no AWS Lambda por 1,9% do custo do AWS Elastic Transcoder”, blog Intoli, 2 de maio de 2018, ***https://oreil.ly/myzOv.***

12 Exemplos incluem OpenFaaS, Knative, e Google Cloud Run.

Machine Translated by Google

13 Justin Olsson e Reynold Xin, “Elimming the Anti-competitive DeWitt Clause for Database Benchmarking”, Databricks, 8 de novembro de 2021, ***https:// oreil.ly/ 3iFOE.***

14 Para um clássico do gênero, ver William McKnight e Jake Dolezal, “Data Warehouse in the Cloud Benchmark”, GigaOm, 7 de fevereiro de 2019, ***https:// oreil.ly/ QjCmA.***

Machine Translated by Google

Parte II. O ciclo de vida da engenharia de dados em profundidade

Machine Translated by Google

Capítulo 5. Geração de Dados em Sistemas de Origem

Bem-vindo ao primeiro estágio do ciclo de vida da engenharia de dados: geração de dados em sistemas de origem. Como descrevemos anteriormente, o trabalho de um engenheiro de dados é pegar dados de sistemas de origem, fazer algo com eles e torná-los úteis para servir casos de uso downstream. Mas antes de obter dados brutos, você deve entender onde os dados existem, como são gerados e suas características e peculiaridades.

Este capítulo abrange alguns padrões de sistema de origem operacional populares e os tipos significativos de sistemas de origem. Existem muitos sistemas de origem para geração de dados e não estamos cobrindo todos eles exaustivamente. Consideraremos os dados que esses sistemas geram e o que você deve considerar ao trabalhar com sistemas de origem. Também discutimos como as correntes ocultas da engenharia de dados se aplicam a essa primeira fase do ciclo de vida da engenharia de dados (Figura 5-1).

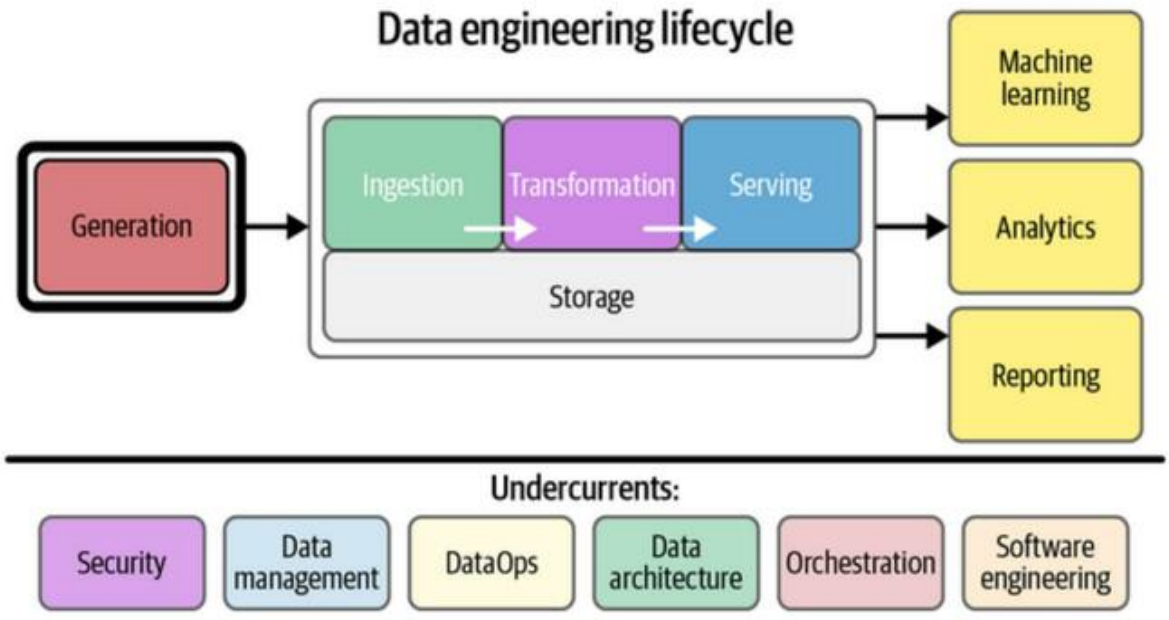


Figura 5-1. Os sistemas de origem geram os dados para o restante do ciclo de vida da engenharia de dados

Machine Translated by Google

À medida que os dados proliferam, especialmente com o aumento do compartilhamento de dados (discutido a seguir), esperamos que a função de um engenheiro de dados mude fortemente para entender a interação entre fontes e destinos de dados. As tarefas básicas de encanamento da engenharia de dados – mover dados de A para B – simplificarão drasticamente. Por outro lado, continuará sendo fundamental entender a natureza dos dados à medida que são criados nos sistemas de origem.

Fontes de dados: como os dados são criados?

À medida que você aprende sobre os vários padrões operacionais subjacentes dos sistemas que geram dados, é essencial entender como os dados são criados.
Os dados são uma coleção desorganizada e sem contexto de fatos e números. Ele pode ser criado de várias maneiras, tanto analógicas quanto digitais.

A criação de ***dados analógicos*** ocorre no mundo real, como fala vocal, linguagem de sinais, escrita em papel ou tocar um instrumento. Esses dados analógicos geralmente são transitórios; com que frequência você teve uma conversa verbal cujo conteúdo se perdeu no éter após o término da conversa?

Os dados digitais são criados pela conversão de dados analógicos em formato digital ou são o produto nativo de um sistema digital. Um exemplo de analógico para digital é um aplicativo de mensagens de texto para dispositivos móveis que converte a fala analógica em texto digital. Um exemplo de criação de dados digitais é uma transação com cartão de crédito em uma plataforma de comércio eletrônico. Um cliente faz um pedido, a transação é cobrada em seu cartão de crédito e as informações da transação são salvas em vários bancos de dados.

Utilizaremos alguns exemplos comuns neste capítulo, como dados criados ao interagir com um site ou aplicativo móvel. Mas, na verdade, os dados estão em todo o mundo ao nosso redor. Capturamos dados de dispositivos IoT, terminais de cartão de crédito, sensores de telescópio, negociações de ações e muito mais.

Familiarize-se com seu sistema de origem e como ele gera dados. Esforce-se para ler a documentação do sistema de origem e entender seus padrões e peculiaridades. Se o seu sistema de origem for um RDBMS, saiba como ele funciona

Machine Translated by Google

(gravações, confirmações, consultas, etc.); aprenda os meandros do sistema de origem que podem afetar sua capacidade de ingerir a partir dele.

Sistemas de origem: ideias principais

Os sistemas de origem produzem dados de várias maneiras. Esta seção discute as principais ideias que você encontrará com frequência ao trabalhar com sistemas de origem.

Arquivos e dados não estruturados

Um ***arquivo*** é uma sequência de bytes, normalmente armazenada em um disco. Os aplicativos geralmente gravam dados em arquivos. Os arquivos podem armazenar parâmetros locais, eventos, logs, imagens e áudio.

Além disso, os arquivos são um meio universal de troca de dados. Por mais que os engenheiros de dados desejem obter dados programaticamente, grande parte do mundo ainda envia e recebe arquivos. Por exemplo, se você estiver obtendo dados de uma agência governamental, há uma grande chance de você baixar os dados como um arquivo Excel ou CSV ou receber o arquivo em um e-mail.

Os principais tipos de formatos de arquivo de origem que você encontrará como engenheiro de dados — arquivos que se originam manualmente ou como saída de um processo do sistema de origem — são Excel, CSV, TXT, JSON e XML. Esses arquivos têm suas peculiaridades e podem ser estruturados (Excel, CSV), semiestruturados (JSON, XML, CSV) ou não estruturados (TXT, CSV). Embora você use fortemente certos formatos como engenheiro de dados (como Parquet, ORC e Avro), abordaremos isso mais tarde e colocaremos o destaque aqui nos arquivos do sistema de origem. O Capítulo 6 cobre os detalhes técnicos dos arquivos.

API

As interfaces de programação de aplicativos (APIs) são uma forma padrão de troca de dados entre sistemas. Em teoria, as APIs simplificam a tarefa de ingestão de dados para engenheiros de dados. Na prática, muitas APIs ainda expõem uma grande quantidade de complexidade de dados para os engenheiros gerenciarem. Mesmo com o surgimento de vários serviços e frameworks e serviços para automatizar dados de API

Machine Translated by Google

ingestão, os engenheiros de dados geralmente devem investir uma boa quantidade de energia na manutenção de conexões de API personalizadas. Discutiremos as APIs com mais detalhes posteriormente neste capítulo.

Bancos de dados de aplicativos (sistemas OLTP)

Um banco de ***dados de aplicativos*** armazena o estado de um aplicativo. Um exemplo padrão é um banco de dados que armazena saldos de contas bancárias. Conforme as transações e pagamentos dos clientes acontecem, o aplicativo atualiza os saldos das contas bancárias.

Normalmente, um banco de dados de aplicativo é um sistema ***de processamento de transações on*** -line (OLTP) — um banco de dados que lê e grava registros de dados individuais em uma taxa alta. Os sistemas OLTP são frequentemente chamados de ***bancos de dados transacionais,*** mas isso não implica necessariamente que o sistema em questão suporte ***transações atômicas.***

De forma mais geral, os bancos de dados OLTP suportam baixa latência e alta simultaneidade. Um banco de dados RDBMS pode selecionar ou atualizar uma linha em menos de um milissegundo (sem considerar a latência da rede) e lidar com milhares de leituras e gravações por segundo. Um cluster de banco de dados de documentos pode gerenciar taxas de confirmação de documentos ainda mais altas às custas de uma possível inconsistência. Alguns bancos de dados gráficos também podem lidar com casos de uso transacionais.

Fundamentalmente, os bancos de dados OLTP funcionam bem como back-ends de aplicativos quando milhares ou até milhões de usuários podem estar interagindo com o aplicativo simultaneamente, atualizando e gravando dados simultaneamente. Os sistemas OLTP são menos adequados para casos de uso orientados por análises em escala, em que uma única consulta deve verificar uma grande quantidade de dados.

ÁCIDO

O suporte para transações atômicas é um de um conjunto crítico de características de banco de dados conhecidas como ACID (como você deve se lembrar do Capítulo 3, isso significa ***atomicidade, consistência, isolamento, durabilidade). Consistência*** significa que qualquer leitura do banco de dados retornará a última versão escrita do item recuperado. ***O isolamento*** implica que, se duas atualizações estiverem em andamento simultaneamente

Machine Translated by Google

da mesma forma, o estado final do banco de dados será consistente com a execução sequencial dessas atualizações na ordem em que foram enviadas.
A durabilidade indica que os dados comprometidos nunca serão perdidos, mesmo em caso de perda de energia.

Observe que as características ACID não são necessárias para oferecer suporte a back-ends de aplicativos e relaxar essas restrições pode ser um benefício considerável para o desempenho e a escala. No entanto, as características do ACID garantem que o banco de dados manterá uma imagem consistente do mundo, simplificando drasticamente a tarefa do desenvolvedor do aplicativo.

Todos os engenheiros (dados ou não) devem entender a operação com e sem ACID.
Por exemplo, para melhorar o desempenho, alguns bancos de dados distribuídos usam restrições de consistência relaxadas, como ***consistência eventual,*** para melhorar o desempenho. Compreender o modelo de consistência com o qual você está trabalhando ajuda a evitar desastres.

Transações atômicas

Uma ***transação atômica*** é um conjunto de várias alterações que são confirmadas como uma unidade. No exemplo da Figura 5-2, um aplicativo bancário tradicional executado em um RDBMS executa uma instrução SQL que verifica os saldos de duas contas, uma na Conta A (a origem) e outra na Conta B (o destino).
O dinheiro é então movido da Conta A para a Conta B se houver fundos suficientes na Conta A. A transação inteira deve ser executada com atualizações nos saldos de ambas as contas ou falhar sem atualizar o saldo de nenhuma das contas. Ou seja, toda a operação deve acontecer como uma ***transação.***

Machine Translated by Google

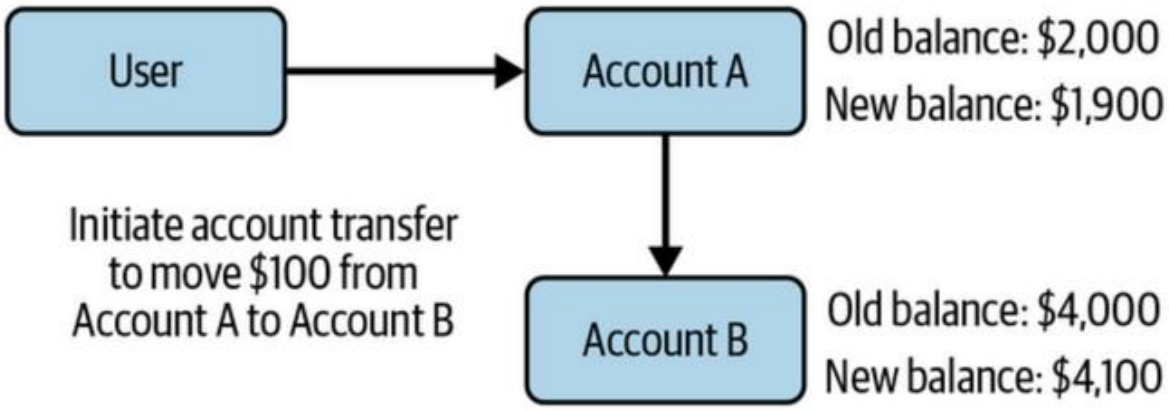


Figura 5-2. Exemplo de uma transação atômica: uma transferência de conta bancária usando OLTP

OLTP e análise

Muitas vezes, as pequenas empresas executam análises diretamente em um OLTP. Esse padrão funciona a curto prazo, mas, em última análise, não é escalável. Em algum momento, a execução de consultas analíticas no OLTP gera problemas de desempenho devido a limitações estruturais do OLTP ou contenção de recursos com cargas de trabalho transacionais concorrentes. Os engenheiros de dados devem entender o funcionamento interno do OLTP e dos back-ends de aplicativos para configurar as integrações apropriadas com sistemas de análise sem prejudicar o desempenho dos aplicativos de produção.

À medida que as empresas oferecem mais recursos analíticos em aplicativos SaaS, a necessidade de recursos híbridos – atualizações rápidas com recursos analíticos combinados – criou novos desafios para engenheiros de dados. Usaremos o termo ***aplicativo de dados*** para nos referirmos a aplicativos que hibridizam cargas de trabalho transacionais e analíticas.

Sistema de processamento analítico online

Em contraste com um sistema OLTP, um sistema ***de processamento analítico online*** (OLAP) é construído para executar grandes consultas analíticas e normalmente é ineficiente ao lidar com pesquisas de registros individuais. Por exemplo, bancos de dados de coluna modernos são otimizados para verificar grandes volumes de dados, dispensando índices para melhorar a escalabilidade e o desempenho da verificação. Qualquer consulta normalmente envolve a verificação de um bloco de dados mínimo, geralmente com 100 MB ou mais de tamanho. Tentando procurar milhares de itens individuais por segundo em tal sistema

Machine Translated by Google

o deixará de joelhos, a menos que seja combinado com uma camada de cache projetada para este caso de uso.

Observe que estamos usando o termo ***OLAP*** para nos referirmos a qualquer sistema de banco de dados que suporte consultas analíticas interativas de alta escala; não estamos nos limitando a sistemas que suportam cubos OLAP (arrays multidimensionais de dados). A parte ***on*** -line do OLAP implica que o sistema escute constantemente as consultas recebidas, tornando os sistemas OLAP adequados para análises interativas.

Embora este capítulo aborde os sistemas de origem, os OLAPs geralmente são sistemas de armazenamento e consulta para análise. Por que estamos falando sobre eles em nosso capítulo sobre sistemas de origem? Em casos de uso práticos, os engenheiros geralmente precisam ler dados de um sistema OLAP. Por exemplo, um data warehouse pode fornecer dados usados para treinar um modelo de ML. Ou um sistema OLAP pode atender a um fluxo de trabalho de ETL reverso, em que os dados derivados em um sistema de análise são enviados de volta a um sistema de origem, como um CRM, plataforma SaaS ou aplicativo transacional.

Alterar captura de dados

A captura de dados de alteração (CDC) é um método para extrair cada evento de alteração (inserir, atualizar, excluir) que ocorre em um banco de dados. O CDC é frequentemente aproveitado para replicar entre bancos de dados quase em tempo real ou criar um fluxo de eventos para processamento downstream.

O CDC é tratado de forma diferente dependendo da tecnologia do banco de dados.
Os bancos de dados relacionais geralmente geram um log de eventos armazenado diretamente no servidor de banco de dados que pode ser processado para criar um fluxo. (Consulte “Logs do banco de dados”.) Muitos bancos de dados NoSQL em nuvem podem enviar um log ou fluxo de eventos para um local de armazenamento de destino.

Histórico

Um ***log*** captura informações sobre eventos que ocorrem nos sistemas. Por exemplo, um log pode capturar padrões de tráfego e uso em um servidor web. Sua área de trabalho

Machine Translated by Google

o sistema operacional do computador (Windows, macOS, Linux) registra eventos à medida que o sistema é inicializado e quando os aplicativos são iniciados ou travados, por exemplo.

Os logs são uma fonte de dados rica, potencialmente valiosa para análise de dados downstream, ML e automação. Aqui estão algumas fontes familiares de logs:

- Sistemas operacionais
- Formulários
- Servidores
- Recipientes
- Redes
- dispositivos IoT

Todos os logs rastreiam eventos e metadados de eventos. No mínimo, um log deve capturar quem, o quê e quando:

Quem

- A conta humana, de sistema ou de serviço associada ao evento (por exemplo, um agente de usuário do navegador da web ou um ID de usuário)

O que aconteceu

- O evento e metadados relacionados

Quando

- O carimbo de data/hora do evento

Codificação de registro

Os logs são codificados de algumas maneiras:

Registros codificados em binário

Machine Translated by Google

Eles codificam dados em um formato compacto personalizado para eficiência de espaço e E/S rápida. Os logs de banco de dados, discutidos em "Database Logs", são um padrão exemplo.

Registros semiestruturados

Eles são codificados como texto em um formato de serialização de objetos (JSON, mais muitas vezes não). Os logs semiestruturados são legíveis por máquina e portáteis.
No entanto, eles são muito menos eficientes que os logs binários. E embora sejam nominalmente legíveis por máquina, extrair valor deles geralmente requer um código personalizado significativo.

Logs de texto simples (não estruturados)

Estes essencialmente armazenam a saída do console do software. Como tal, não existem padrões de propósito geral. Esses logs podem fornecer informações úteis informações para cientistas de dados e engenheiros de ML, extraindo informações úteis dos dados de texto bruto podem ser complicadas.

Resolução de registro

Os logs são criados em várias resoluções e níveis de log. A ***resolução*** do log refere-se à quantidade de dados de eventos capturados em um log. Por exemplo, os logs do banco de dados capturam informações suficientes dos eventos do banco de dados para permitir a reconstrução do estado do banco de dados a qualquer momento.

Por outro lado, capturar todas as alterações de dados em logs para um sistema de big data geralmente não é prático. Em vez disso, esses logs podem observar apenas que ocorreu um tipo específico de evento de confirmação. O ***nível de log*** refere-se às condições necessárias para registrar uma entrada de log, especificamente no que diz respeito a erros e depuração.
O software geralmente é configurável para registrar todos os eventos ou apenas erros, por exemplo.

Latência de log: em lote ou em tempo real

Machine Translated by Google

Os logs em lote geralmente são gravados continuamente em um arquivo. As entradas de log individuais podem ser gravadas em um sistema de mensagens como Kafka ou Pulsar para aplicativos em tempo real.

Registros do banco de dados

Os ***logs de banco*** de dados são essenciais o suficiente para merecerem uma cobertura mais detalhada. Os logs de gravação antecipada — normalmente, arquivos binários armazenados em um formato específico nativo do banco de dados — desempenham um papel crucial nas garantias e na capacidade de recuperação do banco de dados. O servidor de banco de dados recebe solicitações de gravação e atualização em uma tabela de banco de dados (consulte a Figura 5-3), armazenando cada operação no log antes de confirmar a solicitação. A confirmação vem com uma garantia associada ao log: mesmo que o servidor falhe, ele pode recuperar seu estado na reinicialização completando o trabalho inacabado dos logs.

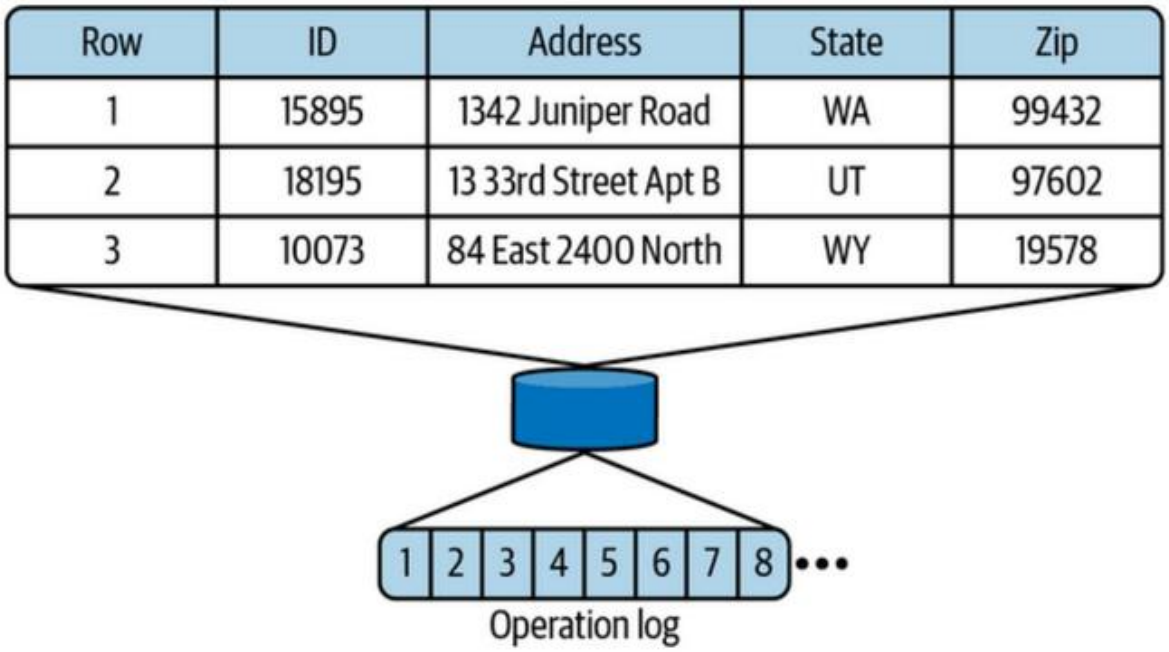


Figura 5-3. Operações de log em uma tabela de banco de dados

Os logs de banco de dados são extremamente úteis na engenharia de dados, especialmente para o CDC gerar fluxos de eventos a partir de alterações no banco de dados.

CRUD

Machine Translated by Google

CRUD, que significa ***criar, ler, atualizar*** e ***excluir,*** é um padrão transacional comumente usado em programação e representa as quatro operações básicas de armazenamento persistente. CRUD é o padrão mais comum para armazenar o estado do aplicativo em um banco de dados. Um princípio básico do CRUD é que os dados devem ser criados antes de serem usados. Após a criação dos dados, os dados podem ser lidos e atualizados. Finalmente, os dados podem precisar ser destruídos. O CRUD garante que essas quatro operações ocorrerão nos dados, independentemente de seu armazenamento.

CRUD é um padrão amplamente usado em aplicativos de software, e você normalmente encontrará CRUD usado em APIs e bancos de dados. Por exemplo, um aplicativo da Web fará uso intenso de CRUD para solicitações HTTP RESTful e armazenar e recuperar dados de um banco de dados.

Como em qualquer banco de dados, podemos usar a extração baseada em instantâneo para obter dados de um banco de dados onde nosso aplicativo aplica operações CRUD. Por outro lado, a extração de eventos com CDC nos fornece um histórico completo de operações e potencialmente permite análises quase em tempo real.

Somente inserção

O ***padrão somente inserção*** retém o histórico diretamente em uma tabela que contém dados. Em vez de atualizar registros, novos registros são inseridos com um carimbo de data/hora indicando quando foram criados (Tabela 5-1). Por exemplo, suponha que você tenha uma tabela de endereços de clientes. Seguindo um padrão CRUD, você simplesmente atualizaria o registro se o cliente mudasse de endereço. Com o padrão somente inserção, um novo registro de endereço é inserido com o mesmo ID do cliente. Para ler o endereço do cliente atual por ID de cliente, você deve procurar o registro mais recente sob esse ID.

Machine Translated by Google

Machine Translated by Google

c
e
s
m
você
eu
t
eu

pl
e
v
e
r
s
eu
o
n
s
o
f
uma
r
e
c
o
r
d

ID do registro	Valor	Carimbo de data e hora
1	40	2021-09-19T00:10:23+00:00
1	51	2021-09-30T00:12:00+00:00

Machine Translated by Google

De certa forma, o padrão insert-only mantém um log do banco de dados diretamente na própria tabela, tornando-o especialmente útil se o aplicativo precisar de acesso ao histórico. Por exemplo, o padrão somente inserção funcionaria bem para um aplicativo bancário projetado para apresentar o histórico de endereços do cliente.

Um padrão somente de inserção analítica separado é frequentemente usado com tabelas de aplicativos CRUD regulares. No padrão ETL somente de inserção, os pipelines de dados inserem um novo registro na tabela analítica de destino sempre que ocorre uma atualização na tabela CRUD.

Insert-only tem algumas desvantagens. Primeiro, as tabelas podem ficar muito grandes, especialmente se os dados forem alterados com frequência, pois cada alteração é inserida na tabela. Às vezes, os registros são limpos com base em uma data de expiração de registro ou em um número máximo de versões de registro para manter o tamanho da tabela razoável. A segunda desvantagem é que as pesquisas de registro incorrem em sobrecarga extra porque pesquisar o estado atual envolve a execução de MAX(created_timestamp). Se centenas ou milhares de registros estiverem em um único ID, essa operação de pesquisa será cara para ser executada.

Mensagens e fluxos

Relacionados à arquitetura orientada a eventos, dois termos que você verá frequentemente usados de forma intercambiável são ***fila de mensagens*** e ***plataforma de streaming,*** mas existe uma diferença sutil, mas essencial, entre os dois. Vale a pena definir e contrastar esses termos, pois eles abrangem muitas grandes ideias relacionadas a sistemas e práticas de origem e tecnologias que abrangem todo o ciclo de vida da engenharia de dados.

Uma ***mensagem*** é um dado bruto comunicado em dois ou mais sistemas (Figura 5-4). Por exemplo, temos o Sistema 1 e o Sistema 2, onde o Sistema 1 envia uma mensagem para o Sistema 2. Esses sistemas podem ser microsserviços diferentes, um servidor enviando uma mensagem para uma função sem servidor, etc. Uma mensagem normalmente é enviada através de uma ***fila de mensagens*** de um publicador para um consumidor e, uma vez entregue, a mensagem é removida da fila.

Machine Translated by Google



Figura 5-4. Uma mensagem passada entre dois sistemas

As mensagens são sinais discretos e singulares em um sistema orientado a eventos. Por exemplo, um dispositivo IoT pode enviar uma mensagem com a leitura de temperatura mais recente para uma fila de mensagens. Esta mensagem é então ingerida por um serviço que determina se o forno deve ser ligado ou desligado. Este serviço envia uma mensagem para um controlador de forno que executa a ação apropriada.
Depois que a mensagem é recebida e a ação é executada, a mensagem é removida da fila de mensagens.

Por outro lado, um ***fluxo*** é um log somente de acréscimo de registros de eventos. (Os fluxos são ingeridos e armazenados em ***plataformas de fluxo de eventos,*** que discutiremos mais detalhadamente em "Mensagens e fluxos".) À medida que os eventos ocorrem, eles são acumulados em uma sequência ordenada (Figura 5-5); um carimbo de data/hora ou um ID pode ordenar eventos. (Observe que os eventos nem sempre são entregues na ordem exata devido às sutilezas dos sistemas distribuídos.)

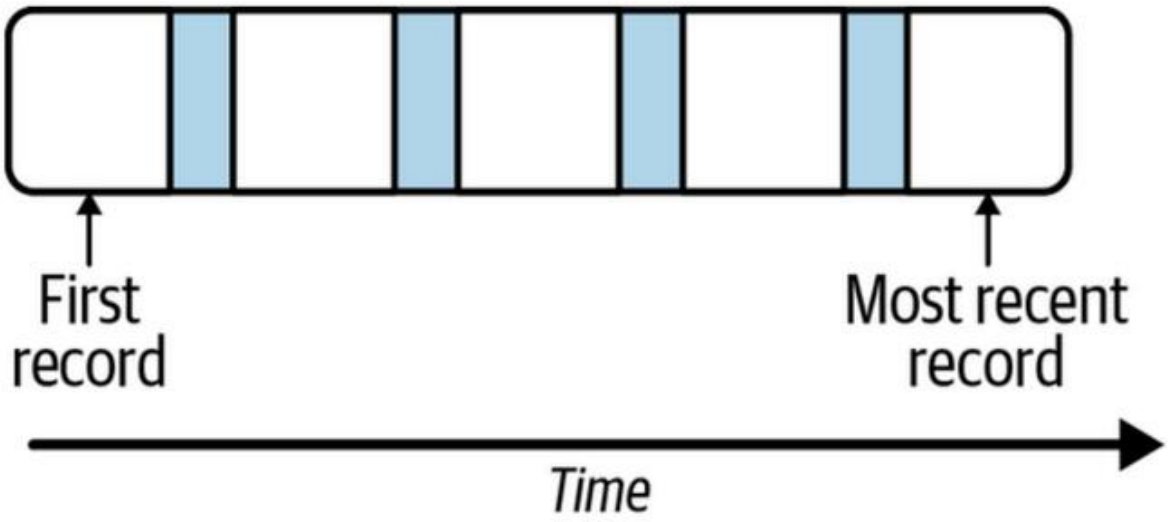


Figura 5-5. Um fluxo, que é um log de registros somente de acréscimo ordenado

Você usará streams quando se preocupar com o que aconteceu em muitos eventos. Devido à natureza somente anexada dos fluxos, os registros em um fluxo são persistidos por uma longa janela de retenção, geralmente semanas ou meses, permitindo

Machine Translated by Google

para operações complexas em registros, como agregações em vários registros ou a capacidade de retroceder até um ponto no tempo no fluxo.

Vale a pena notar que os sistemas que processam fluxos podem processar mensagens, e as plataformas de fluxo são frequentemente usadas para passagem de mensagens. Muitas vezes acumulamos mensagens em fluxos quando queremos realizar análises de mensagens. Em nosso exemplo de IoT, as leituras de temperatura que acionam o forno para ligar ou desligar também podem ser analisadas posteriormente para determinar tendências e estatísticas de temperatura.

Tipos de tempo

Embora o tempo seja uma consideração essencial para toda a ingestão de dados, ele se torna muito mais crítico e sutil no contexto de streaming, onde vemos os dados como contínuos e esperamos consumi-los logo após serem produzidos. Vejamos os principais tipos de tempo que você terá ao ingerir dados: o tempo em que o evento é gerado, quando é ingerido e processado e quanto tempo o processamento levou (Figura 5-6).

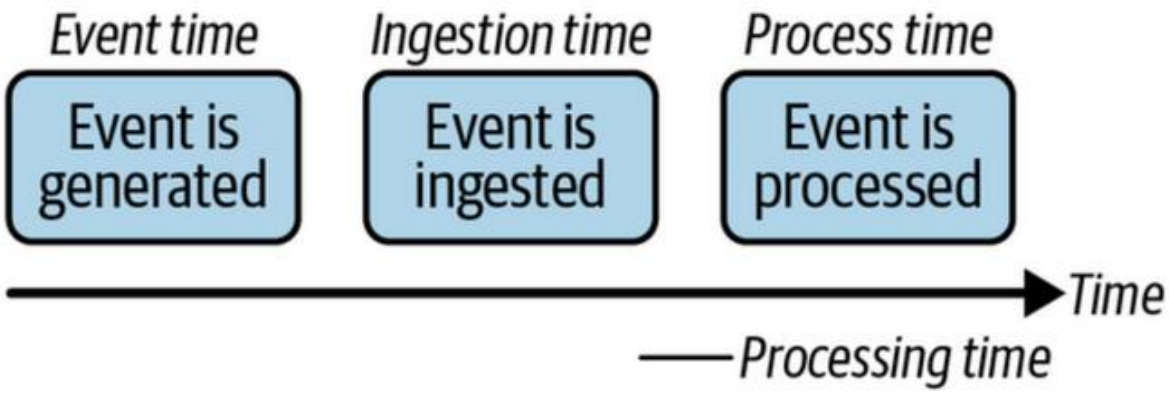


Figura 5-6. Evento, ingestão, processo e tempo de processamento

A ***hora do evento*** indica quando um evento é gerado em um sistema de origem, incluindo o carimbo de data/hora do próprio evento original. Um intervalo de tempo indeterminado ocorrerá na criação do evento, antes que o evento seja ingerido e processado a jusante. Sempre inclua carimbos de data/hora para cada fase pela qual um evento passa. Registre eventos à medida que ocorrem e em cada estágio do tempo — quando são criados, ingeridos e processados. Use esses registros de carimbo de data/hora para rastrear com precisão o movimento de seus dados por meio de seus pipelines de dados.

Machine Translated by Google

Depois que os dados são criados, eles são ingeridos em algum lugar. ***O tempo de ingestão*** indica quando um evento é ingerido dos sistemas de origem em uma fila de mensagens, cache, memória, armazenamento de objetos, banco de dados ou qualquer outro local em que os dados estejam armazenados (consulte o Capítulo 6). Após a ingestão, os dados podem ser processados imediatamente; ou em minutos, horas ou dias; ou simplesmente persistir no armazenamento indefinidamente.

O tempo de processo ocorre após o tempo de ingestão, quando os dados são processados (normalmente, uma transformação). ***O tempo de processamento*** é quanto tempo os dados levaram para processar, medido em segundos, minutos, horas, etc.

Você vai querer gravar essas várias vezes, de preferência de forma automatizada. Configure o monitoramento ao longo de seus fluxos de trabalho de dados para capturar quando os eventos ocorrem, quando são ingeridos e processados e quanto tempo levou para processar os eventos.

Detalhes práticos do sistema de origem

Esta seção discute os detalhes práticos da interação com sistemas de origem modernos. Vamos nos aprofundar nos detalhes de bancos de dados, APIs e outros aspectos comumente encontrados. Essas informações terão um prazo de validade menor do que as principais ideias discutidas anteriormente; estruturas de API populares, bancos de dados e outros detalhes continuarão a mudar rapidamente.

No entanto, esses detalhes são um conhecimento crítico para engenheiros de dados de trabalho. Sugerimos que você estude essas informações como conhecimento básico, mas leia extensivamente para ficar a par dos desenvolvimentos em andamento.

Bancos de dados

Nesta seção, veremos tecnologias comuns de banco de dados do sistema de origem que você encontrará como engenheiro de dados e considerações de alto nível para trabalhar com esses sistemas. Existem tantos tipos de bancos de dados quanto casos de uso para dados.

Principais considerações para entender as tecnologias de banco de dados

Machine Translated by Google

Aqui, apresentamos as principais ideias que ocorrem em uma variedade de tecnologias de banco de dados, incluindo aquelas que suportam aplicativos de software e aquelas que suportam casos de uso de análise:

Sistema de gerenciamento de banco de dados

Um sistema de banco de dados usado para armazenar e servir dados. Abreviado como DBMS, consiste em um mecanismo de armazenamento, otimizador de consultas, recuperação de desastres e outros componentes-chave para gerenciar o sistema de banco de dados.

Pesquisas

Como o banco de dados encontra e recupera dados? Os índices podem ajudar a acelerar pesquisas, mas nem todos os bancos de dados têm índices. Saiba se o seu banco de dados usa índices; em caso afirmativo, quais são os melhores padrões para projetá-los e mantê-los? Entenda como aproveitar para uma extração eficiente.

Também ajuda a ter um conhecimento básico dos principais tipos de índices, incluindo árvore B e árvores de mesclagem estruturadas em log (LSM).

Otimizador de consultas

O banco de dados utiliza um otimizador? Quais são suas características?

Dimensionamento e distribuição

O banco de dados é dimensionado com a demanda? Qual estratégia de dimensionamento ele implanta? Ele escala horizontalmente (mais nós de banco de dados) ou verticalmente (mais recursos em uma única máquina)?

Padrões de modelagem

Quais padrões de modelagem funcionam melhor com o banco de dados (por exemplo, dados normalização ou tabelas largas)? (Veja o Capítulo 8 para nossa discussão de dados modelagem.)

Machine Translated by Google

CRUD

Como os dados são consultados, criados, atualizados e excluídos no banco de dados?

Cada tipo de banco de dados trata as operações CRUD de maneira diferente.

Consistência

O banco de dados é totalmente consistente ou suporta uma consistência relaxada

modelo (por exemplo, consistência eventual)? O banco de dados suporta opcional

modos de consistência para leituras e gravações (por exemplo, leituras fortemente consistentes)?

Dividimos os bancos de dados em categorias relacionais e não relacionais. Na verdade, a categoria não relacional é muito mais diversificada, mas os bancos de dados relacionais ainda ocupam um espaço significativo nos back-ends de aplicativos.

Bancos de dados relacionais

Um ***sistema de gerenciamento de banco de dados relacional*** (RDBMS) é um dos back-ends de aplicativos mais comuns. Os bancos de dados relacionais foram desenvolvidos na IBM na década de 1970 e popularizados pela Oracle na década de 1980. O crescimento da internet viu a ascensão da pilha LAMP (Linux, servidor web Apache, MySQL, PHP) e uma explosão de opções de fornecedores e RDBMS de código aberto. Mesmo com o surgimento dos bancos de dados NoSQL (descritos na seção a seguir), os bancos de dados relacionais permaneceram extremamente populares.

Os dados são armazenados em uma tabela de ***relações*** (linhas), e cada relação contém vários ***campos*** (colunas); consulte a Figura 5-7. Observe que usamos os termos ***coluna*** e ***campo de*** forma intercambiável ao longo deste livro. Cada relação na tabela tem o mesmo ***esquema*** (uma sequência de colunas com tipos estáticos atribuídos, como string, integer ou float). As linhas geralmente são armazenadas como uma sequência contígua de bytes no disco.

Machine Translated by Google

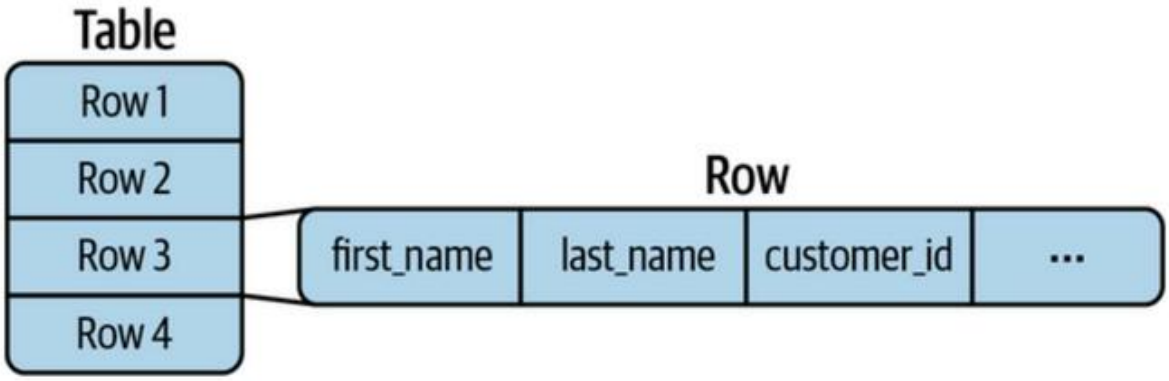


Figura 5-7. O RDBMS armazena e recupera dados em nível de linha

As tabelas são indexadas por uma ***chave primária,*** um campo exclusivo para cada linha da tabela. A estratégia de indexação para a chave primária está intimamente ligada ao layout da tabela no disco.

As tabelas também podem ter várias chaves ***estrangeiras*** — campos com valores conectados aos valores das chaves primárias em outras tabelas, facilitando junções e permitindo esquemas complexos que distribuem dados por várias tabelas. Em particular, é possível projetar um ***esquema normalizado.*** A normalização é uma estratégia para garantir que os dados nos registros não sejam duplicados em vários locais, evitando assim a necessidade de atualizar estados em vários locais ao mesmo tempo e evitando inconsistências (consulte o Capítulo 8).

Os sistemas RDBMS são normalmente compatíveis com ACID. A combinação de um esquema normalizado, conformidade com ACID e suporte para altas taxas de transação torna os sistemas de banco de dados relacionais ideais para armazenar estados de aplicativos que mudam rapidamente. O desafio para engenheiros de dados é determinar como capturar informações de estado ao longo do tempo.

Uma discussão completa da teoria, história e tecnologia do RDBMS está além do escopo deste livro. Incentivamos você a estudar sistemas RDBMS, álgebra relacional e estratégias para normalização porque eles são difundidos e você os encontrará com frequência. Consulte "Recursos Adicionais" para livros sugeridos.

Bancos de dados não relacionais: NoSQL

Embora os bancos de dados relacionais sejam ótimos para muitos casos de uso, eles não são uma solução de tamanho único. Muitas vezes vemos que as pessoas começam com um relacionamento

Machine Translated by Google

banco de dados com a impressão de que é um dispositivo universal e calçadeira em uma tonelada de casos de uso e cargas de trabalho. À medida que os dados e os requisitos de consulta se transformam, o banco de dados relacional entra em colapso sob seu peso. Nesse ponto, você desejará usar um banco de dados apropriado para a carga de trabalho específica sob pressão. Insira bancos de dados não relacionais ou NoSQL. ***NoSQL,*** que significa ***não apenas SQL,*** refere-se a toda uma classe de bancos de dados que abandonam o paradigma relacional.

Por um lado, eliminar as restrições relacionais pode melhorar o desempenho, a escalabilidade e a flexibilidade do esquema. Mas, como sempre na arquitetura, existem trocas. Os bancos de dados NoSQL geralmente abandonam várias características do RDBMS, como consistência forte, junções ou um esquema fixo.

Um grande tema deste livro é que a inovação de dados é constante. Vamos dar uma olhada rápida na história do NoSQL, pois é útil obter uma perspectiva sobre por que e como as inovações de dados afetam seu trabalho como engenheiro de dados. No início dos anos 2000, empresas de tecnologia como Google e Amazon começaram a superar seus bancos de dados relacionais e foram pioneiras em novos bancos de dados não relacionais distribuídos para dimensionar suas plataformas da web.

Enquanto o termo ***NoSQL*** apareceu pela primeira vez em 1998, a versão moderna foi cunhada por Eric Evans na década de 2000. [1] Ele conta a história em um post de blog de 2009:

> ***Passei os últimos dias no nosqleast e um dos tópicos quentes aqui é o nome "nosql". Compreensivelmente, há muitas pessoas que se preocupam que o nome seja ruim, que envie uma mensagem imprópria ou imprecisa. Embora eu não reivindique a ideia, tenho que aceitar alguma culpa pelo que agora está sendo chamado. Como é isso? Johan Oskarsson estava organizando o primeiro encontro e fez a pergunta "O que é um bom nome?" no IRC; foi uma das três ou quatro sugestões que dei no espaço de 45 segundos, sem pensar.***
>
> ***Meu arrependimento, no entanto, não é sobre o que o nome diz; é sobre o que não faz. Quando Johan originalmente teve a ideia para o primeiro encontro, ele parecia estar pensando em Big Data e sistemas distribuídos linearmente escaláveis, mas o nome é tão vago que abriu a porta para falar sobre envios para literalmente qualquer coisa que armazenasse dados e não fosse um RDBMS.***

Machine Translated by Google

O NoSQL permanece vago em 2022, mas tem sido amplamente adotado para descrever um universo de bancos de dados da “nova escola”, alternativas aos bancos de dados relacionais.

Existem vários tipos de banco de dados NoSQL projetados para quase todos os casos de uso imagináveis. Como há muitos bancos de dados NoSQL para cobrir exaustivamente nesta seção, consideramos os seguintes tipos de banco de dados: valor-chave, documento, coluna larga, gráfico, pesquisa e série temporal.
Esses bancos de dados são muito populares e desfrutam de ampla adoção. Um engenheiro de dados deve entender esses tipos de bancos de dados, incluindo considerações de uso, a estrutura dos dados que eles armazenam e como aproveitar cada um no ciclo de vida da engenharia de dados.

Armazenamentos de valores-chave

Um ***banco de dados de valor-chave*** é um banco de dados não relacional que recupera registros usando uma chave que identifica exclusivamente cada registro. Isso é semelhante ao mapa de hash ou às estruturas de dados de dicionário apresentadas em muitas linguagens de programação, mas potencialmente mais escaláveis. Os armazenamentos de valores-chave abrangem vários tipos de banco de dados NoSQL—por exemplo, armazenamentos de documentos e bancos de dados de colunas largas (discutidos a seguir).

Diferentes tipos de bancos de dados de chave-valor oferecem uma variedade de características de desempenho para atender a várias necessidades de aplicativos. Por exemplo, os bancos de dados de valor-chave na memória são populares para armazenar dados de sessão em cache para aplicativos móveis e da Web, onde são necessárias pesquisas ultrarrápidas e alta simultaneidade. O armazenamento nesses sistemas geralmente é temporário; se o banco de dados for encerrado, os dados desaparecerão. Esses caches podem reduzir a pressão no banco de dados principal do aplicativo e fornecer respostas rápidas.

Obviamente, os armazenamentos de valor-chave também podem atender a aplicativos que exigem persistência de alta durabilidade. Um aplicativo de comércio eletrônico pode precisar salvar e atualizar grandes quantidades de alterações de estado de eventos para um usuário e seus pedidos. Um usuário faz login no aplicativo de comércio eletrônico, clica em várias telas, adiciona itens a um carrinho de compras e faz o check-out. Cada evento deve ser armazenado de forma durável para recuperação. Os armazenamentos de valores-chave geralmente persistem dados em disco e em vários nós para dar suporte a esses casos de uso.

Machine Translated by Google

Armazenamentos de documentos

Conforme mencionado anteriormente, um armazenamento de ***documentos*** é um armazenamento de valor-chave especializado. Nesse contexto, um ***documento*** é um objeto aninhado; geralmente podemos pensar em cada documento como um objeto JSON para fins práticos. Os documentos são armazenados em coleções e recuperados por chave. Uma ***coleção*** é aproximadamente equivalente a uma tabela em um banco de dados relacional (consulte a Tabela 5-2).

Machine Translated by Google

Machine Translated by Google

e
n
t
t
e
r
m
eu
n
o
eu
o
g
y

RDBMS	Banco de dados de documentos
Mesa	Coleção
Fileira	Documento, itens, entidade

Uma diferença importante entre bancos de dados relacionais e armazenamentos de documentos é que o último não oferece suporte a junções. Isso significa que os dados não podem ser facilmente ***normalizados,*** ou seja, divididos em várias tabelas. (Aplicativos ainda podem ingressar manualmente. O código pode pesquisar um documento, extrair uma propriedade e recuperar outro documento.) Idealmente, todos os dados relacionados podem ser armazenados no mesmo documento.

Em muitos casos, os mesmos dados devem ser armazenados em vários documentos espalhados por várias coleções; engenheiros de software devem ter o cuidado de atualizar uma propriedade em todos os lugares em que ela está armazenada. (Muitos armazenamentos de documentos suportam uma noção de transações para facilitar isso.)

Os bancos de dados de documentos geralmente adotam toda a flexibilidade do JSON e não impõem esquemas ou tipos; isso é uma benção e uma maldição. Por um lado,

Machine Translated by Google

isso permite que o esquema seja altamente flexível e expressivo. O esquema também pode evoluir à medida que um aplicativo cresce. Por outro lado, vimos bancos de dados de documentos se tornarem pesadelos absolutos para gerenciar e consultar. Se os desenvolvedores não forem cuidadosos ao gerenciar a evolução do esquema, os dados podem se tornar inconsistentes e inchados ao longo do tempo. A evolução do esquema também pode interromper a ingestão de downstream e causar dores de cabeça para engenheiros de dados se não for comunicada em tempo hábil (antes da implantação).

Veja a seguir um exemplo de dados armazenados em uma coleção chamada users. A chave de coleta é o id. Também temos um nome (junto com primeiro e último como elementos filhos) e uma matriz das bandas favoritas da pessoa em cada documento:

```
usuários:
        [ { id:
          1234
                nome:
                        primeiro: "Joe"
                        último: "Reis"
                favorite_bands: ["AC/DC", "Slayer", "WuTang Clan",
"Ação Bronson"]
          },

        { ID: 1235
                nome:
                        primeiro: "Matt" último:
                        "Housley" favorite_bands:
                ["Dave Matthews Band", "Creed",
"Nickelback"]
        }
]
```

Para consultar os dados neste exemplo, você pode recuperar registros por chave. Observe que a maioria dos bancos de dados de documentos também suporta a criação de índices e tabelas de pesquisa para permitir a recuperação de documentos por propriedades específicas. Isso geralmente é inestimável no desenvolvimento de aplicativos quando você precisa pesquisar documentos de várias maneiras. Por exemplo, você pode definir um índice no nome.

Outro detalhe técnico crítico para engenheiros de dados é que os armazenamentos de documentos geralmente não são compatíveis com ACID, ao contrário dos bancos de dados relacionais. Técnico

Machine Translated by Google

a experiência em um armazenamento de documentos específico é essencial para entender o desempenho, o ajuste, a configuração, os efeitos relacionados nas gravações, a consistência, a durabilidade etc. Por exemplo, muitos armazenamentos de documentos ***acabam sendo consistentes.*** Permitir a distribuição de dados em um cluster é um benefício para dimensionamento e desempenho, mas pode levar a catástrofes quando engenheiros e desenvolvedores não entendem as implicações.

2

Para executar análises em armazenamentos de documentos, os engenheiros geralmente devem executar uma verificação completa para extrair todos os dados de uma coleção ou empregar uma estratégia de CDC para enviar eventos a um fluxo de destino. A abordagem de varredura completa pode ter implicações de desempenho e custo. A verificação geralmente diminui a velocidade do banco de dados enquanto ele é executado, e muitas ofertas de nuvem sem servidor cobram uma taxa significativa por cada verificação completa.
Em bancos de dados de documentos, geralmente é útil criar um índice para ajudar a acelerar as consultas. Discutimos índices e padrões de consulta no Capítulo 8.

Coluna larga

Um ***banco de dados de colunas amplas*** é otimizado para armazenar grandes quantidades de dados com altas taxas de transação e latência extremamente baixa. Esses bancos de dados podem ser dimensionados para taxas de gravação extremamente altas e grandes quantidades de dados. Especificamente, bancos de dados de coluna larga podem suportar petabytes de dados, milhões de solicitações por segundo e latência abaixo de 10 ms. Essas características tornaram os bancos de dados de colunas amplas populares em aplicativos de comércio eletrônico, fintech, ad tech, IoT e personalização em tempo real. Os engenheiros de dados devem estar cientes das características operacionais dos bancos de dados de colunas amplas com os quais trabalham para definir uma configuração adequada, projetar o esquema e escolher uma chave de linha apropriada para otimizar o desempenho e evitar problemas operacionais comuns.

Esses bancos de dados suportam verificações rápidas de grandes quantidades de dados, mas não suportam consultas complexas. Eles têm apenas um único índice (a chave de linha) para pesquisas. Os engenheiros de dados geralmente devem extrair dados e enviá-los a um sistema de análise secundário para executar consultas complexas para lidar com essas limitações. Isso pode ser feito executando grandes varreduras para a extração ou empregando o CDC para capturar um fluxo de eventos.

Bancos de dados de gráficos

Machine Translated by Google

Os bancos de ***dados gráficos*** armazenam dados explicitamente com uma estrutura gráfica matemática (como um conjunto de nós e arestas). O Neo4j provou ser extremamente popular, enquanto Amazon, Oracle e outros fornecedores oferecem seus produtos de banco de dados gráfico. Grosso modo, os bancos de dados de gráficos são uma boa opção quando você deseja analisar a conectividade entre os elementos.

Por exemplo, você pode usar um banco de dados de documentos para armazenar um documento para cada usuário descrevendo suas propriedades. Você pode adicionar um elemento de matriz para ***conexões*** que contenham IDs de usuários conectados diretamente em um contexto de mídia social. É muito fácil determinar o número de conexões diretas que um usuário possui, mas suponha que você queira saber quantos usuários podem ser alcançados por meio de duas conexões diretas. Você poderia responder a essa pergunta escrevendo um código complexo, mas cada consulta seria executada lentamente e consumiria recursos significativos. O armazenamento de documentos simplesmente não é otimizado para isso
caso de uso.

Os bancos de dados gráficos são projetados exatamente para esse tipo de consulta. Suas estruturas de dados permitem consultas baseadas na conectividade entre os elementos; bancos de dados de grafos são indicados quando nos preocupamos em entender travessias complexas entre elementos. Na linguagem dos gráficos, armazenamos ***nós*** (usuários no exemplo anterior) e ***arestas*** (conexões entre usuários).
Os bancos de dados de gráfico suportam modelos de dados avançados para nós e arestas. Dependendo do mecanismo de banco de dados gráfico subjacente, os bancos de dados gráficos utilizam linguagens de consulta especializadas, como SPARQL, Resource Description Framework (RDF), Graph Query Language (GQL) e Cypher.

Como exemplo de gráfico, considere uma rede de quatro usuários. O usuário 1 segue o usuário 2, que segue o usuário 3 e o usuário 4; O usuário 3 também segue o usuário 4 (Figura 5-8).

Machine Translated by Google

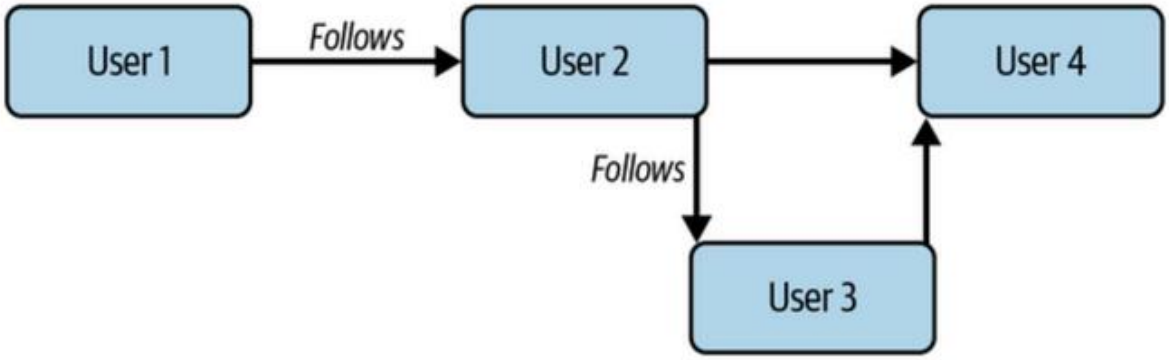


Figura 5-8. Um gráfico de rede social

Prevemos que os aplicativos de banco de dados gráfico crescerão dramaticamente fora das empresas de tecnologia; análises de mercado também preveem um crescimento rápido. É claro que os bancos de dados[4] gráficos são benéficos do ponto de vista operacional e suportam os tipos de relacionamentos sociais complexos críticos para os aplicativos modernos. As estruturas gráficas também são fascinantes do ponto de vista da ciência de dados e ML, potencialmente revelando insights profundos sobre as interações e o comportamento humano.

Isso apresenta desafios exclusivos para engenheiros de dados que podem estar mais acostumados a lidar com relações estruturadas, documentos ou dados não estruturados. Os engenheiros devem escolher se devem fazer o seguinte:

- Mapeie os dados do gráfico do sistema de origem em um de seus paradigmas preferenciais existentes
- Analise os dados do gráfico dentro do próprio sistema de origem
- Adote ferramentas de análise específicas para gráficos

Os dados do gráfico podem ser recodificados em linhas em um banco de dados relacional, o que pode ser uma solução adequada dependendo do caso de uso de análise. Os bancos de dados de gráficos transacionais também são projetados para análises, embora consultas grandes possam sobrecarregar os sistemas de produção. Os bancos de dados de gráficos baseados em nuvem contemporâneos oferecem suporte a análises de gráficos de leitura intensa em grandes quantidades de dados.

Procurar

Um banco de dados de ***pesquisa*** é um banco de dados não relacional usado para pesquisar as características semânticas e estruturais complexas e diretas de seus dados. Dois

Machine Translated by Google

existem casos de uso proeminentes para um banco de dados de pesquisa: pesquisa de texto e análise de log. Vamos cobrir cada um deles separadamente.

A pesquisa de ***texto*** envolve pesquisar um corpo de texto por palavras-chave ou frases, correspondendo a correspondências exatas, difusas ou semanticamente semelhantes. A ***análise de log*** é normalmente usada para detecção de anomalias, monitoramento em tempo real, análise de segurança e análise operacional. As consultas podem ser otimizadas e aceleradas com o uso de índices.

Dependendo do tipo de empresa em que você trabalha, você pode usar bancos de dados de pesquisa regularmente ou não. Independentemente disso, é bom estar ciente de que eles existem, caso você os encontre na natureza. Os bancos de dados de pesquisa são populares para pesquisa e recuperação rápidas e podem ser encontrados em vários aplicativos; um site de comércio eletrônico pode impulsionar sua pesquisa de produtos usando um banco de dados de pesquisa. Como engenheiro de dados, pode-se esperar que você traga dados de um banco de dados de pesquisa (como Elasticsearch, Apache Solr ou Lucene ou Algolia) para relatórios de KPI downstream ou algo semelhante.

Série temporal

Uma ***série temporal*** é uma série de valores organizados por tempo. Por exemplo, os preços das ações podem se mover à medida que os negócios são executados ao longo do dia, ou um sensor climático mede as temperaturas atmosféricas a cada minuto. Quaisquer eventos registrados ao longo do tempo—regular ou esporadicamente—são dados de série temporal. Um ***banco de dados de séries temporais*** é otimizado para recuperação e processamento estatístico de dados de séries temporais.

Embora os dados de séries temporais, como pedidos, remessas, logs e assim por diante, tenham sido armazenados em bancos de dados relacionais por muito tempo, esses tamanhos e volumes de dados geralmente eram pequenos. À medida que os dados cresciam cada vez mais rápido, novos bancos de dados para fins especiais eram necessários. Os bancos de dados de série temporal atendem às necessidades de volumes de dados crescentes e de alta velocidade de IoT, logs de eventos e aplicativos, tecnologia de anúncios e fintech, entre muitos outros casos de uso. Muitas vezes, essas cargas de trabalho são pesadas de gravação. Como resultado, os bancos de dados de séries temporais geralmente utilizam buffer de memória para oferecer suporte a gravações e leituras rápidas.

Devemos distinguir entre dados de medição e baseados em eventos, comuns em bancos de dados de séries temporais. ***Os dados de medição*** são gerados regularmente,

Machine Translated by Google

como sensores de temperatura ou de qualidade do ar. ***Os dados baseados em eventos*** são irregulares e criados sempre que um evento ocorre, por exemplo, quando um sensor de movimento detecta movimento.

O esquema para uma série temporal normalmente contém um carimbo de data/hora e um pequeno conjunto de campos. Como os dados dependem do tempo, os dados são ordenados pelo carimbo de data/hora. Isso torna os bancos de dados de séries temporais adequados para análises operacionais, mas não ótimos para casos de uso de BI. As junções não são comuns, embora alguns bancos de dados quase de séries temporais, como junções de suporte do Apache Druid. Muitos bancos de dados de séries temporais estão disponíveis, tanto como opções de código aberto quanto pagas.

API

As APIs agora são uma maneira padrão e abrangente de troca de dados na nuvem, para plataformas SaaS e entre sistemas internos da empresa. Muitos tipos de interfaces de API existem na web, mas estamos principalmente interessados naquelas construídas em torno do HTTP, o tipo mais popular na web e na nuvem.

DESCANSO

Falaremos primeiro sobre REST, atualmente o paradigma de API dominante. Conforme observado no Capítulo 4, ***REST*** significa ***transferência de estado representacional.*** Esse conjunto de práticas e filosofias para a construção de APIs web HTTP foi apresentado por Roy Fielding em 2000 em uma dissertação de doutorado. REST é construído em torno de verbos HTTP, como GET e PUT; na prática, o REST moderno usa apenas alguns dos mapeamentos verbais descritos na dissertação original.

Uma das principais ideias do REST é que as interações são stateless. Ao contrário de uma sessão de terminal Linux, não há noção de uma sessão com variáveis de estado associadas, como um diretório de trabalho; cada chamada REST é independente.
As chamadas REST podem alterar o estado do sistema, mas essas alterações são globais, aplicando-se ao sistema completo e não a uma sessão atual.

Os críticos apontam que REST não é de forma alguma uma especificação completa. REST estipula propriedades básicas de interações, mas desenvolvedores que utilizam uma API

Machine Translated by Google

deve obter uma quantidade significativa de conhecimento de domínio para criar aplicativos ou extrair dados de forma eficaz.

Vemos uma grande variação nos níveis de abstração da API. Em alguns casos, as APIs são apenas um invólucro fino sobre os internos que fornecem a funcionalidade mínima necessária para proteger o sistema das solicitações do usuário. Em outros exemplos, uma API de dados REST é uma obra-prima de engenharia que prepara dados para aplicativos de análise e oferece suporte a relatórios avançados.

Alguns desenvolvimentos simplificaram a configuração de pipelines de ingestão de dados de APIs REST. Primeiro, os provedores de dados frequentemente fornecem bibliotecas de cliente em várias linguagens, especialmente em Python. As bibliotecas de cliente removem grande parte do trabalho padrão de construção do código de interação da API. As bibliotecas de cliente lidam com detalhes críticos, como autenticação e mapeiam métodos fundamentais em classes acessíveis.

Em segundo lugar, vários serviços e bibliotecas de código aberto surgiram para interagir com APIs e gerenciar a sincronização de dados. Muitos fornecedores de SaaS e de código aberto fornecem conectores prontos para uso para APIs comuns. As plataformas também simplificam o processo de criação de conectores personalizados conforme necessário.

Existem várias APIs de dados sem bibliotecas de cliente ou suporte de conector pronto para uso. Como enfatizamos ao longo do livro, os engenheiros fariam bem em reduzir o trabalho pesado indiferenciado usando ferramentas prontas para uso.
No entanto, as tarefas de ***encanamento*** de baixo nível ainda consomem muitos recursos. Em praticamente qualquer grande empresa, os engenheiros de dados precisarão lidar com o problema de escrever e manter código personalizado para extrair dados de APIs, o que requer entender a estrutura dos dados conforme fornecidos, desenvolver código de extração de dados apropriado e determinar um dado adequado. estratégia de sincronização.

GraphQL

O GraphQL foi criado no Facebook como uma linguagem de consulta para dados de aplicativos e uma alternativa às APIs REST genéricas. Enquanto as APIs REST geralmente restringem suas consultas a um modelo de dados específico, o GraphQL abre a possibilidade de recuperar vários modelos de dados em uma única solicitação. este

Machine Translated by Google

permite consultas mais flexíveis e expressivas do que com REST. O GraphQL é construído em torno de JSON e retorna dados em uma forma semelhante à consulta JSON.

Há uma espécie de guerra santa entre REST e GraphQL, com algumas equipes de engenharia partidárias de um ou de outro e algumas usando ambos. Na realidade, os engenheiros encontrarão ambos ao interagirem com os sistemas de origem.

Webhooks

Webhooks são um padrão simples de transmissão de dados baseado em eventos. A fonte de dados pode ser um back-end de aplicativo, uma página da Web ou um aplicativo móvel. Quando eventos especificados ocorrem no sistema de origem, isso aciona uma chamada para um endpoint HTTP hospedado pelo consumidor de dados. Observe que a conexão vai do sistema de origem ao coletor de dados, o oposto das APIs típicas.
Por esse motivo, os webhooks costumam ser chamados ***de APIs reversas.***

O endpoint pode fazer várias coisas com os dados do evento POST, potencialmente acionando um processo downstream ou armazenando os dados para uso futuro. Para fins de análise, estamos interessados em coletar esses eventos. Os engenheiros geralmente usam filas de mensagens para ingerir dados em alta velocidade e volume.
Falaremos sobre filas de mensagens e fluxos de eventos mais adiante neste capítulo.

RPC e gRPC

Uma ***chamada de procedimento remoto*** (RPC) é comumente usada em computação distribuída. Ele permite que você execute um procedimento em um sistema remoto.

gRPC é uma biblioteca de chamada de procedimento remoto desenvolvida internamente no Google em 2015 e posteriormente lançada como um padrão aberto. Seu uso apenas no Google seria suficiente para merecer inclusão em nossa discussão. Muitos serviços do Google, como o Google Ads e o GCP, oferecem APIs gRPC. O gRPC é construído em torno do padrão de serialização de dados abertos Protocol Buffers, também desenvolvido pelo Google.

O gRPC enfatiza a troca bidirecional eficiente de dados por HTTP/2.
Eficiência refere-se a aspectos como utilização da CPU, consumo de energia, duração da bateria e largura de banda. Assim como o GraphQL, o gRPC impõe muito mais

Machine Translated by Google

padrões técnicos específicos do REST, permitindo assim o uso de bibliotecas de cliente comuns e permitindo que os engenheiros desenvolvam um conjunto de habilidades que se aplicará a qualquer código de interação gRPC.

Compartilhamento de dados

O conceito central de compartilhamento de dados em nuvem é que um sistema multilocatário oferece suporte a políticas de segurança para compartilhamento de dados entre locatários. Concretamente, qualquer sistema de armazenamento de objetos em nuvem pública com um sistema de permissão refinado pode ser uma plataforma para compartilhamento de dados. Plataformas populares de armazenamento de dados em nuvem também suportam recursos de compartilhamento de dados. É claro que os dados também podem ser compartilhados por meio de download ou troca por e-mail, mas um sistema multilocatário torna o processo muito mais fácil.

Muitas plataformas de compartilhamento modernas (especialmente armazéns de dados em nuvem) oferecem suporte à filtragem de dados confidenciais de linha, coluna e. O compartilhamento de dados também simplifica a noção do ***mercado de dados,*** disponível em várias nuvens e plataformas de dados populares. Os mercados de dados fornecem um local centralizado para o comércio de dados, onde os provedores de dados podem anunciar suas ofertas e vendê-las sem se preocupar com os detalhes do gerenciamento do acesso à rede aos sistemas de dados.

O compartilhamento de dados também pode simplificar os pipelines de dados dentro de uma organização. O compartilhamento de dados permite que as unidades de uma organização gerenciem seus dados e os compartilhem seletivamente com outras unidades, enquanto ainda permite que unidades individuais gerenciem seus custos de computação e consulta separadamente, facilitando a descentralização de dados. Isso facilita padrões de gerenciamento de dados descentralizados, como malha de dados.

Compartilhamento de dados e malha de dados estão alinhados com nossa filosofia de componentes de arquitetura comuns. Escolha componentes comuns (Capítulo 3) que permitam o intercâmbio simples e eficiente de dados e conhecimentos, em vez de adotar a tecnologia mais empolgante e sofisticada.

Fontes de dados de terceiros

A consumerização da tecnologia significa que toda empresa é essencialmente agora uma empresa de tecnologia. A consequência é que essas empresas - e

Machine Translated by Google

cada vez mais agências governamentais—querem disponibilizar seus dados para seus clientes e usuários, seja como parte de seu serviço ou como uma assinatura separada. Por exemplo, o Bureau of Labor Statistics dos Estados Unidos publica várias estatísticas sobre o mercado de trabalho dos EUA. A Administração Nacional de Aeronáutica e Espaço (NASA) publica vários dados de suas iniciativas de pesquisa. O Facebook compartilha dados com empresas que anunciam em sua plataforma.

Por que as empresas querem disponibilizar seus dados? Os dados são fixos e um flywheel é criado permitindo que os usuários integrem e estendam seu aplicativo no aplicativo de um usuário. Maior adoção e uso do usuário significa mais dados, o que significa que os usuários podem integrar mais dados em seus aplicativos e sistemas de dados. O efeito colateral é que agora existem fontes quase infinitas de dados de terceiros.

O acesso direto a dados de terceiros geralmente é feito por meio de APIs, por meio de compartilhamento de dados em uma plataforma de nuvem ou por meio de download de dados. As APIs geralmente fornecem recursos de integração profunda, permitindo que os clientes extraiam e enviem dados. Por exemplo, muitos CRMs oferecem APIs que seus usuários podem integrar em seus sistemas e aplicativos. Vemos um fluxo de trabalho comum para obter dados de um CRM, combinar os dados do CRM por meio do modelo de pontuação do cliente e, em seguida, usar o ETL reverso para enviar esses dados de volta ao CRM para que os vendedores entrem em contato com leads mais qualificados.

Filas de mensagens e plataformas de streaming de eventos

As arquiteturas orientadas a eventos são difundidas em aplicativos de software e estão prontas para aumentar ainda mais sua popularidade. Primeiro, as filas de mensagens e as plataformas de streaming de eventos — camadas críticas em arquiteturas orientadas a eventos — são mais fáceis de configurar e gerenciar em um ambiente de nuvem. Em segundo lugar, a ascensão dos aplicativos de dados – aplicativos que integram diretamente a análise em tempo real – está crescendo cada vez mais. As arquiteturas orientadas a eventos são ideais nessa configuração porque os eventos podem acionar o trabalho no aplicativo e alimentar análises quase em tempo real.

Machine Translated by Google

Observe que o streaming de dados (neste caso, mensagens e streams) atravessa muitos estágios do ciclo de vida da engenharia de dados. Ao contrário de um RDBMS, que geralmente é conectado diretamente a um aplicativo, as linhas de dados de streaming às vezes são menos nítidas. Esses sistemas são usados como sistemas de origem, mas geralmente atravessam o ciclo de vida da engenharia de dados devido à sua natureza transitória. Por exemplo, você pode usar uma plataforma de fluxo de eventos para transmissão de mensagens em um aplicativo orientado a eventos, um sistema de origem. A mesma plataforma de streaming de eventos pode ser usada no estágio de ingestão e transformação para processar dados para análises em tempo real.

Como sistemas de origem, as filas de mensagens e as plataformas de streaming de eventos são usadas de várias maneiras, desde o roteamento de mensagens entre microsserviços que ingerem milhões de eventos por segundo de dados de eventos de aplicativos da Web, móveis e IoT. Vamos examinar as filas de mensagens e as plataformas de streaming de eventos um pouco mais de perto.

Filas de mensagens

Uma ***fila de mensagens*** é um mecanismo para enviar dados de forma assíncrona (geralmente como pequenas mensagens individuais, nos kilobytes) entre sistemas discretos usando um modelo de publicação e assinatura. Os dados são publicados em uma fila de mensagens e entregues a um ou mais assinantes (Figura 5-9). O assinante confirma o recebimento da mensagem, removendo-a da fila.



Figura 5-9. Uma fila de mensagens simples

As filas de mensagens permitem que aplicativos e sistemas sejam desacoplados uns dos outros e são amplamente utilizadas em arquiteturas de microsserviços. A fila de mensagens armazena mensagens em buffer para lidar com picos de carga transitórios e torna as mensagens duráveis por meio de uma arquitetura distribuída com replicação.

Machine Translated by Google

As filas de mensagens são um ingrediente crítico para microsserviços desacoplados e arquiteturas orientadas a eventos. Algumas coisas a serem lembradas com as filas de mensagens são a frequência de entrega, a ordem das mensagens e a escalabilidade.

Pedido e entrega de mensagens

A ordem em que as mensagens são criadas, enviadas e recebidas pode afetar significativamente os assinantes downstream. Em geral, a ordem nas filas de mensagens distribuídas é um problema complicado. As filas de mensagens geralmente aplicam uma noção difusa de ordem e primeiro a entrar, primeiro a sair (FIFO). FIFO estrito significa que se a mensagem A for ingerida antes da mensagem B, a mensagem A sempre será entregue antes da mensagem B. Na prática, as mensagens podem ser publicadas e recebidas fora de ordem, especialmente em sistemas de mensagens altamente distribuídos.

Por exemplo, filas padrão do Amazon SQS fazer o melhor esforço para preservar a ordem das mensagens. O SQS também oferece filas FIFO, que oferecem garantias muito mais fortes ao custo de despesas gerais extras.

Em geral, não assuma que suas mensagens serão entregues em ordem, a menos que sua tecnologia de fila de mensagens garanta isso. Normalmente, você precisa projetar para entrega de mensagens fora de ordem.

Frequência de entrega

As mensagens podem ser enviadas exatamente uma vez ou pelo menos uma vez. Se uma mensagem for enviada ***exatamente uma vez,*** depois que o assinante reconhecer a mensagem, a mensagem desaparecerá e não será entregue novamente. As mensagens enviadas ***pelo menos uma vez*** podem ser consumidas por vários assinantes ou pelo mesmo assinante mais de uma vez. Isso é ótimo quando duplicações ou redundância não importam.

Idealmente, os sistemas devem ser ***idempotentes.*** Em um sistema idempotente, o resultado de processar uma mensagem uma vez é idêntico ao resultado de processá-la várias vezes. Isso ajuda a explicar uma variedade de cenários sutis. Por exemplo, mesmo que nosso sistema possa garantir a entrega exatamente uma vez, um consumidor pode processar totalmente uma mensagem, mas falhar logo antes de confirmar o processamento. A mensagem será efetivamente processada duas vezes, mas um sistema idempotente lida com esse cenário normalmente.

Machine Translated by Google

Escalabilidade

As filas de mensagens mais populares utilizadas em aplicativos orientados a eventos são escaláveis horizontalmente, sendo executadas em vários servidores. Isso permite que essas filas aumentem e diminuam dinamicamente, armazenam mensagens em buffer quando os sistemas ficam para trás e armazenam mensagens de forma durável para resiliência contra falhas. No entanto, isso pode criar uma variedade de complicações, conforme mencionado anteriormente (múltiplas entregas e pedidos confusos).

Plataformas de streaming de eventos

De certa forma, uma ***plataforma de fluxo de eventos*** é uma continuação de uma fila de mensagens em que as mensagens são passadas de produtores para consumidores. Conforme discutido anteriormente neste capítulo, a grande diferença entre mensagens e fluxos é que uma fila de mensagens é usada principalmente para rotear mensagens com certas garantias de entrega. Em contraste, uma plataforma de streaming de eventos é usada para ingerir e processar dados em um log ordenado de registros. Em uma plataforma de streaming de eventos, os dados são retidos por um tempo e é possível reproduzir mensagens de um momento anterior.

Vamos descrever um evento relacionado a uma plataforma de streaming de eventos. Conforme mencionado no Capítulo 3, um evento é “algo que aconteceu, normalmente uma mudança no ***estado*** de algo”. Um evento tem os seguintes recursos: uma chave, um valor e um carimbo de data/hora. Vários carimbos de data/hora de valor-chave podem estar contidos em um único evento. Por exemplo, um evento para um pedido de comércio eletrônico pode ter esta aparência:

Chave: "Pedido nº 12345"
Valor: "SKU 123, preço de compra de $ 100"
Carimbo de data/hora: "2023-01-02 06:01:00"

Vejamos algumas das características críticas de uma plataforma de streaming de eventos que você deve conhecer como engenheiro de dados.

Tópicos

Em uma plataforma de streaming de eventos, um produtor transmite eventos para um tópico, uma coleção de eventos relacionados. Um tópico pode conter alertas de fraude,

Machine Translated by Google

pedidos ou leituras de temperatura de dispositivos IoT, por exemplo. Um tópico pode ter zero, um ou vários produtores e clientes na maioria das plataformas de streaming de eventos.

Usando o exemplo de evento anterior, um tópico pode ser pedidos da web. Além disso, vamos enviar este tópico para alguns consumidores, como atendimento e marketing. Este é um excelente exemplo de linhas borradas entre análise e um sistema orientado a eventos. O assinante de atendimento usará eventos para acionar um processo de atendimento, enquanto o marketing executa análises em tempo real para ajustar as campanhas de marketing (Figura 5-10).

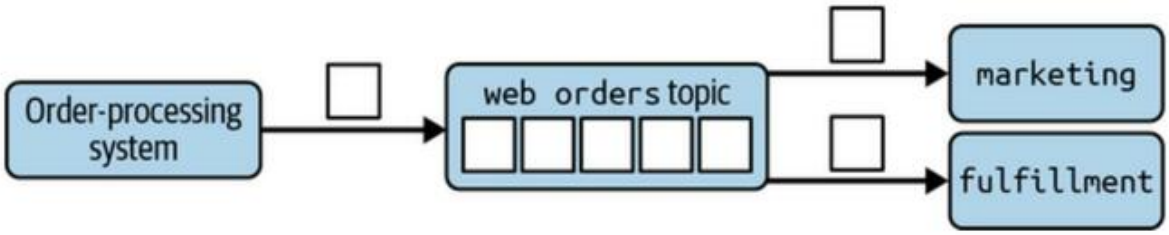


Figura 5-10. Um sistema de processamento de pedidos gera eventos (pequenos retângulos) e os publica no tópico** de pedidos da web . **Dois assinantes — marketing e** atendimento — extraem **eventos do tópico.

Partições de fluxo

As partições de fluxo são subdivisões de um fluxo em vários fluxos. Uma boa analogia é uma rodovia de várias faixas. Ter várias pistas permite paralelismo e maior taxa de transferência. As mensagens são distribuídas entre partições por ***chave de partição.*** As mensagens com a mesma chave de partição sempre terminarão na mesma partição.

Na Figura 5-11, por exemplo, cada mensagem tem um ID numérico — mostrado dentro do círculo que representa a mensagem — que usamos como chave de partição. Para determinar a partição, dividimos por 3 e pegamos o resto. Indo de baixo para cima, as partições têm resto 0, 1 e 2, respectivamente.

Machine Translated by Google

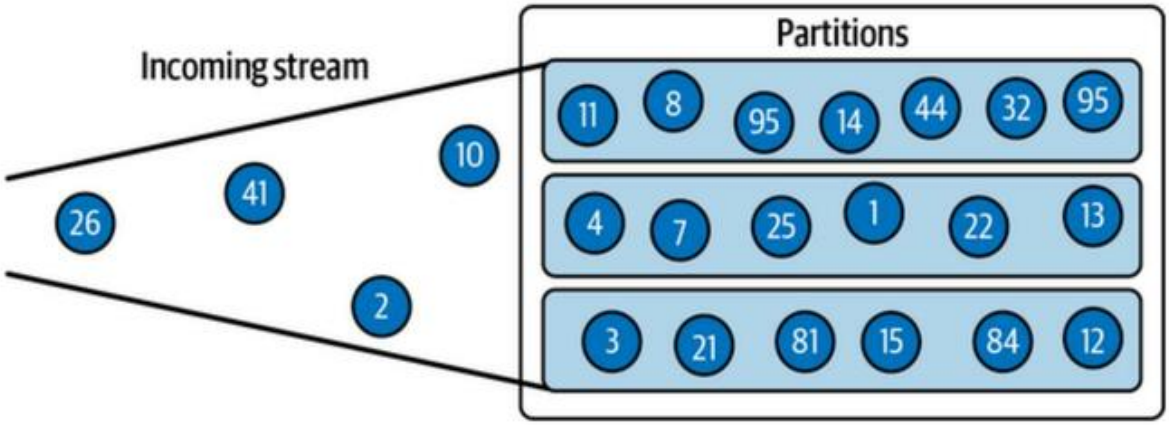


Figura 5-11. Um fluxo de mensagens de entrada dividido em três partições

Defina uma chave de partição para que as mensagens que devem ser processadas juntas tenham a mesma chave de partição. Por exemplo, é comum nas configurações de IoT querer enviar todas as mensagens de um determinado dispositivo para o mesmo servidor de processamento. Podemos conseguir isso usando um ID de dispositivo como chave de partição e, em seguida, configurando um servidor para consumir de cada partição.

Uma preocupação importante com o particionamento de fluxo é garantir que sua chave de partição não gere ***pontos de acesso—um*** número desproporcional de mensagens entregues a uma partição. Por exemplo, se cada dispositivo IoT estiver localizado em um determinado estado dos EUA, podemos usar o estado como a chave de partição. Dada uma distribuição de dispositivos proporcional à população do estado, as partições contendo Califórnia, Texas, Flórida e Nova York podem ficar sobrecarregadas, com outras partições relativamente subutilizadas. Certifique-se de que sua chave de partição distribuirá mensagens uniformemente pelas partições.

Tolerância a falhas e resiliência

As plataformas de fluxo de eventos geralmente são sistemas distribuídos, com fluxos armazenados em vários nós. Se um nó ficar inativo, outro nó o substituirá e o fluxo ainda estará acessível. Isso significa que os registros não são perdidos; você pode optar por excluir registros, mas isso é outra história. Essa tolerância a falhas e resiliência tornam as plataformas de streaming uma boa escolha quando você precisa de um sistema que possa produzir, armazenar e ingerir dados de eventos de forma confiável.

Com quem você vai trabalhar

Machine Translated by Google

Ao acessar os sistemas de origem, é essencial entender as pessoas com quem você trabalhará. Em nossa experiência, a boa diplomacia e os relacionamentos com as partes interessadas dos sistemas de origem são uma parte subestimada e crucial da engenharia de dados bem-sucedida.

Quem são esses interessados? Normalmente, você lidará com duas categorias de partes interessadas: partes interessadas de sistemas e partes interessadas de dados (Figura 5-12). Um ***stakeholder de sistemas*** constrói e mantém os sistemas de origem; estes podem ser engenheiros de software, desenvolvedores de aplicativos e terceiros. As partes interessadas de dados possuem e controlam o acesso aos dados que você deseja, geralmente manipulados pela TI, um grupo de governança de dados ou terceiros. As partes interessadas em sistemas e dados geralmente são pessoas ou equipes diferentes; às vezes, eles são os mesmos.

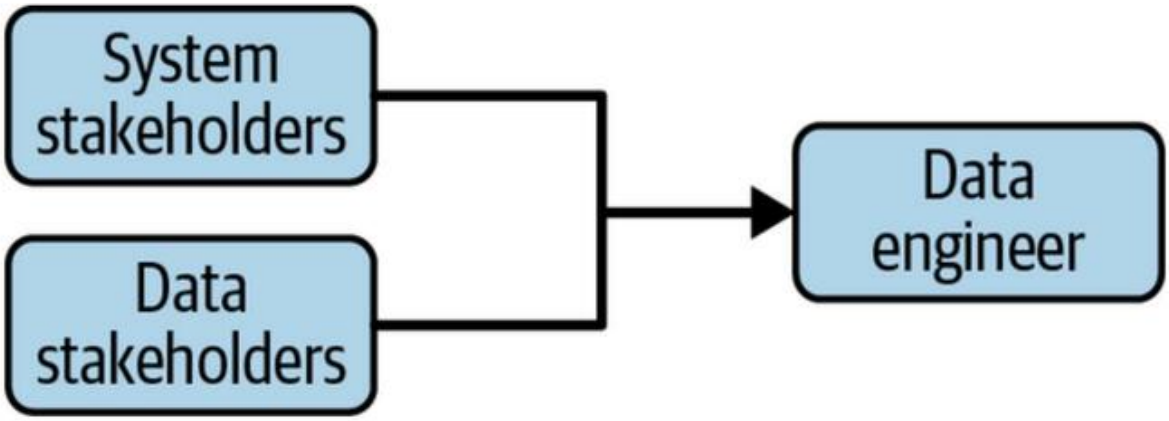


Figura 5-12. As partes interessadas upstream do engenheiro de dados

Muitas vezes, você fica à mercê da capacidade da parte interessada de seguir as práticas corretas de engenharia de software, gerenciamento de banco de dados e desenvolvimento. Idealmente, as partes interessadas estão fazendo DevOps e trabalhando de maneira ágil. Sugerimos a criação de um ciclo de feedback entre engenheiros de dados e partes interessadas dos sistemas de origem para conscientizar sobre como os dados são consumidos e usados. Essa é uma das áreas mais negligenciadas em que os engenheiros de dados podem obter muito valor. Quando algo acontece com os dados de origem upstream - e algo vai acontecer, seja um esquema ou alteração de dados, um servidor ou banco de dados com falha ou outros eventos importantes - você quer ter certeza de que está ciente do impacto que esses problemas causarão tem em seus sistemas de engenharia de dados.

Machine Translated by Google

Pode ser útil ter um contrato de dados com os proprietários do sistema de origem upstream. O que é um contrato de dados? James Denmore oferece esta definição:

> ***Um contrato de dados é um acordo escrito entre o proprietário de um sistema de origem e a equipe que está consumindo dados desse sistema para uso em um pipeline de dados. O contrato deve indicar quais dados estão sendo extraídos, por meio de qual método (completo, incremental), com que frequência e quem (pessoa, equipe) são os contatos tanto para o sistema de origem quanto para a ingestão. Os contratos de dados devem ser armazenados em um local conhecido e fácil de encontrar, como um repositório GitHub ou site de documentação interna. Se possível, formate os contratos de dados de forma padronizada para que possam ser integrados ao processo de desenvolvimento ou consultados programaticamente.***[7]

Além disso, considere estabelecer um SLA com provedores upstream. Um SLA fornece expectativas do que você pode esperar dos sistemas de origem nos quais você confia. Um exemplo de SLA pode ser “os dados dos sistemas de origem estarão disponíveis de forma confiável e de alta qualidade”. Um objetivo de nível de serviço (SLO) mede o desempenho em relação ao que você concordou no SLA. Por exemplo, dado seu exemplo de SLA, um SLO pode ser “os sistemas de origem terão 99% de tempo de atividade”. Se um contrato de dados ou SLA/SLO parecer muito formal, pelo menos defina verbalmente as expectativas para garantias do sistema de origem para tempo de atividade, qualidade dos dados e qualquer outra coisa importante para você. Os proprietários upstream de sistemas de origem precisam entender seus requisitos para que possam fornecer os dados de que você precisa.

Subcorrentes e seu impacto na fonte Sistemas

Ao contrário de outras partes do ciclo de vida da engenharia de dados, os sistemas de origem geralmente estão fora do controle do engenheiro de dados. Há uma suposição implícita (alguns podem chamar de ***esperança)*** de que as partes interessadas e os proprietários dos sistemas de origem - e os dados que eles produzem - estão seguindo as melhores práticas relativas ao gerenciamento de dados, DataOps (e DevOps), DODD (mencionado no Capítulo 2) arquitetura de dados , orquestração e software

Machine Translated by Google

Engenharia. O engenheiro de dados deve obter o máximo de suporte upstream possível para garantir que as subcorrentes sejam aplicadas quando os dados forem gerados nos sistemas de origem. Isso fará com que o restante das etapas do ciclo de vida da engenharia de dados prossiga com muito mais facilidade.

Como as subcorrentes afetam os sistemas de origem? Vamos dar uma olhada.

Segurança

A segurança é fundamental, e a última coisa que você deseja é criar acidentalmente um ponto de vulnerabilidade em um sistema de origem. Aqui estão algumas áreas a serem consideradas:

O sistema de origem é arquitetado para que os dados sejam seguros e criptografados, tanto com os dados em repouso quanto enquanto os dados são transmitidos?

Você precisa acessar o sistema de origem pela Internet pública ou está usando uma rede virtual privada (VPN)?

Mantenha senhas, tokens e credenciais para o sistema de origem bloqueados com segurança. Por exemplo, se você estiver usando chaves Secure Shell (SSH), use um gerenciador de chaves para proteger suas chaves; a mesma regra se aplica a senhas — use um gerenciador de senhas ou um provedor de logon único (SSO).

Você confia no sistema de origem? Sempre certifique-se de confiar, mas verifique se o sistema de origem é legítimo. Você não quer receber dados de um agente mal-intencionado.

Gestão de dados

O gerenciamento de dados de sistemas de origem é um desafio para engenheiros de dados. Na maioria dos casos, você terá apenas controle periférico — se houver algum controle — sobre os sistemas de origem e os dados que eles produzem. Na medida do possível, você deve entender a maneira como os dados são gerenciados nos sistemas de origem, pois isso influenciará diretamente em como você ingere, armazena e transforma os dados.

Aqui estão algumas áreas a serem consideradas:

Machine Translated by Google

Gestão de dados

Os dados e sistemas upstream são governados de forma confiável e fácil de

entende de moda? Quem gerencia os dados?

Qualidade dos dados

Como você garante a qualidade e a integridade dos dados em sistemas upstream?

Trabalhar com equipes de sistema de origem para definir expectativas sobre dados e comunicação.

Esquema

Espere que os esquemas upstream sejam alterados. Sempre que possível, colabore

com as equipes do sistema de origem para serem notificadas sobre mudanças iminentes no esquema.

Gerenciamento de dados mestre

A criação de registros upstream é controlada por um master data

prática ou sistema de gestão?

Privacidade e ética

Você tem acesso a dados brutos ou os dados serão ofuscados? o que

são as implicações dos dados de origem? Quanto tempo fica retido? Ele muda de local com

base em políticas de retenção?

Regulatório

Com base nos regulamentos, você deve acessar os dados?

DataOps

A excelência operacional — DevOps, DataOps, MLOps, XOps — deve se estender para cima e para baixo em toda a pilha e dar suporte à engenharia de dados e ao ciclo de vida.

Machine Translated by Google

Embora isso seja ideal, muitas vezes não é totalmente realizado.

Como você está trabalhando com partes interessadas que controlam os sistemas de origem e os dados que eles produzem, você precisa garantir que possa observar e monitorar o tempo de atividade e o uso dos sistemas de origem e responder quando ocorrerem incidentes. Por exemplo, quando o banco de dados do aplicativo do qual você depende para o CDC excede sua capacidade de E/S e precisa ser redimensionado, como isso afetará sua capacidade de receber dados desse sistema? Você poderá acessar os dados ou eles ficarão indisponíveis até que o banco de dados seja redimensionado? Como isso afetará os relatórios? Em outro exemplo, se a equipe de engenharia de software estiver implantando continuamente, uma alteração de código pode causar falhas inesperadas no próprio aplicativo. Como a falha afetará sua capacidade de acessar os bancos de dados que alimentam o aplicativo? Os dados estarão atualizados?

Configure uma cadeia de comunicação clara entre a engenharia de dados e as equipes que dão suporte aos sistemas de origem. Idealmente, essas equipes de partes interessadas incorporaram o DevOps em seu fluxo de trabalho e cultura. Isso ajudará bastante a atingir os objetivos do DataOps (um irmão do DevOps), para resolver e reduzir erros rapidamente. Como mencionamos anteriormente, os engenheiros de dados precisam se integrar às práticas de DevOps das partes interessadas e vice-versa. O DataOps bem-sucedido funciona quando todas as pessoas estão envolvidas e se concentram em fazer os sistemas funcionarem de forma holística.

Algumas considerações de DataOps são as seguintes:

Automação

Há a automação impactando o sistema de origem, como atualizações de código e novos recursos. Depois, há a automação DataOps que você configurou para seus fluxos de trabalho de dados. Faz um problema na fonte a automação do sistema afeta a automação do fluxo de trabalho de dados? Em caso afirmativo, considere desacoplar esses sistemas para que eles possam realizar a automação de forma independente.

Observabilidade

Machine Translated by Google

Como você saberá quando houver um problema com um sistema de origem, como uma interrupção ou um problema de qualidade de dados? Configure o monitoramento para o sistema de origem uptime (ou use o monitoramento criado pela equipe que possui a fonte sistema). Configure verificações para garantir que os dados do sistema de origem estejam em conformidade com as expectativas de uso downstream. Por exemplo, os dados são de boa qualidade? O esquema está em conformidade? São registros de clientes consistente? Os dados são criptografados conforme estipulado pela política interna?

Resposta a incidentes

Qual é o seu plano se algo ruim acontecer? Por exemplo, como será seu pipeline de dados se comporta se um sistema de origem ficar offline? Qual é o seu planeja preencher os dados “perdidos” assim que o sistema de origem estiver on-line novamente?

Arquitetura de dados

Semelhante ao gerenciamento de dados, a menos que você esteja envolvido no design e manutenção da arquitetura do sistema de origem, você terá pouco impacto na arquitetura do sistema de origem upstream. Você também deve entender como a arquitetura upstream é projetada e seus pontos fortes e fracos.
Converse frequentemente com as equipes responsáveis pelos sistemas de origem para entender os fatores discutidos nesta seção e garantir que seus sistemas atendam às suas expectativas. Saber onde a arquitetura funciona bem e onde ela não afeta a forma como você projeta seu pipeline de dados.

Aqui estão algumas coisas a serem consideradas em relação às arquiteturas do sistema de origem:

Confiabilidade

Todos os sistemas sofrem de entropia em algum ponto, e as saídas serão desviadas do esperado. Bugs são introduzidos e falhas aleatórias acontecem. O sistema produz saídas previsíveis? Quantas vezes pode

Machine Translated by Google

esperamos que o sistema falhe? Qual é o tempo médio de reparo para que o sistema volte a ter confiabilidade suficiente?

Durabilidade

Tudo falha. Um servidor pode morrer, a zona ou região de uma nuvem pode ficar offline ou outros problemas podem surgir. Você precisa explicar como um falhas ou interrupções inevitáveis afetarão seus sistemas de dados gerenciados. Quão o sistema de origem lida com a perda de dados de falhas de hardware ou interrupções de rede? Qual é o plano para lidar com interrupções por um longo período e limitar o raio de explosão de uma interrupção?

Disponibilidade

O que garante que o sistema de origem esteja funcionando, funcionando e disponível quando deveria estar?

Pessoas

Quem é responsável pelo design do sistema de origem e como você saberá se foram feitas alterações importantes na arquitetura? Um engenheiro de dados precisa trabalhar com as equipes que mantêm os sistemas de origem e garantir que esses sistemas são arquitetados de forma confiável. Crie um SLA com a equipe do sistema de origem para definir as expectativas sobre possíveis falhas do sistema.

Orquestração

Ao orquestrar em seu fluxo de trabalho de engenharia de dados, você se preocupará principalmente em garantir que sua orquestração possa acessar o sistema de origem, o que requer o acesso, autenticação e autorização corretos à rede.

Machine Translated by Google

Aqui estão algumas coisas para pensar em relação à orquestração para sistemas de origem:

Cadência e frequência

Os dados estão disponíveis em um horário fixo ou você pode acessar novos dados sempre que quiser?

Estruturas comuns

Os engenheiros de software e dados usam o mesmo gerenciador de contêiner, como o Kubernetes? Faria sentido integrar cargas de trabalho de aplicativos e dados no mesmo cluster Kubernetes? Se você estiver usando um estrutura de orquestração como o Airflow, faz sentido integrá-lo com a equipe de aplicativos upstream? Não há uma resposta correta aqui, mas você precisa equilibrar os benefícios da integração com os riscos do acoplamento rígido.

Engenharia de software

À medida que o cenário de dados muda para ferramentas que simplificam e automatizam o acesso aos sistemas de origem, você provavelmente precisará escrever código. Aqui estão algumas considerações ao escrever código para acessar um sistema de origem:

Rede

Certifique-se de que seu código será capaz de acessar a rede onde o sistema de origem reside. Além disso, sempre pense em redes seguras. São você está acessando um URL HTTPS pela internet pública, SSH ou uma VPN?

Autenticação e autorização

Você tem as credenciais adequadas (tokens, nome de usuário/senhas) para acessar o sistema de origem? Onde você armazenará essas credenciais para que elas

Machine Translated by Google

não aparecem em seu código ou controle de versão? Você tem as funções corretas do IAM para executar as tarefas codificadas?

Padrões de acesso

Como você está acessando os dados? Você está usando uma API e como está lidando com solicitações REST/GraphQL, volumes de dados de resposta e paginação? Se você estiver acessando dados por meio de um driver de banco de dados, o driver compatível com o banco de dados que você está acessando? Para qualquer padrão de acesso, como são tratadas as tentativas e os tempos limite?

Orquestração

Seu código se integra a uma estrutura de orquestração e pode ser executado como um fluxo de trabalho orquestrado?

Paralelização

Como você está gerenciando e dimensionando o acesso paralelo aos sistemas de origem?

Implantação

Como você está lidando com a implantação de alterações no código-fonte?

Conclusão

Os sistemas de origem e seus dados são vitais no ciclo de vida da engenharia de dados. Os engenheiros de dados tendem a tratar os sistemas de origem como “problema de outra pessoa” – faça isso por sua conta e risco! Os engenheiros de dados que abusam dos sistemas de origem podem precisar procurar outro emprego quando a produção cair.

Se há um pau, há também uma cenoura. Uma melhor colaboração com as equipes do sistema de origem pode levar a dados de maior qualidade, resultados mais bem-sucedidos e melhores produtos de dados. Crie um fluxo bidirecional de comunicação com seus colegas nessas equipes; configurar processos para notificar o esquema

Machine Translated by Google

e alterações de aplicativos que afetam análises e ML. Comunique suas necessidades de dados de forma proativa para ajudar as equipes de aplicativos na engenharia de dados processo.

Esteja ciente de que a integração entre engenheiros de dados e equipes de sistemas de origem está crescendo. Um exemplo é o ETL reverso, que há muito vive nas sombras, mas recentemente ganhou destaque. Também discutimos que a plataforma de streaming de eventos poderia desempenhar um papel em arquiteturas e análises orientadas a eventos; um sistema de origem também pode ser um sistema de engenharia de dados. Construir sistemas compartilhados onde faz sentido fazê-lo.

Procure oportunidades para criar produtos de dados voltados para o usuário.
Converse com as equipes de aplicativos sobre análises que eles gostariam de apresentar aos usuários ou locais onde o ML poderia melhorar a experiência do usuário. Torne as equipes de aplicativos partes interessadas na engenharia de dados e encontre maneiras de compartilhar seus sucessos.

Agora que você entende os tipos de sistemas de origem e os dados que eles geram, veremos a seguir maneiras de armazenar esses dados.

Recursos adicionais

"O que, por que e quando do design de tabela única com o DynamoDB" por Alex De Brie

"Testar a qualidade dos dados em escala com o Deequ" por Dustin Lange et al.

"Modernizando a Indexação de Dados Corporativos" por Benjamin Douglas e Mohammad Mohtasham

"Evolução e Compatibilidade do Esquema" da Confluent documentação

"NoSQL: O que há em um nome" por Eric Evans

"O log: o que todo engenheiro de software deve saber sobre o real
A abstração unificadora de dados de tempo" por ay Kreps

Machine Translated by Google

Conceitos de sistema de banco de dados por Abraham (Avi) Silberschatz et al. (McGraw Hill)

Internos do banco de dados por Alex Petrov (O'Reilly)

1 Keith D. Foote, “A Brief History of Non-Relational Databases”, Dataversity, 19 de junho de 2018, ***https://oreil.ly/5Ukg2.***

2 Excelente série de Nemil Dalal sobre a história do MongoDB conta algumas histórias angustiantes de abuso de banco de dados e suas consequências para startups incipientes.

3 Martin Kleppmann, ***Designing Data Intensive Applications*** (Sebastopol, CA: O'Reilly, 2017), 49, ***https://oreil.ly/v1NhG.***

4 Aashish Mehra, “Graph Database Market Worth $ 5,1 Billion by 2026: Exclusive Report by MarketsandMarkets”, Cision PR Newswire, 30 de julho de 2021, ***https://oreil.ly/mGVkY.***

5 Para um exemplo, veja Michael S. Mikowski, “RESTful APIs: The Big Lie”, 10 de agosto de 2015, ***https://oreil.ly/rqja3.***

6 Martin Fowler, “How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh,” MartinFowler.com, 20 de maio de 2019, ***https://oreil.ly/TEdJF.***

7 Leia ***a referência de bolso de pipelines de dados*** (O'Reilly) para obter mais informações sobre como um contrato de dados deve ser escrito.

Machine Translated by Google

Capítulo 6. Armazenamento

O armazenamento é a pedra angular do ciclo de vida da engenharia de dados (Figura 6-1) e está na base de seus principais estágios — ingestão, transformação e atendimento. Os dados são armazenados muitas vezes ao longo do ciclo de vida. Parafraseando um velho ditado, é armazenamento até o fim. Se os dados forem necessários segundos, minutos, dias, meses ou anos depois, eles devem persistir no armazenamento até que os sistemas estejam prontos para consumi-los para processamento e transmissão adicionais. Conhecer o caso de uso dos dados e a maneira como você os recuperará no futuro é o primeiro passo para escolher as soluções de armazenamento adequadas para sua arquitetura de dados.

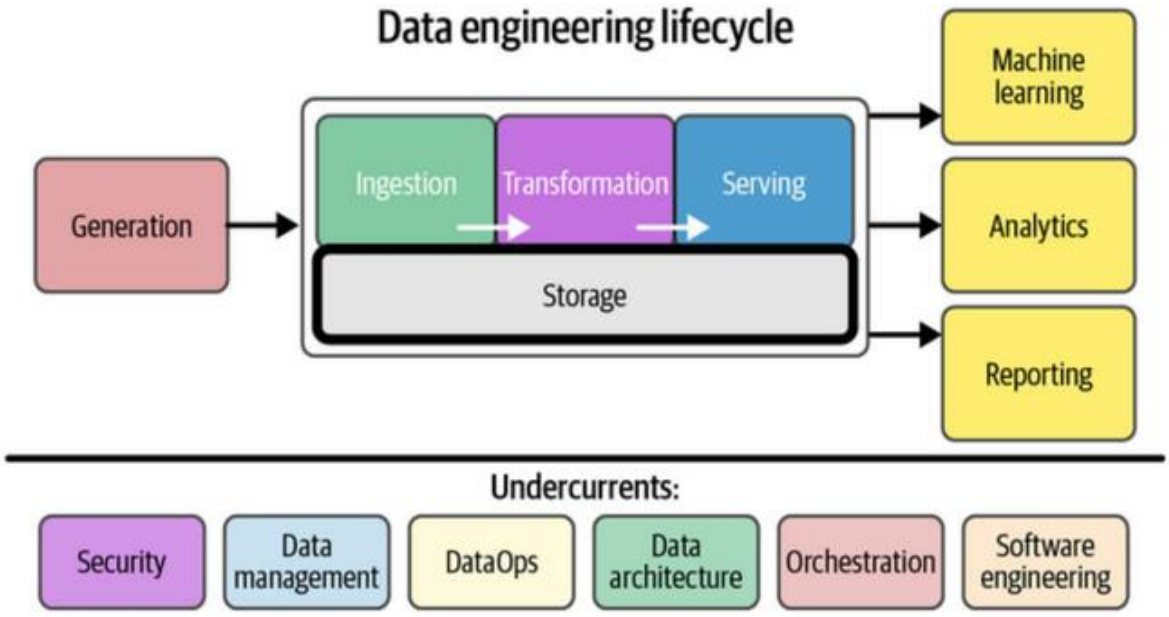


Figura 6-1. O armazenamento desempenha um papel central no ciclo de vida da engenharia de dados

Também discutimos o armazenamento no Capítulo 5, mas com uma diferença de foco e domínio de controle. Os sistemas de origem geralmente não são mantidos ou controlados por engenheiros de dados. O armazenamento que os engenheiros de dados manipulam diretamente, no qual nos concentraremos neste capítulo, abrange os estágios do ciclo de vida da engenharia de dados desde a ingestão de dados dos sistemas de origem até o fornecimento de dados para agregar valor com análises, ciência de dados etc. todo o ciclo de vida da engenharia de dados de alguma forma.

Machine Translated by Google

Para entender o armazenamento, vamos começar estudando os ***ingredientes*** básicos que compõem os sistemas de armazenamento, incluindo discos rígidos, unidades de estado sólido e memória do sistema (veja a Figura 6-2). É essencial entender as características básicas das tecnologias de armazenamento físico para avaliar as vantagens e desvantagens inerentes a qualquer arquitetura de armazenamento. Esta seção também discute serialização e compactação, elementos-chave de software de armazenamento prático. (Adiamos uma discussão técnica mais profunda sobre serialização e compactação para o Apêndice A.) Também discutimos o armazenamento em ***cache,*** que é fundamental na montagem de sistemas de armazenamento.

Machine Translated by Google

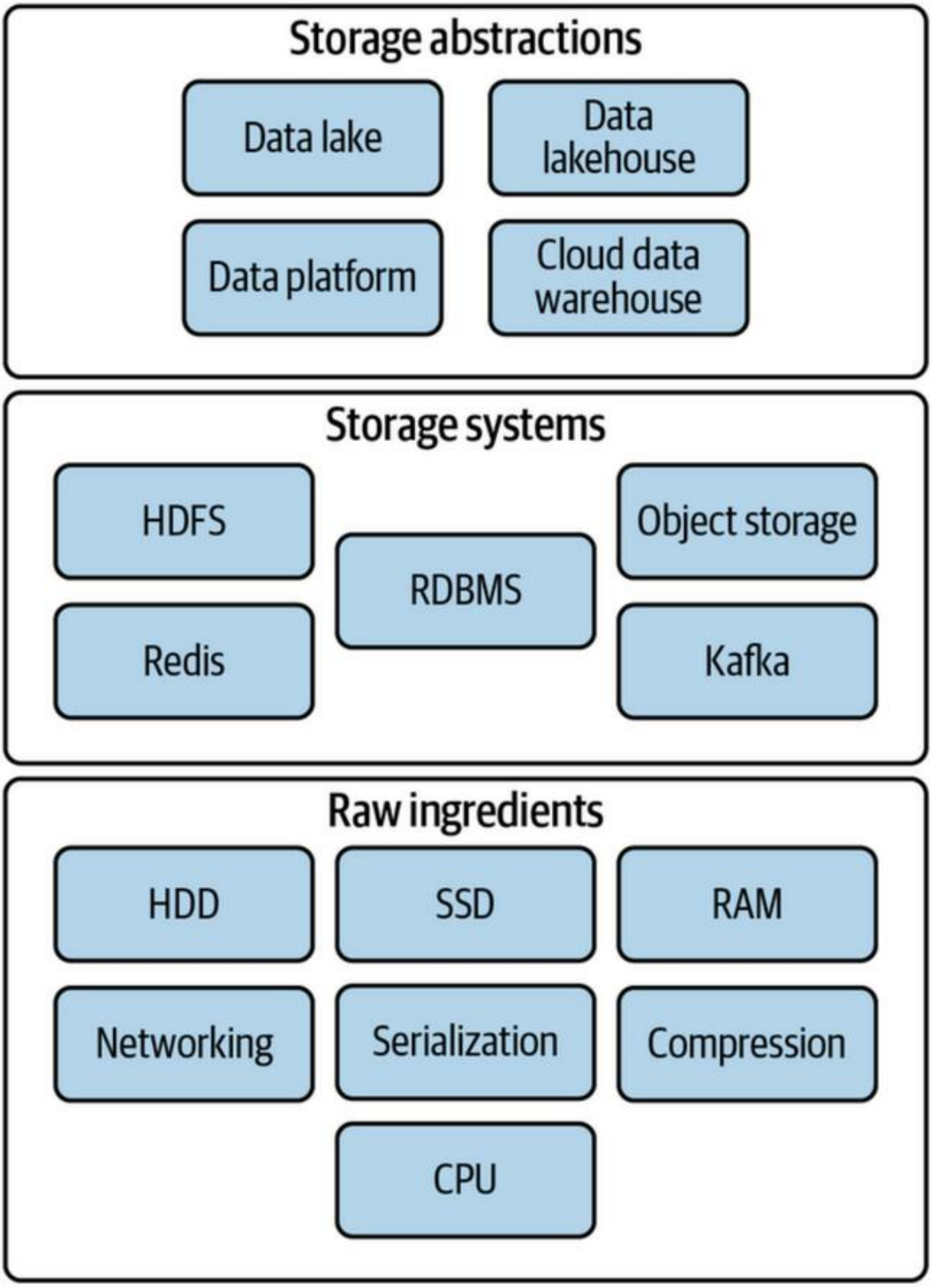


Figura 6-2. Ingredientes brutos, sistemas de armazenamento e abstrações de armazenamento

Machine Translated by Google

A seguir, veremos ***os sistemas de armazenamento.*** Na prática, não acessamos diretamente a memória do sistema ou os discos rígidos. Esses componentes de armazenamento físico existem dentro de servidores e clusters que podem ingerir e recuperar dados usando vários paradigmas de acesso.

Por fim, veremos as ***abstrações de armazenamento.*** Os sistemas de armazenamento são montados em um data warehouse em nuvem, um data lake etc. Ao construir pipelines de dados, os engenheiros escolhem as abstrações apropriadas para armazenar seus dados à medida que passam pelos estágios de ingestão, transformação e serviço.

Ingredientes brutos do armazenamento de dados

O armazenamento é tão comum que é fácil tomá-lo como garantido. Muitas vezes ficamos surpresos com o número de engenheiros de software e de dados que usam armazenamento todos os dias, mas não fazem ideia de como ele funciona nos bastidores ou das vantagens e desvantagens inerentes a várias mídias de armazenamento. Como resultado, vemos o armazenamento usado de algumas maneiras bem... interessantes. Embora os serviços gerenciados atuais liberem potencialmente os engenheiros de dados das complexidades do gerenciamento de servidores, os engenheiros de dados ainda precisam estar cientes das características essenciais dos componentes subjacentes, considerações de desempenho, durabilidade e custos.

Na maioria das arquiteturas de dados, os dados frequentemente passam por armazenamento magnético, SSDs e memória enquanto percorrem as várias fases de processamento de um pipeline de dados. Os sistemas de armazenamento e consulta de dados geralmente seguem receitas complexas envolvendo sistemas distribuídos, vários serviços e várias camadas de armazenamento de hardware. Esses sistemas exigem os ingredientes crus certos para funcionar corretamente.

Vejamos alguns dos ingredientes básicos do armazenamento de dados: unidades de disco, memória, rede e CPU, serialização, compactação e armazenamento em cache.

Unidade de disco magnético

Os discos magnéticos utilizam pratos giratórios revestidos com um filme ferromagnético (Figura 6-3). Este filme é magnetizado por uma cabeça de leitura/gravação durante as operações de gravação para codificar fisicamente os dados binários. A cabeça de leitura/gravação detecta o

Machine Translated by Google

campo magnético e emite um fluxo de bits durante as operações de leitura. As unidades de disco magnético existem há muito tempo. Ainda assim, eles formam a espinha dorsal dos sistemas de armazenamento de dados em massa porque são significativamente mais baratos que os SSDs por gigabyte de dados armazenados.

Machine Translated by Google

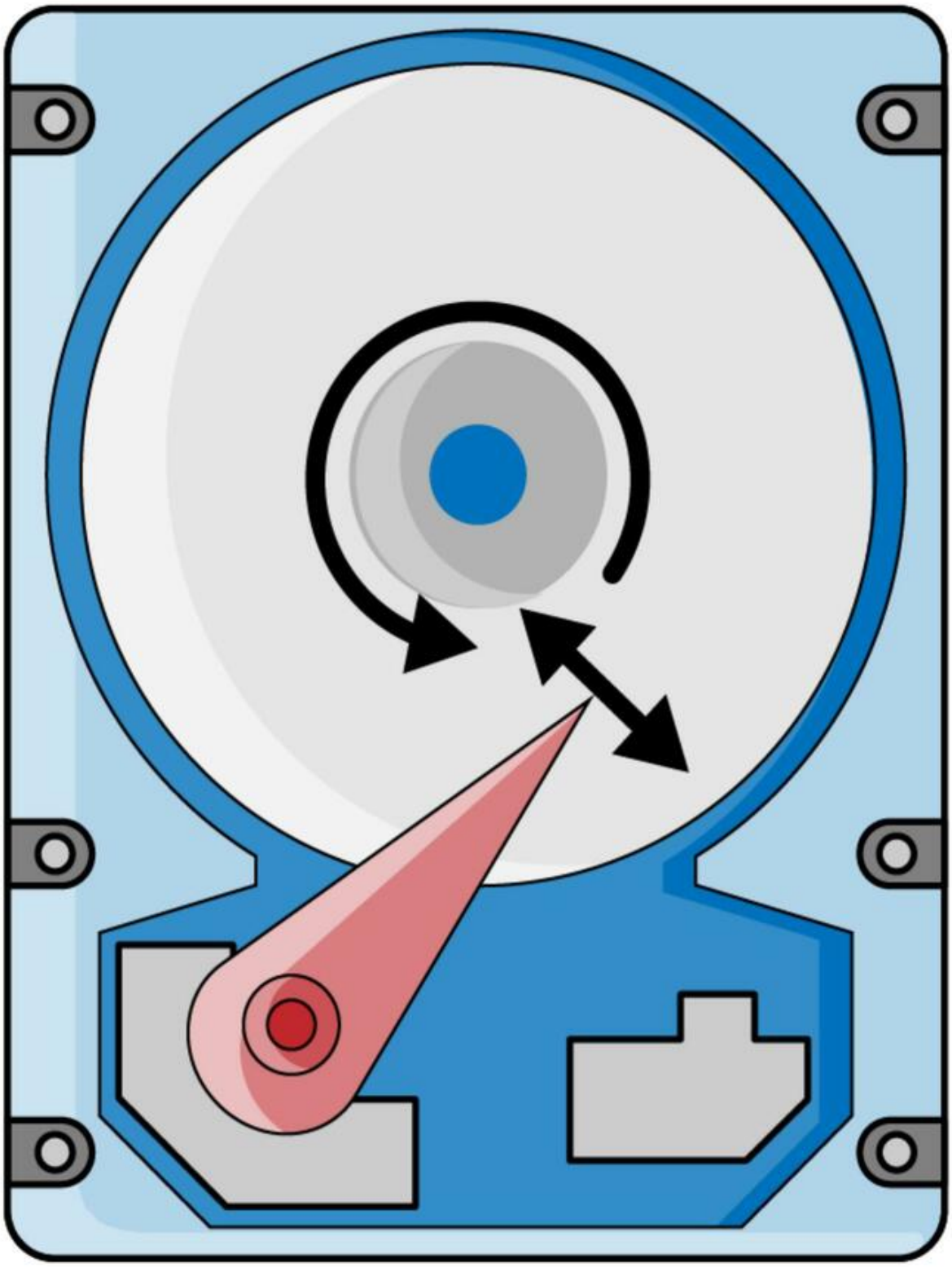


Figura 6-3. O movimento e a rotação da cabeça do disco magnético são essenciais na latência de acesso aleatório

Por um lado, esses discos tiveram melhorias extraordinárias em desempenho, densidade de armazenamento e custo. Por outro lado, os SSDs[1]

Machine Translated by Google

superam drasticamente os discos magnéticos em várias métricas. Atualmente, as unidades de disco magnético comerciais custam cerca de 3 centavos por gigabyte de capacidade. (Observe que usaremos frequentemente as abreviações ***HDD*** e ***SSD*** para denotar disco magnético rotativo e unidades de estado sólido, respectivamente.)

A IBM desenvolveu a tecnologia de unidade de disco magnético na década de 1950. Desde então, as capacidades dos discos magnéticos cresceram de forma constante. O primeiro drive de disco magnético comercial, o IBM 350, tinha capacidade de 3,75 megabytes. No momento da redação deste artigo, unidades magnéticas que armazenam 20 TB estão disponíveis comercialmente. Na verdade, os discos magnéticos continuam a ter uma inovação rápida, com métodos como gravação magnética assistida por calor (HAMR), gravação magnética shingled (SMR) e compartimentos de disco preenchidos com hélio sendo usados para obter densidades de armazenamento cada vez maiores. Apesar das melhorias contínuas na capacidade da unidade, outros aspectos do desempenho do HDD são prejudicados pela física.

Primeiro, ***a velocidade de transferência do disco,*** a taxa na qual os dados podem ser lidos e gravados, não é proporcional à capacidade do disco. A capacidade do disco é dimensionada com ***densidade*** de área (gigabits armazenados por polegada quadrada), enquanto a velocidade de transferência é dimensionada com ***densidade linear*** (bits por polegada). Isso significa que, se a capacidade do disco aumentar por um fator de 4, a velocidade de transferência aumentará apenas por um fator de 2. Consequentemente, as unidades de data center atuais suportam velocidades máximas de transferência de dados de 200–-300 MB/s. Para enquadrar isso de outra maneira, são necessárias mais de 20 horas para ler todo o conteúdo de uma unidade magnética de 30 TB, assumindo uma velocidade de transferência de 300 MB/s.

Uma segunda limitação importante é o tempo de busca. Para acessar os dados, a unidade deve realocar fisicamente as cabeças de leitura/gravação para a trilha apropriada no disco.
Terceiro, para encontrar um dado específico no disco, o controlador de disco deve esperar que esses dados girem sob as cabeças de leitura/gravação. Isso leva à ***latência rotacional.*** Unidades comerciais típicas girando a 7.200 rotações por minuto (RPM) tempo de busca e latência rotacional levam a mais de quatro milissegundos de latência média geral (tempo para acessar um dado selecionado). Uma quarta limitação são as operações de entrada/saída por segundo (IOPS), críticas para bancos de dados transacionais. Uma unidade magnética varia de 50 a 500 IOPS.

Machine Translated by Google

Vários truques podem melhorar a latência e a velocidade de transferência. Usar uma velocidade rotacional mais alta pode aumentar a taxa de transferência e diminuir a latência rotacional. Limitar o raio do prato do disco ou gravar dados em apenas uma faixa estreita do disco reduz o tempo de busca. No entanto, nenhuma dessas técnicas torna as unidades magnéticas remotamente competitivas com os SSDs para pesquisas de acesso aleatório. Os SSDs podem fornecer dados com latência significativamente menor, IOPS mais alta e velocidades de transferência mais altas, em parte porque não há disco fisicamente giratório ou cabeçote magnético para esperar.

Como mencionado anteriormente, os discos magnéticos ainda são valorizados em data centers por seus baixos custos de armazenamento de dados. Além disso, os drives magnéticos podem sustentar taxas de transferência extraordinariamente altas por meio do paralelismo. Essa é a ideia crítica por trás do armazenamento de objetos em nuvem: os dados podem ser distribuídos em milhares de discos em clusters. As taxas de transferência de dados aumentam drasticamente ao ler vários discos simultaneamente, limitados principalmente pelo desempenho da rede em vez da taxa de transferência do disco. Assim, componentes de rede e CPUs também são matérias-primas essenciais em sistemas de armazenamento, e retornaremos a esses tópicos em breve.

Disco de Estado Sólido

As unidades de estado sólido (SSDs) armazenam dados como cargas em células de memória flash. Os SSDs eliminam os componentes mecânicos das unidades magnéticas; os dados são lidos por meios puramente eletrônicos. Os SSDs podem pesquisar dados aleatórios em menos de 0,1 ms (100 microssegundos). Além disso, os SSDs podem dimensionar as velocidades de transferência de dados e IOPS dividindo o armazenamento em partições com vários controladores de armazenamento executados em paralelo. Os SSDs comerciais podem suportar velocidades de transferência de muitos gigabytes por segundo e dezenas de milhares de IOPS.

Devido a essas características de desempenho excepcionais, os SSDs revolucionaram os bancos de dados transacionais e são o padrão aceito para implantações comerciais de sistemas OLTP. Os SSDs permitem que bancos de dados relacionais como PostgreSQL, MySQL e SQL Server lidem com milhares de transações por segundo.

Machine Translated by Google

No entanto, os SSDs não são atualmente a opção padrão para armazenamento de dados analíticos de alta escala. Novamente, isso se resume ao custo. SSDs comerciais normalmente custam de 20 a 30 centavos (USD) por gigabyte de capacidade, quase 10 vezes o custo por capacidade de uma unidade magnética. Assim, o armazenamento de objetos em discos magnéticos surgiu como a principal opção para armazenamento de dados em larga escala em data lakes e data warehouses em nuvem.

Os SSDs ainda desempenham um papel significativo nos sistemas OLAP. Alguns bancos de dados OLAP aproveitam o cache SSD para dar suporte a consultas de alto desempenho em dados acessados com frequência. À medida que o OLAP de baixa latência se torna mais popular, esperamos que o uso de SSD nesses sistemas siga o exemplo.

Memória de acesso aleatório

Costumamos usar os termos ***memória de acesso aleatório*** (RAM) e ***memória*** de forma intercambiável. A rigor, drives magnéticos e SSDs também servem como memória que armazena dados para posterior recuperação de acesso aleatório, mas a RAM tem várias características específicas:

Está conectado a uma CPU e mapeado no espaço de endereço da CPU.

Armazena o código que as CPUs executam e os dados que esse código processa diretamente.

É ***volátil,*** enquanto unidades magnéticas e SSDs são ***não voláteis.*** Embora possam ocasionalmente falhar e corromper ou perder dados, as unidades geralmente retêm dados quando desligadas. A RAM perde dados em menos de um segundo quando está desenergizada.

Oferece velocidades de transferência significativamente mais altas e tempos de recuperação mais rápidos do que o armazenamento SSD. A memória DDR5 – o mais recente padrão amplamente usado para RAM – oferece latência de recuperação de dados na ordem de 100 ns, aproximadamente 1.000 vezes mais rápido que o SSD. Uma CPU típica pode suportar largura de banda de 100 GB/s para memória anexada e milhões de IOPS. (As estatísticas variam drasticamente dependendo do número de canais de memória e outros detalhes de configuração.)

Machine Translated by Google

É significativamente mais caro que o armazenamento SSD, por aproximadamente US$ 10/GB (no momento da redação deste artigo).

É limitado na quantidade de RAM conectada a uma CPU individual e controlador de memória.

Isso aumenta ainda mais a complexidade e o custo. Servidores de alta memória normalmente utilizam muitas CPUs interconectadas em uma placa, cada uma com um bloco de RAM conectado.

Ainda é significativamente mais lento que o cache da CPU, um tipo de memória localizada diretamente na matriz da CPU ou no mesmo pacote. O cache armazena dados acessados com frequência e recentemente para recuperação ultrarrápida durante o processamento. Os designs de CPU incorporam várias camadas de cache de tamanho e características de desempenho variadas.

Quando falamos de memória do sistema, quase sempre queremos dizer ***RAM dinâmica,*** uma forma de memória de alta densidade e baixo custo. A RAM dinâmica armazena dados como cargas em capacitores. Esses capacitores vazam com o tempo, portanto, os dados devem ser ***atualizados*** com frequência (lidos e reescritos) para evitar a perda de dados. O controlador de memória de hardware lida com esses detalhes técnicos; os engenheiros de dados simplesmente precisam se preocupar com as características de largura de banda e latência de recuperação.
Outras formas de memória, como ***RAM estática,*** são usadas em aplicativos especializados, como caches de CPU.

As CPUs atuais quase sempre empregam a ***arquitetura von Neumann,*** com código e dados armazenados juntos no mesmo espaço de memória. No entanto, as CPUs normalmente também suportam a opção de desabilitar a execução de código em páginas específicas de memória para maior segurança. Esse recurso lembra a ***arquitetura Harvard,*** que separa código e dados.

A RAM é usada em vários sistemas de armazenamento e processamento e pode ser usada para armazenamento em cache, processamento de dados ou índices. Vários bancos de dados tratam a RAM como uma camada de armazenamento primária, permitindo um desempenho ultrarrápido de leitura e gravação. Nessas aplicações, os engenheiros de dados devem sempre ter em mente a volatilidade da RAM. Mesmo que os dados armazenados na memória sejam replicados em um cluster, uma queda de energia que desative vários nós pode causar perda de dados. As arquiteturas destinadas a armazenar dados de forma durável podem usar backups de bateria e despejar automaticamente todos os dados no disco em caso de perda de energia.

Machine Translated by Google

Rede e CPU

Por que estamos mencionando a rede e a CPU como ingredientes brutos para armazenar dados? Cada vez mais, os sistemas de armazenamento são distribuídos para melhorar o desempenho, a durabilidade e a disponibilidade. Mencionamos especificamente que os discos magnéticos individuais oferecem desempenho de transferência relativamente baixo, mas um cluster de discos paraleliza as leituras para uma escala de desempenho significativa. Enquanto os padrões de armazenamento, como arrays redundantes de discos independentes (RAID), são paralelizados em um único servidor, os clusters de armazenamento de objetos em nuvem operam em uma escala muito maior, com discos distribuídos em uma rede e até vários data centers e zonas de disponibilidade.

As ***zonas de disponibilidade*** são uma construção de nuvem padrão que consiste em ambientes de computação com energia, água e outros recursos independentes. O armazenamento multizonal melhora a disponibilidade e a durabilidade dos dados.

As CPUs lidam com os detalhes de solicitações de serviço, agregando leituras e distribuindo gravações. O armazenamento se torna um aplicativo da Web com uma API, componentes de serviço de back-end e balanceamento de carga. O desempenho do dispositivo de rede e a topologia da rede são fatores-chave para obter alto desempenho.

Os engenheiros de dados precisam entender como a rede afetará os sistemas que eles constroem e usam. Os engenheiros equilibram constantemente a durabilidade e a disponibilidade alcançadas ao distribuir os dados geograficamente versus o desempenho e os benefícios de custo de manter o armazenamento em uma pequena área geográfica e próximo aos consumidores ou gravadores de dados. O Apêndice B abrange a rede em nuvem e as principais ideias relevantes.

Serialização

A serialização é outro ingrediente de armazenamento bruto e um elemento crítico do design do banco de dados. As decisões sobre a serialização informarão o desempenho das consultas em uma rede, sobrecarga de CPU, latência de consulta e muito mais. Projetar um data lake, por exemplo, envolve a escolha de um sistema de armazenamento básico (por exemplo, Amazon S3) e padrões para serialização que equilibrem interoperabilidade com considerações de desempenho.

Machine Translated by Google

O que é serialização, exatamente? Os dados armazenados na memória do sistema pelo software geralmente não estão em um formato adequado para armazenamento em disco ou transmissão em uma rede. A serialização é o processo de achatamento e empacotamento de dados em um formato padrão que um leitor poderá decodificar. Os formatos de serialização fornecem um padrão de troca de dados. Podemos codificar dados de maneira baseada em linhas como um arquivo XML, JSON ou CSV e passá-los para outro usuário que pode decodificá-los usando uma biblioteca padrão. Um algoritmo de serialização possui lógica para manipulação de tipos, impõe regras na estrutura de dados e permite a troca entre linguagens de programação e CPUs. O algoritmo de serialização também possui regras para tratamento de exceções. Por exemplo, objetos Python podem conter referências cíclicas; o algoritmo de serialização pode gerar um erro ou limitar a profundidade de aninhamento ao encontrar um ciclo.

O armazenamento de banco de dados de baixo nível também é uma forma de serialização. Os bancos de dados relacionais orientados a linhas organizam os dados como linhas no disco para oferecer suporte a pesquisas rápidas e atualizações in-loco. Os bancos de dados colunares organizam os dados em arquivos de coluna para otimizar a compactação altamente eficiente e oferecer suporte a verificações rápidas de grandes volumes de dados. Cada opção de serialização vem com um conjunto de compensações, e os engenheiros de dados ajustam essas opções para otimizar o desempenho de acordo com os requisitos.

Fornecemos um catálogo mais detalhado de técnicas e formatos comuns de serialização de dados no Apêndice A. Sugerimos que os engenheiros de dados se familiarizem com práticas e formatos comuns de serialização, especialmente os formatos atuais mais populares (por exemplo, Apache Parquet), serialização híbrida (por exemplo, Apache Hudi) e serialização na memória (por exemplo, Apache Arrow).

Compressão

A compactação é outro componente crítico da engenharia de armazenamento. Em um nível básico, a compactação torna os dados menores, mas os algoritmos de compactação interagem com outros detalhes dos sistemas de armazenamento de maneiras complexas.

A compressão altamente eficiente tem três vantagens principais em sistemas de armazenamento. Primeiro, os dados são menores e, portanto, ocupam menos espaço no disco. Segundo,

Machine Translated by Google

compressão aumenta a velocidade de varredura prática por disco. Com uma taxa de compactação de 10:1, passamos da varredura de 200 MB/s por disco magnético para uma taxa efetiva de 2 GB/s por disco.

A terceira vantagem está no desempenho da rede. Considerando que uma conexão de rede entre uma instância do Amazon EC2 e o S3 fornece 10 gigabits por segundo (Gbps) de largura de banda, uma taxa de compactação de 10:1 aumenta a largura de banda de rede efetiva para 100 Gbps.

A compactação também vem com desvantagens. A compactação e descompactação de dados envolve tempo extra e consumo de recursos para ler ou gravar dados. Empreendemos uma discussão mais detalhada sobre algoritmos de compressão e trade-offs no Apêndice A.

Cache

Já mencionamos o cache em nossa discussão sobre RAM. A ideia central do cache é armazenar dados acessados com frequência ou recentemente em uma camada de acesso rápido. Quanto mais rápido o cache, maior o custo e menos espaço de armazenamento disponível. Os dados acessados com menos frequência são armazenados em um armazenamento mais barato e mais lento.
Os caches são críticos para o serviço, processamento e transformação de dados.

À medida que analisamos os sistemas de armazenamento, é útil colocar todos os tipos de armazenamento que utilizamos dentro de uma ***hierarquia de cache*** (Tabela 6-1). A maioria dos sistemas de dados práticos depende de muitas camadas de cache montadas a partir do armazenamento com características de desempenho variadas. Isso começa dentro das CPUs; os processadores podem implantar até quatro camadas de cache. Descemos na hierarquia para RAM e SSDs. O armazenamento de objetos em nuvem é um nível inferior que oferece suporte à retenção e durabilidade de dados de longo prazo, ao mesmo tempo em que permite a veiculação de dados e a movimentação dinâmica de dados em pipelines.

Machine Translated by Google

Tabela 4.4.4 Ativação

Ativação/Desativação Ajuda

Machine Translated by Google

Machine Translated by Google

r

eu

c

eu

n

g

uma

n

d

p

e

r

f

o

r

m

uma

n

c

e

c

h

uma

r

uma

c

t

e

r

eu

s

ti

c

s

Machine Translated by Google

Tipo de armazenamento	Latência de busca de dados[a]	Largura de banda	Preço
cache da CPU	1 nanossegundo	1 TB/s	
BATER	0,1 microssegundos	100 GB/s	US$ 10/GB
SSD	0,1 milissegundos	4 GB/s	US$ 0,20/GB
HD	4 milissegundos	300 MB/s	US$ 0,03/GB
Armazenamento de objetos	100 milissegundos	3 GB/s por instância	US$ 0,02/GB por mês
Armazenamento de arquivo	12 horas	O mesmo que armazenamento de objetos quando os dados são acessível	US$ 0,004/GB por mês

a Um microssegundo é 1.000 nanossegundos e um milissegundo é 1.000 microssegundos.

Podemos pensar no armazenamento de arquivo como um ***cache reverso.*** Armazenamento de arquivo oferece características de acesso inferiores a custos baixos. O armazenamento de arquivos é geralmente usado para backups de dados e para atender à conformidade de retenção de dados requisitos. Em cenários típicos, esses dados serão acessados apenas em um emergência (por exemplo, dados em um banco de dados podem ser perdidos e precisam ser recuperados, ou uma empresa pode precisar consultar dados históricos para fins legais descoberta).

Sistemas de armazenamento de dados

Esta seção abrange os principais sistemas de armazenamento de dados que você encontrará como engenheiro de dados. Os sistemas de armazenamento existem em um nível de abstração acima do bruto ingredientes. Por exemplo, os discos magnéticos são um ingrediente bruto de armazenamento, enquanto principais plataformas de armazenamento de objetos em nuvem e HDFS são sistemas de armazenamento que utilizar discos magnéticos. Existem níveis ainda mais altos de abstração de armazenamento, como como data lakes e lakehouses (que abordamos em “Data Engineering Storage Abstrações”).

Máquina única versus armazenamento distribuído

Machine Translated by Google

À medida que o armazenamento de dados e os padrões de acesso se tornam mais complexos e superam a utilidade de um único servidor, a distribuição de dados para mais de um servidor se torna necessária. Os dados podem ser armazenados em vários servidores, conhecidos como ***armazenamento distribuído.*** Este é um sistema distribuído cuja finalidade é armazenar dados de forma distribuída (Figura 6-4).

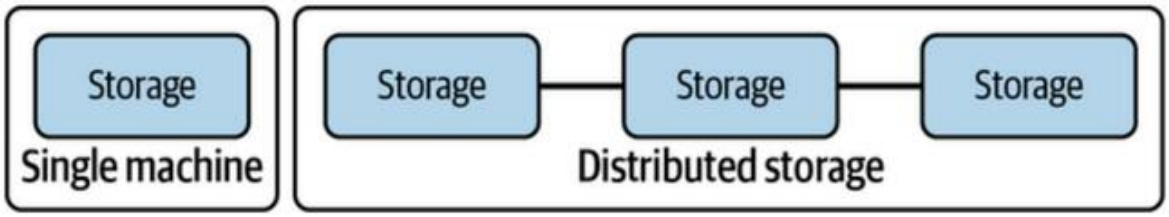


Figura 6-4. Máquina única versus armazenamento distribuído em vários servidores

O armazenamento distribuído coordena as atividades de vários servidores para armazenar, recuperar e processar dados mais rapidamente e em maior escala, ao mesmo tempo em que fornece redundância caso um servidor fique indisponível. O armazenamento distribuído é comum em arquiteturas em que você deseja redundância e escalabilidade integradas para grandes quantidades de dados. Por exemplo, armazenamento de objetos, Apache Spark e data warehouses em nuvem contam com arquiteturas de armazenamento distribuído.

Os engenheiros de dados devem sempre estar cientes dos paradigmas de consistência dos sistemas distribuídos, que exploraremos a seguir.

Consistência eventual versus forte

Um desafio com sistemas distribuídos é que seus dados estão espalhados por vários servidores. Como esse sistema mantém os dados consistentes?
Infelizmente, os sistemas distribuídos representam um dilema para o armazenamento e a precisão das consultas. Leva tempo para replicar as alterações nos nós de um sistema; muitas vezes existe um equilíbrio entre obter dados atuais e obter dados atuais “mais ou menos” em um banco de dados distribuído. Vejamos dois padrões de consistência comuns em sistemas distribuídos: eventual e forte.

Cobrimos a conformidade com ACID ao longo deste livro, começando no Capítulo 5. Outro acrônimo é ***BASE,*** que significa ***basicamente disponível, estado suave, consistência eventual.*** Pense nisso como o oposto de ACID. BASE é a base da consistência eventual. Vamos explorar brevemente seus componentes:

Machine Translated by Google

Basicamente disponível

A consistência não é garantida, mas os esforços nas leituras e gravações do banco de dados são feitos com base no melhor esforço, o que significa que dados consistentes estão disponíveis a maior parte do tempo.

Estado suave

O estado da transação é confuso e não se sabe se o transação é confirmada ou não confirmada.

Consistência eventual

Em ***algum*** momento, a leitura de dados retornará valores consistentes.

Se a leitura de dados em um sistema eventualmente consistente não é confiável, por que usá-lo? A consistência eventual é uma compensação comum em sistemas distribuídos de grande escala. Se você deseja dimensionar horizontalmente (em vários nós) para processar dados em grandes volumes, eventualmente, a consistência geralmente é o preço que você pagará. A consistência eventual permite que você recupere dados rapidamente sem verificar se você tem a versão mais recente em todos os nós.

O oposto da consistência eventual é ***a consistência forte.*** Com consistência forte, o banco de dados distribuído garante que as gravações em qualquer nó sejam distribuídas primeiro com um consenso e que todas as leituras no banco de dados retornem valores consistentes. Você usará uma consistência forte quando puder tolerar uma latência de consulta mais alta e exigir dados corretos toda vez que ler o banco de dados.

Geralmente, os engenheiros de dados tomam decisões sobre consistência em três lugares. Primeiro, a própria tecnologia de banco de dados prepara o terreno para um certo nível de consistência. Em segundo lugar, os parâmetros de configuração do banco de dados terão impacto na consistência. Terceiro, os bancos de dados geralmente suportam alguma configuração de consistência em um nível de consulta individual. Por exemplo, o DynamoDB suporta leituras eventualmente consistentes e leituras fortemente consistentes. Fortemente

Machine Translated by Google

leituras consistentes são mais lentas e consomem mais recursos, portanto, é melhor usá-las com moderação, mas elas estão disponíveis quando a consistência é necessária.

Você deve entender como seu banco de dados lida com a consistência. Novamente, os engenheiros de dados têm a tarefa de entender profundamente a tecnologia e usá-la para resolver problemas de forma adequada. Um engenheiro de dados pode precisar negociar requisitos de consistência com outras partes interessadas técnicas e de negócios.

Observe que isso é um problema de tecnologia e organizacional; certifique-se de ter reunido os requisitos de suas partes interessadas e escolha suas tecnologias adequadamente.

Armazenamento de arquivo

Lidamos com arquivos todos os dias, mas a noção de arquivo é um tanto sutil. Um ***arquivo*** é uma entidade de dados com características específicas de leitura, gravação e referência usadas por software e sistemas operacionais. Definimos um arquivo para ter as seguintes características:

Comprimento finito

Um arquivo é um fluxo de bytes de comprimento finito.

Anexar operações

Podemos anexar bytes ao arquivo até os limites do armazenamento do host sistema.

Acesso aleatório

Podemos ler de qualquer local no arquivo ou gravar atualizações em qualquer localização.

O armazenamento de objetos se comporta muito como o armazenamento de arquivos, mas com diferenças importantes. Enquanto preparamos o cenário para o armazenamento de objetos discutindo primeiro o armazenamento de arquivos, o armazenamento de objetos é sem dúvida muito mais importante para o tipo de dados

Machine Translated by Google

engenharia que você fará hoje. Faremos referências à discussão sobre armazenamento de objetos extensivamente nas próximas páginas.

Os sistemas de armazenamento de arquivos organizam os arquivos em uma árvore de diretórios. A referência de diretório para um arquivo pode ser assim:

```
/Users/matthewousley/output.txt
```

Quando forneço isso ao sistema operacional, ele inicia no diretório raiz /, encontra Users, matthewousley e, finalmente, output.txt. Trabalhando da esquerda, cada diretório está contido dentro de um diretório pai, até que finalmente chegamos ao arquivo output.txt. Este exemplo usa a semântica do Unix, mas a semântica de referência de arquivo do Windows é semelhante. O sistema de arquivos armazena cada diretório como metadados sobre os arquivos e diretórios que ele contém. Esses metadados consistem no nome de cada entidade, detalhes de permissão relevantes e um ponteiro para a entidade real. Para localizar um arquivo no disco, o sistema operacional analisa os metadados em cada nível de hierarquia e segue o ponteiro para a próxima entidade do subdiretório até finalmente chegar ao próprio arquivo.

Observe que outras entidades de dados semelhantes a arquivos geralmente não têm necessariamente todas essas propriedades. Por exemplo, ***objetos*** no armazenamento de objetos suportam apenas a primeira característica, comprimento finito, mas ainda são extremamente úteis. Discutimos isso em “Armazenamento de Objetos”.

Nos casos em que os paradigmas de armazenamento de arquivos são necessários para um pipeline, tenha cuidado com o estado e tente usar ambientes efêmeros o máximo possível. Mesmo que você precise processar arquivos em um servidor com um disco anexado, use o armazenamento de objetos para armazenamento intermediário entre as etapas de processamento. Tente reservar o processamento manual de arquivos de baixo nível para etapas de ingestão únicas ou para os estágios exploratórios do desenvolvimento do pipeline.

Armazenamento em disco

local O tipo mais conhecido de armazenamento de arquivos é um sistema de arquivos gerenciado pelo sistema operacional em uma partição de disco local de SSD ou disco magnético. New Technology File System (NTFS) e ext4 são sistemas de arquivos populares no Windows e Linux, respectivamente. O sistema operacional lida com os detalhes

Machine Translated by Google

de armazenar entidades de diretório, arquivos e metadados. Os sistemas de arquivos são projetados para gravar dados para permitir uma recuperação fácil em caso de perda de energia durante uma gravação, embora quaisquer dados não gravados ainda sejam perdidos.

Sistemas de arquivos locais geralmente suportam leitura completa após consistência de escrita; a leitura imediatamente após uma gravação retornará os dados gravados. Os sistemas operacionais também empregam várias estratégias de bloqueio para gerenciar tentativas de gravação simultâneas em um arquivo.

Os sistemas de arquivos de disco local também podem oferecer suporte a recursos avançados, como journaling, snapshots, redundância, extensão do sistema de arquivos em vários discos, criptografia completa de disco e compactação. Em "Block Storage", também discutimos RAID.

Armazenamento conectado à rede

Os sistemas ***de armazenamento conectado à rede*** (NAS) fornecem um sistema de armazenamento de arquivos para clientes em uma rede. O NAS é uma solução predominante para servidores; eles geralmente são fornecidos com hardware de interface NAS dedicado integrado. Embora haja penalidades de desempenho ao acessar o sistema de arquivos em uma rede, também existem vantagens significativas para a virtualização de armazenamento, incluindo redundância e confiabilidade, controle refinado de recursos, pool de armazenamento em vários discos para grandes volumes virtuais e compartilhamento de arquivos em várias máquinas. Os engenheiros devem estar cientes do modelo de consistência fornecido por sua solução NAS, especialmente quando vários clientes potencialmente acessarão os mesmos dados.

Uma alternativa popular ao NAS é uma rede de área de armazenamento (SAN), mas os sistemas SAN fornecem acesso em nível de bloco sem a abstração do sistema de arquivos. Cobrimos os sistemas SAN em "Block Storage".

Serviços de sistema de arquivos em nuvem

Os serviços de sistema de arquivos em nuvem fornecem um sistema de arquivos totalmente gerenciado para uso com várias VMs e aplicativos em nuvem, incluindo potencialmente clientes fora do ambiente de nuvem. Os sistemas de arquivos em nuvem não devem ser confundidos com armazenamento padrão anexado a VMs - geralmente, armazenamento em bloco com um sistema de arquivos gerenciado pelo sistema operacional da VM. Os sistemas de arquivos em nuvem se comportam

Machine Translated by Google

muito parecido com as soluções NAS, mas os detalhes de rede, gerenciamento de clusters de disco, falhas e configuração são totalmente tratados pelo fornecedor da nuvem.

Por exemplo, o Amazon Elastic File System (EFS) é um exemplo extremamente popular de um serviço de sistema de arquivos em nuvem. O armazenamento é exposto por meio do protocolo NFS 4, que também é usado por sistemas NAS. O EFS oferece dimensionamento automático e preços de pagamento por armazenamento sem necessidade de reserva avançada de armazenamento. O serviço também fornece consistência de leitura após gravação ***local*** (ao ler da máquina que executou a gravação). Ele também oferece consistência aberta após o fechamento em todo o sistema de arquivos. Em outras palavras, quando um aplicativo fecha um arquivo, os leitores subsequentes verão as alterações salvas no arquivo fechado.

Armazenamento em bloco

Fundamentalmente, ***o armazenamento em bloco*** é o tipo de armazenamento bruto fornecido por SSDs e discos magnéticos. Na nuvem, o armazenamento em bloco virtualizado é o padrão para VMs. Essas abstrações de armazenamento em bloco permitem um controle preciso do tamanho do armazenamento, escalabilidade e durabilidade dos dados além do oferecido pelos discos brutos.

Em nossa discussão anterior sobre SSDs e discos magnéticos, mencionamos que, com esses dispositivos de acesso aleatório, o sistema operacional pode buscar, ler e gravar quaisquer dados no disco. Um ***bloco*** é a menor unidade de dados endereçável suportada por um disco. Isso geralmente era 512 bytes de dados utilizáveis em discos mais antigos, mas agora aumentou para 4.096 bytes para a maioria dos discos atuais, tornando as gravações menos refinadas, mas reduzindo drasticamente a sobrecarga do gerenciamento de blocos. Os blocos normalmente contêm bits extras para detecção/correção de erros e outros metadados.

Blocos em discos magnéticos são dispostos geometricamente em um prato físico.
Dois blocos na mesma trilha podem ser lidos sem mover a cabeça, enquanto a leitura de dois blocos em trilhas separadas requer uma busca. O tempo de busca pode ocorrer entre blocos em um SSD, mas é infinitesimal comparado ao tempo de busca para faixas de disco magnético.

Aplicativos de armazenamento em bloco

Machine Translated by Google

Os sistemas de banco de dados transacionais geralmente acessam os discos em um nível de bloco para dispor os dados para um desempenho ideal. Para bancos de dados orientados a linhas, isso originalmente significava que linhas de dados eram gravadas como fluxos contínuos; a situação ficou mais complicada com a chegada dos SSDs e suas melhorias de desempenho de tempo de busca associadas, mas os bancos de dados transacionais ainda contam com o alto desempenho de acesso aleatório oferecido pelo acesso direto a um dispositivo de armazenamento em bloco.

O armazenamento em bloco também continua sendo a opção padrão para discos de inicialização do sistema operacional em VMs na nuvem. O dispositivo de bloco é formatado da mesma forma que seria diretamente em um disco físico, mas o armazenamento geralmente é virtualizado. (consulte “Armazenamento em bloco virtualizado em nuvem”.)

ATAQUE

RAID significa ***matriz redundante de discos independentes,*** conforme observado anteriormente.
O RAID controla simultaneamente vários discos para melhorar a durabilidade dos dados, aprimorar o desempenho e combinar a capacidade de várias unidades. Uma matriz pode aparecer para o sistema operacional como um dispositivo de bloco único. Muitos esquemas de codificação e paridade estão disponíveis, dependendo do equilíbrio desejado entre largura de banda efetiva aprimorada e maior tolerância a falhas (tolerância para muitas falhas de disco).

Rede da área de armazenamento

Os sistemas ***de rede de área de armazenamento*** (SAN) fornecem dispositivos de armazenamento em bloco virtualizados em uma rede, normalmente de um pool de armazenamento. A abstração de SAN pode permitir o dimensionamento de armazenamento refinado e aprimorar o desempenho, a disponibilidade e a durabilidade. Você pode encontrar sistemas SAN se estiver trabalhando com sistemas de armazenamento no local; você também pode encontrar uma versão em nuvem do SAN, como na próxima subseção.

Armazenamento em bloco virtualizado na nuvem

As soluções ***de armazenamento em bloco virtualizado em nuvem*** são semelhantes à SAN, mas dispensam os engenheiros de lidar com clusters SAN e detalhes de rede. Veremos o Amazon Elastic Block Store (EBS) como um exemplo padrão; outro

Machine Translated by Google

nuvens públicas têm ofertas semelhantes. EBS é o armazenamento padrão para máquinas virtuais Amazon EC2; outros provedores de nuvem também tratam o armazenamento de objetos virtualizados como um componente-chave de suas ofertas de VM.

O EBS oferece vários níveis de serviço com diferentes características de desempenho. Geralmente, as métricas de desempenho do EBS são fornecidas em IOPS e taxa de transferência (velocidade de transferência). Os níveis de maior desempenho do armazenamento EBS são suportados por discos SSD, enquanto o armazenamento baseado em disco magnético oferece IOPS mais baixo, mas custa menos por gigabyte.

Os volumes do EBS armazenam dados separados do servidor host da instância, mas na mesma zona para oferecer suporte a alto desempenho e baixa latência (Figura 6-5). Isso permite que os volumes do EBS persistam quando uma instância do EC2 for encerrada, quando um servidor host falhar ou mesmo quando a instância for excluída. O armazenamento EBS é adequado para aplicativos como bancos de dados, onde a durabilidade dos dados é uma alta prioridade. Além disso, o EBS replica todos os dados para pelo menos duas máquinas host separadas, protegendo os dados se um disco falhar.

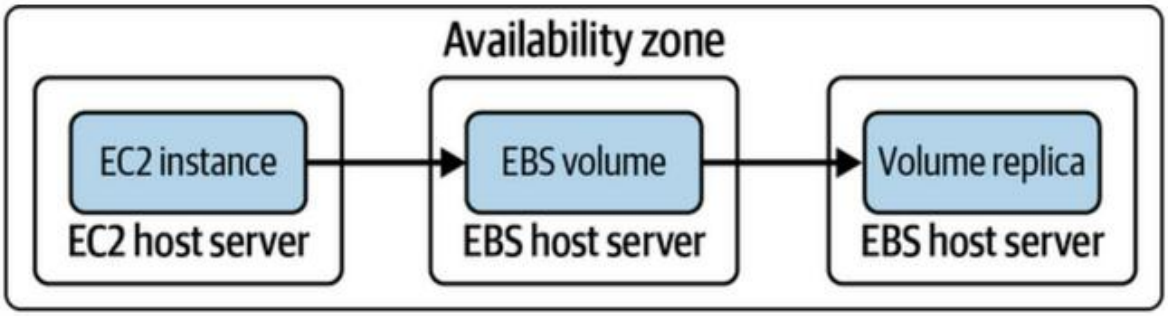


Figura 6-5. Os volumes do EBS replicam dados para vários hosts e discos para alta durabilidade e disponibilidade, mas não são resilientes à falha de uma zona de disponibilidade

A virtualização de armazenamento do EBS também oferece suporte a vários recursos avançados. Por exemplo, os volumes do EBS permitem instantâneos instantâneos de um momento específico enquanto a unidade é usada. Embora ainda demore algum tempo para que o instantâneo seja replicado para o S3, o EBS pode efetivamente congelar o estado dos blocos de dados quando o instantâneo é obtido, enquanto permite que a máquina cliente continue usando o disco. Além disso, os instantâneos após o backup completo inicial são diferenciais; apenas os blocos alterados são gravados no S3 para minimizar os custos de armazenamento e o tempo de backup.

Machine Translated by Google

Os volumes do EBS também são altamente escaláveis. No momento da redação deste artigo, algumas classes de volume do EBS podem ser dimensionadas para até 64 TiB, 256.000 IOPS e 4.000 MiB/s.

Volumes de instâncias locais

Os provedores de nuvem também oferecem volumes de armazenamento em bloco que são fisicamente conectados ao servidor host que executa uma máquina virtual. Esses volumes de armazenamento geralmente têm um custo muito baixo (incluído no preço da VM no caso do armazenamento de instâncias EC2 da Amazon) e fornecem baixa latência e alto IOPS.

Os volumes de armazenamento de instâncias (Figura 6-6) se comportam essencialmente como um disco fisicamente conectado a um servidor em um data center. Uma diferença importante é que, quando uma VM é encerrada ou excluída, o conteúdo do disco anexado localmente é perdido, independentemente de esse evento ter sido causado por ação intencional do usuário. Isso garante que uma nova máquina virtual não possa ler o conteúdo do disco pertencente a um cliente diferente.

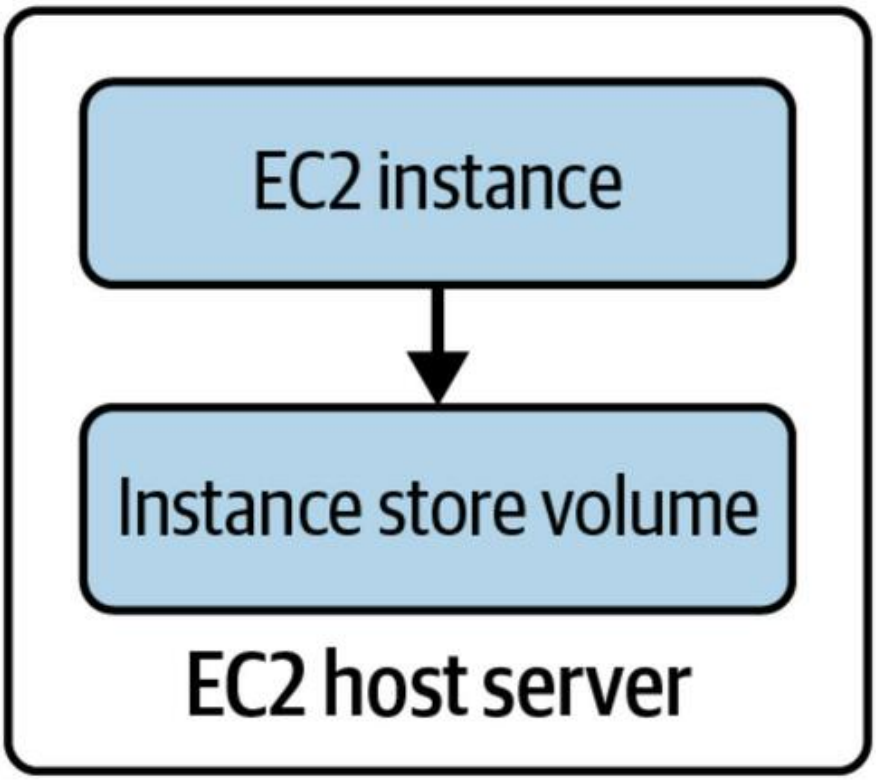


Figura 6-6. Os volumes de armazenamento de instâncias oferecem alto desempenho e baixo custo, mas não protegem os dados em caso de falha de disco ou desligamento da VM

Machine Translated by Google

Os discos conectados localmente não suportam nenhum dos recursos avançados de virtualização oferecidos por serviços de armazenamento virtualizado como o EBS. O disco conectado localmente não é replicado, portanto, uma falha de disco físico pode perder ou corromper dados, mesmo que a VM do host continue em execução. Além disso, os volumes anexados localmente não oferecem suporte a instantâneos ou outros recursos de backup.

Apesar dessas limitações, os discos anexados localmente são extremamente úteis. Em muitos casos, usamos discos como cache local e, portanto, não precisamos de todos os recursos avançados de virtualização de um serviço como o EBS. Por exemplo, suponha que estamos executando o AWS EMR em instâncias do EC2. Podemos estar executando um trabalho efêmero que consome dados do S3, armazena-os temporariamente no sistema de arquivos distribuído em execução nas instâncias, processa os dados e grava os resultados de volta no S3. O sistema de arquivos EMR é construído em replicação e redundância e está servindo como cache em vez de armazenamento permanente. O armazenamento de instâncias do EC2 é uma solução perfeitamente adequada nesse caso e pode melhorar o desempenho, pois os dados podem ser lidos e processados localmente sem fluir pela rede (consulte a Figura 6-7).

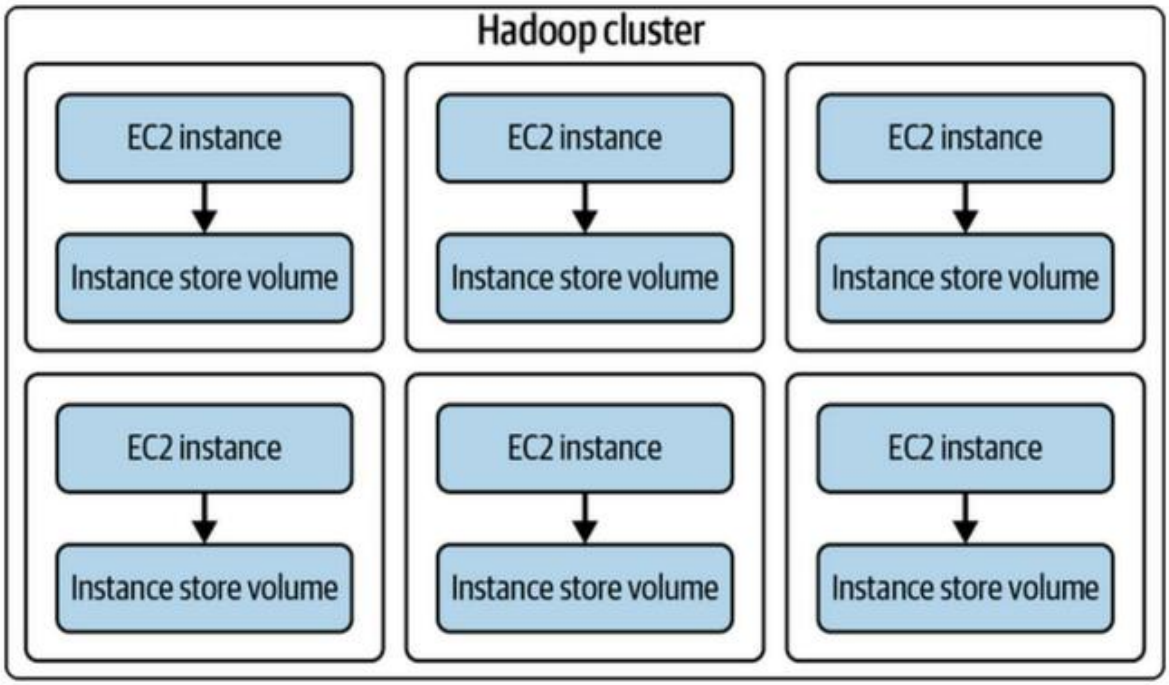


Figura 6-7. Os volumes de armazenamento de instâncias podem ser usados como um cache de processamento em um cluster Hadoop efêmero

Machine Translated by Google

Recomendamos que os engenheiros pensem no armazenamento anexado localmente nos piores cenários. Quais são as consequências de uma falha de disco local? De uma VM acidental ou desligamento de cluster? De uma interrupção de nuvem zonal ou regional? Se nenhum desses cenários tiver consequências catastróficas quando os dados em volumes anexados localmente forem perdidos, o armazenamento local pode ser uma opção econômica e de alto desempenho. Além disso, estratégias simples de mitigação (backups periódicos de checkpoints para S3) podem evitar a perda de dados.

Armazenamento de objetos

O armazenamento de objetos contém ***objetos*** de todas as formas e tamanhos (Figura 6-8). O termo ***armazenamento de objetos*** é um pouco confuso porque ***objeto*** tem vários significados na ciência da computação. Nesse contexto, estamos falando de uma construção especializada em arquivo. Pode ser qualquer tipo de arquivo – TXT, CSV, JSON, imagens, vídeos ou áudio.

Machine Translated by Google

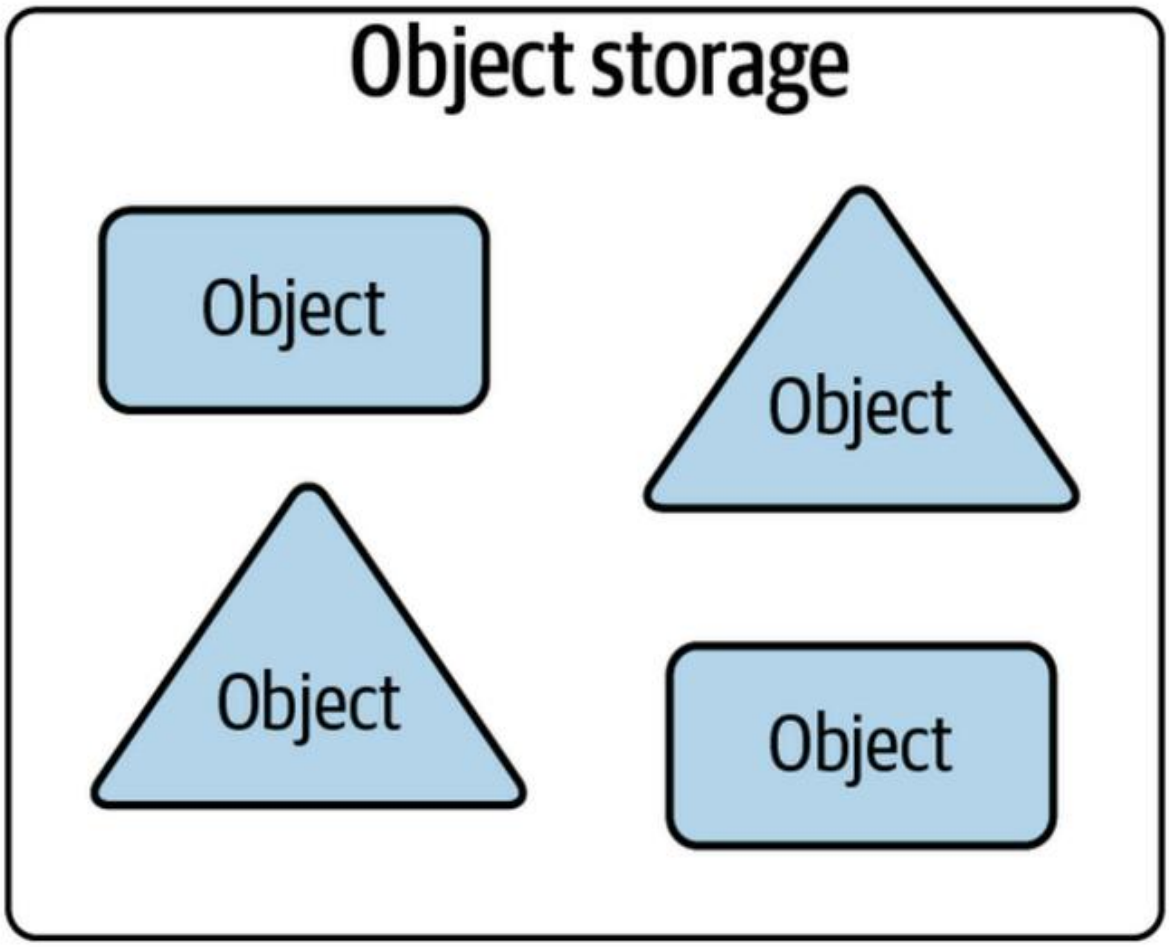


Figura 6-8. O armazenamento de objetos contém objetos imutáveis de todas as formas e tamanhos. Ao contrário dos arquivos em um disco local, os objetos não podem ser modificados no local.

Os armazenamentos de objetos cresceram em importância e popularidade com o surgimento do big data e da nuvem. Amazon S3, Azure Blob Storage e Google Cloud Storage (GCS) são armazenamentos de objetos amplamente usados. Além disso, muitos data warehouses em nuvem (e um número crescente de bancos de dados) utilizam o armazenamento de objetos como sua camada de armazenamento, e os data lakes em nuvem geralmente ficam em armazenamentos de objetos.

Embora muitos sistemas de armazenamento de objetos locais possam ser instalados em clusters de servidores, vamos nos concentrar principalmente em armazenamentos de objetos em nuvem totalmente gerenciados. Do ponto de vista operacional, uma das características mais atraentes do armazenamento de objetos em nuvem é que ele é simples de gerenciar e usar.

O armazenamento de objetos foi sem dúvida um dos primeiros serviços “sem servidor”; os engenheiros não precisam considerar as características dos clusters ou discos de servidor subjacentes.

Machine Translated by Google

Um armazenamento de objeto é um armazenamento de valor-chave para objetos de dados imutáveis. Perdemos muito da flexibilidade de gravação que esperamos com o armazenamento de arquivos em um disco local em um armazenamento de objetos. Objetos não suportam gravações aleatórias ou operações de acréscimo; em vez disso, eles são escritos uma vez como um fluxo de bytes. Após essa gravação inicial, os objetos se tornam imutáveis. Para alterar dados em um objeto ou anexar dados a ele, devemos reescrever o objeto completo. Os armazenamentos de objetos geralmente oferecem suporte a leituras aleatórias por meio de solicitações de intervalo, mas essas pesquisas podem ter um desempenho muito pior do que leituras aleatórias de dados armazenados em um SSD.

Para um desenvolvedor de software acostumado a aproveitar o armazenamento de arquivos de acesso aleatório local, as características dos objetos podem parecer restrições, mas menos é mais; os armazenamentos de objetos não precisam oferecer suporte a bloqueios ou sincronização de alterações, permitindo o armazenamento de dados em clusters de disco massivos. Os armazenamentos de objetos suportam gravações e leituras de fluxo paralelo de alto desempenho em muitos discos, e esse paralelismo é oculto aos engenheiros, que podem simplesmente lidar com o fluxo em vez de se comunicar com discos individuais. Em um ambiente de nuvem, a velocidade de gravação é dimensionada com o número de fluxos sendo gravados até os limites de cota definidos pelo fornecedor. A largura de banda de leitura pode ser dimensionada com o número de solicitações paralelas, o número de máquinas virtuais empregadas para ler dados e o número de núcleos de CPU. Essas características tornam o armazenamento de objetos ideal para atender tráfego da Web de alto volume ou fornecer dados para mecanismos de consulta distribuídos altamente paralelos.

Os armazenamentos de objetos em nuvem típicos salvam dados em várias zonas de disponibilidade, reduzindo drasticamente as chances de que o armazenamento fique totalmente offline ou seja perdido de maneira irrecuperável. Essa durabilidade e disponibilidade são incorporadas ao custo; os fornecedores de armazenamento em nuvem oferecem outras classes de armazenamento a preços com desconto em troca de durabilidade ou disponibilidade reduzida. Discutiremos isso em “Classes e camadas de armazenamento”.

O armazenamento de objetos em nuvem é um ingrediente essencial na separação de computação e armazenamento, permitindo que os engenheiros processem dados com clusters efêmeros e dimensionem esses clusters para cima e para baixo sob demanda. Esse é um fator-chave para disponibilizar big data para organizações menores que não podem ter hardware para tarefas de dados que serão executadas apenas ocasionalmente. Algumas grandes empresas de tecnologia continuarão a executar clusters Hadoop permanentes em seu hardware. Ainda assim, o

Machine Translated by Google

A tendência geral é que a maioria das organizações moverá o processamento de dados para a nuvem, usando um armazenamento de objetos como armazenamento essencial e camada de serviço enquanto processa dados em clusters efêmeros.

No armazenamento de objetos, o espaço de armazenamento disponível também é altamente escalável, uma característica ideal para sistemas de big data. O espaço de armazenamento é limitado pelo número de discos que o provedor de armazenamento possui, mas esses provedores lidam com exabytes de dados. Em um ambiente de nuvem, o espaço de armazenamento disponível é praticamente ilimitado; na prática, o principal limite de espaço de armazenamento para clientes de nuvem pública é o orçamento. Do ponto de vista prático, os engenheiros podem armazenar rapidamente grandes quantidades de dados para projetos sem planejar com meses de antecedência os servidores e discos necessários.

Armazenamentos de objetos para aplicativos de engenharia de dados Do

ponto de vista da engenharia de dados, os armazenamentos de objetos oferecem excelente desempenho para leituras e gravações em lote grandes. Isso corresponde bem ao caso de uso para sistemas OLAP massivos. Um pouco do folclore da engenharia de dados diz que os armazenamentos de objetos não são bons para atualizações, mas isso é apenas parcialmente verdade. Os armazenamentos de objetos são um ajuste inferior para cargas de trabalho transacionais com muitas pequenas atualizações a cada segundo; esses casos de uso são muito mais bem atendidos por bancos de dados transacionais ou sistemas de armazenamento em bloco. Os armazenamentos de objetos funcionam bem para uma taxa baixa de operações de atualização, onde cada operação atualiza um grande volume de dados.

Os armazenamentos de objetos agora são o padrão-ouro de armazenamento para data lakes. Nos primórdios dos data lakes, escrever uma vez, ler muitos (WORM) era o padrão operacional, mas isso tinha mais a ver com as complexidades do gerenciamento de versões e arquivos de dados do que com as limitações do HDFS e armazenamentos de objetos. Desde então, sistemas como Apache Hudi e Delta Lake surgiram para gerenciar essa complexidade, e regulamentações de privacidade, como GDPR e CCPA, tornaram imperativos os recursos de exclusão e atualização. O gerenciamento de atualizações para armazenamento de objetos é a ideia central por trás do conceito data lakehouse, que apresentamos no Capítulo 3.

O armazenamento de objetos é um repositório ideal para dados não estruturados em qualquer formato além desses aplicativos de dados estruturados. O armazenamento de objetos pode abrigar qualquer

Machine Translated by Google

dados binários sem restrições de tipo ou estrutura e frequentemente desempenha um papel nos pipelines de ML para texto bruto, imagens, vídeo e áudio.

Pesquisa de objetos

Como mencionamos, os armazenamentos de objetos são armazenamentos de valores-chave. O que isso significa para os engenheiros? É fundamental entender que, diferentemente dos armazenamentos de arquivos, os armazenamentos de objetos não utilizam uma árvore de diretórios para localizar objetos. O armazenamento de objetos usa um contêiner lógico de nível superior (um bucket no S3 e GCS) e faz referência a objetos por chave. Um exemplo simples no S3 pode ser assim:

```
S3://oreilly-data-engineering-book/data-example.json
```

Nesse caso, S3://oreilly-data-engineering-book/ é o nome do bucket e data-example.json é a chave que aponta para um objeto específico. Os nomes de bucket do S3 devem ser exclusivos em toda a AWS. As chaves são exclusivas em um bucket. Embora os armazenamentos de objetos na nuvem pareçam oferecer suporte à semântica da árvore de diretórios, não existe uma hierarquia de diretórios verdadeira. Podemos armazenar um objeto com o seguinte caminho completo:

```
S3://oreilly-data-engineering-book/project data/11/23/2021/
data.txt
```

Na superfície, isso se parece com subdiretórios que você pode encontrar em um sistema de pastas de arquivos comum: project-data, 11, 23 e 2021. Muitas interfaces de console em nuvem permitem que os usuários visualizem os objetos dentro de um "diretório" e linha de comando na nuvem as ferramentas geralmente suportam comandos no estilo Unix, como ls dentro de um diretório de armazenamento de objetos. No entanto, nos bastidores, o sistema de objetos não percorre uma árvore de diretórios para alcançar o objeto. Em vez disso, ele simplesmente vê uma chave (project-data/11/23/2021/data.txt) que corresponde à semântica do diretório. Isso pode parecer um detalhe técnico menor, mas os engenheiros precisam entender que certas operações no nível de “diretório” são caras em um armazenamento de objetos. Para executar aws ls S3://oreilly-data engineering-book/project-data/11/ o armazenamento de objetos deve filtrar chaves no prefixo de chave project-data/11/. Se o balde contém

Machine Translated by Google

milhões de objetos, esta operação pode levar algum tempo, mesmo que o “subdiretório” abrigue apenas alguns objetos.

Consistência e versionamento de objetos

Conforme mencionado, os armazenamentos de objetos não oferecem suporte a atualizações ou anexos in-loco como regra geral. Escrevemos um novo objeto sob a mesma chave para atualizar um objeto. Quando os engenheiros de dados utilizam atualizações nos processos de dados, eles devem estar cientes do modelo de consistência do armazenamento de objetos que estão usando. Os armazenamentos de objetos podem ser eventualmente consistentes ou fortemente consistentes. Por exemplo, até recentemente, o S3 ***acabou sendo consistente;*** depois que uma nova versão de um objeto foi gravada com a mesma chave, o armazenamento de objetos às vezes pode retornar a versão antiga do objeto. A parte eventual da ***consistência eventual*** significa que, após um tempo suficiente, o cluster de armazenamento atinge um estado em que apenas a versão mais recente do objeto será retornada. Isso contrasta com o modelo de ***consistência forte*** que esperamos de discos locais conectados a um servidor: a leitura após uma gravação retornará os dados gravados mais recentemente.

Pode ser desejável impor uma consistência forte em um armazenamento de objetos por vários motivos, e métodos padrão são usados para conseguir isso. Uma abordagem é adicionar um banco de dados fortemente consistente (por exemplo, PostgreSQL) à mistura. Escrever um objeto agora é um processo de duas etapas:

1. Escreva o objeto.
2. Grave os metadados retornados para a versão do objeto no banco de dados fortemente consistente.

Os metadados de versão (um hash de objeto ou um carimbo de data/hora de objeto) podem identificar exclusivamente uma versão de objeto em conjunto com a chave de objeto. Para ler um objeto, um leitor realiza as seguintes etapas:

1. Busque os metadados de objeto mais recentes do banco de dados fortemente consistente.
2. Consulte os metadados do objeto usando a chave do objeto. Leia os dados do objeto se eles corresponderem aos metadados buscados no banco de dados consistente.

Machine Translated by Google

3. Se os metadados do objeto não corresponderem, repita a etapa 2 até que a versão mais recente do objeto seja retornada.

Uma implementação prática tem exceções e casos extremos a serem considerados, como quando o objeto é reescrito durante esse processo de consulta. Essas etapas podem ser gerenciadas por trás de uma API para que um leitor de objetos veja um armazenamento de objetos fortemente consistente ao custo de maior latência para acesso a objetos.

O versionamento de objetos está intimamente relacionado à consistência do objeto. Quando reescrevemos um objeto sob uma chave existente em um armazenamento de objetos, estamos essencialmente escrevendo um objeto totalmente novo, definindo referências da chave existente para o objeto e excluindo as referências de objetos antigos. A atualização de todas as referências no cluster leva tempo, daí o potencial para leituras obsoletas. Eventualmente, o coletor de lixo do cluster de armazenamento desaloca o espaço dedicado aos dados desreferenciados, reciclando a capacidade do disco para uso por novos objetos.

Com o versionamento de objetos ativado, adicionamos metadados adicionais ao objeto que estipula uma versão. Embora a referência de chave padrão seja atualizada para apontar para o novo objeto, mantemos outros ponteiros para versões anteriores. Também mantemos uma lista de versões para que os clientes possam obter uma lista de todas as versões do objeto e, em seguida, obter uma versão específica. Como as versões antigas do objeto ainda são referenciadas, elas não são limpas pelo coletor de lixo.

Se referenciarmos um objeto com uma versão, o problema de consistência com alguns sistemas de armazenamento de objetos desaparece: os metadados da chave e da versão juntos formam uma referência exclusiva a um objeto de dados imutável específico. Sempre obteremos o mesmo objeto de volta quando usarmos esse par, desde que não o tenhamos excluído. O problema de consistência ainda existe quando um cliente solicita a versão “padrão” ou “mais recente” de um objeto.

A principal sobrecarga que os engenheiros precisam considerar com o versionamento de objetos é o custo de armazenamento. As versões históricas de objetos geralmente têm os mesmos custos de armazenamento associados que as versões atuais. Os custos da versão do objeto podem ser quase insignificantes ou catastroficamente caros, dependendo de vários fatores. O tamanho dos dados é um problema, assim como a frequência de atualização; mais versões de objeto podem levar a um tamanho de dados significativamente maior. Tenha em mente que

Machine Translated by Google

estamos falando de versionamento de objetos de força bruta. Os sistemas de armazenamento de objetos geralmente armazenam dados de objetos completos para cada versão, não instantâneos diferenciais.

Os engenheiros também têm a opção de implantar políticas de ciclo de vida de armazenamento. As políticas de ciclo de vida permitem a exclusão automática de versões antigas de objetos quando determinadas condições são atendidas (por exemplo, quando uma versão de objeto atinge uma determinada idade ou existem muitas versões mais recentes). Os fornecedores de nuvem também oferecem vários níveis de dados de arquivamento a preços com grandes descontos, e o processo de arquivamento pode ser gerenciado usando políticas de ciclo de vida.

Classes e camadas de armazenamento

Os fornecedores de nuvem agora oferecem classes de armazenamento que reduzem os preços de armazenamento de dados em troca de acesso reduzido ou durabilidade reduzida. Usamos o termo ***acesso reduzido*** aqui porque muitas dessas camadas de armazenamento ainda tornam os dados altamente disponíveis, mas com altos custos de recuperação em troca de custos de armazenamento reduzidos.

Vejamos alguns exemplos no S3, pois a Amazon é uma referência para padrões de serviço em nuvem. A classe de armazenamento S3 Standard-Infrequent Access oferece descontos nos custos mensais de armazenamento para aumentar os custos de recuperação de dados. (Consulte “A Brief Detour on Cloud Economics” para uma discussão teórica sobre a economia dos níveis de armazenamento em nuvem.) A Amazon também oferece o nível Amazon S3 One Zone-Infrequent Access, replicando apenas para uma única zona. A disponibilidade projetada cai de 99,9% para 99,5% para levar em conta a possibilidade de uma interrupção zonal. A Amazon ainda alega durabilidade de dados extremamente alta, com a ressalva de que os dados serão perdidos se uma zona de disponibilidade for destruída.

Mais abaixo, os níveis de acesso reduzido estão os níveis de arquivamento no S3 Glacier. O S3 Glacier promete uma redução drástica nos custos de armazenamento de longo prazo para custos de acesso muito mais altos. Os usuários têm várias opções de velocidade de recuperação, de minutos a horas, com custos de recuperação mais altos para acesso mais rápido. Por exemplo, no momento da redação deste artigo, o S3 Glacier Deep Archive reduz ainda mais os custos de armazenamento; A Amazon anuncia que os custos de armazenamento começam em US$ 1 por terabyte por mês. Em troca, a restauração de dados leva 12 horas. Além disso, essa classe de armazenamento é projetada para dados que serão armazenados de 7 a 10 anos e acessados apenas uma a duas vezes por ano.

Machine Translated by Google

Esteja ciente de como você planeja utilizar o armazenamento de arquivamento, pois é fácil de acessar e muitas vezes caro para acessar os dados, especialmente se você precisar deles com mais frequência do que o esperado. Consulte o Capítulo 4 para uma discussão mais ampla sobre a economia do armazenamento de arquivos.

Sistemas de arquivos suportados por armazenamento de objetos

As soluções de sincronização de armazenamento de objetos tornaram-se cada vez mais populares. Ferramentas como s3fs e Amazon S3 File Gateway permitem que os usuários montem um bucket do S3 como armazenamento local. Os usuários dessas ferramentas devem estar cientes das características das gravações no sistema de arquivos e como elas irão interagir com as características e preços do armazenamento de objetos. O File Gateway, por exemplo, lida com alterações em arquivos com bastante eficiência, combinando partes de objetos em um novo objeto usando os recursos avançados do S3. No entanto, a gravação transacional de alta velocidade sobrecarregará os recursos de atualização de um armazenamento de objetos. A montagem do armazenamento de objetos como um sistema de arquivos local funciona bem para arquivos que são atualizados com pouca frequência.

Sistemas de armazenamento baseados em cache e memória

Conforme discutido em "Raw Ingredients of Data Storage", a RAM oferece excelente latência e velocidades de transferência. No entanto, a RAM tradicional é extremamente vulnerável à perda de dados porque uma queda de energia que dura até um segundo pode apagar os dados. Os sistemas de armazenamento baseados em RAM geralmente são focados em aplicativos de cache, apresentando dados para acesso rápido e alta largura de banda. Os dados geralmente devem ser gravados em um meio mais durável para fins de retenção.

Esses sistemas de cache ultrarrápidos são úteis quando os engenheiros de dados precisam fornecer dados com latência de recuperação ultrarrápida.

Exemplo: cache de objetos Memcached e leve

O ***Memcached*** é um armazenamento de valor-chave projetado para armazenar em cache resultados de consultas de banco de dados, respostas de chamadas de API e muito mais. O Memcached usa estruturas de dados simples, suportando tipos string ou inteiros. O Memcached pode entregar

Machine Translated by Google

resultados com latência muito baixa, além de aliviar a carga dos sistemas de back-end.

Exemplo: Redis, cache de memória com persistência opcional

Assim como o Memcached, o ***Redis*** é um armazenamento de chave-valor, mas suporta tipos de dados um pouco mais complexos (como listas ou conjuntos). O Redis também cria vários mecanismos de persistência, incluindo snapshots e journaling. Com uma configuração típica, o Redis grava dados aproximadamente a cada dois segundos. O Redis é, portanto, adequado para aplicativos de desempenho extremamente alto, mas pode tolerar uma pequena perda de dados.

O sistema de arquivos distribuídos do Hadoop

O Hadoop Distributed File System é baseado no Google File System (GFS) e foi inicialmente projetado para processar dados com o modelo de programação MapReduce . O Hadoop é semelhante ao armazenamento de objetos, mas com uma diferença fundamental: o Hadoop combina computação e armazenamento nos mesmos nós, onde os armazenamentos de objetos normalmente têm suporte limitado para processamento interno.

O Hadoop divide arquivos grandes em ***blocos,*** pedaços de dados com menos de algumas centenas de megabytes de tamanho. O sistema de arquivos é gerenciado pelo ***NameNode,*** que mantém diretórios, metadados de arquivos e um catálogo detalhado que descreve a localização dos blocos de arquivos no cluster. Em uma configuração típica, cada bloco de dados é replicado para três nós. Isso aumenta a durabilidade e a disponibilidade dos dados. Se um disco ou nó falhar, o fator de replicação para alguns blocos de arquivo ficará abaixo de 3. O NameNode instruirá outros nós a replicar esses blocos de arquivo para que eles atinjam novamente o fator de replicação correto. Assim, a probabilidade de perda de dados é muito baixa, exceto uma ***falha correlacionada*** (por exemplo, um asteroide atingindo o data center).

O Hadoop não é simplesmente um sistema de armazenamento. O Hadoop combina recursos de computação com nós de armazenamento para permitir o processamento de dados no local. Isso foi originalmente alcançado usando o modelo de programação MapReduce, que discutimos no Capítulo 8.

Hadoop está morto. Viva o Hadoop!

Machine Translated by Google

Muitas vezes vemos alegações de que o Hadoop está morto. Isso é apenas parcialmente verdade. O Hadoop não é mais uma tecnologia de ponta. Muitas ferramentas do ecossistema Hadoop, como o Apache Pig, agora estão em suporte de vida e são usadas principalmente para executar trabalhos legados. O modelo de programação MapReduce puro caiu no esquecimento. O HDFS continua sendo amplamente utilizado em vários aplicativos e organizações.

O Hadoop ainda aparece em muitas instalações legadas. Muitas organizações que adotaram o Hadoop durante o pico da febre do big data não têm planos imediatos de migrar para tecnologias mais recentes. Essa é uma boa opção para empresas que executam clusters Hadoop massivos (de mil nós) e têm recursos para manter sistemas locais com eficiência. As empresas menores podem querer reconsiderar a sobrecarga de custos e as limitações de escala da execução de um pequeno cluster Hadoop em relação à migração para soluções em nuvem.

Além disso, o HDFS é um ingrediente-chave de muitos mecanismos de big data atuais, como o Amazon EMR. Na verdade, o Apache Spark ainda é comumente executado em clusters HDFS. Discutimos isso com mais detalhes em “Separação de computação do armazenamento”.

Armazenamento de streaming

Os dados de streaming têm requisitos de armazenamento diferentes dos dados que não são de streaming. No caso de filas de mensagens, os dados armazenados são temporais e devem desaparecer após um determinado período. No entanto, estruturas de streaming distribuídas e escaláveis, como o Apache Kafka, agora permitem a retenção de dados de streaming de duração extremamente longa. O Kafka oferece suporte à retenção de dados indefinida, enviando mensagens antigas e acessadas com pouca frequência para o armazenamento de objetos. Os concorrentes do Kafka (incluindo Amazon Kinesis, Apache Pulsar e Google Cloud Pub/Sub) também oferecem suporte a longa retenção de dados

Intimamente relacionada à retenção de dados nesses sistemas está a noção de repetição. ***Replay*** permite que um sistema de streaming retorne uma série de dados históricos armazenados. Replay é o mecanismo padrão de recuperação de dados para sistemas de armazenamento de streaming. A repetição pode ser usada para executar consultas em lote em um intervalo de tempo ou para

Machine Translated by Google

reprocessar dados em um pipeline de streaming. O Capítulo 7 cobre a reprodução com mais profundidade.

Outros mecanismos de armazenamento surgiram para aplicativos de análise em tempo real. De certa forma, os bancos de dados transacionais surgiram como os primeiros mecanismos de consulta em tempo real; os dados se tornam visíveis para as consultas assim que são gravados. No entanto, esses bancos de dados têm limitações conhecidas de dimensionamento e bloqueio, especialmente para consultas analíticas executadas em grandes volumes de dados. Embora as versões escaláveis de bancos de dados transacionais orientados a linhas tenham superado algumas dessas limitações, elas ainda não são realmente otimizadas para análises em escala.

Índices, particionamento e clustering

Os índices fornecem um mapa da tabela para campos específicos e permitem uma pesquisa extremamente rápida de registros individuais. Sem índices, um banco de dados precisaria varrer uma tabela inteira para encontrar os registros que satisfaçam a condição WHERE.

Na maioria dos RDBMSs, os índices são usados para chaves de tabela primárias (permitindo identificação exclusiva de linhas) e chaves estrangeiras (permitindo junções com outras tabelas).
Os índices também podem ser aplicados a outras colunas para atender às necessidades de aplicativos específicos. Usando índices, um RDBMS pode pesquisar e atualizar milhares de linhas por segundo.

Não abordamos os registros de banco de dados transacionais em profundidade neste livro; vários recursos técnicos estão disponíveis sobre este tópico. Em vez disso, estamos interessados na evolução dos índices em sistemas de armazenamento orientados a análise e alguns novos desenvolvimentos em índices para casos de uso de análise.

A evolução das linhas para as colunas

Um data warehouse inicial era normalmente construído no mesmo tipo de RDBMS usado para aplicativos transacionais. A crescente popularidade dos sistemas MPP significou uma mudança para o processamento paralelo para melhorias significativas no desempenho de varredura em grandes quantidades de dados para fins de análise.
No entanto, esses MPPs orientados a linhas ainda usavam índices para dar suporte a junções e verificação de condição.

Machine Translated by Google

Em “Raw Ingredients of Data Storage”, discutimos a serialização colunar.
A serialização colunar permite que um banco de dados verifique apenas as colunas necessárias para uma consulta específica, às vezes reduzindo drasticamente a quantidade de dados lidos do disco. Além disso, organizar dados por coluna agrupa valores semelhantes próximos uns dos outros, gerando altas taxas de compactação com sobrecarga de compactação mínima. Isso permite que os dados sejam verificados mais rapidamente do disco e pela rede.

Bancos de dados colunares têm um desempenho ruim para casos de uso transacionais, ou seja, quando tentamos pesquisar um grande número de linhas individuais de forma assíncrona.
No entanto, eles têm um desempenho extremamente bom quando grandes quantidades de dados precisam ser verificadas, por exemplo, para transformações complexas de dados, agregações, cálculos estatísticos ou avaliação de condições complexas em grandes conjuntos de dados.

No passado, os bancos de dados colunares tinham um desempenho ruim em junções, então o conselho para os engenheiros de dados era desnormalizar os dados, usando esquemas amplos, matrizes e dados aninhados sempre que possível. O desempenho de junção para bancos de dados colunares melhorou drasticamente nos últimos anos, portanto, embora ainda possa haver vantagens de desempenho na desnormalização, isso não é mais uma necessidade.
Você aprenderá mais sobre normalização e desnormalização no Capítulo 8.

De índices a partições e clustering

Embora os bancos de dados colunares permitam velocidades de varredura rápidas, ainda é útil reduzir ao máximo a quantidade de dados verificados. Além de verificar apenas os dados em colunas relevantes para uma consulta, podemos particionar uma tabela em várias subtabelas dividindo-a em um campo. É bastante comum em casos de uso de análise e ciência de dados fazer a varredura em um intervalo de tempo, portanto, o particionamento baseado em data e hora é extremamente comum. Bancos de dados colunares geralmente suportam uma variedade de outros esquemas de partição também.

Os ***clusters*** permitem uma organização mais refinada de dados dentro de partições. Um esquema de cluster aplicado em um banco de dados colunar classifica os dados por um ou alguns campos, colocando valores semelhantes. Isso melhora o desempenho para filtrar, classificar e unir esses valores.

Machine Translated by Google

Exemplo: microparticionamento de floco de neve

Mencionamos o microparticionamento Snowflake porque é um bom exemplo de desenvolvimentos recentes e evolução nas abordagens de armazenamento colunar.
As ***micropartições*** são conjuntos de linhas entre 50 e 500 megabytes em tamanho não compactado. Snowflake usa uma abordagem algorítmica que tenta agrupar linhas semelhantes. Isso contrasta com a abordagem ingênua tradicional de particionamento em um único campo designado, como uma data.
O Snowflake procura especificamente valores que são repetidos em um campo em várias linhas. Isso permite a ***poda*** agressiva de consultas com base em predicados.
Por exemplo, uma cláusula WHERE pode estipular o seguinte:

```
ONDE created_date='2017-01-02'
```

Nessa consulta, o Snowflake exclui todas as micropartições que não incluem essa data, eliminando efetivamente esses dados. O Snowflake também permite micropartições sobrepostas, potencialmente particionando em vários campos mostrando repetições significativas.

A poda eficiente é facilitada pelo banco de dados de metadados do Snowflake, que armazena uma descrição de cada micropartição, incluindo o número de linhas e intervalos de valores para campos. Em cada estágio de consulta, o Snowflake analisa as micropartições para determinar quais precisam ser verificadas. Snowflake usa o termo ***armazenamento colunar híbrido,*** referindo-se parcialmente[2] ao fato de que suas tabelas são divididas em pequenos grupos de linhas, embora o armazenamento seja fundamentalmente colunar. O banco de dados de metadados desempenha um papel semelhante a um índice em um banco de dados relacional tradicional.

Abstrações de armazenamento de engenharia de dados

As abstrações de armazenamento de engenharia de dados são padrões de organização e consulta de dados que estão no centro do ciclo de vida de engenharia de dados e são construídos sobre os sistemas de armazenamento de dados discutidos anteriormente (consulte a Figura 6-3). Introduzimos muitas dessas abstrações no Capítulo 3 e as revisitaremos aqui.

Machine Translated by Google

Os principais tipos de abstrações com os quais nos preocuparemos são aqueles que dão suporte à ciência de dados, análise e casos de uso de relatórios. Isso inclui data warehouse, data lake, data lakehouse, plataformas de dados e catálogos de dados. Não abordaremos os sistemas de origem, pois eles são discutidos no Capítulo 5.

A abstração de armazenamento que você precisa como engenheiro de dados se resume a algumas considerações importantes:

Finalidade e caso de uso

Você deve primeiro identificar a finalidade de armazenar os dados. O que é usado por?

Atualizar padrões

A abstração é otimizada para atualizações em massa, inserções de streaming ou upserts?

Custo

Quais são os custos financeiros diretos e indiretos? A hora de valorizar? o custos de oportunidade?

Armazenamento e computação separados

A tendência é separar armazenamento e computação, mas a maioria dos sistemas hibridizar separação e colocation. Cobrimos isso em "Separação de Compute from Storage" , pois afeta a finalidade, a velocidade e o custo.

Você deve saber que a popularidade de separar o armazenamento da computação significa que as linhas entre os bancos de dados OLAP e os data lakes estão cada vez mais embaçadas. Os principais data warehouses e data lakes na nuvem estão em rota de colisão. No futuro, as diferenças entre esses dois podem estar apenas no nome, pois podem ser funcional e tecnicamente muito semelhantes sob o capô.

Machine Translated by Google

O armazém de dados

Os data warehouses são uma arquitetura de dados OLAP padrão. Conforme discutido no Capítulo 3, o termo ***data warehouse*** refere-se a plataformas de tecnologia (por exemplo, Google BigQuery e Teradata), uma arquitetura para centralização de dados e um padrão organizacional dentro de uma empresa. Em termos de tendências de armazenamento, evoluímos da construção de data warehouses sobre bancos de dados transacionais convencionais, sistemas MPP baseados em linhas (por exemplo, Teradata e IBM Netezza) e sistemas MPP colunares (por exemplo, Vertica e Teradata Columnar) para data warehouses em nuvem e plataformas de dados. (Veja nossa discussão sobre armazenamento de dados no Capítulo 3 para mais detalhes sobre sistemas MPP.)

Na prática, os data warehouses em nuvem são frequentemente usados para organizar dados em um data lake, uma área de armazenamento para grandes quantidades de dados brutos não processados, conforme originalmente concebido por James Dixon. Os data warehouses na nuvem podem lidar com grandes quantidades de texto bruto e documentos JSON complexos. A limitação é que os data warehouses em nuvem não podem lidar com dados realmente não estruturados, como imagens, vídeo ou áudio, ao contrário de um verdadeiro data lake. Os data warehouses em nuvem podem ser acoplados ao armazenamento de objetos para fornecer uma solução completa de data lake.

O lago de dados

O ***data lake*** foi originalmente concebido como um armazenamento massivo onde os dados eram retidos em forma bruta e não processada. Inicialmente, os data lakes foram construídos principalmente em sistemas Hadoop, onde o armazenamento barato permitia a retenção de grandes quantidades de dados sem a sobrecarga de custos de um sistema MPP proprietário.

Os últimos cinco anos viram dois grandes desenvolvimentos na evolução do armazenamento de data lake. Primeiro, ocorreu uma grande migração para a ***separação de computação e armazenamento .*** Na prática, isso significa uma mudança do Hadoop para o armazenamento de objetos em nuvem para retenção de dados a longo prazo.

Em segundo lugar, os engenheiros de dados descobriram que grande parte da funcionalidade oferecida pelos sistemas MPP (gerenciamento de esquema; recursos de atualização, mesclagem e exclusão) e inicialmente descartada na corrida para os data lakes era, de fato, extremamente útil. Isso levou à noção de data lakehouse.

Machine Translated by Google

A Casa do Data Lake

O ***data lakehouse*** é uma arquitetura que combina aspectos do data warehouse e do data lake. Como é geralmente concebido, o lakehouse armazena dados no armazenamento de objetos como um lago. No entanto, o lakehouse adiciona a esse arranjo recursos projetados para simplificar o gerenciamento de dados e criar uma experiência de engenharia semelhante a um data warehouse. Isso significa suporte robusto a tabelas e esquemas e recursos para gerenciar atualizações e exclusões incrementais. Lakehouses normalmente também suportam histórico de tabela e rollback; isso é feito retendo versões antigas de arquivos e metadados.

Um sistema lakehouse é uma camada de metadados e gerenciamento de arquivos implantada com ferramentas de gerenciamento e transformação de dados. A Databricks promoveu fortemente o conceito de lakehouse com o Delta Lake, um sistema de gerenciamento de armazenamento de código aberto.

Seríamos negligentes em não apontar que a arquitetura do data lakehouse é semelhante à arquitetura usada por várias plataformas de dados comerciais, incluindo BigQuery e Snowflake. Esses sistemas armazenam dados no armazenamento de objetos e fornecem gerenciamento automatizado de metadados, histórico de tabelas e recursos de atualização/exclusão. As complexidades do gerenciamento de arquivos e armazenamento subjacentes são totalmente ocultas do usuário.

A principal vantagem do data lakehouse sobre as ferramentas proprietárias é a interoperabilidade. É muito mais fácil trocar dados entre ferramentas quando armazenados em um formato de arquivo aberto. A reserialização de dados de um formato de banco de dados proprietário gera sobrecarga de processamento, tempo e custo. Em uma arquitetura de data lakehouse, várias ferramentas podem se conectar à camada de metadados e ler dados diretamente do armazenamento de objetos.

É importante enfatizar que muitos dos dados em um data lakehouse podem não ter uma estrutura de tabela imposta. Podemos impor recursos de data warehouse onde precisamos deles em um lakehouse, deixando outros dados em um formato bruto ou até mesmo não estruturado.

A tecnologia de data lakehouse está evoluindo rapidamente. Surgiram vários novos concorrentes para o Delta Lake, incluindo Apache Hudi e

Machine Translated by Google

Apache Iceberg. Consulte o Apêndice A para obter mais detalhes.

Plataformas de dados

Cada vez mais, os fornecedores estão estilizando seus produtos como ***plataformas de dados.*** Esses fornecedores criaram seus ecossistemas de ferramentas interoperáveis com forte integração na camada principal de armazenamento de dados. Ao avaliar as plataformas, os engenheiros devem garantir que as ferramentas oferecidas atendam às suas necessidades. Ferramentas não fornecidas diretamente na plataforma ainda podem interoperar, com sobrecarga de dados extra para troca de dados. As plataformas também enfatizam a integração próxima com o armazenamento de objetos para casos de uso não estruturados, conforme mencionado em nossa discussão sobre data warehouses em nuvem.

Neste ponto, a noção de plataforma de dados francamente ainda precisa ser totalmente desenvolvida. No entanto, a corrida está em andamento para criar um jardim murado de ferramentas de dados, simplificando o trabalho de engenharia de dados e gerando uma dependência significativa do fornecedor.

Arquitetura de armazenamento de fluxo para lote

A arquitetura de armazenamento de fluxo para lote tem muitas semelhanças com a arquitetura Lambda, embora alguns possam discordar dos detalhes técnicos. Essencialmente, os dados que passam por um tópico no sistema de armazenamento de streaming são gravados para vários consumidores.

Alguns desses consumidores podem ser sistemas de processamento em tempo real que geram estatísticas sobre o fluxo. Além disso, um consumidor de armazenamento em lote grava dados para retenção de longo prazo e consultas em lote. O consumidor do lote pode ser o AWS Kinesis Firehose, que pode gerar objetos do S3 com base em gatilhos configuráveis (por exemplo, tempo e tamanho do lote). Sistemas como o BigQuery ingerem dados de streaming em um buffer de streaming. Esse buffer de streaming é reserializado automaticamente no armazenamento de objeto colunar. O mecanismo de consulta oferece suporte à consulta contínua do buffer de streaming e dos dados do objeto para fornecer aos usuários uma visão atual da tabela quase em tempo real.

Machine Translated by Google

Grandes ideias e tendências em armazenamento

Nesta seção, discutiremos algumas grandes ideias em armazenamento — as principais considerações que você precisa ter em mente ao construir sua arquitetura de armazenamento. Muitas dessas considerações fazem parte de tendências maiores. Por exemplo, os catálogos de dados se encaixam na tendência de engenharia de dados e gerenciamento de dados "empresariais". A separação da computação do armazenamento agora é um fato realizado em sistemas de dados em nuvem. E o compartilhamento de dados é uma consideração cada vez mais importante à medida que as empresas adotam a tecnologia de dados.

Catálogo de dados

Um ***catálogo de dados*** é um repositório de metadados centralizado para todos os dados de uma organização. Estritamente falando, um catálogo de dados não é uma abstração de armazenamento de dados de nível superior, mas se integra a vários sistemas e abstrações. Os catálogos de dados geralmente funcionam em fontes de dados operacionais e analíticas, integram a linhagem de dados e a apresentação de relacionamentos de dados e permitem a edição de descrições de dados pelo usuário.

Os catálogos de dados são frequentemente usados para fornecer um local central onde as pessoas podem visualizar seus dados, consultas e armazenamento de dados. Como engenheiro de dados, você provavelmente será responsável por configurar e manter as várias integrações de dados de pipeline de dados e sistemas de armazenamento que se integrarão ao catálogo de dados e à integridade do próprio catálogo de dados.

Integração de aplicativos de catálogo

Idealmente, os aplicativos de dados são projetados para integração com APIs de catálogo para lidar diretamente com seus metadados e atualizações. Como os catálogos são mais amplamente utilizados em uma organização, fica mais fácil aproximar-se desse ideal.

Verificação automatizada

Na prática, os sistemas de catalogação geralmente precisam contar com uma camada de varredura automatizada que coleta metadados de vários sistemas, como data lakes, data warehouses e bancos de dados operacionais. Os catálogos de dados podem coletar

Machine Translated by Google

metadados existentes e também podem usar ferramentas de varredura para inferir metadados, como relacionamentos de chaves ou a presença de dados confidenciais.

Portal de dados e camada social

Os catálogos de dados geralmente também fornecem uma camada de acesso humano por meio de uma interface da Web, na qual os usuários podem pesquisar dados e visualizar relacionamentos de dados. Os catálogos de dados podem ser aprimorados com uma camada social que oferece a funcionalidade Wiki.
Isso permite que os usuários forneçam informações sobre seus conjuntos de dados, solicitem informações de outros usuários e publiquem atualizações assim que estiverem disponíveis.

Casos de uso do catálogo de dados

Os catálogos de dados têm casos de uso organizacionais e técnicos. Os catálogos de dados tornam os metadados facilmente disponíveis para os sistemas. Por exemplo, um catálogo de dados é um ingrediente chave do data lakehouse, permitindo a descoberta de tabelas para consultas.

Organizacionalmente, os catálogos de dados permitem que usuários de negócios, analistas, cientistas de dados e engenheiros pesquisem dados para responder a perguntas. Os catálogos de dados simplificam as comunicações e a colaboração entre organizações.

Compartilhamento de dados

O compartilhamento de dados permite que organizações e indivíduos compartilhem dados específicos e permissões cuidadosamente definidas com entidades específicas. O compartilhamento de dados permite que os cientistas de dados compartilhem dados de um sandbox com seus colaboradores dentro de uma organização. Entre as organizações, o compartilhamento de dados facilita a colaboração entre empresas parceiras. Por exemplo, uma empresa de tecnologia de anúncios pode compartilhar dados de publicidade com seus clientes.

Um ambiente multilocatário na nuvem torna a colaboração interorganizacional muito mais fácil. No entanto, também apresenta novos desafios de segurança.
As organizações devem controlar cuidadosamente as políticas que controlam quem pode compartilhar dados com quem para evitar exposição acidental ou exfiltração deliberada.

O compartilhamento de dados é um recurso central de muitas plataformas de dados em nuvem. Consulte o Capítulo 5 para uma discussão mais ampla sobre compartilhamento de dados.

Machine Translated by Google

Esquema

Qual é a forma esperada dos dados? Qual é o formato do arquivo? É estruturado, semiestruturado ou não estruturado? Que tipos de dados são esperados?
Como os dados se encaixam em uma hierarquia maior? Ele está conectado a outros dados por meio de chaves compartilhadas ou outros relacionamentos?

Observe que o esquema não precisa ser ***relacional.*** Em vez disso, os dados se tornam mais úteis quando temos o máximo de informações sobre sua estrutura e organização. Para imagens armazenadas em um data lake, essas informações de esquema podem explicar o formato da imagem, a resolução e a maneira como as imagens se encaixam em uma hierarquia maior.

O esquema pode funcionar como uma espécie de pedra de Roseta, instruções que nos dizem como ler os dados. Existem dois padrões de esquema principais: esquema na gravação e esquema na leitura. O ***esquema na gravação*** é essencialmente o padrão tradicional de data warehouse: uma tabela tem um esquema integrado; quaisquer gravações na tabela devem estar em conformidade. Para dar suporte ao esquema na gravação, um data lake deve integrar um metastore de esquema.

Com o ***esquema em leitura,*** o esquema é criado dinamicamente quando os dados são gravados e um leitor deve determinar o esquema ao ler os dados.
Idealmente, o esquema na leitura é implementado usando formatos de arquivo que implementam informações de esquema internas, como Parquet ou JSON. Os arquivos CSV são notórios pela inconsistência de esquema e não são recomendados nessa configuração.

A principal vantagem do esquema na gravação é que ele impõe padrões de dados, tornando os dados mais fáceis de consumir e utilizar no futuro. O esquema na leitura enfatiza a flexibilidade, permitindo que praticamente todos os dados sejam gravados.
Isso vem ao custo de maior dificuldade em consumir dados no futuro.

Separação de computação do armazenamento

Uma ideia-chave que revisitamos ao longo deste livro é a separação entre computação e armazenamento. Isso surgiu como um padrão de acesso a dados e consulta padrão na era da nuvem de hoje. Os data lakes, conforme discutimos, armazenam dados em armazenamentos de objetos e aumentam a capacidade de computação temporária para lê-los e processá-los. Mais completamente

Machine Translated by Google

produtos OLAP gerenciados agora contam com armazenamento de objetos nos bastidores. Para entender as motivações para separar computação e armazenamento, devemos primeiro examinar a colocação de computação e armazenamento.

Colocação de computação e armazenamento

A colocação de computação e armazenamento tem sido um método padrão para melhorar o desempenho do banco de dados. Para bancos de dados transacionais, a colocação de dados permite leituras de disco rápidas e de baixa latência e alta largura de banda. Mesmo quando virtualizamos o armazenamento (por exemplo, usando o Amazon EBS), os dados ficam relativamente próximos da máquina host.

A mesma ideia básica se aplica a sistemas de consulta analítica executados em um cluster de máquinas. Por exemplo, com HDFS e MapReduce, a abordagem padrão é localizar blocos de dados que precisam ser verificados no cluster e, em seguida, enviar tarefas de ***mapas*** individuais para esses blocos. A varredura de dados e o processamento para a etapa do mapa são estritamente locais. A etapa de ***redução*** envolve o embaralhamento de dados em todo o cluster, mas manter as etapas de mapa locais preserva efetivamente mais largura de banda para embaralhamento, proporcionando melhor desempenho geral; etapas do mapa que filtram pesadamente também reduzem drasticamente a quantidade de dados a serem embaralhados.

Separação de computação e armazenamento Se a

colocação de computação e armazenamento oferece alto desempenho, por que a mudança para a separação de computação e armazenamento? Existem várias motivações.

Efemeridade e escalabilidade

Na nuvem, vimos uma mudança dramática em direção à efemeridade. Em geral, é mais barato comprar e hospedar um servidor do que alugá-lo de um provedor de nuvem, ***desde que você o execute 24 horas por dia sem parar por anos a fio.*** Na prática, as cargas de trabalho variam drasticamente e eficiências significativas são obtidas com um modelo de pagamento conforme o uso, se os servidores puderem ser dimensionados para cima e para baixo. Isso vale para servidores da Web no varejo online e também para trabalhos em lote de big data que podem ser executados apenas periodicamente.

Machine Translated by Google

Os recursos de computação efêmeros permitem que os engenheiros criem clusters massivos para concluir os trabalhos no prazo e, em seguida, excluam os clusters quando esses trabalhos forem concluídos. Os benefícios de desempenho de operar temporariamente em escala ultra-alta podem superar as limitações de largura de banda do armazenamento de objetos.

Durabilidade e disponibilidade de dados

Os armazenamentos de objetos em nuvem reduzem significativamente o risco de perda de dados e geralmente fornecem um tempo de atividade (disponibilidade) extremamente alto. Por exemplo, o S3 armazena dados em várias zonas; se um desastre natural destruir uma zona, os dados ainda estarão disponíveis nas zonas restantes. Ter várias zonas disponíveis também reduz as chances de uma interrupção de dados. Se os recursos em uma zona ficarem inativos, os engenheiros poderão ativar os mesmos recursos em uma zona diferente.

O potencial de uma configuração incorreta que destrói dados no armazenamento de objetos ainda é um pouco assustador, mas mitigações simples de implantar estão disponíveis.
Copiar dados para várias regiões de nuvem reduz esse risco, pois as alterações de configuração geralmente são implantadas em apenas uma região por vez. A replicação de dados para vários provedores de armazenamento pode reduzir ainda mais o risco.

Separação híbrida e colocation

As realidades práticas de separar a computação do armazenamento são mais complicadas do que sugerimos. Na realidade, constantemente hibridizamos a colocação e a separação para obter os benefícios de ambas as abordagens. Essa hibridização geralmente é feita de duas maneiras: armazenamento em cache multicamadas e armazenamento de objetos híbridos.

Com o ***cache multicamada,*** utilizamos o armazenamento de objetos para retenção e acesso de dados a longo prazo, mas aumentamos o armazenamento local para ser usado durante consultas e vários estágios de pipelines de dados. Tanto o Google quanto a Amazon oferecem versões de armazenamento de objetos híbridos (armazenamento de objetos totalmente integrado à computação).

Vejamos exemplos de como alguns mecanismos de processamento populares hibridizam a separação e a colocação de armazenamento e computação.

Exemplo: AWS EMR com S3 e HDFS

Machine Translated by Google

Serviços de big data, como o Amazon EMR, criam clusters HDFS temporários para processar dados. Os engenheiros têm a opção de referenciar o S3 e o HDFS como um sistema de arquivos. Um padrão comum é colocar o HDFS em unidades SSD, extrair do S3 e salvar dados de etapas de processamento intermediárias no HDFS local.
Fazer isso pode obter ganhos de desempenho significativos em relação ao processamento diretamente do S3. Os resultados completos são gravados de volta no S3 assim que o cluster conclui suas etapas e o cluster e o HDFS são excluídos. Outros consumidores lêem os dados de saída diretamente do S3.

Exemplo: Apache Spark

Na prática, o Spark geralmente executa trabalhos no HDFS ou em algum outro sistema de arquivos distribuído efêmero para oferecer suporte ao armazenamento de dados de alto desempenho entre as etapas de processamento. Além disso, o Spark depende muito do armazenamento de dados na memória para melhorar o processamento. O problema de possuir a infraestrutura para executar o Spark é que a RAM dinâmica (DRAM) é extremamente cara; separando computação e armazenamento na nuvem, podemos alugar grandes quantidades de memória e liberar essa memória quando o trabalho for concluído.

Exemplo: Apache Druid

O Apache Druid depende muito de SSDs para obter alto desempenho. Como os SSDs são significativamente mais caros que os discos magnéticos, o Druid mantém apenas uma cópia dos dados em seu cluster, reduzindo os custos de armazenamento “ao vivo” por um fator de três.

É claro que manter a durabilidade dos dados ainda é fundamental, então o Druid usa um armazenamento de objetos como sua camada de durabilidade. Quando os dados são ingeridos, eles são processados, serializados em colunas compactadas e gravados em SSDs de cluster e armazenamento de objetos. No caso de falha do nó ou corrupção de dados do cluster, os dados podem ser recuperados automaticamente para novos nós. Além disso, o cluster pode ser encerrado e recuperado totalmente do armazenamento SSD.

Exemplo: armazenamento de objetos híbridos

O sistema de armazenamento de arquivos Colossus do Google oferece suporte ao controle refinado da localização do bloco de dados, embora essa funcionalidade não seja exposta diretamente ao público. O BigQuery usa esse recurso para colocar tabelas de clientes em um único

Machine Translated by Google

local, permitindo largura de banda ultra-alta para consultas nesse local. Referimo-nos a isso como ***armazenamento de objetos híbridos*** porque combina as abstrações limpas do armazenamento de objetos com algumas vantagens de colocar computação e armazenamento. A Amazon também oferece alguma noção de armazenamento de objetos híbridos por meio do S3 Select, um recurso que permite aos usuários filtrar dados do S3 diretamente em clusters do S3 antes que os dados sejam retornados pela rede.[4]

Especulamos que as nuvens públicas adotarão o armazenamento de objetos híbridos mais amplamente para melhorar o desempenho de suas ofertas e fazer uso mais eficiente dos recursos de rede disponíveis. Alguns podem já estar fazendo isso sem divulgar isso publicamente.

O conceito de armazenamento de objetos híbridos ressalta que ainda pode haver vantagens em ter acesso de baixo nível ao hardware em vez de depender da nuvem pública de outra pessoa. Os serviços de nuvem pública não expõem detalhes de baixo nível de hardware e sistemas (por exemplo, localizações de blocos de dados para Colossus), mas esses detalhes podem ser extremamente úteis na otimização e aprimoramento de desempenho. Veja nossa discussão sobre economia de nuvem no Capítulo 4.

Enquanto estamos vendo uma migração em massa de dados para nuvens públicas, acreditamos que muitos fornecedores de serviços de dados em hiperescala que atualmente rodam em nuvens públicas fornecidas por outros fornecedores podem construir seus data centers no futuro, embora com profunda integração de rede em nuvens públicas.

Clonagem de cópia zero

Sistemas baseados em nuvem baseados em armazenamento de objetos suportam ***clonagem de cópia zero.*** Isso normalmente significa que uma nova cópia virtual de um objeto é criada (por exemplo, uma nova tabela) sem necessariamente copiar fisicamente os dados subjacentes. Normalmente, novos ponteiros são criados para os arquivos de dados brutos e alterações futuras nessas tabelas não serão registradas na tabela antiga. Para aqueles familiarizados com o funcionamento interno de linguagens orientadas a objetos, como Python, esse tipo de cópia “superficial” é familiar em outros contextos.

A clonagem de cópia zero é um recurso atraente, mas os engenheiros devem entender seus pontos fortes e limitações. Por exemplo, clonando um objeto em um data lake

Machine Translated by Google

ambiente e, em seguida, excluir os arquivos no objeto original também pode eliminar o novo objeto.

Para sistemas baseados em armazenamento de objetos totalmente gerenciados (por exemplo, Snowflake e BigQuery), os engenheiros precisam estar extremamente familiarizados com os limites exatos da cópia superficial. Os engenheiros têm mais acesso ao armazenamento de objetos subjacentes em sistemas de data lake, como Databricks - uma bênção e uma maldição. Os engenheiros de dados devem ter muito cuidado antes de excluir qualquer arquivo bruto no armazenamento de objeto subjacente. Databricks e outras tecnologias de gerenciamento de data lake às vezes também suportam uma noção de ***cópia profunda,*** na qual todos os objetos de dados subjacentes são copiados. Este é um processo mais caro, mas também mais robusto no caso de arquivos serem perdidos ou excluídos involuntariamente.

Ciclo de vida do armazenamento de dados e retenção de dados

Armazenar dados não é tão simples quanto salvá-los no armazenamento de objetos ou em disco e esquecê-los. Você precisa pensar no ciclo de vida do armazenamento de dados e na retenção de dados. Ao pensar na frequência de acesso e nos casos de uso, pergunte: “Qual a importância dos dados para os usuários downstream e com que frequência eles precisam acessá-los?” Este é o ciclo de vida do armazenamento de dados. Outra pergunta que você deve fazer é: “Por quanto tempo devo manter esses dados?” Você precisa reter dados indefinidamente ou pode descartá-los após um determinado período de tempo?
Isso é retenção de dados. Vamos mergulhar em cada um deles.

Dados quentes, mornos e frios

Você sabia que os dados têm uma temperatura? Dependendo da frequência com que os dados são acessados, podemos agrupar aproximadamente a maneira como eles são armazenados em três categorias de persistência: quente, morno e frio. Os padrões de acesso à consulta diferem para cada conjunto de dados (Figura 6-9). Normalmente, os dados mais recentes são consultados com mais frequência do que os dados mais antigos. Vejamos os dados quentes, frios e quentes nessa ordem.

Machine Translated by Google

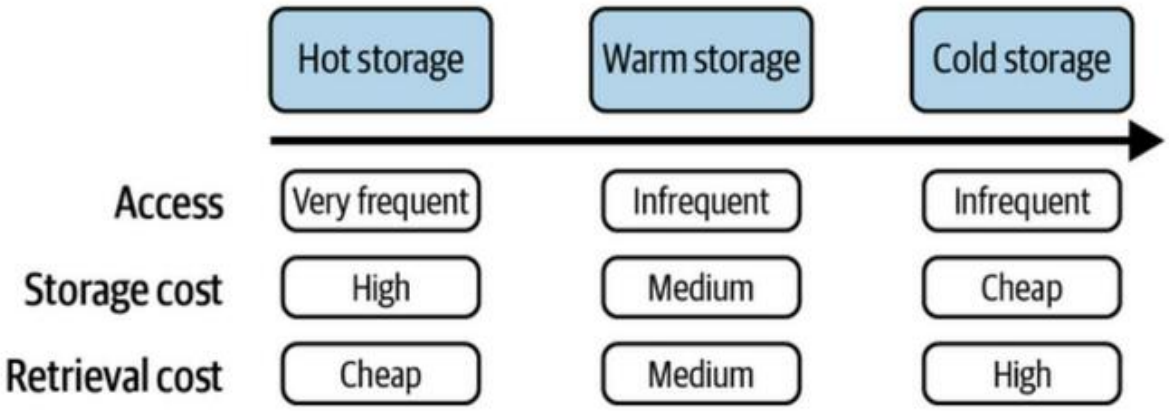


Figura 6-9. Custos de dados quentes, mornos e frios associados à frequência de acesso

Dados quentes

Os ***dados quentes*** têm requisitos de acesso instantâneo ou frequente. O armazenamento subjacente para dados ativos é adequado para acesso e leituras rápidos, como SSD ou memória. Por causa do tipo de hardware envolvido com dados quentes, armazenar dados quentes geralmente é a forma mais cara de armazenamento. Exemplos de casos de uso para dados quentes incluem a recuperação de recomendações de produtos e resultados da página do produto. O custo de armazenamento de dados quentes é o mais alto desses três níveis de armazenamento, mas a recuperação geralmente é barata.

O cache de resultados da consulta é outro exemplo de dados quentes. Quando uma consulta é executada, alguns mecanismos de consulta persistirão os resultados da consulta no cache. Por um tempo limitado, quando a mesma consulta é executada, em vez de executar novamente a mesma consulta no armazenamento, o cache de resultados da consulta exibe os resultados armazenados em cache. Isso permite tempos de resposta de consulta muito mais rápidos em comparação com a emissão redundante da mesma consulta repetidamente. Nos próximos capítulos, abordaremos os caches de resultados de consulta com mais detalhes.

Dados quentes

Os dados quentes são acessados semi-regularmente, digamos, uma vez por mês. Nenhuma regra rígida e rápida indica com que frequência os dados quentes são acessados, mas são menos do que dados quentes e mais do que dados frios. Os principais provedores de nuvem oferecem camadas de armazenamento de objetos que acomodam dados quentes. Por exemplo, o S3 oferece um nível de acesso pouco frequente e o Google Cloud tem um nível de armazenamento semelhante chamado Nearline. Os fornecedores fornecem seus modelos de frequência de acesso recomendada e os engenheiros também podem fazer sua modelagem e monitoramento de custos.

Machine Translated by Google

O armazenamento de dados quentes é mais barato do que os dados quentes, com custos de recuperação um pouco mais caros.

Dados frios

No outro extremo, ***os dados frios*** são dados acessados com pouca frequência. O hardware usado para arquivar dados frios geralmente é barato e durável, como HDD, armazenamento em fita e sistemas de arquivamento baseados em nuvem. Os dados frios destinam-se principalmente ao arquivamento de longo prazo, quando há pouca ou nenhuma intenção de acessar os dados.
Embora armazenar dados frios seja barato, recuperar dados frios geralmente é caro.

Considerações sobre o nível de armazenamento

Ao considerar a camada de armazenamento para seus dados, considere os custos de cada camada. Se você armazenar todos os seus dados em armazenamento a quente, todos os dados poderão ser acessados rapidamente. Mas isso tem um preço tremendo! Por outro lado, se você armazenar todos os dados em armazenamento frio para economizar custos, certamente reduzirá seus custos de armazenamento, mas às custas de tempos de recuperação prolongados e altos custos de recuperação se precisar acessar dados. O preço do armazenamento diminui de armazenamento mais rápido/com desempenho superior para armazenamento mais baixo.

O armazenamento a frio é popular para arquivar dados. Historicamente, o armazenamento a frio envolvia backups físicos e muitas vezes o envio desses dados para terceiros que os arquivavam em um cofre literal. O armazenamento a frio é cada vez mais popular na nuvem. Todo fornecedor de nuvem oferece uma solução de dados frios e você deve pesar o custo de enviar dados para o armazenamento frio versus o custo e o tempo para recuperar os dados.

Os engenheiros de dados precisam levar em conta o transbordamento do armazenamento quente para o quente/frio. A memória é cara e finita. Por exemplo, se os dados quentes estiverem armazenados na memória, eles poderão ser despejados no disco quando houver muitos dados novos para armazenar e memória insuficiente. Alguns bancos de dados podem mover dados acessados com pouca frequência para camadas quentes ou frias, descarregando os dados para HDD ou armazenamento de objetos. Este último é cada vez mais comum devido à relação custo-benefício do armazenamento de objetos. Se você estiver na nuvem e usando serviços gerenciados, o transbordamento de disco ocorrerá automaticamente.

Machine Translated by Google

Se você estiver usando armazenamento de objetos baseado em nuvem, crie políticas de ciclo de vida automatizadas para seus dados. Isso reduzirá drasticamente seus custos de armazenamento. Por exemplo, se seus dados precisarem ser acessados apenas uma vez por mês, mova os dados para uma camada de armazenamento de acesso pouco frequente. Se seus dados tiverem 180 dias e não forem acessados para consultas atuais, mova-os para uma camada de armazenamento de arquivamento. Em ambos os casos, você pode automatizar a migração de dados para fora do armazenamento regular de objetos e economizará dinheiro. Dito isso, considere os custos de recuperação — tanto em tempo quanto em dinheiro — usando camadas de armazenamento pouco frequentes ou de estilo de arquivamento. Os tempos e custos de acesso e recuperação podem variar dependendo do provedor de nuvem. Alguns provedores de nuvem tornam simples e barato migrar dados para armazenamento de arquivo, mas é caro e lento para recuperar seus dados.

Retenção de dados

Nos primórdios do “big data”, havia uma tendência a errar por acumular todos os dados possíveis, independentemente de sua utilidade.
A expectativa era: “podemos precisar desses dados no futuro”. Esse armazenamento de dados inevitavelmente se tornou pesado e sujo, dando origem a pântanos de dados e repressão regulatória à retenção de dados, entre outras consequências e pesadelos. Atualmente, os engenheiros de dados precisam considerar a retenção de dados: quais dados você ***precisa*** manter e por quanto ***tempo*** você deve mantê-los? Aqui estão algumas coisas para pensar com a retenção de dados.

Valor

Os dados são um ativo, portanto, você deve saber o valor dos dados que está armazenando. É claro que o valor é subjetivo e depende de quanto vale para seu caso de uso imediato e sua organização mais ampla. Esses dados são impossíveis de recriar ou podem ser facilmente recriados consultando sistemas upstream?

Qual é o impacto para os usuários a jusante se esses dados estiverem disponíveis versus se não estiverem?

Tempo

O valor para os usuários a jusante também depende da idade dos dados. Os novos dados geralmente são mais valiosos e acessados com frequência do que os dados mais antigos.
As limitações técnicas podem determinar por quanto tempo você pode armazenar dados em determinados

Machine Translated by Google

camadas de armazenamento. Por exemplo, se você armazenar dados quentes no cache ou na memória, provavelmente precisará definir um tempo de vida (TTL), para poder expirar os dados após um determinado ponto ou mantê-los no armazenamento quente ou frio. Caso contrário, seu armazenamento ativo ficará cheio e as consultas nos dados ativos sofrerão atrasos de desempenho.

Observância

Certos regulamentos (por exemplo, HIPAA e Payment Card Industry, ou PCI) podem exigir que você mantenha os dados por um determinado período. Nessas situações, os dados simplesmente precisam estar acessíveis mediante solicitação, mesmo que a probabilidade de uma solicitação de acesso seja baixa. Outras regulamentações podem exigir que você mantenha os dados apenas por um período limitado de tempo, e você precisará ter a capacidade de excluir informações específicas no prazo e dentro das diretrizes de conformidade. Você precisará de um processo de armazenamento e arquivamento de dados — juntamente com a capacidade de pesquisar os dados — que atenda aos requisitos de retenção do regulamento específico que você precisa cumprir. Claro, você vai querer equilibrar a conformidade com o custo.

Custo

Os dados são um ativo que (espero) tem um ROI. No lado do custo do ROI, uma despesa óbvia de armazenamento está associada aos dados. Considere a linha do tempo na qual você precisa reter dados. Dada nossa discussão sobre dados quentes, mornos e frios, implemente práticas de gerenciamento de ciclo de vida de dados automáticos e mova os dados para armazenamento frio se você não precisar dos dados após a data de retenção exigida. Ou exclua dados se realmente não forem necessários.

Armazenamento de locatário único versus multilocatário

No Capítulo 3, abordamos as vantagens e desvantagens entre a arquitetura de locatário único e multilocatário. Para recapitular, com a arquitetura ***de locatário único ,*** cada grupo de locatários (por exemplo, usuários individuais, grupos de usuários, contas ou clientes) obtém seu próprio conjunto dedicado de recursos, como rede, computação e armazenamento. Uma arquitetura ***multilocatário*** inverte isso e compartilha esses recursos entre grupos de usuários. Ambas as arquiteturas são amplamente utilizadas.

Machine Translated by Google

Esta seção examina as implicações do armazenamento de locatário único e multilocatário.

Adotar o armazenamento de locatário único significa que cada locatário obtém seu armazenamento dedicado. No exemplo da Figura 6-10, cada locatário obtém um banco de dados. Nenhum dado é compartilhado entre esses bancos de dados e o armazenamento é totalmente isolado. Um exemplo de uso de armazenamento de locatário único é que os dados de cada cliente devem ser armazenados isoladamente e não podem ser combinados com os dados de nenhum outro cliente. Nesse caso, cada cliente recebe seu próprio banco de dados.



Figura 6-10. No armazenamento de locatário único, cada locatário obtém seu próprio banco de dados

O armazenamento de dados separado implica em esquemas separados e independentes, estruturas de bucket e tudo relacionado ao armazenamento. Isso significa que você tem a liberdade de projetar o ambiente de armazenamento de cada locatário para ser uniforme ou deixá-los evoluir de qualquer maneira. A variação de esquema entre os clientes pode ser uma vantagem e uma complicação; como sempre, considere os trade-offs. Se o esquema de cada locatário não for uniforme em todos os locatários, isso terá consequências importantes se você precisar consultar as tabelas de vários locatários para criar uma exibição unificada de todos os dados de locatários.

O armazenamento multilocatário permite o armazenamento de vários locatários em um único banco de dados. Por exemplo, em vez do cenário de locatário único em que os clientes obtêm seu próprio banco de dados, vários clientes podem residir nos mesmos esquemas de banco de dados ou tabelas em um banco de dados multilocatário. Armazenar dados de vários locatários significa que os dados de cada locatário são armazenados no mesmo local (Figura 6-11).

Machine Translated by Google

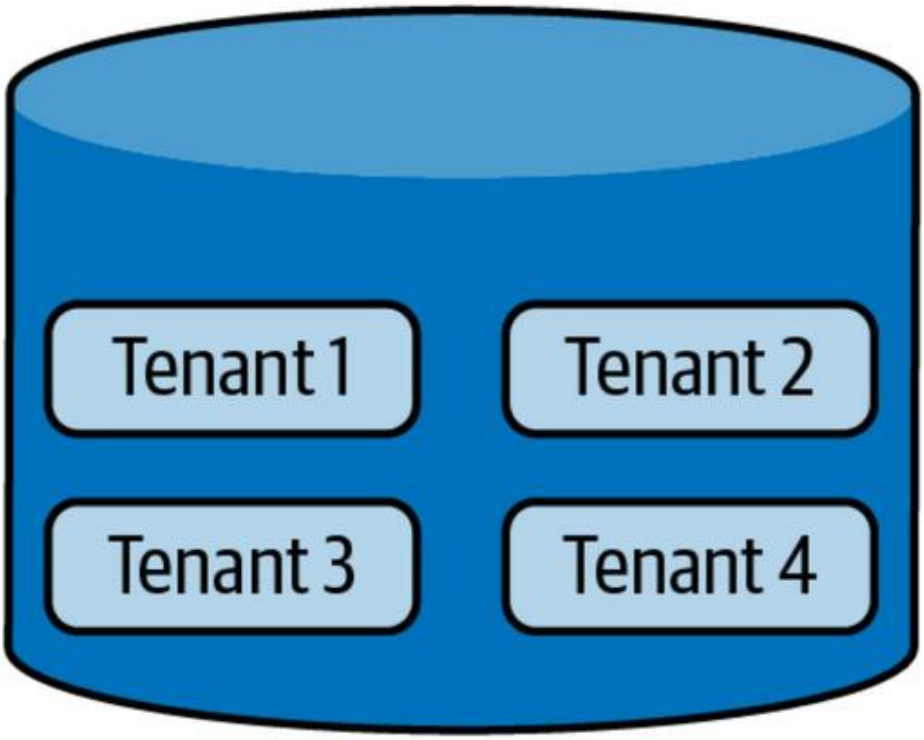


Figura 6-11. Neste armazenamento multilocatário, quatro inquilinos ocupam o mesmo banco de dados

Você precisa estar ciente da consulta de armazenamento único e multilocatário, que abordaremos com mais detalhes no Capítulo 8.

Com quem você vai trabalhar

O armazenamento está no centro da infraestrutura de engenharia de dados. Você interagirá com as pessoas que possuem sua infraestrutura de TI — normalmente, DevOps, segurança e arquitetos de nuvem. Definir domínios de responsabilidade entre a engenharia de dados e outras equipes é fundamental. Os engenheiros de dados têm autoridade para implantar sua infraestrutura em uma conta da AWS ou outra equipe deve lidar com essas alterações? Trabalhe com outras equipes para definir processos simplificados para que as equipes possam trabalhar juntas com eficiência e rapidez.

A divisão de responsabilidades pelo armazenamento de dados dependerá significativamente da maturidade da organização envolvida. O engenheiro de dados provavelmente gerenciará os sistemas de armazenamento e o fluxo de trabalho se a empresa estiver no início de sua maturidade de dados. Se a empresa estiver mais avançada em sua maturidade de dados, o engenheiro de dados provavelmente gerenciará uma seção do sistema de armazenamento. Este engenheiro de dados irá

Machine Translated by Google

também provavelmente interagem com engenheiros em ambos os lados do armazenamento — ingestão e transformação.

O engenheiro de dados precisa garantir que os sistemas de armazenamento usados pelos usuários downstream estejam disponíveis com segurança, contenham dados de alta qualidade, tenham ampla capacidade de armazenamento e funcionem quando consultas e transformações são
corre.

Subcorrentes

As subcorrentes do armazenamento são significativas porque o armazenamento é um hub crítico para todos os estágios do ciclo de vida da engenharia de dados. Ao contrário de outras subcorrentes para as quais os dados podem estar em movimento (ingestão) ou consultados e transformados, as subcorrentes para armazenamento diferem porque o armazenamento é tão onipresente.

Segurança

Embora os engenheiros muitas vezes vejam a segurança como um impedimento ao seu trabalho, eles devem abraçar a ideia de que a segurança é um facilitador fundamental. A segurança robusta com controle de acesso a dados refinado permite que os dados sejam compartilhados e consumidos de forma mais ampla dentro de uma empresa. O valor dos dados aumenta significativamente quando isso é possível.

Como sempre, exerça o princípio do privilégio mínimo. Não dê acesso total ao banco de dados a ninguém, a menos que seja necessário. Isso significa que a maioria dos engenheiros de dados não precisa de acesso total ao banco de dados na prática. Além disso, preste atenção aos controles de acesso em nível de coluna, linha e célula em seu banco de dados. Dê aos usuários apenas as informações de que precisam e nada mais.

Gestão de dados

O gerenciamento de dados é fundamental à medida que lemos e gravamos dados com sistemas de armazenamento.

Catálogos de dados e gerenciamento de metadados

Machine Translated by Google

Os dados são aprimorados por metadados robustos. A catalogação habilita cientistas de dados, analistas e engenheiros de ML, permitindo a descoberta de dados. A linhagem de dados acelera o tempo para rastrear problemas de dados e permite que os consumidores localizem fontes brutas upstream. À medida que você constrói seus sistemas de armazenamento, invista em seus metadados. A integração de um dicionário de dados com essas outras ferramentas permite que os usuários compartilhem e registrem o conhecimento institucional de forma robusta.

O gerenciamento de metadados também melhora significativamente a governança de dados. Além de simplesmente habilitar a catalogação e a linhagem de dados passiva, considere implementar análises nesses sistemas para obter uma imagem clara e ativa do que está acontecendo com seus dados.

Versionamento de dados no armazenamento de objetos

Os principais sistemas de armazenamento de objetos em nuvem permitem o controle de versão de dados. O controle de versão de dados pode ajudar na recuperação de erros quando os processos falham e os dados são corrompidos. O controle de versão também é benéfico para rastrear o histórico de conjuntos de dados usados para criar modelos. Assim como o controle de versão de código permite que os desenvolvedores rastreiem commits que causam bugs, o controle de versão de dados pode ajudar os engenheiros de ML a rastrear alterações que levam à degradação do desempenho do modelo.

Privacidade

O GDPR e outros regulamentos de privacidade afetaram significativamente o design do sistema de armazenamento. Quaisquer dados com implicações de privacidade têm um ciclo de vida que os engenheiros de dados devem gerenciar. Os engenheiros de dados devem estar preparados para responder a solicitações de exclusão de dados e remover dados seletivamente conforme necessário. Além disso, os engenheiros podem acomodar privacidade e segurança por meio de anonimização e mascaramento.

DataOps

O DataOps não é ortogonal ao gerenciamento de dados e existe uma área significativa de sobreposição. A DataOps se preocupa com o monitoramento operacional tradicional de sistemas de armazenamento e o monitoramento dos próprios dados, indissociáveis de metadados e qualidade.

Machine Translated by Google

Monitoramento de sistemas

Os engenheiros de dados devem monitorar o armazenamento de várias maneiras. Isso inclui o monitoramento de componentes de armazenamento de infraestrutura, onde eles existem, mas também o monitoramento de armazenamento de objetos e outros sistemas “sem servidor”. Os engenheiros de dados devem assumir a liderança em FinOps (gerenciamento de custos), monitoramento de segurança e monitoramento de acesso.

Observando e monitorando dados

Embora os sistemas de metadados, como descrevemos, sejam críticos, uma boa engenharia deve considerar a natureza entrópica dos dados, procurando ativamente entender suas características e observando grandes mudanças. Os engenheiros podem monitorar estatísticas de dados, aplicar métodos de detecção de anomalias ou regras simples e testar e validar ativamente inconsistências lógicas.

Arquitetura de dados

O Capítulo 3 aborda os fundamentos da arquitetura de dados, pois o armazenamento é o ponto crítico do ciclo de vida da engenharia de dados.

Considere as seguintes dicas de arquitetura de dados. Design para confiabilidade e durabilidade exigidas. Entenda os sistemas de origem upstream e como esses dados, uma vez ingeridos, serão armazenados e acessados. Entenda os tipos de modelos de dados e consultas que ocorrerão no downstream.

Se for esperado que os dados cresçam, você pode negociar o armazenamento com seu provedor de nuvem? Adote uma abordagem ativa para FinOps e trate-o como uma parte central das conversas sobre arquitetura. Não otimize prematuramente, mas prepare-se para escalar se houver oportunidades de negócios operando em grandes volumes de dados.

Inclinar-se para sistemas totalmente gerenciados e entender os SLAs do provedor. Os sistemas totalmente gerenciados geralmente são muito mais robustos e escaláveis do que os sistemas que você precisa cuidar.

Orquestração

Machine Translated by Google

A orquestração é altamente emaranhada com o armazenamento. O armazenamento permite que os dados fluam por meio de pipelines, e a orquestração é a bomba. A orquestração também ajuda os engenheiros a lidar com a complexidade dos sistemas de dados, combinando potencialmente muitos sistemas de armazenamento e mecanismos de consulta.

Engenharia de software

Podemos pensar em engenharia de software no contexto de armazenamento de duas maneiras. Primeiro, o código que você escreve deve funcionar bem com seu sistema de armazenamento. Certifique-se de que o código que você escreve armazena os dados corretamente e não causa dados acidentalmente, vazamentos de memória ou problemas de desempenho. Segundo, defina sua infraestrutura de armazenamento como código e use recursos de computação efêmeros na hora de processar seus dados. Como o armazenamento é cada vez mais distinto da computação, você pode aumentar e diminuir os recursos automaticamente enquanto mantém seus dados no armazenamento de objetos. Isso mantém sua infraestrutura limpa e evita o acoplamento de suas camadas de armazenamento e consulta.

Conclusão

O armazenamento está em toda parte e está subjacente a muitos estágios do ciclo de vida da engenharia de dados. Neste capítulo, você aprendeu sobre os ingredientes brutos, tipos, abstrações e grandes ideias em torno dos sistemas de armazenamento. Obtenha um conhecimento profundo do funcionamento interno e das limitações dos sistemas de armazenamento que você usará. Conheça os tipos de dados, atividades e cargas de trabalho apropriados para seu armazenamento.

Recursos adicionais

"SGBD orientado a colunas" Página da Wikipédia

"Banco de dados Rowise vs. Colunar? Teoria e prática" por Mangat Rai Modi

"O Projeto e Implementação de Sistemas Modernos de Banco de Dados Orientados a Colunas" por Daniel Abadi et al.

Machine Translated by Google

"Mergulhando no Lago Delta: Aplicação e Evolução do Esquema" por Burak Yavuz et al.

"Antipadrões de solução de floco de neve: o provável cientista de dados" por John Aven

"O que, quando, por que e como de cargas incrementais" por Tim Mitchell

A "Criação e replicação de dados da IDC crescerão a uma taxa mais rápida do que Capacidade de armazenamento instalada, de acordo com o comunicado de imprensa do IDC Global DataSphere e StorageSphere Forecasts"

"O que é um banco de dados de vetores?" por Bryan Turriff

"Dados quentes versus dados frios: por que é importante" por Afzaal Ahmad Zeeshan

1 Andy Klein, "Hard Disk Drive (HDD) vs. Solid-State Drive (SSD): Qual é a diferença?" Blog Backblaze, 5 de outubro de 2021, ***https:// oreil.ly/ XBps8.***

2 Benoit Dageville, "The Snowflake Elastic Data Warehouse," ***SIGMOD '16: Proceedings of the 2016 International Conference on Management of Data*** (junho de 2016): 215–226, ***https:// oreil.ly/ Tc1su.***

3 James Dixon, "Data Lakes Revisited", ***James Dixon's Blog,*** 25 de setembro de 2014, ***https:// oreil.ly/ FH25v.***

4 Valliappa Lakshmanan e Jordan Tigani, ***GoogleBig Query: The Definitive Guide*** (Seastopol, CA: O'Reilly, 2019), página 15, ***https:// oreil.ly/ 5aXXu***

Machine Translated by Google

Capítulo 7. Ingestão

Você aprendeu sobre os vários sistemas de origem que provavelmente encontrará como engenheiro de dados e sobre maneiras de armazenar dados. Vamos agora voltar nossa atenção para os padrões e escolhas que se aplicam à ingestão de dados de vários sistemas de origem. Neste capítulo, discutimos a ingestão de dados (consulte a Figura 7-1), as principais considerações de engenharia para a fase de ingestão, os principais padrões para ingestão em lote e streaming, tecnologias que você encontrará, com quem trabalhará ao desenvolver seu pipeline de ingestão de dados e como as correntes secundárias aparecem na fase de ingestão.

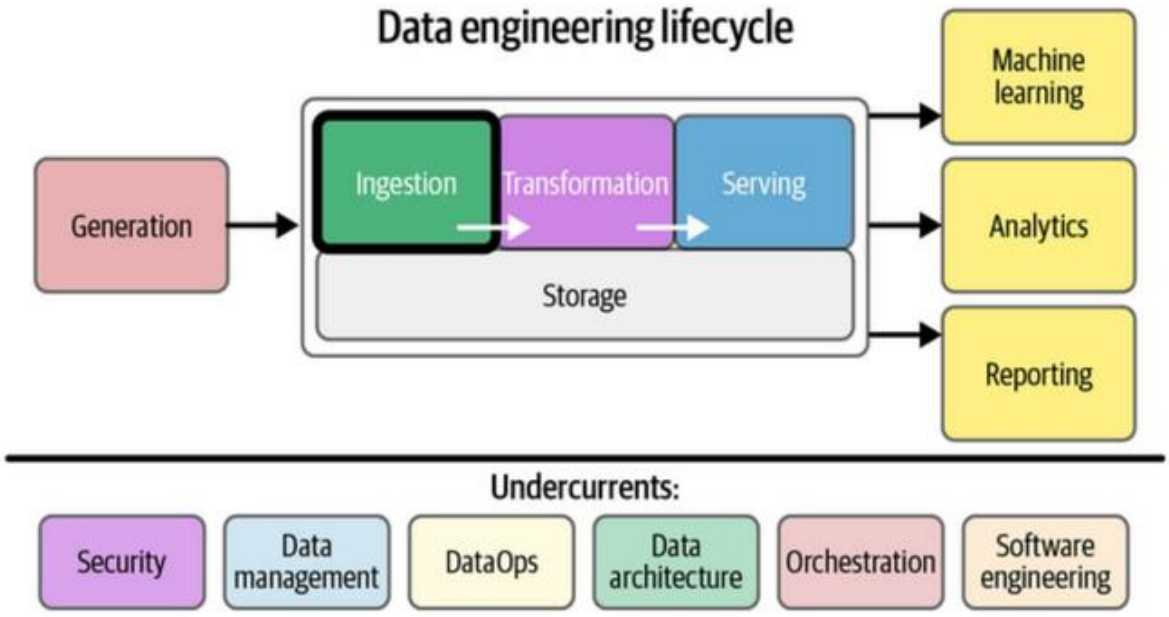


Figura 7-1. Para começar a processar dados, devemos ingeri-los

O que é ingestão de dados?

A ingestão de dados é o processo de mover dados de um lugar para outro.
A ingestão de dados implica a movimentação de dados dos sistemas de origem para o armazenamento no ciclo de vida da engenharia de dados, com a ingestão como uma etapa intermediária (Figura 7-2).

Machine Translated by Google



Figura 7-2. Os dados do sistema 1 são ingeridos no sistema 2

Vale a pena comparar rapidamente a ingestão de dados com a integração de dados. Enquanto ***a ingestão*** de dados é a movimentação de dados do ponto A para o B, ***a integração*** de dados combina dados de fontes diferentes em um novo conjunto de dados. Por exemplo, você pode usar a integração de dados para combinar dados de um sistema de CRM, dados de análise de publicidade e análise da web para criar um perfil de usuário, que é salvo em seu data warehouse. Além disso, usando o ETL reverso, você pode enviar esse perfil de usuário recém-criado de ***volta*** ao seu CRM para que os vendedores possam usar os dados para priorizar leads. Descrevemos a integração de dados mais detalhadamente no Capítulo 8, onde discutimos as transformações de dados; ETL reverso é abordado no Capítulo 9.

Também destacamos que a ingestão de dados é diferente da ***ingestão interna*** dentro de um sistema. Os dados armazenados em um banco de dados são copiados de uma tabela para outra ou os dados em um fluxo são armazenados em cache temporariamente. Consideramos isso outra parte do processo geral de transformação de dados abordado no Capítulo 8.

Machine Translated by Google

PIPELINES DE DADOS DEFINIDOS

Os pipelines de dados começam nos sistemas de origem, mas a ingestão é o estágio em que os engenheiros de dados começam a projetar ativamente as atividades do pipeline de dados. No espaço de engenharia de dados, uma grande quantidade de cerimônias ocorre em torno de padrões de movimentação e processamento de dados, com padrões estabelecidos como ETL, padrões mais recentes como ELT e novos nomes para práticas estabelecidas há muito tempo (ETL reverso) e compartilhamento de dados.

Todos esses conceitos estão englobados na ideia de um ***pipeline de dados.*** É essencial entender os detalhes desses vários padrões e saber que um pipeline de dados moderno inclui todos eles. À medida que o mundo se afasta de uma abordagem monolítica tradicional com restrições rígidas no movimento de dados e em direção a um ecossistema aberto de serviços em nuvem que são montados como peças LEGO para realizar produtos, os engenheiros de dados priorizam o uso das ferramentas certas para alcançar o resultado desejado em vez de aderir a uma filosofia estreita de movimentação de dados.

Em geral, aqui está nossa definição de um pipeline de dados:

> ***Um pipeline de dados é a combinação de arquitetura, sistemas e processos que movem os dados pelos estágios do ciclo de vida da engenharia de dados.***

Nossa definição é deliberadamente fluida – e intencionalmente vaga – para permitir que os engenheiros de dados conectem o que for necessário para realizar a tarefa em questão. Um pipeline de dados pode ser um sistema ETL tradicional, em que os dados são ingeridos de um sistema transacional local, passados por um processador monolítico e gravados em um data warehouse. Ou pode ser um pipeline de dados baseado em nuvem que extrai dados de 100 fontes, combina-os em 20 tabelas amplas, treina cinco outros modelos de ML, implanta-os em produção e monitora o desempenho contínuo. Um pipeline de dados deve ser flexível o suficiente para atender a qualquer necessidade ao longo do ciclo de vida da engenharia de dados.

Vamos manter essa noção de pipelines de dados em mente à medida que avançamos neste capítulo.

Machine Translated by Google

Principais Considerações de Engenharia para o Fase de ingestão

Ao se preparar para arquitetar ou construir um sistema de ingestão, aqui estão algumas considerações e perguntas principais a serem feitas relacionadas à ingestão de dados:

- Qual é o caso de uso para os dados que estou ingerindo?
- Posso reutilizar esses dados e evitar a ingestão de várias versões do mesmo conjunto de dados?
- Para onde vão os dados? Qual é o destino?
- Com que frequência os dados devem ser atualizados da fonte?
- Qual é o volume de dados esperado?
- Em que formato estão os dados? O armazenamento e a transformação downstream podem aceitar esse formato?
- Os dados de origem estão em boas condições para uso imediato a jusante? Ou seja, os dados são de boa qualidade? Que pós-processamento é necessário para atendê-lo? Quais são os riscos de qualidade de dados (por exemplo, o tráfego de bots para um site pode contaminar os dados)?
- Os dados exigem processamento em andamento para ingestão de downstream se os dados forem de uma fonte de streaming?

Essas perguntas reduzem a ingestão de lote e streaming e se aplicam à arquitetura subjacente que você criará, construirá e manterá. Independentemente da frequência com que os dados são ingeridos, você deve considerar estes fatores ao projetar sua arquitetura de ingestão:

- Limitado versus ilimitado
- Frequência
- Síncrono versus assíncrono

Machine Translated by Google

- Serialização e desserialização
- Taxa de transferência e escalabilidade elástica
- Confiabilidade e durabilidade
- Carga útil
- Padrões push versus pull versus poll

Vejamos cada um deles.

Limitado versus ilimitado

Como você deve se lembrar do Capítulo 3, os dados vêm em duas formas: limitados e ilimitados (Figura 7-3). ***Dados ilimitados*** são dados como existem na realidade, à medida que os eventos acontecem, esporadicamente ou continuamente, em andamento e fluindo. ***Dados limitados*** são uma maneira conveniente de agrupar dados em algum tipo de limite, como o tempo.

Machine Translated by Google

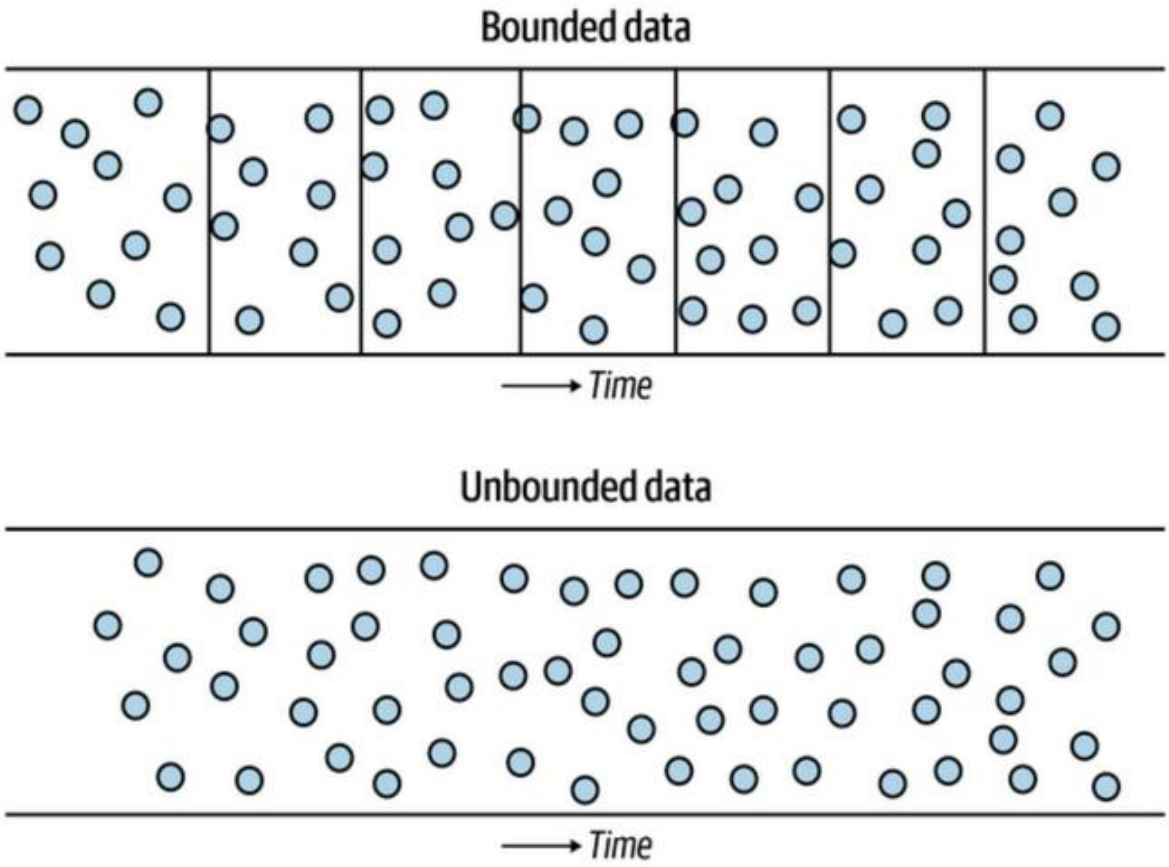


Figura 7-3. Dados limitados versus dados ilimitados

Vamos adotar este mantra: ***Todos os dados são ilimitados até que sejam limitados.*** Como muitos mantras, este não é preciso 100% do tempo. A lista de compras que rabisquei esta tarde é de dados limitados. Eu o escrevi como um fluxo de consciência (dados ilimitados) em um pedaço de papel de rascunho, onde os pensamentos agora existem como uma lista de coisas (dados limitados) que preciso comprar no supermercado. No entanto, a ideia é correta para fins práticos para a grande maioria dos dados que você manipulará em um contexto de negócios. Por exemplo, um varejista on-line processará transações de clientes 24 horas por dia até que o negócio falhe, a economia pare ou o sol exploda.

Os processos de negócios há muito impõem limites artificiais aos dados cortando lotes discretos. Tenha em mente a verdadeira falta de limites de seus dados; os sistemas de ingestão de streaming são simplesmente uma ferramenta para preservar a natureza ilimitada dos dados para que as etapas subsequentes do ciclo de vida também possam processá-los continuamente.

Machine Translated by Google

Frequência

Uma das decisões críticas que os engenheiros de dados devem tomar ao projetar processos de ingestão de dados é a frequência de ingestão de dados. Os processos de ingestão podem ser em lote, microlote ou em tempo real.

As frequências de ingestão variam drasticamente de lenta a rápida (Figura 7-4). No lado mais lento, uma empresa pode enviar seus dados fiscais para uma empresa de contabilidade uma vez por ano. No lado mais rápido, um sistema CDC pode recuperar novas atualizações de log de um banco de dados de origem uma vez por minuto. Ainda mais rápido, um sistema pode ingerir continuamente eventos de sensores de IoT e processá-los em segundos. As frequências de ingestão de dados geralmente são misturadas em uma empresa, dependendo do caso de uso e das tecnologias.

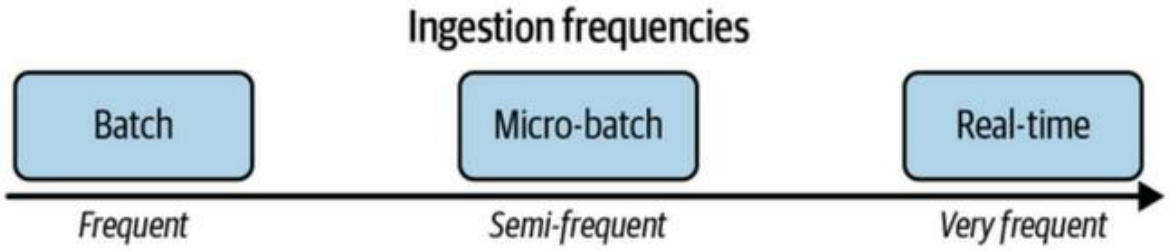


Figura 7-4. O lote de espectro para frequências de ingestão em tempo real

Observamos que os padrões de ingestão “em tempo real” estão se tornando cada vez mais comuns. Colocamos “tempo real” entre aspas porque nenhum sistema de ingestão é genuinamente em tempo real. Qualquer banco de dados, fila ou pipeline tem latência inerente na entrega de dados a um sistema de destino. É mais preciso falar ***em tempo quase real,*** mas geralmente usamos ***o tempo real*** para ser breve. O padrão quase em tempo real geralmente elimina uma frequência de atualização explícita; os eventos são processados no pipeline um por um à medida que chegam ou em microlotes (ou seja, lotes em intervalos de tempo concisos). Para este livro, usaremos ***tempo*** real e ***streaming de*** forma intercambiável.

Mesmo com um processo de ingestão de dados de streaming, o processamento em lote downstream é relativamente padrão. No momento da redação deste artigo, os modelos de ML geralmente são treinados em lote, embora o treinamento on-line contínuo esteja se tornando mais prevalente. Raramente os engenheiros de dados têm a opção de construir um pipeline puramente quase em tempo real sem componentes em lote. Em vez disso, eles escolhem onde ocorrerão os limites do lote - ou seja, os dados do ciclo de vida da engenharia de dados

Machine Translated by Google

será dividido em lotes. Quando os dados atingem um processo em lote, a frequência do lote se torna um gargalo para todo o processamento posterior.

Além disso, os sistemas de streaming são os mais adequados para muitos tipos de fontes de dados. Em aplicativos de IoT, o padrão típico é que cada sensor grave eventos ou medições em sistemas de streaming à medida que eles acontecem. Embora esses dados possam ser gravados diretamente em um banco de dados, uma plataforma de ingestão de streaming, como Amazon Kinesis ou Apache Kafka, é mais adequada para o aplicativo.
Os aplicativos de software podem adotar padrões semelhantes gravando eventos em uma fila de mensagens à medida que eles acontecem, em vez de esperar que um processo de extração extraia eventos e informações de estado de um banco de dados de back-end. Esse padrão funciona excepcionalmente bem para arquiteturas orientadas a eventos que já trocam mensagens por meio de filas. E, novamente, as arquiteturas de streaming geralmente coexistem com o processamento em lote.

Ingestão síncrona versus assíncrona

Com a ***ingestão síncrona,*** a origem, a ingestão e o destino têm dependências complexas e são fortemente acopladas. Como você pode ver na Figura 7-5 , cada estágio do ciclo de vida da engenharia de dados tem os processos A, B e C diretamente dependentes um do outro. Se o processo A falhar, os processos B e C não poderão ser iniciados; se o processo B falhar, o processo C não será iniciado. Esse tipo de fluxo de trabalho síncrono é comum em sistemas ETL mais antigos, em que os dados extraídos de um sistema de origem devem ser transformados antes de serem carregados em um data warehouse. Os processos a jusante da ingestão não podem ser iniciados até que todos os dados do lote tenham sido ingeridos. Se o processo de ingestão ou transformação falhar, todo o processo deverá ser executado novamente.

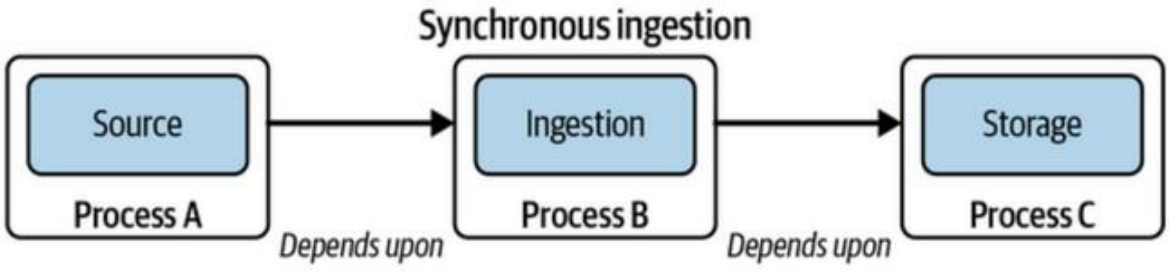


Figura 7-5. Um processo de ingestão síncrona é executado como etapas em lote discretas

Machine Translated by Google

Aqui está um mini estudo de caso de como ***não*** projetar seus pipelines de dados. Em uma empresa, o próprio processo de transformação era uma série de dezenas de fluxos de trabalho síncronos fortemente acoplados, com todo o processo levando mais de 24 horas para ser concluído. Se alguma etapa desse pipeline de transformação falhar, todo o processo de transformação terá que ser reiniciado desde o início! Nesse caso, vimos processo após processo falhar e, devido a mensagens de erro inexistentes ou enigmáticas, consertar o pipeline foi um jogo de golpe que levou mais de uma semana para diagnosticar e curar. Enquanto isso, a empresa não tinha relatórios atualizados durante esse período. As pessoas não estavam felizes.

Com a ***ingestão assíncrona,*** as dependências agora podem operar no nível de eventos individuais, da mesma forma que em um back-end de software criado a partir de microsserviços (Figura 7-6). Os eventos individuais ficam disponíveis no armazenamento assim que são ingeridos individualmente. Veja o exemplo de um aplicativo web na AWS que emite eventos no Amazon Kinesis Data Streams (aqui atuando como um buffer). O stream é lido pelo Apache Beam, que analisa e enriquece os eventos e os encaminha para um segundo stream do Kinesis; O Kinesis Data Firehose acumula eventos e grava objetos no Amazon S3.

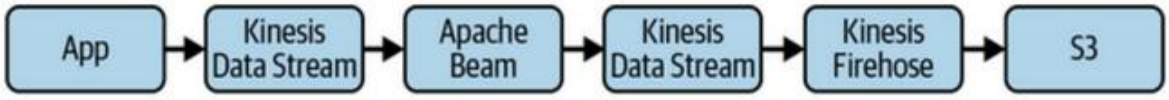


Figura 7-6. Processamento assíncrono de um fluxo de eventos na AWS

A grande ideia é que, em vez de depender do processamento assíncrono, onde um processo em lote é executado para cada estágio à medida que o lote de entrada é fechado e determinadas condições de tempo são atendidas, cada estágio do pipeline assíncrono pode processar itens de dados à medida que se tornam disponíveis em paralelo no Conjunto de feixes. A taxa de processamento depende dos recursos disponíveis. O Kinesis Data Stream atua como amortecedor, moderando a carga para que os picos de taxa de eventos não sobrecarreguem o processamento downstream. Os eventos passarão pelo pipeline rapidamente quando a taxa de eventos for baixa e qualquer backlog for limpo. Observe que poderíamos modificar o cenário e usar um Kinesis Data Stream para armazenamento, eventualmente extraindo eventos para o S3 antes que eles expirem fora do fluxo.

Machine Translated by Google

Serialização e desserialização

Mover dados da origem para o destino envolve serialização e desserialização. Como lembrete, ***serialização*** significa codificar os dados de uma fonte e preparar estruturas de dados para transmissão e estágios intermediários de armazenamento.

Ao ingerir dados, certifique-se de que seu destino possa desserializar os dados que recebe. Vimos dados ingeridos de uma origem, mas que ficam inertes e inutilizáveis no destino porque os dados não podem ser desserializados adequadamente.
Veja a discussão mais extensa sobre serialização no Apêndice A.

Taxa de transferência e escalabilidade

Em teoria, sua ingestão nunca deve ser um gargalo. Na prática, os gargalos de ingestão são bastante comuns. A taxa de transferência de dados e a escalabilidade do sistema tornam-se críticas à medida que seus volumes de dados crescem e os requisitos mudam.
Projete seus sistemas para dimensionar e reduzir para corresponder com flexibilidade à taxa de transferência de dados desejada.

De onde você está ingerindo dados importa muito. Se você estiver recebendo dados conforme são gerados, o sistema upstream terá algum problema que possa afetar seus pipelines de ingestão downstream? Por exemplo, suponha que um banco de dados de origem fique inativo. Quando ele voltar a ficar on-line e tentar preencher as cargas de dados decorridos, sua ingestão será capaz de acompanhar esse influxo repentino de dados em atraso?

Outra coisa a considerar é sua capacidade de lidar com a ingestão de dados em rajadas.
A geração de dados raramente acontece a uma taxa constante e muitas vezes vai e vem.
O buffer integrado é necessário para coletar eventos durante picos de taxa para evitar que os dados sejam perdidos. O armazenamento em buffer preenche o tempo enquanto o sistema é dimensionado e permite que os sistemas de armazenamento acomodem intermitências mesmo em um sistema dinamicamente escalável.

Sempre que possível, use serviços gerenciados que lidam com o dimensionamento da taxa de transferência para você. Embora você possa realizar essas tarefas manualmente adicionando mais servidores, fragmentos ou trabalhadores, muitas vezes isso não é um trabalho de valor agregado e

Machine Translated by Google

há uma boa chance de você perder alguma coisa. Grande parte desse trabalho pesado agora é automatizado. Não reinvente a roda de ingestão de dados se você não tiver
para.

Confiabilidade e Durabilidade

Confiabilidade e durabilidade são vitais nos estágios de ingestão de pipelines de dados.
A confiabilidade envolve alto tempo de atividade e failover adequado para sistemas de ingestão.
A durabilidade implica garantir que os dados não sejam perdidos ou corrompidos.

Algumas fontes de dados (por exemplo, dispositivos IoT e caches) podem não reter dados se não forem ingeridos corretamente. Uma vez perdido, ele se foi para sempre. Nesse sentido, a ***confiabilidade*** dos sistemas de ingestão leva diretamente à ***durabilidade*** dos dados gerados. Se os dados forem ingeridos, os processos downstream podem, teoricamente, ficar atrasados se forem interrompidos temporariamente.

Nosso conselho é avaliar os riscos e construir um nível adequado de redundância e autorrecuperação com base no impacto e custo da perda de dados.
Confiabilidade e durabilidade têm custos diretos e indiretos. Por exemplo, seu processo de ingestão continuará se uma zona da AWS ficar inativa? Que tal uma região inteira? Que tal a rede elétrica ou a internet? Claro, nada é de graça. Quanto isso vai te custar? Você pode construir um sistema altamente redundante e ter uma equipe de plantão 24 horas por dia para lidar com interrupções. Isso também significa que seus custos de nuvem e mão-de-obra se tornam proibitivos (custos diretos) e o trabalho contínuo afeta significativamente sua equipe (custos indiretos). Não há resposta correta e você precisa avaliar os custos e benefícios de suas decisões de confiabilidade e durabilidade.

Não assuma que você pode construir um sistema que irá ingerir dados de forma confiável e durável em todos os cenários possíveis. Mesmo o orçamento quase infinito do governo federal dos EUA não pode garantir isso. Em muitos cenários extremos, a ingestão de dados realmente não importa. Haverá pouco a ingerir se a internet cair, mesmo se você construir vários data centers com lacunas de ar em bunkers subterrâneos com energia independente. Avalie continuamente as compensações e os custos de confiabilidade e durabilidade.

Machine Translated by Google

Carga útil

Uma ***carga útil*** é o conjunto de dados que você está ingerindo e tem características como tipo, forma, tamanho, esquema e tipos de dados e metadados. Vejamos algumas dessas características para entender por que isso é importante.

Gentil

O ***tipo*** de dados que você manipula afeta diretamente como eles são tratados no downstream no ciclo de vida da engenharia de dados. Tipo consiste em tipo e formato. Os dados têm um tipo — tabular, imagem, vídeo, texto, etc. O tipo influencia diretamente o formato dos dados ou a forma como são expressos em bytes, nomes e extensões de arquivo. Por exemplo, um tipo de dados tabular pode estar em formatos como CSV ou Parquet, com cada um desses formatos tendo diferentes padrões de bytes para serialização e desserialização. Outro tipo de dado é uma imagem, que tem o formato JPG ou PNG e é inerentemente não estruturada.

Forma

Cada carga útil tem uma ***forma*** que descreve suas dimensões. A forma dos dados é crítica em todo o ciclo de vida da engenharia de dados. Por exemplo, as dimensões de pixel e vermelho, verde, azul (RGB) de uma imagem são necessárias para treinar modelos de aprendizado profundo. Como outro exemplo, se você estiver tentando importar um arquivo CSV para uma tabela de banco de dados e seu CSV tiver mais colunas do que a tabela de banco de dados, você provavelmente receberá um erro durante o processo de importação. Aqui estão alguns exemplos das formas de vários tipos de dados:

Tabular

> O número de linhas e colunas no conjunto de dados, comumente expresso como ***M*** linhas e ***N*** colunas

JSON semiestruturado

> Os pares de valores-chave e a profundidade de aninhamento ocorrem com subelementos

Texto não estruturado

Machine Translated by Google

Número de palavras, caracteres ou bytes no corpo do texto

Imagens

A largura, altura e profundidade de cor RGB (por exemplo, 8 bits por pixel)

Áudio descompactado

Número de canais (por exemplo, dois para estéreo), profundidade da amostra (por exemplo, 16 bits por amostra), taxa de amostragem (por exemplo, 48 kHz) e duração (por exemplo, 10.003 segundos)

Tamanho

O ***tamanho*** dos dados descreve o número de bytes de uma carga útil. Uma carga útil pode variar em tamanho de bytes únicos a terabytes e maiores. Para reduzir o tamanho de uma carga útil, ela pode ser compactada em vários formatos, como ZIP e TAR (consulte a discussão sobre compactação no Apêndice A).

Uma carga útil massiva também pode ser dividida em partes, o que reduz efetivamente o tamanho da carga útil em subseções menores. Ao carregar um arquivo enorme em um armazenamento de objetos em nuvem ou data warehouse, essa é uma prática comum, pois os pequenos arquivos individuais são mais fáceis de transmitir em uma rede (especialmente se estiverem compactados). Os arquivos fragmentados menores são enviados ao seu destino e, em seguida, remontados após a chegada de todos os dados.

Esquema e tipos de dados Muitas

cargas de dados têm um esquema, como dados tabulares e semiestruturados. Conforme mencionado anteriormente neste livro, um esquema descreve os campos e tipos de dados dentro desses campos. Outros dados, como texto não estruturado, imagens e áudio, não terão um esquema ou tipos de dados explícitos. No entanto, eles podem vir com descrições de arquivos técnicos sobre forma, dados e formato de arquivo, codificação, tamanho etc.

Embora você possa se conectar a bancos de dados de várias maneiras (como exportação de arquivos, CDC, JDBC/ODBC), a conexão é fácil. A grande engenharia

Machine Translated by Google

desafio é entender o esquema subjacente. Os aplicativos organizam os dados de várias maneiras, e os engenheiros precisam estar intimamente familiarizados com a organização dos dados e os padrões de atualização relevantes para compreendê-los.
O problema foi um pouco exacerbado pela popularidade do mapeamento relacional de objetos (ORM), que gera automaticamente esquemas baseados na estrutura de objetos em linguagens como Java ou Python. Estruturas naturais em uma linguagem orientada a objetos geralmente são mapeadas para algo confuso em um banco de dados operacional. Os engenheiros de dados podem precisar se familiarizar com a estrutura de classes do código do aplicativo.

Schema não é apenas para bancos de dados. Conforme discutimos, as APIs apresentam suas complicações de esquema. Muitas APIs de fornecedores têm métodos de relatório amigáveis que preparam dados para análise. Em outros casos, os engenheiros não têm tanta sorte.
A API é um invólucro fino em torno de sistemas subjacentes, exigindo que os engenheiros entendam os componentes internos do aplicativo para usar os dados.

Grande parte do trabalho associado à ingestão de esquemas de origem acontece no estágio de transformação do ciclo de vida da engenharia de dados, que discutimos no Capítulo 8. Colocamos essa discussão aqui porque os engenheiros de dados precisam começar a estudar esquemas de origem assim que planejam ingerir dados de uma nova
fonte.

A comunicação é fundamental para entender os dados de origem, e os engenheiros também têm a oportunidade de reverter o fluxo de comunicação e ajudar os engenheiros de software a melhorar os dados onde são produzidos. Mais adiante neste capítulo, retornaremos a este tópico em “Com quem você trabalhará”.

Detectando e manipulando alterações de esquema em sistemas upstream e downstream

As alterações de esquema ocorrem com frequência em sistemas de origem e geralmente estão fora do controle dos engenheiros de dados. Exemplos de alterações de esquema incluem o seguinte:

Adicionando uma nova coluna

Alterando um tipo de coluna

Machine Translated by Google

Criando uma nova tabela

Renomeando uma coluna

Está se tornando cada vez mais comum que as ferramentas de ingestão automatizem a detecção de alterações de esquema e até mesmo atualizem automaticamente as tabelas de destino. Em última análise, isso é uma espécie de bênção mista. As alterações de esquema ainda podem interromper os pipelines a jusante do preparo e da ingestão.

Os engenheiros ainda devem implementar estratégias para responder às mudanças automaticamente e alertar sobre as mudanças que não podem ser acomodadas automaticamente. A automação é excelente, mas os analistas e cientistas de dados que dependem desses dados devem ser informados sobre as alterações de esquema que violam as suposições existentes. Mesmo que a automação possa acomodar uma mudança, o novo esquema pode afetar negativamente o desempenho de relatórios e modelos.
A comunicação entre aqueles que fazem alterações no esquema e os afetados por essas alterações é tão importante quanto a automação confiável que verifica as alterações no esquema.

Registros de esquema

No streaming de dados, cada mensagem tem um esquema, e esses esquemas podem evoluir entre produtores e consumidores. Um ***registro de esquema*** é um repositório de metadados usado para manter a integridade do esquema e do tipo de dados diante de esquemas em constante mudança. Os registros de esquema também podem rastrear versões e histórico de esquema. Ele descreve o modelo de dados para mensagens, permitindo serialização e desserialização consistente entre produtores e consumidores. Os registros de esquema são usados na maioria das principais ferramentas de dados e nuvens.

Metadados

Além das características aparentes que acabamos de abordar, uma carga útil geralmente contém metadados, que discutimos primeiro no Capítulo 2. Metadados são dados sobre dados. Os metadados podem ser tão críticos quanto os próprios dados. Uma das limitações significativas da abordagem inicial do data lake – ou pântano de dados, que poderia se tornar um site de superfundo de dados – foi a completa falta de atenção aos metadados. Sem uma descrição detalhada dos dados, pode ser

Machine Translated by Google

de pouco valor. Já discutimos alguns tipos de metadados (por exemplo, esquema) e os abordaremos muitas vezes ao longo deste capítulo.

Padrões Push versus Pull versus Poll

Mencionamos push versus pull quando apresentamos o ciclo de vida de engenharia de dados no Capítulo 2. Uma estratégia ***push*** (Figura 7-7) envolve um sistema de origem enviando dados para um destino, enquanto uma estratégia ***pull*** (Figura 7-8) envolve um destino lendo dados diretamente de uma fonte. Como mencionamos nessa discussão, as linhas entre essas estratégias são embaçadas.

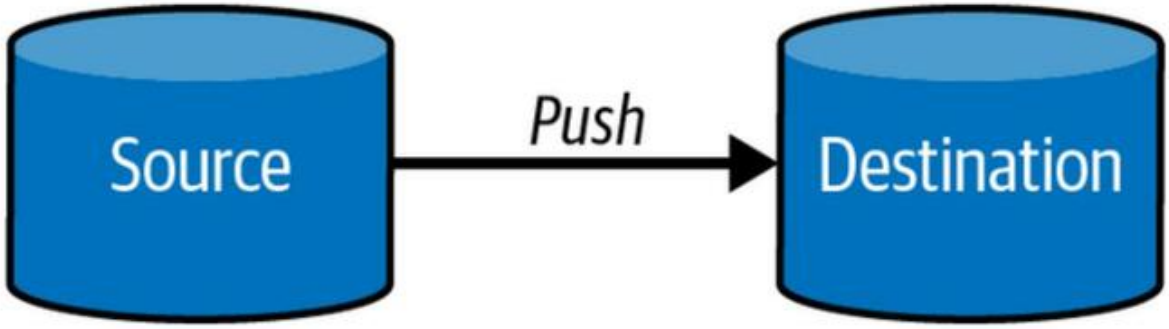


Figura 7-7. Empurrando dados da origem para o destino

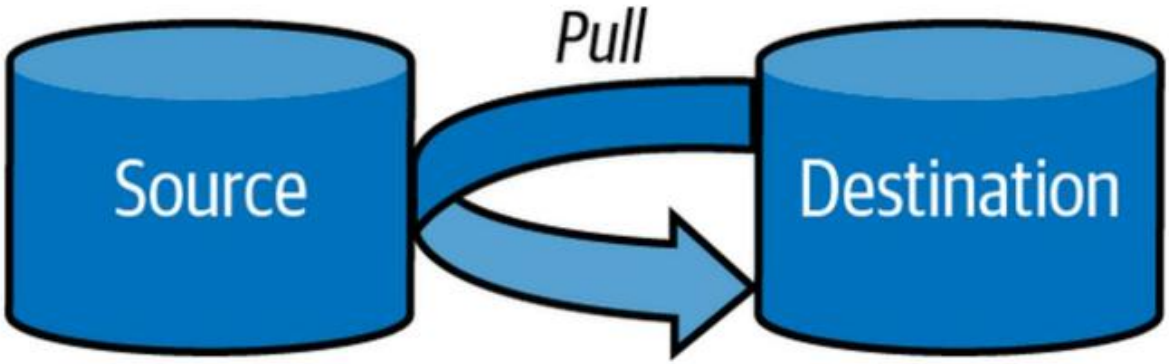


Figura 7-8. Um destino extraindo dados de uma origem

Outro padrão relacionado à extração é a ***pesquisa*** de dados (Figura 7-9). A sondagem envolve a verificação periódica de uma fonte de dados para quaisquer alterações. Quando as alterações são detectadas, o destino extrai os dados como faria em uma situação normal de recebimento.

Machine Translated by Google

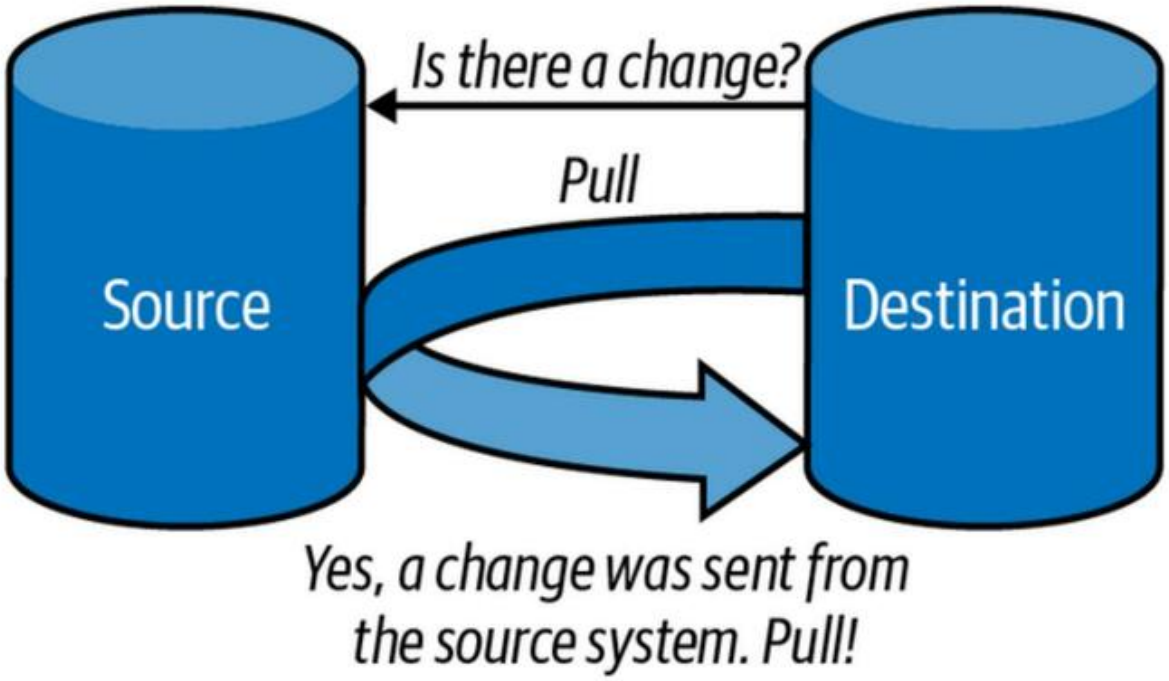


Figura 7-9. Pesquisa de alterações em um sistema de origem

Considerações sobre ingestão de lote

A ingestão em lote, que envolve o processamento de dados em massa, geralmente é uma maneira conveniente de ingerir dados. Isso significa que os dados são ingeridos tomando um subconjunto de dados de um sistema de origem, com base em um intervalo de tempo ou no tamanho dos dados acumulados (Figura 7-10).

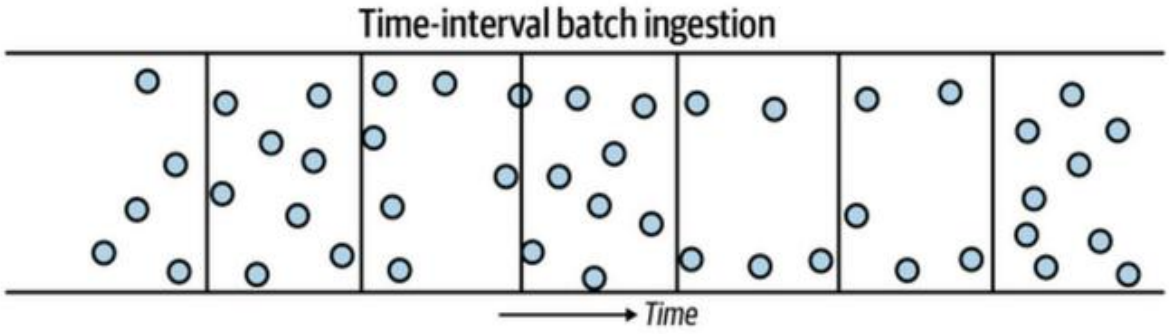


Figura 7-10. Ingestão de lote com intervalo de tempo

A ingestão de lote com intervalo de tempo é generalizada no ETL de negócios tradicional para armazenamento de dados. Esse padrão é frequentemente usado para processar dados uma vez por dia,

Machine Translated by Google

durante a noite fora do horário comercial, para fornecer relatórios diários, mas outras frequências também podem ser usadas.

A ingestão de lotes baseada em tamanho (Figura 7-11) é bastante comum quando os dados são movidos de um sistema baseado em streaming para o armazenamento de objetos; em última análise, você deve cortar os dados em blocos discretos para processamento futuro em um data lake. Alguns sistemas de ingestão baseados em tamanho podem dividir dados em objetos com base em vários critérios, como o tamanho em bytes do número total de eventos.

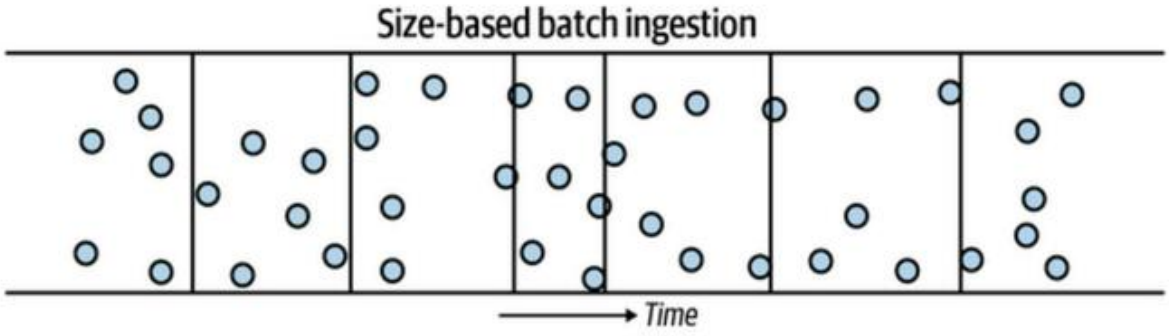


Figura 7-11. Ingestão de lote com base no tamanho

Alguns padrões de ingestão em lote comumente usados, que discutimos nesta seção, incluem o seguinte:

- Snapshot ou extração diferencial
- Exportação e ingestão com base em arquivo
- ETL versus ELT
- Inserções, atualizações e tamanho do lote
- Migração de dados

Snapshot ou Extração Diferencial

Os engenheiros de dados devem escolher entre capturar instantâneos completos de um sistema de origem ou atualizações diferenciais (às vezes chamadas ***de incrementais) .*** Com ***instantâneos completos,*** os engenheiros obtêm todo o estado atual do sistema de origem em cada leitura de atualização. Com o padrão de ***atualização diferencial ,*** os engenheiros podem extrair apenas as atualizações e alterações desde a última leitura do sistema de origem.

Machine Translated by Google

Embora as atualizações diferenciais sejam ideais para minimizar o tráfego de rede e o uso de armazenamento de destino, as leituras completas de instantâneos permanecem extremamente comuns devido à sua simplicidade.

Exportação e ingestão baseada em arquivo

Os dados são frequentemente movidos entre bancos de dados e sistemas usando arquivos. Os dados são serializados em arquivos em um formato intercambiável e esses arquivos são fornecidos a um sistema de ingestão. Consideramos a exportação baseada em arquivo como um padrão de ingestão ***baseado em push .*** Isso ocorre porque o trabalho de exportação e preparação de dados é feito no lado do sistema de origem.

A ingestão baseada em arquivo tem várias vantagens em potencial sobre uma abordagem de conexão direta com o banco de dados. Muitas vezes, é indesejável permitir o acesso direto aos sistemas de back-end por motivos de segurança. Com a ingestão baseada em arquivo, os processos de exportação são executados no lado da fonte de dados, dando aos engenheiros do sistema de origem controle total sobre quais dados são exportados e como os dados são pré-processados. Depois que os arquivos são concluídos, eles podem ser fornecidos ao sistema de destino de várias maneiras. Os métodos comuns de troca de arquivos são armazenamento de objetos, protocolo de transferência segura de arquivos (SFTP), intercâmbio eletrônico de dados (EDI) ou cópia segura (SCP).

ETL versus ELT

O Capítulo 3 apresentou ETL e ELT, ambos padrões extremamente comuns de ingestão, armazenamento e transformação que você encontrará em cargas de trabalho em lote. Esta seção abrange as partes de extração ***(E)*** e carga ***(L)*** de ETL e ELT.

Extrair

Extrair significa obter dados de um sistema de origem. Embora a ***extração*** pareça implicar a ***extração*** de dados, ela também pode ser baseada em push. A extração também pode exigir a leitura de metadados e alterações de esquema.

Carregar

Machine Translated by Google

Depois que os dados são extraídos, eles podem ser transformados (ETL) antes de carregá-los em um destino de armazenamento ou simplesmente carregados no armazenamento para transformação futura. Ao carregar dados, você deve estar atento ao tipo de sistema que está carregando, ao esquema dos dados e ao impacto no desempenho do carregamento. Cobrimos ETL e ELT no Capítulo 8.

Inserções, atualizações e tamanho do lote

Os sistemas orientados a lotes geralmente têm um desempenho ruim quando os usuários tentam executar muitas operações em lotes pequenos em vez de um número menor de operações grandes. Por exemplo, embora seja comum inserir uma linha por vez em um banco de dados transacional, esse é um padrão ruim para muitos bancos de dados colunares, pois força a criação de muitos arquivos pequenos e subótimos e força o sistema a executar um grande número de ***criar*** operações de objeto. A execução de muitas pequenas operações de atualização no local é um problema ainda maior porque faz com que o banco de dados verifique cada arquivo de coluna existente para executar a atualização.

Entenda os padrões de atualização apropriados para o banco de dados ou armazenamento de dados com o qual você está trabalhando. Além disso, entenda que certas tecnologias são desenvolvidas especificamente para altas taxas de inserção. Por exemplo, Apache Druid e Apache Pinot podem lidar com altas taxas de inserção. O SingleStore pode gerenciar cargas de trabalho híbridas que combinam características OLAP e OLTP. O BigQuery tem um desempenho ruim em uma alta taxa de inserções de linha única do SQL vanilla, mas extremamente bom se os dados forem alimentados por meio do buffer de fluxo. Conheça os limites e as características de suas ferramentas.

Migração de dados

A migração de dados para um novo banco de dados ou ambiente geralmente não é trivial e os dados precisam ser movidos em massa. Às vezes, isso significa mover tamanhos de dados de centenas de terabytes ou muito maiores, geralmente envolvendo a migração de tabelas específicas e a movimentação de bancos de dados e sistemas inteiros.

As migrações de dados provavelmente não são uma ocorrência regular como engenheiro de dados, mas você deve estar familiarizado com elas. Como costuma ser o caso da ingestão de dados, o gerenciamento de esquema é uma consideração crucial. Suponha que você esteja migrando

Machine Translated by Google

dados de um sistema de banco de dados para outro (digamos, SQL Server para Snowflake). Não importa o quanto os dois bancos de dados se pareçam, diferenças sutis quase sempre existem na maneira como eles lidam com o esquema.
Felizmente, geralmente é fácil testar a ingestão de uma amostra de dados e encontrar problemas de esquema antes de realizar uma migração completa da tabela.

A maioria dos sistemas de dados funciona melhor quando os dados são movidos em massa e não como linhas ou eventos individuais. O armazenamento de arquivos ou objetos geralmente é um excelente estágio intermediário para a transferência de dados. Além disso, um dos maiores desafios da migração de banco de dados não é a movimentação dos dados em si, mas a movimentação das conexões de pipeline de dados do sistema antigo para o novo.

Esteja ciente de que muitas ferramentas estão disponíveis para automatizar vários tipos de migrações de dados. Especialmente para migrações grandes e complexas, sugerimos examinar essas opções antes de fazer isso manualmente ou escrever sua própria solução de migração.

Ingestão de mensagens e fluxos Considerações

A ingestão de dados de eventos é comum. Esta seção aborda questões que você deve considerar ao ingerir eventos, com base nos tópicos abordados nos Capítulos 5 e 6.

Evolução do esquema

A evolução do esquema é comum ao manipular dados de eventos; campos podem ser adicionados ou removidos, ou tipos de valor podem mudar (digamos, uma string para um inteiro). A evolução do esquema pode ter impactos não intencionais em seus pipelines e destinos de dados. Por exemplo, um dispositivo IoT obtém uma atualização de firmware que adiciona um novo campo ao evento que transmite ou uma API de terceiros introduz alterações na carga útil do evento ou em inúmeros outros cenários. Tudo isso afeta potencialmente seus recursos de downstream.

Para aliviar os problemas relacionados à evolução do esquema, aqui estão algumas sugestões. Primeiro, se sua estrutura de processamento de eventos tiver um registro de esquema (discutido

Machine Translated by Google

anteriormente neste capítulo), use-o para versionar suas mudanças de esquema. Em seguida, uma fila de mensagens mortas (descrita em “Tratamento de erros e filas de mensagens mortas”) pode ajudá-lo a investigar problemas com eventos que não são tratados adequadamente.
Por fim, a rota de baixa fidelidade (e a mais eficaz) é comunicar regularmente com as partes interessadas upstream sobre possíveis alterações de esquema e abordar proativamente as alterações de esquema com as equipes que introduzem essas alterações, em vez de reagir ao destinatário das alterações importantes.

Dados de chegada tardia

Embora você provavelmente prefira que todos os dados do evento cheguem no horário, os dados do evento podem chegar atrasados. Um grupo de eventos pode ocorrer no mesmo período (horários de eventos semelhantes), mas alguns podem chegar mais tarde do que outros (tempos de ingestão tardia) devido a várias circunstâncias.

Por exemplo, um dispositivo IoT pode atrasar o envio de uma mensagem devido a problemas de latência da Internet. Isso é comum ao ingerir dados. Você deve estar ciente dos dados de chegada tardia e do impacto nos sistemas e usos downstream.
Suponha que você suponha que o tempo de ingestão ou processo seja o mesmo que o tempo do evento. Você pode obter alguns resultados estranhos se seus relatórios ou análises dependerem de um retrato preciso de quando os eventos ocorrem. Para lidar com dados de chegada tardia, você precisa definir um horário limite para quando os dados de chegada tardia não serão mais processados.

Encomenda e entrega múltipla

As plataformas de streaming geralmente são construídas a partir de sistemas distribuídos, o que pode causar algumas complicações. Especificamente, as mensagens podem ser entregues fora de ordem e mais de uma vez (entrega pelo menos uma vez). Consulte a discussão sobre plataformas de streaming de eventos no Capítulo 5 para obter mais detalhes.

Repetir

O ***Replay*** permite que os leitores solicitem uma série de mensagens do histórico, permitindo que você retroceda o histórico de eventos até um determinado momento.
O replay é um recurso essencial em muitas plataformas de ingestão de streaming e é

Machine Translated by Google

particularmente útil quando você precisa re-ingerir e reprocessar dados para um intervalo de tempo específico. Por exemplo, o RabbitMQ normalmente exclui mensagens depois que todos os assinantes as consomem. Kafka, Kinesis e Pub/Sub são compatíveis com retenção e repetição de eventos.

Tempo de Viver

Por quanto tempo você preservará o registro do seu evento? Um parâmetro chave é ***o tempo máximo de retenção de mensagens,*** também conhecido como ***tempo de vida*** (TTL). O TTL geralmente é uma configuração que você definirá por quanto tempo você deseja que os eventos vivam antes de serem reconhecidos e ingeridos. Qualquer evento não confirmado que não seja ingerido após a expiração do TTL desaparece automaticamente.
Isso é útil para reduzir a contrapressão e o volume de eventos desnecessários em seu pipeline de ingestão de eventos.

Encontre o equilíbrio certo do impacto do TTL em nosso pipeline de dados. Um TTL extremamente curto (milissegundos ou segundos) pode fazer com que a maioria das mensagens desapareça antes do processamento. Um TTL muito longo (várias semanas ou meses) criará uma lista de pendências de muitas mensagens não processadas, resultando em longos tempos de espera.

Vejamos como algumas plataformas populares lidam com o TTL no momento da redação deste artigo. O Google Cloud Pub/Sub é compatível com períodos de retenção de até 7 dias.
A retenção do Amazon Kinesis Data Streams pode ser ativada em até 365 dias.
O Kafka pode ser configurado para retenção indefinida, limitada pelo espaço em disco disponível. (O Kafka também suporta a opção de gravar mensagens mais antigas no armazenamento de objetos em nuvem, desbloqueando espaço de armazenamento e retenção praticamente ilimitados.)

Tamanho da mensagem

O tamanho da mensagem é um problema facilmente esquecido: você deve garantir que a estrutura de streaming em questão possa lidar com o tamanho máximo esperado da mensagem. O Amazon Kinesis oferece suporte a um tamanho máximo de mensagem de 1 MB.
O Kafka é padronizado para esse tamanho máximo, mas pode ser configurado para um máximo de 20 MB ou mais. (A configurabilidade pode variar em plataformas de serviços gerenciados.)

Machine Translated by Google

Tratamento de erros e filas de mensagens mortas

Às vezes, os eventos não são ingeridos com êxito. Talvez um evento seja enviado para um tópico ou fila de mensagens inexistente, o tamanho da mensagem pode ser muito grande ou o evento expirou além de seu TTL. Os eventos que não podem ser ingeridos precisam ser redirecionados e armazenados em um local separado chamado de ***dead letter queue.***

Uma dead letter queue separa eventos problemáticos de eventos que podem ser aceitos pelo consumidor (Figura 7-12). Se os eventos não forem redirecionados para uma fila de mensagens mortas, esses eventos errôneos correm o risco de bloquear a ingestão de outras mensagens. Os engenheiros de dados podem usar uma fila de mensagens mortas para diagnosticar por que ocorrem erros de ingestão de eventos e resolver problemas de pipeline de dados e podem reprocessar algumas mensagens na fila após corrigir a causa subjacente dos erros.

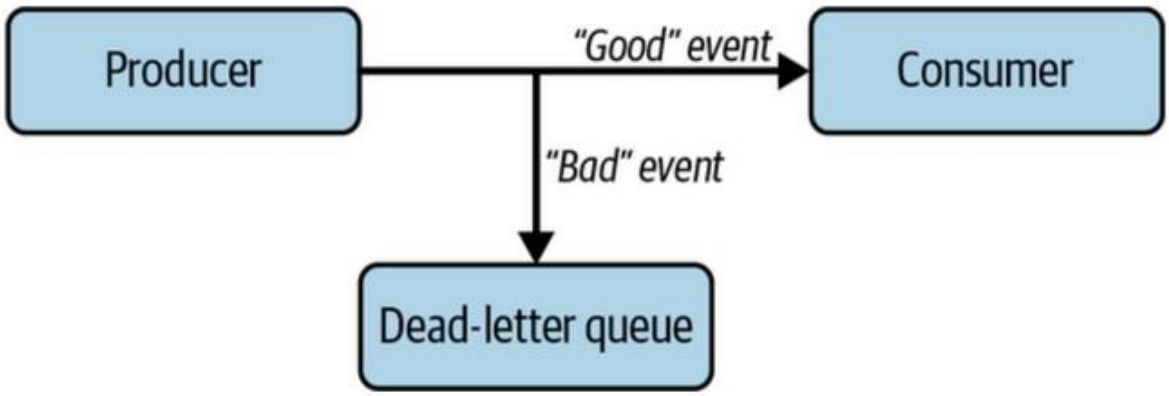


Figura 7-12. Eventos "bons" são passados para o consumidor, enquanto eventos "ruins" são armazenados em uma dead letter queue

Puxar e empurrar do consumidor

Um consumidor que assina um tópico pode obter eventos de duas maneiras: push e pull. Vejamos como algumas tecnologias de streaming extraem e enviam dados. Kafka e Kinesis suportam apenas assinaturas pull. Os assinantes lêem as mensagens de um tópico e confirmam quando elas foram processadas. Além disso, para receber assinaturas, Pub/Sub e RabbitMQ são compatíveis com assinaturas push, permitindo que esses serviços gravem mensagens para um ouvinte.

As assinaturas pull são a escolha padrão para a maioria dos aplicativos de engenharia de dados, mas você pode querer considerar os recursos push para

Machine Translated by Google

formulários. Observe que os sistemas de ingestão de mensagens somente pull ainda podem enviar por push se você adicionar uma camada extra para lidar com isso.

Localização

Muitas vezes, é desejável integrar o streaming em vários locais para obter redundância aprimorada e consumir dados próximos de onde são gerados.
Como regra geral, quanto mais próxima sua ingestão estiver de onde os dados se originam, melhor será a largura de banda e a latência. No entanto, você precisa equilibrar isso com os custos de movimentação de dados entre regiões para executar análises em um conjunto de dados combinado. Como sempre, os custos de saída de dados podem aumentar rapidamente. Faça uma avaliação cuidadosa das compensações à medida que você cria sua arquitetura.

Maneiras de ingerir dados

Agora que descrevemos alguns dos padrões significativos subjacentes à ingestão de lote e streaming, vamos nos concentrar nas maneiras de ingerir dados. Apesar de citarmos algumas formas comuns, tenha em mente que o universo de práticas e tecnologias de ingestão de dados é vasto e cresce a cada dia.

Conexão direta de banco de dados

Os dados podem ser extraídos de bancos de dados para ingestão consultando e lendo em uma conexão de rede. Mais comumente, essa conexão é feita usando ODBC ou JDBC.

O ODBC usa um driver hospedado por um cliente que acessa o banco de dados para traduzir os comandos emitidos para a API ODBC padrão em comandos emitidos para o banco de dados.
O banco de dados retorna os resultados da consulta pela rede, onde o driver os recebe e os traduz de volta em um formulário padrão e lido pelo cliente. Para ingestão, o aplicativo que utiliza o driver ODBC é uma ferramenta de ingestão. A ferramenta de ingestão pode extrair dados por meio de várias consultas pequenas ou de uma única consulta grande.

JDBC é conceitualmente muito semelhante ao ODBC. Um driver Java se conecta a um banco de dados remoto e serve como uma camada de tradução entre o padrão

Machine Translated by Google

API JDBC e a interface de rede nativa do banco de dados de destino. Pode parecer estranho ter uma API de banco de dados dedicada a uma única linguagem de programação, mas há fortes motivações para isso. A Java Virtual Machine (JVM) é padrão, portátil em arquiteturas de hardware e sistemas operacionais e fornece o desempenho de código compilado por meio de um compilador just-in-time (JIT). A JVM é uma VM de compilação extremamente popular para executar código de maneira portátil.

O JDBC fornece portabilidade extraordinária do driver de banco de dados. Drivers ODBC são enviados como binários nativos de SO e arquitetura; os fornecedores de banco de dados devem manter versões para cada versão de arquitetura/SO que desejam oferecer suporte. Por outro lado, os fornecedores podem enviar um único driver JDBC compatível com qualquer linguagem JVM (por exemplo, Java, Scala, Clojure ou Kotlin) e estrutura de dados JVM (por exemplo, Spark). JDBC tornou-se tão popular que também é usado como uma interface para linguagens não-JVM como Python; o ecossistema Python fornece ferramentas de tradução que permitem que o código Python se comunique com um driver JDBC em execução em uma JVM local.

JDBC e ODBC são usados extensivamente para ingestão de dados de bancos de dados relacionais, retornando ao conceito geral de conexões diretas de banco de dados. Vários aprimoramentos são usados para acelerar a ingestão de dados. Muitas estruturas de dados podem paralelizar várias conexões simultâneas e consultas de partição para extrair dados em paralelo. Por outro lado, nada é de graça; o uso de conexões paralelas também aumenta a carga no banco de dados de origem.

JDBC e ODBC foram por muito tempo os padrões ouro para ingestão de dados de bancos de dados, mas esses padrões de conexão estão começando a mostrar sua idade para muitos aplicativos de engenharia de dados. Esses padrões de conexão lutam com dados aninhados e enviam dados como linhas. Isso significa que os dados aninhados nativos devem ser recodificados como dados de string a serem enviados pela rede, e as colunas de bancos de dados colunares devem ser serializadas novamente como linhas.

Conforme discutido em “Exportação e ingestão baseada em arquivo”, muitos bancos de dados agora suportam exportação de arquivo nativo que ignora JDBC/ODBC e exporta dados diretamente em formatos como Parquet, ORC e Avro. Como alternativa, muitos data warehouses em nuvem fornecem APIs REST diretas.

Machine Translated by Google

As conexões JDBC geralmente devem ser integradas a outras tecnologias de ingestão. Por exemplo, geralmente usamos um processo de leitura para conectar a um banco de dados com JDBC, gravar os dados extraídos em vários objetos e, em seguida, orquestrar a ingestão em um sistema downstream (consulte a Figura 7-13). O processo de leitura pode ser executado em uma instância de nuvem totalmente efêmera ou em um sistema de orquestração.

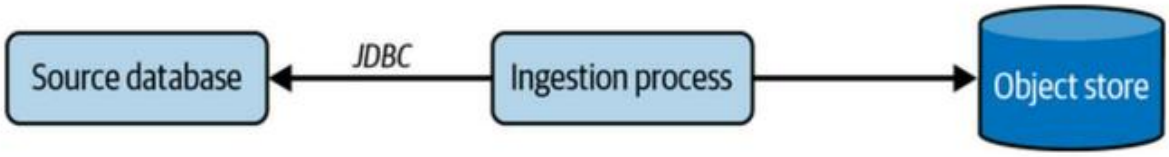


Figura 7-13. Um processo de ingestão lê de um banco de dados de origem usando JDBC e, em seguida, grava objetos no armazenamento de objetos. Um banco de dados de destino (não mostrado) pode ser acionado para ingerir os dados com uma chamada de API de um sistema de orquestração.

Alterar captura de dados

Captura de dados de alteração (CDC). apresentado no Capítulo 2, é o processo de ingestão de alterações de um sistema de banco de dados de origem. Por exemplo, podemos ter um sistema PostgreSQL de origem que suporte um aplicativo e ingere periodicamente ou continuamente alterações de tabela para análise.

Observe que nossa discussão aqui não é exaustiva. Apresentamos padrões comuns, mas sugerimos que você leia a documentação em um banco de dados específico para lidar com os detalhes das estratégias do CDC.

CDC orientado a lote

Se a tabela de banco de dados em questão tiver um campo updated_at contendo a última vez que um registro foi gravado ou atualizado, podemos consultar a tabela para encontrar todas as linhas atualizadas desde um horário especificado. Definimos o carimbo de data/hora do filtro com base na última vez em que capturamos as linhas alteradas das tabelas. Esse processo nos permite extrair alterações e atualizar diferencialmente uma tabela de destino.

Essa forma de CDC orientado a lotes tem uma limitação importante: embora possamos determinar facilmente quais linhas foram alteradas desde um determinado momento, não obtemos necessariamente todas as alterações aplicadas a essas linhas. Considere o exemplo de execução do CDC em lote em uma tabela de conta bancária a cada 24 horas.

Machine Translated by Google

Esta tabela operacional mostra o saldo da conta corrente para cada conta.
Quando o dinheiro entra e sai das contas, o aplicativo bancário executa uma transação para atualizar o saldo.

Quando executarmos uma consulta para retornar todas as linhas da tabela de contas que foram alteradas nas últimas 24 horas, veremos os registros de cada conta que registrou uma transação. Suponha que um determinado cliente tenha sacado dinheiro cinco vezes usando um cartão de débito nas últimas 24 horas. Nossa consulta retornará apenas o último saldo da conta registrado no período de 24 horas; outros registros durante o período não aparecerão. Esse problema pode ser mitigado utilizando um esquema somente de inserção, em que cada transação de conta é registrada como um novo registro na tabela (consulte “Somente inserção”).

CDC contínuo

O CDC contínuo captura todo o histórico da tabela e pode oferecer suporte à ingestão de dados quase em tempo real, seja para replicação de banco de dados em tempo real ou para alimentar análises de streaming em tempo real. Em vez de executar consultas periódicas para obter um lote de alterações de tabela, o CDC contínuo trata cada gravação no banco de dados como um evento.

Podemos capturar um fluxo de eventos para CDC contínuo de duas maneiras.
Uma das abordagens mais comuns com um banco de dados transacional como o PostgreSQL é ***o CDC baseado em log.*** O log binário do banco de dados registra todas as alterações no banco de dados sequencialmente (consulte “Logs do banco de dados”) Uma ferramenta CDC pode ler esse log e enviar os eventos para um destino, como a plataforma de streaming Apache Kafka Debezium.

Alguns bancos de dados oferecem suporte a um paradigma de CDC gerenciado e simplificado. Por exemplo, muitos bancos de dados hospedados em nuvem podem ser configurados para acionar diretamente uma função sem servidor ou gravar em um fluxo de eventos sempre que uma alteração ocorrer no banco de dados. Isso libera completamente os engenheiros de se preocuparem com os detalhes de como os eventos são capturados no banco de dados e encaminhados.

CDC e replicação de banco de dados

O CDC pode ser usado para replicar entre bancos de dados: os eventos são armazenados em buffer em um fluxo e gravados de ***forma assíncrona*** em um segundo banco de dados. Porém muitos

Machine Translated by Google

os bancos de dados oferecem suporte nativo a uma versão de replicação fortemente acoplada (replicação síncrona) que mantém a réplica totalmente sincronizada com o banco de dados primário. A replicação síncrona normalmente requer que o banco de dados primário e a réplica sejam do mesmo tipo (por exemplo, PostgreSQL para PostgreSQL). A vantagem da replicação síncrona é que o banco de dados secundário pode descarregar o trabalho do banco de dados primário agindo como uma réplica de leitura; as consultas de leitura podem ser redirecionadas para a réplica. A consulta retornará os mesmos resultados que seriam retornados do banco de dados primário.

As réplicas de leitura geralmente são usadas em padrões de ingestão de dados em lote para permitir que grandes varreduras sejam executadas sem sobrecarregar o banco de dados de produção primário. Além disso, um aplicativo pode ser configurado para fazer failover para a réplica se o banco de dados primário ficar indisponível. Nenhum dado será perdido no failover porque a réplica está totalmente sincronizada com o banco de dados primário.

A vantagem da replicação assíncrona do CDC é um padrão de arquitetura fracamente acoplado. Embora a réplica possa estar um pouco atrasada em relação ao banco de dados primário, isso geralmente não é um problema para aplicativos de análise, e os eventos agora podem ser direcionados para vários destinos; podemos executar a replicação do CDC ao mesmo tempo em que direcionamos eventos para o armazenamento de objetos e um processador de análise de streaming.

Considerações do CDC

Como tudo em tecnologia, o CDC não é gratuito. O CDC consome vários recursos de banco de dados, como memória, largura de banda do disco, armazenamento, tempo de CPU e largura de banda da rede. Os engenheiros devem trabalhar com equipes de produção e executar testes antes de ativar o CDC em sistemas de produção para evitar problemas operacionais. Considerações semelhantes se aplicam à replicação síncrona.

Para CDC em lote, esteja ciente de que executar qualquer consulta em lote grande em um sistema de produção transacional pode causar carga excessiva. Execute essas consultas apenas fora do horário comercial ou use uma réplica de leitura para evitar sobrecarregar o banco de dados primário.

Machine Translated by Google

API

A maior parte da engenharia de software é apenas encanamento.

—Karl Hughes

Como mencionamos no Capítulo 5, as APIs são uma fonte de dados que continua crescendo em importância e popularidade. Uma organização típica pode ter centenas de fontes de dados externas, como plataformas SaaS ou empresas parceiras. A dura realidade é que não existe um padrão adequado para troca de dados por APIs. Os engenheiros de dados podem gastar uma quantidade significativa de tempo lendo documentação, comunicando-se com proprietários de dados externos e escrevendo e mantendo o código de conexão da API.

Três tendências estão mudando lentamente essa situação. Primeiro, muitos fornecedores fornecem bibliotecas cliente de API para várias linguagens de programação que removem grande parte da complexidade do acesso à API.

Em segundo lugar, várias plataformas de conector de dados estão disponíveis agora como SaaS, código aberto ou código aberto gerenciado. Essas plataformas fornecem conectividade de dados pronta para uso a muitas fontes de dados; eles oferecem estruturas para escrever conectores personalizados para fontes de dados sem suporte. Consulte “Conectores de Dados Gerenciados”.

A terceira tendência é o surgimento do compartilhamento de dados (discutido no Capítulo 5), ou seja, a capacidade de trocar dados por meio de uma plataforma padrão, como BigQuery, Snowflake, Redshift ou S3. Quando os dados chegam a uma dessas plataformas, é fácil armazená-los, processá-los ou movê-los para outro lugar. O compartilhamento de dados teve um grande e rápido impacto na engenharia de dados
espaço.

Não reinvente a roda quando o compartilhamento de dados não for uma opção e o acesso direto à API for necessário. Embora um serviço gerenciado possa parecer uma opção cara, considere o valor do seu tempo e o custo de oportunidade de criar conectores de API quando você pode gastar seu tempo em trabalhos de maior valor.

Além disso, muitos serviços gerenciados agora oferecem suporte à criação de conectores de API personalizados. Isso pode fornecer especificações técnicas de API em um padrão

Machine Translated by Google

formatar ou escrever o código do conector que é executado em uma estrutura de função sem servidor (por exemplo, AWS Lambda) enquanto permite que o serviço gerenciado lide com os detalhes de agendamento e sincronização. Novamente, esses serviços podem economizar muito tempo para os engenheiros, tanto para desenvolvimento quanto para manutenção contínua.

Reserve seu trabalho de conexão personalizado para APIs que não são bem suportadas por estruturas existentes; você descobrirá que ainda há muitos deles para trabalhar. O manuseio de conexões de API personalizadas tem dois aspectos principais: desenvolvimento de software e operações. Seguir as melhores práticas de desenvolvimento de software; você deve usar controle de versão, entrega contínua e testes automatizados. Além de seguir as melhores práticas de DevOps, considere uma estrutura de orquestração, que pode simplificar drasticamente a carga operacional da ingestão de dados.

Filas de mensagens e plataformas de streaming de eventos

As filas de mensagens e as plataformas de streaming de eventos são formas generalizadas de ingerir dados em tempo real de aplicativos móveis e da Web, sensores de IoT e dispositivos inteligentes. À medida que os dados em tempo real se tornam mais onipresentes, muitas vezes você se verá introduzindo ou adaptando maneiras de lidar com dados em tempo real em seus fluxos de trabalho de ingestão. Como tal, é essencial saber como ingerir dados em tempo real. A ingestão de dados em tempo real popular inclui filas de mensagens ou plataformas de fluxo de eventos, que abordamos no Capítulo 5. Embora ambos sejam sistemas de origem, eles também atuam como formas de ingerir dados. Em ambos os casos, você consome eventos do editor ao qual se inscreveu.

Lembre-se das diferenças entre mensagens e fluxos. Uma ***mensagem*** é tratada no nível de evento individual e deve ser transitória. Depois que uma mensagem é consumida, ela é reconhecida e removida da fila. Por outro lado, um ***fluxo*** ingere eventos em um log ordenado. O log persiste pelo tempo que você desejar, permitindo que os eventos sejam consultados em vários intervalos, agregados e combinados com outros fluxos para criar novas transformações publicadas para consumidores downstream. Na Figura 7-14, temos dois produtores (produtores 1 e 2) enviando eventos para dois consumidores (consumidores

Machine Translated by Google

1 e 2). Esses eventos são combinados em um novo conjunto de dados e enviados a um produtor para consumo posterior.

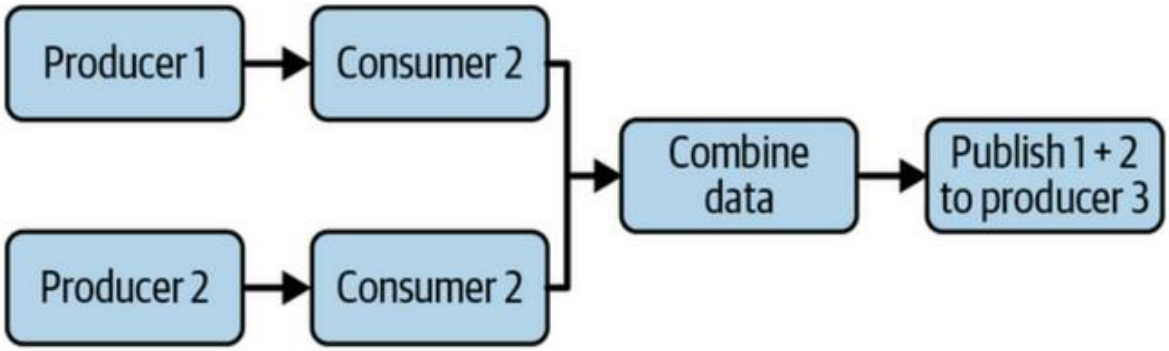


Figura 7-14. Dois conjuntos de dados são produzidos e consumidos (produtores 1 e 2) e, em seguida, combinados, com os dados combinados publicados para um novo produtor (produtor 3)

O último ponto é uma diferença essencial entre ingestão em lote e streaming. Enquanto o lote geralmente envolve fluxos de trabalho estáticos (ingestão de dados, armazená-los, transformá-los e servi-los), as mensagens e os fluxos são fluidos. A ingestão pode ser não linear, com dados sendo publicados, consumidos, republicados e consumidos novamente. Ao projetar seus fluxos de trabalho de ingestão em tempo real, lembre-se de como os dados fluirão.

Outra consideração é a taxa de transferência de seus pipelines de dados em tempo real. Mensagens e eventos devem fluir com a menor latência possível, o que significa que você deve provisionar largura de banda e taxa de transferência de partição (ou fragmento) adequados. Forneça recursos suficientes de memória, disco e CPU para processamento de eventos e, se estiver gerenciando seus pipelines em tempo real, incorpore o dimensionamento automático para lidar com picos e economizar dinheiro à medida que a carga diminui. Por esses motivos, gerenciar sua plataforma de streaming pode gerar uma sobrecarga significativa. Considere os serviços gerenciados para seus pipelines de ingestão em tempo real e concentre sua atenção em maneiras de obter valor de seus dados em tempo real.

Conectores de dados gerenciados

Atualmente, se você estiver pensando em escrever um conector de ingestão de dados em um banco de dados ou API, pergunte a si mesmo: isso já foi criado? Além disso, existe um serviço que irá gerenciar os detalhes dessa conexão para mim? “APIs” menciona a popularidade do conector de dados gerenciados

Machine Translated by Google

plataformas e frameworks. Essas ferramentas visam fornecer um conjunto padrão de conectores prontos para uso para poupar os engenheiros de dados que constroem encanamentos complicados para se conectar a uma fonte específica. Em vez de criar e gerenciar um conector de dados, você terceiriza esse serviço para terceiros.

Geralmente, as opções no espaço permitem que os usuários definam um destino e uma origem, ingeram de várias maneiras (por exemplo, CDC, replicação, trunque e recarregue), definam permissões e credenciais, configurem uma frequência de atualização e comecem a sincronizar dados. O fornecedor ou a nuvem nos bastidores gerencia e monitora totalmente as sincronizações de dados. Se a sincronização de dados falhar, você receberá um alerta com informações registradas sobre a causa do erro.

Sugerimos o uso de plataformas de conectores gerenciados em vez de criar e gerenciar seus conectores. Fornecedores e projetos de OSS normalmente têm centenas de opções de conectores pré-construídos e podem criar conectores personalizados facilmente. A criação e gerenciamento de conectores de dados é um trabalho pesado indiferenciado nos dias de hoje e deve ser terceirizado sempre que possível.

Movendo dados com armazenamento de objetos

O armazenamento de objetos é um sistema multilocatário em nuvens públicas e suporta o armazenamento de grandes quantidades de dados. Isso torna o armazenamento de objetos ideal para mover dados dentro e fora de data lakes, entre equipes e transferir dados entre organizações. Você pode até mesmo fornecer acesso de curto prazo a um objeto com uma URL assinada, dando permissão temporária ao usuário.

Em nossa opinião, o armazenamento de objetos é a maneira mais otimizada e segura de lidar com a troca de arquivos. O armazenamento em nuvem pública implementa os mais recentes padrões de segurança, possui um histórico robusto de escalabilidade e confiabilidade, aceita arquivos de tipos e tamanhos arbitrários e fornece movimentação de dados de alto desempenho. Discutimos o armazenamento de objetos muito mais extensivamente no Capítulo 6.

EDI

Outra realidade prática para engenheiros de dados é ***o intercâmbio eletrônico de dados*** (EDI). O termo é vago o suficiente para se referir a qualquer método de movimentação de dados. Isto

Machine Translated by Google

geralmente se refere a meios um tanto arcaicos de troca de arquivos, como por e-mail ou pen drive. Os engenheiros de dados descobrirão que algumas fontes de dados não suportam meios mais modernos de transporte de dados, geralmente devido a sistemas de TI arcaicos ou limitações de processos humanos.

Os engenheiros podem pelo menos aprimorar o EDI por meio da automação. Por exemplo, eles podem configurar um servidor de e-mail baseado em nuvem que salva arquivos no armazenamento de objetos da empresa assim que são recebidos. Isso pode acionar processos de orquestração para ingerir e processar dados. Isso é muito mais robusto do que um funcionário baixar o arquivo anexado e carregá-lo manualmente em um sistema interno, o que ainda vemos com frequência.

Bancos de dados e exportação de arquivos

Os engenheiros devem estar cientes de como os sistemas de banco de dados de origem lidam com a exportação de arquivos. A exportação envolve grandes varreduras de dados que carregam significativamente o banco de dados para muitos sistemas transacionais. Os engenheiros do sistema de origem devem avaliar quando essas verificações podem ser executadas sem afetar o desempenho do aplicativo e podem optar por uma estratégia para mitigar a carga. As consultas de exportação podem ser divididas em exportações menores consultando intervalos de chaves ou uma partição por vez. Como alternativa, uma réplica de leitura pode reduzir a carga. As réplicas de leitura são especialmente apropriadas se as exportações ocorrerem muitas vezes ao dia e coincidirem com uma alta carga do sistema de origem.

Os principais data warehouses em nuvem são altamente otimizados para exportação direta de arquivos. Por exemplo, Snowflake, BigQuery, Redshift e outros são compatíveis com exportação direta para armazenamento de objetos em vários formatos.

Problemas práticos com formatos de arquivo comuns

Os engenheiros também devem estar cientes dos formatos de arquivo a serem exportados. O CSV ainda é onipresente e altamente propenso a erros no momento da redação deste artigo. Ou seja, o delimitador padrão do CSV também é um dos caracteres mais familiares no idioma inglês - a vírgula! Mas fica pior.

O CSV não é de forma alguma um formato uniforme. Os engenheiros devem estipular o delimitador, os caracteres de aspas e o escape para lidar adequadamente com a exportação

Machine Translated by Google

de dados de cadeia. O CSV também não codifica nativamente informações de esquema nem dá suporte direto a estruturas aninhadas. A codificação do arquivo CSV e as informações de esquema devem ser configuradas no sistema de destino para garantir a ingestão apropriada. A detecção automática é um recurso de conveniência fornecido em muitos ambientes de nuvem, mas é inadequado para ingestão de produção. Como prática recomendada, os engenheiros devem registrar a codificação CSV e os detalhes do esquema nos metadados do arquivo.

Formatos de exportação mais robustos e expressivos incluem Parquet, Avro, Flecha, e ORC ou JSON. Esses formatos codificam nativamente informações de esquema e manipulam dados de string arbitrários sem nenhuma intervenção específica. Muitos deles também lidam com estruturas de dados aninhadas nativamente para que os campos JSON sejam armazenados usando estruturas aninhadas internas em vez de strings simples. Para bancos de dados colunares, os formatos colunares (Parquet, Arrow, ORC) permitem uma exportação de dados mais eficiente porque as colunas podem ser transcodificadas diretamente entre os formatos.
Esses formatos também são geralmente mais otimizados para mecanismos de consulta. O formato de arquivo Arrow foi projetado para mapear dados diretamente na memória do mecanismo de processamento, proporcionando alto desempenho em ambientes de data lake.

A desvantagem desses formatos mais recentes é que muitos deles não são suportados nativamente pelos sistemas de origem. Os engenheiros de dados geralmente são forçados a trabalhar com dados CSV e, em seguida, criar um tratamento robusto de exceções e detecção de erros para garantir a qualidade dos dados na ingestão. Consulte o Apêndice A para uma discussão mais ampla sobre os formatos de arquivo.

Casca

O ***shell*** é uma interface pela qual você pode executar comandos para ingerir dados. O shell pode ser usado para scripts de fluxos de trabalho para praticamente qualquer ferramenta de software, e o shell script ainda é usado extensivamente em processos de ingestão. Um script de shell pode ler dados de um banco de dados, reserializá-los em um formato de arquivo diferente, carregá-los no armazenamento de objetos e acionar um processo de ingestão em um banco de dados de destino. Embora o armazenamento de dados em uma única instância ou servidor não seja altamente escalável, muitas de nossas fontes de dados não são particularmente grandes e essas abordagens funcionam bem.

Machine Translated by Google

Além disso, os fornecedores de nuvem geralmente fornecem ferramentas robustas baseadas em CLI. É possível executar processos de ingestão complexos simplesmente emitindo comandos para a AWS CLI. À medida que os processos de ingestão se tornam mais complicados e o SLA se torna mais rigoroso, os engenheiros devem considerar a mudança para um sistema de orquestração adequado.

SSH

O SSH não é uma estratégia de ingestão, mas um protocolo usado com outras estratégias de ingestão. Usamos SSH de algumas maneiras. Primeiro, o SSH pode ser usado para transferência de arquivos com SCP, conforme mencionado anteriormente. Em segundo lugar, os túneis SSH são usados para permitir conexões seguras e isoladas com bancos de dados.

Os bancos de dados de aplicativos nunca devem ser expostos diretamente na Internet. Em vez disso, os engenheiros podem configurar um bastion host, ou seja, uma instância de host intermediária que pode se conectar ao banco de dados em questão. Essa máquina host está exposta na Internet, embora bloqueada para acesso mínimo apenas de endereços IP especificados a portas especificadas. Para se conectar ao banco de dados, uma máquina remota primeiro abre uma conexão de túnel SSH com o bastion host e, em seguida, conecta-se da máquina host ao banco de dados.

SFTP e SCP

Acessar e enviar dados de FTP seguro (SFTP) e cópia segura (SCP) são técnicas com as quais você deve estar familiarizado, mesmo que os engenheiros de dados normalmente não as usem regularmente (TI ou segurança/secOps cuidarão disso).

Os engenheiros se assustam com a menção do SFTP (ocasionalmente, até ouvimos casos de FTP sendo usados na produção). Independentemente disso, o SFTP ainda é uma realidade prática para muitas empresas. Eles trabalham com empresas parceiras que consomem ou fornecem dados usando SFTP e não estão dispostos a confiar em outros padrões. Para evitar vazamentos de dados, a análise de segurança é fundamental nessas situações.

SCP é um protocolo de troca de arquivos que é executado em uma conexão SSH relacionada ao SSH. O SCP pode ser uma opção segura de transferência de arquivos se estiver configurado corretamente.

Machine Translated by Google

Novamente, é altamente recomendável adicionar controle de acesso à rede adicional (defesa em profundidade) para aprimorar a segurança do SCP.

Webhooks

Webhooks, como discutimos no Capítulo 5, são frequentemente chamados de ***APIs reversas.*** Para uma API de dados REST típica, o provedor de dados fornece especificações de API aos engenheiros que eles usam para escrever seu código de ingestão de dados. O código faz solicitações e recebe dados em respostas.

Com um webhook (Figura 7-15), o provedor de dados define uma especificação de solicitação de API, mas o provedor de dados ***faz chamadas de API*** em vez de recebê-las; é responsabilidade do consumidor de dados fornecer um endpoint de API para o provedor chamar. O consumidor é responsável por ingerir cada solicitação e manipular a agregação, armazenamento e processamento de dados.

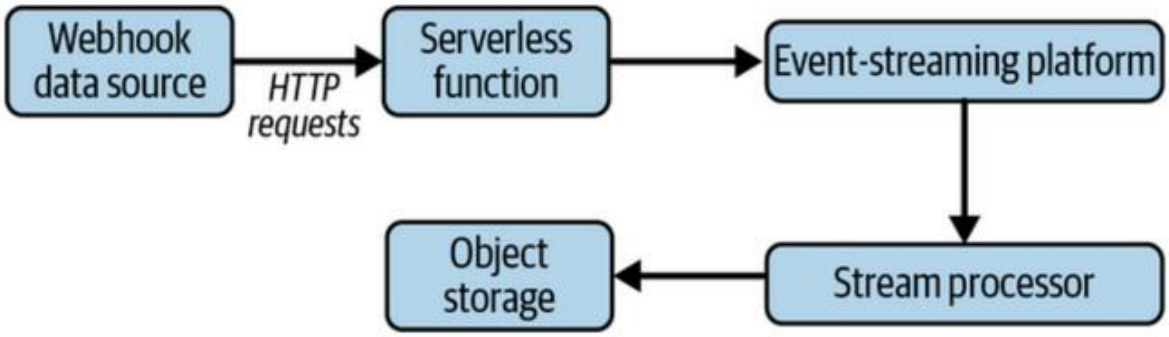


Figura 7-15. Uma arquitetura básica de ingestão de webhook criada a partir de serviços em nuvem

As arquiteturas de ingestão de dados baseadas em webhook podem ser frágeis, difíceis de manter e ineficientes. Usando ferramentas apropriadas prontas para uso, os engenheiros de dados podem criar arquiteturas de webhook mais robustas com menores custos de manutenção e infraestrutura. Por exemplo, um padrão de webhook na AWS pode usar uma estrutura de função sem servidor (Lambda) para receber eventos de entrada, uma plataforma gerenciada de streaming de eventos para armazenar e armazenar mensagens em buffer (Kinesis), uma estrutura de processamento de fluxo para lidar com análises em tempo real (Flink) e um armazenamento de objetos para armazenamento de longo prazo (S3).

Você notará que essa arquitetura faz muito mais do que simplesmente ingerir os dados. Isso ressalta o emaranhamento da ingestão com os outros estágios do

Machine Translated by Google

ciclo de vida da engenharia de dados; muitas vezes é impossível definir sua arquitetura de ingestão sem tomar decisões sobre armazenamento e processamento.

Interface web

As interfaces da Web para acesso a dados continuam sendo uma realidade prática para engenheiros de dados. Frequentemente nos deparamos com situações em que nem todos os dados e funcionalidades em uma plataforma SaaS são expostos por meio de interfaces automatizadas, como APIs e descartes de arquivos. Em vez disso, alguém deve acessar manualmente uma interface da Web, gerar um relatório e baixar um arquivo para uma máquina local. Isso tem desvantagens óbvias, como pessoas que se esquecem de executar o relatório ou que seus laptops morrem. Sempre que possível, escolha ferramentas e fluxos de trabalho que permitam o acesso automatizado aos dados.

Raspagem da web

A raspagem da Web extrai automaticamente os dados das páginas da Web, geralmente combinando os vários elementos HTML da página da Web. Você pode raspar sites de comércio eletrônico para extrair informações de preços de produtos ou raspar vários sites de notícias para seu agregador de notícias. A raspagem da Web é generalizada e você pode encontrá-la como engenheiro de dados. É também uma área obscura onde as linhas éticas e legais são borradas.

Aqui estão alguns conselhos de alto nível a serem observados antes de iniciar qualquer projeto de raspagem na web. Primeiro, pergunte a si mesmo se você deveria estar raspando na web ou se os dados estão disponíveis de terceiros. Se sua decisão é raspar na web, seja um bom cidadão. Não crie inadvertidamente um ataque de negação de serviço (DoS) e não bloqueie seu endereço IP. Entenda a quantidade de tráfego que você gera e regule adequadamente suas atividades de rastreamento na Web. Só porque você pode criar milhares de funções Lambda simultâneas para raspar não significa que você deveria; a extração excessiva da web pode levar à desativação de sua conta da AWS.

Em segundo lugar, esteja ciente das implicações legais de suas atividades. Novamente, gerar ataques DoS pode acarretar consequências legais. Ações que violam os termos de serviço podem causar dores de cabeça para seu empregador ou para você pessoalmente.

Machine Translated by Google

Terceiro, as páginas da web mudam constantemente sua estrutura de elementos HTML, tornando difícil manter seu web scraper atualizado. Pergunte a si mesmo, a dor de cabeça de manter esses sistemas vale o esforço?

A raspagem da Web tem implicações interessantes para o estágio de processamento do ciclo de vida da engenharia de dados; engenheiros devem pensar em vários fatores no início de um projeto de web-scraping. O que você pretende fazer com os dados? Você está apenas puxando os campos obrigatórios do HTML raspado usando o código Python e, em seguida, gravando esses valores em um banco de dados? Você pretende manter o código HTML completo dos sites raspados e processar esses dados usando uma estrutura como o Spark? Essas decisões podem levar a arquiteturas muito diferentes a jusante da ingestão.

Transfer Appliances para migração de dados

Para dados massivos (100 TB ou mais), a transferência de dados diretamente pela Internet pode ser um processo lento e caro. Nessa escala, a maneira mais rápida e eficiente de mover dados não é por fio, mas por caminhão. Os fornecedores de nuvem oferecem a capacidade de enviar seus dados por meio de uma “caixa de discos rígidos” física.
Basta solicitar um dispositivo de armazenamento, chamado de ***dispositivo de transferência,*** carregar seus dados de seus servidores e enviá-los de volta ao fornecedor da nuvem, que fará o upload de seus dados.

A sugestão é considerar o uso de um dispositivo de transferência se o tamanho dos dados estiver em torno de 100 TB. No extremo, a AWS ainda oferece Snowmobile, um dispositivo de transferência enviado a você em um semirreboque! O Snowmobile destina-se a elevar e deslocar um data center inteiro, no qual os tamanhos dos dados estão na casa dos petabytes ou mais.

Os dispositivos de transferência são úteis para criar configurações de nuvem híbrida ou multicloud. Por exemplo, o dispositivo de transferência de dados da Amazon (AWS Snowball) oferece suporte à importação e exportação. Para migrar para uma segunda nuvem, os usuários podem exportar seus dados para um dispositivo Snowball e importá-los para um segundo dispositivo de transferência para mover dados para o GCP ou Azure. Isso pode parecer estranho, mas mesmo quando for viável enviar dados pela Internet entre

Machine Translated by Google

nuvens, as taxas de saída de dados tornam isso uma proposta cara. Os dispositivos de transferência física são uma alternativa mais barata quando os volumes de dados são significativos.

Lembre-se de que os dispositivos de transferência e os serviços de migração de dados são eventos de ingestão de dados únicos e não são sugeridos para cargas de trabalho contínuas. Suponha que você tenha cargas de trabalho que exigem movimentação constante de dados em um cenário híbrido ou multicloud. Nesse caso, seus tamanhos de dados provavelmente estão agrupando ou transmitindo tamanhos de dados muito menores continuamente.

Compartilhamento de dados

O compartilhamento de dados está crescendo como uma opção popular para o consumo de dados (consulte os Capítulos 5 e 6.) Os provedores de dados oferecerão conjuntos de dados para assinantes de terceiros, gratuitamente ou a um custo. Esses conjuntos de dados geralmente são compartilhados de forma somente leitura, o que significa que você pode integrar esses conjuntos de dados com seus próprios dados (e outros conjuntos de dados de terceiros), mas você não possui o conjunto de dados compartilhado. No sentido estrito, isso não é ingestão, onde você obtém a posse física do conjunto de dados. Se o provedor de dados decidir remover seu acesso a um conjunto de dados, você não terá mais acesso a ele.

Muitas plataformas de nuvem oferecem compartilhamento de dados, permitindo que você compartilhe seus dados e consuma dados de vários provedores. Algumas dessas plataformas também fornecem marketplaces de dados onde empresas e organizações podem oferecer seus dados para venda.

Com quem você vai trabalhar

A ingestão de dados fica em vários limites organizacionais. No desenvolvimento e gerenciamento de pipelines de ingestão de dados, os engenheiros de dados trabalharão com pessoas e sistemas localizados a montante (produtores de dados) e a jusante (consumidores de dados).

Partes interessadas a montante

Muitas vezes existe uma desconexão significativa entre os responsáveis pela ***geração*** de dados – normalmente, engenheiros de software – e os engenheiros de dados

Machine Translated by Google

quem preparará esses dados para análise e ciência de dados. Engenheiros de software e engenheiros de dados geralmente ficam em silos organizacionais separados; se eles pensam em engenheiros de dados, eles normalmente os veem simplesmente como consumidores downstream dos dados exaustos de seu aplicativo, não como partes interessadas.

Vemos essa situação atual como um problema e uma oportunidade significativa. Os engenheiros de dados podem melhorar a qualidade de seus dados convidando engenheiros de software para serem partes interessadas nos resultados da engenharia de dados. A grande maioria dos engenheiros de software está bem ciente do valor da análise e da ciência de dados, mas não necessariamente tem incentivos alinhados para contribuir diretamente com os esforços de engenharia de dados.

Simplesmente melhorar a comunicação é um primeiro passo significativo. Muitas vezes, os engenheiros de software já identificaram dados potencialmente valiosos para consumo posterior. A abertura de um canal de comunicação incentiva os engenheiros de software a colocar os dados em forma para os consumidores e comunicar sobre as alterações de dados para evitar regressões de pipeline.

Além da comunicação, os engenheiros de dados podem destacar as contribuições dos engenheiros de software para os membros da equipe, executivos e especialmente gerentes de produto. Envolver os gerentes de produto no resultado e tratar os dados downstream processados como parte de um produto os incentiva a alocar o desenvolvimento de software escasso para a colaboração com engenheiros de dados. Idealmente, os engenheiros de software podem trabalhar parcialmente como extensões da equipe de engenharia de dados; isso permite que eles colaborem em vários projetos, como a criação de uma arquitetura orientada a eventos para permitir análises em tempo real.

Partes interessadas a jusante

Quem é o cliente final para ingestão de dados? Os engenheiros de dados se concentram em profissionais de dados e líderes de tecnologia, como cientistas de dados, analistas e diretores técnicos. Eles também fariam bem em lembrar seu círculo mais amplo de partes interessadas nos negócios, como diretores de marketing, vice-presidentes da cadeia de suprimentos e CEOs.

Machine Translated by Google

Muitas vezes, vemos engenheiros de dados buscando projetos sofisticados (por exemplo, barramentos de streaming em tempo real ou sistemas de dados complexos), enquanto os gerentes de marketing digital ao lado ficam baixando relatórios do Google Ads manualmente. Veja a engenharia de dados como um negócio e reconheça quem são seus clientes. Muitas vezes, a automação básica dos processos de ingestão tem um valor significativo, especialmente para departamentos como marketing, que controlam orçamentos enormes e estão no centro da receita dos negócios. O trabalho básico de ingestão pode parecer tedioso, mas entregar valor a essas partes principais da empresa abrirá mais orçamento e oportunidades de engenharia de dados de longo prazo mais empolgantes.

Os engenheiros de dados também podem convidar mais executivos a participar desse processo colaborativo. Por uma boa razão, a cultura orientada a dados está na moda nos círculos de liderança empresarial. Ainda assim, cabe aos engenheiros de dados e outros profissionais de dados fornecer aos executivos orientação sobre a melhor estrutura para um negócio orientado a dados. Isso significa comunicar o valor de reduzir as barreiras entre produtores de dados e engenheiros de dados, ao mesmo tempo em que apoia os executivos na quebra de silos e na criação de incentivos para levar a uma cultura orientada por dados mais unificada.

Mais uma vez, ***comunicação*** é a palavra de ordem. A comunicação honesta desde o início e com frequência com as partes interessadas ajudará bastante a garantir que sua ingestão de dados agregue valor.

Subcorrentes

Praticamente todas as subcorrentes tocam a fase de ingestão, mas vamos enfatizar as mais salientes aqui.

Segurança

A movimentação de dados apresenta vulnerabilidades de segurança porque você precisa transferir dados entre locais. A última coisa que você deseja é capturar ou comprometer os dados durante a movimentação.

Machine Translated by Google

Considere onde os dados estão e para onde estão indo. Os dados que precisam ser movidos em sua VPC devem usar endpoints seguros e nunca sair dos limites da VPC. Use uma VPN ou uma conexão privada dedicada se precisar enviar dados entre a nuvem e uma rede local. Isso pode custar dinheiro, mas a segurança é um bom investimento. Se seus dados atravessarem a internet pública, certifique-se de que a transmissão seja criptografada. É sempre uma boa prática criptografar dados pela rede.

Gestão de dados

Naturalmente, o gerenciamento de dados começa na ingestão de dados. Este é o ponto de partida para catalogação de linhagem e dados; a partir deste ponto, os engenheiros de dados precisam pensar no gerenciamento de dados mestre, ética, privacidade e conformidade.

Alterações de esquema

Alterações de esquema (como adicionar, alterar ou remover colunas em uma tabela de banco de dados) permanecem, do nosso ponto de vista, um problema não resolvido no gerenciamento de dados. A abordagem tradicional é um processo cuidadoso de revisão de comando e controle. Trabalhando com clientes em grandes empresas, foram cotados prazos de entrega de seis meses para a adição de um único campo. Este é um impedimento inaceitável para a agilidade.

Na extremidade oposta do espectro, qualquer alteração de esquema na origem aciona as tabelas de destino a serem recriadas com o novo esquema. Isso resolve problemas de esquema no estágio de ingestão, mas ainda pode interromper pipelines downstream e sistemas de armazenamento de destino.

Uma solução possível, sobre a qual nós, os autores, meditamos por um tempo, é uma abordagem pioneira no controle de versão do Git. Quando Linus Torvalds estava desenvolvendo o Git, muitas de suas escolhas foram inspiradas nas limitações do Concurrent Versions System (CVS). O CVS é totalmente centralizado; ele suporta apenas uma versão oficial atual do código, armazenada em um servidor de projeto central. Para tornar o Git um sistema verdadeiramente distribuído, Torvalds

Machine Translated by Google

usou a noção de árvore; cada desenvolvedor pode manter sua ramificação processada do código e, em seguida, mesclar de ou para outras ramificações.

Há alguns anos, tal abordagem aos dados era impensável. Os sistemas MPP locais são normalmente operados perto da capacidade máxima de armazenamento.
No entanto, o armazenamento é barato em ambientes de big data e data warehouse em nuvem. Pode-se facilmente manter várias versões de uma tabela com diferentes esquemas e até mesmo diferentes transformações upstream. As equipes podem oferecer suporte a várias versões de "desenvolvimento" de uma tabela usando ferramentas de orquestração, como Airflow; alterações de esquema, transformação upstream e alterações de código podem aparecer nas tabelas de desenvolvimento antes das alterações oficiais na tabela ***principal .***

Ética de dados, privacidade e conformidade Os

clientes geralmente pedem nossos conselhos sobre criptografar dados confidenciais em bancos de dados, o que geralmente nos leva a fazer uma pergunta fundamental: você precisa dos dados confidenciais que está tentando criptografar? Como se vê, essa questão geralmente é negligenciada ao criar requisitos e resolver problemas.

Os engenheiros de dados devem sempre se treinar para fazer essa pergunta ao configurar pipelines de ingestão. Eles inevitavelmente encontrarão dados confidenciais; a tendência natural é ingeri-lo e encaminhá-lo para a próxima etapa do pipeline. Mas se esses dados não são necessários, por que coletá-los? Por que não simplesmente descartar os campos confidenciais antes que os dados sejam armazenados? Os dados não podem vazar se nunca forem coletados.

Onde for realmente necessário acompanhar identidades confidenciais, é prática comum aplicar tokenização para anonimizar identidades no treinamento e análise de modelos. Mas os engenheiros devem observar onde essa tokenização é usada. Se possível, faça o hash dos dados no momento da ingestão.

Os engenheiros de dados não podem evitar trabalhar com dados altamente confidenciais em alguns casos. Alguns sistemas analíticos devem apresentar informações confidenciais e identificáveis. Os engenheiros devem agir de acordo com os mais altos padrões éticos sempre que lidarem com dados confidenciais. Além disso, eles podem implementar uma variedade de práticas para reduzir o manuseio direto de dados confidenciais. Apontar como

Machine Translated by Google

tanto quanto possível para a ***produção sem toque*** onde estão envolvidos dados sensíveis. Isso significa que os engenheiros desenvolvem e testam código em dados simulados ou limpos em ambientes de desenvolvimento e preparação, mas implantações de código automatizadas para produção.

A produção sem toque é um ideal pelo qual os engenheiros devem lutar, mas inevitavelmente surgem situações que não podem ser totalmente resolvidas em ambientes de desenvolvimento e preparação. Alguns bugs podem não ser reproduzíveis sem olhar para os dados ao vivo que estão acionando uma regressão. Para esses casos, implemente um processo de vidro quebrado: exija pelo menos duas pessoas para aprovar o acesso a dados confidenciais no ambiente de produção. Esse acesso deve ter um escopo restrito para um problema específico e ter uma data de expiração.

Nosso último conselho sobre dados confidenciais: desconfie de soluções tecnológicas ingênuas para problemas humanos. Tanto a criptografia quanto a tokenização são frequentemente tratadas como balas mágicas de privacidade. A maioria dos sistemas de armazenamento baseados em nuvem e quase todos os bancos de dados criptografam dados em repouso e em movimento por padrão. Geralmente, não vemos problemas de criptografia, mas sim problemas de acesso a dados. A solução é aplicar uma camada extra de criptografia a um único campo ou controlar o acesso a esse campo? Afinal, ainda é preciso gerenciar rigidamente o acesso à chave de criptografia. Existem casos de uso legítimos para criptografia de campo único, mas cuidado com a criptografia ritualística.

Na frente de tokenização, use o bom senso e avalie os cenários de acesso a dados. Se alguém tivesse o e-mail de um de seus clientes, eles poderiam facilmente fazer o hash do e-mail e encontrar o cliente em seus dados? O hashing de dados sem pensar, sem sal e outras estratégias, pode não proteger a privacidade tão bem quanto você pensa.

DataOps

Pipelines de dados confiáveis são a base do ciclo de vida da engenharia de dados. Quando eles falham, todas as dependências downstream param bruscamente. Data warehouses e data lakes não são reabastecidos com dados atualizados, e os cientistas e analistas de dados não podem fazer seu trabalho com eficiência; o negócio é forçado a voar às cegas.

Machine Translated by Google

Garantir que seus pipelines de dados sejam monitorados adequadamente é uma etapa crucial para a confiabilidade e a resposta eficaz a incidentes. Se há um estágio no ciclo de vida da engenharia de dados em que o monitoramento é crítico, é no estágio de ingestão. Monitoramento fraco ou inexistente significa que as tubulações podem ou não estar funcionando. Voltando à nossa discussão anterior sobre o tempo, certifique-se de acompanhar os vários aspectos do tempo – criação de eventos, ingestão, processo e tempos de processamento. Seus pipelines de dados devem processar dados de forma previsível em lotes ou fluxos. Vimos inúmeros exemplos de relatórios e modelos de ML gerados a partir de dados obsoletos. Em um caso extremo, uma falha no pipeline de ingestão não foi detectada por seis meses. (Pode-se questionar a utilidade concreta dos dados neste caso, mas isso é outra questão.) Isso era muito evitável por meio de monitoramento adequado.

O que você deve monitorar? O tempo de atividade, a latência e os volumes de dados processados são bons pontos de partida. Se um trabalho de ingestão falhar, como você responderá? Em geral, crie monitoramento em seus pipelines desde o início, em vez de aguardar a implantação.

O monitoramento é fundamental e o conhecimento do comportamento dos sistemas upstream dos quais você depende e como eles geram dados. Você deve estar ciente do número de eventos gerados por intervalo de tempo com o qual está preocupado (eventos/minuto, eventos/segundo e assim por diante) e o tamanho médio de cada evento. Seu pipeline de dados deve lidar com a frequência e o tamanho dos eventos que você está ingerindo.

Isso também se aplica a serviços de terceiros. No caso desses serviços, o que você ganhou em termos de eficiência operacional enxuta (redução do número de funcionários) é substituído por sistemas dos quais você depende de estar fora de seu controle. Se você estiver usando um serviço de terceiros (nuvem, serviço de integração de dados etc.), como você será alertado se houver uma interrupção? Qual é o seu plano de resposta se um serviço do qual você depende de repente ficar offline?

Infelizmente, não existe um plano de resposta universal para falhas de terceiros. Se você puder fazer failover para outros servidores, de preferência em outra zona ou região, configure isso definitivamente.

Machine Translated by Google

Se seus processos de ingestão de dados são criados internamente, você tem a automação adequada de teste e implantação para garantir que o código funcione em produção? E se o código estiver com bugs ou falhar, você pode reverter para uma versão funcional?

Testes de qualidade de dados

Muitas vezes nos referimos aos dados como um assassino silencioso. Se dados válidos e de qualidade são a base do sucesso nos negócios de hoje, usar dados ruins para tomar decisões é muito pior do que não ter dados. Dados incorretos causaram danos incalculáveis às empresas; esses desastres de dados às vezes são chamados ***de datastrophes.***
1

Os dados são entrópicos; muitas vezes muda de maneiras inesperadas sem aviso prévio. Uma das diferenças inerentes entre DevOps e DataOps é que esperamos regressões de software apenas quando implantamos alterações, enquanto os dados geralmente apresentam regressões de forma independente devido a eventos fora de nosso controle.

Os engenheiros de DevOps normalmente são capazes de detectar problemas usando condições binárias. A taxa de falha de solicitação violou um determinado limite? Como sobre a latência de resposta? No espaço de dados, as regressões geralmente se manifestam como distorções estatísticas sutis. Uma mudança nas estatísticas dos termos de pesquisa é resultado do comportamento do cliente? De um pico no tráfego de bots que escapou da rede? De uma ferramenta de teste de site implantada em alguma outra parte da empresa?

Assim como as falhas do sistema no DevOps, algumas regressões de dados são imediatamente visíveis. Por exemplo, no início dos anos 2000, o Google fornecia termos de pesquisa para sites quando os usuários chegavam da pesquisa. Em 2011, o Google começou a reter essas informações em alguns casos para proteger melhor a privacidade do usuário.
Os analistas rapidamente viram “não fornecido” borbulhando no topo de seus relatórios. 2

As regressões de dados verdadeiramente perigosas são silenciosas e podem vir de dentro ou de fora de uma empresa. Os desenvolvedores de aplicativos podem alterar o significado dos campos do banco de dados sem se comunicar adequadamente com as equipes de dados.
Alterações nos dados de fontes de terceiros podem passar despercebidas. Na melhor das hipóteses, os relatórios quebram de maneiras óbvias. Muitas vezes, as métricas de negócios são distorcidas sem o conhecimento dos tomadores de decisão.

Machine Translated by Google

Sempre que possível, trabalhe com engenheiros de software para corrigir problemas de qualidade de dados na origem. É surpreendente quantos problemas de qualidade de dados podem ser tratados respeitando as melhores práticas básicas de engenharia de software, como logs para capturar o histórico de alterações de dados, verificações (nulls, etc.) e tratamento de exceções (try, catch, etc.) .

As ferramentas tradicionais de teste de dados geralmente são construídas em lógica binária simples. Os nulos estão aparecendo em um campo não anulável? Os itens novos e inesperados estão aparecendo em uma coluna categórica? O teste de dados estatísticos é uma área nova, mas que provavelmente crescerá dramaticamente nos próximos cinco anos.

Orquestração

A ingestão geralmente fica no início de um gráfico de dados grande e complexo; como a ingestão é o primeiro estágio do ciclo de vida da engenharia de dados, os dados ingeridos fluirão para muitas outras etapas de processamento de dados e os dados de muitas fontes se misturarão de maneiras complexas. Como enfatizamos ao longo deste livro, a orquestração é um processo crucial para coordenar essas etapas.

As organizações em um estágio inicial de maturidade de dados podem optar por implantar processos de ingestão como tarefas cron agendadas simples. No entanto, é crucial reconhecer que essa abordagem é frágil e pode diminuir a velocidade de implantação e desenvolvimento de engenharia de dados.

À medida que a complexidade do pipeline de dados aumenta, é necessária uma verdadeira orquestração. Por verdadeira orquestração, queremos dizer um sistema capaz de agendar gráficos de tarefas completos em vez de tarefas individuais. Uma orquestração pode iniciar cada tarefa de ingestão no horário agendado apropriado. O processamento downstream e as etapas de transformação começam quando as tarefas de ingestão são concluídas. Mais a jusante, as etapas de processamento levam a etapas de processamento adicionais.

Engenharia de software

O estágio de ingestão do ciclo de vida da engenharia de dados é intensivo em engenharia. Esse estágio fica na borda do domínio da engenharia de dados e geralmente faz interface com sistemas externos, onde os engenheiros de software e dados precisam construir uma variedade de encanamentos personalizados.

Machine Translated by Google

Nos bastidores, a ingestão é incrivelmente complicada, geralmente com equipes operando estruturas de código aberto como Kafka ou Pulsar, ou algumas das maiores empresas de tecnologia executando suas próprias soluções de ingestão bifurcadas ou caseiras. Conforme discutido neste capítulo, os conectores de dados gerenciados simplificaram o processo de ingestão, como Fivetran, Matillion e Airbyte.
Os engenheiros de dados devem aproveitar as melhores ferramentas disponíveis - principalmente ferramentas e serviços gerenciados que fazem muito trabalho pesado para você - e desenvolver alta competência de desenvolvimento de software nas áreas em que isso é importante. Vale a pena usar processos adequados de controle de versão e revisão de código e implementar testes apropriados mesmo para qualquer código relacionado à ingestão.

Ao escrever software, seu código precisa ser desacoplado. Evite escrever sistemas monolíticos com fortes dependências nos sistemas de origem ou destino.

Conclusão

Em seu trabalho como engenheiro de dados, a ingestão provavelmente consumirá uma parte significativa de sua energia e esforço. No centro, a ingestão é o encanamento, conectando canos a outros canos, garantindo que os dados fluam de forma consistente e segura para seu destino. Às vezes, as minúcias da ingestão podem parecer tediosas, mas os aplicativos de dados empolgantes (por exemplo, análise e ML) não podem acontecer sem ela.

Como enfatizamos, também estamos no meio de uma mudança radical, passando de lote para pipelines de dados de streaming. Esta é uma oportunidade para os engenheiros de dados descobrirem aplicativos interessantes para streaming de dados, comunicá-los aos negócios e implantar novas tecnologias interessantes.

Recursos adicionais

- Os "pipelines de streaming" do Google Cloud página da Internet
- "Janela de instantâneo da Microsoft (Azure Stream Analytics)" documentação

Machine Translated by Google

"Conexões e modos de sincronização" da Airbyte página da Internet

1 Andy Petrella, "Datastrophes", ***Medium,*** 1º de março de 2021, ***https:// oreil.ly/ h6FRW.***

2 Danny Sullivan, "Dark Google: um ano desde que os termos de pesquisa foram 'não fornecidos'" ***MarTech,*** 19 de outubro de 2012, https://oreil.ly/Fp8ta

Machine Translated by Google

Capítulo 8. Consultas, Modelagem e Transformação

Até este ponto, os estágios do ciclo de vida da engenharia de dados consistiam principalmente em passar dados de um lugar para outro ou armazená-los. Neste capítulo, você aprenderá como tornar os dados úteis. Ao compreender as consultas, modelagem e transformações (consulte a Figura 8-1), você terá as ferramentas para transformar os ingredientes de dados brutos em algo consumível pelas partes interessadas a jusante.

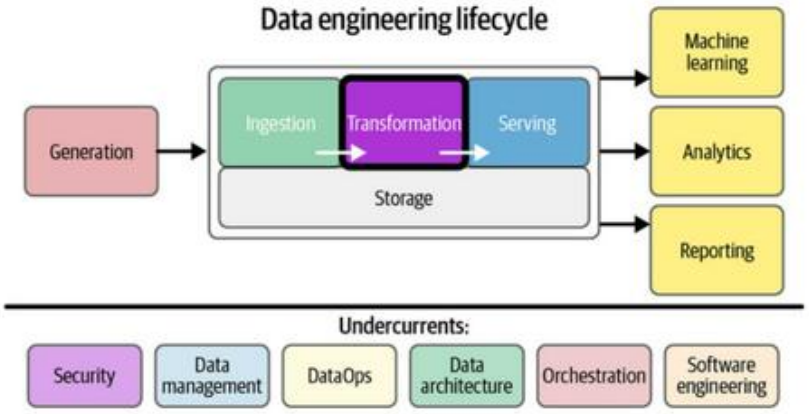


Figura 8-1. As transformações nos permitem criar valor a partir dos dados

Discutiremos primeiro as consultas e os padrões significativos subjacentes a elas. Em segundo lugar, veremos os principais padrões de modelagem de dados que você pode usar para introduzir a lógica de negócios em seus dados. Em seguida, abordaremos as transformações, que pegam a lógica de seus modelos de dados e os resultados das consultas e os tornam úteis para um consumo downstream mais direto. Por fim, abordaremos com quem você trabalhará e as tendências ocultas relacionadas a este capítulo.

Uma variedade de técnicas pode ser usada para consultar, modelar e transformar dados em bancos de dados SQL e NoSQL. Esta seção se concentra nas consultas feitas em um sistema OLAP, como um data warehouse ou data lake. Embora existam muitas linguagens para consulta, por conveniência e familiaridade, ao longo da maior parte deste capítulo, vamos nos concentrar fortemente no SQL, a linguagem de consulta mais popular e universal. A maioria dos conceitos para bancos de dados OLAP e SQL serão traduzidos para outros tipos de bancos de dados e linguagens de consulta. Este capítulo pressupõe que você entenda a linguagem SQL e conceitos relacionados, como chaves primárias e estrangeiras. Se essas ideias não são familiares para você, inúmeros recursos estão disponíveis para ajudá-lo a começar.

Uma nota sobre os termos usados neste capítulo. Por conveniência, usaremos o termo ***banco*** de dados como um atalho para um mecanismo de consulta e o armazenamento que ele está consultando; pode ser um data warehouse na nuvem ou o Apache Spark consultando dados armazenados no S3. Assumimos que o banco de dados possui um mecanismo de armazenamento que organiza os dados sob o capô. Isso se estende a consultas baseadas em arquivo (carregar um arquivo CSV em um bloco de anotações Python) e consultas em formatos de arquivo como Parquet.

Além disso, observe que este capítulo se concentra principalmente na consulta, nos padrões de modelagem e nas transformações relacionadas a dados estruturados e semiestruturados, que os engenheiros de dados usam com frequência. Muitas das práticas discutidas também podem ser aplicadas ao trabalho com dados não estruturados, como imagens, vídeo e texto bruto.

Antes de começarmos a modelar e transformar dados, vejamos as consultas — o que são, como funcionam, considerações para melhorar o desempenho da consulta e consultas sobre dados de streaming.

Consultas

Machine Translated by Google

As consultas são uma parte fundamental da engenharia de dados, ciência de dados e análise. Antes de aprender sobre os padrões e tecnologias subjacentes para transformações, você precisa entender o que são as consultas, como elas funcionam em vários dados e técnicas para melhorar o desempenho da consulta.

Esta seção se preocupa principalmente com consultas em dados tabulares e semiestruturados. Como engenheiro de dados, você consultará e transformará com mais frequência esses tipos de dados. Antes de entrarmos em tópicos mais complicados sobre consultas e transformações, vamos começar respondendo a uma pergunta bem simples: o que é uma consulta?

O que é uma consulta?

Muitas vezes encontramos pessoas que sabem escrever SQL, mas não estão familiarizadas com o funcionamento de uma consulta. Parte desse material introdutório sobre consultas será familiar para engenheiros de dados experientes; sinta-se à vontade para pular adiante se isso se aplicar a você.

Uma ***consulta*** permite que você recupere e atue nos dados. Lembre-se de nossa conversa no Capítulo 5 sobre CRUD. Quando uma consulta recupera dados, ela está emitindo uma solicitação para ler um padrão de registros. Este é o ***R*** (lido) no CRUD. Você pode emitir uma consulta que obtém todos os registros de uma tabela foo, como SELECT * FROM foo. Ou, você pode aplicar um predicado (condição lógica) para filtrar seus dados recuperando apenas registros onde o id é 1, usando a consulta SQL SELECT * DE foo ONDE id=1.

Muitos bancos de dados permitem que você crie, atualize e exclua dados. Estes são os ***CUD*** em CRUD; sua consulta criará, modificará ou destruirá registros existentes. Vamos revisar alguns outros acrônimos comuns que você encontrará ao trabalhar com linguagens de consulta.

Linguagem de definição de dados

Em alto nível, primeiro você precisa criar os objetos de banco de dados antes de adicionar dados. Você usará comandos de ***linguagem de definição de dados*** (DDL) para realizar operações em objetos de banco de dados, como o próprio banco de dados, esquemas, tabelas ou usuários; DDL define o estado dos objetos em seu banco de dados.

Os engenheiros de dados usam expressões SQL DDL comuns: CREATE, DROP e UPDATE. Por exemplo, você pode criar um banco de dados usando a barra CREATE DATABASE da expressão DDL. Depois disso, você também pode criar novas tabelas (CREATE table bar_table) ou excluir uma tabela (DROP table bar_table).

Linguagem de manipulação de dados

Depois de usar DDL para definir objetos de banco de dados, você precisa adicionar e alterar dados nesses objetos, que é o objetivo principal da ***linguagem de manipulação de dados*** (DML). Alguns comandos DML comuns que você usará como engenheiro de dados são os seguintes:

SELECIONAR
INSERIR
ATUALIZAR
EXCLUIR
CÓPIA DE
MERGE

Por exemplo, você pode INSERIR novos registros em uma tabela de banco de dados, ATUALIZAR os existentes e SELECIONAR registros específicos.

Linguagem de controle de dados

Você provavelmente deseja limitar o acesso a objetos de banco de dados e controlar com precisão ***quem*** tem acesso a ***quê. A linguagem de controle de dados*** (DCL) permite controlar o acesso aos objetos do banco de dados ou aos dados usando comandos SQL como GRANT, DENY e REVOKE.

Vamos ver um breve exemplo usando comandos DCL. Uma nova cientista de dados chamada Sarah ingressa na sua empresa e precisa de acesso somente leitura a um banco de dados chamado ***data_science_db.*** Você concede a Sarah acesso a esse banco de dados usando o seguinte comando DCL:

Machine Translated by Google

```
GRANT SELECT ON data_science_db TO user_name Sarah;
```

É um mercado de trabalho aquecido, e Sarah trabalhou na empresa por apenas alguns meses antes de ser roubada por uma grande empresa de tecnologia. Até mais, Sara! Sendo um engenheiro de dados preocupado com a segurança, você remove a capacidade de Sarah de ler o banco de dados:

```
REVOKE SELECT ON data_science_db TO user_name Sarah;
```

Solicitações e problemas de controle de acesso são comuns, e entender a DCL ajudará você a resolver problemas se você ou um membro da equipe não puder acessar os dados de que precisam, bem como impedir o acesso a dados de que não precisam.

TCL

TCL significa ***linguagem de controle de transações.*** Como o nome sugere, o TCL suporta comandos que controlam os detalhes das transações. Com o TCL, podemos definir pontos de verificação de confirmação, condições em que as ações serão revertidas e muito mais. Dois comandos TCL comuns incluem COMMIT e ROLLBACK.

A vida de uma consulta Como

funciona uma consulta e o que acontece quando uma consulta é executada? Vamos abordar os fundamentos de alto nível da execução de consultas (Figura 8-2), usando um exemplo de uma consulta SQL típica em execução em um banco de dados.

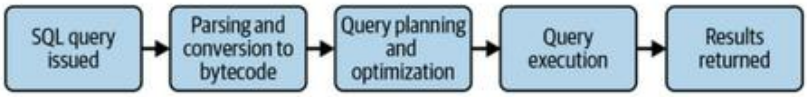


Figura 8-2. A vida de uma consulta SQL em um banco de dados

Embora a execução de uma consulta possa parecer simples – escreva código, execute-o e obtenha resultados – muita coisa está acontecendo nos bastidores. Quando você executa uma consulta SQL, aqui está um resumo do que acontece:

1. O mecanismo de banco de dados compila o SQL, analisando o código para verificar a semântica adequada e garantindo que os objetos de banco de dados referenciados existam e que o usuário atual tenha o acesso apropriado a esses objetos.
2. O código SQL é convertido em bytecode. Este bytecode expressa os passos que devem ser executados no mecanismo de banco de dados em um formato eficiente e legível por máquina.
3. O otimizador de consultas do banco de dados analisa o bytecode para determinar como executar a consulta, reordenando e etapas de refatoração para usar os recursos disponíveis da maneira mais eficiente possível.
4. A consulta é executada e os resultados são produzidos.

As consultas do Query Optimizer

podem ter tempos de execução muito diferentes, dependendo de como são executadas. O trabalho de um otimizador de consulta é otimizar o desempenho da consulta e minimizar os custos dividindo a consulta em etapas apropriadas em uma ordem eficiente. O otimizador avaliará junções, índices, tamanho da varredura de dados e outros fatores. O otimizador de consulta tenta executar a consulta da maneira menos dispendiosa.

Os otimizadores de consulta são fundamentais para o desempenho da sua consulta. Cada banco de dados é diferente e executa consultas de maneiras que são óbvia e sutilmente diferentes umas das outras. Você não trabalhará diretamente com um otimizador de consulta, mas entender algumas de suas funcionalidades o ajudará a escrever consultas com melhor desempenho. Você precisará saber como analisar o desempenho de uma consulta, usando itens como um plano de explicação ou análise de consulta, descritos na seção a seguir.

Melhorando o desempenho da consulta

Machine Translated by Google

Na engenharia de dados, você inevitavelmente encontrará consultas com desempenho insatisfatório. Saber identificar e corrigir essas consultas é inestimável. Não lute contra seu banco de dados. Aprenda a trabalhar com seus pontos fortes e aumentar suas fraquezas. Esta seção mostra várias maneiras de melhorar o desempenho da consulta.

Otimize sua estratégia e esquema de JOIN Um único

conjunto de dados (como uma tabela ou arquivo) raramente é útil sozinho; criamos valor combinando-o com outros conjuntos de dados. As ***junções*** são um dos meios mais comuns de combinar conjuntos de dados e criar novos. Presumimos que você esteja familiarizado com os tipos significativos de junções (por exemplo, interna, externa, esquerda, cruzada) e os tipos de relacionamentos de junção (por exemplo, um para um, um para muitos, muitos para um e muitos para muitos) .

As junções são críticas na engenharia de dados e têm bom suporte e desempenho em muitos bancos de dados. Mesmo bancos de dados colunares, que no passado tinham uma reputação de desempenho lento de junção, agora geralmente oferecem excelente desempenho.

Uma técnica comum para melhorar o desempenho da consulta é ***pré-juntar*** dados. Se você achar que as consultas de análise estão juntando os mesmos dados repetidamente, geralmente faz sentido juntar os dados com antecedência e fazer com que as consultas sejam lidas da versão pré-juntada dos dados para que você não repita o trabalho computacionalmente intensivo. Isso pode significar alterar o esquema e relaxar as condições de normalização para ampliar as tabelas e utilizar estruturas de dados mais recentes (como arrays ou structs) para substituir relacionamentos de entidade frequentemente associados. Outra estratégia é manter um esquema mais normalizado, mas pré-juntar tabelas para os casos de uso de análise e ciência de dados mais comuns. Podemos simplesmente criar tabelas pré-juntadas e treinar os usuários para utilizá-las ou ingressar em visualizações materializadas (consulte “Visualizações materializadas, federação e virtualização de consultas”).

Em seguida, considere os detalhes e a complexidade de suas condições de associação. A lógica de junção complexa pode consumir recursos computacionais significativos. Podemos melhorar o desempenho de junções complexas de algumas maneiras.

Muitos bancos de dados orientados a linhas permitem indexar um resultado calculado a partir de uma linha. Por exemplo, o PostgreSQL permite que você crie um índice em um campo de string convertido para minúsculas; quando o otimizador encontra uma consulta onde a função lower() aparece dentro de um predicado, ele pode aplicar o índice. Você também pode criar uma nova coluna derivada para ingressar, embora precise treinar os usuários para ingressar nessa coluna.

EXPLOSÃO DE LINHA

Um problema obscuro, mas frustrante, é a explosão de linhas. Isso ocorre quando temos um grande número de correspondências muitos para muitos, seja por repetição em chaves de junção ou como consequência da lógica de junção. Suponha que a chave de junção na tabela A tenha o valor repetido cinco vezes e a chave de junção na tabela B contenha esse mesmo valor repetido 10 vezes. Isso leva a uma junção cruzada dessas linhas: cada linha this da tabela A emparelhada com cada linha this da tabela B. Isso cria 5 × 10 = 50 linhas na saída. Agora suponha que muitas outras repetições estejam na chave de junção. A explosão de linhas geralmente gera linhas suficientes para consumir uma grande quantidade de recursos de banco de dados ou até mesmo causar a falha de uma consulta.

Também é essencial saber como seu otimizador de consulta lida com junções. Alguns bancos de dados podem reordenar junções e predicados, enquanto outros não. Uma explosão de linha em um estágio de consulta inicial pode fazer com que a consulta falhe, mesmo que um predicado posterior remova corretamente muitas das repetições na saída. A reordenação de predicados pode reduzir significativamente os recursos computacionais exigidos por uma consulta.

Finalmente, use expressões de tabela comuns (CTEs) em vez de subconsultas aninhadas ou tabelas temporárias. Os CTEs permitem que os usuários componham consultas complexas de forma legível, ajudando você a entender o fluxo de sua consulta. A importância da legibilidade para consultas complexas não pode ser subestimada.

Em muitos casos, os CTEs também oferecem melhor desempenho do que um script que cria tabelas intermediárias; se você precisar criar tabelas intermediárias, considere criar tabelas temporárias. Se você quiser saber mais sobre CTEs, uma rápida pesquisa na web trará muitas informações úteis.

Use o plano de explicação e entenda o desempenho da sua consulta

Machine Translated by Google

Como você aprendeu na seção anterior, o otimizador de consulta do banco de dados influencia a execução de uma consulta. O plano de explicação do otimizador de consulta mostrará como o otimizador de consulta determinou sua consulta de custo mais baixo ideal, os objetos de banco de dados usados (tabelas, índices, cache etc.) e várias estatísticas de consumo de recursos e desempenho em cada estágio de consulta. Alguns bancos de dados fornecem uma representação visual dos estágios de consulta. Em contrapartida, outros disponibilizam o plano de explicação via SQL com o comando EXPLAIN, que exibe a sequência de etapas que o banco de dados seguirá para executar a consulta.

Além de usar EXPLAIN para entender ***como*** sua consulta será executada, você deve monitorar o desempenho de sua consulta, visualizando métricas de consumo de recursos do banco de dados. A seguir estão algumas áreas para monitorar:

- Uso de recursos-chave, como disco, memória e rede.
- Tempo de carregamento de dados versus tempo de processamento.
- Tempo de execução da consulta, número de registros, tamanho dos dados verificados e quantidade de dados embaralhados.
- Consultas concorrentes que podem causar contenção de recursos em seu banco de dados.
- Número de conexões simultâneas usadas versus conexões disponíveis. Conexões simultâneas com excesso de assinaturas podem ter efeitos negativos em seus usuários que podem não conseguir se conectar ao banco de dados.

Evite varreduras de tabela completa

Todas as consultas verificam os dados, mas nem todas as verificações são criadas da mesma forma. Como regra geral, você deve consultar apenas os dados necessários. Ao executar SELECT * sem predicados, você está varrendo a tabela inteira e recuperando cada linha e coluna. Isso é muito ineficiente em termos de desempenho e caro, especialmente se você estiver usando um banco de dados de pagamento conforme o uso que cobra por bytes verificados ou recursos de computação utilizados durante a execução de uma consulta.

Sempre que possível, use a ***remoção*** para reduzir a quantidade de dados verificados em uma consulta. Bancos de dados colunares e orientados a linhas requerem diferentes estratégias de poda. Em um banco de dados orientado a colunas, você deve selecionar apenas as colunas necessárias. A maioria dos bancos de dados OLAP orientados a colunas também fornece ferramentas adicionais para otimizar suas tabelas para um melhor desempenho de consulta. Por exemplo, se você tiver uma tabela muito grande (vários terabytes de tamanho ou mais)
O Snowflake e o BigQuery oferecem a opção de definir uma chave de cluster em uma tabela, que ordena os dados da tabela de forma a permitir que as consultas acessem com mais eficiência partes de conjuntos de dados muito grandes. O BigQuery também permite particionar uma tabela em segmentos menores, permitindo consultar apenas partições específicas em vez de toda a tabela.
(Esteja ciente de que estratégias inadequadas de clustering e distribuição de chaves podem prejudicar o desempenho.)

Em bancos de dados orientados a linhas, a poda geralmente se concentra em índices de tabela, que você aprendeu no Capítulo 6. A estratégia geral é criar índices de tabela que melhorarão o desempenho de suas consultas mais sensíveis ao desempenho, sem sobrecarregar a tabela com tantos índices que você degrada o desempenho.

Saiba como seu banco de dados lida com confirmações Uma

confirmação de banco de dados é uma alteração em um banco de dados, como criar, atualizar ou excluir um registro, tabela ou outros objetos de banco de dados. Muitos bancos de dados suportam ***transações — ou seja,*** uma noção de confirmação de várias operações simultaneamente de forma a manter um estado consistente. Observe que o termo ***transação*** está um pouco sobrecarregado; consulte o Capítulo 5. O propósito de uma transação é manter um estado consistente de um banco de dados enquanto estiver ativo e em caso de falha. As transações também tratam do isolamento quando vários eventos simultâneos podem estar lendo, gravando e excluindo dos mesmos objetos de banco de dados. Sem transações, os usuários obteriam informações potencialmente conflitantes ao consultar um banco de dados.

Você deve estar intimamente familiarizado com a forma como seu banco de dados lida com confirmações e transações e determinar a consistência esperada dos resultados da consulta. Seu banco de dados lida com gravações e atualizações de maneira compatível com ACID? Sem conformidade ACID, sua consulta pode retornar resultados inesperados. Isso pode resultar de uma leitura suja, que acontece quando uma linha é lida e uma transação não confirmada alterou a linha. As leituras sujas são um comportamento esperado do seu banco de dados? Se sim, como você lida com isso? Além disso, esteja ciente de que durante as transações de atualização e exclusão, alguns bancos de dados criam novos arquivos para representar o novo estado do banco de dados e retêm os arquivos antigos para

Machine Translated by Google

referências de ponto de verificação de falha. Nesses bancos de dados, a execução de um grande número de pequenas confirmações pode causar confusão e consumir um espaço de armazenamento significativo que pode precisar ser limpo periodicamente.

Vamos considerar brevemente três bancos de dados para entender o impacto dos commits (observe que esses exemplos são atuais no momento da redação deste artigo). Primeiro, suponha que estamos olhando para um RDBMS PostgreSQL e aplicando transações ACID.
Cada transação consiste em um pacote de operações que falharão ou serão bem-sucedidas como um grupo. Também podemos executar consultas de análise em várias linhas; essas consultas apresentarão uma imagem consistente do banco de dados em um determinado momento.

A desvantagem da abordagem do PostgreSQL é que ela requer ***bloqueio de linha*** (bloqueio de leituras e gravações em determinadas linhas), o que pode prejudicar o desempenho de várias maneiras. O PostgreSQL não é otimizado para grandes varreduras ou grandes quantidades de dados apropriados para aplicativos de análise de grande escala.

Em seguida, considere o Google BigQuery. Ele utiliza um modelo de confirmação de tabela completa point-in-time. Quando uma consulta de leitura é emitida, o BigQuery lê o último snapshot confirmado da tabela. Quer a consulta seja executada por um segundo ou duas horas, ela lerá apenas a partir desse instantâneo e não verá nenhuma alteração subsequente. O BigQuery não bloqueia a tabela enquanto eu leio nela. Em vez disso, as operações de gravação subsequentes criarão novas confirmações e novos instantâneos enquanto a consulta continua a ser executada no instantâneo em que foi iniciada.

Para evitar o estado inconsistente, o BigQuery permite apenas uma operação de gravação por vez. Nesse sentido, o BigQuery não oferece nenhuma simultaneidade de gravação. (No sentido de que ele pode gravar grandes quantidades de dados em paralelo ***dentro de uma única consulta de gravação,*** é altamente concorrente.) Se mais de um cliente tentar gravar simultaneamente, as consultas de gravação serão enfileiradas por ordem de chegada. O modelo de confirmação do BigQuery é semelhante aos modelos de confirmação usados pelo Snowflake, Spark e outros.

Por último, vamos considerar o MongoDB. Nos referimos ao MongoDB como um ***banco de dados de consistência variável.*** Os engenheiros têm várias opções de consistência configuráveis, tanto para o banco de dados quanto no nível de consultas individuais. O MongoDB é celebrado por sua extraordinária escalabilidade e simultaneidade de gravação, mas é um tanto notório por problemas que surgem quando os engenheiros o abusam.
2

Por exemplo, em certos modos, o MongoDB suporta desempenho de gravação ultra-alto. No entanto, isso tem um custo: o banco de dados descartará gravações sem cerimônia e silenciosamente se ficar sobrecarregado com tráfego. Isso é perfeitamente adequado para aplicativos que podem perder alguns dados, por exemplo, aplicativos de IoT em que simplesmente queremos muitas medições, mas não nos preocupamos em capturar todas as medições. Não é uma ótima opção para aplicativos que precisam capturar dados e estatísticas exatos.

Nada disso quer dizer que esses são bancos de dados ruins. Eles são todos bancos de dados fantásticos quando são escolhidos para aplicativos apropriados e configurados corretamente. O mesmo vale para praticamente qualquer tecnologia de banco de dados.

As empresas não contratam engenheiros simplesmente para hackear o código isoladamente. Para serem dignos de seu título, os engenheiros devem desenvolver uma compreensão profunda dos problemas que têm a tarefa de resolver e das ferramentas de tecnologia. Isso se aplica a modelos de compromisso e consistência e a todos os outros aspectos do desempenho da tecnologia. Escolhas e configurações de tecnologia apropriadas podem diferenciar o sucesso extraordinário de um fracasso maciço. Consulte o Capítulo 6 para uma discussão mais profunda sobre consistência.

Registros mortos a vácuo

Como acabamos de discutir, as transações incorrem na sobrecarga de criar novos registros durante certas operações, como atualizações, exclusões e operações de índice, enquanto mantêm os registros antigos como ponteiros para o último estado do banco de dados.
À medida que esses registros antigos se acumulam no sistema de arquivos do banco de dados, eles não precisam mais ser referenciados. Você deve remover esses registros mortos em um processo chamado de ***aspiração.***

Você pode limpar uma única tabela, várias tabelas ou todas as tabelas em um banco de dados. Não importa como você opte por limpar, excluir registros de banco de dados mortos é importante por alguns motivos. Primeiro, ele libera espaço para novos registros, levando a menos volume de tabela e consultas mais rápidas. Em segundo lugar, registros novos e relevantes significam que os planos de consulta são mais precisos; registros desatualizados podem levar o otimizador de consulta a gerar planos abaixo do ideal e imprecisos. Por fim, a aspiração limpa os índices ruins, permitindo um melhor desempenho do índice.

Machine Translated by Google

As operações de vácuo são tratadas de forma diferente dependendo do tipo de banco de dados. Por exemplo, em bancos de dados suportados por armazenamento de objetos (BigQuery, Snowflake, Databricks), a única desvantagem da retenção de dados antigos é que ela usa espaço de armazenamento, podendo custar dinheiro dependendo do modelo de preço de armazenamento do banco de dados. No Snowflake, os usuários não podem aspirar diretamente. Em vez disso, eles controlam um intervalo de "viagem no tempo" que determina por quanto tempo os instantâneos de tabela são retidos antes de serem limpos automaticamente. O BigQuery utiliza uma janela de histórico fixa de sete dias. Databricks geralmente retém dados indefinidamente até que sejam manualmente limpos; a aspiração é importante para controlar os custos diretos de armazenamento do S3.

O Amazon Redshift lida com seus discos de cluster em várias configurações, e a limpeza pode afetar o desempenho e o armazenamento disponível. O VACUUM é executado automaticamente nos bastidores, mas os usuários às vezes podem querer executá-lo manualmente para fins de ajuste.

A aspiração torna-se ainda mais crítica para bancos de dados relacionais como PostgreSQL e MySQL. Um grande número de operações transacionais pode causar um rápido acúmulo de registros mortos, e os engenheiros que trabalham nesses sistemas precisam se familiarizar com os detalhes e o impacto da aspiração.

Aproveite os resultados da consulta em cache

Digamos que você tenha uma consulta intensiva que costuma executar em um banco de dados que cobra pela quantidade de dados consultados. Cada vez que uma consulta é executada, isso custa dinheiro. Em vez de executar novamente a mesma consulta no banco de dados repetidamente e incorrer em cobranças enormes, não seria bom se os resultados da consulta fossem armazenados e disponíveis para recuperação instantânea? Felizmente, muitos bancos de dados OLAP na nuvem armazenam em cache os resultados das consultas.

Quando uma consulta é executada inicialmente, ela recupera dados de várias fontes, os filtra e os associa e gera um resultado. Essa consulta inicial — uma consulta fria — é semelhante à noção de dados frios que exploramos no Capítulo 6. Para fins de argumentação, digamos que essa consulta levou 40 segundos para ser executada. Supondo que seu banco de dados armazene em cache os resultados da consulta, executar novamente a mesma consulta pode retornar resultados em 1 segundo ou menos. Os resultados foram armazenados em cache e a consulta não precisou ser executada a frio. Sempre que possível, aproveite os resultados do cache de consulta para ajudar a reduzir a pressão em seu banco de dados e, ao mesmo tempo, fornecer uma melhor experiência do usuário para consultas executadas com frequência. Observe também que ***as visualizações materializadas*** fornecem outra forma de cache de consulta (consulte "Visualizações materializadas, federação e virtualização de consulta").

Consultas sobre streaming de dados Os dados

de streaming estão constantemente em andamento. Como você pode imaginar, consultar dados de streaming é diferente de dados em lote. Para aproveitar totalmente um fluxo de dados, devemos adaptar padrões de consulta que reflitam sua natureza em tempo real. Por exemplo, sistemas como Kafka e Pulsar facilitam a consulta de fontes de dados de streaming. Vejamos algumas maneiras comuns de fazer isso.

Padrões básicos de consulta em fluxos Lembre-

se do CDC contínuo, discutido no Capítulo 7. O CDC, neste formato, basicamente configura um banco de dados analítico como um seguidor rápido de um banco de dados de produção. Um dos padrões de consulta de streaming mais antigos envolve simplesmente consultar o banco de dados de análise, recuperando resultados estatísticos e agregações com um pequeno atraso em relação ao banco de dados de produção.

A abordagem do seguidor rápido

Como isso é um padrão de consulta de streaming? Não poderíamos fazer a mesma coisa simplesmente executando nossas consultas no banco de dados de produção? Em princípio, sim; na prática, não. Os bancos de dados de produção geralmente não estão equipados para lidar com cargas de trabalho de produção e executar simultaneamente grandes varreduras analíticas em quantidades significativas de dados.
A execução de tais consultas pode tornar o aplicativo de produção lento ou até mesmo fazer com que ele falhe. O padrão de consulta CDC básico nos permite fornecer análises em tempo real com um impacto mínimo no sistema de produção.

O padrão de seguidor rápido pode utilizar um banco de dados transacional convencional como seguidor, mas há vantagens significativas em usar um sistema orientado a OLAP adequado (Figura 8-3). Tanto o Druid quanto o BigQuery combinam um buffer de streaming com armazenamento colunar de longo prazo em uma configuração semelhante à arquitetura Lambda (consulte o Capítulo 3).
Isso funciona muito bem para calcular estatísticas de rastreamento em dados históricos vastos com atualizações quase em tempo real.

Machine Translated by Google

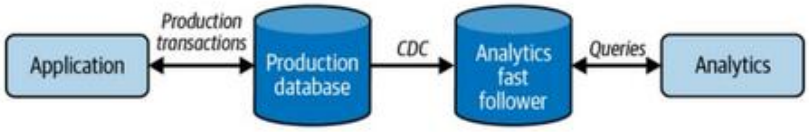


Figura 8-3. CDC com um banco de dados analítico de seguidor rápido

A abordagem do CDC seguidor rápido tem limitações críticas. Ele não repensa fundamentalmente os padrões de consulta em lote. Você ainda está executando consultas SELECT no estado atual da tabela e perdendo a oportunidade de acionar dinamicamente eventos de alterações no fluxo.

A arquitetura Kappa

Em seguida, lembre-se da arquitetura Kappa que discutimos no Capítulo 3. A ideia principal dessa arquitetura é manipular todos os dados como eventos e armazenar esses eventos como um fluxo em vez de uma tabela (Figura 8-4). Quando os bancos de dados do aplicativo de produção são a origem, a arquitetura Kappa armazena eventos do CDC. Os fluxos de eventos também podem fluir diretamente de um back-end de aplicativo, de um enxame de dispositivos IoT ou de qualquer sistema que gere eventos e possa enviá-los por uma rede. Em vez de simplesmente tratar um sistema de armazenamento de streaming como um buffer, a arquitetura Kappa retém eventos no armazenamento durante um período de retenção mais prolongado, e os dados podem ser consultados diretamente desse armazenamento. O período de retenção pode ser bastante longo (meses ou anos). Observe que isso é muito mais longo do que o período de retenção usado em sistemas orientados puramente em tempo real, geralmente uma semana no máximo.

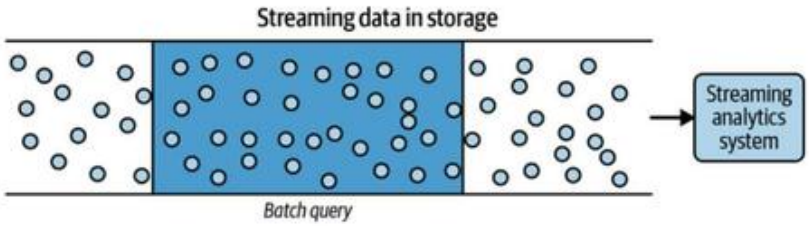


Figura 8-4. A arquitetura Kappa é construída em torno de sistemas de armazenamento e ingestão de streaming

A “grande ideia” da arquitetura Kappa é tratar o armazenamento de streaming como uma camada de transporte em tempo real e um banco de dados para recuperar e consultar dados históricos. Isso acontece por meio dos recursos de consulta direta do sistema de armazenamento de streaming ou com a ajuda de ferramentas externas. Por exemplo, o Kafka KSQL suporta agregação, cálculos estatísticos e até mesmo sessionalização. Se os requisitos de consulta forem mais complexos ou os dados precisarem ser combinados com outras fontes de dados, uma ferramenta externa, como o Spark, lê um intervalo de tempo de dados do Kafka e calcula os resultados da consulta. O sistema de armazenamento de streaming também pode alimentar outros aplicativos ou um processador de stream, como Flink ou Beam.

Windows, gatilhos, estatísticas emitidas e dados de chegada tardia Uma limitação

fundamental das consultas em lote tradicionais é que esse paradigma geralmente trata o mecanismo de consulta como um observador externo. Um ator externo aos dados faz com que a consulta seja executada – talvez um cron job de hora em hora ou um gerente de produto abrindo um painel.

Os sistemas de streaming mais usados, por outro lado, suportam a noção de computações acionadas diretamente dos próprios dados. Eles podem emitir estatísticas médias e medianas toda vez que um determinado número de registros é coletado no buffer ou gerar um resumo quando uma sessão de usuário é fechada.

O Windows é um recurso essencial em consultas e processamento de streaming. Windows são pequenos lotes processados com base em gatilhos dinâmicos. As janelas são geradas dinamicamente ao longo do tempo de algumas maneiras. Vejamos alguns tipos comuns de janelas: sessão, tempo fixo e deslizante. Também veremos as marcas d'água.

Janela de sessão

Uma ***janela de sessão*** agrupa eventos que ocorrem juntos e filtra os períodos de inatividade quando nenhum evento ocorre. Podemos dizer que uma sessão de usuário é qualquer intervalo de tempo sem intervalo de inatividade de cinco minutos ou mais. Nosso sistema de lote coleta dados por uma chave de ID do usuário, ordena eventos, determina as lacunas e os limites da sessão e

Machine Translated by Google

calcula estatísticas para cada sessão. Os engenheiros de dados costumam fazer sessões de dados retrospectivamente, aplicando condições de tempo à atividade do usuário em aplicativos da Web e de desktop.

Em uma sessão de streaming, esse processo pode acontecer dinamicamente. Observe que as janelas de sessão são por chave; no exemplo anterior, cada usuário obtém seu próprio conjunto de janelas. O sistema acumula dados por usuário. Se ocorrer um intervalo de cinco minutos sem atividade, o sistema fecha a janela, envia seus cálculos e libera os dados. Caso cheguem novos eventos para uso, o sistema inicia uma nova janela de sessão.

As janelas de sessão também podem fazer uma provisão para dados de chegada tardia. Permitindo que os dados cheguem com até cinco minutos de atraso para levar em conta as condições da rede e a latência do sistema, o sistema abrirá a janela se um evento de chegada tardia indicar atividade menos de cinco minutos após o último evento. Teremos mais a dizer sobre dados de chegada tardia ao longo deste capítulo. A Figura 8-5 mostra três janelas de sessão, cada uma separada por cinco minutos de inatividade.

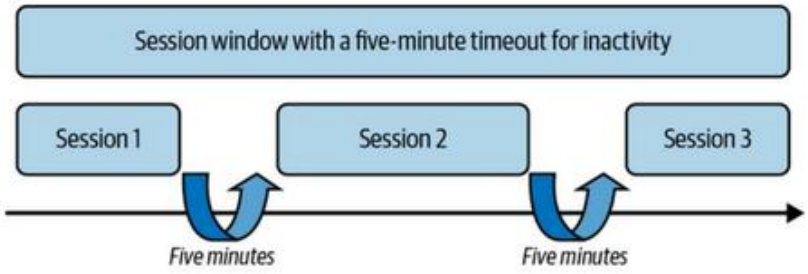


Figura 8-5. Janela de sessão com um tempo limite de cinco minutos para inatividade

Tornar a sessionalização dinâmica e quase em tempo real muda fundamentalmente sua utilidade. Com a sessionalização retrospectiva, poderíamos automatizar ações específicas um dia ou uma hora após o encerramento de uma sessão do usuário (por exemplo, um e-mail de acompanhamento com um cupom para um produto visualizado pelo usuário). Com a sessionalização dinâmica, o usuário pode receber um alerta em um aplicativo móvel que é imediatamente útil com base em sua atividade nos últimos 15 minutos.

Janelas de tempo fixo

Uma janela de tempo ***fixo*** (também conhecida como ***rolante)*** apresenta períodos de tempo fixos executados em uma programação fixa e processa todos os dados desde que a janela anterior foi fechada. Por exemplo, podemos fechar uma janela a cada 20 segundos e processar todos os dados que chegam da janela anterior para fornecer uma estatística média e mediana (Figura 8-6). As estatísticas seriam emitidas assim que pudessem ser calculadas após o fechamento da janela.

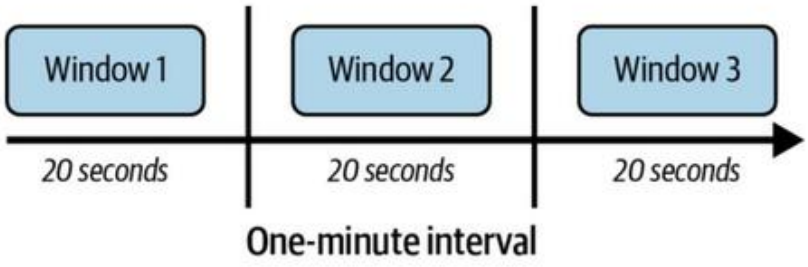


Figura 8-6. Janela basculante/ fixa

Isso é semelhante ao processamento ETL em lote tradicional, no qual podemos executar um trabalho de atualização de dados todos os dias ou a cada hora. O sistema de streaming nos permite gerar janelas com mais frequência e entregar resultados com menor latência. Como enfatizaremos repetidamente, o lote é um caso especial de streaming.

Janelas deslizantes

Os eventos em uma janela deslizante são agrupados em janelas de duração fixa. Por exemplo, poderíamos gerar uma nova janela de 60 segundos a cada 30 segundos (Figura 8-7). Assim como fizemos antes, podemos emitir estatísticas médias e medianas.
Janelas deslizantes também são conhecidas como ***janelas saltitantes.***

Machine Translated by Google

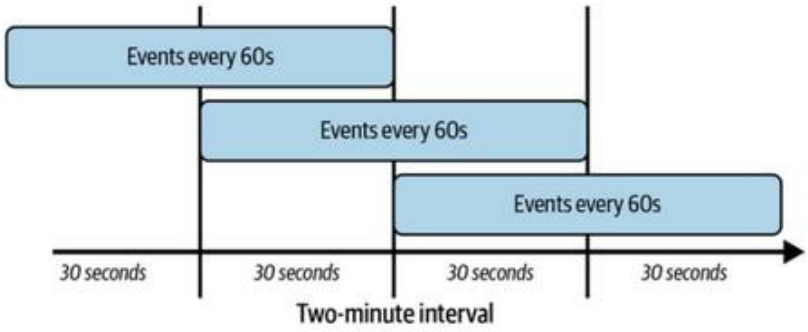


Figura 8-7. Janelas deslizantes

O deslizamento pode variar. Por exemplo, podemos pensar na janela como realmente deslizando continuamente, mas emitindo estatísticas apenas quando certas condições (gatilhos) são atendidas. Suponha que usamos uma janela deslizante contínua de 30 segundos, mas calculamos uma estatística apenas quando um usuário clica em um banner específico. Isso levaria a uma taxa de saída extremamente alta quando muitos usuários clicam no banner e nenhum cálculo durante uma pausa.

Marcas d'água

Cobrimos vários tipos de janelas e seus usos. Conforme discutido no Capítulo 7, os dados às vezes são ingeridos fora da ordem em que se originaram. Uma ***marca d'água*** (Figura 8-8) é um limite usado por uma janela para determinar se os dados em uma janela estão dentro do intervalo de tempo estabelecido ou se são considerados atrasados. Se chegarem dados novos na janela, mas mais antigos que o carimbo de data/hora da marca d'água, eles serão considerados dados de chegada tardia.

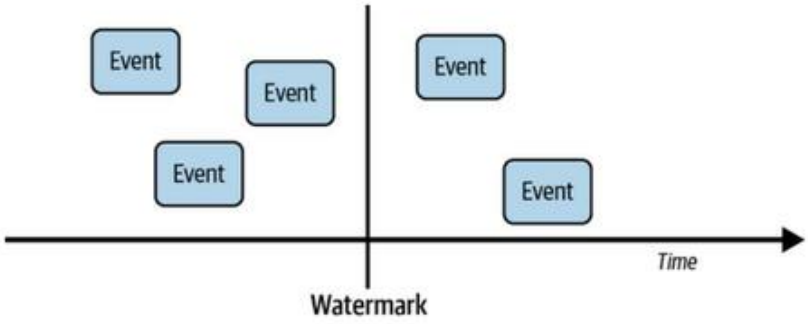


Figura 8-8. Marca d'água atuando como um limite para dados de chegada tardia

Combinando fluxos com outros dados Como

mencionamos anteriormente, geralmente derivamos valor dos dados combinando-os com outros dados. Os dados de streaming não são diferentes. Por exemplo, vários fluxos podem ser combinados ou um fluxo pode ser combinado com dados históricos de lote.

Junções de tabelas convencionais

Algumas tabelas podem ser alimentadas por streams (Figura 8-9). A abordagem mais básica para esse problema é simplesmente juntar essas duas tabelas em um banco de dados. Um fluxo pode alimentar uma ou ambas as tabelas.

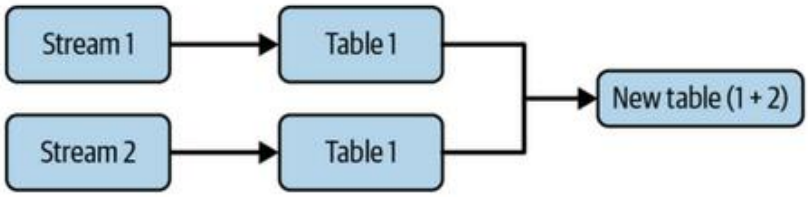


Figura 8-9. Unindo duas tabelas alimentadas por streams

Enriquecimento

Enriquecimento significa que juntamos um fluxo a outros dados (Figura 8-10). Normalmente, isso é feito para fornecer dados aprimorados em outro fluxo. Por exemplo, suponha que um varejista online receba um fluxo de eventos de um parceiro

Machine Translated by Google

negócios contendo IDs de produtos e usuários. O varejista deseja aprimorar esses eventos com detalhes do produto e informações demográficas sobre os usuários. O varejista alimenta esses eventos para uma função sem servidor que pesquisa o produto e o usuário em um banco de dados na memória (digamos, um cache), adiciona as informações necessárias ao evento e envia os eventos aprimorados para outro fluxo.

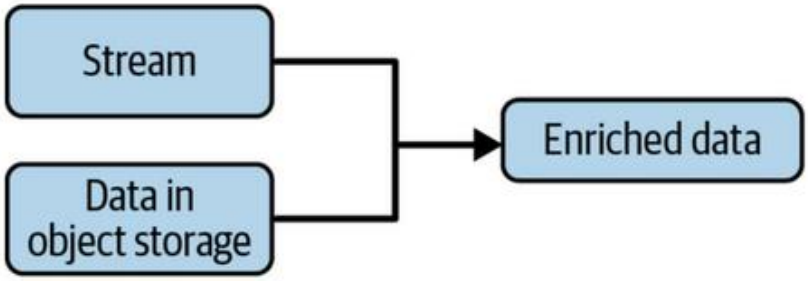


Figura 8-10. Neste exemplo, um fluxo é enriquecido com dados que residem no armazenamento de objetos, resultando em um novo conjunto de dados enriquecido

Na prática, a fonte de enriquecimento pode se originar em quase qualquer lugar — uma tabela em um data warehouse ou RDBMS na nuvem ou um arquivo no armazenamento de objetos. É simplesmente uma questão de ler a partir da fonte e armazenar os dados de enriquecimento necessários em um local apropriado para recuperação pelo fluxo.

Junção de fluxo a fluxo

Cada vez mais, os sistemas de streaming suportam a junção direta de stream a stream. Suponha que um varejista on-line deseje unir seus dados de eventos da web com dados de streaming de uma plataforma de anúncios. A empresa pode alimentar os dois fluxos no Spark, mas surgem várias complicações. Por exemplo, os fluxos podem ter latências significativamente diferentes para chegada no ponto em que a junção é tratada no sistema de streaming. A plataforma de anúncios pode fornecer seus dados com um atraso de cinco minutos. Além disso, alguns eventos podem ser significativamente atrasados, por exemplo, um evento de fechamento de sessão para um usuário ou um evento que ocorre no telefone offline e aparece no fluxo somente depois que o usuário está de volta ao alcance da rede móvel.

Como tal, as arquiteturas típicas de junção de streaming dependem de buffers de streaming. O intervalo de retenção do buffer é configurável; um intervalo de retenção mais longo requer mais armazenamento e outros recursos. Os eventos se juntam aos dados no buffer e são eventualmente despejados após o intervalo de retenção ter passado (Figura 8-11).[5]

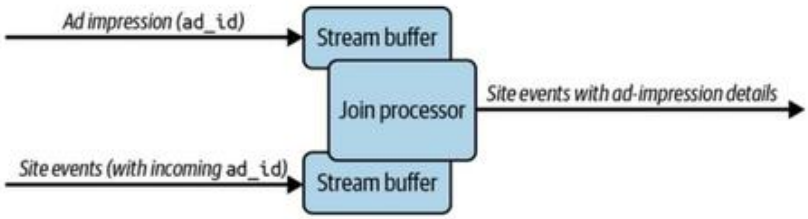


Figura 8-11. Uma arquitetura para juntar fluxos armazena em buffer cada fluxo e junta eventos se eventos relacionados forem encontrados durante o intervalo de retenção do buffer

Agora que abordamos como as consultas funcionam para dados em lote e streaming, vamos discutir como tornar seus dados úteis modelando-os.

Modelagem de dados

A modelagem de dados é algo que vemos negligenciado com frequência perturbadora. Frequentemente, vemos equipes de dados pulando na construção de sistemas de dados sem um plano de jogo para organizar seus dados de uma maneira que seja útil para os negócios. Isto é um erro. Arquiteturas de dados bem construídas devem refletir os objetivos e a lógica de negócios da organização que depende desses dados. A modelagem de dados envolve a escolha deliberada de uma estrutura coerente para os dados e é uma etapa crítica para tornar os dados úteis para os negócios.

A modelagem de dados tem sido uma prática há décadas de uma forma ou de outra. Por exemplo, vários tipos de técnicas de normalização (discutidas em "Normalização") têm sido usadas para modelar dados desde os primeiros dias dos RDBMSs; dados

Machine Translated by Google

técnicas de modelagem de armazenamento existem desde pelo menos o início da década de 1990, e sem dúvida há mais tempo. Como os pêndulos na tecnologia costumam ir, a modelagem de dados tornou-se um pouco fora de moda no início e meados da década de 2010. A ascensão do data lake 1.0, NoSQL e sistemas de big data permitiram que os engenheiros ignorassem a modelagem de dados tradicional, às vezes para obter ganhos de desempenho legítimos. Outras vezes, a falta de modelagem de dados rigorosa criava pântanos de dados, juntamente com muitos dados redundantes, incompatíveis ou simplesmente errados.

Hoje em dia, o pêndulo parece estar voltando para a modelagem de dados. A crescente popularidade do gerenciamento de dados (em particular, governança de dados e qualidade de dados) está aumentando a necessidade de uma lógica de negócios coerente. A ascensão meteórica da proeminência dos dados nas empresas cria um reconhecimento crescente de que a modelagem é fundamental para obter valor nos níveis mais altos da pirâmide de Hierarquia de Necessidades da Ciência de Dados. Por outro lado, acreditamos que novos paradigmas são necessários para realmente abraçar as necessidades de streaming de dados e ML. Nesta seção, examinamos as técnicas atuais de modelagem de dados e refletimos brevemente sobre o futuro da modelagem de dados.

O que é um modelo de dados?

Um ***modelo de dados*** representa a maneira como os dados se relacionam com o mundo real. Ele reflete como os dados devem ser estruturados e padronizados para refletir melhor os processos, definições, fluxos de trabalho e lógica da sua organização. Um bom modelo de dados captura como a comunicação e o trabalho fluem naturalmente em sua organização. Em contraste, um modelo de dados pobre (ou inexistente) é aleatório, confuso e incoerente.

Alguns profissionais de dados veem a modelagem de dados como tediosa e reservada para “grandes empresas”. Como a maioria das boas práticas de higiene – como passar fio dental e dormir bem – a modelagem de dados é reconhecida como uma boa coisa a se fazer, mas muitas vezes é ignorada na prática. Idealmente, toda organização deve modelar seus dados apenas para garantir que a lógica e as regras de negócios sejam traduzidas na camada de dados.

Ao modelar dados, é fundamental focar em traduzir o modelo para resultados de negócios. Um bom modelo de dados deve se correlacionar com decisões de negócios impactantes. Por exemplo, um ***cliente*** pode significar coisas diferentes para diferentes departamentos de uma empresa. Alguém que comprou de você nos últimos 30 dias é um cliente? E se eles não compraram de você nos últimos seis meses ou um ano? Definir e modelar cuidadosamente esses dados do cliente pode ter um impacto enorme nos relatórios de downstream sobre o comportamento do cliente ou na criação de modelos de rotatividade de clientes em que o tempo desde a última compra é uma variável crítica.

GORJETA

Você consegue pensar em conceitos ou termos em sua empresa que podem significar coisas diferentes para pessoas diferentes?

Nossa discussão se concentra principalmente na modelagem de dados em lote, pois é onde surgiram a maioria das técnicas de modelagem de dados. Também veremos algumas abordagens para modelar dados de streaming e considerações gerais para modelagem.

Modelos de dados conceituais, lógicos e físicos

Ao modelar dados, a ideia é passar de conceitos de modelagem abstratos para implementação concreta. Ao longo desse continuum (Figura 8-12), três modelos de dados principais são conceituais, lógicos e físicos. Esses modelos formam a base para as várias técnicas de modelagem que descrevemos neste capítulo:

Conceptual

Contém lógica e regras de negócios e descreve os dados do sistema, como esquemas, tabelas e campos (nomes e tipos). Ao criar um modelo conceitual, geralmente é útil visualizá-lo em um diagrama entidade-relacionamento (ER), que é uma ferramenta padrão para visualizar os relacionamentos entre várias entidades em seus dados (pedidos, clientes, produtos etc.). Por exemplo, um diagrama ER pode codificar as conexões entre o ID do cliente,

Machine Translated by Google

nome do cliente, endereço do cliente e pedidos do cliente. A visualização de relacionamentos de entidade é altamente recomendada para projetar um modelo de dados conceitual coerente.

Lógico

Detalha como o modelo conceitual será implementado na prática, adicionando muito mais detalhes. Por exemplo, adicionaríamos informações sobre os tipos de ID de cliente, nomes de clientes e endereços personalizados. Além disso, mapeamos as chaves primárias e estrangeiras.

Fisica

Define como o modelo lógico será implementado em um sistema de banco de dados. Adicionaríamos bancos de dados, esquemas e tabelas específicos ao nosso modelo lógico, incluindo detalhes de configuração.

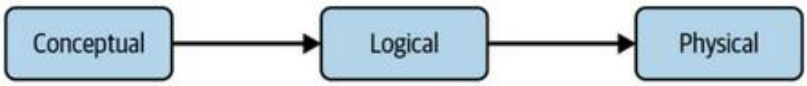


Figura 8-12. O continuum de modelos de dados: conceitual, lógico e físico

A modelagem de dados bem-sucedida envolve as partes interessadas do negócio no início do processo. Os engenheiros precisam obter definições e metas de negócios para os dados. A modelagem de dados deve ser um esporte de contato total, cujo objetivo é fornecer à empresa dados de qualidade para insights acionáveis e automação. Esta é uma prática da qual todos devem participar continuamente.

Outra consideração importante para a modelagem de dados é a ***granulação*** dos dados, que é a resolução na qual os dados são armazenados e consultados. A granulação geralmente está no nível de uma chave primária em uma tabela, como ID do cliente, ID do pedido e ID do produto; geralmente é acompanhado por uma data ou carimbo de data/hora para aumentar a fidelidade.

Por exemplo, suponha que uma empresa tenha acabado de começar a implantar relatórios de BI. A empresa é pequena o suficiente para que a mesma pessoa esteja desempenhando o papel de engenheiro de dados e analista. Uma solicitação chega para um relatório que resume os pedidos diários dos clientes. Especificamente, o relatório deve listar todos os clientes que fizeram pedidos, o número de pedidos que fizeram naquele dia e o valor total que gastaram.

Este relatório é inerentemente grosseiro. Ele não contém detalhes sobre gastos por pedido ou os itens em cada pedido. É tentador para o engenheiro/analista de dados ingerir dados do banco de dados de ordens de produção e reduzi-los a uma tabela de relatórios com apenas os dados agregados básicos necessários para o relatório. No entanto, isso implicaria recomeçar quando chegasse uma solicitação para um relatório com agregação de dados mais detalhada.

Como o engenheiro de dados é bastante experiente, ele escolhe criar tabelas com dados detalhados sobre os pedidos dos clientes, incluindo cada pedido, item, custo do item, IDs do item, etc. Essencialmente, suas tabelas contêm todos os detalhes dos pedidos dos clientes. A granulação dos dados está no nível do pedido do cliente. Esses dados do pedido do cliente podem ser analisados como estão ou agregados para estatísticas resumidas sobre a atividade do pedido do cliente.

Em geral, você deve se esforçar para modelar seus dados no nível mais baixo de granulação possível. A partir daqui, é fácil agregar esse conjunto de dados altamente granular. O inverso não é verdadeiro e geralmente é impossível restaurar detalhes que foram agregados.

Normalização

A normalização é uma prática de modelagem de dados de banco de dados que impõe controle estrito sobre os relacionamentos de tabelas e colunas em um banco de dados. O objetivo da normalização é remover a redundância de dados dentro de um banco de dados e garantir a integridade referencial. Basicamente, ***não se repita*** (DRY) aplicado a dados em um banco de dados.

A normalização é normalmente aplicada a bancos de dados relacionais contendo tabelas com linhas e colunas (usamos os termos ***coluna*** e ***campo*** de forma intercambiável nesta seção). Foi introduzido pela primeira vez pelo pioneiro do banco de dados relacional Edgar Codd no início dos anos 1970.

Machine Translated by Google

Codd delineou quatro objetivos principais da normalização: [7]

- Para liberar a coleção de relações de dependências indesejáveis de inserção, atualização e exclusão
- Para reduzir a necessidade de reestruturação da coleção de relações, à medida que novos tipos de dados são introduzidos e, assim, aumentar a vida útil dos programas aplicativos
- Para tornar o modelo relacional mais informativo para os usuários
- Para tornar a coleta de relações neutra para as estatísticas de consulta, onde essas estatísticas podem mudar com o passar do tempo

Codd introduziu a ideia de ***formas normais.*** As formas normais são sequenciais, com cada forma incorporando as condições das formas anteriores. Descrevemos as três primeiras formas normais de Codd aqui:

Desnormalizado

Sem normalização. Dados aninhados e redundantes são permitidos.

Primeira forma normal (1FN)

Cada coluna é única e tem um único valor. A tabela tem uma chave primária exclusiva.

Segunda forma normal (2FN)

Os requisitos da 1NF, mais as dependências parciais, são removidos.

Terceira forma normal (3FN)

Os requisitos da 2NF, mais cada tabela contém apenas campos relevantes relacionados à sua chave primária e não possui dependências transitivas.

Vale a pena gastar um momento para descompactar alguns termos que acabamos de jogar para você. Uma ***chave primária exclusiva*** é um único campo ou conjunto de vários campos que determina exclusivamente as linhas na tabela. Cada valor de chave ocorre no máximo uma vez; caso contrário, um valor seria mapeado para várias linhas na tabela. Assim, todos os outros valores em uma linha dependem (podem ser determinados) da chave. Uma ***dependência parcial*** ocorre quando um subconjunto de campos em uma chave composta pode ser usado para determinar uma coluna não chave da tabela. Uma ***dependência transitiva*** ocorre quando um campo não-chave depende de outro campo não-chave.

Vejamos os estágios de normalização — de desnormalizado a 3NF — usando um exemplo de comércio eletrônico de pedidos de clientes (Tabela 8-1). Forneceremos explicações concretas de cada um dos conceitos apresentados no parágrafo anterior.

Machine Translated by Google

Tabela 8-1. OrderDetail

Código do pedido	Itens de ordem	Identificação do Cliente	Nome do Cliente	Data do Pedido
100	[{ "sku": 1, "preço": 50, "quantidade": 1, "name:": "Thingamajig" }, { "sku": 2, "preço": 25, "quantidade": 2, "name:": "Whatchamacallit" }]	5	Joe Reis	01-03-2022

Primeiro, esta tabela OrderDetail desnormalizada contém cinco campos. A chave primária é OrderID. Observe que o campo OrderItems contém um objeto aninhado com dois SKUs junto com seu preço, quantidade e nome.

Para converter esses dados para 1NF, vamos mover OrderItems para quatro campos (Tabela 8-2). Agora temos uma tabela OrderDetail na qual os campos não contêm repetições ou dados aninhados.

Machine Translated by Google

Tabela 8-2. OrderDetailswithoutreplacementsorNestedd

Código do pedido	Sku	Preço	Quantidade	ProductName CustomerID	Nome do Cliente Pedido

Machine Translated by Google

100	1	50	1	Thingamajig	5	Joe Reis	202
100	2	25	2	Objeto ou pessoa desconhecido	5	Joe Reis	202

O problema é que agora não temos uma chave primária única. Ou seja, 100 ocorre na coluna OrderID em dois fileiras diferentes. Para entender melhor a situação, vejamos uma amostra maior da nossa tabela (Tabela 8-3).

Machine Translated by Google

Tabela 8-3. OrderDetailswithregistersmapeado

Código do pedido	Sku	Preço	Quantidade	ProductName	CustomerID	Nome do Cliente	Pedido
100	1	50	1	Thingamajig	5	Joe Reis	202
100	2	25	2	Objeto ou pessoa desconhecido	5	Joe Reis	202
101	3	75	1	Whozeewhatzit	7	Matt Housley	202
102	1	50	1	Thingamajig	7	Matt Housley	202

Para criar uma chave primária (composta) exclusiva, vamos numerar as linhas em cada ordem adicionando uma coluna chamada LineItemNumber (Tabela 8-4).

Machine Translated by Google

Tabela 8-4. OrderDetailswithItemNumbercolumn

Código do pedido	LinhaItemNúmero	Sku	Preço	Quantidade	ProductName	CustomerID	Cus
100	1	1	50	1	Thingamajig	5	João

Machine Translated by Google

100	2	2	25	2	Objeto ou pessoa desconhecido	5	João
101	1	3	75	1	Whozeewhatzit	7	Esteira
102	1	1	50	1	Thingamajig	7	Esteira

A chave composta (OrderID, LineItemNumber) agora é uma chave primária exclusiva.

Para alcançar 2NF, precisamos garantir que não existam dependências parciais. Uma ***dependência parcial*** é uma coluna não chave que é totalmente determinado por um subconjunto das colunas na chave primária (composta) exclusiva; dependências parciais podem ocorrem apenas quando a chave primária é composta. No nosso caso, as três últimas colunas são determinadas pelo número do pedido. Para corrigir esse problema, vamos dividir OrderDetail em duas tabelas: Orders e OrderLineItem (Tabelas 8-5 e 8-6).

Tabela 8-5. Pedidos

Código do pedido	Identificação do Cliente	Nome do Cliente	Data do Pedido
100	5	Joe Reis	01-03-2022
101	7	Matt Housley	01-03-2022
102	7	Matt Housley	01-03-2022

Machine Translated by Google

Tabela 8-6. OrderLineItem

Código do pedido	LinhaItemNúmero Sku		Preço	Quantidade	Nome do Produto
100	1	1	50	1	Thingamajig
100	2	2	25	2	Objeto ou pessoa desconhecido
101	1	3	75	1	Whozeewhatzit
102	1	1	50	1	Thingamajig

A chave composta (OrderID, LineItemNumber) é uma chave primária exclusiva para OrderLineItem, enquanto OrderID é uma chave primária para Pedidos.

Observe que Sku determina ProductName em OrderLineItem. Ou seja, Sku depende da chave composta, e ProductName depende do Sku. Esta é uma dependência transitiva. Vamos dividir OrderLineItem em OrderLineItem e Skus (Tabelas 8-7 e 8-8).

Machine Translated by Google

Tabela 8-7. OrderLineItem

Código do pedido	LinhaItemNúmero Sku		Preço	Quantidade
100	1	1	50	1
100	2	2	25	2
101	1	3	75	1
102	1	1	50	1

Machine Translated by Google

T
uma

bl
e

8
-
8
.
S
k
você
s

Sku	Nome do Produto
1	Thingamajig
2	Objeto ou pessoa desconhecido
3	Whozeewhatzit

Agora, tanto OrderLineItem quanto Skus estão na 3NF. Observe que Orders não satisfaz a 3NF. Que dependências transitivas estão presentes? Como você consertaria isso?

Existem formas normais adicionais (até 6NF no sistema Boyce-Codd), mas estas são muito menos comuns do que as três primeiras. Um banco de dados geralmente é considerado normalizado se estiver na terceira forma normal, e essa é a convenção que usamos neste livro.

O grau de normalização que você deve aplicar aos seus dados depende do seu caso de uso. Não existe uma solução de tamanho único, especialmente em bancos de dados em que alguma desnormalização apresenta vantagens de desempenho. Embora a desnormalização possa parecer um antipadrão, é comum em muitos sistemas OLAP que armazenam dados semiestruturados.
Estude convenções de normalização e práticas recomendadas de banco de dados para escolher uma estratégia apropriada.

Técnicas para Modelagem de Dados Analíticos em Lote

Ao descrever a modelagem de dados para data lakes ou data warehouses, você deve presumir que os dados brutos assumem muitas formas (por exemplo, estruturados e semiestruturados), mas a saída é um modelo de dados estruturado de linhas e colunas.
No entanto, várias abordagens para modelagem de dados podem ser usadas nesses ambientes. As grandes abordagens que você provavelmente encontrará são Kimball, Inmon e cofre de dados.

Na prática, algumas dessas técnicas podem ser combinadas. Por exemplo, vemos algumas equipes de dados começarem com o cofre de dados e, em seguida, adicionarem um esquema em estrela Kimball ao lado dele. Também veremos modelos de dados amplos e desnormalizados e outras técnicas de modelagem de dados em lote que você deve ter em seu arsenal. À medida que discutimos cada uma dessas técnicas, usaremos o exemplo de modelagem de transações que ocorrem em um sistema de pedidos de comércio eletrônico.

NOTA

Nossa cobertura das três primeiras abordagens – Inmon, Kimball e cofre de dados – é superficial e dificilmente faz justiça à sua respectiva complexidade e nuance. Ao final de cada seção, listamos os livros canônicos de seus criadores. Para um engenheiro de dados, esses livros são leituras obrigatórias e recomendamos que você os leia, mesmo que seja apenas para entender como e por que a modelagem de dados é fundamental para dados analíticos em lote.

Machine Translated by Google

Inmon

O pai do data warehouse, Bill Inmon, criou sua abordagem para modelagem de dados em 1990. Antes do data warehouse, a análise geralmente ocorria diretamente no próprio sistema de origem, com a consequência óbvia de atolar bancos de dados transacionais de produção com consultas. O objetivo do data warehouse era separar o sistema de origem do sistema analítico.

A Inmon define um data warehouse da seguinte maneira:

> ***Um data warehouse é uma coleção de dados orientada por assunto, integrada, não volátil e variável no tempo para apoiar as decisões da administração. O data warehouse contém dados corporativos granulares. Os dados no data warehouse podem ser usados para muitas finalidades diferentes, incluindo sentar e esperar por requisitos futuros que são desconhecidos hoje .***

As quatro partes críticas de um data warehouse significam o seguinte:

Orientado ao assunto

O data warehouse se concentra em uma área de assunto específica, como vendas ou marketing.

Não volátil

Os dados permanecem inalterados depois que os dados são armazenados em um data warehouse.

Integrado

Dados de fontes diferentes são consolidados e normalizados.

Tempo variável

Vários intervalos de tempo podem ser consultados.

Vejamos cada uma dessas partes para entender sua influência em um modelo de dados do Inmon. Primeiro, o modelo lógico deve se concentrar em uma área específica. Por exemplo, se a ***orientação do assunto*** for "vendas", o modelo lógico contém todos os detalhes relacionados a vendas - chaves de negócios, relacionamentos, atributos etc. Em seguida, esses detalhes são ***integrados*** a um modelo de dados consolidado e altamente normalizado. Por fim, os dados são armazenados inalterados de maneira ***não volátil*** e ***variante no tempo ,*** o que significa que você pode (teoricamente) consultar os dados originais pelo tempo que o histórico de armazenamento permitir. O data warehouse da Inmon deve aderir estritamente a todas essas quatro partes críticas ***para apoiar as decisões da administração.***
Este é um ponto sutil, mas posiciona o data warehouse para análise, não OLTP.

Aqui está outra característica importante do data warehouse da Inmon:

> ***A segunda característica saliente do data warehouse é que ele é integrado. De todos os aspectos de um data warehouse, a integração é o mais importante. Os dados são alimentados de várias fontes diferentes no data warehouse. À medida que os dados são alimentados, eles são convertidos, reformatados, resequenciados, resumidos etc. O resultado é que os dados — uma vez que residem no data warehouse — têm uma única imagem corporativa física.***

Com o data warehouse da Inmon, os dados são integrados em toda a organização em um modelo de ER granular e altamente normalizado, com ênfase incansável em ETL. Devido à natureza orientada por assunto do data warehouse, o data warehouse da Inmon consiste em bancos de dados de origem chave e sistemas de informação usados em uma organização. Os dados dos principais sistemas de origem de negócios são ingeridos e integrados em um data warehouse altamente normalizado (3NF) que geralmente se assemelha muito à estrutura de normalização do próprio sistema de origem; os dados são trazidos de forma incremental, começando pelas áreas de negócios de maior prioridade. O requisito rigoroso de normalização garante o mínimo de duplicação de dados possível, o que leva a menos erros analíticos downstream porque os dados não divergirão nem sofrerão redundâncias. O data warehouse representa uma “fonte única da verdade”, que dá suporte aos requisitos gerais de informações da empresa. Os dados são apresentados para relatórios e análises downstream por meio de data marts específicos de negócios e departamentos, que também podem ser desnormalizados.

Machine Translated by Google

Vejamos como um data warehouse Inmon é usado para comércio eletrônico (Figura 8-13). Os sistemas de origem do negócio são pedidos, estoque e marketing. Os dados desses sistemas de origem são enviados por ETL para o data warehouse e armazenados no 3NF. Idealmente, o data warehouse engloba holisticamente as informações do negócio. Para servir dados para solicitações de informações específicas do departamento, os processos ETL pegam dados do data warehouse, transformam os dados e os colocam em data marts downstream para serem visualizados em relatórios.

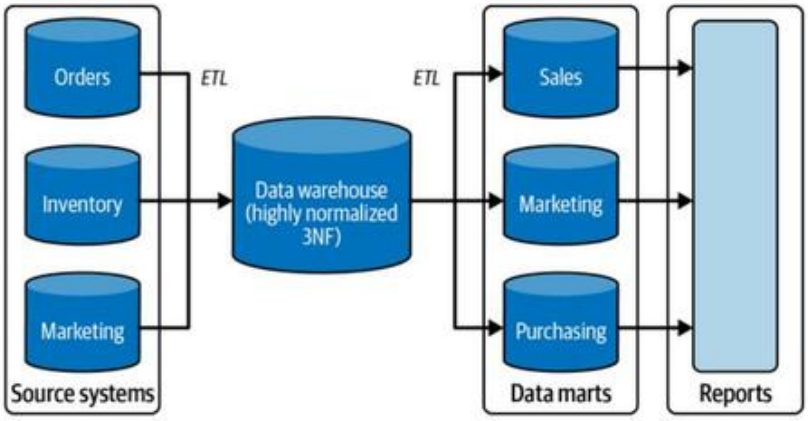


Figura 8-13. Um armazém de dados de comércio eletrônico

Uma opção popular para modelar dados em um data mart é um esquema em estrela (discutido na próxima seção sobre Kimball), embora qualquer modelo de dados que forneça informações facilmente acessíveis também seja adequado. No exemplo anterior, vendas, marketing e compras têm seu próprio esquema em estrela, alimentado a montante dos dados granulares no data warehouse.
Isso permite que cada departamento tenha sua própria estrutura de dados única e otimizada para suas necessidades específicas.

A Inmon continua inovando no espaço de data warehouse, atualmente focando em ETL textual no data warehouse.
Ele também é um escritor e pensador prolífico, escrevendo mais de 60 livros e inúmeros artigos. Para ler mais sobre o data warehouse da Inmon, consulte seus livros listados em "Recursos Adicionais".

Kimball

Se houver espectros para modelagem de dados, Kimball está muito no extremo oposto de Inmon. Criada por Ralph Kimball no início da década de 1990, essa abordagem de modelagem de dados se concentra menos na normalização e, em alguns casos, aceita a desnormalização. Como Inmon diz sobre a diferença entre o data warehouse e o data mart, "um data [10] mart nunca é um substituto para um data warehouse".

Enquanto a Inmon integra dados de toda a empresa no data warehouse e fornece análises específicas do departamento por meio de data marts, o modelo Kimball é de baixo para cima, incentivando você a modelar e servir análises de departamento ou de negócios no próprio data warehouse (Inmon argumenta que abordagem distorce a definição de um data warehouse). Isso pode permitir uma iteração e modelagem mais rápida do que o Inmon, com a compensação de uma potencial integração de dados mais flexível, redundância de dados e duplicação.

Na abordagem de Kimball, os dados são modelados com dois tipos gerais de tabelas: fatos e dimensões. Você pode pensar em uma ***tabela de fatos*** como uma tabela de números e ***as tabelas de dimensões*** como dados qualitativos que fazem referência a um fato. As tabelas de dimensão envolvem uma única tabela de fatos em um relacionamento chamado ***esquema em estrela*** [11] (Figura 8-14). Vejamos fatos, dimensões e esquemas em estrela.

Machine Translated by Google

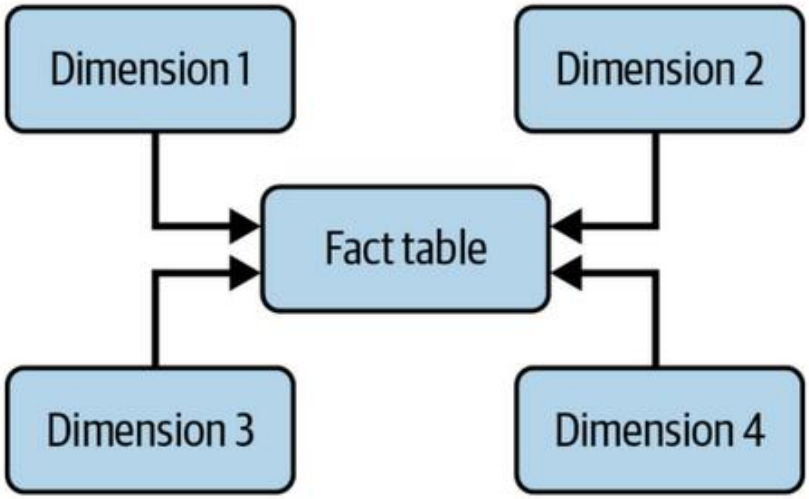


Figura 8-14. Um esquema em estrela Kimball, com fatos e dimensões

Tabelas de fatos

O primeiro tipo de tabela em um esquema em estrela é a tabela de fatos, que contém dados ***factuais,*** quantitativos e relacionados a eventos. Os dados em uma tabela de fatos são imutáveis porque os fatos estão relacionados a eventos. Portanto, as tabelas de fatos não mudam e são somente anexadas. As tabelas de fatos geralmente são estreitas e longas, o que significa que não têm muitas colunas, mas muitas linhas que representam eventos. As tabelas de fatos devem ter a granularidade mais baixa possível.

As consultas em um esquema em estrela começam com a tabela de fatos. Cada linha de uma tabela de fatos deve representar a granulação dos dados. Evite agregar ou derivar dados em uma tabela de fatos. Se você precisar realizar agregações ou derivações, faça isso em uma consulta downstream, tabela de data mart ou exibição. Finalmente, as tabelas de fatos não fazem referência a outras tabelas de fatos; eles fazem referência apenas às dimensões.

Vejamos um exemplo de uma tabela de fatos elementares (Tabela 8-9). Uma pergunta comum em sua empresa pode ser: “Mostre-me vendas brutas, por pedido de cada cliente, por data”. Mais uma vez, os fatos devem estar no nível mais baixo possível — nesse caso, o orderID da venda, cliente, data e valor bruto da venda. Observe que os tipos de dados na tabela de fatos são todos números (inteiros e flutuantes); não há cordas. Além disso, neste exemplo, CustomerKey 7 tem dois pedidos no mesmo dia, refletindo a granulação da tabela. Em vez disso, a tabela de fatos possui chaves que fazem referência a tabelas de dimensões contendo seus respectivos atributos, como informações de cliente e data. O valor bruto de vendas representa a venda total para o ***evento de vendas.***

Machine Translated by Google

Tabela 8-9. Tabela de fatos

Código do pedido	Chave de data da chave do cliente		Valor Bruto de Vendas
100	5	20220301	100,00
101	7	20220301	75,00
102	7	20220301	50,00

Tabelas de dimensão

O segundo tipo principal de tabela em um modelo de dados Kimball é chamado de ***dimensão.*** As tabelas de dimensões fornecem a dados de referência, atributos e contexto relacional para os eventos armazenados em tabelas de fatos. As tabelas de dimensão são menores do que as tabelas de fatos e assumem uma forma oposta, geralmente larga e curta. Quando associadas a uma tabela de fatos, as dimensões podem descrever os eventos o quê, onde e quando. As dimensões são desnormalizadas, com possibilidade de dados duplicados. Isso está correto no modelo de dados Kimball. Vejamos as duas dimensões referenciadas na tabela de fatos anterior exemplo.

Em um modelo de dados Kimball, as datas geralmente são armazenadas em uma dimensão de data, permitindo que você faça referência à chave de data (DateKey) entre a tabela de dimensão de fato e data. Com a tabela de dimensão de data, você pode responder facilmente perguntas como: "Quais são minhas vendas totais no primeiro trimestre de 2022?" ou “Quantos clientes mais compram em terça do que quarta?” Observe que temos cinco campos além da chave de data (Tabela 8-10). A beleza de um dimensão de data é que você pode adicionar quantos novos campos fizer sentido para analisar seus dados.

Machine Translated by Google

Tabela 8-10. Uma dimensão de mês contendo uma linha

Chave de data	Data-ISO	Ano	Trimestre	Mês	Dia da semana
20220301	01-03-2022	2022	1	3	Terça-feira
20220302	02-03-2022	2022	1	3	Quarta-feira
20220303	03-03-2022	2022	1	3	Quinta-feira

A Tabela 8-11 também faz referência a outra dimensão — a dimensão do cliente — pelo campo CustomerKey. o
A dimensão do cliente contém vários campos que descrevem o cliente: nome e sobrenome, CEP e alguns
de campos de data de aparência peculiar. Vejamos esses campos de data, pois eles ilustram outro conceito nos dados de Kimball
modelo: uma dimensão de mudança lenta do Tipo 2, que descreveremos com mais detalhes a seguir.

Machine Translated by Google

Tabela 8-11. Tipo 2 como você armazena mensalmente uma tabela

CustomerKey	FirstName	Sobrenome	Código postal	EFF_StartDate	EFF_EndDate
5	João	Reis	84108	04-01-2019	9999-01-01
7	Matt	Housley	84101	04-05-2020	19-09-2021
7	Matt	Housley	84123	19-09-2021	9999-01-01
11	Lana	Bela	90210	04-02-2022	9999-01-01

Por exemplo, dê uma olhada em CustomerKey 5, com o EFF_StartDate (EFF_StartDate significa ***efetiva data de início)*** de 2019-01-04 e uma EFF_EndDate de 9999-01-01. Isso significa que o cadastro de clientes de Joe Reis

Machine Translated by Google

foi criado na tabela de dimensões do cliente em 04-01-2019 e tem uma data final de 01-01-9999. Interessante. O que significa essa data de término? Isso significa que o registro do cliente está ativo e não foi alterado.

Agora vamos ver o registro do cliente de Matt Housley (CustomerKey = 7). Observe as duas entradas para a data de início de Housley: 2020-05-04 e 2021-09-19. Parece que Housley mudou seu CEP em 19/09/2021, resultando em uma alteração em seu registro de cliente. Quando os dados forem consultados para os registros de clientes mais recentes, você consultará onde a data de término é igual a 9999-01-01.

Uma dimensão de alteração lenta (SCD) é necessária para rastrear alterações nas dimensões. O exemplo anterior é um SCD Tipo 2: um novo registro é inserido quando um registro existente é alterado. Embora os SCDs possam ir até sete níveis, vejamos os três mais comuns:

Tipo 1

Substituir registros de dimensão existentes. Isso é super simples e significa que você não tem acesso aos arquivos excluídos registros de dimensão histórica.

Tipo 2

Mantenha um histórico completo dos registros de dimensão. Quando um registro é alterado, esse registro específico é sinalizado como alterado e é criado um novo registro de dimensão que reflete o status atual dos atributos. Em nosso exemplo, Housley mudou para um novo CEP, o que acionou seu registro inicial para refletir uma data de término efetiva, e um novo registro foi criado para mostrar seu novo CEP.

Tipo 3

Um SCD Tipo 3 é semelhante a um SCD Tipo 2, mas em vez de criar uma nova linha, uma alteração em um SCD Tipo 3 cria um novo campo. Usando o exemplo anterior, vamos ver como é um SCD Tipo 3 no seguinte
mesas.

Na Tabela 8-12, Housley mora no CEP 84101. Quando Housley muda para um novo CEP, o SCD Tipo 3 cria dois novos campos, um para seu novo CEP e a data da alteração (Tabela 8-13). O campo de CEP original também é renomeado para refletir que este é o registro mais antigo.

Machine Translated by Google

Tabela 8-12. Typesetting row meanings

CustomerKey	FirstName	Sobrenome	Código postal
7	Matt	Housley	84101

Machine Translated by Google

Tabela 8-13. Tipo 3 cliente dimensão tabela

CustomerKey	FirstName	Sobrenome	CEP original	CEP atual	Data CEP atual
7	Matt	Housley	84101	84123	19-09-2021

Dos tipos de SCDs descritos, o Tipo 1 é o comportamento padrão da maioria dos data warehouses e o Tipo 2 é o que vemos mais comumente usado na prática. Há muito o que saber sobre dimensões, e sugerimos usar esta seção como ponto de partida para se familiarizar com o funcionamento das dimensões e como elas são usadas.

Esquema em estrela

Agora que você tem um entendimento básico de fatos e dimensões, é hora de integrá-los em um esquema em estrela.
O ***esquema em estrela*** representa o modelo de dados do negócio. Ao contrário de abordagens altamente normalizadas para modelagem de dados, o esquema em estrela é uma tabela de fatos cercada pelas dimensões necessárias. Isso resulta em menos junções do que outros dados

Machine Translated by Google

modelos, o que acelera o desempenho da consulta. Outra vantagem de um esquema em estrela é que é sem dúvida mais fácil para os usuários de negócios entenderem e usarem.

Observe que o esquema em estrela não deve refletir um relatório específico, embora você possa modelar um relatório em um data mart downstream ou diretamente em sua ferramenta de BI. O esquema em estrela deve capturar os fatos e atributos de sua ***lógica de negócios*** e ser flexível o suficiente para responder às respectivas perguntas críticas.

Como um esquema em estrela tem uma tabela de fatos, às vezes você terá vários esquemas em estrela que abordam diferentes fatos do negócio. Você deve se esforçar para reduzir o número de dimensões sempre que possível, pois esses dados de referência podem ser reutilizados entre diferentes tabelas de fatos. Uma dimensão que é reutilizada em vários esquemas em estrela, compartilhando assim os mesmos campos, é chamada de ***dimensão conformada.*** Uma dimensão conformada permite combinar várias tabelas de fatos em vários esquemas em estrela. Lembre-se, dados redundantes são aceitáveis com o método Kimball, mas evite replicar as mesmas tabelas de dimensão para evitar definições de negócios flutuantes e integridade de dados.

O modelo de dados Kimball e o esquema em estrela têm muitas nuances. Você deve estar ciente de que este modo é apropriado apenas para dados em lote e não para dados de streaming. Como o modelo de dados Kimball é popular, há uma boa chance de você encontrá-lo

Cofre de dados

Enquanto Kimball e Inmon se concentram na estrutura da lógica de negócios no data warehouse, o ***data vault*** oferece uma abordagem diferente para a modelagem de dados. Criado na década de 1990 por Dan Linstedt, o modelo de cofre de dados separa os aspectos estruturais dos dados de um sistema de origem de seus atributos. Em vez de representar a lógica de negócios em fatos, dimensões ou tabelas altamente normalizadas, um cofre de dados simplesmente carrega dados de sistemas de origem diretamente em um punhado de tabelas criadas para fins específicos de uma maneira somente de inserção. Ao contrário das outras abordagens de modelagem de dados que você aprendeu, não há noção de dados bons, ruins ou conformados em um cofre de dados.

Os dados se movem rapidamente nos dias de hoje e os modelos de dados precisam ser ágeis, flexíveis e escaláveis; o modelo de cofre de dados visa atender a essa necessidade. O objetivo desse modelo é manter os dados o mais alinhados possível com o negócio, mesmo enquanto os dados do negócio evoluem.

Um cofre de dados consiste em três tipos principais de tabelas: hubs, links e satélites (Figura 8-15). Em resumo, um ***hub*** armazena chaves de negócios, um ***link*** mantém relacionamentos entre chaves de negócios e um ***satélite*** representa os atributos e o contexto de uma chave de negócios. Um usuário consultará um hub, que se vinculará a uma tabela satélite contendo os atributos relevantes da consulta. Vamos explorar hubs, links e satélites com mais detalhes.



Figura 8-15. Tabelas de cofre de dados: hubs, links e satélites conectados entre si

Hubs

As consultas geralmente envolvem a pesquisa por uma chave comercial, como um ID de cliente ou um ID de pedido do nosso exemplo de comércio eletrônico. Um hub é a entidade central de um cofre de dados que retém um registro de todas as chaves comerciais carregadas no cofre de dados. O hub deve refletir o estado do sistema de origem do qual carrega os dados.

Um hub sempre contém os seguintes campos padrão:

Chave de hash

A chave primária usada para unir dados entre sistemas. Este é um campo de hash calculado (MD5 ou similar)

Data de carregamento

A data em que os dados foram carregados no hub.

Fonte de registro

O sistema de origem.

Machine Translated by Google

Chave(s) de negócios

A chave comercial, ou ID de registro, no sistema de origem.

É importante observar que um hub é somente para inserção e os dados dos sistemas de origem não são alterados em um hub. Depois que os dados são carregados em um hub, eles se tornam permanentes.

Ao projetar um hub, identificar a chave de negócios é fundamental. Pergunte a si mesmo: Qual é o ***elemento comercial identificável*** em um sistema de origem? Dito de outra forma, como os usuários de um sistema de origem geralmente procuram dados? Idealmente, isso é descoberto à medida que você cria o modelo de dados conceitual de sua organização e antes de começar a construir seu cofre de dados.

Usando nosso cenário de comércio eletrônico, vejamos um exemplo de um hub para produtos. Primeiro, vejamos o design físico de um hub de produto (Tabela 8-14).

Machine Translated by Google

Tabela 8-14. Uma representação física de uma tabela de hub de produto

Produto Hub
ProductHashKey
LoadDate
Origem do registro
ID do produto

Na prática, o hub do produto fica assim quando preenchido com dados (Tabela 8-15). Neste exemplo, três produtos diferentes são carregados em um hub de um sistema ERP em duas datas separadas.

Machine Translated by Google

T
uma
bl
e
8
-
1
5
.
UMA
p
r
o
d
você
c
t
h
você
b
p
o
p
você
eu
uma
t
e
d
W
isto
HD
uma
t
uma

ProductHashKey LoadDate	Código do Produto da Origem do Registro	
4041fd80ab68e267490522b4a99048f5 2020-01-02	ERP	1
de8435530d6ca93caabc00cf88982dc1 2021-03-09	ERP	2
cf27369bd80f53d0e60d394817d77bab 09-03-2021	ERP	3

Enquanto estamos nisso, vamos criar outro hub para pedidos (Tabela 8-16) usando o mesmo esquema do HubProduct e preenchê-lo com alguns dados de pedido de amostra.

Machine Translated by Google

T

uma

bl
e
8
-
1
6
.
UMA
n
o
r
d
e
r
h
você
b
p
o
p
você
eu
uma
t
e
d
W
eu
t

HD

uma
t
uma

OrderHashKey LoadDate	Código do Pedido da Origem do Registro		
f899139df5e1059396431415e770c6dd 2022-03-01	Local na rede Internet	100	
38b3eff8baf56627478ec76a704e9b52 2022-03-01	Local na rede Internet	101	
ec8956637a99787bd197eacd77acce5e 2022-03-01	Local na rede Internet	102	

Links

Uma ***tabela de links*** rastreia os relacionamentos das chaves de negócios entre os hubs. As tabelas de links devem conectar pelo menos dois hubs, de preferência com a granulação mais baixa possível. Como as tabelas de links conectam dados de vários hubs, elas são muitos para muitos.
As relações do modelo de cofre de dados são diretas e tratadas por meio de alterações nos links. Isso fornece excelente flexibilidade no caso inevitável de alteração dos dados de origem subjacentes. Você adiciona um novo hub, e isso

Machine Translated by Google

hub precisa se conectar a hubs existentes. Basta adicionar um novo relacionamento a esse hub em sua tabela de links. É isso! Agora vamos ver maneiras de visualizar dados contextualmente usando satélites.

De volta ao nosso exemplo de comércio eletrônico, gostaríamos de associar pedidos a produtos. Vamos ver como pode ser uma tabela de links para pedidos e produtos (Tabela 8-17).

Machine Translated by Google

Tabela 8-17. Uma tabela de link para pedidos de produtos

LinkPedidoProduto
OrderProductHashKey
LoadDate
Origem do registro
ProductHashKey
OrderHashKey

Machine Translated by Google

Quando a tabela LinkOrderProduct é preenchida, esta é a aparência (Tabela 8-18). Observe que estamos usando a fonte de registro do pedido neste exemplo.

Machine Translated by Google

Tabela 8-18. Tabela de link de conexão de pedidos e produtos

OrderProductHashKey	LoadDate	RecordSource	ProductHashKey	OrderHashKey
ff64ec193d7bacf05e8b97ea04b3906 6	01-03-2022	Local na rede Internet	4041fd80ab68e267490522b4a990 48f5	f899139df5e1059396431415e770 c6dd
ff64ec193d7bacf05e8b97ea04b3906	2022-03-01	Local na rede Internet	de8435530d6ca93caabc00cf88982	f899139df5e1059396431415e770

Machine Translated by Google

6			dc1	c6dd
e232628c251e3cad0f53f7e6e13cdea7	01-03-2022	Local na rede Internet	cf27369bd80f53d0e60d394817d77bab	38b3eff8baf56627478ec76a704e9b52
26166a5871b6d21ae12e9c464262be67	01-03-2022	Local na rede Internet	4041fd80ab68e267490522b4a99048f5	ec8956637a99787bd197eacd77acce5e

Satélites

Descrevemos relacionamentos entre hubs e links que envolvem chaves, datas de carregamento e fontes de registro. Como você tem uma noção do que essas relações significam? ***Satélites*** são atributos descritivos que dão significado e contexto para hubs. Os satélites podem se conectar a hubs ou links. Os únicos campos obrigatórios em um satélite são um chave que consiste na chave comercial do hub pai e uma data de carregamento. Além disso, um satélite pode conter muitos atributos que fazem sentido.

Vejamos um exemplo de satélite para o hub Produto (Tabela 8-19). Neste exemplo, o
A tabela SatelliteProduct contém informações adicionais sobre o produto, como nome e preço do produto.

Machine Translated by Google

Tabela 8-19. Satélite Produto

Produto Satélite
ProductHashKey
LoadDate
Origem do registro
Nome do Produto
Preço

E aqui está a tabela SatelliteProduct com alguns dados de amostra (Tabela 8-20).

Machine Translated by Google

Table 8-20. product_satellite_ewshsmpled_table

ProductHashKey	LoadDate	RecordSource	ProductName	Price
4041fd80ab68e267490522b4a99048f5	2020-01-02	ERP	Thingamajig	50
de8435530d6ca93caabc00cf88982dc1	2021-03-09	ERP	Whatchamacallit	25
cf27369bd80f53d0e60d394817d77bab	09-03-2021	ERP	Whozeewhatzit	75

Machine Translated by Google

Vamos juntar tudo isso e juntar as tabelas de hub, produto e link em um cofre de dados (Figura 8-16).



Figura 8-16. O cofre de dados para pedidos e produtos

Existem outros tipos de tabelas de cofre de dados, incluindo tabelas de ponto no tempo (PIT) e de ponte. Não os abordamos aqui, mas os mencionamos porque o cofre de dados é bastante abrangente. Nosso objetivo é simplesmente fornecer uma visão geral do poder do cofre de dados.

Ao contrário de outras técnicas de modelagem de dados que discutimos, em um cofre de dados, a lógica de negócios é criada e interpretada quando os dados dessas tabelas são consultados. Esteja ciente de que o modelo de cofre de dados pode ser usado com outras técnicas de modelagem de dados. Não é incomum que um cofre de dados seja a zona de destino para dados analíticos, após o que é modelado separadamente em um data warehouse, geralmente usando um esquema em estrela. O modelo de cofre de dados também pode ser adaptado para NoSQL e fontes de dados de streaming. O cofre de dados é um tópico enorme, e esta seção destina-se simplesmente a torná-lo ciente de sua existência.

Tabelas amplas desnormalizadas

As abordagens de modelagem estritas que descrevemos, especialmente Kimball e Inmon, foram desenvolvidas quando os data warehouses eram caros, no local e altamente limitados por recursos com computação e armazenamento fortemente acoplados. Embora a modelagem de dados em lote tenha sido tradicionalmente associada a essas abordagens rígidas, abordagens mais relaxadas estão se tornando mais comuns.

Há razões para isso. Primeiro, a popularidade da nuvem significa que o armazenamento é muito barato. É mais barato armazenar dados do que se preocupar com a melhor maneira de representar os dados armazenados. Em segundo lugar, a popularidade de dados aninhados (JSON e similares) significa que os esquemas são flexíveis em sistemas de origem e analíticos.

Você tem a opção de modelar rigidamente seus dados conforme descrevemos ou pode optar por lançar todos os seus dados em uma única tabela ampla. Uma ***tabela ampla*** é exatamente o que parece: uma coleção altamente desnormalizada e muito ampla de muitos campos, normalmente criada em um banco de dados colunar. Um campo pode ser um valor único ou conter dados aninhados. Os dados são organizados junto com uma ou várias chaves; essas chaves estão intimamente ligadas à ***granulação*** dos dados.

Uma tabela ampla pode potencialmente ter milhares de colunas, enquanto menos de 100 são comuns em bancos de dados relacionais. Mesas largas geralmente são esparsas; a grande maioria das entradas em um determinado campo pode ser nula. Isso é extremamente caro em um banco de dados relacional tradicional porque o banco de dados aloca uma quantidade fixa de espaço para cada entrada de campo; nulos praticamente não ocupam espaço em um banco de dados colunar. Um esquema amplo em um banco de dados relacional diminui drasticamente a leitura porque cada linha deve alocar todo o espaço especificado pelo esquema amplo e o banco de dados deve ler o conteúdo de cada linha em sua totalidade. Por outro lado, um banco de dados colunar lê apenas as colunas selecionadas em uma consulta e a leitura de nulos é essencialmente gratuita.

Tabelas amplas geralmente surgem por meio da evolução do esquema; engenheiros adicionam campos gradualmente ao longo do tempo. A evolução do esquema em um banco de dados relacional é um processo lento e com muitos recursos. Em um banco de dados colunar, adicionar um campo é inicialmente apenas uma alteração nos metadados. À medida que os dados são gravados no novo campo, novos arquivos são adicionados à coluna.

As consultas de análise em tabelas amplas geralmente são executadas mais rapidamente do que consultas equivalentes em dados altamente normalizados que exigem muitas junções. A remoção de junções pode ter um grande impacto no desempenho da verificação. A tabela ampla simplesmente contém todos os dados que você juntaria em uma abordagem de modelagem mais rigorosa. Fatos e dimensões são representados na mesma tabela. A falta de rigor no modelo de dados também significa que não há muito pensamento envolvido. Carregue seus dados em uma tabela ampla e comece a consultá-los. Especialmente com esquemas em sistemas de origem se tornando mais adaptáveis e flexíveis, esses dados geralmente resultam de transações de alto volume, o que significa que há muitos dados. Armazenar isso como dados aninhados em seu armazenamento analítico tem muitos benefícios.

Machine Translated by Google

Jogar todos os seus dados em uma única tabela pode parecer uma heresia para um modelador de dados hardcore, e temos visto muitas críticas. Quais são algumas dessas críticas? A maior crítica é que, à medida que você mistura seus dados, perde a lógica de negócios em suas análises. Outra desvantagem é o desempenho de atualizações para coisas como um elemento em uma matriz, o que pode ser muito doloroso.

Vejamos um exemplo de uma tabela ampla (Tabela 8-21), usando a tabela desnormalizada original de nosso exemplo de normalização anterior. Essa tabela pode ter muito mais colunas — centenas ou mais! — e incluímos apenas algumas colunas para concisão e facilidade de compreensão. Como você pode ver, esta tabela combina vários tipos de dados, representados ao longo de um grão de pedidos de um cliente em uma data.

Machine Translated by Google

Tabela 8-21. Um exemplo de um arquivo de log de pedidos

Código do pedido	Itens de ordem	Identificação do Cliente	Nome do Cliente	Data do Pedido	Local	SiteRegião
100	[{ "sku": 1, "preço": 50, "quantidade": 1, "nome": "Thingamajig" }, { "sku": 2, "preço": 25,	5	Joe Reis	01-03-2022	abc.com	NÓS

Machine Translated by Google

"quantidade": 2,
"n / D
eu:": "Whatchamac
tudo isso"
}]

Sugerimos o uso de uma tabela ampla quando você não se importa com a modelagem de dados ou quando tem muitos dados que precisam de mais flexibilidade do que o rigor da modelagem de dados tradicional oferece. Tabelas amplas também se prestam a dados de streaming, que discutiremos a seguir. À medida que os dados avançam em direção a esquemas de movimento rápido e streaming em primeiro lugar, esperamos ver uma nova onda de modelagem de dados, talvez algo como "normalização relaxada".

E SE VOCÊ NÃO MODELO SEUS DADOS?

Você também tem a opção de ***não*** modelar seus dados. Nesse caso, basta consultar as fontes de dados diretamente. Esse padrão é frequentemente usado, especialmente quando as empresas estão apenas começando e desejam obter insights rápidos ou compartilhar análises com seus usuários. Embora permita obter respostas para várias perguntas, você deve considerar o seguinte:

Se eu não modelar meus dados, como saberei se os resultados das minhas consultas são consistentes?

Tenho definições adequadas de lógica de negócios no sistema de origem e minha consulta produzirá respostas verdadeiras?

Que carga de consulta estou colocando em meus sistemas de origem e como isso afeta os usuários desses sistemas?

Em algum momento, você provavelmente vai gravitar em direção a um paradigma de modelo de dados em lote mais estrito e uma arquitetura de dados dedicada que não depende dos sistemas de origem para o trabalho pesado.

Modelagem de dados de streaming

Embora muitas técnicas de modelagem de dados estejam bem estabelecidas para lotes, esse não é o caso para dados de streaming. Devido à natureza ilimitada e contínua dos dados de streaming, traduzir técnicas de lote como Kimball para um paradigma de streaming é complicado, se não impossível. Por exemplo, dado um fluxo de dados, como você atualizaria continuamente uma dimensão de mudança lenta do Tipo 2 sem deixar seu data warehouse de joelhos?

O mundo está evoluindo do lote para o streaming e do local para a nuvem. As restrições dos métodos de lote mais antigos não se aplicam mais. Dito isso, permanecem grandes questões sobre como modelar dados para equilibrar a necessidade de lógica de negócios em relação a mudanças fluidas de esquema, dados em movimento rápido e autoatendimento. Qual é o equivalente de streaming das abordagens de modelo de dados em lote anteriores? Não há (ainda) uma abordagem de consenso sobre modelagem de dados de streaming.
Conversamos com muitos especialistas em sistemas de dados de streaming, muitos dos quais nos disseram que a modelagem tradicional de dados orientada a lotes não se aplica ao streaming. Alguns sugeriram o cofre de dados como uma opção para modelagem de dados de streaming.

Como você deve se lembrar, existem dois tipos principais de fluxos: fluxos de eventos e CDC. Na maioria das vezes, a forma dos dados nesses fluxos é semiestruturada, como JSON. O desafio com a modelagem de dados de streaming é que o esquema da carga útil pode mudar por capricho. Por exemplo, suponha que você tenha um dispositivo IoT que atualizou recentemente seu firmware e introduziu um novo campo. Nesse caso, é possível que seu data warehouse de destino downstream ou pipeline de processamento não esteja ciente dessa alteração e seja interrompido. Isso não é ótimo. Como outro exemplo, um sistema CDC pode reformular um campo como um tipo diferente — digamos, uma string em vez de um formato de data e hora da International Organization for Standardization (ISO). Novamente, como o destino lida com essa mudança aparentemente aleatória?

Os especialistas em dados de streaming com quem conversamos sugerem que você preveja mudanças nos dados de origem e mantenha um esquema flexível. Isso significa que não há um modelo de dados rígido no banco de dados analítico. Em vez disso, suponha que os sistemas de origem estejam fornecendo os dados corretos com a definição e a lógica de negócios corretas, como existem hoje. E como o armazenamento é barato, armazene o streaming recente e os dados históricos salvos de forma que possam ser consultados juntos.

Machine Translated by Google

Otimize para análises abrangentes em um conjunto de dados com um esquema flexível. Além disso, em vez de reagir a relatórios, por que não criar uma automação que responda a anomalias e alterações nos dados de streaming?

O mundo da modelagem de dados está mudando e acreditamos que uma mudança radical ocorrerá em breve nos paradigmas do modelo de dados. Essas novas abordagens provavelmente incorporarão métricas e camadas semânticas, pipelines de dados e fluxos de trabalho analíticos tradicionais em uma camada de streaming que fica diretamente na parte superior do sistema de origem. Como os dados estão sendo gerados em tempo real, a noção de separar artificialmente os sistemas de origem e análise em dois buckets distintos pode não fazer tanto sentido quanto quando os dados são movidos de forma mais lenta e previsível. O tempo vai dizer…

Temos mais a dizer sobre o futuro do streaming de dados no Capítulo 11.

Transformações

> ***O resultado líquido da transformação de dados é a capacidade de unificar e integrar dados. Uma vez que os dados são transformados, os dados podem ser vistos como uma única entidade. Mas sem transformar os dados, você não pode ter uma visão unificada dos dados em toda a organização.***
>
> —Bill Inmon

Agora que abordamos consultas e modelagem de dados, você deve estar se perguntando: se posso modelar dados, consultá-los e obter resultados, por que preciso pensar em transformações? As transformações manipulam, aprimoram e salvam dados para uso posterior, aumentando seu valor de maneira escalável, confiável e econômica.

Imagine executar uma consulta sempre que quiser visualizar os resultados de um conjunto de dados específico. Você executaria a mesma consulta dezenas ou centenas de vezes por dia. Imagine que essa consulta envolva análise, limpeza, junção, união e agregação em 20 conjuntos de dados. Para agravar ainda mais a dor, a consulta leva 30 minutos para ser executada, consome recursos significativos e incorre em cobranças de nuvem substanciais em várias repetições. Você provavelmente ficaria louco.
Felizmente, você pode ***salvar os resultados de sua consulta*** ou, pelo menos, executar as partes mais intensivas em computação apenas uma vez, para que as consultas subsequentes sejam simplificadas.

Uma transformação difere de uma consulta. Uma ***consulta*** recupera os dados de várias fontes com base na filtragem e na lógica de junção. Uma ***transformação*** mantém os resultados para consumo por transformações ou consultas adicionais. Esses resultados podem ser armazenados de forma efêmera ou permanente.

Além da persistência, um segundo aspecto que diferencia transformações de consultas é a complexidade. Você provavelmente criará pipelines complexos que combinam dados de várias fontes e reutilizam resultados intermediários para várias saídas finais. Esses pipelines complexos podem normalizar, modelar, agregar ou caracterizar dados. Embora você possa criar fluxos de dados complexos em consultas únicas usando expressões de tabela, scripts ou DAGs comuns, isso rapidamente se torna pesado, inconsistente e intratável. Insira as transformações.

As transformações dependem criticamente de uma das principais correntes ocultas deste livro: a orquestração. A orquestração combina muitas operações discretas, como transformações intermediárias, que armazenam dados temporária ou permanentemente para consumo por transformações ou serviços downstream. Cada vez mais, os pipelines de transformação abrangem não apenas várias tabelas e conjuntos de dados, mas também vários sistemas.

Transformações em lote

As transformações em lote são executadas em blocos discretos de dados, em contraste com as transformações de streaming, em que os dados são processados continuamente à medida que chegam. As transformações em lote podem ser executadas em uma programação fixa (por exemplo, diariamente, de hora em hora ou a cada 15 minutos) para oferecer suporte a relatórios contínuos, análises e modelos de ML. Nesta seção, você aprenderá vários padrões e tecnologias de transformação em lote.

Junções distribuídas A

idéia básica por trás das junções distribuídas é que precisamos quebrar uma ***junção lógica*** (a junção definida pela lógica de consulta) em ***junções de nós*** muito menores que são executadas em servidores individuais no cluster. Os padrões básicos de junção distribuída aplicam-se a MapReduce (discutido em “MapReduce”), BigQuery, Snowflake ou Spark, embora os detalhes de

Machine Translated by Google

armazenamento intermediário entre as etapas de processamento variam (no disco ou na memória). Na melhor das hipóteses, os dados em um lado da junção são pequenos o suficiente para caber em um único nó ***(junção de difusão).*** Frequentemente, é necessária uma ***junção de hash aleatória*** mais intensiva em recursos .

Ingressar na transmissão

Uma ***junção de broadcast*** é geralmente assimétrica, com uma grande tabela distribuída entre os nós e uma pequena tabela que pode caber facilmente em um único nó (Figura 8-17). O mecanismo de consulta "transmite" a tabela pequena (tabela A) para todos os nós, onde ela é unida às partes da tabela grande (tabela B). As junções de transmissão são muito menos intensivas em computação do que as junções de hash aleatórias.

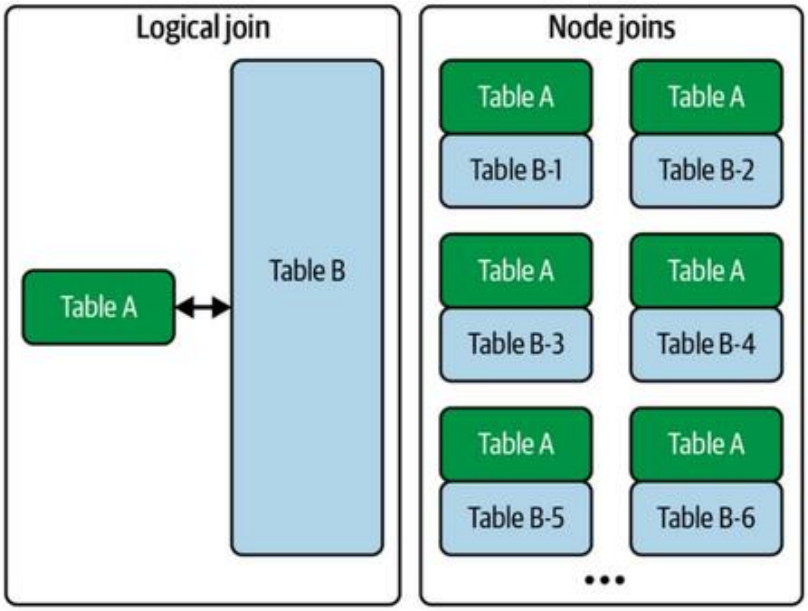


Figura 8-17. Em uma junção de transmissão, o mecanismo de consulta envia a tabela A para todos os nós do cluster para serem unidos às várias partes da tabela B

Na prática, a tabela A geralmente é uma tabela maior filtrada para baixo que o mecanismo de consulta coleta e transmite. Uma das principais prioridades nos otimizadores de consulta é a reordenação de junção. Com a aplicação antecipada de filtros e o movimento de pequenas tabelas para a esquerda (para junções à esquerda), muitas vezes é possível reduzir drasticamente a quantidade de dados processados em cada junção. A pré-filtragem de dados para criar junções de transmissão sempre que possível pode melhorar drasticamente o desempenho e reduzir o consumo de recursos.

Junção de hash aleatória

Se nenhuma tabela for pequena o suficiente para caber em um único nó, o mecanismo de consulta usará uma ***junção de hash aleatória.*** Na Figura 8-18, os mesmos nós são representados acima e abaixo da linha pontilhada. A área acima da linha pontilhada representa o particionamento inicial das tabelas A e B entre os nós. Em geral, esse particionamento não terá relação com a chave de junção. Um esquema de hash é usado para reparticionar dados por chave de junção.

Machine Translated by Google

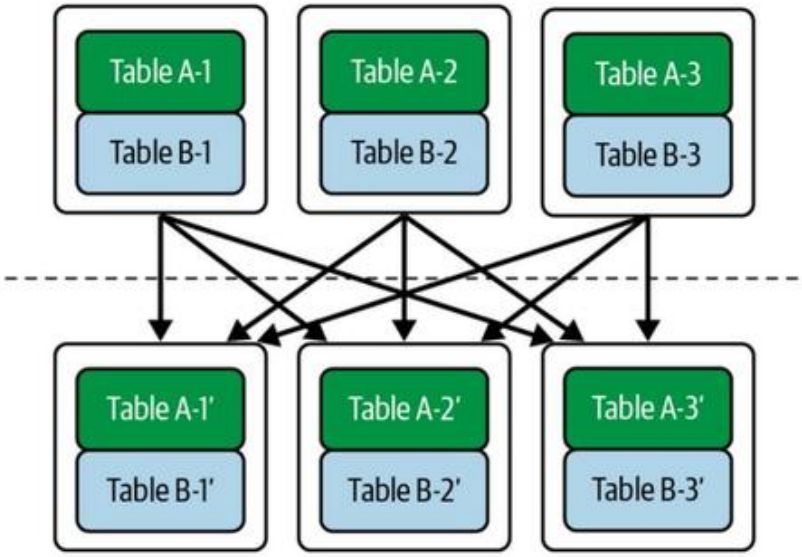


Figura 8-18. Junção de hash aleatória

Neste exemplo, o esquema de hash dividirá a chave de junção em três partes, com cada parte atribuída a um nó.
Os dados são então reordenados para o nó apropriado e as novas partições para as tabelas A e B em cada nó são unidas. As junções de hash aleatórias geralmente consomem mais recursos do que as junções de transmissão.

ETL, ELT e pipelines de dados

Conforme discutimos no Capítulo 3, um padrão de transformação difundido que data dos primeiros dias dos bancos de dados relacionais é um ETL em lote. O ETL tradicional depende de um sistema de transformação externo para extrair, transformar e limpar dados enquanto os prepara para um esquema de destino, como um data mart ou um esquema em estrela Kimball. Os dados transformados seriam então carregados em um sistema de destino, como um data warehouse, onde a análise de negócios poderia ser realizada.

O próprio padrão ETL foi impulsionado pelas limitações dos sistemas de origem e destino. A fase de extração tendia a ser um grande gargalo, com as restrições do RDBMS de origem limitando a taxa na qual os dados podiam ser extraídos.
Além disso, a transformação foi tratada em um sistema dedicado porque o sistema de destino tinha recursos extremamente limitados em capacidade de armazenamento e CPU.

Uma evolução agora popular de ETL é ELT. À medida que os sistemas de data warehouse cresceram em desempenho e capacidade de armazenamento, tornou-se comum simplesmente extrair dados brutos de um sistema de origem, importá-los para um data warehouse com transformação mínima e depois limpá-los e transformá-los diretamente no sistema de warehouse. (Veja nossa discussão sobre data warehouses no Capítulo 3 para uma discussão mais detalhada sobre a diferença entre ETL e ELT.)

Uma segunda noção ligeiramente diferente de ELT foi popularizada com o surgimento dos data lakes. Nesta versão, os dados não são transformados no momento em que são carregados. De fato, grandes quantidades de dados podem ser carregadas sem nenhuma preparação e nenhum plano. A suposição é que a etapa de transformação acontecerá em algum tempo futuro indeterminado. A ingestão de dados sem um plano é uma ótima receita para um pântano de dados. Como diz Inmon:

> ***Eu sempre fui um fã de ETL devido ao fato de que ETL força você a transformar dados antes de colocá-los em um formulário onde você possa trabalhar com eles. Mas algumas organizações querem simplesmente pegar os dados, colocá-los em um banco de dados e depois fazer a transformação... Já vi muitos casos em que a organização diz, ah, vamos apenas colocar os dados e transformá-los mais tarde. E adivinha? Seis meses depois, esses dados nunca foram tocados.***[14]

Também vimos que a linha entre ETL e ELT pode ficar um pouco embaçada em um ambiente de data lakehouse. Com o armazenamento de objetos como camada base, não fica mais claro o que está no banco de dados e o que está fora do banco de dados. A ambiguidade é ainda mais exacerbada com o surgimento da federação de dados, virtualização e tabelas ativas. (Discutiremos esses tópicos mais adiante nesta seção.)

Cada vez mais, sentimos que os termos ***ETL*** e ***ELT*** devem ser aplicados apenas no nível micro (dentro de pipelines de transformação individuais) e não no nível macro (para descrever um padrão de transformação para um todo

Machine Translated by Google

organização). As organizações não precisam mais padronizar em ETL ou ELT, mas podem se concentrar na aplicação da técnica adequada caso a caso à medida que constroem pipelines de dados.

SQL e ferramentas de transformação de código de uso geral

Neste momento, a distinção entre sistemas de transformação baseados em SQL e não baseados em SQL parece um tanto sintética. Desde a introdução do Hive na plataforma Hadoop, o SQL tornou-se um cidadão de primeira classe no ecossistema de big data. Por exemplo, o Spark SQL foi um recurso inicial do Apache Spark. Estruturas que priorizam o streaming, como Kafka, Flink e Beam, também suportam SQL, com recursos e funcionalidades variados.

É mais apropriado pensar em ferramentas somente SQL versus aquelas que suportam paradigmas de programação de propósito geral mais poderosos. As ferramentas de transformação somente SQL abrangem uma ampla variedade de opções proprietárias e de código aberto.

O SQL é declarativo, mas ainda pode criar fluxos de trabalho de dados complexos

Muitas vezes ouvimos SQL descartado porque “não é processual”. Isso é tecnicamente correto. SQL é uma linguagem declarativa: em vez de codificar um procedimento de processamento de dados, os escritores SQL estipulam as características de seus dados finais em linguagem de teoria de conjuntos; o compilador e o otimizador SQL determinam as etapas necessárias para colocar os dados nesse estado.

As pessoas às vezes insinuam que, como o SQL não é procedural, ele não pode construir pipelines complexos. Isto é falso. O SQL pode ser usado efetivamente para criar DAGs complexos usando expressões de tabela comuns, scripts SQL ou uma ferramenta de orquestração.

Para ser claro, o SQL tem limites, mas muitas vezes vemos engenheiros fazendo coisas em Python e Spark que poderiam ser feitas com mais facilidade e eficiência no SQL. Para ter uma ideia melhor das compensações sobre as quais estamos falando, vejamos alguns exemplos de Spark e SQL.

Exemplo: quando evitar SQL para transformações em lote no Spark

Ao determinar se deve usar o código nativo Spark ou PySpark em vez do Spark SQL ou outro mecanismo SQL, faça a si mesmo as seguintes perguntas:

1. Quão difícil é codificar a transformação em SQL?
2. Quão legível e sustentável será o código SQL resultante?
3. Se algum código de transformação for enviado para uma biblioteca personalizada para reutilização futura no organização?

Em relação à questão 1, muitas transformações codificadas no Spark podem ser realizadas em instruções SQL bastante simples. Por outro lado, se a transformação não for realizável em SQL, ou se for extremamente difícil de implementar, o Spark nativo é uma opção melhor. Por exemplo, podemos implementar o radical de palavras em SQL colocando sufixos de palavras em uma tabela, unindo-os a essa tabela, usando uma função de análise sintática para encontrar sufixos em palavras e, em seguida, reduzindo a palavra ao seu radical usando uma função de substring. No entanto, isso soa como um processo extremamente complexo com vários casos extremos a serem considerados. Uma linguagem de programação procedural mais poderosa se encaixa melhor aqui.

A questão 2 está intimamente relacionada. A consulta de raiz de palavras não será legível nem sustentável.

Em relação à questão 3, uma das maiores limitações do SQL é que ele não inclui uma noção natural de bibliotecas ou código reutilizável. Uma exceção é que alguns mecanismos SQL permitem que você mantenha funções definidas pelo usuário (UDFs) como objetos dentro de um banco de dados. No entanto, eles não estão comprometidos com um repositório Git sem um sistema CI/CD externo para gerenciar a implantação. Além disso, o SQL não tem uma boa noção de reutilização para componentes de consulta mais complexos. É claro que as bibliotecas reutilizáveis são fáceis de criar no Spark e no PySpark.

Acrescentaremos que é possível reciclar o SQL de duas maneiras. Primeiro, podemos reutilizar facilmente os ***resultados*** de uma consulta SQL confirmando em uma tabela ou criando uma visualização. Esse processo geralmente é mais bem tratado em uma ferramenta de orquestração, como o Airflow, para que as consultas downstream possam ser iniciadas assim que a consulta de origem for concluída. Em segundo lugar, a Data Build Tool (dbt) facilita a reutilização de instruções SQL e oferece uma linguagem de modelagem que facilita a personalização.

Machine Translated by Google

Exemplo: otimizando o Spark e outras estruturas de processamento

Os acólitos do Spark frequentemente reclamam que o SQL não lhes dá controle sobre o processamento de dados. O mecanismo SQL pega suas instruções, as otimiza e as compila em suas etapas de processamento. (Na prática, a otimização pode acontecer antes ou depois da compilação, ou ambos.)

Esta é uma reclamação justa, mas existe um corolário. Com o Spark e outras estruturas de processamento de código pesado, o escritor de código se torna responsável por grande parte da otimização que é tratada automaticamente em um mecanismo baseado em SQL. A API Spark é poderosa e complexa, o que significa que não é tão fácil identificar candidatos para reordenação, combinação ou decomposição. Ao adotar o Spark, as equipes de engenharia de dados precisam se envolver ativamente com os problemas de otimização do Spark, especialmente para trabalhos caros e de longa duração. Isso significa criar experiência em otimização na equipe e ensinar engenheiros individuais a otimizar.

Algumas coisas de nível superior a serem lembradas ao codificar no Spark nativo:

1. Filtre cedo e com frequência.
2. Confie fortemente na API central do Spark e aprenda a entender a maneira nativa do Spark de fazer as coisas. Tente confiar em bibliotecas públicas bem mantidas se a API do Spark nativa não oferecer suporte ao seu caso de uso. Um bom código Spark é substancialmente declarativo.
3. Tenha cuidado com UDFs.
4. Considere misturar SQL.

A recomendação 1 também se aplica à otimização do SQL, com a diferença de que o Spark pode não conseguir reordenar algo que o SQL manipularia para você automaticamente. O Spark é uma estrutura de processamento de big data, mas quanto menos dados você precisar processar, menos recursos pesados e mais desempenho será seu código.

Se você estiver escrevendo um código personalizado extremamente complexo, faça uma pausa e determine se há uma maneira mais nativa de fazer o que você está tentando realizar. Aprenda a entender o Spark idiomático lendo exemplos públicos e trabalhando em tutoriais. Existe algo na API do Spark que pode realizar o que você está tentando fazer? Existe uma biblioteca pública bem mantida e otimizada que possa ajudar?

A terceira recomendação é crucial para o PySpark. Em geral, o PySpark é um wrapper de API para Scala Spark. Seu código envia o trabalho para o código Scala nativo em execução na JVM chamando a API. A execução de UDFs do Python força os dados a serem passados para o Python, onde o processamento é menos eficiente. Se você estiver usando UDFs do Python, procure uma maneira mais nativa do Spark de realizar o que está fazendo. Volte para a recomendação: existe uma maneira de realizar sua tarefa usando a API principal ou uma biblioteca bem mantida? Se você precisar usar UDFs, considere reescrevê-los em Scala ou Java para melhorar o desempenho.

Quanto à recomendação 4, o uso do SQL nos permite aproveitar o otimizador Spark Catalyst, que pode extrair mais desempenho do que com o código Spark nativo. O SQL geralmente é mais fácil de escrever e manter para operações simples. A combinação de Spark e SQL nativos nos permite obter o melhor dos dois mundos — funcionalidade poderosa e de uso geral combinada com simplicidade, quando aplicável.

Muitos dos conselhos de otimização nesta seção são bastante genéricos e também se aplicam ao Apache Beam, por exemplo. O ponto principal é que as APIs de processamento de dados programáveis exigem um pouco mais de sutileza de otimização do que o SQL, que talvez seja menos poderoso e mais fácil de usar.

Atualizar padrões

Como as transformações persistem nos dados, geralmente atualizamos os dados persistentes no local. A atualização de dados é um grande problema para as equipes de engenharia de dados, especialmente à medida que fazem a transição entre as tecnologias de engenharia de dados. Estamos discutindo DML em SQL, que apresentamos anteriormente neste capítulo.

Mencionamos várias vezes ao longo do livro que o conceito original de data lake não levava em conta a atualização de dados. Isso agora parece absurdo por várias razões. A atualização de dados tem sido uma parte fundamental do tratamento dos resultados da transformação de dados, embora a comunidade de big data a tenha rejeitado. É tolice repetir significativamente

Machine Translated by Google

quantidade de trabalho porque não temos recursos de atualização. Assim, o conceito de data lakehouse agora é construído em atualizações.
Além disso, o GDPR e outros padrões de exclusão de dados agora ***exigem*** que as organizações excluam dados de maneira direcionada, mesmo em conjuntos de dados brutos.

Vamos considerar vários padrões básicos de atualização.

Truncar e recarregar

Truncar é um padrão de atualização que não atualiza nada. Ele simplesmente limpa os dados antigos. Em um padrão de atualização truncar e recarregar, uma tabela é limpa de dados e as transformações são executadas novamente e carregadas nessa tabela, gerando efetivamente uma nova versão da tabela.

Inserir somente

Inserir somente insere novos registros sem alterar ou excluir registros antigos. Padrões somente de inserção podem ser usados para manter uma exibição atual dos dados, por exemplo, se novas versões de registros forem inseridas sem excluir registros antigos.
Uma consulta ou exibição pode apresentar o estado atual dos dados localizando o registro mais recente por chave primária. Observe que os bancos de dados colunares normalmente não impõem chaves primárias. A chave primária seria uma construção usada pelos engenheiros para manter uma noção do estado atual da tabela. A desvantagem dessa abordagem é que pode ser extremamente caro do ponto de vista computacional encontrar o registro mais recente no momento da consulta. Alternativamente, podemos usar uma visão materializada (abordada mais adiante no capítulo), uma tabela somente de inserção que mantém todos os registros e uma tabela de destino truncar e recarregar que mantém o estado atual para servir dados.

AVISO

Ao inserir dados em um banco de dados OLAP orientado a colunas, o problema comum é que os engenheiros que fazem a transição de sistemas orientados a linhas tentam usar inserções de uma única linha. Esse antipadrão coloca uma carga enorme no sistema. Também faz com que os dados sejam gravados em muitos arquivos separados; isso é extremamente ineficiente para leituras subsequentes e os dados devem ser reclusterizados posteriormente. Em vez disso, recomendamos o carregamento de dados de forma periódica em microlotes ou lotes.

Mencionaremos uma exceção ao conselho para não inserir com frequência: a arquitetura Lambda aprimorada usada pelo BigQuery e Apache Druid, que hibridiza um buffer de streaming com armazenamento colunar. Exclusões e atualizações in-loco ainda podem ser caras, como discutiremos a seguir.

Excluir

A exclusão é crítica quando um sistema de origem exclui dados e atende às alterações regulatórias recentes. Em sistemas colunares e data lakes, as exclusões são mais caras do que as inserções.

Ao excluir dados, considere se você precisa fazer uma exclusão física ou temporária. Uma ***exclusão*** definitiva remove permanentemente um registro de um banco de dados, enquanto uma ***exclusão reversível*** marca o registro como "excluído". Exclusões definitivas são úteis quando você precisa remover dados por motivos de desempenho (digamos, uma tabela é muito grande) ou se houver um motivo legal ou de conformidade para isso.
As exclusões temporárias podem ser usadas quando você não deseja excluir um registro permanentemente, mas também deseja filtrá-lo dos resultados da consulta.

Uma terceira abordagem para exclusões está intimamente relacionada às exclusões reversíveis: ***inserir exclusão*** insere um novo registro com um sinalizador excluído sem modificar a versão anterior do registro. Isso nos permite seguir um padrão somente de inserção, mas ainda leva em conta as exclusões. Apenas observe que nossa consulta para obter o estado mais recente da tabela fica um pouco mais complicada. Agora devemos desduplicar, encontrar a versão mais recente de cada registro por chave e não mostrar nenhum registro cuja versão mais recente mostre excluído.

Upsert/ merge

Desses padrões de atualização, os padrões upsert e merge são os que causam mais problemas para as equipes de engenharia de dados, especialmente para pessoas que estão fazendo a transição de data warehouses baseados em linhas para sistemas em nuvem baseados em colunas.

Machine Translated by Google

O upserting usa um conjunto de registros de origem e procura correspondências em uma tabela de destino usando uma chave primária ou outra condição lógica. (Novamente, é responsabilidade da equipe de engenharia de dados gerenciar essa chave primária executando consultas apropriadas. A maioria dos sistemas colunares não impõe exclusividade.) Quando ocorre uma correspondência de chave, o registro de destino é atualizado (substituído pelo novo registro). Quando não existe correspondência, o banco de dados insere o novo registro. O padrão de mesclagem adiciona a isso a capacidade de excluir registros.

Então qual é o problema? O padrão upsert/merge foi originalmente projetado para bancos de dados baseados em linha. Em bancos de dados baseados em linhas, as atualizações são um processo natural: o banco de dados procura o registro em questão e o altera no lugar.

Por outro lado, os sistemas baseados em arquivos não oferecem suporte a atualizações de arquivos in-loco. Todos esses sistemas utilizam cópia na gravação (COW). Se um registro em um arquivo for alterado ou excluído, todo o arquivo deverá ser reescrito com as novas alterações.

Isso é parte do motivo pelo qual os primeiros adeptos de big data e data lakes rejeitaram as atualizações: gerenciar arquivos e atualizações parecia muito complicado. Assim, eles simplesmente usaram um padrão somente de inserção e presumiram que os consumidores de dados determinariam o estado atual dos dados no momento da consulta ou em transformações downstream. Na realidade, bancos de dados colunares como o Vertica há muito oferecem suporte a atualizações in-loco, ocultando a complexidade do COW dos usuários.
Eles verificam arquivos, alteram os registros relevantes, gravam novos arquivos e alteram ponteiros de arquivo para a tabela. Os principais data warehouses em nuvem colunar suportam atualizações e fusões, embora os engenheiros devam investigar o suporte a atualizações se considerarem adotar uma tecnologia exótica.

Há algumas coisas importantes para entender aqui. Embora os sistemas de dados colunares distribuídos suportem comandos de atualização nativos, as mesclagens têm um custo: o impacto no desempenho da atualização ou exclusão de um único registro pode ser bastante alto. Por outro lado, as mesclagens podem ser extremamente eficientes para grandes conjuntos de atualizações e podem até superar os bancos de dados transacionais.

Além disso, é importante entender que COW raramente envolve reescrever a tabela inteira. Dependendo do sistema de banco de dados em questão, o COW pode operar em várias resoluções (partição, cluster, bloco). Para realizar atualizações de desempenho, concentre-se no desenvolvimento de uma estratégia apropriada de particionamento e clustering com base em suas necessidades e nas entranhas do banco de dados em questão.

Assim como nas inserções, tenha cuidado com a frequência de atualização ou mesclagem. Vimos muitas equipes de engenharia fazerem a transição entre os sistemas de banco de dados e tentarem executar mesclagens quase em tempo real do CDC, exatamente como faziam em seu sistema antigo. Simplesmente não funciona. Não importa quão bom seja seu sistema CDC, essa abordagem deixará a maioria dos data warehouses colunares de joelhos. Vimos os sistemas ficarem semanas atrasados nas atualizações, onde uma abordagem que simplesmente se fundisse a cada hora faria muito mais sentido.

Podemos usar várias abordagens para aproximar os bancos de dados colunares do tempo real. Por exemplo, o BigQuery nos permite fazer streaming de novos registros em uma tabela e, em seguida, oferece suporte a visualizações materializadas especializadas que apresentam uma visualização de tabela desduplicada eficiente e quase em tempo real. O Druid usa armazenamento de duas camadas e SSDs para oferecer suporte a consultas ultrarrápidas em tempo real.

Atualizações de esquema

Os dados têm entropia e podem mudar sem o seu controle ou consentimento. As fontes de dados externas podem alterar seu esquema ou as equipes de desenvolvimento de aplicativos podem adicionar novos campos ao esquema. Uma vantagem dos sistemas colunares sobre os sistemas baseados em linhas é que, embora a atualização dos dados seja mais difícil, a atualização do esquema é mais fácil. As colunas geralmente podem ser adicionadas, excluídas e renomeadas.

Apesar dessas melhorias tecnológicas, o gerenciamento prático do esquema organizacional é mais desafiador.
Algumas atualizações de esquema serão automatizadas? (Esta é a abordagem que o Fivetran usa ao replicar a partir de fontes.)
Por mais conveniente que isso pareça, existe o risco de que as transformações downstream sejam interrompidas.

Existe um processo de solicitação de atualização de esquema simples? Suponha que uma equipe de ciência de dados queira adicionar uma coluna de uma fonte que não foi ingerida anteriormente. Como será o processo de revisão? Os processos a jusante serão interrompidos? (Existem consultas que executam SELECT * em vez de usar seleção de coluna explícita? Isso geralmente é uma prática ruim em bancos de dados colunares.) Quanto tempo levará para implementar a alteração? É possível criar uma bifurcação de tabela — ou seja, uma nova versão de tabela específica para este projeto?

Machine Translated by Google

Uma nova opção interessante surgiu para dados semiestruturados. Tomando emprestada uma ideia de armazenamentos de documentos, muitos data warehouses na nuvem agora oferecem suporte a tipos de dados que codificam dados JSON arbitrários. Uma abordagem armazena JSON bruto em um campo enquanto armazena dados acessados com frequência em campos nivelados adjacentes. Isso ocupa espaço de armazenamento adicional, mas permite a conveniência de dados achatados, com a flexibilidade de dados semiestruturados para usuários avançados.
Os dados acessados com frequência no campo JSON podem ser adicionados diretamente ao esquema ao longo do tempo.

Essa abordagem funciona extremamente bem quando os engenheiros de dados precisam ingerir dados de um armazenamento de documentos de aplicativo com um esquema que muda com frequência. Dados semiestruturados disponíveis como cidadãos de primeira classe em data warehouses são extremamente flexíveis e abrem novas oportunidades para analistas de dados e cientistas de dados, pois os dados não são mais restritos a linhas e colunas.

A disputa *de dados* A

disputa de dados pega dados confusos e malformados e os transforma em dados úteis e limpos. Isso geralmente é um processo de transformação em lote.

A disputa de dados tem sido uma grande fonte de dor e segurança no trabalho para desenvolvedores de ETL. Por exemplo, suponha que os desenvolvedores recebam dados EDI (consulte o Capítulo 7) de um parceiro comercial sobre transações e faturas, potencialmente uma combinação de dados estruturados e texto. O processo típico de disputar esses dados envolve primeiro tentar ingeri-los. Freqüentemente, os dados são tão malformados que uma boa quantidade de pré-processamento de texto está envolvida. Os desenvolvedores podem optar por ingerir os dados como uma única tabela de campo de texto — uma linha inteira ingerida como um único campo. Os desenvolvedores então começam a escrever consultas para analisar e separar os dados. Com o tempo, eles descobrem anomalias de dados e casos extremos.
Eventualmente, eles vão obter os dados em forma aproximada. Só então o processo de transformação downstream pode começar.

As ferramentas de organização de dados visam simplificar partes significativas desse processo. Essas ferramentas geralmente afastam os engenheiros de dados porque afirmam não haver código, o que parece pouco sofisticado. Preferimos pensar nas ferramentas de manipulação de dados como ambientes de desenvolvimento integrados (IDEs) para dados malformados. Na prática, os engenheiros de dados gastam muito tempo analisando dados desagradáveis; as ferramentas de automação permitem que os engenheiros de dados gastem tempo em tarefas mais interessantes. As ferramentas de disputa também podem permitir que os engenheiros entreguem alguns trabalhos de análise e ingestão aos analistas.

As ferramentas gráficas de manipulação de dados normalmente apresentam uma amostra de dados em uma interface visual, com tipos inferidos, estatísticas incluindo distribuições, dados anômalos, valores discrepantes e nulos. Os usuários podem adicionar etapas de processamento para corrigir problemas de dados. Uma etapa pode fornecer instruções para lidar com dados digitados incorretamente, dividir um campo de texto em várias partes ou unir-se a uma tabela de pesquisa.

Os usuários podem executar as etapas em um conjunto de dados completo quando o trabalho completo estiver pronto. O trabalho normalmente é enviado para um sistema de processamento de dados escalável, como o Spark, para grandes conjuntos de dados. Depois que o trabalho for executado, ele retornará erros e exceções não tratadas. O usuário pode refinar ainda mais a receita para lidar com esses valores discrepantes.

É altamente recomendável que engenheiros aspirantes e experientes experimentem ferramentas de disputa; os principais provedores de nuvem vendem sua versão de ferramentas de manipulação de dados e muitas opções de terceiros estão disponíveis. Os engenheiros de dados podem descobrir que essas ferramentas simplificam significativamente certas partes de seus trabalhos. Organizacionalmente, as equipes de engenharia de dados podem querer considerar o treinamento de especialistas em disputa de dados se eles ingerirem frequentemente dados novos e confusos
fontes.

Exemplo: transformação de dados no Spark

Vejamos um exemplo prático e concreto de transformação de dados. Suponha que construímos um pipeline que ingere dados de três fontes de API no formato JSON. Essa etapa de ingestão inicial é tratada no Airflow. Cada fonte de dados obtém seu prefixo (caminho de arquivo) em um bucket do S3.

O Airflow aciona um trabalho do Spark chamando uma API. Esse trabalho do Spark ingere cada uma das três fontes em um dataframe, convertendo os dados em um formato relacional, com aninhamento em determinadas colunas. O trabalho do Spark combina as três fontes em uma única tabela e filtra os resultados com uma instrução SQL. Os resultados são finalmente gravados em uma tabela Delta Lake formatada em Parquet armazenada no S3.

Na prática, o Spark cria um DAG de etapas com base no código que escrevemos para ingerir, juntar e gravar os dados. A ingestão básica de dados acontece na memória do cluster, embora uma das fontes de dados seja grande o suficiente

Machine Translated by Google

que deve ser derramado no disco durante o processo de ingestão. (Esses dados são gravados no armazenamento do cluster; eles serão recarregados na memória para as etapas de processamento subsequentes.)

A junção requer uma operação de embaralhamento. Uma chave é usada para redistribuir dados pelo cluster; mais uma vez, ocorre um spill to disk à medida que os dados são gravados em cada nó. A transformação SQL filtra as linhas na memória e descarta as linhas não utilizadas. Por fim, o Spark converte os dados no formato Parquet, os compacta e os grava de volta no S3. O Airflow liga periodicamente para o Spark para ver se o trabalho foi concluído. Depois de confirmar que o trabalho foi concluído, ele marca o Airflow DAG completo como concluído. (Observe que temos duas construções de DAG aqui, um Airflow DAG e um DAG específico para o trabalho do Spark.)

Lógica de negócios e dados derivados

Um dos casos de uso mais comuns para transformação é renderizar a lógica de negócios. Colocamos essa discussão em transformações em lote porque é onde esse tipo de transformação acontece com mais frequência, mas observe que isso também pode acontecer em um pipeline de streaming.

Suponha que uma empresa use vários cálculos internos especializados de lucro. Uma versão pode analisar os lucros antes dos custos de marketing e outra pode analisar o lucro após subtrair os custos de marketing. Mesmo que isso pareça ser um exercício de contabilidade simples, cada uma dessas métricas é altamente complexa para renderizar.

O lucro antes dos custos de marketing pode precisar contabilizar pedidos fraudulentos; determinar uma estimativa de lucro razoável para o dia útil anterior implica estimar qual porcentagem de receita e lucro será perdida para pedidos cancelados nos próximos dias, à medida que a equipe de fraude investiga pedidos suspeitos. Existe um sinalizador especial no banco de dados que indica um pedido com alta probabilidade de fraude ou que foi cancelado automaticamente? A empresa supõe que uma certa porcentagem de pedidos será cancelada por causa de fraude antes mesmo que o processo de avaliação de risco de fraude tenha sido concluído para pedidos específicos?

Para lucros após os custos de marketing, devemos levar em conta todas as complexidades da métrica anterior, mais os custos de marketing atribuídos ao pedido específico. A empresa tem um modelo de atribuição ingênuo – por exemplo, custos de marketing atribuídos a itens ponderados por preço? Os custos de marketing também podem ser atribuídos por departamento ou categoria de item ou, nas organizações mais sofisticadas, por item individual com base nos cliques nos anúncios do usuário.

A transformação da lógica de negócios que gera essa versão diferenciada de lucro deve integrar todas as sutilezas da atribuição — ou seja, um modelo que vincule pedidos a anúncios específicos e custos de publicidade. Os dados de atribuição são armazenados nas entranhas dos scripts ETL ou são extraídos de uma tabela que é gerada automaticamente a partir dos dados da plataforma de anúncios?

Esse tipo de dados de relatório é um exemplo por excelência de dados ***derivados*** – dados calculados a partir de outros dados armazenados em um sistema de dados. Os críticos de dados derivados apontarão que é um desafio para o ETL manter a consistência nas métricas derivadas. Por exemplo, se a empresa atualizar seu modelo de atribuição, essa alteração pode precisar ser mesclada em muitos scripts ETL para geração de relatórios. (Os scripts ETL são conhecidos por quebrar o princípio DRY.) A atualização desses scripts ETL é um processo manual e trabalhoso, envolvendo experiência de domínio em lógica de processamento e alterações anteriores. Os scripts atualizados também devem ser validados para consistência e precisão.

Do nosso ponto de vista, essas são críticas legítimas, mas não necessariamente muito construtivas, porque a alternativa aos dados derivados neste caso é igualmente desagradável. Os analistas precisarão executar suas consultas de relatórios se os dados de lucro não estiverem armazenados no data warehouse, incluindo a lógica de lucro. A atualização de scripts ETL complexos para representar as alterações na lógica de negócios com precisão é uma tarefa trabalhosa e trabalhosa, mas fazer com que os analistas atualizem suas consultas de relatórios de forma consistente é quase impossível.

Uma alternativa interessante é empurrar a lógica de negócios para uma ***camada de métricas,*** mas ainda aproveitar o data warehouse ou outra ferramenta para fazer o trabalho pesado computacional. Uma camada de métricas codifica a lógica de negócios e permite que analistas e usuários de painéis criem análises complexas a partir de uma biblioteca de métricas definidas. A camada de métricas gera consultas a partir das métricas e as envia para o banco de dados. Discutimos camadas semânticas e métricas com mais detalhes no Capítulo 9.

MapReduce

Machine Translated by Google

Nenhuma discussão sobre transformação em lote pode ser concluída sem tocar em MapReduce. Isso não ocorre porque o MapReduce é amplamente usado por engenheiros de dados atualmente. O MapReduce foi o padrão definidor de transformação de dados em lote da era do big data, ainda influencia muitos sistemas distribuídos que os engenheiros de dados usam hoje e é útil para os engenheiros de dados entenderem em um nível básico. MapReduce foi introduzido pelo Google na sequência de seu artigo sobre GFS. Foi inicialmente o padrão de processamento de fato do Hadoop, a tecnologia analógica de código aberto do GFS que apresentamos no Capítulo 6.

Um trabalho MapReduce simples consiste em uma coleção de tarefas de mapa que lêem blocos de dados individuais espalhados pelos nós, seguidos por um shuffle que redistribui os dados de resultados pelo cluster e uma etapa de redução que agrega dados em cada nó. Por exemplo, suponha que queiramos executar a seguinte consulta SQL:

```
SELECT COUNT(*), user_id
DE user_events
GROUP BY user_id;
```

Os dados da tabela são distribuídos entre nós em blocos de dados; o trabalho MapReduce gera uma tarefa de mapa por bloco. Cada tarefa de mapa executa essencialmente a consulta em um único bloco, ou seja, gera uma contagem para cada ID de usuário que aparece no bloco. Enquanto um bloco pode conter centenas de megabytes, a tabela completa pode ter petabytes de tamanho. No entanto, a parte do mapa do trabalho é um exemplo quase perfeito de paralelismo embaraçoso; a taxa de varredura de dados em todo o cluster basicamente é dimensionada linearmente com o número de nós.

Em seguida, precisamos agregar (reduzir) para reunir os resultados do cluster completo. Não estamos reunindo resultados em um único nó; em vez disso, redistribuímos os resultados por chave para que cada chave termine em um e apenas um nó. Esta é a etapa de embaralhamento, que geralmente é executada usando um algoritmo de hash nas teclas. Uma vez que os resultados do mapa tenham sido embaralhados, somamos os resultados para cada chave. Os pares chave/contagem podem ser gravados no disco local no nó onde são calculados.
Coletamos os resultados armazenados nos nós para visualizar os resultados completos da consulta.

Os trabalhos MapReduce do mundo real podem ser muito mais complexos do que descrevemos aqui. Uma consulta complexa que filtra com uma cláusula WHERE une três tabelas e aplica uma função de janela que consistiria em muitos estágios de mapeamento e redução.

Após MapReduce

O modelo MapReduce original do Google é extremamente poderoso, mas agora é visto como excessivamente rígido. Ele utiliza várias tarefas efêmeras de curta duração que lêem e gravam no disco. Em particular, nenhum estado intermediário é preservado na memória; todos os dados são transferidos entre tarefas armazenando-os em disco ou enviando-os pela rede. Isso simplifica o gerenciamento do estado e do fluxo de trabalho e minimiza o consumo de memória, mas também pode aumentar a utilização da largura de banda do disco e aumentar o tempo de processamento.

O paradigma MapReduce foi construído em torno da ideia de que a capacidade e a largura de banda do disco magnético eram tão baratas que fazia sentido simplesmente lançar uma enorme quantidade de disco nos dados para realizar consultas ultrarrápidas. Isso funcionou até certo ponto; O MapReduce estabeleceu repetidamente registros de processamento de dados durante os primeiros dias do Hadoop.

No entanto, vivemos em um mundo pós-MapReduce por algum tempo. O processamento pós-MapReduce não descarta verdadeiramente o MapReduce; ele ainda inclui os elementos map, shuffle e reduce, mas relaxa as restrições do MapReduce para permitir o armazenamento em cache na memória. Lembre-se de que a RAM é muito mais rápida que SSD e HDDs em velocidade de transferência e tempo de busca. Persistindo até mesmo uma pequena quantidade de dados criteriosamente escolhidos na memória pode acelerar drasticamente tarefas específicas de processamento de dados e esmagar totalmente o desempenho do MapReduce.

Por exemplo, Spark, BigQuery e várias outras estruturas de processamento de dados foram projetadas em torno do processamento na memória. Essas estruturas tratam os dados como um conjunto distribuído que reside na memória. Se os dados estourarem a memória disponível, isso causará um ***transbordamento no disco.*** O disco é tratado como uma camada de armazenamento de dados de segunda classe para processamento, embora ainda seja altamente valioso.

A nuvem é um dos impulsionadores para a adoção mais ampla do cache de memória; é muito mais eficaz alugar memória durante um trabalho de processamento específico do que possuí-la 24 horas por dia. Os avanços no aproveitamento da memória para transformações continuarão a gerar ganhos no futuro próximo.

Machine Translated by Google

Visualizações materializadas, federação e virtualização de consultas

Nesta seção, examinamos várias técnicas que virtualizam os resultados da consulta apresentando-os como objetos semelhantes a tabelas.
Essas técnicas podem se tornar parte de um pipeline de transformação ou ficar logo antes do consumo de dados do usuário final.

Visualizações

Primeiro, vamos revisar as visualizações para definir o cenário para visualizações materializadas. Uma ***visão*** é um objeto de banco de dados que podemos selecionar como qualquer outra tabela. Na prática, uma visão é apenas uma consulta que faz referência a outras tabelas. Quando selecionamos de uma visualização, esse banco de dados cria uma nova consulta que combina a subconsulta da visualização com nossa consulta. O otimizador de consulta otimiza e executa a consulta completa.

As visualizações desempenham várias funções em um banco de dados. Primeiro, as exibições podem servir a uma função de segurança. Por exemplo, as exibições podem selecionar apenas colunas específicas e filtrar linhas, fornecendo acesso restrito aos dados. Várias visualizações podem ser criadas para funções de trabalho, dependendo do acesso aos dados do usuário.

Em segundo lugar, uma visualização pode ser usada para fornecer uma imagem desduplicada atual dos dados. Se estivermos usando um padrão somente de inserção, uma exibição poderá ser usada para retornar uma versão desduplicada de uma tabela mostrando apenas a versão mais recente de cada registro.

Terceiro, as visualizações podem ser usadas para apresentar padrões comuns de acesso a dados. Suponha que os analistas de marketing devam executar frequentemente uma consulta que una cinco tabelas. Poderíamos criar uma visão que una essas cinco tabelas em uma tabela ampla.
Os analistas podem então escrever consultas que filtram e agregam sobre essa visualização.

Visualizações materializadas

Mencionamos visualizações materializadas em nossa discussão anterior sobre cache de consulta. Uma desvantagem potencial das visualizações (não materializadas) é que elas não fazem nenhuma pré-computação. No exemplo de uma exibição que une cinco tabelas, essa união deve ser executada sempre que um analista de marketing executa uma consulta nessa exibição e a união pode ser extremamente cara.

Uma visão materializada faz parte ou todo o cálculo da visão com antecedência. Em nosso exemplo, uma visualização materializada pode salvar os cinco resultados de junção de tabela sempre que ocorrer uma alteração nas tabelas de origem. Então, quando um usuário faz referência à exibição, ele está consultando os dados pré-juntados. Uma visão materializada é uma etapa de transformação de fato, mas o banco de dados gerencia a execução por conveniência.

As exibições materializadas também podem desempenhar uma função significativa de otimização de consulta, dependendo do banco de dados, mesmo para consultas que não as referenciam diretamente. Muitos otimizadores de consulta podem identificar consultas que “se parecem” com uma visualização materializada. Um analista pode executar uma consulta que usa um filtro que aparece em uma visualização materializada. O otimizador reescreverá a consulta para selecionar os resultados pré-computados.

Visualizações materializadas combináveis

Em geral, as visualizações materializadas não permitem composição, ou seja, uma visualização materializada não pode selecionar a partir de outra visualização materializada. No entanto, vimos recentemente o surgimento de ferramentas que suportam esse recurso. Por exemplo, Databricks introduziu a noção de ***tabelas ativas.*** Cada tabela é atualizada à medida que os dados chegam das origens.
Os dados fluem para as tabelas subsequentes de forma assíncrona.

Consultas federadas

As ***consultas federadas*** são um recurso de banco de dados que permite que um banco de dados OLAP selecione uma fonte de dados externa, como armazenamento de objeto ou RDBMS. Por exemplo, digamos que você precise combinar dados no armazenamento de objetos e várias tabelas nos bancos de dados MySQL e PostgreSQL. Seu data warehouse pode emitir uma consulta federada para essas fontes e retornar os resultados combinados (Figura 8-19).

Machine Translated by Google

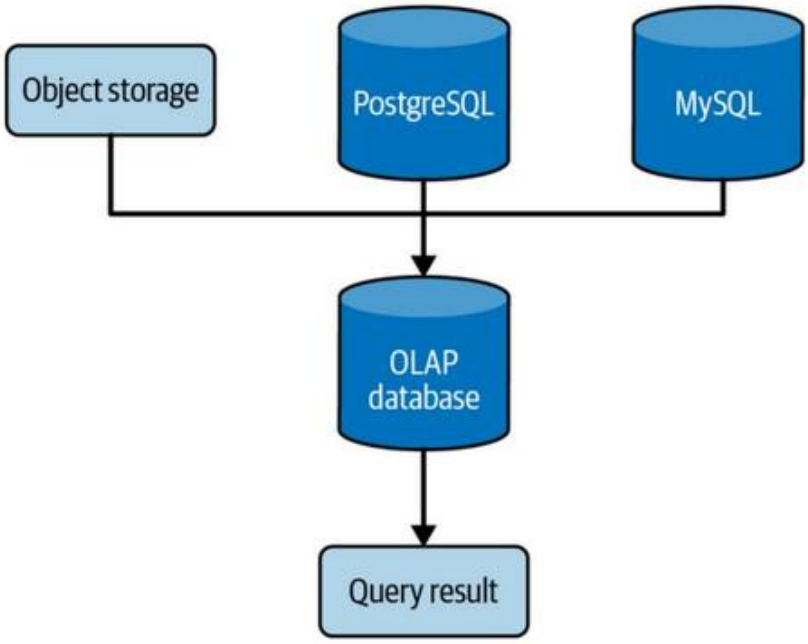


Figura 8-19. Um banco de dados OLAP emite uma consulta federada que obtém dados do armazenamento de objetos, MySQL e PostgreSQL e retorna um resultado de consulta com os dados combinados

Como outro exemplo, o Snowflake suporta a noção de tabelas externas definidas em buckets do S3. Um local de dados externo e um formato de arquivo são definidos ao criar a tabela, mas os dados ainda não foram ingeridos na tabela. Quando a tabela externa é consultada, o Snowflake lê do S3 e processa os dados com base nos parâmetros definidos no momento da criação da tabela. Podemos até juntar dados do S3 a tabelas internas do banco de dados. Isso torna o Snowflake e bancos de dados semelhantes mais compatíveis com um ambiente de data lake.

Alguns sistemas OLAP podem converter consultas federadas em visualizações materializadas. Isso nos dá muito do desempenho de uma tabela nativa sem a necessidade de ingerir dados manualmente toda vez que a fonte externa muda. A visão materializada é atualizada sempre que os dados externos são alterados.

Virtualização de dados

A virtualização de dados está intimamente relacionada a consultas federadas, mas isso normalmente envolve um sistema de processamento e consulta de dados que não armazena dados internamente. Neste momento, Trino (por exemplo, Starburst) e Presto são exemplos por excelência. Qualquer mecanismo de consulta/processamento que suporte tabelas externas pode servir como mecanismo de virtualização de dados. As considerações mais significativas com a virtualização de dados são as fontes externas e o desempenho com suporte.

Um conceito intimamente relacionado é a noção de ***empilhamento de consulta.*** Suponha que eu queira consultar dados do Snowflake, juntar dados de um banco de dados MySQL e filtrar os resultados. O pushdown de consulta visa mover o máximo de trabalho possível para os bancos de dados de origem. O mecanismo pode procurar maneiras de enviar predicados de filtragem para as consultas nos sistemas de origem. Isso serve a dois propósitos: primeiro, descarrega a computação da camada de virtualização, aproveitando o desempenho de consulta da fonte. Em segundo lugar, reduz potencialmente a quantidade de dados que devem ser enviados pela rede, um gargalo crítico para o desempenho da virtualização.

A virtualização de dados é uma boa solução para organizações com dados armazenados em várias fontes de dados. No entanto, a virtualização de dados não deve ser usada aleatoriamente. Por exemplo, a virtualização de um banco de dados MySQL de produção não resolve o problema central das consultas analíticas que afetam negativamente o sistema de produção – como o Trino não armazena dados internamente, ele extrairá do MySQL toda vez que executar uma consulta.

Alternativamente, a virtualização de dados pode ser usada como um componente de ingestão de dados e pipelines de processamento. Por exemplo, Trino pode ser usado para selecionar MySQL uma vez por dia à meia-noite quando a carga no sistema de produção é baixa. Os resultados podem ser salvos no S3 para consumo por transformações downstream e consultas diárias, protegendo o MySQL de consultas analíticas diretas.

Machine Translated by Google

A virtualização de dados pode ser vista como uma ferramenta que expande o data lake para muitas outras fontes, abstraindo as barreiras usadas para armazenar dados em silos entre unidades organizacionais. Uma organização pode armazenar dados transformados e acessados com frequência no S3 e virtualizar o acesso entre várias partes da empresa. Isso se encaixa perfeitamente com a noção de ***malha de dados*** (discutida no Capítulo 3), em que pequenas equipes são responsáveis por preparar seus dados para análise e compartilhá-los com o restante da empresa; a virtualização pode servir como uma camada de acesso crítica para compartilhamento prático.

Transformações e processamento de streaming

Já discutimos o processamento de fluxo no contexto de consultas. A diferença entre transformações de streaming e consultas de streaming é sutil e merece mais explicações.

Fundamentos

As consultas de streaming são executadas dinamicamente para apresentar uma visão atual dos dados, conforme discutido anteriormente. ***As transformações de streaming*** visam preparar os dados para consumo downstream.

Por exemplo, uma equipe de engenharia de dados pode ter um fluxo de entrada carregando eventos de uma fonte de IoT. Esses eventos de IoT carregam uma ID de dispositivo e dados de evento. Desejamos enriquecer dinamicamente esses eventos com outros metadados do dispositivo, que são armazenados em um banco de dados separado. O mecanismo de processamento de fluxo consulta um banco de dados separado contendo esses metadados por ID de dispositivo, gera novos eventos com os dados adicionados e os repassa para outro fluxo. As consultas ao vivo e as métricas acionadas são executadas nesse fluxo enriquecido (consulte a Figura 8-20).

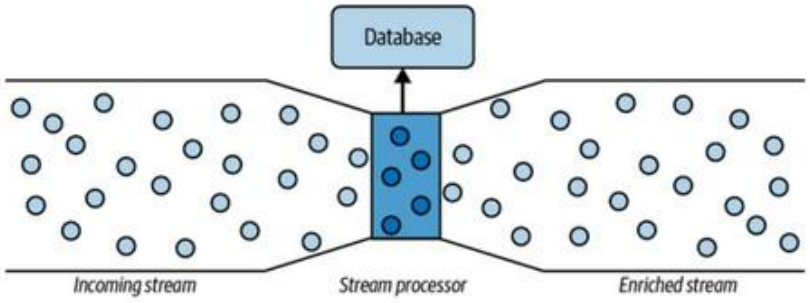


Figura 8-20. Um fluxo de entrada é transportado por uma plataforma de eventos de streaming e passado para um processador de fluxo

Transformações e consultas são um continuum

A linha entre transformações e consultas também é tênue no processamento em lote, mas as diferenças se tornam ainda mais sutis no domínio do streaming. Por exemplo, se calcularmos dinamicamente estatísticas de roll-up em janelas e, em seguida, enviarmos a saída para um fluxo de destino, isso é uma transformação ou uma consulta?

Talvez eventualmente adotemos uma nova terminologia para processamento de fluxo que represente melhor os casos de uso do mundo real. Por enquanto, faremos o nosso melhor com a terminologia que temos.

DAGs de streaming

Uma noção interessante intimamente relacionada ao enriquecimento e junções de fluxo é o ***DAG de streaming.*** Falamos[19] sobre essa ideia pela primeira vez em nossa discussão sobre orquestração no Capítulo 2. A orquestração é inerentemente um conceito de lote, mas e se quisermos enriquecer, mesclar e dividir vários fluxos em tempo real?

Vamos dar um exemplo simples em que o streaming de DAG seria útil. Suponha que queremos combinar dados de clickstream do site com dados de IoT. Isso nos permitirá obter uma visão unificada da atividade do usuário combinando eventos de IoT com cliques. Além disso, cada fluxo de dados precisa ser pré-processado em um formato padrão (veja a Figura 8-21).

Machine Translated by Google

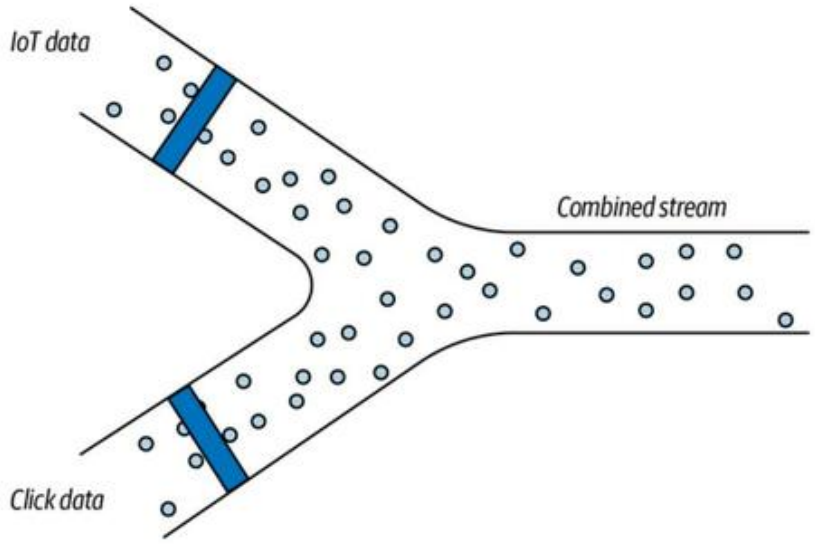


Figura 8-21. Um DAG de streaming simples

Isso é possível há muito tempo combinando uma loja de streaming (por exemplo, Kafka) com um processador de fluxo (por exemplo, Flink). Criar o DAG equivalia a construir uma máquina complexa de Rube Goldberg, com vários tópicos e trabalhos de processamento conectados.

O Pulsar simplifica drasticamente esse processo tratando os DAGs como uma abstração de streaming central. Em vez de gerenciar fluxos em vários sistemas, os engenheiros podem definir seus DAGs de streaming como código dentro de um único sistema.

Micro-lote versus streaming verdadeiro

Uma longa batalha está em andamento entre as abordagens de micro-lote e de streaming verdadeiro. Fundamentalmente, é importante entender seu caso de uso, os requisitos de desempenho e os recursos de desempenho da estrutura em questão.

O micro-lote é uma maneira de pegar uma estrutura orientada a lote e aplicá-la em uma situação de streaming. Um micro-lote pode ser executado a cada dois minutos a cada segundo. Algumas estruturas de microlote (por exemplo, Apache Spark Streaming) são projetadas para esse caso de uso e funcionarão bem com recursos alocados adequadamente em uma alta frequência de lote. (Na verdade, DBAs e engenheiros há muito usam micro-lotes com bancos de dados mais tradicionais; isso geralmente leva a um desempenho e consumo de recursos horríveis.)

Os verdadeiros sistemas de streaming (por exemplo, Beam e Flink) são projetados para processar um evento por vez. No entanto, isso vem com uma sobrecarga significativa. Além disso, é importante observar que, mesmo nesses verdadeiros sistemas de streaming, muitos processos ainda ocorrerão em lotes. Um processo de enriquecimento básico que adiciona dados a eventos individuais pode entregar um evento por vez com baixa latência. No entanto, uma métrica acionada no Windows pode ser executada a cada poucos segundos, a cada poucos minutos, etc.

Quando você está usando janelas e gatilhos (portanto, processamento em lote), qual é a frequência da janela? Qual é a latência aceitável? Se você estiver coletando métricas de vendas da Black Friday publicadas a cada poucos minutos, os micro-lotes provavelmente funcionarão bem, desde que você defina uma frequência de micro-lote apropriada. Por outro lado, se sua equipe de operações está computando métricas a cada segundo para detectar ataques DDoS, o streaming verdadeiro pode ser necessário.

Quando você deve usar um sobre o outro? Francamente, não existe uma resposta universal. O termo ***micro-lote*** tem sido frequentemente usado para descartar tecnologias concorrentes, mas pode funcionar bem para o seu caso de uso e pode ser superior em muitos aspectos, dependendo de suas necessidades. Se sua equipe já tiver experiência em Spark, você poderá criar uma solução de streaming Spark (micro-lote) com extrema rapidez.

Não há substituto para experiência de domínio e testes do mundo real. Converse com especialistas que podem apresentar uma opinião imparcial. Você também pode testar facilmente as alternativas criando testes na infraestrutura de nuvem. Além disso, atente para benchmarks espúrios fornecidos pelos fornecedores. Os fornecedores são famosos por escolher benchmarks e configurar

Machine Translated by Google

exemplos artificiais que não correspondem à realidade (lembre-se de nossa conversa sobre benchmarks no Capítulo 4). Frequentemente, os fornecedores mostram grandes vantagens em seus resultados de benchmark, mas não entregam no mundo real para seu uso caso.

Com quem você vai trabalhar

Consultas, transformações e modelagem impactam todas as partes interessadas no ciclo de vida da engenharia de dados. O engenheiro de dados é responsável por várias coisas nesta fase do ciclo de vida. Do ponto de vista técnico, o engenheiro de dados projeta, constrói e mantém a integridade dos sistemas que consultam e transformam dados. O engenheiro de dados também implementa modelos de dados nesse sistema. Este é o estágio mais "full-contact" onde seu foco é agregar o máximo de valor possível, tanto em termos de sistemas funcionais quanto de dados confiáveis e confiáveis.

Partes interessadas a montante

Quando se trata de transformações, os stakeholders upstream podem ser divididos em duas grandes categorias: aqueles que controlam as definições de negócios e aqueles que controlam os sistemas que geram dados.

Ao interagir com as partes interessadas upstream sobre definições e lógica de negócios, você precisará conhecer as fontes de dados — o que são, como são usadas e a lógica e as definições de negócios envolvidas. Você trabalhará com os engenheiros responsáveis por esses sistemas de origem e as partes interessadas do negócio que supervisionam os produtos e aplicativos complementares. Um engenheiro de dados pode trabalhar ao lado de "negócios" e partes interessadas técnicas em um modelo de dados.

O engenheiro de dados precisa estar envolvido no projeto do modelo de dados e nas atualizações posteriores devido a mudanças na lógica de negócios ou novos processos. As transformações são bastante fáceis de fazer; basta escrever uma consulta e colocar os resultados em uma tabela ou exibição. Criá-los para que tenham desempenho e valor para o negócio é outra questão. Sempre mantenha os requisitos e expectativas do negócio em mente ao transformar dados.

As partes interessadas dos sistemas upstream querem garantir que suas consultas e transformações afetem minimamente seus sistemas. Garanta a comunicação bidirecional sobre as alterações nos modelos de dados (alterações de coluna e índice, por exemplo) nos sistemas de origem, pois podem afetar diretamente as consultas, transformações e modelos de dados analíticos.
Os engenheiros de dados devem conhecer as alterações de esquema, incluindo a adição ou exclusão de campos, alterações de tipo de dados e qualquer outra coisa que possa afetar materialmente a capacidade de consultar e transformar dados.

Partes interessadas a jusante

As transformações são onde os dados começam a fornecer utilidade para as partes interessadas a jusante. Suas partes interessadas downstream incluem muitas pessoas, incluindo analistas de dados, cientistas de dados, engenheiros de ML e "o negócio". Colabore com eles para garantir que o modelo de dados e as transformações que você fornece sejam eficazes e úteis. Em termos de desempenho, as consultas devem ser executadas o mais rápido possível da maneira mais econômica. O que queremos dizer com ***útil?*** Analistas, cientistas de dados e engenheiros de ML devem poder consultar uma fonte de dados com a confiança de que os dados são da mais alta qualidade e integridade e podem ser integrados a seus fluxos de trabalho e produtos de dados. A empresa deve ser capaz de confiar que os dados transformados são precisos e acionáveis.

Subcorrentes

O estágio de transformação é onde seus dados se transformam e se transformam em algo útil para os negócios. Como há muitas partes móveis, as subcorrentes são especialmente críticas neste estágio.

Segurança

Consultas e transformações combinam conjuntos de dados diferentes em novos conjuntos de dados. Quem tem acesso a este novo conjunto de dados? Se alguém tiver acesso a um conjunto de dados, continue controlando quem tem acesso à coluna, linha e célula de um conjunto de dados.

Machine Translated by Google

nível de acesso.

Esteja ciente dos vetores de ataque contra seu banco de dados no momento da consulta. Os privilégios de leitura/gravação no banco de dados devem ser rigidamente monitorados e controlados. O acesso de consulta ao banco de dados deve ser controlado da mesma forma que você normalmente controla o acesso aos sistemas e ambientes de sua organização.

Mantenha as credenciais ocultas; evite copiar e colar senhas, tokens de acesso ou outras credenciais em código ou arquivos não criptografados. É surpreendentemente comum ver código nos repositórios do GitHub com nomes de usuário e senhas de banco de dados colados diretamente na base de código! Escusado será dizer, não compartilhe senhas com outros usuários. Por fim, nunca permita que dados não seguros ou não criptografados atravessem a Internet pública.

Gestão de dados

Embora o gerenciamento de dados seja essencial no estágio do sistema de origem (e em todos os outros estágios do ciclo de vida da engenharia de dados), ele é especialmente crítico no estágio de transformação. A transformação cria inerentemente novos conjuntos de dados que precisam ser gerenciados. Assim como em outros estágios do ciclo de vida da engenharia de dados, é fundamental envolver todas as partes interessadas nos modelos e transformações de dados e gerenciar suas expectativas. Além disso, certifique-se de que todos concordem com as convenções de nomenclatura que se alinham às respectivas definições de negócios dos dados. Convenções de nomenclatura adequadas devem ser refletidas em nomes de campo fáceis de entender. Os usuários também podem verificar em um catálogo de dados para obter mais clareza sobre o que o campo significa quando foi criado, quem mantém o conjunto de dados e outras informações relevantes.

A contabilização da precisão da definição é fundamental no estágio de transformação. A transformação segue a lógica de negócios esperada? Cada vez mais, a noção de uma camada semântica ou de métricas que fica independente de transformações está se tornando popular. Em vez de impor a lógica de negócios dentro da transformação em tempo de execução, por que não manter essas definições como um estágio autônomo antes de sua camada de transformação? Embora ainda seja cedo, espere ver as camadas semânticas e métricas se tornando mais populares e comuns na engenharia de dados e no gerenciamento de dados.

Como as transformações envolvem dados mutantes, é fundamental garantir que os dados que você está usando estejam livres de defeitos e representem a verdade. Se o MDM for uma opção em sua empresa, busque sua implementação. Dimensões conformadas e outras transformações contam com o MDM para preservar a integridade original dos dados e a verdade do terreno. Se o MDM não for possível, trabalhe com as partes interessadas upstream que controlam os dados para garantir que todos os dados que você está transformando estejam corretos e estejam em conformidade com a lógica de negócios acordada.

As transformações de dados tornam potencialmente difícil saber como um conjunto de dados foi derivado nas mesmas linhas. No Capítulo 6, discutimos os catálogos de dados. À medida que transformamos os dados, as ferramentas ***de linhagem*** de dados se tornam inestimáveis. As ferramentas de linhagem de dados ajudam os engenheiros de dados, que devem entender as etapas anteriores da transformação à medida que criam novas transformações, e os analistas, que precisam entender de onde os dados vieram enquanto executam consultas e criam relatórios.

Por fim, que impacto a conformidade regulatória tem em seu modelo de dados e transformações? Os dados de campos confidenciais são mascarados ou ofuscados, se necessário? Você tem a capacidade de excluir dados em resposta a solicitações de exclusão? Seu rastreamento de linhagem de dados permite que você veja dados derivados de dados excluídos e execute novamente as transformações para remover dados downstream de fontes brutas?

DataOps

Com consultas e transformações, o DataOps tem duas áreas de preocupação: dados e sistemas. Você precisa monitorar e ser alertado sobre alterações ou anomalias nessas áreas. O campo da observabilidade de dados está explodindo agora, com um grande foco na confiabilidade dos dados. Existe até um cargo recente chamado ***engenheiro de confiabilidade de dados.*** Esta seção enfatiza a observabilidade e a integridade dos dados, que se concentra no estágio de consulta e transformação.

Vamos começar com o lado dos dados do DataOps. Quando você consulta dados, as entradas e saídas estão corretas? Como você sabe? Se esta consulta for salva em uma tabela, o esquema está correto? E quanto ao formato dos dados e estatísticas relacionadas, como valores mín./máx., contagens nulas e muito mais? Você deve executar testes de qualidade de dados nos conjuntos de dados de entrada e no conjunto de dados transformado, o que garantirá que os dados atendam às expectativas dos usuários upstream e downstream. Se

Machine Translated by Google

há um problema de qualidade de dados na transformação, você deve ter a capacidade de sinalizar esse problema, reverter as alterações e investigar a causa raiz.

Agora vamos ver a parte Ops do DataOps. Como estão o desempenho dos sistemas? Monitore métricas como comprimento da fila de consulta, simultaneidade de consulta, uso de memória, utilização de armazenamento, latência de rede e E/S de disco. Use dados de métrica para identificar gargalos e consultas de baixo desempenho que podem ser candidatas a refatoração e ajuste. Se a consulta estiver perfeitamente correta, você terá uma boa ideia de onde ajustar o próprio banco de dados (por exemplo, agrupando uma tabela para obter um desempenho de pesquisa mais rápido). Ou talvez seja necessário atualizar os recursos de computação do banco de dados. Os bancos de dados em nuvem e SaaS de hoje oferecem muita flexibilidade para atualizar (e fazer downgrade) rapidamente do seu sistema. Adote uma abordagem orientada a dados e use suas métricas de observabilidade para identificar se você tem uma consulta ou um problema relacionado ao sistema.

A mudança para bancos de dados analíticos baseados em SaaS altera o perfil de custo do consumo de dados. Nos dias de data warehouses locais, o sistema e as licenças eram adquiridos antecipadamente, sem custo adicional de uso.
Enquanto os engenheiros de dados tradicionais se concentram na otimização do desempenho para extrair o máximo de utilidade de suas compras caras, os engenheiros de dados que trabalham com data warehouses em nuvem que cobram com base no consumo precisam se concentrar no gerenciamento de custos e na otimização de custos. Essa é a prática do ***FinOps*** (veja o Capítulo 4).

Arquitetura de dados

As regras gerais de uma boa arquitetura de dados no Capítulo 3 se aplicam ao estágio de transformação. Crie sistemas robustos que possam processar e transformar dados sem implodir. Suas escolhas para ingestão e armazenamento afetarão diretamente a capacidade de sua arquitetura geral de realizar consultas e transformações confiáveis. Se a ingestão e o armazenamento forem adequados aos seus padrões de consulta e transformação, você deve estar em um ótimo lugar. Por outro lado, se suas consultas e transformações não funcionarem bem com seus sistemas upstream, você enfrentará um mundo de problemas.

Por exemplo, muitas vezes vemos equipes de dados usando os pipelines e bancos de dados de dados errados para o trabalho. Uma equipe de dados pode conectar um pipeline de dados em tempo real a um RDBMS ou Elasticsearch e usá-lo como seu data warehouse. Esses sistemas não são otimizados para consultas OLAP agregadas de alto volume e implodirão sob essa carga de trabalho. Essa equipe de dados claramente não entendia como suas escolhas de arquitetura afetariam o desempenho da consulta. Reserve um tempo para entender as vantagens e desvantagens inerentes às suas escolhas de arquitetura; seja claro sobre como seu modelo de dados funcionará com sistemas de ingestão e armazenamento e como as consultas serão executadas.

Orquestração

As equipes de dados geralmente gerenciam seus pipelines de transformação usando agendamentos simples baseados em tempo, por exemplo, cron jobs. Isso funciona razoavelmente bem no início, mas se torna um pesadelo à medida que os fluxos de trabalho se tornam mais complicados. Use a orquestração para gerenciar pipelines complexos usando uma abordagem baseada em dependência. A orquestração também é a cola que nos permite montar pipelines que abrangem vários sistemas.

Engenharia de software Ao escrever

o código de transformação, você pode usar muitas linguagens, como SQL, Python e linguagens baseadas em JVM, e plataformas que variam de data warehouses a clusters de computação distribuídos e tudo mais. Cada linguagem e plataforma tem seus pontos fortes e peculiaridades, então você deve conhecer as melhores práticas de suas ferramentas. Por exemplo, você pode escrever transformações de dados em Python, alimentadas por um sistema distribuído, como Spark ou Dask.
Ao executar uma transformação de dados, você está usando uma UDF quando uma função nativa pode funcionar muito melhor? Vimos casos em que UDFs lentas e mal escritas foram substituídas por um comando SQL integrado, com melhoria instantânea e dramática no desempenho.

A ascensão da engenharia analítica traz práticas de engenharia de software para os usuários finais, com a noção de ***análise como código.*** Ferramentas de transformação de engenharia analítica, como dbt, explodiram em popularidade, dando a analistas e cientistas de dados a capacidade de escrever transformações no banco de dados usando SQL, sem a intervenção direta de um DBA ou engenheiro de dados. Nesse caso, o engenheiro de dados é responsável por configurar o repositório de código e o pipeline de CI/CD usado pelos analistas e cientistas de dados. Esta é uma grande mudança no papel de um engenheiro de dados, que historicamente

Machine Translated by Google

construir e gerenciar a infraestrutura subjacente e criar as transformações de dados. À medida que as ferramentas de dados diminuem as barreiras de entrada e se tornam mais democratizadas entre as equipes de dados, será interessante ver como os fluxos de trabalho das equipes de dados mudam.

Usando uma ferramenta de baixo código baseada em GUI, você obterá visualizações úteis do fluxo de trabalho de transformação. Você ainda precisa entender o que está acontecendo sob o capô. Essas ferramentas de transformação baseadas em GUI geralmente geram SQL ou alguma outra linguagem nos bastidores. Embora o objetivo de uma ferramenta de baixo código seja aliviar a necessidade de se envolver em detalhes de baixo nível, entender o código nos bastidores ajudará na depuração e na otimização do desempenho. Assumir cegamente que a ferramenta está gerando código de alto desempenho é um erro.

Sugerimos que os engenheiros de dados prestem atenção especial às melhores práticas de engenharia de software no estágio de consulta e transformação. Embora seja tentador simplesmente lançar mais recursos de processamento em um conjunto de dados, saber escrever um código limpo e de alto desempenho é uma abordagem muito melhor.

Conclusão

As transformações estão no centro dos pipelines de dados. Francamente, as transformações são a parte “sexy” da engenharia de dados.
É fundamental ter em mente o propósito das transformações. Em última análise, os engenheiros não são contratados para brincar com os brinquedos tecnológicos mais recentes, mas para atender seus clientes.

Nossa opinião é que é possível adotar tecnologias de transformação empolgantes ***e*** atender as partes interessadas. O Capítulo 11 fala sobre a ***pilha de dados ao vivo,*** essencialmente reconfigurando a pilha de dados em torno da ingestão de dados de streaming e aproximando os fluxos de trabalho de transformação dos próprios aplicativos do sistema de origem. As equipes de engenharia que pensam em dados em tempo real como a tecnologia pela tecnologia repetirão os erros da era do big data.
Mas, na realidade, a maioria das organizações com as quais trabalhamos tem um caso de uso de negócios que se beneficiaria do streaming de dados. Identificar esses casos de uso e focar no valor antes de escolher tecnologias e sistemas complexos é fundamental.

À medida que avançamos para o estágio de serviço do ciclo de vida da engenharia de dados no Capítulo 9, reflita sobre a tecnologia como uma ferramenta para atingir as metas organizacionais. Se você é um engenheiro de dados que trabalha, pense em como as melhorias nos sistemas de transformação podem ajudá-lo a atender melhor seus clientes finais. Se você está apenas iniciando um caminho para a engenharia de dados, pense nos tipos de problemas de negócios que você está interessado em resolver com tecnologia.

Recursos adicionais

- “Como funciona um mecanismo de banco de dados SQL,” por Dennis Pham
- “Um guia detalhado sobre otimização de consulta SQL” tutorial de Mega
- “Modelagem de dados de streaming em tempo real?” Thread do Stack Exchange
- “Consistência Eventual vs. Forte em Bancos de Dados Distribuídos” por Saurabh.v
- “Caching no Snowflake Data Warehouse” Página da Comunidade Snowflake
- "Como usar resultados de consulta em cache" do Google Cloud documentação
- Holistics "Não é possível combinar campos devido a problemas de distribuição?" Página de perguntas frequentes
- “Uma explicação simples de agregados simétricos ou 'Por que diabos meu SQL se parece com isso?'” por Lloyd Tab
- “Construindo um cofre de dados em tempo real no Snowflake” por Dmytro Yaroshenko e Kent Graziano
- “Modelagem de eventos de streaming” por Paul Stanton
- As “dimensões que mudam lentamente” da Oracle tutorial

Machine Translated by Google

"O novo paradigma 'Unified Star Schema' na análise de modelagem de dados do Analytics" por Andriy Zabavskyy

"Data Warehouse: A Escolha de Inmon vs. Kimball" por Ian Abramson

"Inmon ou Kimball: qual abordagem é adequada para seu data warehouse?" por Sansu George

A "Fábrica de Informações Corporativas" da ScienceDirect página da Internet

"Bill Inmon Data Warehouse" de Zentut página da Internet

"A Evolução da Fábrica de Informações Corporativas" por Bill Inmon

"Tipos de Arquitetura de Data Warehousing" por Amrita Fernando

"Diferença entre Kimball e Inmon" por manmeetjuneja5

"Introdução ao Data Warehousing", "Introdução à Modelagem Dimensional para Data Warehousing", e "Introdução ao Data Vault para Data Warehousing" por Simon Kitching

Gavroshe EUA "DW 2.0" página da Internet

Patente dos EUA para "Método e Aparelho para Integração Funcional de Metadados"

O "Processo de Projeto Dimensional em Quatro Etapas" do Kimball Group , "Dimensões Conformadas" e "Dimensões Técnicas de Modelagem" páginas da web

"Kimball vs. Inmon vs. Vault" Tópico do Reddit

"Como as organizações devem estruturar seus dados?" por Michael Berk

"Cofre de Dados - Uma Visão Geral" por John Ryan

Documento "Introdução à modelagem de cofre de dados" compilado por Kent Graziano e Dan Linstedt

"Noções básicas de modelagem do Data Vault 2.0" por Kent Graciano

Construindo o Data Warehouse (Wiley), ***Corporate Information Factory*** e ***The Unified Star Schema*** (Technics Publications) por WH (Bill) Inmon

O Data Warehouse Toolkit por Ralph Kimball e Margy Ross (Wiley)

Construindo um Data Warehouse Escalável com Data Vault 2.0 (Morgan Kaufmann) por Daniel Linstedt e Michael Olschimke

1 "Reescrevendo consultas de execução lenta que têm condições de junção equi-explosiva e condições de junção adicionais com base no operador semelhante", Site da Comunidade Snowflake, 1º de maio de 2020, ***https:// oreil.ly/ kUsO9.***

2 Veja, por exemplo, Emin Gün Sirer, "NoSQL Meets Bitcoin and Brings Down Two Exchanges: The Story of Flexcoin and Poloniex," Hacking, Distribuído, 6 de abril de 2014, ***https:// oreil.ly/ RM3QX.***

3 Algumas configurações do Redshift em vez disso, confie no armazenamento de objetos.

4 Os autores estão cientes de um incidente envolvendo um novo analista em uma grande rede de supermercados executando SELECT * em um banco de dados de produção e derrubando um banco de dados de inventário crítico por três dias.

5 A Figura 8-11 e o exemplo que ela descreve são significativamente baseados em "Introducing Stream—Stream Joins in Apache 2.3" por Tathagata Das e Joseph Torres, 13 de março de 2018.

6 Para mais detalhes sobre o princípio DRY, veja ***The Pragmatic Programmer*** de David Thomas e Andrew Hunt (Addison-Wesley Professional, 2019).

7 EF Codd, "Further Normalization of the Data Base Relational Model", IBM Research Laboratory (1971), ***https:// oreil.ly/ Muajm.***

8 HW Inmon, ***Construindo o Data Warehouse*** (Hoboken: Wiley, 2005).

9 Inmon, ***Construindo o Data Warehouse.***

10 Inmon, ***Construindo o Data Warehouse.***

Machine Translated by Google

11 Embora dimensões e fatos sejam frequentemente associados a Kimball, eles foram usados pela primeira vez na General Mills e na Dartmouth University na década de 1960, e tiveram adoção precoce na Nielsen e IRI, entre outras empresas.

12 O cofre de dados tem duas versões, 1.0 e 2.0. Esta seção se concentra no cofre de dados 2.0, mas vamos chamá-lo ***de cofre de dados*** por questões de brevidade.

13 Kent Graziano, “Data Vault 2.0 Modeling Basics,” Vertabelo, 20 de outubro de 2015, ***https://oreil.ly/iuW1U.***

14 Alex Woodie, “Lakehouses Prevent Data Swamps, Bill Inmon Says”, Datanami, 1º de junho de 2021, ***https://oreil.ly/XMwWc.***

15 Lembramos que você deve usar UDFs com responsabilidade. As UDFs SQL geralmente têm um desempenho razoavelmente bom. Vimos UDFs JavaScript aumentarem o tempo de consulta de alguns minutos a várias horas.

16 Michael Blaha, “Be Careful with Derived Data,” Dataversity, 5 de dezembro de 2016, ***https://oreil.ly/garoL.***

17 Benn Stancil, “The Missing Piece of the Modern Data Stack”, ***benn.substack,*** 22 de abril de 2021, ***https://oreil.ly/GYf3Z.***

18 “Qual é a diferença entre Apache Spark e Hadoop MapReduce?”, vídeo do YouTube Knowledge Powerhouse, 20 de maio de 2017, ***https://oreil.ly/WN0eX.***

19 Para uma aplicação detalhada do conceito de streaming DAG, veja “Why We Moved from Apache Kafka to Apache Pulsar” de Simba Khadder, Blog StreamNative, 21 de abril de 2020, ***https://oreil.ly/Rxfko.***

Machine Translated by Google

Capítulo 9. Servindo dados para análise, aprendizado de máquina e ETL reverso

Parabéns! Você chegou ao estágio final do ciclo de vida da engenharia de dados — fornecendo dados para casos de uso downstream (consulte a Figura 9-1). Neste capítulo, você aprenderá sobre várias maneiras de fornecer dados para três casos de uso principais que você encontrará como engenheiro de dados. Primeiro, você fornecerá dados para análise e BI. Você preparará dados para uso em análises estatísticas, relatórios e painéis. Esta é a área mais tradicional de serviço de dados.
Indiscutivelmente, ele antecede a TI e os bancos de dados, mas é tão importante como sempre que as partes interessadas tenham visibilidade dos processos de negócios, organizacionais e financeiros.

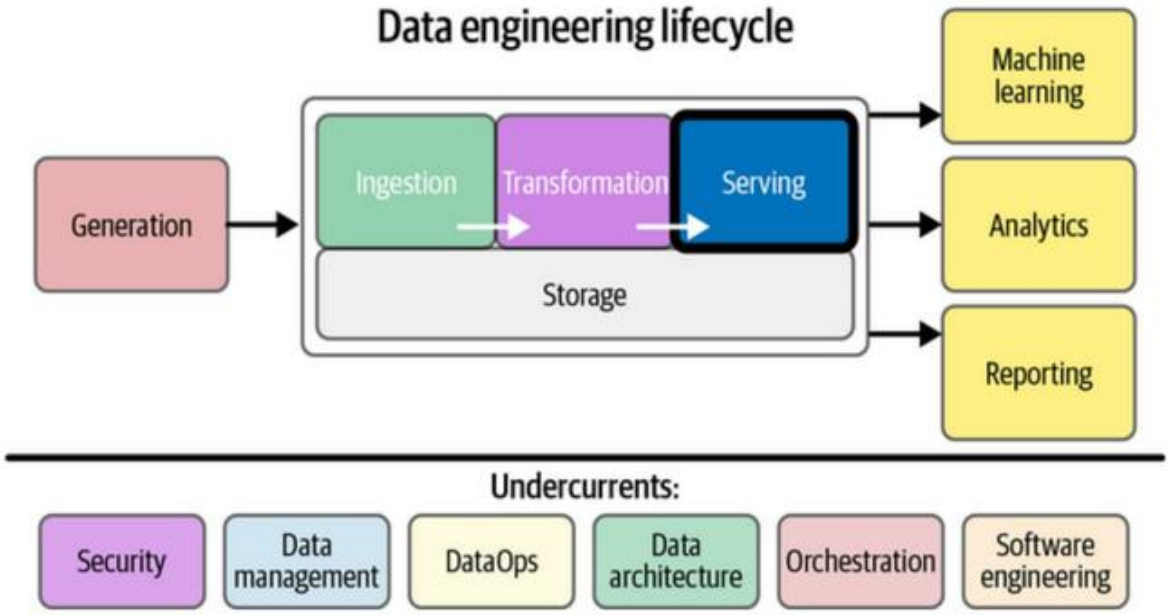


Figura 9-1. A veiculação entrega dados para casos de uso

Em segundo lugar, você fornecerá dados para aplicativos de ML. O ML não é possível sem dados de alta qualidade, devidamente preparados. Engenheiros de dados trabalham com dados

Machine Translated by Google

cientistas e engenheiros de ML para adquirir, transformar e fornecer os dados necessários para o treinamento de modelos.

Terceiro, você fornecerá dados por meio de ETL reverso. ***O ETL reverso*** é o processo de envio de dados de volta às fontes de dados. Por exemplo, podemos adquirir dados de uma plataforma de tecnologia de anúncios, executar um processo estatístico nesses dados para determinar os lances de custo por clique e, em seguida, alimentar esses dados de volta na plataforma de tecnologia de anúncios. O ETL reverso é altamente emaranhado com BI e ML.

Antes de entrarmos nessas três principais formas de veicular dados, vejamos algumas considerações gerais.

Considerações gerais para veiculação de dados

Antes de prosseguirmos na veiculação de dados, temos algumas considerações importantes. Em primeiro lugar é a confiança. As pessoas precisam confiar nos dados que você está fornecendo. Além disso, você precisa entender seus casos de uso e usuários, os produtos de dados que serão produzidos, como você servirá os dados (autoatendimento ou não), definições e lógica de dados e malha de dados. As considerações que discutiremos aqui são gerais e se aplicam a qualquer uma das três formas de veicular dados.
Compreender essas considerações ajudará você a ser muito mais eficaz no atendimento aos seus clientes de dados.

Confiar

> ***Leva 20 anos para construir uma reputação e cinco minutos para arruiná-la. Se você pensar sobre isso, você vai fazer as coisas de forma diferente.***
>
> —Warren Buffet

Acima de tudo, a confiança é a principal consideração no fornecimento de dados; os usuários finais precisam confiar nos dados que estão recebendo. A arquitetura de dados e a camada de serviço mais sofisticadas e sofisticadas são irrelevantes se os usuários finais não acreditarem que os dados são uma representação confiável de seus negócios. A perda de confiança geralmente é uma sentença de morte silenciosa para um projeto de dados, mesmo que o projeto não seja oficialmente cancelado até meses ou anos depois. O trabalho de um engenheiro de dados é servir

Machine Translated by Google

os melhores dados possíveis, portanto, você deve certificar-se de que seus produtos de dados sempre contenham dados confiáveis e de alta qualidade.

À medida que você aprender a fornecer dados ao longo deste capítulo, reforçaremos a ideia de incutir confiança em seus dados e discutiremos maneiras pragmáticas de fazer isso. Vemos muitos casos em que as equipes de dados estão fixadas em enviar dados sem perguntar se as partes interessadas confiam neles. Muitas vezes, as partes interessadas perdem a confiança nos dados. Uma vez que a confiança se foi, reconquistá-la é insanamente difícil. Isso inevitavelmente leva a empresa a não atingir todo o seu potencial com dados e as equipes de dados perdendo credibilidade (e possivelmente sendo dissolvidas).

Para obter a qualidade dos dados e construir a confiança das partes interessadas, utilize os processos de validação de dados e observabilidade de dados, em conjunto com a inspeção visual e a confirmação da validade com as partes interessadas. ***A validação*** de dados está analisando dados para garantir que eles representem com precisão informações financeiras, interações com clientes e vendas. ***A observabilidade*** de dados fornece uma visão contínua de dados e processos de dados. Esses processos devem ser aplicados ***em todo o ciclo de vida da engenharia de dados*** para obter um bom resultado à medida que chegamos ao fim. Discutiremos isso mais adiante em “Subcorrentes”.

Além de criar confiança na qualidade dos dados, cabe aos engenheiros criar confiança em seus SLAs e SLOs com seus usuários finais e partes interessadas upstream. Quando os usuários dependerem dos dados para realizar os processos de negócios, eles exigirão que os dados estejam consistentemente disponíveis e atualizados de acordo com os compromissos assumidos pelos engenheiros de dados. Dados de alta qualidade têm pouco valor se não estiverem disponíveis conforme o esperado na hora de tomar uma decisão crítica de negócios. Observe que os SLAs e SLOs também podem assumir a forma de ***contratos de dados*** (consulte o Capítulo 5), formal ou informalmente.

Falamos sobre SLAs no Capítulo 5, mas vale a pena discuti-los novamente aqui. SLAs vêm em uma variedade de formas. Independentemente de sua forma, um SLA informa aos usuários o que esperar de seu produto de dados; é um contrato entre você e seus stakeholders. Um exemplo de SLA pode ser: “Os dados estarão disponíveis de forma confiável e de alta qualidade”. Um SLO é uma parte essencial de um SLA e descreve as maneiras pelas quais você medirá o desempenho em relação ao que

Machine Translated by Google

você concordou. Por exemplo, dado o exemplo de SLA anterior, um SLO pode ser: "Nossos pipelines de dados para seu painel ou fluxo de trabalho de ML terão 99% de tempo de atividade, com 95% dos dados livres de defeitos". Certifique-se de que as expectativas estejam claras e que você possa verificar se está operando dentro dos parâmetros de SLA e SLO acordados.

Não basta simplesmente concordar com um SLA. A comunicação contínua é uma característica central de um bom SLA. Você comunicou possíveis problemas que podem afetar suas expectativas de SLA ou SLO? Qual é o seu processo de remediação e melhoria?

Confiança é tudo. Leva muito tempo para ganhar, e é fácil perder.

Qual é o caso de uso e quem é o usuário?

O estágio de veiculação é sobre dados em ação. Mas o que é um uso ***produtivo*** de dados? Você precisa considerar duas coisas para responder a essa pergunta: qual é o caso de uso e quem é o usuário?

O caso de uso de dados vai muito além da visualização de relatórios e painéis.
Os dados estão no seu melhor quando levam à ***ação.*** Um executivo tomará uma decisão estratégica a partir de um relatório? Um usuário de um aplicativo móvel de entrega de comida receberá um cupom que o atrai a comprar nos próximos dois minutos? Os dados geralmente são usados em mais de um caso de uso, por exemplo, para treinar um modelo de ML que faz a pontuação de leads e preenche um CRM (ETL reverso). Dados de alta qualidade e alto impacto atrairão muitos casos de uso interessantes. Mas, ao buscar casos de uso, sempre pergunte: "Que ***ação*** esses dados acionarão e ***quem*** executará essa ação?", com a pergunta de acompanhamento apropriada: "Esta ação pode ser automatizada?"

Sempre que possível, priorize os casos de uso com o maior ROI possível. Os engenheiros de dados adoram ficar obcecados com os detalhes técnicos de implementação dos sistemas que constroem, ignorando a questão básica do propósito. Os engenheiros querem fazer o que sabem fazer melhor: projetar coisas. Quando os engenheiros reconhecem a necessidade de se concentrar no valor e nos casos de uso, eles se tornam muito mais valiosos e eficazes em suas funções.

Machine Translated by Google

Ao iniciar um novo projeto de dados, trabalhar de trás para frente é útil. Embora seja tentador se concentrar nas ferramentas, recomendamos que você comece com o caso de uso e os usuários. Aqui estão algumas perguntas para se fazer ao começar:

> Quem usará os dados e como eles os usarão?
>
> O que as partes interessadas esperam?
>
> Como posso colaborar com as partes interessadas de dados (por exemplo, cientistas de dados, analistas, usuários de negócios) para entender como os dados com os quais estou trabalhando serão usados?

Novamente, sempre aborde a engenharia de dados da perspectiva do usuário e de seu caso de uso. Ao entender suas expectativas e objetivos, você pode trabalhar para trás para criar produtos de dados incríveis com mais facilidade. Vamos dedicar um momento para expandir nossa discussão sobre um produto de dados.

Produtos de dados

> ***Uma boa definição de um produto de dados é um produto que facilita um objetivo final por meio do uso de dados.***
>
> —DJ Patil

Os produtos de dados não são criados no vácuo. Como tantos outros processos organizacionais que discutimos, fazer produtos de dados é um esporte de contato total, envolvendo uma mistura de produto e negócios ao lado de tecnologia. É importante envolver os principais interessados no desenvolvimento de um produto de dados. Na maioria das empresas, um engenheiro de dados está a poucos passos dos usuários finais de um produto de dados; um bom engenheiro de dados procurará entender completamente os resultados para usuários diretos, como analistas de dados e cientistas de dados ou clientes externos à empresa.

Ao criar um produto de dados, é útil pensar nos “trabalhos a serem feitos”.
Um usuário “contrata” um produto para um “trabalho a ser feito”. Isso significa que você precisa saber o que o usuário quer, ou seja, sua motivação para “contratar” seu produto.
Um erro clássico de engenharia é simplesmente construir sem entender o

Machine Translated by Google

requisitos, necessidades do usuário final ou ajuste de produto/mercado. Esse desastre acontece quando você cria produtos de dados que ninguém quer usar.

Um bom produto de dados tem ciclos de feedback positivo. Mais uso de um produto de dados gera dados mais úteis, que são usados para melhorar o produto de dados. Enxague e repita.

Ao criar um produto de dados, lembre-se destas considerações:

> Quando alguém usa o produto de dados, o que espera realizar? Com muita frequência, os produtos de dados são feitos sem uma compreensão clara do resultado esperado pelo usuário.
>
> O produto de dados atenderá a usuários internos ou externos? No Capítulo 2, discutimos a engenharia de dados interna e externa. Ao criar um produto de dados, saber se seu cliente é interno ou externo afetará a maneira como os dados são atendidos.
>
> Quais são os resultados e o ROI do produto de dados que você está construindo?

Construir produtos de dados que as pessoas vão usar e amar é fundamental. Nada arruinará mais a adoção de um produto de dados do que utilidade indesejada e perda de confiança nas saídas de dados. Preste atenção à adoção e uso de produtos de dados e esteja disposto a se ajustar para deixar os usuários felizes.

Autoatendimento ou não?

Como os usuários irão interagir com seu produto de dados? Um diretor de negócios solicitará um relatório da equipe de dados ou esse diretor pode simplesmente criar o relatório? Os produtos de dados de autoatendimento — dando ao usuário a capacidade de criar produtos de dados por conta própria — têm sido uma aspiração comum dos usuários de dados por muitos anos. O que é melhor do que apenas dar ao usuário final a capacidade de criar diretamente relatórios, análises e modelos de ML?

Hoje, BI de autoatendimento e ciência de dados ainda são principalmente aspiracionais. Embora ocasionalmente vejamos empresas fazendo autoatendimento com dados com sucesso, isso é raro. Na maioria das vezes, as tentativas de autoatendimento de dados começam com grandes intenções, mas acabam falhando; dados de autoatendimento são difíceis de implementar em

Machine Translated by Google

prática. Assim, o analista ou cientista de dados é deixado para realizar o trabalho pesado de fornecer relatórios ad hoc e manter painéis.

Por que os dados de autoatendimento são tão difíceis? A resposta é matizada, mas geralmente envolve entender o usuário final. Se o usuário for um executivo que precisa entender como o negócio está indo, essa pessoa provavelmente só quer um painel predefinido de métricas claras e acionáveis. O executivo provavelmente ignorará quaisquer ferramentas de autoatendimento para criar visualizações de dados personalizadas. Se os relatórios provocarem mais perguntas, eles podem ter analistas à sua disposição para buscar uma investigação mais profunda. Por outro lado, um usuário que é analista pode já estar buscando análises de autoatendimento por meio de ferramentas mais poderosas, como SQL. A análise de autoatendimento por meio de uma camada de BI não é útil. As mesmas considerações se aplicam à ciência de dados. Embora conceder ML de autoatendimento a “cientistas de dados cidadãos” tenha sido uma meta de muitos fornecedores de ML automatizados, a adoção ainda é incipiente pelos mesmos motivos da análise de autoatendimento. Nesses dois casos extremos, um produto de dados de autoatendimento é uma ferramenta errada para o trabalho.

Projetos de dados de autoatendimento bem-sucedidos se resumem a ter o público certo. Identifique os usuários de autoatendimento e o “trabalho” que eles desejam fazer. O que eles estão tentando realizar usando um produto de dados de autoatendimento em vez de fazer parceria com um analista de dados para realizar o trabalho? Um grupo de executivos com experiência em dados forma um público ideal para o autoatendimento; eles provavelmente querem fatiar os dados sozinhos sem precisar tirar a poeira de suas habilidades de SQL definhantes. Os líderes de negócios dispostos a investir tempo para aprender habilidades de dados por meio de uma iniciativa da empresa e um programa de treinamento também podem obter um valor significativo do autoatendimento.

Determine como você fornecerá dados para esse grupo. Quais são seus requisitos de tempo para novos dados? O que acontece se eles inevitavelmente desejarem mais dados ou alterarem o escopo do que é exigido do autoatendimento? Mais dados geralmente significam mais perguntas, o que requer mais dados. Você precisará antecipar as necessidades crescentes de seus usuários de autoatendimento. Você também precisa entender o bom equilíbrio entre flexibilidade e proteções que ajudarão seu público a encontrar valor e insights sem resultados incorretos e confusão.

Machine Translated by Google

Definições de dados e lógica

Como discutimos enfaticamente, a utilidade dos dados em uma organização deriva, em última análise, de sua exatidão e confiabilidade. Criticamente, a exatidão dos dados vai além da reprodução fiel dos valores dos eventos dos sistemas de origem. A correção dos dados também abrange definições e lógica de dados apropriadas; eles devem ser incorporados aos dados em todos os estágios do ciclo de vida, desde sistemas de origem até pipelines de dados, ferramentas de BI e muito mais.

A definição de dados refere-se ao significado dos dados como são entendidos em toda a organização. Por exemplo, ***cliente*** tem um significado preciso dentro de uma empresa e entre departamentos. Quando a definição de um cliente varia, eles devem ser documentados e disponibilizados para todos que usam os dados.

A lógica de dados estipula fórmulas para derivar métricas de dados – digamos, vendas brutas ou valor da vida útil do cliente. A lógica de dados adequada deve codificar definições de dados e detalhes de cálculos estatísticos. Para calcular as métricas de rotatividade de clientes, precisaríamos de uma definição: quem é um cliente? Para calcular o lucro líquido, precisaríamos de um conjunto de regras lógicas para determinar quais despesas deduzir da receita bruta.

Freqüentemente, vemos definições de dados e lógica tidas como certas, muitas vezes passadas pela organização na forma de conhecimento tribal. ***O conhecimento tribal*** ganha vida própria, muitas vezes à custa de anedotas que substituem insights, decisões e ações baseadas em dados. Em vez disso, declarar formalmente as definições de dados e a lógica em um catálogo de dados e nos sistemas do ciclo de vida de engenharia de dados ajuda bastante a garantir a exatidão, consistência e confiabilidade dos dados.

As definições de dados podem ser veiculadas de várias maneiras, às vezes explicitamente, mas na maioria das vezes de forma implícita. Por ***implícito,*** queremos dizer que sempre que você fornece dados para uma consulta, um painel ou um modelo de ML, os dados e as métricas derivadas são apresentados de forma consistente e correta. Ao escrever uma consulta SQL, você assume implicitamente que as entradas para essa consulta estão corretas, incluindo a lógica e as definições do pipeline upstream. É aqui que a modelagem de dados

Machine Translated by Google

(descrito no Capítulo 8) é incrivelmente útil para capturar definições de dados e lógica de uma forma que seja compreensível e utilizável por vários usuários finais.

Usando uma camada semântica, você consolida as definições e a lógica de negócios de maneira reutilizável. Escreva uma vez, use em qualquer lugar. Esse paradigma é uma abordagem orientada a objetos para métricas, cálculos e lógica. Teremos mais a dizer em "Camadas Semânticas e Métricas".

Malha de dados

A malha de dados será cada vez mais uma consideração ao servir dados. A malha de dados muda fundamentalmente a forma como os dados são servidos dentro de uma organização. Em vez de equipes de dados em silos atendendo seus constituintes internos, cada equipe de domínio assume dois aspectos do serviço descentralizado de dados ponto a ponto.

Primeiro, as equipes são responsáveis por fornecer dados ***a outras equipes ,*** preparando-os para consumo. Os dados devem ser bons para uso em aplicativos de dados, painéis, análises e ferramentas de BI em toda a organização. Em segundo lugar, cada equipe potencialmente executa seus painéis e análises para ***autoatendimento.*** As equipes consomem dados de toda a organização com base nas necessidades específicas de seu domínio. Os dados consumidos de outras equipes também podem entrar no software projetado por uma equipe de domínio por meio de análises voltadas para o usuário ou um recurso de ML.

Isso muda drasticamente os detalhes e a estrutura do serviço. Apresentamos o conceito de malha de dados no Capítulo 3. Agora que abordamos algumas considerações gerais para fornecer dados, vamos examinar a primeira área principal: análise.

Análise

O primeiro caso de uso de serviço de dados que você provavelmente encontrará é a ***análise,*** que é descobrir, explorar, identificar e tornar visíveis os principais insights e padrões nos dados. A análise tem muitos aspectos. Como prática, a análise é realizada usando métodos estatísticos, relatórios, ferramentas de BI e muito mais. Como um

Machine Translated by Google

engenheiro de dados, conhecer os vários tipos e técnicas de análise é fundamental para realizar seu trabalho. Esta seção tem como objetivo mostrar como você servirá dados para análise e apresenta alguns pontos a serem considerados para ajudar seus analistas a ter sucesso.

Antes mesmo de fornecer dados para análise, a primeira coisa que você precisa fazer (o que deve parecer familiar depois de ler a seção anterior) é identificar o caso de uso final. O usuário está olhando para as tendências históricas? Os usuários devem ser notificados imediata e automaticamente sobre uma anomalia, como um alerta de fraude? Alguém está consumindo um painel em tempo real em um aplicativo móvel? Esses exemplos destacam as diferenças entre análise de negócios (geralmente BI), análise operacional e análise voltada para o usuário. Cada uma dessas categorias de análise tem objetivos diferentes e requisitos de veiculação exclusivos. Vejamos como você fornecerá dados para esses tipos de análise.

Analista de negócios

A análise de negócios usa dados históricos e atuais para tomar decisões estratégicas e acionáveis. Os tipos de decisões tendem a levar em consideração tendências de longo prazo e geralmente envolvem uma mistura de análise estatística e de tendências, juntamente com experiência de domínio e julgamento humano. A análise de negócios é tanto uma arte quanto uma ciência.

A análise de negócios normalmente se enquadra em algumas grandes áreas – painéis, relatórios e análises ad hoc. Um analista de negócios pode se concentrar em uma ou todas essas categorias. Vejamos rapidamente as diferenças entre essas práticas e ferramentas relacionadas. Compreender o fluxo de trabalho de um analista ajudará você, o engenheiro de dados, a entender como fornecer dados.

Um ***painel*** mostra de forma concisa aos tomadores de decisão o desempenho de uma organização em relação a algumas métricas principais, como vendas e retenção de clientes. Essas métricas principais são apresentadas como visualizações (por exemplo, gráficos ou mapas de calor), estatísticas resumidas ou até mesmo um único número. Isso é semelhante ao painel de um carro, que fornece uma única leitura das coisas críticas que você precisa saber ao dirigir um veículo. Uma organização pode ter mais de

Machine Translated by Google

um painel, com executivos de nível C usando um painel abrangente e seus subordinados diretos usando painéis com suas métricas específicas, KPIs ou objetivos e resultados-chave (OKRs). Os analistas ajudam a criar e manter esses painéis. Depois que as partes interessadas de negócios adotam e confiam em um painel, o analista geralmente responde a solicitações para analisar um possível problema com uma métrica ou adicionar uma nova métrica ao painel. Atualmente, você pode usar plataformas de BI para criar painéis, como Tableau, Looker, Sisense, Power BI ou Apache Superset/Preset.

Os analistas geralmente são encarregados pelas partes interessadas do negócio de criar um ***relatório.*** O objetivo de um relatório é usar dados para gerar insights e ações. Um analista que trabalha em uma empresa de varejo on-line é solicitado a investigar quais fatores estão gerando uma taxa de retorno acima do esperado para shorts femininos. O analista executa algumas consultas SQL no data warehouse, agrega os códigos de retorno que os clientes fornecem como motivo de sua devolução e descobre que o tecido do shorts de corrida é de qualidade inferior, muitas vezes se desgastando em poucos usos. As partes interessadas, como fabricação e controle de qualidade, são notificadas dessas descobertas.

Além disso, as descobertas são resumidas em um relatório e distribuídas na mesma ferramenta de BI onde reside o painel.

O analista foi solicitado a investigar um problema em potencial e retornar com insights. Isso representa um exemplo de ***análise ad hoc.*** Os relatórios geralmente começam como solicitações ad hoc. Se os resultados da análise ad hoc forem impactantes, eles geralmente acabam em um relatório ou painel. As tecnologias usadas para relatórios e análises ad hoc são semelhantes aos painéis, mas podem incluir Excel, Python, notebooks baseados em R, consultas SQL e muito mais.

Bons analistas se envolvem constantemente com os negócios e mergulham nos dados para responder a perguntas e descobrir tendências e insights ocultos e contra-intuitivos. Eles também trabalham com engenheiros de dados para fornecer feedback sobre qualidade de dados, problemas de confiabilidade e solicitações de novos conjuntos de dados. O engenheiro de dados é responsável por abordar esse feedback e fornecer novos conjuntos de dados para o analista usar.

Machine Translated by Google

Voltando ao exemplo dos shorts de corrida, suponha que, depois de comunicar suas descobertas, os analistas saibam que a manufatura pode fornecer a eles vários detalhes da cadeia de suprimentos sobre os materiais usados nos shorts de corrida. Os engenheiros de dados realizam um projeto para ingerir esses dados no data warehouse. Uma vez que os dados da cadeia de suprimentos estejam presentes, os analistas podem correlacionar números de série de roupas específicas com o fornecedor do tecido usado no item. Eles descobrem que a maioria das falhas está ligada a um de seus três fornecedores, e a fábrica para de usar tecido desse fornecedor.

Os dados para análise de negócios são frequentemente servidos em modo de lote a partir de um data warehouse ou de um data lake. Isso varia muito entre empresas, departamentos e até mesmo equipes de dados dentro das empresas. Novos dados podem estar disponíveis a cada segundo, minuto, a cada 30 minutos, todos os dias ou uma vez por semana. A frequência dos lotes pode variar por vários motivos. Uma coisa importante a ser observada é que os engenheiros que trabalham em problemas de análise devem considerar várias aplicações potenciais de dados, atuais e futuras. É comum ter frequências de atualização de dados mistas para atender os casos de uso adequadamente, mas lembre-se de que a frequência de ingestão define um teto na frequência downstream. Se existirem aplicativos de streaming para os dados, eles devem ser ingeridos como um stream, mesmo que algumas etapas de processamento e veiculação downstream sejam tratadas em lotes.

Obviamente, os engenheiros de dados devem abordar várias considerações técnicas de back-end ao servir a análise de negócios. Algumas ferramentas de BI armazenam dados em uma camada de armazenamento interno. Outras ferramentas executam consultas em seu data lake ou data warehouse. Isso é vantajoso porque você pode aproveitar ao máximo o poder do seu banco de dados OLAP. Como discutimos em capítulos anteriores, a desvantagem é o custo, o controle de acesso e a latência.

Análise Operacional

Se a análise de negócios é sobre o uso de dados para descobrir insights acionáveis, a análise operacional usa dados para realizar ***ações imediatas:***

> Análise operacional versus análise de negócios = ação imediata versus insights acionáveis

Machine Translated by Google

A grande diferença entre análise operacional e de negócios é ***o tempo.*** Os dados usados na análise de negócios têm uma visão mais ampla da questão em consideração. Atualizações atualizadas são boas de saber, mas não afetarão materialmente a qualidade ou o resultado. A análise operacional é exatamente o oposto, pois as atualizações em tempo real podem ter impacto na resolução de um problema quando ele ocorre.

Um exemplo de análise operacional é o monitoramento de aplicativos em tempo real. Muitas equipes de engenharia de software querem saber como está o desempenho de seus aplicativos; se surgirem problemas, eles querem ser notificados imediatamente. A equipe de engenharia pode ter um painel (veja, por exemplo, a Figura 9-2) que mostra as principais métricas, como solicitações por segundo, E/S do banco de dados ou quaisquer métricas importantes. Certas condições podem acionar eventos de dimensionamento, adicionando mais capacidade se os servidores estiverem sobrecarregados. Se determinados limites forem violados, o sistema de monitoramento também poderá enviar alertas por mensagem de texto, bate-papo em grupo e e-mail.

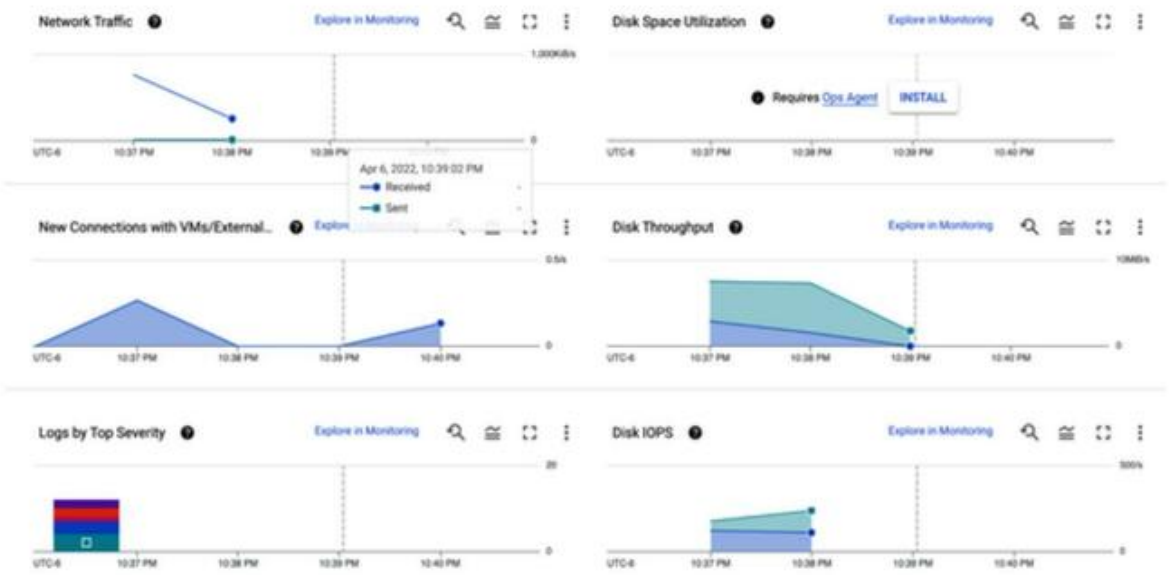


Figura 9-2. Um painel de análise operacional mostrando algumas métricas importantes do Google Compute Engine

Machine Translated by Google

ANÁLISES DE NEGÓCIOS E OPERACIONAIS

A linha entre análise de negócios e operacional começou a se confundir.
À medida que o streaming e os dados de baixa latência se tornam mais difundidos, é natural aplicar abordagens operacionais a problemas de análise de negócios; além de monitorar o desempenho do site na Black Friday, um varejista online também pode analisar e apresentar vendas, receitas e o impacto das campanhas publicitárias em tempo real.

As arquiteturas de dados serão alteradas para se adequarem a um mundo onde você pode ter seus dados quentes e quentes em um só lugar. A pergunta central que você deve sempre fazer a si mesmo e aos seus stakeholders é esta: se você tem dados de streaming, o que você vai fazer com eles? Que ação você deve tomar? A ação correta cria impacto e valor. Dados em tempo real sem ação são uma distração implacável.

A longo prazo, prevemos que o streaming suplantará o lote. Os produtos de dados nos próximos 10 anos provavelmente serão os primeiros a serem transmitidos, com a capacidade de combinar dados históricos perfeitamente. Após a coleta em tempo real, os dados ainda podem ser consumidos e processados em lotes conforme necessário.

Vamos voltar mais uma vez ao nosso exemplo de shorts de corrida. Usar a análise para descobrir tecidos ruins na cadeia de suprimentos foi um grande sucesso; líderes de negócios e engenheiros de dados querem encontrar mais oportunidades de utilizar dados para melhorar a qualidade do produto. Os engenheiros de dados sugerem a implantação de análises em tempo real na fábrica. A planta já usa uma variedade de máquinas capazes de transmitir dados em tempo real. Além disso, a planta possui câmeras gravando vídeo na linha de fabricação. No momento, os técnicos assistem às filmagens em tempo real, procuram itens defeituosos e alertam aqueles que estão na linha quando veem uma alta taxa de problemas aparecendo nos itens.

Os engenheiros de dados percebem que podem usar uma ferramenta de visão de máquina em nuvem pronta para uso para identificar defeitos em tempo real automaticamente. Os dados de defeitos são vinculados a números de série de itens específicos e transmitidos. A partir daqui, um processo de análise em tempo real pode vincular itens defeituosos a eventos de streaming de máquinas mais adiante na linha de montagem.

Machine Translated by Google

Usando essa abordagem, os analistas de chão de fábrica descobrem que a qualidade do estoque de tecido bruto varia significativamente de caixa para caixa. Quando o sistema de monitoramento mostra uma alta taxa de defeitos, os trabalhadores da linha podem remover a caixa defeituosa e devolvê-la ao fornecedor.

Vendo o sucesso deste projeto de melhoria de qualidade, o fornecedor decide adotar processos de controle de qualidade semelhantes. Os engenheiros de dados do varejista trabalham com o fornecedor para implantar sua análise de dados em tempo real, melhorando drasticamente a qualidade do estoque de tecidos.

Análise incorporada

Enquanto as análises de negócios e operacionais são focadas internamente, uma tendência recente é a análise externa ou voltada para o usuário. Com tantos dados alimentando aplicativos, as empresas fornecem cada vez mais análises aos usuários finais. Normalmente, eles são chamados de ***aplicativos de dados,*** geralmente com painéis de análise incorporados no próprio aplicativo. Também conhecidos como ***análises incorporadas,*** esses painéis voltados para o usuário final fornecem aos usuários as principais métricas sobre seu relacionamento com o aplicativo.

Um termostato inteligente possui um aplicativo móvel que mostra a temperatura em tempo real e métricas de consumo de energia atualizadas, permitindo ao usuário criar uma programação de aquecimento ou resfriamento com melhor eficiência energética. Em outro exemplo, uma plataforma de comércio eletrônico de terceiros fornece a seus vendedores um painel em tempo real sobre vendas, estoque e devoluções. O vendedor tem a opção de usar essas informações para oferecer ofertas aos clientes quase em tempo real. Em ambos os casos, um aplicativo permite que os usuários tomem decisões em tempo real (manualmente ou automaticamente) com base em dados.

O cenário da análise incorporada está crescendo como uma bola de neve, e esperamos que esses aplicativos de dados se tornem cada vez mais difundidos nos próximos anos. Como engenheiro de dados, você provavelmente não está criando o front-end analítico incorporado, pois os desenvolvedores de aplicativos lidam com isso. Como você é responsável pelos bancos de dados que atendem às análises incorporadas, você precisará entender os requisitos de velocidade e latência para análises voltadas para o usuário.

Machine Translated by Google

O desempenho para análises incorporadas abrange três problemas. Primeiro, os usuários de aplicativos não são tão tolerantes com o processamento em lote infrequente quanto os analistas internos da empresa; os usuários de uma plataforma SaaS de recrutamento podem esperar ver uma mudança em suas estatísticas assim que fizerem o upload de um novo currículo. Os usuários desejam baixa ***latência de dados.*** Em segundo lugar, os usuários de aplicativos de dados esperam um ***desempenho de consulta rápido.***
Quando eles ajustam os parâmetros em um painel de análise, eles querem que os resultados atualizados apareçam em segundos. Terceiro, os aplicativos de dados geralmente devem oferecer suporte a taxas de consulta extremamente altas em muitos painéis e vários clientes. A alta ***simultaneidade*** é crítica.

O Google e outros grandes players no espaço de aplicativos de dados desenvolveram tecnologias exóticas para lidar com esses desafios. Para novas startups, o padrão é usar bancos de dados transacionais convencionais para aplicativos de dados.
À medida que suas bases de clientes se expandem, eles superam sua arquitetura inicial.
Eles têm acesso a uma nova geração de bancos de dados que combinam alto desempenho — consultas rápidas, alta simultaneidade e atualizações quase em tempo real — com relativa facilidade de uso (por exemplo, análise baseada em SQL).

Aprendizado de máquina

A segunda área principal para servir dados é o aprendizado de máquina. ML é cada vez mais comum, então vamos supor que você esteja pelo menos familiarizado com o conceito. Com a ascensão da engenharia de ML (em si quase um universo paralelo à engenharia de dados), você pode se perguntar onde um engenheiro de dados se encaixa na imagem.

É certo que a fronteira entre ML, ciência de dados, engenharia de dados e engenharia de ML está cada vez mais difusa, e essa fronteira varia drasticamente entre as organizações. Em algumas organizações, os engenheiros de ML assumem o processamento de dados para aplicativos de ML logo após a coleta de dados ou podem até formar uma organização de dados totalmente separada e paralela que lida com todo o ciclo de vida de todos os aplicativos de ML. Os engenheiros de dados lidam com todo o processamento de dados em outras configurações e, em seguida, entregam os dados aos engenheiros de ML para treinamento do modelo.

Os engenheiros de dados podem até mesmo lidar com algumas tarefas extremamente específicas de ML, como a caracterização de dados.

Machine Translated by Google

Voltemos ao nosso exemplo de qualidade para controle de shorts de corrida produzidos por um varejista online. Suponha que os dados de streaming tenham sido implementados na fábrica que fabrica o estoque de tecido bruto para os shorts.

Os cientistas de dados descobriram que a qualidade do tecido fabricado é suscetível às características do poliéster bruto de entrada, temperatura, umidade e vários parâmetros ajustáveis do tear que tece o tecido. Os cientistas de dados desenvolvem um modelo básico para otimizar os parâmetros do tear.

Os engenheiros de ML automatizam o treinamento do modelo e configuram um processo para ajustar automaticamente o tear com base nos parâmetros de entrada. Os engenheiros de dados e ML trabalham juntos para projetar um pipeline de recursos, e os engenheiros de dados implementam e mantêm o pipeline.

O que um engenheiro de dados deve saber sobre ML

Antes de discutirmos a veiculação de dados para ML, você pode se perguntar quanto ML você precisa saber como engenheiro de dados. ML é um tópico incrivelmente vasto, e não tentaremos ensiná-lo a área; inúmeros livros e cursos estão disponíveis para aprender ML.

Embora um engenheiro de dados não precise ter uma compreensão profunda do ML, é muito útil conhecer os conceitos básicos de como o ML clássico funciona e os fundamentos do aprendizado profundo. Conhecer os conceitos básicos de ML ajudará você a trabalhar ao lado de cientistas de dados na criação de produtos de dados.

Aqui estão algumas áreas do ML com as quais achamos que um engenheiro de dados deve estar familiarizado:

- A diferença entre aprendizado supervisionado, não supervisionado e semisupervisionado.
- A diferença entre técnicas de classificação e regressão.
- As várias técnicas para lidar com dados de séries temporais. Isso inclui análise de séries temporais, bem como previsão de séries temporais.
- Quando usar as técnicas “clássicas” (regressão logística, aprendizado baseado em árvore, máquinas de vetor de suporte) versus aprendizado profundo. Nós constantemente

Machine Translated by Google

veja os cientistas de dados pularem imediatamente para o aprendizado profundo quando for um exagero. Como engenheiro de dados, seu conhecimento básico de ML pode ajudá-lo a identificar se uma técnica de ML é apropriada e dimensionar os dados que você precisará fornecer.

Quando você usaria o aprendizado de máquina automatizado (AutoML) em vez de criar um modelo de ML à mão? Quais são os trade-offs com cada abordagem em relação aos dados que estão sendo usados?

Quais são as técnicas de organização de dados usadas para dados estruturados e não estruturados?

Todos os dados usados para ML são convertidos em números. Se você estiver fornecendo dados estruturados ou semiestruturados, certifique-se de que os dados possam ser convertidos adequadamente durante o processo de engenharia de recursos.

Como codificar dados categóricos e os embeddings para vários tipos de dados.

A diferença entre aprendizagem em lote e online. Qual abordagem é apropriada para o seu caso de uso?

Como o ciclo de vida de engenharia de dados se cruza com o ciclo de vida de ML em sua empresa? Você será responsável pela interface ou suporte a tecnologias de ML, como repositórios de recursos ou observabilidade de ML?

Saiba quando é apropriado treinar localmente, em um cluster ou na borda.
Quando você usaria uma GPU em vez de uma CPU? O tipo de hardware que você usa depende muito do tipo de problema de ML que você está resolvendo, da técnica que está usando e do tamanho do seu conjunto de dados.

Conheça a diferença entre os aplicativos de dados em lote e streaming em modelos de ML de treinamento. Por exemplo, os dados em lote geralmente se ajustam bem ao treinamento de modelo offline, enquanto os dados de streaming funcionam com o treinamento online.

O que são cascatas de dados, e como eles podem afetar os modelos de ML?

Machine Translated by Google

Os resultados são retornados em tempo real ou em lote? Por exemplo, um modelo de transcrição de fala em lote pode processar amostras de fala e retornar texto em lote após uma chamada de API. Um modelo de recomendação de produto pode precisar operar em tempo real à medida que o cliente interage com um site de varejo online.

O uso de dados estruturados versus não estruturados. Podemos agrupar dados tabulares (estruturados) do cliente ou reconhecer imagens (não estruturadas) usando uma rede neural.

ML é uma ***vasta*** área de assunto, e este livro não vai te ensinar esses tópicos, ou mesmo generalidades de ML. Se você quiser saber mais sobre ML, sugerimos a leitura ***Hands on Machine Learning with Scikit-Learn, Keras e TensorFlow*** de Aurélien Géron (O'Reilly); inúmeros outros cursos e livros de ML estão disponíveis online. Como os livros e os cursos on-line evoluem tão rapidamente, faça sua pesquisa sobre o que parece ser uma boa opção para você.

Maneiras de fornecer dados para análise e ML

Assim como na análise, os engenheiros de dados fornecem aos cientistas de dados e engenheiros de ML os dados de que precisam para realizar seus trabalhos. Colocamos a veiculação de ML ao lado de análises porque os pipelines e processos são extremamente semelhantes. Há muitas maneiras de fornecer dados para análise e ML. Algumas maneiras comuns de fornecer esses dados incluem arquivos, bancos de dados, mecanismos de consulta e compartilhamento de dados. Vejamos brevemente cada um.

Troca de arquivos

A troca de arquivos é onipresente no serviço de dados. Processamos dados e geramos arquivos para passar aos consumidores de dados.

Tenha em mente que um arquivo pode ser usado para muitas finalidades. Um cientista de dados pode carregar um arquivo de texto (dados não estruturados) de mensagens de clientes para analisar os sentimentos das reclamações dos clientes. Uma unidade de negócios pode receber dados de fatura de uma empresa parceira como uma coleção de CSVs (dados estruturados) e um analista deve realizar algumas análises estatísticas nesses arquivos.

Machine Translated by Google

Ou um fornecedor de dados pode fornecer a um varejista on-line imagens de produtos no site de um concorrente (dados não estruturados) para classificação automatizada usando visão computacional.

A maneira como você veicula arquivos depende de vários fatores, como estes:

- Caso de uso – análise de negócios, análise operacional, análise voltada para o usuário
- Os processos de manipulação de dados do consumidor de dados
- O tamanho e o número de arquivos individuais no armazenamento
- Quem está acessando este arquivo
- Tipo de dados: estruturado, semiestruturado ou não estruturado

O segundo ponto é uma das principais considerações. Muitas vezes, é necessário fornecer dados por meio de arquivos em vez de compartilhamento de dados porque o consumidor de dados não pode usar uma plataforma de compartilhamento.

O arquivo mais simples de servir é algo como enviar um único arquivo Excel por e-mail. Este ainda é um fluxo de trabalho comum, mesmo em uma época em que os arquivos podem ser compartilhados de forma colaborativa. O problema com o envio de arquivos por e-mail é que cada destinatário recebe sua versão do arquivo. Se um destinatário editar o arquivo, essas edições serão específicas do arquivo desse usuário. Desvios entre os arquivos resultam inevitavelmente. Se você precisar de uma versão coerente e consistente de um arquivo, sugerimos usar uma plataforma de colaboração, como Microsoft 365 ou Google Docs.

É claro que servir arquivos únicos é difícil de dimensionar, e suas necessidades acabarão superando o armazenamento simples de arquivos em nuvem. Você provavelmente se tornará um bucket de armazenamento de objetos se tiver um punhado de arquivos grandes ou um data lake se tiver um suprimento constante de arquivos. O armazenamento de objetos pode armazenar qualquer tipo de arquivo blob e é especialmente útil para arquivos semiestruturados ou não estruturados.

Observaremos que geralmente consideramos a troca de arquivos por meio do armazenamento de objetos (data lake) como “compartilhamento de dados” em vez de troca de arquivos, pois o processo pode ser significativamente mais escalável e simplificado do que a troca de arquivos ad hoc.

Machine Translated by Google

Bancos de dados

Os bancos de dados são uma camada crítica no fornecimento de dados para análise e ML. Para esta discussão, vamos implicitamente manter nosso foco em servir dados de bancos de dados OLAP (por exemplo, data warehouses e data lakes). No capítulo anterior, você aprendeu sobre como consultar bancos de dados. Servir dados envolve consultar um banco de dados e, em seguida, consumir esses resultados para um caso de uso. Um analista ou cientista de dados pode consultar um banco de dados usando um editor SQL e exportar esses resultados para um arquivo CSV para consumo por um aplicativo downstream ou analisar os resultados em um notebook (descrito em "Servindo dados em notebooks").

Servir dados de um banco de dados traz uma variedade de benefícios. Um banco de dados impõe ordem e estrutura aos dados por meio de esquema; os bancos de dados podem oferecer controles de permissão refinados em nível de tabela, coluna e linha, permitindo que os administradores de banco de dados criem políticas de acesso complexas para várias funções; e os bancos de dados podem oferecer alto desempenho de serviço para consultas grandes e computacionalmente intensivas, alta simultaneidade de consultas ou ambos.

Os sistemas de BI geralmente compartilham a carga de trabalho de processamento de dados com um banco de dados de origem, mas o limite entre o processamento nos dois sistemas varia. Por exemplo, um servidor do Tableau executa uma consulta inicial para extrair dados de um banco de dados e armazená-los localmente. Slicing e dicing OLAP/BI básicos (filtragem e agregação interativas) são executados diretamente no servidor a partir da cópia de dados local. Por outro lado, o Looker conta com um modelo computacional chamado ***query pushdown;*** O Looker codifica a lógica de processamento de dados em uma linguagem especializada (LookML), combina isso com a entrada dinâmica do usuário para gerar consultas SQL, executa-as no banco de dados de origem e apresenta a saída. (Consulte "Camadas semânticas e métricas".) O Tableau e o Looker têm várias opções de configuração para armazenar resultados em cache para reduzir a carga de processamento de consultas executadas com frequência.

Um cientista de dados pode se conectar a um banco de dados, extrair dados e realizar engenharia e seleção de recursos. Esse conjunto de dados convertido é então alimentado em um modelo de ML; o modelo offline é treinado e produz resultados preditivos.

Machine Translated by Google

Os engenheiros de dados geralmente são encarregados de gerenciar a camada de serviço do banco de dados. Isso inclui gerenciamento de desempenho e custos. Em bancos de dados que separam computação e armazenamento, esse é um problema de otimização um pouco mais sutil do que nos dias de infraestrutura fixa no local.
Por exemplo, agora é possível criar um novo cluster Spark ou depósito Snowflake para cada carga de trabalho analítica ou de ML. Geralmente, é recomendado pelo menos dividir os clusters pelos principais casos de uso, como ETL e servir para análise e ciência de dados. Muitas vezes, as equipes de dados optam por dividir mais detalhadamente, atribuindo um warehouse por área principal. Isso possibilita que diferentes equipes orçamentem seus custos de consulta sob a supervisão de uma equipe de engenharia de dados.

Além disso, lembre-se das três considerações de desempenho que discutimos em "Análise incorporada". Estes são latência de dados, desempenho de consulta e simultaneidade. Um sistema que pode ingerir diretamente de um fluxo pode diminuir a latência de dados. E muitas arquiteturas de banco de dados contam com SSD ou cache de memória para aprimorar o desempenho e a simultaneidade da consulta para atender aos casos de uso desafiadores inerentes à análise voltada para o usuário.

Cada vez mais, plataformas de dados como Snowflake e Databricks permitem que analistas e cientistas de dados operem em um único ambiente, fornecendo editores SQL e notebooks de ciência de dados sob o mesmo teto. Como a computação e o armazenamento são separados, os analistas e os cientistas de dados podem consumir os dados subjacentes de várias maneiras sem interferir uns nos outros. Isso permitirá alto rendimento e entrega mais rápida de produtos de dados para as partes interessadas.

Sistemas de transmissão

A análise de streaming é cada vez mais importante no campo da veiculação. Em alto nível, entenda que esse tipo de atendimento pode envolver ***métricas emitidas,*** que são diferentes das consultas tradicionais.

Além disso, vemos bancos de dados de análise operacional desempenhando um papel crescente nessa área (consulte “Análise Operacional”). Esses bancos de dados permitem que as consultas sejam executadas em uma grande variedade de dados históricos, abrangendo

Machine Translated by Google

dados atuais. Essencialmente, eles combinam aspectos de bancos de dados OLAP com sistemas de processamento de fluxo. Cada vez mais, você trabalhará com sistemas de streaming para fornecer dados para análises e ML, portanto, familiarize-se com esse paradigma.

Você aprendeu sobre sistemas de streaming ao longo do livro. Para ter uma ideia de para onde está indo, leia sobre a pilha de dados ao vivo no Capítulo 11.

Federação de consultas

Conforme você aprendeu no Capítulo 8, a federação de consultas extrai dados de várias fontes, como data lakes, RDBMSs e data warehouses. A federação está se tornando mais popular à medida que os mecanismos de virtualização de consultas distribuídas ganham reconhecimento como formas de atender consultas sem passar pelo problema de centralizar dados em um sistema OLAP. Hoje, você pode encontrar opções de OSS como Trino e Presto e serviços gerenciados como Starburst. Algumas dessas ofertas se descrevem como formas de habilitar a malha de dados; o tempo dirá como isso se desenrola.

Ao servir dados para consultas federadas, você deve estar ciente de que o usuário final pode estar consultando vários sistemas – OLTP, OLAP, APIs, sistemas de arquivos etc. (Figura 9-3). Em vez de fornecer dados de um único sistema, agora você está servindo dados de vários sistemas, cada um com seus padrões de uso, peculiaridades e nuances. Isso apresenta desafios para o fornecimento de dados. Se as consultas federadas tocarem nos sistemas de origem de produção ao vivo, você deverá garantir que a consulta federada não consuma recursos excessivos na origem.

Machine Translated by Google

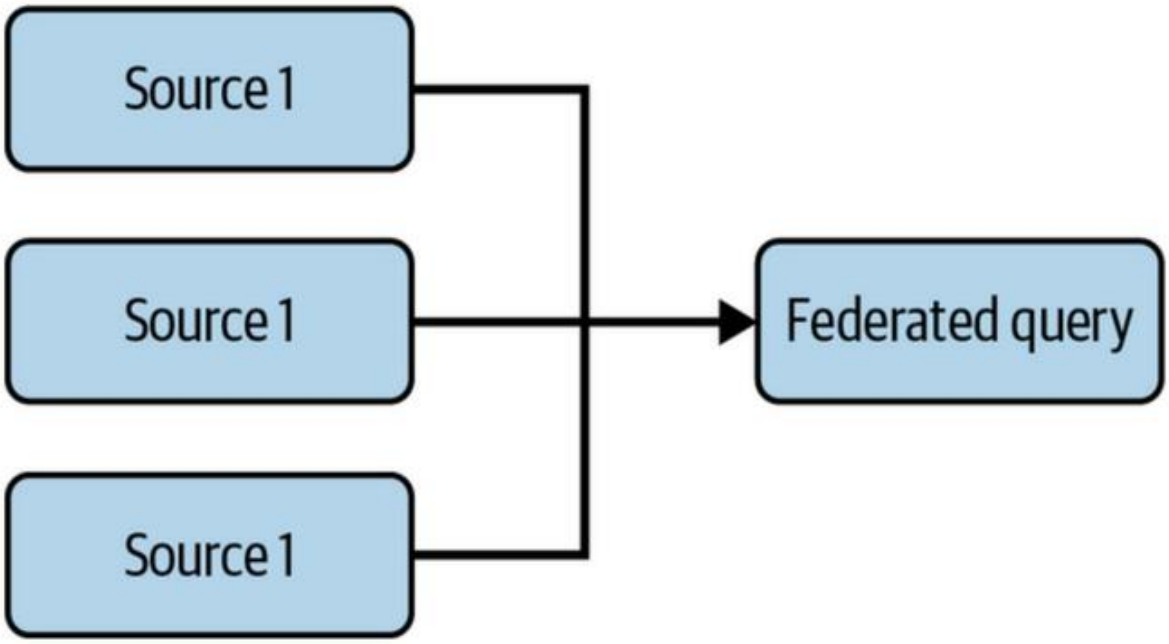


Figura 9-3. Uma consulta federada com três fontes de dados

Em nossa experiência, as consultas federadas são ideais quando você deseja flexibilidade na análise de dados ou os dados de origem precisam ser rigidamente controlados.
A federação permite consultas ad hoc para realizar análises exploratórias, combinando dados de vários sistemas sem a complexidade de configurar pipelines de dados ou ETL. Isso permitirá que você determine se o desempenho de uma consulta federada é suficiente para fins contínuos ou se você precisa configurar a ingestão em algumas ou todas as fontes de dados e centralizar os dados em um banco de dados OLAP ou data lake.

As consultas federadas também fornecem acesso somente leitura aos sistemas de origem, o que é ótimo quando você não deseja fornecer arquivos, acesso ao banco de dados ou despejos de dados. O usuário final lê apenas a versão dos dados que deve acessar e nada mais. A federação de consultas é uma ótima opção a ser explorada para situações em que o acesso e a conformidade são críticos.

Compartilhamento de dados

O Capítulo 5 inclui uma extensa discussão sobre compartilhamento de dados. Qualquer troca de dados entre organizações ou unidades dentro de uma organização maior pode ser vista como compartilhamento de dados. Ainda assim, queremos dizer especificamente compartilhar através

Machine Translated by Google

sistemas de armazenamento massivamente multilocatários em um ambiente de nuvem. O compartilhamento de dados geralmente transforma o serviço de dados em um problema de segurança e controle de acesso.

As consultas reais agora são tratadas pelos consumidores de dados (analistas e cientistas de dados) em vez dos engenheiros que fornecem os dados. Seja fornecendo dados em uma malha de dados dentro de uma organização, fornecendo dados ao público ou servindo a empresas parceiras, o compartilhamento de dados é um modelo de serviço atraente.
O compartilhamento de dados é cada vez mais um recurso central das principais plataformas de dados, como Snowflake, Redshift e BigQuery, permitindo que as empresas compartilhem dados com segurança entre si.

Camadas Semânticas e Métricas

Quando os engenheiros de dados pensam em servir, eles naturalmente tendem a gravitar em torno das tecnologias de processamento e armazenamento de dados, ou seja, você usará o Spark ou um data warehouse na nuvem? Seus dados são armazenados no armazenamento de objetos ou armazenados em cache em uma frota de SSDs? Mas mecanismos de processamento poderosos que fornecem resultados de consulta rápidos em grandes conjuntos de dados não contribuem inerentemente para análises de negócios de qualidade. Quando alimentados com dados de baixa qualidade ou consultas de baixa qualidade, mecanismos de consulta poderosos retornam rapidamente resultados ruins.

Onde a qualidade dos dados se concentra nas características dos próprios dados e em várias técnicas para filtrar ou melhorar os dados incorretos, a qualidade da consulta é uma questão de construir uma consulta com lógica apropriada que retorna respostas precisas para perguntas de negócios. Escrever consultas e relatórios ETL de alta qualidade é um trabalho detalhado e demorado. Várias ferramentas podem ajudar a automatizar esse processo, facilitando a consistência, a manutenção e a melhoria contínua.

Fundamentalmente, uma ***camada de métricas*** é uma ferramenta para manter e computar a lógica de negócios. (Uma ***camada semântica***[2] é extremamente similar conceitualmente, e ***BI sem cabeça***[3] é outro termo intimamente relacionado.) Essa camada pode residir em uma ferramenta de BI ou em um software que cria consultas de transformação. Dois exemplos concretos são Looker e Data Build Tool (dbt).

Por exemplo, o LookML da Looker permite que os usuários definam lógica de negócios complexa e virtual. Relatórios e painéis apontam para LookML específico para métricas de computação. O Looker permite que os usuários definam métricas padrão e

Machine Translated by Google

faça referência a eles em muitas consultas downstream; isso se destina a resolver o problema tradicional de repetição e inconsistência em scripts ETL tradicionais. O Looker usa o LookML para gerar consultas SQL, que são enviadas para o banco de dados. Os resultados podem ser persistidos no servidor Looker ou no próprio banco de dados para grandes conjuntos de resultados.

O dbt permite que os usuários definam fluxos de dados SQL complexos, abrangendo muitas consultas e definições padrão de métricas de negócios, como o Looker.
Ao contrário do Looker, o dbt é executado exclusivamente na camada de transformação, embora isso possa incluir o envio de consultas em exibições que são computadas no momento da consulta. Enquanto o Looker se concentra em atender consultas e relatórios, o dbt pode servir como uma ferramenta robusta de orquestração de pipeline de dados para engenheiros de análise.

Acreditamos que as ferramentas da camada de métricas se tornarão mais populares com uma adoção mais ampla e mais participantes, além de avançar para o aplicativo. As ferramentas da camada de métricas ajudam a resolver uma questão central na análise que tem atormentado as organizações desde que as pessoas analisaram os dados: “Esses números estão corretos?” Muitos novos participantes estão no espaço ao lado dos que mencionamos.

Servindo dados em notebooks

Os cientistas de dados costumam usar notebooks em seu trabalho diário. Seja explorando dados, recursos de engenharia ou treinando um modelo, o cientista de dados provavelmente usará um notebook. No momento da redação deste artigo, a plataforma de notebook mais popular é o Jupyter Notebook, juntamente com sua iteração de próxima geração, o JupyterLab. O Jupyter é de código aberto e pode ser hospedado localmente em um laptop, em um servidor ou por meio de vários serviços gerenciados em nuvem. ***Jupyter*** significa ***Julia, Python e R*** — os dois últimos são populares para aplicativos de ciência de dados, especialmente notebooks. Independentemente do idioma usado, a primeira coisa que você precisa considerar é como os dados podem ser acessados de um notebook.

Os cientistas de dados se conectarão programaticamente a uma fonte de dados, como uma API, um banco de dados, um data warehouse ou um data lake (Figura 9-4). Em um notebook, todas as conexões são criadas usando o

Machine Translated by Google

bibliotecas importadas para carregar um arquivo de um caminho de arquivo, conectar-se a um endpoint de API ou fazer uma conexão ODBC com um banco de dados. Uma conexão remota pode exigir as credenciais e privilégios corretos para estabelecer uma conexão. Uma vez conectado, um usuário pode precisar do acesso correto a tabelas (e linhas/colunas) ou arquivos armazenados no armazenamento de objetos. O engenheiro de dados geralmente ajudará o cientista de dados a encontrar os dados corretos e, em seguida, garantirá que eles tenham as permissões corretas para acessar as linhas e colunas necessárias.

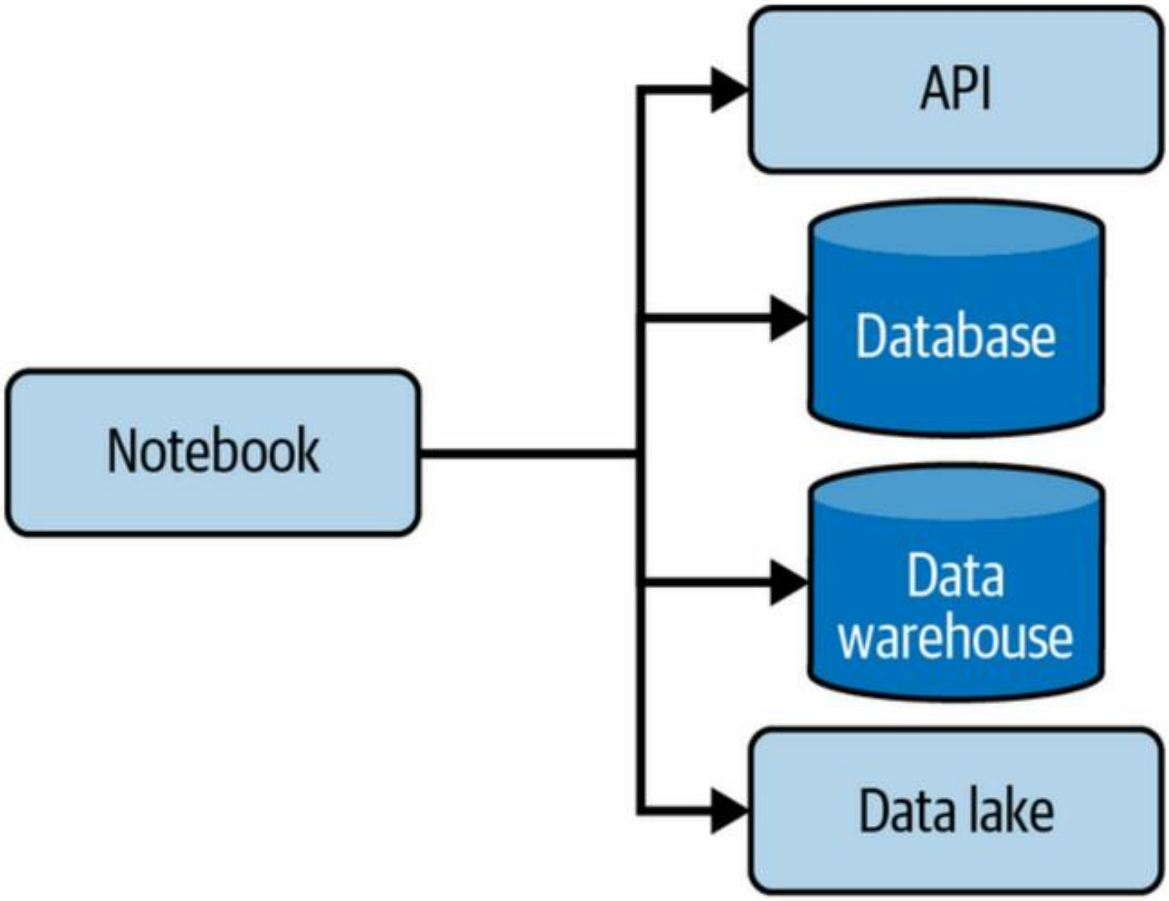


Figura 9-4. Um notebook pode receber dados de várias fontes, como armazenamento de objetos, banco de dados, data warehouse ou data lake

Machine Translated by Google

TRATAMENTO DE CREDENCIAIS

Credenciais manipuladas incorretamente em notebooks e código de ciência de dados são um grande risco de segurança; constantemente vemos credenciais mal tratadas neste domínio. É comum incorporar credenciais diretamente no código, onde muitas vezes vazam em repositórios de controle de versão. As credenciais também são frequentemente repassadas por meio de mensagens e e-mail.

Incentivamos os engenheiros de dados a auditar as práticas de segurança da ciência de dados e trabalhar de forma colaborativa em melhorias. Os cientistas de dados são altamente receptivos a essas conversas se tiverem alternativas. Os engenheiros de dados devem definir padrões para lidar com credenciais. As credenciais nunca devem ser incorporadas ao código; idealmente, os cientistas de dados usam gerenciadores de credenciais ou ferramentas CLI para gerenciar o acesso.

Vejamos um fluxo de trabalho incrivelmente comum para cientistas de dados: executar um notebook local e carregar dados em um dataframe de pandas. ***Pandas*** é uma biblioteca Python predominante usada para manipulação e análise de dados e é comumente usada para carregar dados (digamos, um arquivo CSV) em um notebook Jupyter.

Quando o pandas carrega um conjunto de dados, ele armazena esse conjunto de dados na memória.

O que acontece quando o tamanho do conjunto de dados excede a memória disponível da máquina local? Isso inevitavelmente acontece devido à memória limitada de laptops e estações de trabalho: ele interrompe um projeto de ciência de dados em seu caminho. É hora de considerar opções mais escaláveis. Primeiro, mude para um notebook baseado em nuvem, onde o armazenamento e a memória subjacentes do notebook possam ser dimensionados com flexibilidade. Ao superar essa opção, observe os sistemas de execução distribuída; opções populares baseadas em Python incluem Dask, Ray e Spark. Se uma oferta totalmente gerenciada em nuvem parecer atraente, considere configurar um fluxo de trabalho de ciência de dados usando Amazon SageMaker, Google Cloud Vertex AI ou Microsoft Azure Machine Learning. Por fim, as opções de fluxo de trabalho de ML de ponta a ponta de código aberto, como Kubeflow e MLflow, facilitam o dimensionamento de cargas de trabalho de ML no Kubernetes e no Spark, respectivamente. O objetivo é tirar os cientistas de dados de seus laptops e aproveitar o poder e a escalabilidade da nuvem.

Machine Translated by Google

Engenheiros de dados e engenheiros de ML desempenham um papel fundamental para facilitar a mudança para uma infraestrutura de nuvem escalável. A divisão exata do trabalho depende muito dos detalhes de sua organização. Eles devem assumir a liderança na configuração da infraestrutura em nuvem, supervisionar o gerenciamento de ambientes e treinar cientistas de dados em ferramentas baseadas em nuvem.

Os ambientes de nuvem exigem um trabalho operacional significativo, como gerenciamento de versões e atualizações, controle de acesso e manutenção de SLAs. Assim como em outros trabalhos operacionais, uma recompensa significativa pode resultar quando as “operações de ciência de dados” são bem feitas.

Os notebooks podem até se tornar parte da ciência de dados de produção; notebooks são amplamente implantados na Netflix. Esta é uma abordagem interessante com vantagens e compensações. Os notebooks produzidos permitem que os cientistas de dados coloquem seu trabalho em produção com muito mais rapidez, mas também são inerentemente uma forma de produção abaixo do padrão. A alternativa é fazer com que os engenheiros de ML e dados convertam os notebooks para uso em produção, sobrecarregando significativamente essas equipes. Um híbrido dessas abordagens pode ser ideal, com notebooks usados para produção “leve” e um processo de produção completo para projetos de alto valor.

ETL reverso

Hoje, ***o ETL reverso*** é uma palavra da moda que descreve o fornecimento de dados carregando-os de um banco de dados OLAP de volta para um sistema de origem. Dito isso, qualquer engenheiro de dados que trabalhou no campo por mais de alguns anos provavelmente fez alguma variação de ETL reverso. O ETL reverso cresceu em popularidade no final de 2010/início de 2020 e é cada vez mais reconhecido como uma responsabilidade formal de engenharia de dados.

Um engenheiro de dados pode extrair clientes e solicitar dados de um CRM e armazená-los em um data warehouse. Esses dados são usados para treinar um modelo de pontuação de lead, cujos resultados são retornados ao data warehouse. A equipe de vendas da sua empresa quer ter acesso a esses leads pontuados para tentar gerar mais vendas. Você tem algumas opções para obter os resultados deste modelo de lead scoring no

Machine Translated by Google

mãos da equipe de vendas. Você pode colocar os resultados em um painel para que eles visualizem. Ou você pode enviar os resultados para eles como um arquivo Excel.

O desafio dessas abordagens é que elas não estão conectadas ao CRM, onde um vendedor faz seu trabalho. Por que não colocar os leads pontuados de volta no CRM? Como mencionamos, produtos de dados bem-sucedidos reduzem o atrito com o usuário final. Nesse caso, o usuário final é a equipe de vendas.

Usar o ETL reverso e carregar os leads pontuados de volta ao CRM é a abordagem mais fácil e melhor para esse produto de dados. O ETL reverso pega os dados processados do lado de saída do ciclo de vida da engenharia de dados e os envia de volta aos sistemas de origem (Figura 9-5).

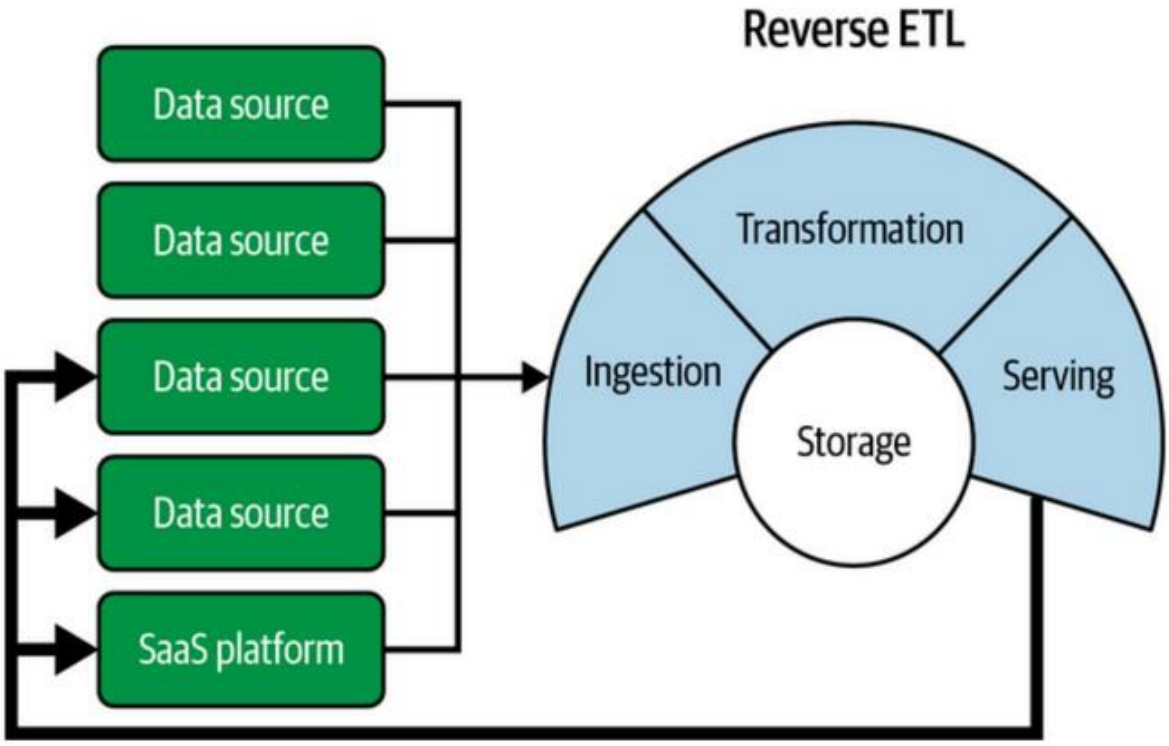


Figura 9-5. ETL reverso

Machine Translated by Google

NOTA

Em vez de ETL reverso, nós, os autores, meio brincando, chamamos isso de carga e transformação bidirecional (BLT). O termo ***ETL reverso*** não descreve com muita precisão o que está acontecendo nesse processo. Independentemente disso, o termo ficou na imaginação popular e na imprensa, então vamos usá-lo ao longo do livro. De forma mais ampla, se o termo ***ETL reverso*** permanece é uma incógnita, mas a prática de carregar dados de sistemas OLAP de volta aos sistemas de origem continuará sendo importante.

Maneiras de servir dados com ETL reverso

Como você começa a servir dados com ETL reverso? Embora você possa implementar sua solução de ETL reverso, muitas opções de ETL reverso prontas para uso estão disponíveis.
Sugerimos o uso de código aberto ou um serviço gerenciado comercial. Dito isso, o espaço de ETL reverso está mudando de forma extremamente rápida. Nenhum vencedor claro surgiu, e muitos produtos ETL reversos serão absorvidos pelas principais nuvens ou outros fornecedores de produtos de dados. Escolha cuidadosamente.

Temos algumas palavras de advertência sobre o ETL reverso. O ETL reverso cria inerentemente loops de feedback. Por exemplo, imagine que fazemos download de dados do Google Ads, usamos um modelo para calcular novos lances, carregamos os lances de volta no Google Ads e iniciamos o processo novamente. Suponha que, devido a um erro em seu modelo de lance, os lances tendam a subir cada vez mais e seus anúncios recebam cada vez mais cliques. Você pode rapidamente desperdiçar enormes quantias de dinheiro! Tenha cuidado e construa monitoramento e guarda-corpos.

Com quem você vai trabalhar

Conforme discutimos, no estágio de veiculação, um engenheiro de dados fará interface com muitas partes interessadas. Estes incluem (mas não estão limitados a) o seguinte:

- Analistas de dados
- Cientistas de dados
- Engenheiros de MLOps/ML

Machine Translated by Google

O negócio – partes interessadas, gerentes e executivos não relacionados a dados ou não técnicos

Como lembrete, o engenheiro de dados opera em uma função de ***suporte*** para essas partes interessadas e não é necessariamente responsável pelos usos finais dos dados. Por exemplo, um engenheiro de dados fornece os dados para um relatório que os analistas interpretam, mas o engenheiro de dados não é responsável por essas interpretações. Em vez disso, o engenheiro de dados é responsável por produzir os produtos de dados da mais alta qualidade possível.

Um engenheiro de dados deve estar ciente dos ciclos de feedback entre o ciclo de vida da engenharia de dados e o uso mais amplo dos dados, uma vez que estejam nas mãos das partes interessadas. Os dados raramente são estáticos, e o mundo exterior influenciará os dados que são ingeridos e servidos e reingeridos e reservidos.

Uma grande consideração para engenheiros de dados no estágio de serviço do ciclo de vida é a separação de tarefas e preocupações. Se você estiver em uma empresa em estágio inicial, o engenheiro de dados também pode ser um engenheiro de ML ou um cientista de dados; isso não é sustentável. À medida que a empresa cresce, você precisa estabelecer uma divisão clara de tarefas com outros membros da equipe de dados.

Adotar uma malha de dados reorganiza drasticamente as responsabilidades da equipe, e cada equipe de domínio assume aspectos de atendimento. Para que uma malha de dados seja bem-sucedida, cada equipe deve trabalhar efetivamente em suas responsabilidades de serviço de dados, e as equipes também devem colaborar efetivamente para garantir o sucesso organizacional.

Subcorrentes

As correntes ocultas chegam ao fim com o serviço. Lembre-se de que o ciclo de vida da engenharia de dados é apenas isso: um ciclo de vida. O que vai volta. Vemos muitos casos em que a veiculação de dados destaca algo perdido no início do ciclo de vida. Esteja sempre atento a como as tendências ocultas podem ajudá-lo a identificar maneiras de melhorar os produtos de dados.

Gostamos de dizer: “Data é um assassino silencioso”, e as correntes ocultas vêm à tona no estágio de servir. Servir é sua última chance de ter certeza

Machine Translated by Google

seus dados estão em ótima forma antes de chegarem às mãos dos usuários finais.

Segurança

Os mesmos princípios de segurança se aplicam ao compartilhar dados com pessoas ou sistemas. Muitas vezes vemos dados compartilhados indiscriminadamente, com pouco ou nenhum controle de acesso ou pensamento sobre para que os dados serão usados. Este é um grande erro que pode ter resultados catastróficos, como uma violação de dados e as multas resultantes, má imprensa e empregos perdidos. Leve a segurança a sério, especialmente nesta fase do ciclo de vida. De todos os estágios do ciclo de vida, o atendimento apresenta a maior superfície de segurança.

Como sempre, exerça o princípio de privilégio mínimo tanto para pessoas quanto para sistemas e forneça apenas o acesso necessário para o propósito em questão e o trabalho a ser feito. De quais dados um executivo precisa versus um analista ou cientista de dados? E quanto a um pipeline de ML ou processo de ETL reverso? Esses usuários e destinos têm necessidades de dados diferentes e o acesso deve ser fornecido de acordo. Evite dar permissões de carta branca a todos e a tudo.

O fornecimento de dados geralmente é somente leitura, a menos que uma pessoa ou processo precise atualizar os dados no sistema do qual são consultados. As pessoas devem ter acesso somente leitura a bancos de dados e conjuntos de dados específicos, a menos que sua função exija algo mais avançado, como acesso de gravação ou atualização. Isso pode ser feito combinando grupos de usuários com determinadas funções do IAM (ou seja, grupo de analistas, grupo de cientistas de dados) ou funções personalizadas do IAM, se isso fizer sentido. Para sistemas, forneça contas de serviço e funções de maneira semelhante. Para usuários e sistemas, restrinja o acesso aos campos, linhas, colunas e células de um conjunto de dados, se isso for garantido. Os controles de acesso devem ser tão refinados quanto possível e revogados quando o acesso não for mais necessário.

Os controles de acesso são essenciais ao servir dados em um ambiente multilocatário. Certifique-se de que os usuários possam acessar apenas ***seus*** dados e nada mais. Uma boa abordagem é mediar o acesso por meio de visualizações filtradas, aliviando assim os riscos de segurança inerentes ao compartilhamento de acesso a uma tabela comum. Outro

Machine Translated by Google

A sugestão é usar o compartilhamento de dados em seus fluxos de trabalho, o que permite controles granulares somente leitura entre você e as pessoas que consomem seus dados.

Verifique com que frequência os produtos de dados são usados e se faz sentido parar de compartilhar determinados produtos de dados. É extremamente comum que um executivo solicite urgentemente a um analista que crie um relatório, apenas para que esse relatório fique rapidamente sem uso. Se os produtos de dados não forem usados, pergunte aos usuários se eles ainda são necessários. Caso contrário, elimine o produto de dados. Isso significa uma vulnerabilidade de segurança a menos flutuando por aí.

Por fim, você deve ver o controle de acesso e a segurança não como impedimentos para servir, mas como facilitadores principais. Estamos cientes de muitos casos em que sistemas de dados complexos e avançados foram criados, potencialmente causando um impacto significativo em uma empresa. Como a segurança não foi implementada corretamente, poucas pessoas tiveram permissão para acessar os dados, por isso definhou. O controle de acesso robusto e refinado significa que análises de dados e ML mais interessantes podem ser feitas enquanto ainda protegem os negócios e seus clientes.

Gestão de dados

Você vem incorporando o gerenciamento de dados ao longo do ciclo de vida da engenharia de dados, e o impacto de seus esforços logo se tornará aparente à medida que as pessoas usarem seus produtos de dados. No estágio de veiculação, você está preocupado principalmente em garantir que as pessoas possam acessar dados confiáveis e de alta qualidade.

Como mencionamos no início deste capítulo, a confiança talvez seja a variável mais crítica no fornecimento de dados. Se as pessoas confiarem em seus dados, elas os usarão; dados não confiáveis não serão usados. Certifique-se de tornar a confiança dos dados e a melhoria dos dados um processo ativo, fornecendo loops de feedback. À medida que os usuários interagem com os dados, eles podem relatar problemas e solicitar melhorias.
Comunique-se ativamente com seus usuários à medida que as alterações são feitas.

De quais dados as pessoas precisam para realizar seus trabalhos? Especialmente com preocupações regulatórias e de conformidade pesando sobre as equipes de dados, dar às pessoas acesso aos dados brutos – mesmo com campos e linhas limitados – representa um problema de rastreamento de dados de volta a uma entidade, como uma pessoa ou um grupo de pessoas. Agradecidamente,

Machine Translated by Google

os avanços na ofuscação de dados permitem que você forneça dados sintéticos, codificados ou anônimos aos usuários finais. Esses conjuntos de dados “falsos” devem permitir que um analista ou cientista de dados obtenha o sinal necessário dos dados, mas de uma maneira que dificulte a identificação de informações protegidas. Embora este não seja um processo perfeito - com esforço suficiente, muitos conjuntos de dados podem ser anônimos ou sofrer engenharia reversa - pelo menos reduz o risco de vazamento de dados.

Além disso, incorpore camadas semânticas e métricas em sua camada de serviço, juntamente com uma modelagem de dados rigorosa que expresse adequadamente a lógica e as definições de negócios. Isso fornece uma única fonte de verdade, seja para análise, ML, ETL reverso ou outros usos de serviço.

DataOps

As etapas que você executa no gerenciamento de dados — qualidade de dados, governança e segurança — são monitoradas no DataOps. Essencialmente, o DataOps operacionaliza o gerenciamento de dados. A seguir estão algumas coisas para monitorar:

Saúde de dados e tempo de inatividade de dados

Latência de sistemas servindo dados – painéis, bancos de dados, etc.

Qualidade dos dados

Segurança e acesso a dados e sistemas

Versões de dados e modelos sendo veiculadas

Tempo de atividade para alcançar um SLO

Uma variedade de novas ferramentas surgiram para abordar vários aspectos de monitoramento. Por exemplo, muitas ferramentas populares de observabilidade de dados visam minimizar ***o tempo de inatividade dos dados*** e maximizar a qualidade dos dados. As ferramentas de observabilidade podem passar de dados para ML, apoiando o monitoramento de modelos e o desempenho do modelo. O monitoramento de DevOps mais convencional também é fundamental para o DataOps — por exemplo, você precisa monitorar se as conexões estão estáveis entre armazenamento, transformação e serviço.

Machine Translated by Google

Como em todas as etapas do ciclo de vida da engenharia de dados, codifique o controle de versão e operacionalize a implantação. Isso se aplica a código analítico, código de lógica de dados, scripts de ML e trabalhos de orquestração. Use vários estágios de implantação (dev, test, prod) para relatórios e modelos.

Arquitetura de dados

A veiculação de dados deve ter as mesmas considerações de arquitetura que outros estágios do ciclo de vida de engenharia de dados. Na fase de servir, os ciclos de feedback devem ser rápidos e apertados. Os usuários devem poder acessar os dados de que precisam o mais rápido possível quando precisam.

Os cientistas de dados são famosos por fazer a maior parte do desenvolvimento em suas máquinas locais. Conforme discutido anteriormente, incentive-os a migrar esses fluxos de trabalho para sistemas comuns em um ambiente de nuvem, onde as equipes de dados podem colaborar em ambientes de desenvolvimento, teste e produção e criar arquiteturas de produção adequadas. Facilite seus analistas e cientistas de dados por meio de ferramentas de suporte para publicar insights de dados com pouca dificuldade.

Orquestração

O serviço de dados é o último estágio do ciclo de vida da engenharia de dados. Como o atendimento está a jusante de tantos processos, é uma área de sobreposição extremamente complexa. A orquestração não é simplesmente uma maneira de organizar e automatizar o trabalho complexo, mas um meio de coordenar o fluxo de dados entre as equipes para que os dados sejam disponibilizados aos consumidores no tempo prometido.

A propriedade da orquestração é uma decisão organizacional chave. A orquestração será centralizada ou descentralizada? Uma abordagem descentralizada permite que pequenas equipes gerenciem seus fluxos de dados, mas pode aumentar a carga de coordenação entre equipes. Em vez de simplesmente gerenciar fluxos dentro de um único sistema, acionando diretamente a conclusão de DAGs ou tarefas pertencentes a outras equipes, as equipes devem passar mensagens ou consultas entre os sistemas.

Uma abordagem centralizada significa que o trabalho é mais fácil de coordenar, mas também deve existir um gatekeeping significativo para proteger um único ativo de produção.

Machine Translated by Google

Por exemplo, um DAG mal escrito pode interromper o Airflow. A abordagem centralizada significaria derrubar os processos de dados e servir em toda a organização. O gerenciamento de orquestração centralizado requer altos padrões, testes automatizados de DAGs e manutenção de portas.

Se a orquestração for centralizada, quem será o dono dela? Quando uma empresa tem uma equipe de DataOps, a orquestração geralmente chega aqui. Muitas vezes, uma equipe envolvida no atendimento é um ajuste natural porque tem uma visão bastante holística de todos os estágios do ciclo de vida da engenharia de dados. Podem ser os DBAs, engenheiros de análise, engenheiros de dados ou engenheiros de ML. Os engenheiros de ML coordenam processos complexos de treinamento de modelos, mas podem ou não querer adicionar a complexidade operacional do gerenciamento da orquestração a um conjunto de responsabilidades já lotado.

Engenharia de software

Em comparação com alguns anos atrás, o fornecimento de dados tornou-se mais simples. A necessidade de escrever código foi drasticamente simplificada. Os dados também passaram a priorizar o código, com a proliferação de estruturas de código aberto focadas em simplificar o fornecimento de dados. Existem muitas maneiras de fornecer dados aos usuários finais, e o foco de um engenheiro de dados deve ser saber como esses sistemas funcionam e como os dados são entregues.

Apesar da simplicidade de servir dados, se o código estiver envolvido, um engenheiro de dados ainda deve entender como funcionam as principais interfaces de serviço. Por exemplo, um engenheiro de dados pode precisar traduzir o código que um cientista de dados está executando localmente em um notebook e convertê-lo em um relatório ou modelo básico de ML para operar.

Outra área em que os engenheiros de dados serão úteis é entender o impacto de como o código e as consultas serão executados nos sistemas de armazenamento.
Os analistas podem gerar SQL de várias maneiras programáticas, incluindo LookML, Jinja via dbt, várias ferramentas de mapeamento relacional de objeto (ORM) e camadas de métricas. Quando essas camadas programáticas forem compiladas para SQL, como esse SQL será executado? Um engenheiro de dados pode sugerir otimizações em que o código SQL pode não funcionar tão bem quanto o SQL escrito à mão.

Machine Translated by Google

A ascensão da análise e do ML IaC significa que o papel de escrever código está se movendo em direção à construção de sistemas que dão suporte a cientistas e analistas de dados. Os engenheiros de dados podem ser responsáveis por configurar os pipelines de CI/CD e criar processos para sua equipe de dados. Eles também fariam bem em treinar e apoiar sua equipe de dados no uso da infraestrutura de dados/MLOps que criaram para que essas equipes de dados possam ser o mais autossuficientes possível.

Para análises incorporadas, os engenheiros de dados podem precisar trabalhar com desenvolvedores de aplicativos para garantir que as consultas sejam retornadas de forma rápida e econômica. O desenvolvedor do aplicativo controlará o código front-end com o qual os usuários lidam. O engenheiro de dados está lá para garantir que os desenvolvedores recebam as cargas úteis corretas conforme solicitado.

Conclusão

O ciclo de vida da engenharia de dados tem um final lógico no estágio de veiculação. Como em todos os ciclos de vida, ocorre um ciclo de feedback (Figura 9-6). Você deve ver o estágio de veiculação como uma chance de aprender o que está funcionando e o que pode ser melhorado. Ouça seus stakeholders. Se eles levantarem problemas – e inevitavelmente o farão – tente não se ofender. Em vez disso, use isso como uma oportunidade para melhorar o que você construiu.

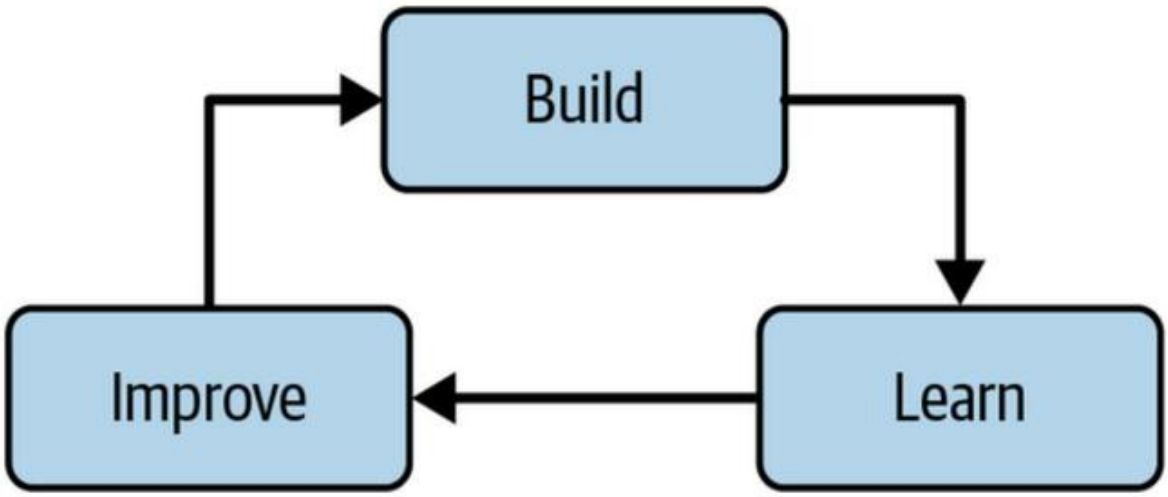


Figura 9-6. Construir, aprender, melhorar

Machine Translated by Google

Um bom engenheiro de dados está sempre aberto a novos comentários e constantemente encontra maneiras de melhorar seu ofício. Agora que fizemos uma jornada pelo ciclo de vida da engenharia de dados, você sabe como projetar, arquitetar, construir, manter e melhorar seus sistemas e produtos de engenharia de dados. Vamos voltar nossa atenção para a Parte III do livro, onde abordaremos alguns aspectos da engenharia de dados sobre os quais somos constantemente questionados e, francamente, merecem mais atenção.

Recursos adicionais

"Projetando produtos de dados" por Seth O'Regan

"Data Jujitsu: A Arte de Transformar Dados em Produto" por DJ Paty

"O que é análise voltada para o usuário?" por Chinmon Soman

"Dados como um produto versus produtos de dados: quais são as diferenças?" por Xavier Gumara Rigol

"Como construir ótimos produtos de dados" por Emily Glassberg Sands

"A evolução dos produtos de dados" e "O que é ciência de dados" por Mike Loukides

"Conheça as 'tarefas a serem feitas' de seus clientes" por Clayton M. Christensen et ai.

"Princípios de malha de dados e arquitetura lógica" por Martin Fowler

"Compreendendo a Camada Semântica do Superconjunto" por Srini Kadamati

"O futuro do BI é sem cabeça" por ZD

"Como estruturar uma equipe de análise de dados" por Niall Napier

"O que é análise operacional (e como ela está mudando a forma como trabalhamos com dados)?" por Sylvain Giuliani

Machine Translated by Google

“Fundamentos do aprendizado de máquina de autoatendimento” por Paramita (Guha) Ghosh

“O que o BI de autoatendimento moderno e a análise de dados realmente significam?” por Harry Dix

“Análise de autoatendimento” no Glossário do Gartner

“A peça que faltava na pilha de dados moderna” e “Por que o autosserviço ainda é um problema?” por Ben Stancil

Artigo do blog “Self-Service Business Intelligence: Dissolving the Barriers to Creative Decision-Support Solutions” da Forrester

Malha de dados por Zhamak Dehghani (O'Reilly)

1 “Know Your Customers' 'Jobs to Be Done'” por Clayton M. Christensen et al., ***Harvard Business Review,*** setembro de 2016, ***https:// oreil.ly/ 3uU4j.***

2 Benn Stancil, “The Missing Piece of the Modern Data Stack”, ***benn.substack,*** 22 de abril de 2021, ***https:// oreil.ly/ wQyPb.***

3 Srini Kadamati, “Understanding the Superset Semantic Layer”, blog Preset, 21 de dezembro de 2021, ***https:// oreil.ly/ 6smWC.***

Machine Translated by Google

Parte III. Segurança, privacidade e o futuro da engenharia de dados

Machine Translated by Google

Capítulo 10. Segurança e Privacidade

A segurança é vital para a prática da engenharia de dados. Isso deveria ser incrivelmente óbvio, mas estamos constantemente surpresos com a frequência com que os engenheiros de dados veem a segurança como uma reflexão tardia. Acreditamos que a segurança é a primeira coisa que um engenheiro de dados precisa pensar em todos os aspectos de seu trabalho e em cada estágio do ciclo de vida da engenharia de dados. Você lida com dados, informações e acessos sensíveis diariamente. Sua organização, clientes e parceiros de negócios esperam que esses ativos valiosos sejam tratados com o máximo cuidado e preocupação. Uma violação de segurança ou um vazamento de dados pode deixar seu negócio na água; sua carreira e reputação são arruinadas se a culpa for sua.

A segurança é um ingrediente chave para a privacidade. A privacidade tem sido fundamental para confiar no espaço corporativo de tecnologia da informação; engenheiros lidam direta ou indiretamente com dados relacionados à vida privada das pessoas. Isso inclui informações financeiras, dados sobre comunicações privadas (e-mails, textos, telefonemas), histórico médico, registros educacionais e histórico de trabalho. Uma empresa que vazou essa informação ou a usou indevidamente pode se tornar um pária quando a violação viesse à tona.

Cada vez mais, a privacidade é uma questão de importância legal significativa. Por exemplo, o Family Educational Rights and Privacy Act (FERPA) entrou em vigor nos EUA na década de 1970; o Health Insurance Portability and Accountability Act (HIPAA) foi seguido na década de 1990; O GDPR foi aprovado na Europa em meados da década de 2010. Várias leis de privacidade baseadas nos EUA foram aprovadas ou serão aprovadas em breve. Esta é apenas uma pequena amostra de estatutos relacionados à privacidade (e acreditamos ser apenas o começo). Ainda assim, as penalidades pela violação de qualquer uma dessas leis podem ser significativas, até mesmo devastadoras, para um negócio. E como os sistemas de dados estão integrados na estrutura da educação, saúde e negócios, os engenheiros de dados lidam com dados confidenciais relacionados a cada uma dessas leis.

Machine Translated by Google

As responsabilidades exatas de segurança e privacidade de um engenheiro de dados variam significativamente entre as organizações. Em uma pequena startup, um engenheiro de dados pode fazer dupla função como engenheiro de segurança de dados. Uma grande empresa de tecnologia terá exércitos de engenheiros de segurança e pesquisadores de segurança. Mesmo nessa situação, os engenheiros de dados geralmente conseguem identificar práticas de segurança e vulnerabilidades tecnológicas em suas próprias equipes e sistemas que podem relatar e mitigar em colaboração com a equipe de segurança dedicada.

Como a segurança e a privacidade são essenciais para a engenharia de dados (sendo a segurança uma tendência), queremos passar mais tempo cobrindo segurança e privacidade. Neste capítulo, apresentamos algumas coisas que os engenheiros de dados devem considerar em relação à segurança, principalmente em pessoas, processos e tecnologia (nessa ordem). Esta não é uma lista completa, mas apresenta as principais coisas que gostaríamos que melhorassem com base em nossa experiência.

Pessoas

O elo mais fraco em segurança e privacidade é ***você.*** A segurança é muitas vezes comprometida no nível humano, portanto, comporte-se como se você fosse sempre um alvo. Um bot ou ator humano está tentando se infiltrar em suas credenciais e informações confidenciais a qualquer momento. Esta é a nossa realidade, e não vai desaparecer. Assuma uma postura defensiva com tudo o que você faz online e offline.
Exercite o poder do pensamento negativo e seja sempre paranóico.

O poder do pensamento negativo

Em um mundo obcecado pelo pensamento positivo, o pensamento negativo é desagradável. No entanto, o cirurgião americano Atul Gawande escreveu um artigo de opinião em 2007 no ***New York Times*** precisamente sobre esse assunto. Sua tese central é que o pensamento positivo pode nos cegar para a possibilidade de ataques terroristas ou emergências médicas e impedir a preparação. O pensamento negativo nos permite considerar cenários desastrosos e agir para evitá-los.

Os engenheiros de dados devem pensar ativamente nos cenários de utilização de dados e coletar dados confidenciais somente se houver uma necessidade real

Machine Translated by Google

Rio abaixo. A melhor maneira de proteger dados privados e confidenciais é evitar a ingestão desses dados em primeiro lugar.

Os engenheiros de dados devem pensar nos cenários de ataque e vazamento com qualquer pipeline de dados ou sistema de armazenamento que utilizam. Ao decidir sobre estratégias de segurança, certifique-se de que sua abordagem ofereça segurança adequada e não apenas a ilusão de segurança.

Seja sempre paranóico

Sempre tenha cuidado quando alguém pedir suas credenciais.
Em caso de dúvida - e você deve sempre estar em dúvida extrema quando solicitado para credenciais - espere e obtenha segundas opiniões de seus colegas de trabalho e amigos. Confirme com outras pessoas que o pedido é realmente legítimo. Um bate-papo rápido ou uma ligação telefônica é mais barato do que um ataque de ransomware desencadeado por um clique de e-mail. Não confie em ninguém quando for solicitado credenciais, dados confidenciais ou informações confidenciais, inclusive de seus colegas de trabalho.

Você também é a primeira linha de defesa no respeito à privacidade e à ética. Você se sente desconfortável com os dados confidenciais que recebeu a tarefa de coletar? Você tem dúvidas éticas sobre a forma como os dados estão sendo tratados em um projeto?
Levante suas preocupações com colegas e liderança. Certifique-se de que seu trabalho seja legalmente compatível e ético.

Processos

Quando as pessoas seguem os processos regulares de segurança, a segurança se torna parte do trabalho. Faça da segurança um hábito, pratique regularmente a segurança real, exerça o princípio do privilégio mínimo e entenda o modelo de responsabilidade compartilhada na nuvem.

Teatro de segurança versus hábito de segurança

Com nossos clientes corporativos, vemos um foco generalizado em compliance (com normas internas, leis, recomendações de órgãos normativos), mas não

Machine Translated by Google

atenção suficiente para cenários potencialmente ruins. Infelizmente, isso cria uma ilusão de segurança, mas muitas vezes deixa lacunas que seriam evidentes com alguns minutos de reflexão.

A segurança precisa ser simples e eficaz o suficiente para se tornar habitual em toda a organização. Estamos impressionados com o número de empresas com políticas de segurança nas centenas de páginas que ninguém lê, a revisão anual de políticas de segurança que as pessoas esquecem imediatamente, tudo em uma auditoria de segurança. Este é o teatro de segurança, onde a segurança é feita na carta de conformidade (SOC-2, ISO 27001 e afins) sem ***compromisso real.***

Em vez disso, busque o espírito de segurança genuína e habitual; assar uma mentalidade de segurança em sua cultura. A segurança não precisa ser complicada. Por exemplo, em nossa empresa, realizamos treinamento de segurança e revisão de políticas pelo menos uma vez por mês para incorporar isso ao DNA de nossa equipe e atualizar uns aos outros sobre as práticas de segurança que podemos melhorar. A segurança não deve ser uma reflexão tardia para sua equipe de dados. Todos são responsáveis e têm um papel a desempenhar. Deve ser a prioridade para você e todos os outros com quem você trabalha.

Segurança Ativa

Voltando à ideia de pensamento negativo, ***a segurança ativa*** implica pensar e pesquisar as ameaças à segurança em um mundo dinâmico e em mudança.
Em vez de simplesmente implantar ataques de phishing simulados programados, você pode adotar uma postura de segurança ativa pesquisando ataques de phishing bem-sucedidos e analisando as vulnerabilidades de segurança de sua organização.
Em vez de simplesmente adotar uma lista de verificação de conformidade padrão, você pode pensar nas vulnerabilidades internas específicas da sua organização e nos incentivos que os funcionários podem ter para vazar ou fazer uso indevido de informações privadas.

Temos mais a dizer sobre segurança ativa em “Tecnologia”.

O Princípio do Mínimo Privilégio

O ***princípio do privilégio mínimo*** significa que uma pessoa ou sistema deve receber apenas os privilégios e dados necessários para concluir a tarefa em questão.

Machine Translated by Google

e nada mais. Muitas vezes, vemos um antipadrão na nuvem: um usuário comum recebe acesso administrativo a tudo, quando essa pessoa pode precisar de apenas algumas funções do IAM para fazer seu trabalho. Dar a alguém acesso administrativo carta branca é um grande erro e nunca deveria acontecer sob o princípio do privilégio mínimo.

Em vez disso, forneça ao usuário (ou grupo ao qual ele pertence) as funções do IAM necessárias quando precisar. Quando esses papéis não forem mais necessários, retire-os. A mesma regra se aplica às contas de serviço. Trate humanos e máquinas da mesma maneira: dê a eles apenas os privilégios e os dados de que precisam para realizar seus trabalhos e apenas pelo período de tempo, quando necessário.

É claro que o princípio do privilégio mínimo também é fundamental para a privacidade. Seus usuários e clientes esperam que as pessoas analisem seus dados confidenciais somente quando necessário. Certifique-se de que este é o caso. Implemente controles de acesso em nível de coluna, linha e célula em torno de dados confidenciais; considere mascarar PII e outros dados confidenciais e crie exibições que contenham apenas as informações que o visualizador precisa acessar. Alguns dados devem ser retidos, mas devem ser acessados apenas em caso de emergência. Coloque esses dados atrás de um ***processo de vidro quebrado:*** os usuários podem acessá-lo somente após passar por um processo de aprovação de emergência para corrigir um problema, consultar informações históricas críticas etc. O acesso é revogado imediatamente assim que o trabalho é concluído.

Responsabilidade compartilhada na nuvem

A segurança é uma responsabilidade compartilhada na nuvem. O fornecedor de nuvem é responsável por garantir a segurança física de seu data center e hardware. Ao mesmo tempo, você é responsável pela segurança dos aplicativos e sistemas que cria e mantém na nuvem. A maioria das violações de segurança na nuvem continua sendo causada por usuários finais, não pela nuvem.
As violações ocorrem devido a configurações incorretas não intencionais, erros, descuidos e desleixo.

Sempre faça backup de seus dados

Machine Translated by Google

Os dados desaparecem. Às vezes é um disco rígido ou servidor morto; em outros casos, alguém pode excluir acidentalmente um banco de dados ou um bucket de armazenamento de objetos. Um mau ator também pode bloquear dados. Os ataques de ransomware são generalizados nos dias de hoje. Algumas companhias de seguros estão reduzindo os pagamentos no caso de um ataque, deixando você no gancho tanto para recuperar seus dados quanto para pagar o mau ator que os mantém reféns. Você precisa fazer backup de seus dados regularmente, tanto para recuperação de desastres quanto para continuidade das operações de negócios, se uma versão de seus dados for comprometida em um ataque de ransomware. Além disso, teste a restauração de seus backups de dados regularmente.

O backup de dados não se enquadra estritamente nas práticas de segurança e privacidade; ele está sob o título maior de ***prevenção de desastres,*** mas é adjacente à segurança, especialmente na era dos ataques de ransomware.

Um exemplo de política de segurança

Esta seção apresenta um exemplo de política de segurança com relação a credenciais, dispositivos e informações confidenciais. Observe que não complicamos demais as coisas; em vez disso, damos às pessoas uma pequena lista de ações práticas que elas podem tomar imediatamente.

Machine Translated by Google

EXEMPLO DE POLÍTICA DE SEGURANÇA

Proteja suas credenciais

Proteja suas credenciais a todo custo. Aqui estão algumas regras básicas para credenciais:

Use um logon único (SSO) para tudo. Evite senhas sempre que possível e use SSO como padrão.

Use a autenticação multifator com SSO.

Não compartilhe senhas ou credenciais. Isso inclui senhas e credenciais de clientes. Em caso de dúvida, consulte a pessoa a quem você reporta. Se essa pessoa estiver em dúvida, continue cavando até encontrar uma
responda.

Cuidado com phishing e chamadas fraudulentas. Nunca divulgue suas senhas. (Novamente, priorize o SSO.)

Desative ou exclua credenciais antigas. De preferência o último.

Não coloque suas credenciais no código. Trate segredos como configuração e nunca os confirme no controle de versão. Use um gerenciador de segredos sempre que possível.

Sempre exerça o princípio do menor privilégio. Nunca dê mais acesso do que o necessário para fazer o trabalho. Isso se aplica a todas as credenciais e privilégios na nuvem e no local.

Proteja seus dispositivos

Use o gerenciamento de dispositivos para todos os dispositivos usados pelos funcionários. Se um funcionário sair da empresa ou seu dispositivo for perdido, o dispositivo poderá ser apagado remotamente.

Use a autenticação multifator para todos os dispositivos.

Faça login no seu dispositivo usando as credenciais de e-mail da sua empresa.

Machine Translated by Google

Todas as políticas que abrangem credenciais e comportamento se aplicam ao(s) seu(s) dispositivo(s).

Trate seu dispositivo como uma extensão de si mesmo. Não deixe o(s) dispositivo(s) atribuído(s) fora de vista.

Ao compartilhar a tela, saiba exatamente o que você está compartilhando para proteger informações e comunicações confidenciais. Compartilhe apenas documentos individuais, guias do navegador ou janelas e evite compartilhar sua área de trabalho completa. Compartilhe apenas o que for necessário para transmitir seu ponto de vista.

Use o modo “não perturbe” nas videochamadas; isso evita que as mensagens apareçam durante as chamadas ou gravações.

Política de atualização de software

Reinicie o navegador da Web quando vir um alerta de atualização.

Execute pequenas atualizações do sistema operacional em dispositivos pessoais e da empresa.

A empresa identificará as principais atualizações críticas do sistema operacional e fornecerá orientação.

Não use a versão beta de um sistema operacional.

Aguarde uma semana ou duas para novos lançamentos de versões principais do sistema operacional.

Estes são alguns exemplos básicos de como a segurança pode ser simples e eficaz.
Com base no perfil de segurança da sua empresa, pode ser necessário adicionar mais requisitos para as pessoas seguirem. E, novamente, lembre-se sempre de que as pessoas são seu elo mais fraco em segurança.

Tecnologia

Depois de abordar a segurança com pessoas e processos, é hora de ver como você aproveita a tecnologia para proteger seus sistemas e ativos de dados. A seguir estão algumas áreas importantes que você deve priorizar.

Machine Translated by Google

Sistemas de patch e atualização

O software fica obsoleto e as vulnerabilidades de segurança são constantemente descobertas. Para evitar expor uma falha de segurança em uma versão mais antiga das ferramentas que você está usando, sempre corrija e atualize os sistemas operacionais e o software à medida que novas atualizações forem disponibilizadas. Felizmente, muitos serviços gerenciados em nuvem e SaaS executam automaticamente atualizações e outras manutenções sem sua intervenção. Para atualizar seu próprio código e dependências, automatize compilações ou defina alertas sobre versões e vulnerabilidades para que você possa ser solicitado a executar as atualizações manualmente.

Criptografia

A criptografia não é uma bala mágica. Ele fará pouco para protegê-lo no caso de uma violação de segurança ***humana*** que conceda acesso a credenciais. A criptografia é um requisito básico para qualquer organização que respeite a segurança e a privacidade.
Ele irá protegê-lo de ataques básicos, como interceptação de tráfego de rede.

Vejamos separadamente a criptografia em repouso e em trânsito.

Criptografia em repouso

Certifique-se de que seus dados estejam criptografados quando estiverem em repouso (em um dispositivo de armazenamento). Os laptops da sua empresa devem ter a criptografia de disco completa habilitada para proteger os dados em caso de roubo de um dispositivo. Implemente a criptografia do lado do servidor para todos os dados armazenados em servidores, sistemas de arquivos, bancos de dados e armazenamento de objetos na nuvem. Todos os backups de dados para fins de arquivamento também devem ser criptografados. Por fim, incorpore a criptografia em nível de aplicativo, quando aplicável.

Criptografia por fio

A criptografia por fio agora é o padrão para os protocolos atuais. Por exemplo, HTTPS geralmente é necessário para APIs de nuvem modernas. Os engenheiros de dados devem sempre estar cientes de como as chaves são tratadas; o manuseio incorreto de chaves é uma fonte significativa de vazamentos de dados. Além disso, o HTTPS não faz nada para proteger os dados se as permissões de bucket forem deixadas abertas ao público, outra causa de vários escândalos de dados na última década.

Machine Translated by Google

Os engenheiros também devem estar cientes das limitações de segurança dos protocolos mais antigos. Por exemplo, o FTP simplesmente não é seguro em uma rede pública. Embora isso possa não parecer um problema quando os dados já são públicos, o FTP é vulnerável a ataques man-in-the-middle, em que um invasor intercepta os dados baixados e os altera antes que cheguem ao cliente. É melhor simplesmente evitar o FTP.

Certifique-se de que tudo esteja criptografado por fio, mesmo com protocolos legados. Em caso de dúvida, use tecnologia robusta com criptografia integrada.

Registro, monitoramento e alerta

Hackers e maus atores normalmente não anunciam que estão se infiltrando em seus sistemas. A maioria das empresas não fica sabendo dos incidentes de segurança até bem depois do fato. Parte do DataOps é observar, detectar e alertar sobre incidentes. Como engenheiro de dados, você deve configurar monitoramento, registro e alerta automatizados para estar ciente de eventos peculiares quando eles ocorrerem em seus sistemas. Se possível, configure a detecção automática de anomalias.

Aqui estão algumas áreas que você deve monitorar:

Acesso

Quem está acessando o quê, quando e de onde? Que novos acessos foram concedidos? Existem padrões estranhos com seus usuários atuais que podem indicar que a conta deles está comprometida, como tentar acessar sistemas eles geralmente não acessam ou não deveriam ter acesso? Você vê novo usuários não reconhecidos acessando seu sistema? Certifique-se de vasculhar regularmente os logs de acesso, usuários e suas funções para garantir que tudo Parece bom.

Recursos

Monitore seu disco, CPU, memória e E/S em busca de padrões que pareçam fora do comum. Seus recursos mudaram de repente? Se sim, isso pode

Machine Translated by Google

indicar uma violação de segurança.

Cobrança

Especialmente com SaaS e serviços gerenciados em nuvem, você precisa supervisionar os custos. Configure alertas de orçamento para garantir que seus gastos estejam dentro das expectativas. Se ocorrer um pico inesperado em seu faturamento, isso pode
indicar que alguém ou algo está utilizando seus recursos para fins maliciosos
propósitos.

Permissões em excesso

Cada vez mais, os fornecedores estão fornecendo ferramentas que monitoram as permissões que ***não são utilizadas*** por um usuário ou conta de serviço por algum tempo. Essas ferramentas geralmente podem ser configuradas para alertar automaticamente um administrador ou remover permissões após um tempo especificado.

Por exemplo, suponha que um determinado analista não tenha acessado o Redshift por seis meses. Essas permissões podem ser removidas, fechando uma possível falha de segurança. Se o analista precisar acessar o Redshift no futuro, ele poderá inserir um tíquete para restaurar as permissões.

É melhor combinar essas áreas em seu monitoramento para obter uma visão transversal do seu recurso, acesso e perfil de cobrança. Sugerimos configurar um painel para que todos na equipe de dados visualizem o monitoramento e recebam alertas quando algo parecer fora do comum. Junte isso a um plano de resposta a incidentes eficaz para gerenciar violações de segurança quando elas ocorrerem e execute o plano regularmente para que você esteja preparado.

Acesso à rede

Muitas vezes vemos engenheiros de dados fazendo coisas bem loucas em relação ao acesso à rede. Em vários casos, vimos buckets do Amazon S3 disponíveis publicamente abrigando muitos dados confidenciais. Também testemunhamos instâncias do Amazon EC2 com acesso SSH de entrada aberto para todo o mundo para 0.0.0.0/0

Machine Translated by Google

(todos os IPs) ou bancos de dados com acesso aberto a todas as solicitações de entrada pela Internet pública. Estes são apenas alguns exemplos de práticas terríveis de segurança de rede.

Em princípio, a segurança da rede deve ser deixada para os especialistas em segurança da sua empresa. (Na prática, você pode precisar assumir uma responsabilidade significativa pela segurança da rede em uma pequena empresa.) Como engenheiro de dados, você encontrará bancos de dados, armazenamento de objetos e servidores com tanta frequência que deve pelo menos estar ciente das medidas simples que pode tomar para garantir que você esteja de acordo com as boas práticas de acesso à rede. Entenda quais IPs e portas estão abertos, para quem e por quê. Permita os endereços IP de entrada dos sistemas e usuários que acessarão essas portas e evite a abertura ampla de conexões por qualquer motivo. Ao acessar a nuvem ou uma ferramenta SaaS, use uma conexão criptografada. Por exemplo, não use um site não criptografado de uma cafeteria.

Além disso, embora este livro tenha se concentrado quase inteiramente na execução de cargas de trabalho na nuvem, adicionamos uma breve observação aqui sobre a hospedagem de servidores locais. Lembre-se de que, no Capítulo 3, discutimos a diferença entre um perímetro protegido e uma segurança de confiança zero. A nuvem geralmente está mais próxima da segurança de confiança zero — cada ação requer autenticação. Acreditamos que a nuvem é uma opção mais segura para a maioria das organizações porque impõe práticas de confiança zero e permite que as empresas aproveitem o exército de engenheiros de segurança empregados pelas nuvens públicas.

No entanto, às vezes, a segurança de perímetro reforçada ainda faz sentido; encontramos algum consolo no conhecimento de que os silos de mísseis nucleares são air gaped (não conectados a nenhuma rede). Servidores air-gapped são o melhor exemplo de um perímetro de segurança reforçado. Lembre-se de que, mesmo no local, os servidores com gap de ar são vulneráveis a falhas de segurança humana.

Segurança para engenharia de dados de baixo nível

Para engenheiros que trabalham nas entranhas dos sistemas de armazenamento e processamento de dados, é fundamental considerar as implicações de segurança de cada elemento. Qualquer biblioteca de software, sistema de armazenamento ou nó de computação é um potencial de segurança

Machine Translated by Google

vulnerabilidade. Uma falha em uma biblioteca de log obscura pode permitir que invasores ignorem controles de acesso ou criptografia. Mesmo arquiteturas de CPU e microcódigo representam vulnerabilidades potenciais; dados confidenciais podem ser vulneráveis quando está em repouso na memória ou em um cache da CPU. Nenhum elo na cadeia pode ser tomado como certo.

É claro que este livro trata principalmente de engenharia de dados de alto nível — unindo ferramentas para lidar com todo o ciclo de vida. Assim, deixaremos para você explorar os detalhes técnicos sangrentos.

Pesquisa de segurança interna

Discutimos a ideia de ***segurança ativa*** em "Processos". Também recomendamos a adoção de uma abordagem de ***segurança ativa*** para a tecnologia. Especificamente, isso significa que todo funcionário de tecnologia deve pensar nos problemas de segurança.

Por que isso é importante? Cada colaborador de tecnologia desenvolve um domínio de conhecimento técnico. Mesmo que sua empresa empregue um exército de pesquisadores de segurança, os engenheiros de dados se familiarizarão intimamente com sistemas de dados específicos e serviços em nuvem sob sua alçada. Especialistas em uma determinada tecnologia estão bem posicionados para identificar falhas de segurança nessa tecnologia.

Incentive todos os engenheiros de dados a se envolverem ativamente na segurança. Quando identificam possíveis riscos de segurança em seus sistemas, eles devem pensar em mitigações e ter um papel ativo na implantação delas.

Conclusão

A segurança precisa ser um hábito mental e de ação; tratar dados como sua carteira ou smartphone. Embora você provavelmente não seja responsável pela segurança de sua empresa, conhecer as práticas básicas de segurança e manter a segurança em mente ajudará a reduzir o risco de violações de segurança de dados em sua organização.

Machine Translated by Google

Recursos adicionais

Open Web Application Security Project (OWASP) publicações

Construindo Sistemas Seguros e Confiáveis por Heather Adkins et al. (O'Reilly)

Segurança prática na nuvem por Chris Dotson (O'Reilly)

Machine Translated by Google

Capítulo 11. O Futuro da Engenharia de Dados

Este livro surgiu do reconhecimento dos autores de que as mudanças de velocidade de dobra no campo criaram uma lacuna de conhecimento significativa para engenheiros de dados existentes, pessoas interessadas em seguir uma carreira em engenharia de dados, gerentes de tecnologia e executivos que desejam entender melhor como os dados engenharia se encaixa em suas empresas. Quando começamos a pensar em como organizar este livro, recebemos um pouco de reação de amigos que perguntavam: “Como você se atreve a escrever sobre um campo que está mudando tão rapidamente?!” De muitas maneiras, eles estão certos. Certamente parece que o campo da engenharia de dados – e, na verdade, todas as coisas relacionadas a dados – está mudando diariamente. Peneirar o ruído e encontrar o sinal do ***que provavelmente não mudará*** foi uma das partes mais desafiadoras da organização e da escrita deste livro.

Neste livro, nos concentramos em grandes ideias que achamos que serão úteis nos próximos anos — daí a continuidade do ciclo de vida da engenharia de dados e suas correntes. A ordem das operações e os nomes das melhores práticas e tecnologias podem mudar, mas os estágios primários do ciclo de vida provavelmente permanecerão intactos por muitos anos. Estamos cientes de que a tecnologia continua a mudar em um ritmo exaustivo; trabalhar no setor de tecnologia em nossa era atual pode parecer uma montanha-russa ou talvez uma sala de espelhos.

Vários anos atrás, a engenharia de dados nem existia como um campo ou cargo. Agora você está lendo um livro chamado ***Fundamentos da Engenharia de Dados!*** Você aprendeu tudo sobre os fundamentos da engenharia de dados — seu ciclo de vida, subcorrentes, tecnologias e melhores práticas. Você pode estar se perguntando, o que vem a seguir na engenharia de dados? Embora ninguém possa prever o futuro, temos uma boa perspectiva sobre o passado, o presente e as tendências atuais. Tivemos a sorte de observar a gênese e a evolução da engenharia de dados da primeira fila. Este capítulo final apresenta nossos pensamentos

Machine Translated by Google

sobre o futuro, incluindo observações de desenvolvimentos em curso e especulação de futuro selvagem.

O ciclo de vida da engenharia de dados não está indo Um jeito

Embora a ciência de dados tenha recebido a maior parte da atenção nos últimos anos, a engenharia de dados está amadurecendo rapidamente em um campo distinto e visível. É uma das carreiras de mais rápido crescimento em tecnologia, sem sinais de perda de impulso. À medida que as empresas percebem que precisam primeiro construir uma base de dados antes de passar para coisas “mais sexy” como IA e ML, a engenharia de dados continuará crescendo em popularidade e importância. Esse progresso gira em torno do ciclo de vida da engenharia de dados.

Alguns questionam se ferramentas e práticas cada vez mais simples levarão ao desaparecimento de engenheiros de dados. Esse pensamento é superficial, preguiçoso e míope. À medida que as organizações aproveitam os dados de novas maneiras, novas fundações, sistemas e fluxos de trabalho serão necessários para atender a essas necessidades. Os engenheiros de dados estão no centro do projeto, arquitetura, construção e manutenção desses sistemas. Se as ferramentas se tornarem mais fáceis de usar, os engenheiros de dados subirão na cadeia de valor para se concentrar no trabalho de nível superior. O ciclo de vida da engenharia de dados não vai acabar tão cedo.

O declínio da complexidade e a ascensão da Ferramentas de dados fáceis de usar

Ferramentas simplificadas e fáceis de usar continuam a reduzir a barreira de entrada para a engenharia de dados. Isso é ótimo, especialmente considerando a escassez de engenheiros de dados que discutimos. A tendência para a simplicidade continuará. A engenharia de dados não depende de uma determinada tecnologia ou tamanho de dados. Também não é apenas para grandes empresas. Nos anos 2000, a implantação de tecnologias de “big data” exigia uma equipe grande e bolsos profundos. A ascensão dos serviços gerenciados por SaaS eliminou em grande parte a complexidade do

Machine Translated by Google

entender as entranhas de vários sistemas de “big data”. A engenharia de dados agora é algo que ***todas as*** empresas podem fazer.

Big data é uma vítima de seu sucesso extraordinário. Por exemplo, o Google BigQuery, descendente de GFS e MapReduce, pode consultar petabytes de dados. Antes reservada para uso interno no Google, essa tecnologia incrivelmente poderosa agora está disponível para qualquer pessoa com uma conta do GCP. Os usuários simplesmente pagam pelos dados que armazenam e consultam, em vez de ter que construir uma pilha de infraestrutura enorme. Snowflake, Amazon EMR e muitas outras soluções de dados em nuvem hiperescaláveis competem no espaço e oferecem recursos semelhantes.

A nuvem é responsável por uma mudança significativa no uso de ferramentas de código aberto. Mesmo no início de 2010, o uso de código aberto normalmente implicava baixar o código e configurá-lo você mesmo. Atualmente, muitas ferramentas de dados de código aberto estão disponíveis como serviços de nuvem gerenciados que competem diretamente com serviços proprietários. O Linux está disponível pré-configurado e instalado em instâncias de servidor em todas as principais nuvens. Plataformas sem servidor, como AWS Lambda e Google Cloud Functions, permitem que você implante aplicativos orientados a eventos em minutos, usando linguagens convencionais como Python, Java e Go em execução no Linux nos bastidores. Os engenheiros que desejam usar o Apache Airflow podem adotar o Cloud Composer do Google ou o serviço Airflow gerenciado da AWS. O Kubernetes gerenciado nos permite construir arquiteturas de microsserviços altamente escaláveis. E assim por diante.

Isso muda fundamentalmente a conversa em torno do código-fonte aberto. Em muitos casos, o código aberto gerenciado é tão fácil de usar quanto seus concorrentes de serviços proprietários. As empresas com necessidades altamente especializadas também podem implantar o código aberto gerenciado e, em seguida, migrar para o código aberto autogerenciado posteriormente, caso precisem personalizar o código subjacente.

Outra tendência significativa é o crescimento da popularidade dos conectores de dados prontos para uso (no momento da redação deste artigo, os populares incluem Fivetran e Airbyte). Os engenheiros de dados tradicionalmente gastam muito tempo e recursos construindo e mantendo o encanamento para se conectar a fontes de dados externas. A nova geração de conectores gerenciados é altamente atraente, mesmo para

Machine Translated by Google

engenheiros altamente técnicos, à medida que começam a reconhecer o valor de recapturar o tempo e a largura de banda mental para outros projetos. Os conectores de API serão um problema terceirizado para que os engenheiros de dados possam se concentrar nos problemas exclusivos que impulsionam seus negócios.

A interseção da concorrência acirrada no espaço de ferramentas de dados com um número crescente de engenheiros de dados significa que as ferramentas de dados continuarão diminuindo em complexidade, adicionando ainda mais funcionalidades e recursos. Essa simplificação só fará crescer a prática da engenharia de dados, à medida que mais e mais empresas encontrarem oportunidades para descobrir valor nos dados.

O SO de dados em escala de nuvem e as melhorias Interoperabilidade

Vamos revisar brevemente alguns dos funcionamentos internos dos sistemas operacionais (de dispositivo único) e, em seguida, vinculá-los aos dados e à nuvem. Esteja você utilizando um smartphone, um laptop, um servidor de aplicativos ou um termostato inteligente, esses dispositivos contam com um sistema operacional para fornecer serviços essenciais e orquestrar tarefas e processos. Por exemplo, posso ver cerca de 300 processos em execução no MacBook Pro em que estou digitando. Entre outras coisas, vejo serviços como WindowServer (responsável por fornecer janelas em uma interface gráfica) e CoreAudio (incumbido de fornecer recursos de áudio de baixo nível).

Quando executo um aplicativo nesta máquina, ele não acessa diretamente o hardware de som e gráficos. Em vez disso, ele envia comandos aos serviços do sistema operacional para desenhar janelas e reproduzir som. Esses comandos são emitidos para APIs padrão; uma especificação informa aos desenvolvedores de software como se comunicar com os serviços do sistema operacional. O sistema operacional ***orquestra*** um processo de inicialização para fornecer esses serviços, iniciando cada serviço na ordem correta com base nas dependências entre eles; também mantém os serviços monitorando-os e reiniciando-os na ordem correta em caso de falha.

Machine Translated by Google

Agora vamos voltar aos dados na nuvem. Os serviços de dados simplificados que mencionamos ao longo deste livro (por exemplo, Google Cloud BigQuery, Azure Blob Storage, Snowflake e AWS Lambda) se assemelham a serviços de sistema operacional, mas em uma escala muito maior, executados em muitas máquinas em vez de um único servidor .

Agora que esses serviços simplificados estão disponíveis, a próxima fronteira de evolução para essa noção de sistema operacional de dados em nuvem acontecerá em um nível mais alto de abstração. Benn Stancil pediu o surgimento de APIs de dados padronizadas para a construção de pipelines de dados e aplicativos de dados. [1] Prevemos que a engenharia de dados se fundirá gradualmente em torno de um punhado de padrões de interoperabilidade de dados. O armazenamento de objetos na nuvem crescerá em importância como uma camada de interface em lote entre vários serviços de dados. Formatos de arquivo de nova geração (como Parquet e Avro) já estão tomando conta do intercâmbio de dados na nuvem, melhorando significativamente a terrível interoperabilidade do CSV e o baixo desempenho do JSON bruto.

Outro ingrediente crítico de um ecossistema de API de dados é um catálogo de metadados que descreve esquemas e hierarquias de dados. Atualmente, essa função é amplamente preenchida pelo Hive Metastore herdado. Esperamos que novos entrantes surjam para ocupar seu lugar. Os metadados desempenharão um papel crucial na interoperabilidade de dados, tanto entre aplicativos e sistemas, quanto entre nuvens e redes, impulsionando a automação e a simplificação.

Também veremos melhorias significativas no andaime que gerencia os serviços de dados em nuvem. O Apache Airflow surgiu como a primeira plataforma de orquestração de dados verdadeiramente orientada para a nuvem, mas estamos à beira de um aprimoramento significativo. O Airflow crescerá em capacidades, aproveitando seu enorme compartilhamento de ideias. Novos participantes, como Dagster e Prefect, competirão reconstruindo a arquitetura de orquestração do zero.

Esta próxima geração de plataformas de orquestração de dados contará com integração e conscientização de dados aprimoradas. As plataformas de orquestração se integrarão à catalogação e linhagem de dados, tornando-se significativamente mais conscientes dos dados no processo. Além disso, as plataformas de orquestração construirão recursos IaC (semelhantes ao Terraform) e recursos de implantação de código (como

Machine Translated by Google

GitHub Actions e Jenkins). Isso permitirá que os engenheiros codifiquem um pipeline e o passem para a plataforma de orquestração para criar, testar, implantar e monitorar automaticamente. Os engenheiros poderão escrever especificações de infraestrutura diretamente em seus pipelines; a infraestrutura e os serviços ausentes (por exemplo, bancos de dados Snowflake, clusters Databricks e fluxos do Amazon Kinesis) serão implantados na primeira vez que o pipeline for executado.

Também veremos melhorias significativas no domínio de dados ***ao vivo*** – por exemplo, pipelines de streaming e bancos de dados capazes de ingerir e consultar dados de streaming. No passado, construir um DAG de streaming era um processo extremamente complexo com uma alta carga operacional contínua (consulte o Capítulo 8). Ferramentas como o Apache Pulsar apontam o caminho para um futuro no qual os DAGs de streaming podem ser implantados com transformações complexas usando código relativamente simples. Já vimos o surgimento de processadores de fluxo gerenciados (como Amazon Kinesis Data Analytics e Google Cloud Dataflow), mas veremos uma nova geração de ferramentas de orquestração para gerenciar esses serviços, juntá-los e monitorá-los. Discutimos dados ao vivo em "The Live Data Stack".

O que essa abstração aprimorada significa para engenheiros de dados? Como já discutimos neste capítulo, o papel do engenheiro de dados não desaparecerá, mas evoluirá significativamente. Em comparação, sistemas operacionais e estruturas móveis mais sofisticados não eliminaram os desenvolvedores de aplicativos móveis. Em vez disso, os desenvolvedores de aplicativos móveis agora podem se concentrar na criação de aplicativos mais sofisticados e de melhor qualidade. Esperamos desenvolvimentos semelhantes para a engenharia de dados, pois o paradigma de SO de dados em escala de nuvem aumenta a interoperabilidade e a simplicidade em vários aplicativos e sistemas.

Engenharia de Dados "Empresa"

A crescente simplificação das ferramentas de dados e o surgimento e documentação das melhores práticas significa que a engenharia de dados se tornará mais "empresarial". Isso fará muitos leitores se encolherem violentamente. O termo ***empreendimento,*** para alguns, evoca pesadelos kafkianos de

Machine Translated by Google

comitês vestidos com camisas azuis e calças cáqui excessivamente engomadas, burocracia interminável e projetos de desenvolvimento gerenciados em cascata com cronogramas constantemente escorregando e orçamentos inflados. Em suma, alguns de vocês lêem "empresa" e imaginam um lugar sem alma onde a inovação morre.

Felizmente, não é disso que estamos falando; estamos nos referindo a algumas das coisas ***boas*** que empresas maiores fazem com dados — gerenciamento, operações, governança e outras coisas "chatas". Atualmente, estamos vivendo a era de ouro das ferramentas de gerenciamento de dados "empresariais".
Tecnologias e práticas antes reservadas para organizações gigantes estão se espalhando. As partes difíceis de big data e streaming de dados agora foram amplamente abstraídas, com o foco mudando para facilidade de uso, interoperabilidade e outros refinamentos.

Isso permite que os engenheiros de dados que trabalham em novas ferramentas encontrem oportunidades nas abstrações de gerenciamento de dados, DataOps e todas as outras correntes ocultas da engenharia de dados. Os engenheiros de dados se tornarão "empresariais". Falando nisso…

Títulos e responsabilidades se transformarão...

Embora o ciclo de vida da engenharia de dados não vá a lugar nenhum tão cedo, os limites entre engenharia de software, engenharia de dados, ciência de dados e engenharia de ML estão cada vez mais confusos. De fato, como os autores, muitos cientistas de dados são transformados em engenheiros de dados por meio de um processo orgânico; encarregados de fazer "ciência de dados", mas sem as ferramentas para fazer seu trabalho, eles assumem o trabalho de projetar e construir sistemas para atender ao ciclo de vida da engenharia de dados.

À medida que a simplicidade sobe na pilha, os cientistas de dados gastarão uma fatia menor de seu tempo coletando e processando dados. Mas essa tendência se estenderá além dos cientistas de dados. A simplificação também significa que os engenheiros de dados gastarão menos tempo em tarefas de baixo nível no ciclo de vida da engenharia de dados (gerenciamento de servidores, configuração etc.), e a engenharia de dados "empresarial" se tornará mais predominante.

Machine Translated by Google

À medida que os dados se tornam mais incorporados aos processos de cada negócio, novas funções surgirão no domínio dos dados e dos algoritmos. Uma possibilidade é uma função que fica entre a engenharia de ML e a engenharia de dados. À medida que os conjuntos de ferramentas de ML se tornam mais fáceis de usar e os serviços de ML em nuvem gerenciados crescem em recursos, o ML está deixando de lado a exploração ad hoc e o desenvolvimento de modelos para se tornar uma disciplina operacional.

Esse novo engenheiro com foco em ML que atravessa essa divisão conhecerá algoritmos, técnicas de ML, otimização de modelos, monitoramento de modelos e monitoramento de dados. No entanto, sua função principal será criar ou utilizar os sistemas que treinam automaticamente os modelos, monitoram o desempenho e operacionalizam o processo de ML completo para tipos de modelo que são bem compreendidos.
Eles também monitorarão pipelines e qualidade de dados, sobrepondo-se ao domínio atual da engenharia de dados. Os engenheiros de ML se tornarão mais especializados para trabalhar em tipos de modelos mais próximos da pesquisa e menos compreendidos.

Outra área em que os títulos podem se transformar está na interseção da engenharia de software e da engenharia de dados. Os aplicativos de dados, que combinam aplicativos de software tradicionais com análises, impulsionarão essa tendência.
Os engenheiros de software precisarão ter uma compreensão muito mais profunda da engenharia de dados. Eles desenvolverão experiência em coisas como streaming, pipelines de dados, modelagem de dados e qualidade de dados. Iremos além da abordagem de “jogar por cima do muro” que agora é difundida. Os engenheiros de dados serão integrados às equipes de desenvolvimento de aplicativos e os desenvolvedores de software adquirirão habilidades de engenharia de dados. Os limites que existem entre os sistemas de back-end de aplicativos e as ferramentas de engenharia de dados também serão reduzidos, com profunda integração por meio de streaming e arquiteturas orientadas a eventos.

Indo além da pilha de dados moderna, Em direção à pilha de dados ao vivo

Seremos francos: a pilha de dados moderna (MDS) não é tão moderna. Aplaudimos o MDS por trazer uma grande seleção de ferramentas de dados poderosas para as massas,

Machine Translated by Google

reduzindo os preços e capacitando os analistas de dados para assumir o controle de sua pilha de dados. A ascensão do ELT, dos data warehouses em nuvem e a abstração dos pipelines de dados SaaS certamente mudaram o jogo para muitas empresas, abrindo novos poderes para BI, análise e ciência de dados.

Dito isso, o MDS é basicamente um reempacotamento de antigas práticas de data warehouse usando tecnologias modernas de nuvem e SaaS; como o MDS é construído em torno do paradigma de armazenamento de dados em nuvem, ele tem algumas limitações sérias quando comparado ao potencial dos aplicativos de dados em tempo real da próxima geração. Do nosso ponto de vista, o mundo está indo além do uso de análises internas e ciência de dados baseadas em data warehouse, para impulsionar negócios e aplicativos inteiros em tempo real com bancos de dados em tempo real de próxima geração.

O que está impulsionando essa evolução? Em muitos casos, a análise (BI e análise operacional) será substituída pela automação. Atualmente, a maioria dos painéis e relatórios responde a perguntas sobre ***o quê*** e ***quando.*** Pergunte a si mesmo: “Se estou fazendo uma pergunta ***sobre o que*** ou ***quando ,*** que ação devo tomar em seguida?” Se a ação for repetitiva, é candidata à automação. Por que examinar um relatório para determinar se deve agir quando você pode automatizar a ação com base nos eventos à medida que eles ocorrem?

E vai muito além disso. Por que usar um produto como TikTok, Uber, Google ou DoorDash parece mágico? Embora pareça para você clicar em um botão para assistir a um vídeo curto, pedir uma carona ou uma refeição ou encontrar um resultado de pesquisa, muita coisa está acontecendo nos bastidores. Esses produtos são exemplos de verdadeiros aplicativos de dados em tempo real, fornecendo as ações de que você precisa com o clique de um botão enquanto executam processamento de dados extremamente sofisticado e ML nos bastidores com latência minúscula. Atualmente, esse nível de sofisticação está bloqueado atrás de tecnologias personalizadas em grandes empresas de tecnologia, mas essa sofisticação e poder estão se democratizando, semelhante à maneira como o MDS trouxe data warehouses e pipelines em escala de nuvem para as massas. O mundo dos dados em breve estará “ao vivo”.

A pilha de dados ao vivo

Machine Translated by Google

Essa democratização das tecnologias em tempo real nos levará ao sucessor do MDS: a ***pilha de dados ao vivo*** em breve estará acessível e difundida. A pilha de dados ao vivo, representada na Figura 11-1, fundirá análises em tempo real e ML em aplicativos usando tecnologias de streaming, cobrindo todo o ciclo de vida dos dados, desde sistemas de origem de aplicativos até processamento de dados para ML e vice-versa.

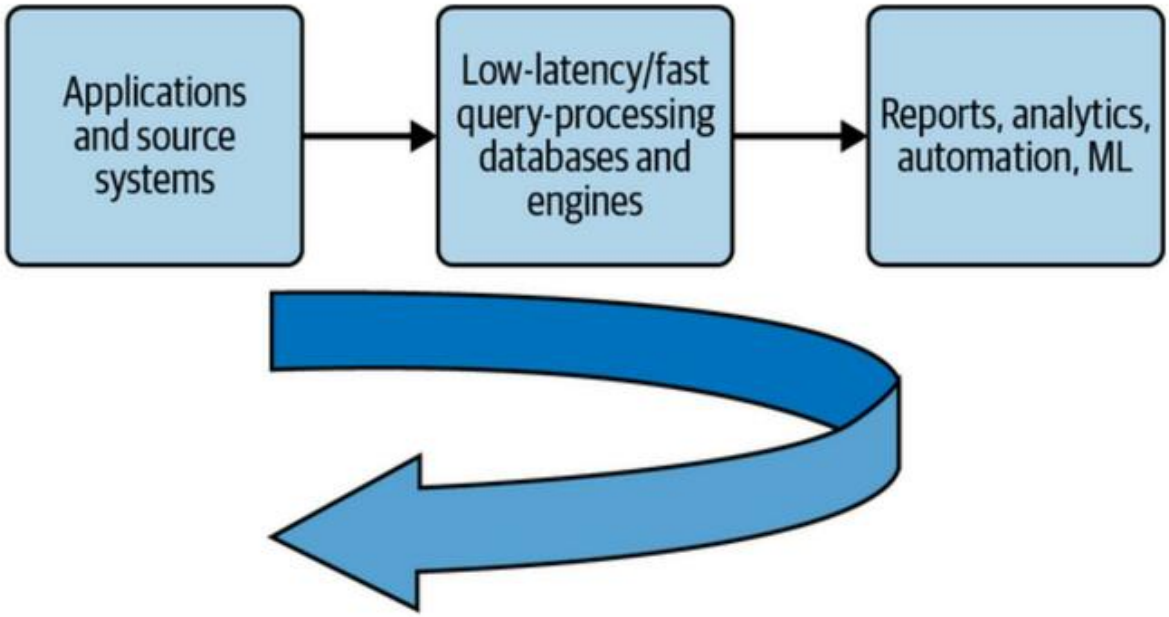


Figura 11-1. Na pilha de dados ao vivo, os dados e a inteligência se movem em tempo real entre o aplicativo e os sistemas de suporte

Assim como o MDS aproveitou a nuvem e trouxe tecnologias de armazenamento de dados e pipeline no local para as massas, a pilha de dados ao vivo usa tecnologias de aplicativos de dados em tempo real usadas em empresas de tecnologia de elite e as disponibiliza para empresas de todos os tamanhos com a mesma facilidade -to-use ofertas baseadas em nuvem. Isso abrirá um novo mundo de possibilidades para criar experiências de usuário e valor comercial ainda melhores.

Pipelines de streaming e bancos de dados analíticos em tempo real

O MDS se limita a técnicas de lote que tratam os dados como limitados. Em contraste, os aplicativos de dados em tempo real tratam os dados como um fluxo contínuo e ilimitado. Pipelines de streaming e bancos de dados analíticos em tempo real são as duas tecnologias principais que facilitarão a mudança do MDS para os dados ao vivo

Machine Translated by Google

pilha. Embora essas tecnologias já existam há algum tempo, os serviços de nuvem gerenciados que amadurecem rapidamente farão com que sejam implantados muito mais amplamente.

As tecnologias de streaming continuarão a ver um crescimento extremo no futuro próximo. Isso acontecerá em conjunto com um foco mais claro na utilidade comercial do streaming de dados. Até o presente, os sistemas de streaming têm sido frequentemente tratados como uma novidade cara ou um canal idiota para obter dados de A para B. No futuro, o streaming transformará radicalmente a tecnologia organizacional e os processos de negócios; arquitetos e engenheiros de dados assumirão a liderança nessas mudanças fundamentais.

Os bancos de dados analíticos em tempo real permitem a ingestão rápida e consultas de subsegundo nesses dados. Esses dados podem ser enriquecidos ou combinados com conjuntos de dados históricos. Quando combinado com um pipeline de streaming e automação, ou painel capaz de análises em tempo real, um novo nível de possibilidades se abre. Você não está mais limitado por processos ELT lentos, atualizações de 15 minutos ou outras partes lentas. Os dados se movem em um fluxo contínuo. À medida que a ingestão de streaming se torna mais prevalente, a ingestão em lote será cada vez menos comum. Por que criar um gargalo de lote no início de seu pipeline de dados? Eventualmente, veremos a ingestão em lote da mesma forma que agora analisamos os modems dial-up.

Em conjunto com o aumento dos fluxos, esperamos um momento de volta ao futuro para transformações de dados. Vamos mudar do ELT — em transformações de banco de dados — para algo que se pareça mais com o ETL. Nos referimos provisoriamente a isso como ***fluxo, transformação e carga*** (STL). Em um contexto de streaming, a extração é um processo contínuo e contínuo. É claro que as transformações em lote não desaparecerão totalmente. O Lote ainda será muito útil para treinamento de modelos, relatórios trimestrais e muito mais. Mas a transformação de streaming se tornará a norma.

Embora o data warehouse e o data lake sejam ótimos para abrigar grandes quantidades de dados e realizar consultas ad hoc, eles não são tão bem otimizados para ingestão de dados de baixa latência ou consultas em dados em movimento rápido. A pilha de dados ao vivo será alimentada por bancos de dados OLAP criados especificamente para

Machine Translated by Google

transmissão. Hoje, bancos de dados como Druid, ClickHouse, Rockset e Firebolt estão liderando o caminho para alimentar o back-end da próxima geração de aplicativos de dados. Esperamos que as tecnologias de streaming continuem a evoluir rapidamente e que novas tecnologias proliferem.

Outra área que achamos que está pronta para disrupção é a modelagem de dados, onde não houve inovação séria desde o início dos anos 2000. As técnicas tradicionais de modelagem de dados orientadas a lotes que você aprendeu no Capítulo 8 não são adequadas para streaming de dados. Novas técnicas de modelagem de dados ocorrerão não no data warehouse, mas nos sistemas que geram os dados. Esperamos que a modelagem de dados envolva alguma noção de uma camada de definições upstream – incluindo semântica, métricas, linhagem e definições de dados (consulte o Capítulo 9) – começando onde os dados são gerados no aplicativo. A modelagem também acontecerá em todas as etapas à medida que os dados fluem e evoluem ao longo de todo o ciclo de vida.

A fusão de dados com aplicativos

Esperamos que a próxima revolução seja a fusão das camadas de aplicativos e dados. No momento, os aplicativos ficam em uma área e o MDS em outra.
Para piorar a situação, os dados são criados sem levar em conta como serão usados para análise. Conseqüentemente, muita fita adesiva é necessária para fazer os sistemas conversarem entre si. Essa configuração de retalhos e silos é desajeitada e desajeitada.

Em breve, as pilhas de aplicativos serão pilhas de dados e vice-versa. Os aplicativos integrarão a automação e a tomada de decisões em tempo real, alimentadas pelos pipelines de streaming e ML. O ciclo de vida da engenharia de dados não mudará necessariamente, mas o tempo entre os estágios do ciclo de vida diminuirá drasticamente. Muita inovação ocorrerá em novas tecnologias e práticas que melhorarão a experiência de engenharia da pilha de dados ao vivo.

Preste atenção às tecnologias emergentes de banco de dados projetadas para abordar a combinação de casos de uso de OLTP e OLAP; os armazenamentos de recursos também podem desempenhar um papel semelhante para casos de uso de ML.

Machine Translated by Google

O feedback apertado entre aplicativos e ML

Outra área com a qual estamos entusiasmados é a fusão de aplicativos e ML.
Hoje, aplicativos e ML são sistemas desconexos, como aplicativos e análises. Engenheiros de software fazem suas coisas aqui, cientistas de dados e engenheiros de ML fazem suas coisas lá.

O ML é adequado para cenários em que os dados são gerados em uma taxa e volume tão altos que os humanos não podem processá-los manualmente. À medida que o tamanho e a velocidade dos dados aumentam, isso se aplica a todos os cenários. Grandes volumes de dados em movimento rápido, juntamente com fluxos de trabalho e ações sofisticados, são candidatos ao ML. À medida que os ciclos de feedback de dados se tornam mais curtos, esperamos que a maioria dos aplicativos integre o ML. À medida que os dados se movem mais rapidamente, o ciclo de feedback entre os aplicativos e o ML ficará mais estreito. Os aplicativos na pilha de dados ao vivo são inteligentes e capazes de se adaptar em tempo real às mudanças nos dados. Isso cria um ciclo de aplicativos cada vez mais inteligentes e aumenta o valor comercial.

Dados da Matéria Escura e a Ascensão das... Planilhas?!

Falamos sobre dados em movimento rápido e como os loops de feedback diminuirão à medida que aplicativos, dados e ML trabalharem mais juntos. Esta seção pode parecer estranha, mas precisamos abordar algo que é amplamente ignorado no mundo dos dados de hoje, especialmente pelos engenheiros.

Qual é a plataforma de dados mais utilizada? É a planilha humilde.
Dependendo das estimativas que você lê, a base de usuários de planilhas está entre 700 milhões e 2 bilhões de pessoas. As planilhas são a matéria escura do mundo dos dados. Boa parte da análise de dados é executada em planilhas e nunca chega aos sofisticados sistemas de dados que descrevemos neste livro. Em muitas organizações, as planilhas lidam com relatórios financeiros, análises da cadeia de suprimentos e até CRM.

No fundo, o que é uma planilha? Uma ***planilha*** é um aplicativo de dados interativo que suporta análises complexas. Ao contrário de ferramentas puramente baseadas em código, como pandas (Python Data Analysis Library), as planilhas são acessíveis a todo um espectro de usuários, desde aqueles que apenas sabem como

Machine Translated by Google

abra arquivos e veja relatórios para usuários avançados que podem criar scripts de processamento de dados processuais sofisticados. Até agora, as ferramentas de BI não conseguiram trazer interatividade comparável aos bancos de dados. Os usuários que interagem com a interface do usuário normalmente estão limitados a fatiar e dividir dados em determinadas proteções, não a análises programáveis de uso geral.

Prevemos que surgirá uma nova classe de ferramentas que combinam os recursos analíticos interativos de uma planilha com o poder de back-end dos sistemas OLAP em nuvem. De fato, alguns candidatos já estão concorrendo.
O vencedor final nesta categoria de produto pode continuar a usar paradigmas de planilhas ou pode definir idiomas de interface totalmente novos para interagir com dados.

Conclusão

Obrigado por se juntar a nós nesta jornada pela engenharia de dados! Percorremos a boa arquitetura, os estágios do ciclo de vida da engenharia de dados e as melhores práticas de segurança. Discutimos estratégias para escolher tecnologias em um momento em que nosso campo continua a mudar em um ritmo extraordinário. Neste capítulo, apresentamos nossa louca especulação sobre o futuro próximo e intermediário.

Alguns aspectos de nosso prognóstico assentam em bases relativamente seguras. A simplificação das ferramentas gerenciadas e o surgimento da engenharia de dados “empresarial” continuaram dia após dia enquanto escrevíamos este livro. Outras previsões são de natureza muito mais especulativa; vemos indícios de uma ***pilha de dados em tempo real emergente,*** mas isso implica uma mudança de paradigma significativa tanto para os engenheiros individuais quanto para as organizações que os empregam. Talvez a tendência de dados em tempo real pare mais uma vez, com a maioria das empresas continuando a se concentrar no processamento básico em lote. Certamente, existem outras tendências que falhamos completamente em identificar. A evolução da tecnologia envolve interações complexas de tecnologia e cultura. Ambos são imprevisíveis.

A engenharia de dados é um tópico vasto; embora não pudéssemos entrar em profundidade técnica em áreas individuais, esperamos ter conseguido criar uma espécie de guia de viagem que ajudará os engenheiros de dados atuais, os dados futuros

Machine Translated by Google

engenheiros e aqueles que trabalham adjacentes ao campo para encontrar seu caminho em um domínio que está em fluxo. Aconselhamos que você continue a exploração por conta própria. À medida que você descobre tópicos e ideias interessantes neste livro, continue a conversa como parte de uma comunidade. Identifique especialistas de domínio que podem ajudá-lo a descobrir os pontos fortes e as armadilhas das tecnologias e práticas modernas. Leia extensivamente os últimos livros, postagens de blogs e artigos.
Participe de encontros e ouça palestras. Faça perguntas e compartilhe seus próprios conhecimentos. Fique de olho nos anúncios dos fornecedores para ficar a par dos últimos desenvolvimentos, levando todas as reivindicações com um grão de sal saudável.

Através deste processo, você pode escolher a tecnologia. Em seguida, você precisará adotar a tecnologia e desenvolver conhecimentos, talvez como um colaborador individual, talvez dentro de sua equipe como líder, talvez em toda uma organização de tecnologia. Ao fazer isso, não perca de vista os objetivos maiores da engenharia de dados. Concentre-se no ciclo de vida, em atender seus clientes – internos e externos – em seus negócios, em servir e em seus objetivos maiores.

Em relação ao futuro, muitos de vocês desempenharão um papel na determinação do que vem a seguir. As tendências tecnológicas são definidas não apenas por aqueles que criam a tecnologia subjacente, mas por aqueles que a adotam e a colocam em bom uso.
O ***uso*** bem- sucedido da ferramenta é tão crítico quanto a ***criação da ferramenta.*** Encontre oportunidades para aplicar tecnologia em tempo real que melhorará a experiência do usuário, criará valor e definirá tipos totalmente novos de aplicativos. É esse tipo de aplicação prática que materializará a ***pilha de dados ao vivo*** como um novo padrão da indústria; ou talvez alguma outra nova tendência tecnológica que não conseguimos identificar vencerá o dia.

Finalmente, desejamos-lhe uma carreira emocionante! Escolhemos trabalhar em engenharia de dados, fazer consultoria e escrever este livro não apenas porque estava na moda, mas porque era fascinante. Esperamos ter conseguido transmitir a você um pouco da alegria que encontramos trabalhando neste campo.

1 Benn Stancil, “The Data OS”, ***benn.substack,*** 3 de setembro de 2021, ***https:// oreil.ly/ HetE9.***

Machine Translated by Google

2 Ben Rogojan, “Três especialistas em engenharia de dados compartilham seus pensamentos sobre onde os dados estão Headed”, ***Better Programming,*** 27 de maio de 2021, ***https:// oreil.ly/ IsY4W.***

Machine Translated by Google

Apêndice A. Detalhes técnicos de serialização e compactação

Os engenheiros de dados que trabalham na nuvem geralmente estão livres das complexidades do gerenciamento de sistemas de armazenamento de objetos. Ainda assim, eles precisam entender os detalhes dos formatos de serialização e desserialização. Como mencionamos no Capítulo 6 sobre armazenamento de ingredientes brutos, os algoritmos de serialização e compactação andam de mãos dadas.

Formatos de serialização

Muitos algoritmos e formatos de serialização estão disponíveis para engenheiros de dados. Embora a abundância de opções seja uma fonte significativa de problemas na engenharia de dados, elas também são uma grande oportunidade para melhorias de desempenho. Às vezes, vimos o desempenho do trabalho melhorar em um fator de 100 simplesmente alternando da serialização CSV para Parquet. À medida que os dados passam por um pipeline, os engenheiros também gerenciam a reserialização — conversão de um formato para outro. Às vezes, os engenheiros de dados não têm escolha a não ser aceitar dados em uma forma antiga e desagradável; eles devem projetar processos para desserializar esse formato e lidar com exceções e, em seguida, limpar e converter dados para processamento e consumo consistentes e rápidos de downstream.

Serialização baseada em linha

Como o próprio nome sugere, ***a serialização baseada em linha*** organiza os dados por linha. O formato CSV é um formato arquetípico baseado em linhas. Para dados semiestruturados (objetos de dados que suportam aninhamento e variação de esquema), a serialização orientada a linhas envolve o armazenamento de cada objeto como uma unidade.

CSV: O padrão fora do padrão

Machine Translated by Google

Discutimos o CSV no Capítulo 7. O CSV é um formato de serialização que os engenheiros de dados adoram odiar. O termo ***CSV*** é essencialmente um termo genérico para texto delimitado, mas há flexibilidade nas convenções de escape, caracteres de aspas, delimitador e muito mais.

Os engenheiros de dados devem evitar o uso de arquivos CSV em pipelines porque eles são altamente propensos a erros e apresentam desempenho insatisfatório. Os engenheiros geralmente precisam usar o formato CSV para trocar dados com sistemas e processos de negócios fora de seu controle. CSV é um formato comum para arquivamento de dados.
Se você usa CSV para arquivamento, inclua uma descrição técnica completa da configuração de serialização de seus arquivos para que futuros consumidores possam ingerir os dados.

XML

Extensible Markup Language (XML) era popular quando o HTML e a Internet eram novos, mas agora é visto como legado; geralmente é lento para desserializar e serializar para aplicativos de engenharia de dados. XML é outro padrão com o qual os engenheiros de dados são frequentemente forçados a interagir enquanto trocam dados com sistemas e softwares legados. O JSON substituiu amplamente o XML para serialização de objetos de texto simples.

JSON e JSONL

JavaScript Object Notation (JSON) surgiu como o novo padrão para troca de dados sobre APIs, e também se tornou um formato extremamente popular para armazenamento de dados. No contexto de bancos de dados, a popularidade do JSON cresceu rapidamente com o surgimento do MongoDB e de outros repositórios de documentos.
Bancos de dados como Snowflake, BigQuery e SQL Server também oferecem amplo suporte nativo, facilitando a troca de dados entre aplicativos, APIs e sistemas de banco de dados.

JSON Lines (JSONL) é uma versão especializada do JSON para armazenar dados semiestruturados em massa em arquivos. JSONL armazena uma sequência de objetos JSON, com objetos delimitados por quebras de linha. Do nosso ponto de vista, JSONL é um formato extremamente útil para armazenar dados logo após serem ingeridos da API ou dos aplicativos. No entanto, muitos formatos colunares oferecem

Machine Translated by Google

atuação. Considere mudar para outro formato para estágios intermediários de pipeline e veiculação.

Avro

Avro é um formato de dados orientado a linhas projetado para RPCs e serialização de dados. O Avro codifica dados em um formato binário, com metadados de esquema especificados em JSON. O Avro é popular no ecossistema Hadoop e também é suportado por várias ferramentas de dados em nuvem.

Serialização Colunar

Os formatos de serialização que discutimos até agora são orientados por linha. Os dados são codificados como relações completas (CSV) ou documentos (XML e JSON), e estes são gravados em arquivos sequencialmente.

Com a ***serialização colunar,*** a organização de dados é essencialmente dinamizada armazenando cada coluna em seu próprio conjunto de arquivos. Uma vantagem óbvia do armazenamento colunar é que ele nos permite ler dados de apenas um subconjunto de campos, em vez de ter que ler linhas inteiras de uma só vez. Esse é um cenário comum em aplicativos de análise e pode reduzir drasticamente a quantidade de dados que devem ser verificados para executar uma consulta.

Armazenar dados como colunas também coloca valores semelhantes próximos uns dos outros, permitindo-nos codificar dados colunares de forma eficiente. Uma técnica comum envolve procurar valores repetidos e tokenizá-los, um método de compactação simples, mas altamente eficiente para colunas com grande número de repetições.

Mesmo quando as colunas não contêm um grande número de valores repetidos, elas podem apresentar alta redundância. Suponha que organizamos as mensagens de suporte ao cliente em uma única coluna de dados. Provavelmente veremos os mesmos temas e palavreado repetidamente nessas mensagens, permitindo que os algoritmos de compactação de dados realizem uma alta proporção. Por esse motivo, o armazenamento colunar geralmente é combinado com a compactação, permitindo maximizar os recursos de largura de banda do disco e da rede.

Machine Translated by Google

O armazenamento colunar e a compactação também apresentam algumas desvantagens. Não podemos acessar facilmente registros de dados individuais; devemos reconstruir registros lendo dados de vários arquivos de coluna. Atualizações de registros também são desafiadoras. Para alterar um campo em um registro, devemos descompactar o arquivo de coluna, modificá-lo, recompactá-lo e gravá-lo novamente no armazenamento. Para evitar a regravação de colunas completas em cada atualização, as colunas são divididas em muitos arquivos, geralmente usando estratégias de particionamento e clustering que organizam os dados de acordo com os padrões de consulta e atualização da tabela. Mesmo assim, a sobrecarga para atualizar uma única linha é horrível. Bancos de dados colunares são um ajuste terrível para cargas de trabalho transacionais, portanto, bancos de dados transacionais geralmente utilizam alguma forma de armazenamento orientado por linha ou registro.

Parquet

O Parquet armazena dados em formato colunar e foi projetado para obter um excelente desempenho de leitura e gravação em um ambiente de data lake. O Parquet resolve alguns problemas que frequentemente atormentam os engenheiros de dados. Os dados codificados em parquet são criados em informações de esquema e oferecem suporte nativo a dados aninhados, ao contrário de CSV. Além disso, o Parquet é portátil; enquanto bancos de dados como BigQuery e Snowflake serializam dados em formatos colunares proprietários e oferecem excelente desempenho de consulta em dados armazenados internamente, ocorre um grande impacto no desempenho ao interoperar com ferramentas externas. Os dados devem ser desserializados, reserializados em um formato intercambiável e exportados para usar ferramentas de data lake, como Spark e Presto. Os arquivos Parquet em um data lake podem ser uma opção superior aos data warehouses em nuvem proprietários em um ambiente de ferramentas poliglotas.

O formato Parquet é usado com vários algoritmos de compactação; algoritmos de compressão de velocidade otimizada como Snappy (discutido mais adiante neste apêndice) são especialmente populares.

ORC

Otimizado Row Columnar (ORC) é um formato de armazenamento colunar semelhante ao Parquet. ORC era muito popular para uso com o Apache Hive; embora ainda amplamente usado, geralmente o vemos muito menos do que o Apache Parquet, e

Machine Translated by Google

desfruta de um pouco menos de suporte nas ferramentas modernas do ecossistema de nuvem. Por exemplo, o Snowflake e o BigQuery são compatíveis com importação e exportação de arquivos Parquet; enquanto eles podem ler de arquivos ORC, nenhuma ferramenta pode exportar para ORC.

Apache Arrow ou serialização na memória

Quando introduzimos a serialização como um ingrediente bruto de armazenamento no início deste capítulo, mencionamos que o software pode armazenar dados em objetos complexos dispersos na memória e conectados por ponteiros, ou estruturas mais ordenadas e densamente empacotadas, como matrizes Fortran e C. Geralmente, estruturas de dados densamente empacotadas na memória eram limitadas a tipos simples (por exemplo, INT64) ou estruturas de dados de largura fixa (por exemplo, strings de largura fixa). Estruturas mais complexas (por exemplo, documentos JSON) não podiam ser densamente armazenadas na memória e a serialização necessária para armazenamento e transferência entre sistemas.

A ideia do Apache Arrow é repensar a serialização utilizando um formato de dados binários adequado tanto para processamento na memória quanto para exportação. Isso nos permite evitar a sobrecarga de serialização e desserialização; simplesmente usamos o mesmo formato para processamento na memória, exportação pela rede e armazenamento de longo prazo. Arrow depende do armazenamento colunar, onde cada coluna obtém essencialmente seus próprios pedaços de memória. Para dados aninhados, usamos uma técnica chamada ***fragmentação,*** que mapeia cada local no esquema de documentos JSON em uma coluna separada.

Essa técnica significa que podemos armazenar um arquivo de dados em disco, trocá-lo diretamente no espaço de endereço do programa usando a memória virtual e começar a executar uma consulta nos dados sem sobrecarga de desserialização. Na verdade, podemos trocar partes do arquivo na memória enquanto o examinamos e, em seguida, trocá-los de volta para evitar ficar sem memória para grandes conjuntos de dados.

Uma dor de cabeça óbvia com essa abordagem é que diferentes linguagens de programação serializam dados de maneiras diferentes. Para resolver esse problema, o Projeto Arrow criou bibliotecas de software para uma variedade de linguagens de programação (incluindo C, Go, Java, JavaScript, MATLAB, Python, R e Rust) que permitem que essas linguagens interoperem com dados do Arrow na memória. Em alguns casos, essas bibliotecas usam uma interface entre os

Machine Translated by Google

linguagem e código de baixo nível em outra linguagem (por exemplo, C) para ler e escrever a partir do Arrow. Isso permite alta interoperabilidade entre linguagens sem sobrecarga extra de serialização. Por exemplo, um programa Scala pode usar a biblioteca Java para escrever dados de seta e depois passá-los como uma mensagem para um Python programa.

A Arrow está tendo uma rápida aceitação com uma variedade de estruturas populares, como o Apache Spark. A Arrow também abrangeu um novo produto de data warehouse; Dremio é um mecanismo de consulta e data warehouse construído em torno da serialização Arrow para oferecer suporte a consultas rápidas.

Serialização híbrida

Usamos o termo ***serialização híbrida*** para nos referirmos a tecnologias que combinam várias técnicas de serialização ou integram a serialização com camadas de abstração adicionais, como gerenciamento de esquema. Citamos como exemplos Apache Hudi e Apache Iceberg.

Hudi

Hudi significa ***Hadoop Update Delete Incremental.*** Essa tecnologia de gerenciamento de tabela combina várias técnicas de serialização para permitir desempenho de banco de dados colunar para consultas analíticas, ao mesmo tempo em que oferece suporte a atualizações atômicas e transacionais. Um aplicativo Hudi típico é uma tabela atualizada de um fluxo CDC de um banco de dados de aplicativo transacional. O fluxo é capturado em um formato de serialização orientado a linhas, enquanto a maior parte da tabela é mantida em um formato colunar. Uma consulta é executada em arquivos orientados a colunas e linhas para retornar resultados para o estado atual da tabela. Periodicamente, é executado um processo de reempacotamento que combina os arquivos de linha e colunar em arquivos colunares atualizados para maximizar a eficiência da consulta.

Iceberg

Assim como o Hudi, o Iceberg é uma tecnologia de gerenciamento de tabelas. Iceberg pode rastrear todos os arquivos que compõem uma tabela. Ele também pode rastrear arquivos em cada instantâneo de tabela

Machine Translated by Google

time, permitindo a viagem no tempo da tabela em um ambiente de data lake. O Iceberg suporta a evolução do esquema e pode gerenciar prontamente tabelas em uma escala de petabytes.

Mecanismos de armazenamento de banco de dados

Para completar a discussão sobre serialização, discutimos brevemente os mecanismos de armazenamento de banco de dados. Todos os bancos de dados têm um mecanismo de armazenamento subjacente; muitos não expõem seus mecanismos de armazenamento como uma abstração separada (por exemplo, BigQuery, Snowflake). Alguns (notavelmente, MySQL) suportam mecanismos de armazenamento totalmente conectáveis. Outros (por exemplo, SQL Server) oferecem as principais opções de configuração do mecanismo de armazenamento (armazenamento em coluna versus armazenamento baseado em linha) que afetam drasticamente o comportamento do banco de dados.

Normalmente, o mecanismo de armazenamento é uma camada de software separada do mecanismo de consulta. O mecanismo de armazenamento gerencia todos os aspectos de como os dados são armazenados em um disco, incluindo a serialização, o arranjo físico dos dados e os índices.

Os mecanismos de armazenamento tiveram uma inovação significativa nas décadas de 2000 e 2010. Enquanto os mecanismos de armazenamento no passado eram otimizados para acesso direto a discos giratórios, os mecanismos de armazenamento modernos são muito mais otimizados para oferecer suporte às características de desempenho dos SSDs. Os mecanismos de armazenamento também oferecem suporte aprimorado para tipos e estruturas de dados modernos, como strings de comprimento variável, arrays e dados aninhados.

Outra grande mudança nos mecanismos de armazenamento é uma mudança para armazenamento colunar para aplicativos de análise e data warehouse. SQL Server, PostgreSQL e MySQL oferecem suporte robusto de armazenamento colunar.

Compressão: gzip, bzip2, Snappy, etc.

A matemática por trás dos algoritmos de compactação é complexa, mas a ideia básica é fácil de entender: os algoritmos de compactação procuram redundância e repetição nos dados e, em seguida, recodificam os dados para reduzir a redundância. Quando queremos ler os dados brutos, nós os ***descompactamos*** invertendo o algoritmo e colocando a redundância de volta.

Machine Translated by Google

Por exemplo, você notou que certas palavras aparecem repetidamente na leitura deste livro. Executando algumas análises rápidas no texto, você pode identificar as palavras que ocorrem com mais frequência e criar tokens abreviados para essas palavras. Para compactar, você substituiria palavras comuns por seus tokens; para descompactar, você substituiria os tokens por suas respectivas palavras.

Talvez pudéssemos usar essa técnica ingênua para obter uma taxa de compressão de 2:1 ou mais. Os algoritmos de compressão utilizam técnicas matemáticas mais sofisticadas para identificar e remover redundância; eles geralmente podem realizar taxas de compactação de 10: 1 em dados de texto.

Observe que estamos falando de ***algoritmos de compactação sem perdas.***
A descompactação de dados codificados com um algoritmo sem perdas recupera uma cópia exata bit a bit dos dados originais. ***Algoritmos de compressão com perdas*** para áudio, imagens e vídeo visam a fidelidade sensorial; descompressão recupera algo que soa ou se parece com o original, mas não é uma cópia exata. Os engenheiros de dados podem lidar com algoritmos de compactação com perdas em pipelines de processamento de mídia, mas não em serialização para análise, onde a fidelidade de dados exata é necessária.

Mecanismos de compactação tradicionais, como gzip e bzip2, compactam dados de texto extremamente bem; eles são frequentemente aplicados a JSON, JSONL, XML, CSV e outros formatos de dados baseados em texto. Os engenheiros criaram uma nova geração de algoritmos de compactação que priorizam a velocidade sobre a taxa de compactação nos últimos anos. Os principais exemplos são Snappy, Zstandard, LZFSE e LZ4.
Esses algoritmos são frequentemente usados para compactar dados em data lakes ou bancos de dados colunares para otimizar o desempenho de consultas rápidas.

1 Dejan Simic, “Apache Arrow: Read DataFrame with Zero Memory,” Towards Data Science, 25 de junho de 2020, ***https:// oreil.ly/ TDAdY.***

Machine Translated by Google

Apêndice B. Rede em Nuvem

Este apêndice discute alguns fatores que os engenheiros de dados devem considerar sobre a rede na nuvem. Os engenheiros de dados frequentemente encontram redes em suas carreiras e muitas vezes as ignoram, apesar de sua importância.

Topologia de rede em nuvem

Uma ***topologia de rede em nuvem*** descreve como vários componentes na nuvem são organizados e conectados, como serviços de nuvem, redes, locais (zonas, regiões) e muito mais. Os engenheiros de dados devem sempre saber como a topologia de rede em nuvem afetará a conectividade nos sistemas de dados que eles constroem. Microsoft Azure, Google Cloud Platform (GCP) e Amazon Web Services (AWS) usam hierarquias de recursos notavelmente semelhantes de zonas e regiões de disponibilidade. No momento da redação deste artigo, o GCP adicionou uma camada adicional, discutida em "Rede específica do GCP e redundância multirregional".

Cobranças de saída de dados

O Capítulo 4 discute a economia da nuvem e como os custos reais do provedor não necessariamente determinam os preços da nuvem. Em relação à rede, as nuvens permitem o tráfego de entrada gratuitamente, mas cobram pelo tráfego de saída para a Internet. O tráfego de saída não é inerentemente mais barato, mas as nuvens usam esse método para criar um fosso em torno de seus serviços e aumentar a rigidez dos dados armazenados, uma prática que tem sido amplamente criticada. Observe que as cobranças de saída de dados também podem ser aplicadas à passagem de dados entre zonas de disponibilidade e regiões em uma nuvem.

Zonas de disponibilidade

Machine Translated by Google

A ***zona de disponibilidade*** é a menor unidade de topologia de rede que as nuvens públicas tornam visível para os clientes (Figura B-1). Embora uma zona possa consistir em vários data centers, os clientes de nuvem não podem controlar o posicionamento de recursos nesse nível.

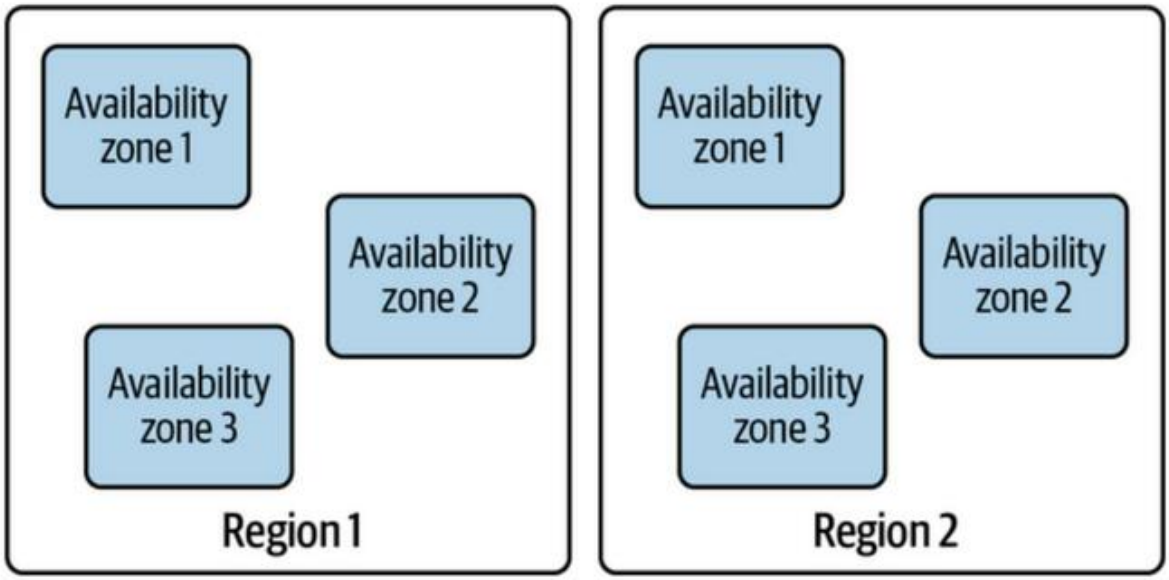


Figura B-1. Zonas de disponibilidade em duas regiões separadas

Geralmente, as nuvens suportam sua maior largura de banda de rede e menor latência entre sistemas e serviços dentro de uma zona. As cargas de trabalho de dados de alta taxa de transferência devem ser executadas em clusters localizados em uma única zona por motivos de desempenho e custo. Por exemplo, um cluster efêmero do Amazon EMR geralmente deve estar em uma única zona de disponibilidade.

Além disso, o tráfego de rede enviado para VMs dentro de uma zona é gratuito, mas com uma ressalva significativa: o tráfego deve ser enviado para endereços IP privados. As principais nuvens utilizam redes virtuais conhecidas como ***nuvens privadas virtuais*** (VPCs). As máquinas virtuais têm endereços IP privados na VPC. Eles também podem receber endereços IP públicos para se comunicar com o mundo exterior e receber tráfego da Internet, mas as comunicações usando endereços IP externos podem incorrer em cobranças de saída de dados.

Regiões

Machine Translated by Google

Uma ***região*** é uma coleção de duas ou mais zonas de disponibilidade. Os data centers exigem muitos recursos para funcionar (energia elétrica, água, etc.). Os recursos de zonas de disponibilidade separadas são independentes para que uma queda de energia local não desative várias zonas de disponibilidade. Os engenheiros podem criar uma infraestrutura separada e altamente resiliente, mesmo em uma única região, executando servidores em várias zonas ou criando failover automatizado entre zonas
processos.

Oferecer várias regiões permite que os engenheiros coloquem recursos perto de qualquer um de seus usuários. ***Fechar*** significa que os usuários podem obter um bom desempenho da rede na conexão com os serviços, minimizando a distância física ao longo do caminho da rede e um número mínimo de saltos pelos roteadores. Tanto a distância física quanto os saltos de rede podem aumentar a latência e diminuir o desempenho. Os principais provedores de nuvem continuam a adicionar novas regiões.

Em geral, as regiões oferecem suporte a redes rápidas e de baixa latência entre zonas; o desempenho de rede entre zonas será pior do que dentro de uma única zona e incorrerá em cobranças nominais de saída de dados entre VMs. A movimentação de dados de rede entre regiões é ainda mais lenta e pode gerar taxas de saída mais altas.

Em geral, o armazenamento de objetos é um recurso regional. Alguns dados podem passar entre zonas para alcançar uma máquina virtual, mas isso é invisível principalmente para os clientes da nuvem e não há cobranças diretas de rede para isso. (É claro que os clientes ainda são responsáveis pelos custos de acesso ao objeto.)

Apesar do design georredundante das regiões, muitas falhas importantes no serviço de nuvem afetaram regiões inteiras, um exemplo de ***falha correlacionada.*** Os engenheiros geralmente implantam código e configuração em regiões inteiras; as falhas regionais que observamos geralmente resultam de problemas de código ou configuração que ocorrem no nível regional.

Redes específicas do GCP e redundância multirregional

O GCP oferece várias abstrações exclusivas que os engenheiros devem conhecer se trabalharem nessa nuvem. A primeira é a ***multirregião,*** uma camada na hierarquia de recursos; uma multirregião contém várias regiões. Atual

Machine Translated by Google

multirregiões são EUA (centros de dados nos Estados Unidos), UE (centros de dados nos estados membros da União Europeia) e ÁSIA.

Vários recursos do GCP são compatíveis com multirregiões, incluindo Cloud Storage e BigQuery. Os dados são armazenados em várias zonas dentro da multirregião de maneira georedundante para que permaneçam disponíveis no caso de uma falha regional. O armazenamento multirregional também foi projetado para fornecer dados com eficiência aos usuários na multirregião sem configurar processos complexos de replicação entre regiões. Além disso, não há taxas de saída de dados para VMs em uma multirregião para acessar dados do Cloud Storage na mesma multirregião.

Os clientes da nuvem podem configurar a infraestrutura multirregional na AWS ou no Azure. No caso de bancos de dados ou armazenamento de objetos, isso envolve a duplicação de dados entre regiões para aumentar a redundância e aproximar os dados dos usuários.

O Google também possui essencialmente mais recursos de rede em escala global do que outros provedores de nuvem, algo que oferece a seus clientes como ***rede de nível premium.*** A rede de nível premium permite que o tráfego entre zonas e regiões passe inteiramente por redes de propriedade do Google sem atravessar a Internet pública.

Conexões de rede diretas para as nuvens

Cada grande nuvem pública oferece opções de conectividade aprimoradas, permitindo que os clientes integrem suas redes diretamente a uma região de nuvem ou VPC.
Por exemplo, a Amazon oferece o AWS Direct Connect. Além de fornecer maior largura de banda e menor latência, essas opções de conexão geralmente oferecem grandes descontos nas cobranças de saída de dados. Em um cenário típico nos EUA, as cobranças de saída da AWS caem de 9 centavos por gigabyte na Internet pública para 2 centavos por gigabyte na conexão direta.

CDN

As redes de entrega de conteúdo (CDNs) podem oferecer melhorias dramáticas de desempenho e descontos para a entrega de ativos de dados ao público ou

Machine Translated by Google

clientes. Os provedores de nuvem oferecem opções de CDN e muitos outros provedores, como o Cloudflare. As CDNs funcionam melhor ao fornecer os mesmos dados repetidamente, mas certifique-se de ler as letras miúdas. Lembre-se de que as CDNs não funcionam em todos os lugares e alguns países podem bloquear o tráfego da Internet e a entrega da CDN.

O futuro das taxas de saída de dados

As taxas de saída de dados são um impedimento significativo à interoperabilidade, compartilhamento de dados e movimentação de dados para a nuvem. No momento, as taxas de saída de dados são um fosso projetado para impedir que os clientes de nuvem pública saiam ou implantem em várias nuvens.

Mas sinais interessantes indicam que a mudança pode estar no horizonte. Em particular, o anúncio da Zoom em 2020, perto do início da pandemia do COVID-19, de que escolheu a Oracle como seu provedor de infraestrutura em nuvem chamou a atenção de muitos observadores de nuvem. Como a Oracle ganhou esse importante[2] contrato de nuvem para infraestrutura crítica de trabalho remoto em relação aos pesos-pesados da nuvem? O especialista em AWS Corey Quinn oferece uma resposta razoavelmente direta. De acordo com seu cálculo detalhado, as taxas[3] mensais de saída de dados da AWS da Zoom seriam superiores a US$ 11 milhões pelo preço de tabela; O da Oracle custaria menos de US$ 2 milhões.

Suspeitamos que um GCP, AWS ou Azure anuncie cortes significativos nas taxas de saída nos próximos anos, levando a uma mudança radical no modelo de negócios da nuvem.

1 Matthew Prince e Nitin Rao, “AWS's Egregious Egress,” ***The Cloudflare Blog,*** 23 de julho de 2021, ***https://oreil.ly/NZqKa.***

2 Mark Haranas e Steven Burke, “Oracle vence rivais da nuvem para ganhar a Blockbuster Cloud Deal”, CRN, 28 de abril de 2020, ***https://oreil.ly/LkqOi.***

3 Corey Quinn, “Por que o Zoom escolheu o Oracle Cloud em vez da AWS e talvez você também deva” Última semana na AWS, 28 de abril de 2020, ***https://oreil.ly/Lx5uu.***

Machine Translated by Google

Machine Translated by Google

sobre os autores

Joe Reis é um nerd de dados com mentalidade de negócios que trabalha no setor de dados há 20 anos, com responsabilidades que variam de modelagem estatística, previsão, aprendizado de máquina, engenharia de dados, arquitetura de dados e quase tudo mais. Joe é o CEO e cofundador da Ternary Data, uma empresa de consultoria em arquitetura e engenharia de dados com sede em Salt Lake City, Utah. Além disso, ele é voluntário em vários grupos de tecnologia e leciona na Universidade de Utah. Em seu tempo livre, Joe gosta de escalar rochas, produzir música eletrônica e levar seus filhos em aventuras malucas.

Matt Housley é consultor de engenharia de dados e especialista em nuvem. Após alguma experiência inicial de programação com Logo, Basic e assembly 6502, ele concluiu um doutorado em matemática na Universidade de Utah. Matt então começou a trabalhar em ciência de dados, eventualmente se especializando em engenharia de dados baseada em nuvem. Ele cofundou a Ternary Data com Joe Reis, onde aproveita sua experiência de ensino para treinar futuros engenheiros de dados e aconselhar equipes sobre arquitetura de dados robusta. Matt e Joe também pontificam sobre todos os dados no The Monday Morning Data Chat.

Machine Translated by Google

Colofão

O animal na capa de ***Fundamentos de Engenharia de Dados*** é o puffbird de orelhas brancas ***(Nystalus chacuru).***

Assim chamados por causa da mancha branca em suas orelhas, bem como por sua plumagem fofa, essas pequenas e rotundas aves são encontradas em uma ampla faixa da América do Sul central, onde habitam as bordas das florestas e savana.

Os puffbirds de orelhas brancas são caçadores de sentar e esperar, empoleirando-se em espaços abertos por longos períodos e se alimentando de forma oportunista de insetos, lagartos e até pequenos mamíferos que se aproximam. Eles são mais frequentemente encontrados sozinhos ou em pares e são pássaros relativamente quietos, vocalizando apenas raramente.

A União Internacional para a Conservação da Natureza listou o puffbird de orelha branca como sendo de ***menor preocupação,*** devido, em parte, à sua extensa área de distribuição e população estável. Muitos dos animais nas capas da O'Reilly estão ameaçados de extinção; todos eles são importantes para o mundo.

A ilustração da capa é de Karen Montgomery, baseada em uma gravura de linha antiga da ***Zoologia Geral de Shaw.*** As fontes da capa são Gilroy Semibold e Guardian Sans. A fonte do texto é Adobe Minion Pro; a fonte do título é Adobe Myriad Condensed; e a fonte do código é Ubuntu Mono da Dalton Maag.